

%ifdef __X64

LoadGsBaseToRbp

%else

mov ebp, [base]

%endif

mov ax,

;;

;;

test DWORD

邓志 著

;;

;; 检测是否支持 VMX

;;

bt DWORD [ebp + PCB.FeatureEcx], 5

mov eax, STATUS_UNSUCCESS

jnc vmx_operation_enter.done

;;

;; 开启 VMX operation 允许

;;

REX.Wrb

mov eax, cr4

REX.Wrb

bts eax, 13

REX.Wrb

mov cr4, eax

处理器 虚拟化技术



电子工业出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY
http://www.phei.com.cn



内 容 简 介

本书针对在Intel处理器端的虚拟化技术（Intel Virtualization Technology for x86，即Intel VT-x）进行全面讲解。在Intel VT-x技术下实现了VMX（Virtual-Machine Extensions，虚拟机扩展）架构平台来支持对处理器的虚拟化管理。因此，VMX架构是Intel VT-x技术的核心。本书内容围绕VMX架构实现细节展开全面讲解。但Intel VT-d（Virtualization Technology for Directed I/O）和Intel VT-c（Virtualization Technology for Connectivity）技术并不在本书的描述范围。同时，也不针对AMD-v技术进行讨论。

全书共分为7章，书的整体结构也较为规整，可读性比较强。本书共提供14个例子，对VMX架构的一些特色功能进行辅助讲解。

读者阅读本书，可以学习Intel VT-x技术的VMX架构知识，并且对整个x86/x64体系有更深入的了解！可以说，不了解VMX架构，根本算不上对x86/x64体系熟悉，因为，在处理器的虚拟化技术里需要使用全方位的体系知识，对处理器在非常细节的地方进行虚拟化处理。

因此，本书适合有一定x86/x64体系知识基础或者想更深入学习x86/x64体系知识的读者。

未经许可，不得以任何方式复制或抄袭本书之部分或全部内容。

版权所有，侵权必究。

图书在版编目（CIP）数据

处理器虚拟化技术 / 邓志著. —北京：电子工业出版社，2014.6

ISBN 978-7-121-23019-6

I. ①处… II. ①邓… III. ①微处理器—虚拟技术IV. ①TP332

中国版本图书馆 CIP 数据核字(2014)第 080434 号

策划编辑：李 冰

责任编辑：李 冰 高洪霞

印 刷：北京京科印刷有限公司

装 订：三河市皇庄路通装订厂

出版发行：电子工业出版社

北京市海淀区万寿路 173 信箱 邮编 100036

开 本：787×1092 1/16 印张：41.75 字数：1076 千字

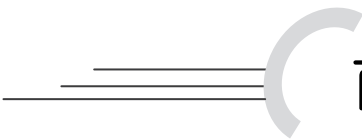
印 次：2014 年 6 月第 1 次印刷

印 数：2000 册 定价：109.00 元

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：（010）88254888。

质量投诉请发邮件至 zlts@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

服务热线：（010）88258888。



前言

虚拟化技术大概可以分为软件虚拟化和硬件虚拟化两大类。软件上的虚拟化实际上很多地方都可以看到，例如常见的系统虚拟机软件，Java 虚拟机和 Android 上的虚拟机可以算是广义上的软件虚拟机了。

在硬件虚拟化技术出现之前，虚拟机只能靠软件来模拟实现。硬件上的虚拟化技术可以算是比较热的一门技术了，但它并不是什么新鲜的技术。据资料显示，实际上 Intel 第一代 VT-x 技术 Vanderpool 在 2005 年就已经推出了，但 Intel 发展得很快，从处理器端到芯片组端，再到网络端都已经可以布署硬件虚拟化技术了。

《处理器虚拟化技术》一书围绕 Intel VT-x 处理器端的虚拟化技术而写，可以算是前作《x86/x64 体系探索及编程》的下册。尽管此书只围绕一个主题，但笔者认为它的知识面并不比《x86/x64 体系探索及编程》一书窄，甚至还要超出。因为，完整地虚拟化一个“虚拟处理器”，必须结合原来的 x86/x64 体系知识并扩展开来。编写本书所花的时间比前作还要多许多，从开始动笔到完稿用了 11 个月的时间，期间不可谓不艰辛。

本书内容

全书共 7 章。

第 1 章 系统平台

本书的代码使用汇编语言编写，运行在裸机平台上。本章主要介绍如何实现一个简易平台来支撑书中所有例子的运行。这个平台可以被编译生成 32 位或者 64 位环境。也实现了一些特色机制，例如，开启了多处理器环境，并且为了更好地、直观地了解代码的执行流程而加入了调试信息机制。

第 2 章 VMX 架构基础

VMX 架构是 Intel 处理器端虚拟化技术 (Intel VT-x) 的实现框架。本章对虚拟化技术进行概括讲解，全面了解 VMX 架构的基础知识。首先介绍了 VMX 架构下引入 VMX root operation 与 VMX non-root operation 模式，以及这两个模式之间的切换。然后介绍了

VMX 架构下的各种处理器能力，以及这些能力的检测。最后全面讲解新引进的 VMX 指令集，以及 VMX 指令产生的各种失败的情形。

第 3 章 VMCS 结构

VMCS 结构是 VMX 架构下最基本、最重要的数据结构。VMCS 结构内分为 6 个区域，每个区域有若干个字段。整个 VMX operation 模式的运行环境由 VMCS 结构内的这些字段来配置与管理，例如配置 guest 与 host 的运行环境。全面地讲解了这 6 个区域内每个字段的设置与使用。

第 4 章 VM-entry 处理

在 VMX 架构下，guest 运行在 VMX non-root operation 模式下，而 host 运行在 VMX root operation 模式下。如果需要运行 guest 软件，则处理器需要从 VMX root operation 切换到 VMX non-root operation 模式运行，这个行为被称为“VM-entry”。

本章详细地讲解处理器在执行 VM-entry 操作时的每个流程。例如，在 VM-entry 时会检查 VMCS 结构内字段配置是否合理，并且加载 guest 的运行环境。处理器也会根据 VMCS 内字段来控制 guest 软件的运行。

第 5 章 VM-exit 处理

在 guest 运行过程中，会因为某些事件而被迫返回 host 环境。处理器由 VMX non-root operation 模式切换回 VMX root operation 模式，这个行为被称为“VM-exit”。

本章详细地讲解处理器在执行 VM-exit 操作时的每个流程。例如，在 VM-exit 处理器的相应状态信息会得到更新，guest 的环境信息会保存在 VMCS 结构内的 guest-state 区域，并且从 host-state 区域加载 host 环境信息。最后转入 host 入口执行 VMM 的管理代码。

第 6 章 内存虚拟化

VMX 架构引进了 EPT 机制来实现对物理内存的虚拟化管理，使得每个 VM 在物理平台上拥有自己独立的物理内存区域，而不受 VMM 及其他 VM 的干扰。同样也避免了 VM 对 VMM 的干扰。

本章全面讲解了 EPT 机制的实现细节，也着重介绍了 VMX 架构下的 cache 管理。在随书例子中，我们实现了 EPT 机制来管理 VM 内存。

第 7 章 中断虚拟化

VMX 架构下也实现了对 local APIC 的虚拟化管理，引进了两个重要的页面：APIC-access page 页面与 virtual-APIC page 页面。因此而形成了 virtual-APIC 的概念。它是物理平台上的 local APIC 的 shadow。

在这一章里，我们将看到处理器如何处理 APIC-access page 与 virtual-APIC page 之间的关系，将对 APIC-access page 的访问转化为对 virtual-APIC page 的访问。从而实现访问和管理 virtual-APIC 组件。

本章也着重介绍了 VMM 应该如何处理 guest 产生的异常与任务切换。最后以实际例子来展现 VMM 如何监控 INT 指令及 NMI 与外部中断。

如何阅读本书

本书将理论结合实际地进行讲解，但每章内容还是会有侧重点。以章节的篇幅来说，第 2 章至第 5 章偏重于理论知识。而第 1 章、第 6 章和第 7 章偏于实际例子。

因此，如果读者并不想通篇阅读，可以选择偏重理论知识的章节来看。第 6 章与第 7 章介绍的重点知识主要是 EPT（扩展页表）与 local APIC 虚拟化。这两个内容也是必须掌握的。

在第 1 章中并不涉及 VMX 架构知识，而是通过代码的讲解来介绍基础平台。书中的全部例子将构建于这个基础平台之上。如果读者对此章不感兴趣，可以直接跳过。但是，如果读者对一些 OS 的基础元素感兴趣，大可详细加以阅读。

读者对象

读者最好有一些 x86/x64 体系知识基础，如果感到这方面的基础知识相对薄弱，可以阅读作者的另一本著作《x86/x64 体系探索及编程》或者 Intel 官方手册。

如果读者只是想了解 VMX 的理论知识，那么可以选择相应章节，对汇编知识的要求不高。如果读者想深入全面地了解整个 VMX 架构知识，希望能够仔细阅读所有章节（第 1 章可以略过），那么需要对汇编有一定了解。

随书例子

书中的每个章节有若干例子，全书共 14 个例子。读者请登录网址 <http://www.mouseos.com/books/vt/index.html>，选择相应的栏目下载源码包。

源码包解压后有 7 个 chapXX 目录，对应于每一章。还有 commom、inc 以及 lib 目录，存放所有例子共用的代码。每个 chapXX 目录下有多个 ex 实例目录，对应于每个例子。以第 1 章为例，chap01 目录下有 ex1-1 和 ex1-2 实例目录。

每个实例目录里有下面的文件。

- Bs: bochs 的配置文件。
- Build: 一个 DOS 批处理的编译工具，用来编译源码。
- c.img: 硬盘与 U 盘映像文件。
- demo.img: 软盘映像文件。
- ex.asm: 例子实体代码的汇编源文件。
- ex.inc: 例子实体代码头文件。

在时间宽裕的情况下，我会在 Windows 平台上用 C 语言来实现代码示例。目前已经写了大部分代码，但为了保证代码的正确性，力求在所有代码完成后再放出来。

勘误反馈

由于作者学识有限，尽管在书写时已经力求认真细致，但难免有错漏之处。读者如果发现有不妥之处，敬请不吝指出，邮件请发至：mik@mouseos.com，或登录www.mouseos.com 网站留言。

参考资料

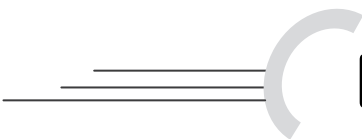
- 《Intel® 64 and IA-32 Architectures Software Developer's Manual》volumes 1, 2A, 2B, 2C, 3A, 3B and 3C
- 《Intel Virtualization Technology for Directed I/O Architecture Specification》

致谢

感谢电子工业出版社博文视点公司的全体工作人员，特别感谢本书的策划编辑李冰，正是有你们的辛勤劳动，才使得本书可以顺利出版。

邓志

2014 年 5 月



目 录

第 1 章 系统平台	1
1.1 环境及工具	1
1.1.1 使用 VMware	2
1.1.2 使用 Bochs	4
1.1.3 在真实机器上运行	4
1.1.4 Build 工具	4
1.2 64 位与 32 位代码的混合编译	7
1.2.1 使用符号 __X64	7
1.2.2 指令操作数	8
1.2.3 64-bit 模式下其他指令处理	11
1.2.4 函数重定义表	15
1.3 地址空间	17
1.4 数据结构	23
1.4.1 PCB 结构	23
1.4.2 LSB 结构	37
1.4.3 初始化 PCB	38
1.4.4 SDA 结构	42
1.4.5 初始化 SDA	56
1.4.6 DRS 结构	57
1.5 系统启动	59
1.5.1 Boot 阶段	59
1.5.2 stage1 阶段	62

1.5.2.1	stage1 阶段的多处理器初始化	66
1.5.2.2	BSP 的收尾工作	68
1.5.2.3	APs 的 stage1 阶段工作	70
1.5.3	stage2 阶段	73
1.5.3.1	BSP 在 stage2 最后处理	80
1.5.3.2	APs 在 stage2 阶段收尾工作	81
1.5.4	stage3 阶段	83
1.5.4.1	BSP 在 stage3 阶段的最后工作	87
1.5.4.2	APs 在 stage3 阶段收尾工作	88
1.5.5	例子 1-1	90
1.6	系统机制	91
1.6.1	分页机制	91
1.6.1.1	PAE 分页模式实现	91
1.6.1.2	IA-32e 分页模式实现	98
1.6.2	多处理器机制	102
1.6.2.1	调度任务	102
1.6.2.2	处理器切换	109
1.6.3	调试记录机制	113
1.6.3.1	例子 1-2	120
1.6.3.2	运行结果	121
第 2 章	VMX 架构基础	122
2.1	虚拟化概述	123
2.1.1	虚拟设备	124
2.1.2	地址转换	125
2.1.3	设备的 I/O 访问	125
2.2	VMX 架构	126
2.2.1	VMM 与 VM	127
2.2.2	VMXON 与 VMCS 区域	127
2.2.3	检测 VMX 支持	128
2.2.4	开启 VMX 进入允许	128
2.3	VMX operation 模式	129

2.3.1	进入 VMX operation 模式	130
2.3.2	进入 VMX operation 的制约	131
2.3.2.1	IA32_FEATURE_CONTROL 寄存器	131
2.3.2.2	CR0 与 CR4 固定位	133
2.3.2.3	A20M 模式	135
2.3.3	设置 VMXON 区域	135
2.3.3.1	分配 VMXON 区域	135
2.3.3.2	VMXON 区域初始设置	135
2.3.4	退出 VMX operation 模式	136
2.4	VMX operation 模式切换	137
2.4.1	VM entry	138
2.4.2	VM exit	139
2.4.3	SMM 双重监控处理下	140
2.5	VMX 能力的检测	141
2.5.1	检测是否支持 VMX	141
2.5.2	通过 MSR 组检查 VMX 能力	141
2.5.3	例子 2-1	146
2.5.4	基本信息检测	147
2.5.5	允许为 0 以及允许为 1 位	149
2.5.5.1	决定 VMX 支持的功能	150
2.5.5.2	控制字段设置算法	150
2.5.6	VM-execution 控制字段	151
2.5.6.1	Pin-based VM-execution control 字段	151
2.5.6.2	primary processor-based VM-execution control 字段	152
2.5.6.3	secondary processor-based VM-execution control 字段	152
2.5.7	VM-exit control 字段	152
2.5.8	VM-entry control 字段	153
2.5.9	VM-function control 字段	153
2.5.10	CR0 与 CR4 的固定位	154
2.5.10.1	CR0 与 CR4 寄存器设置算法	155
2.5.11	VMX 杂项信息	156
2.5.12	VMCS 区域字段 index 值	157

2.5.13	VPID 与 EPT 能力	157
2.6	VMX 指令	158
2.6.1	VMX 指令执行环境	159
2.6.2	指令执行的状态	159
2.6.3	VMfailValid 事件原因	160
2.6.4	指令异常优先级	161
2.6.5	VMCS 管理指令	161
2.6.5.1	VMPTRLD 指令	162
2.6.5.2	VMPTRST 指令	162
2.6.5.3	VMCLEAR 指令	162
2.6.5.4	VMREAD 指令	163
2.6.5.5	VMWRITE 指令	165
2.6.6	VMX 模式管理指令	166
2.6.6.1	VMXON 指令	167
2.6.6.2	VMXOFF 指令	167
2.6.6.3	VMLAUNCH 指令	167
2.6.6.4	VMRESUME 指令	168
2.6.6.5	返回到 executive monitor	168
2.6.7	cache 刷新指令	169
2.6.7.1	INVEPT 指令	170
2.6.7.2	INVVPID 指令	170
2.6.8	调用服务例程指令	171
2.6.8.1	VMCALL 指令	171
2.6.8.2	VMFUNC 指令	172
第 3 章	VMCS 结构	173
3.1	VMCS 状态	173
3.1.1	activity 属性	174
3.1.2	current 属性	174
3.1.3	launch 属性	174
3.2	VMCS 区域	175
3.2.1	VMXON 区域	176

3.2.2	Executive-VMCS 与 SMM-transfer VMCS	176
3.2.3	VMCS 区域格式	176
3.3	访问 VMCS 字段	177
3.3.1	字段 ID 格式	178
3.3.2	不同宽度的字段处理	179
3.4	字段 ID 值	181
3.4.1	16 位字段 ID	181
3.4.2	64 位字段 ID	182
3.4.3	32 位字段 ID	184
3.4.4	natural-width 字段 ID	185
3.5	VM-execution 控制类字段	187
3.5.1	Pin-based VM-execution control 字段	188
3.5.2	processor-based VM-execution control 字段	190
3.5.2.1	primary processor-based VM-execution control 字段	191
3.5.2.2	secondary processor-based VM-execution control 字段	195
3.5.3	exception bitmap 字段	200
3.5.4	PFEC_MASK 与 PFEC_MATCH 字段	200
3.5.5	I/O bitmap address 字段	202
3.5.6	TSC offset 字段	202
3.5.7	guest/host mask 与 read shadow 字段	202
3.5.8	CR3-target 字段	203
3.5.9	APIC-access address 字段	203
3.5.10	virtual-APIC address 字段	204
3.5.11	TPR threshold 字段	204
3.5.12	EOI-exit bitmap 字段	204
3.5.13	posted-interrupt notification vector 字段	205
3.5.14	posted-interrupt descriptor address 字段	205
3.5.15	MSR bitmap address 字段	205
3.5.16	executive-VMCS pointer	206
3.5.17	EPTP 字段	206
3.5.18	virtual-processor identifier 字段	207
3.5.19	PLE_Gap 与 PLE_Window 字段	207

3.5.20	VM-function control 字段	209
3.5.21	EPTP-list address 字段	210
3.6	VM-entry 控制类字段	210
3.6.1	VM-entry control 字段	211
3.6.2	VM-entry MSR-load 字段	214
3.6.3	事件注入控制字段	214
3.6.3.1	VM-entry interruption information 字段	215
3.6.3.2	VM-entry exception error code 字段	217
3.6.3.3	VM-entry instruction length 字段	217
3.7	VM-exit 控制类字段	218
3.7.1	VM-exit control 字段	218
3.7.2	VM-exit MSR-store 与 MSR-load 字段	220
3.8	guest-state 区域字段	221
3.8.1	段寄存器字段	224
3.8.1.1	access right 字段	224
3.8.2	GDTR 与 IDTR 字段	229
3.8.3	MSR 字段	229
3.8.4	SMBASE 字段	229
3.8.5	activity state 字段	230
3.8.6	interruptibility state 字段	232
3.8.7	pending debug exceptions 字段	235
3.8.7.1	#DB 异常的处理	237
3.8.8	VMCS link pointer 字段	243
3.8.9	VMX-preemption timer value 字段	243
3.8.10	PDPTEs 字段	243
3.8.11	guest interrupt status 字段	244
3.9	host-state 区域字段	245
3.10	VM-exit 信息类字段	247
3.10.1	基本信息类字段	248
3.10.1.1	Exit reason 字段	248
3.10.1.2	VM-exit 原因	249

3.10.1.3	Exit qualification 字段	255
3.10.1.4	由某些指令引发的 VM-exit	256
3.10.1.5	由#DB 异常引发的 VM-exit	256
3.10.1.6	由#PF 异常引发的 VM-exit	257
3.10.1.7	由 SIPI 引发的 VM-exit	257
3.10.1.8	由 I/O SMI 引发的 VM-exit	257
3.10.1.9	由任务切换引发的 VM-exit	258
3.10.1.10	访问控制寄存器引发的 VM-exit	259
3.10.1.11	由 MOV-DR 指令引发的 VM-exit	260
3.10.1.12	由 I/O 指令引发的 VM-exit	260
3.10.1.13	由于访问 APIC-access page 引发的 VM-exit	261
3.10.1.14	由 EPT violation 引发的 VM-exit	262
3.10.1.15	由 EOI 虚拟化引发的 VM-exit	264
3.10.1.16	由 APIC-write 引发的 VM-exit	264
3.10.1.17	guest-linear address 字段	264
3.10.1.18	guest-physical address 字段	265
3.10.2	直接向量事件类信息字段	265
3.10.2.1	VM-exit interruption information 字段	265
3.10.2.2	VM-exit interruption error code 字段	267
3.10.3	间接向量事件类信息字段	267
3.10.3.1	IDT-vectoring information 字段	268
3.10.3.2	IDT-vectoring error code 字段	269
3.10.4	指令类信息字段	269
3.10.4.1	VM-exit instruction length 字段	269
3.10.4.2	VM-exit instruction information 字段	272
3.10.5	I/O SMI 信息类字段	280
3.10.6	指令错误类字段	280
3.11	VMM 初始化实例	280
3.11.1	VMCS 相关的数据结构	281
3.11.1.1	VMB 结构	281
3.11.1.2	VSB 结构	284
3.11.1.3	VMCS buffer 结构	287

3.11.2	初始化 VMXON 区域	288
3.11.3	初始化 VMCS 区域	289
3.11.3.1	分配 VMCS 区域	290
3.11.3.2	VMCS 初始化模式	291
3.11.3.3	VMCS buffer 初始化	293
3.11.4	例子 3-1	297
第 4 章	VM-entry 处理	301
4.1	发起 VM-entry 操作	302
4.2	VM-entry 执行流程	303
4.3	指令执行的基本检查	303
4.4	检查控制区域及 host-state 区域	305
4.4.1	VM-execution 控制区域检查	305
4.4.1.1	检查 pin-based VM-execution control 字段	306
4.4.1.2	检查 primary processor-based VM-execution control 字段	306
4.4.1.3	检查 secondary processor-based VM-execution control 字段	307
4.4.1.4	检查 CR3-target 字段	308
4.4.2	VM-exit 控制区域检查	308
4.4.2.1	VM-exit control 字段的检查	308
4.4.2.2	MSR-store 与 MSR-load 相关字段的检查	308
4.4.3	VM-entry 控制区域检查	309
4.4.3.1	VM-entry control 字段的检查	309
4.4.3.2	MSR-load 相关字段的检查	309
4.4.3.3	事件注入相关字段的检查	309
4.4.4	Host-state 区域的检查	310
4.4.4.1	Host 控制寄存器字段的检查	310
4.4.4.2	Host-RIP 的检查	310
4.4.4.3	段 selector 字段的检查	311
4.4.4.4	段基址字段的检查	311
4.4.4.5	MSR 字段的检查	311
4.5	检查 guest-state 区域	311
4.5.1	检查控制寄存器字段	312

4.5.2	检查 RIP 与 RFLAGS 字段	312
4.5.3	检查 DR7 与 IA32_DEBUGCTL 字段	313
4.5.4	检查段寄存器字段	313
4.5.4.1	virtual-8086 模式下的检查	314
4.5.4.2	unrestricted guest 位为 0 时的检查	315
4.5.4.3	unrestricted guest 位为 1 时的检查	318
4.5.5	检查 GDTR 与 IDTR 字段	320
4.5.6	检查 MSR 字段	320
4.5.7	检查 activity state 字段	321
4.5.8	检查 interruptibility state 字段	321
4.5.9	检查 pending debug exception 字段	322
4.5.10	检查 VMCS link pointer 字段	322
4.5.11	检查 PDPTE 字段	323
4.5.11.1	由加载 CR3 引发的 PDPTE 检查	323
4.6	检查 guest state 引起的 VM-entry 失败	324
4.7	加载 guest 环境信息	324
4.7.1	加载控制寄存器	325
4.7.2	加载 DR7 与 IA32_DEBUGCTL	325
4.7.3	加载 MSR	325
4.7.4	SMBASE 字段处理	326
4.7.5	加载段寄存器与描述符表寄存器	326
4.7.5.1	unusable 段寄存器	327
4.7.5.2	加载 GDTR 与 IDTR	327
4.7.6	加载 RIP、RSP 和 RFLAGS	327
4.7.7	加载 PDPTE 表项	327
4.8	刷新处理器 cache	328
4.9	更新 Vritual-APIC 状态	328
4.9.1	PPR 虚拟化	329
4.9.2	虚拟中断评估与 delivery	329
4.10	加载 MSR-load 列表	329
4.10.1	IA32_EFER 的加载处理	330

4.10.2	其他 MSR 字段的加载处理	331
4.11	由加载 guest state 引起的 VM-entry 失败	331
4.12	事件注入	332
4.12.1	注入事件的 delivery	335
4.12.1.1	保护模式下的事件注入	335
4.12.1.2	实模式下的事件注入	338
4.12.1.3	virtual-8086 模式下的事件注入	338
4.12.2	注入事件的间接 VM-exit	339
4.13	执行 pending debug exception	341
4.13.1	注入事件下的 #DB 异常 delivery	342
4.13.2	例子 4-1	346
4.13.3	非注入事件下的 #DB 异常 delivery	351
4.14	使用 MTF VM-exit 功能	354
4.14.1	注入事件下的 MTF VM-exit	354
4.14.2	非注入事件下的 MTF VM-exit	355
4.14.3	MTF VM-exit 与其他 VM-exit	355
4.14.4	MTF VM-exit 的优先级别	356
4.14.5	例子 4-2	356
4.15	VM-entry 后直接导致 VM-exit 的事件	362
4.15.1	VM-exit 事件的优先级别	362
4.15.2	TPR below threshold VM-exit	363
4.15.3	pending MTF VM-exit	364
4.15.4	由 pending debug exception 引发的 VM-exit	364
4.15.5	VMX-preemption timer	364
4.15.6	NMI-window exiting	366
4.15.7	interrupt-window exiting	367
4.16	处理器的可中断状态	367
4.16.1	中断的阻塞状态	367
4.16.2	阻塞状态的解除	368
4.16.3	中断的阻塞	369
4.16.4	VM-entry 后的可中断状态	370
4.17	处理器的活动状态	370

4.17.1 active 与 inactive 状态	371
4.17.2 事件的阻塞	371
4.17.3 inactive 状态的唤醒	372
4.17.4 VM-entry 后的活动状态	372
4.18 VM-entry 的机器检查事件	373
第 5 章 VM-exit 处理	374
5.1 无条件引发 VM-exit 的指令	374
5.2 有条件引发 VM-exit 的指令	375
5.3 引发 VM-exit 的事件	377
5.4 由于 VM-entry 失败导致的 VM-exit	380
5.5 例子 5-1	380
5.6 指令引发的异常与 VM-exit	385
5.6.1 优先级高于 VM-exit 的异常	386
5.6.2 VM-exit 优先级高于指令的异常	387
5.6.3 例子 5-2	387
5.7 VM-exit 的处理流程	389
5.8 记录 VM-exit 的相关信息	390
5.9 更新 VM-entry 区域字段	391
5.10 更新处理器状态信息	391
5.10.1 直接 VM-exit 事件下的状态更新	393
5.10.2 间接 VM-exit 事件下的状态更新	394
5.10.3 其他情况下的状态更新	395
5.11 保存 guest 环境信息	397
5.11.1 保存控制寄存器, debug 寄存器及 MSR	397
5.11.2 保存 RIP 与 RSP	397
5.11.3 保存 RFLAGS	399
5.11.4 保存段寄存器	399
5.11.5 保存 GDTR 与 IDTR	400
5.11.6 保存 activity 与 interruptibility 状态信息	400
5.11.7 保存 pending debug exception 信息	400
5.11.8 保存 VMX-preemption timer 值	402

5.11.9 保存 PDPTE	402
5.11.10 保存 SMBASE 与 VMCS-link pointer	403
5.12 保存 MSR-store 列表	403
5.13 加载 host 环境	404
5.13.1 加载控制寄存器	404
5.13.2 加载 DR7 与 MSR	405
5.13.3 加载 host 段寄存器	405
5.13.3.1 加载 selector	406
5.13.3.2 加载 base	406
5.13.3.3 加载 limit	406
5.13.3.4 加载 access rights	407
5.13.4 加载 GDTR 与 IDTR	408
5.13.5 加载 RIP, RSP 及 RFLAGS	408
5.13.6 加载 PDPTE	408
5.14 更新 host 处理器状态信息	409
5.15 刷新处理器 cache 信息	409
5.16 加载 MSR-load 列表	410
5.17 VMX-abort	411
第 6 章 内存虚拟化	412
6.1 EPT (扩展页表) 机制	412
6.1.1 EPT 机制概述	413
6.1.1.1 guest 分页机制与 EPT	413
6.1.2 EPT 页表结构	416
6.1.3 guest-physical address	417
6.1.4 EPTP	417
6.1.5 4K 页面下的 EPT 页表结构	418
6.1.6 2M 页面下的 EPT 页表结构	422
6.1.7 1G 页面下的 EPT 页表结构	424
6.1.8 EPT 导致的 VM-exit	426
6.1.8.1 EPT violation	426
6.1.8.2 EPT misconfiguration	427

6.1.8.3	EPT 页故障的优先级	428
6.1.8.4	修复 EPT 页故障	431
6.1.9	accessed 与 dirty 标志位	436
6.1.10	EPT 内存类型	438
6.1.11	EPTP switching	440
6.1.12	实现 EPT 机制	442
6.2	Cache 管理	454
6.2.1	linear mapping (线性映射)	455
6.2.2	guest-physical mapping (guest 物理映射)	456
6.2.3	combined mapping (合并映射)	457
6.2.4	cache 域	458
6.2.5	cache 建立	463
6.2.6	cache 刷新	465
6.2.6.1	INVLPG 指令刷新 cache	468
6.2.6.2	INVPCID 指令刷新 cache	468
6.2.6.3	INVVPID 指令刷新 cache	469
6.2.6.4	INVEPT 指令刷新 cache	470
6.2.6.5	INVVPID 指令使用指南	470
6.2.6.6	INVEPT 指令使用指南	471
6.3	内存虚拟化管理	473
6.3.1	分配物理内存	473
6.3.2	实模式 guest OS 内存处理	475
6.3.3	guest 内存虚拟化	476
6.3.3.1	guest 虚拟地址转换	477
6.3.3.2	guest OS 的 cache 管理	479
6.4	例子 6-1	482
6.4.1	GuestBoot 模块	483
6.4.2	GuestKernel 模块	486
6.4.3	VSIB 结构	495
6.4.4	VMM 初始化 guest	498
6.4.5	使用 VMX-preemption timer	503
6.4.6	host 处理流程	507

6.4.7 运行结果	511
第 7 章 中断虚拟化	522
7.1 异常处理	522
7.1.1 反射异常给 guest	523
7.1.2 恢复 guest 异常	526
7.1.2.1 直接恢复	526
7.1.2.2 例子 7-1	527
7.1.2.3 恢复原始向量事件	533
7.1.3 处理任务切换	535
7.1.3.1 检查任务切换条件	535
7.1.3.2 VMM 处理任务切换	537
7.1.3.3 恢复 guest 运行	547
7.1.3.4 例子 7-2	551
7.2 Local APIC 虚拟化	554
7.2.1 监控 guest 访问 local APIC	554
7.2.1.1 例子 7-3	555
7.2.2 local APIC 虚拟化机制	571
7.2.3 APIC-access page	573
7.2.3.1 APIC-access page 的设置	574
7.2.4 虚拟化 x2APIC MSR 组	577
7.2.5 virtual-APIC page	578
7.2.6 APIC-access VM-exit	581
7.2.6.1 APIC-access VM-exit 优先级	581
7.2.7 虚拟化读取 APIC-access page	582
7.2.8 虚拟化写入 APIC-access page	584
7.2.9 虚拟化基于 MSR 读 local APIC	587
7.2.10 虚拟化基于 MSR 写 local APIC	588
7.2.11 虚拟化基于 CR8 访问 TPR	589
7.2.12 local APIC 虚拟化操作	589
7.2.12.1 TPR 虚拟化	590
7.2.12.2 PPR 虚拟化	591

7.2.12.3	EOI 虚拟化	591
7.2.12.4	Self-IPI 虚拟化	593
7.2.13	虚拟中断的评估与 delivery	593
7.2.13.1	虚拟中断的评估	594
7.2.13.2	虚拟中断的 delivery	596
7.2.14	posted-interrupt 处理	597
7.3	中断处理	601
7.3.1	拦截 INT 指令	601
7.3.1.1	处理 IDTR.limit	602
7.3.1.2	处理#GP 异常	605
7.3.1.3	处理中断 delivery	608
7.3.1.4	完成中断的 delivery 操作	618
7.3.1.5	例子 7-4	628
7.3.2	处理 NMI	632
7.3.2.1	拦截 NMI	632
7.3.2.2	虚拟 NMI	634
7.3.3	处理外部中断	634
7.3.3.1	拦截外部中断	634
7.3.3.2	转发外部中断	635
7.3.3.3	监控 guest 设置 8259	637
7.3.3.4	例子 7-5	642

第 1 章

系统平台

开宗明义，本书讨论基于 Intel VMX 架构的处理器虚拟化技术，并且实现了若干个例子辅助说明。在展开本书内容之前，我们首先需要构建一个系统平台来支撑本书的所有例子。

系统平台组件引导机器进入最终环境。我们要求，书中的例子既可以运行在 64 位环境，也可以运行在 32 位环境。为了方便测试与观察信息，我们还需要开启多处理器环境。

1.1 环境及工具

以 4 个逻辑处理器平台为例，机器启动后最终进入一个等待状态，如图 1-1 所示。

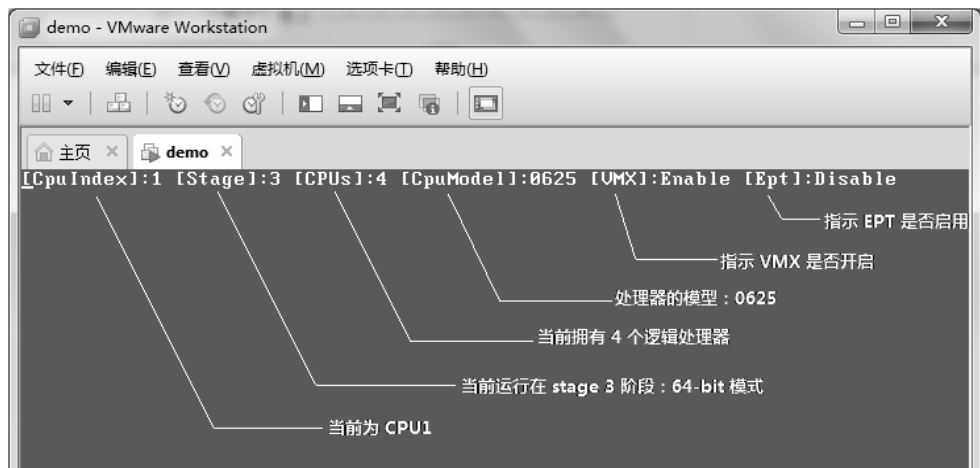


图 1-1

图 1-1 所显示的信息是处理器最后调用 `update_system_status` 例程的结果，它输出当前系统的信息，在 `lib\crt.asm` 模块里实现。

处理器编号

CpuIndex 指示处理器的编号，每个处理器从 0 开始编号（记为 **CPU0**）。假如平台上拥有 4 个逻辑处理器，那么 **CPU3** 对应第 4 个处理器。在 **PCB**（Processor Control Block，处理器控制块）结构里的 ProcessorIndex 值用来记录处理器当前的编号。

图 1-1 显示，当前处理器为 **CPU1**。

运行阶段

Stage 指示系统当前运行在哪个阶段。系统平台分三个阶段（参见第 1.5 节）。

- Stage1：处理器当前处于 32 位**未分页**的保护模式，这个时候是关闭分页机制。
- Stage2：处理器当前处于 32 位**已分页**的保护模式，这是 legacy 模式的最终环境。
- Stage3：处理器当前处于 **IA-32e**（即 long-mode）下的 64-bit 模式。

图 1-1 显示，处理器当前处于 stage3 阶段（也就是 64 位模式）。

处理器个数

CPUs 指示系统当前可用的处理器个数，这是在启动阶段检测出来的。实际上，使用代码枚举出来更可靠些。因为，存在 BIOS 关闭某些处理器的可能性，当然这个可能性是极小的。

图 1-1 显示，当前系统存在 4 个可用的逻辑处理器，从 CPU0 到 CPU3。

处理器模型

CpuModel 指示处理器的模型信息，包括了处理器家族与型号。显示当前处理器是 **0625H**。06H 指 P6 家族（包括了 Core 家族），25H 属于第 1 代的 core 架构，具体为 Westmere 架构。

VMX 开启

VMX enable 指示 VMX 模式已经开启，VMX disable 则指示 VMX 模式关闭。实际上，每个处理器都开启了 VMX 模式。由调用 vmx_operation_enter 函数进入，实现在 lib\Vmx\VmxInit.asm 模块里。

EPT 开启

Ept enable 指示 EPT 映射机制已经开启，Ept disable 则指示 EPT 映射机制关闭。处理器在进入 guest 前，根据 guest 的设置决定是否开启 EPT 映射机制。

1.1.1 使用 VMware

本书的大部分例子可以使用 VMware 来运行（VMware-9 或者 VMware-10），只有少数例子需要在真实机器上运行。下面将介绍如何使用 VMware 来运行例子中的 **demo.img** 映像文件。

运行 VMware 后，使用典型配置新建一个虚拟机，命名为“**demo**”。“客户机操作

系统”选择“其他(O)”，版本为“其他 64 位”。虚拟机内存使用 **256MB** 就足够了。

接下来设置处理器的个数，以及虚拟化引擎，如图 1-2 所示。

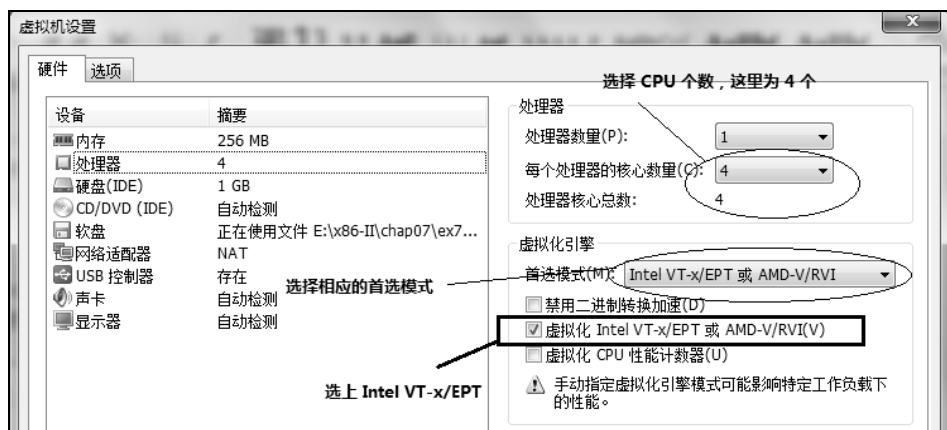


图 1-2

物理处理器数量为 1，而可以选择的处理器核心数量取决于**真实的逻辑处理器**个数。以 4 个逻辑处理器平台为例，那么可以选择核心数量从 1 到 4，超过 4 个将不被支持。

在一个双核四线程的处理器上，假如**选择了 2 个核心数量**，运行 demo.img 的结果如图 1-3 所示：检测到的 CPU 个数为 2。



图 1-3

当物理处理器支持 Intel VT-x 虚拟化技术时，我们需要在虚拟化引擎设置里，选上“虚拟化 Intel VT-x/EPT 或 AMD-V/RVI(V)”选项。并且首选模式选择“Intel VT-x/EPT 或 AMD-V/RVI”此项。这样才能在 VMware 中使用 VMX 模式。

在 VMware 里，我们使用软盘映像文件 demo.img，这个映像文件位于每个例子目录里。例如图 1-4 中的 chap07\ex7-5\demo.img。



图 1-4

1.1.2 使用 Bochs

最新预编译的 Bochs 2.6.2 只支持基本的 VMX 特性。因此，本书例子基本上没有几个能在 Bochs 上运行，但我们可以使用 Bochs 调试系统平台。

1.1.3 在真实机器上运行

在后面我们将看到，实现了一个 Build 工具，用于编译和生成所需的映像文件。其中包括写入物理 U 盘，我们将使用 U 盘启动，在真实机器上运行本书中的例子。

1.1.4 Build 工具

在每个例子目录下，含有一个 Build 工具，它是一个批处理命令。Build 工具用来编译例子用到的所有源代码，并且生成 merge 的配置文件 **config.txt**，用来将编译好的 binary 文件写入目标映像文件。

下面是 Build 工具的批处理命令源代码。

代码片段 1-1:

```
@echo off

set Params=
set ConfigString1=
set ConfigString2=
set Target=x86
set GuestEnable=No
set GuestTarget=x86

:Loop
set s=%1
set s=%s:~2%
if "%s%"=="__X64" (set Target=x64)
if "%s%"=="GUEST_ENABLE" (set GuestEnable=Yes)
if "%s%"=="GUEST_X64" (
    set GuestTarget=x64
```

```

        goto :CommandLineNext
    )
set Params=%Params% %1
:CommandLineNext
shift
if "%1"==" " (goto :Next) else goto :Loop

:Next

if %GuestTarget%==x64 (
    if %Target%==x64 (
        set Params=%Params% -DGUEST_X64
    )
)

::
:: 注意:
:: 1) setup.asm 模块编译时加入 -D__STAGE1 参数
:: 2) proected.asm 模块编译时加入 -D__STAGE2 参数
:: 3) long.asm 模块编译时加入 -D__STAGE3 参数
::

nasm -I..\ ..\..\common\boot.asm %Params%
if %errorlevel%==0 (nasm -I..\ ..\..\common\setup.asm %Params% -D__STAGE1) else
goto :END
if %errorlevel%==0 (
    if "%Target%"=="x64" (
        set ConfigString1=..\..\common\long,0,c.img,256,200
        set ConfigString2=..\..\common\long,0,demo.img,256,200
        nasm -I..\ ..\..\common\long.asm %Params% -D__STAGE3
    ) else (
        set ConfigString1=..\..\common\protected,0,c.img,64,200
        set ConfigString2=..\..\common\protected,0,demo.img,64,200
        nasm -I..\ ..\..\common\protected.asm %Params% -D__STAGE2
    )
) else goto :END
if %errorlevel%==0 (
    if "%GuestEnable%"=="Yes" (
        nasm -I..\ ..\..\lib\Guest\GuestBoot.asm %Params% -D__STAGE4 &&
nasm -I..\ ..\..\lib\Guest\GuestKernel.asm %Params% -D__STAGE4
    )
) else goto :END
if %errorlevel%==0 (goto DoMerge) else goto :END

:DoMerge

:: $$$ 生成 config.txt 文件, 并使用 merge 命令写入映像文件 $$$

::
:: #### 为 fat32 文件格式的 U 盘启动 ####
::
echo #### 下面是 build.bat 自动生成的配置信息! #### > config.txt
echo. >> config.txt
echo. >> config.txt
echo #### 为 fat32 文件格式的 U 盘启动 #### >> config.txt
echo ..\..\common\boot,0,c.img,63,1 >> config.txt
echo ..\..\common\setup,0,c.img,1,60 >> config.txt
echo %ConfigString1% >> config.txt

```

```

if "%GuestEnable%"=="Yes" (
    echo ..\..\lib\Guest\GuestBoot,0,c.img,512,6 >> config.txt
    echo ..\..\lib\Guest\GuestKernel,0,c.img,520,20 >> config.txt
)
echo. >> config.txt

::
:: #### 写入 floppy 盘映像 ####
::
echo #### 写入 floppy 盘映像 #### >> config.txt
echo ..\..\common\boot,0,demo.img,0,1 >> config.txt
echo ..\..\common\setup,0,demo.img,1,60 >> config.txt
echo %ConfigString2% >> config.txt
if "%GuestEnable%"=="Yes" (
    echo ..\..\lib\Guest\GuestBoot,0,demo.img,512,6 >> config.txt
    echo ..\..\lib\Guest\GuestKernel,0,demo.img,520,20 >> config.txt
)
echo. >> config.txt

::
:: #### 写入 u 盘 ####
::
echo #### 写入 u 盘 #### >> config.txt
echo c.img,0,\\.g:,0,600 >> config.txt

::
:: 执行 merge 命令
::
call merge

:END

```

Build 工具所做的主要工作如下。

(1) 解析命令行参数，主要是分析是否定义了__X64，GUEST_ENABLE 以及 GUEST_X64 符号。

- 源码中将根据定义的__X64 符号来决定是否生成 64 位代码。
- 定义了 GUEST_ENABLE 符号，表明需要编译 guest 端代码。否则，不需要编译 guest 端代码。
- 定义了 GUEST_X64 符号，表明需要将 guest 端代码编译为 64 位代码。否则，表明 guest 端代码为 legacy 模式。

(2) 分别编译 boot.asm，setup.asm 模块代码，并根据__X64 与 GUEST_ENALBE 符号决定：

- 假如定义了__X64 符号，则编译 long.asm 模块代码。否则，编译 protected.asm 模块代码。
- 假如定义了 GUEST_ENABLE 符号，则编译 GuestBoot.asm 与 GuestKernel.asm 模块代码。否则不进行编译。

(3) 生成 merge 工具的配置文件，分别向 config.txt 文件添加 c.img 与 demo.img 镜像文件的写入记录。

- 当定义了 GUEST_ENABLE 符号后，还需要写入 GuestBoot 与 GuestKernel 模块。
- C.img 与 demo.img 镜像文件除了 boot 模块写入的位置不同外，其他模块写入位置都是一样的。

(4) 执行 merge 命令，根据 config.txt 配置文件写入 c.img, demo.img 以及 U 盘（插入 U 盘时）。

符号定义

本书源码内使用了若干个符号，用来控制生成的代码类型，包括主要的 __X64, DEBUG_RECORD_ENABLE, GUEST_ENABLE 以及 GUEST_X64 符号。

假如不需要编译 guest 端代码，我们可以选择使用下面的命令行：

- Build
- Build -D __X64

定义符号 __X64，可以指示 nasm 将源码编译为 64 位。当定义了 __X64 符号时，我们的代码最终运行在 stage 3 阶段（64-bit）。否则将运行在 stage 2 阶段。

当我们需要编译 guest 端代码时，可以使用下面的命令行：

- build -DGUEST_ENABLE -D __X64 -DGUEST_X64
- build -DGUEST_ENABLE -D __X64
- build -DGUEST_ENABLE

定义符号 GUEST_X64，可以指示 nasm 将 guest 端代码编译为 64 位，但 host 也必须是 64 位。因此，GUEST_X64 符号与 __X64 符号必须同时定义。没有定义 GUEST_X64 符号时，guest 端代码被编译为 32 位。

如果需要开启 debug record（调试记录）功能，则需要定义 DEBUG_RECORD_ENABLE 符号。在 Build 批处理文件里，在编译源码时也定义了 __STAGE1, __STAGE2 以及 __STAGE3 符号，为源码处于的目标阶段进行相关控制。

1.2 64 位与 32 位代码的混合编译

我们可以将本书源码编译为 64 位版本。但是对于大多数模块，我们只维护一份代码。也就是，并不需要分别编写 64 位版本以及 32 位版本的代码。对于少量较为复杂的代码以及遗留下来的代码，本书源码里才区分 64 位以及 32 位版本。

1.2.1 使用符号 __X64

为了做到 64 位以及 32 位版本只维护一份代码，我们需要将代码以 32 位形式进行编译。同时，引进 __X64 符号控制机器码的生成。

bits 32 ;; 重要：以 32 位形式进行编译！

```

    ... ..
#ifdef __x64
    ;;
    ;; 此处插入 64 位特定的代码!
    ;;
#else
    ;;
    ;; 此处插入 32 位特定的代码!
    ;;
#endif
    ... ..

```

注意：由于在 64-bit 模式下，绝大多数指令的操作数默认为 32 位，导致大多数指令机器码在 64 位以及 32 位环境下是一样的！只有少量的指令，我们需要进行特别处理。因此，引进__X64 符号是为了解决某些特定指令的使用。

代码统一在 32 位下编译后，处理器无论运行在 64-bit 模式下，还是 32 位保护模式下，都能够正确执行我们的代码。当然，除了某些需要特殊处理的指令，下面我们将一一了解。

1.2.2 指令操作数

在继续讲解 64 位与 32 位混合编译之前，我们有必要先了解一下 x86/x64 体系中指令的 operand size（操作数宽度），以及 address size（地址宽度）相关知识。更详细的介绍请浏览作者站点的以下页面 <http://www.mouseos.com/x64/index.html>。

指令的操作数大小及地址大小有 default（默认）和 effective（有效）两种情况。表 1-1 列出了处理器各种工作模式下的 default operand size（默认操作数宽度）与 default address size（默认地址宽度），effective operand size（有效的操作数宽度）与 effective address size（有效的地址宽度）。

表 1-1

处理器模式	默认的 operand size	有效的 operand size	默认的 address size	有效的 address size
64-bit	64	64	64	64
		16		32
	32	64		
		32		
		16		
		16		
compatibility protected real virtual-8086	32	32	32	32
		16		16
	16	32	16	32
		16		16

指令的默认 operand size、address size 由 CS.D 和 CS.L 位 (IA-32e/long-mode 可用) 决定。

- 在 IA-32e (long-mode) 模式下, 当 CS.L = 1 并且 CS.D = 0 时, 指示处理器当前处于 64-bit 模式里。有下面的情况:
 - 大部分指令的操作数宽度默认为 32 位, 少量指令操作数宽度默认为 64 位。
 - 地址宽度默认为 64 位。
- 在 IA-32e (long-mode) 模式下, 当 CS.L = 0 时, 指示处理器当前处于 compatibility (兼容) 模式里。操作数宽度与地址宽度的情景与 legacy 模式下是一样的。
- 在 protected, real 以及 virtual-8086 模式下, CS.L 为保留位, 必须为 0 值。在 compatibility 模式下, CS.L 也为 0 值。它们有以下情况:
 - 当 CS.D = 1 时, 指示操作数宽度以及地址宽度默认为 32 位。
 - 当 CS.D = 0 时, 指示操作数宽度以及地址宽度默认为 16 位。

另外, stack address (栈地址, 或栈指针) 的宽度由 SS.B 位决定, 但不能被 override (改写)。

- 在 64-bit 模式下, stack address 固定为 64 位。
- 在 compatibility, protected, real 以及 virtual-8086 模式下, 有以下情况:
 - 当 SS.B = 1 时, stack address 为 32 位。
 - 当 SS.B = 0 时, stack address 为 16 位。

栈地址宽度不能被 address-size override prefix (地址宽度改写前缀) 修改, 例如下面的代码。

```
DB 67h                ; address override prefix
push eax              ; 32 位的栈地址不能被改写
```

上面的示例, 企图使用地址宽度改写前缀 67H 修改栈地址为 16 位是不起作用的。PUSH 指令中的 32 位栈地址依然是 32 位。

然而, 指令的 default operand size (默认操作数宽度), 以及 default address size (默认地址宽度) 都可以使用 override prefix (改写前缀) 进行修改。

- **66H** (operand-size override prefix), 使用 66H 字节改变指令的默认操作数宽度。
- **67H** (address-size override prefix), 使用 67H 字节改变内存寻址中的地址宽度。

编译器根据情况使用 66H 与 67H 字节, 在默认操作数宽度及地址宽度的基础上, 改变为表 1-1 中所列出的有效操作数宽度及地址宽度。另外, 我们可以手动在指令前面添加 override prefix 字节。

```
DB 66h                ; operand size override prefix
lgdt [GdtPointer]     ; 改变实模式下 16 位默认操作数宽度
```

如上面指令示例是 66H 字节的典型应用。实模式下, 我们可以使用 66H 字节改写 16 位的默认操作数宽度, 变为 32 位, 使得 LGDT 指令可以读取 32 位的 GDT 基址。

REX prefix

在 64-bit 模式下，使用 REX prefix（从 40H 到 4FH）来改写指令操作数宽度以及地址寻址模式，8 位的 REX prefix 的结构如表 2-2 所示。

表 2-2

位域	名字	描 述
0	B	扩展 ModRM.base, ModRM.r/m 及 opcode.reg 域
1	X	扩展 SIB.index 域
2	R	扩展 ModRM.reg 域
3	W	width 标志位，W=1 时使用 64 位操作数，W=0 时使用默认操作数宽度
7:4	0100B	40H - 4FH

注意：从 40H 到 4FH 的 REX prefix，它们的 W 位为 0，表明使用默认的操作数宽度。如果默认操作数为 32 位，则有效的操作数依然为 32 位。

```

89 D8 ; mov eax, ebx
... ..
48 89 D8 ; mov rax, rbx

```

如上面所示，指令的机器编码 89 D8 对应汇编语句为“mov eax, ebx”。当加上 48H 这个 REX prefix 后形成了汇编语句“mov rax, rbx”。这是由于 **MOV 指令的默认操作数宽度为 32 位**。利用这一点，我们可以实现 64 位与 32 位代码的混合编译。

在 inc\cpu.inc 文件里，以**宏的形式**实现了对全部 REX prefix 字节的定义。最为常用的是 REX.Wrbx 宏，它的定义如以下代码所示。

代码片段 1-2:

```

; -----
; 宏: REX.Wrbx
; 描述:
;      1) 定义 REX prefix
; -----
%macro REX.Wrbx 0
%ifndef __STAGE1
    %ifdef __X64
        DB 48h
    %endif
%endif
%endmacro

```

当定义了__X64 符号后，REX.Wrbx 宏的值为 **48H**。当不存在__X64 符号时，REX.Wrbx 宏为空。因此，我们可以像下面代码所示使用这个宏。

```

REX.Wrbx ; 插入 REX.Wrbx 宏
mov eax, ebx ; 在 32 位下进行编译

```

把这条指令放在 32 位下进行编译。当定义了__X64 符号时，指令被编译为 **48 89 D8**，它就可以执行在 64-bit 模式下。当没有定义__X64 符号时，指令被编译为 **89 D8**，它

在 legacy 以及 compatibility 模式下执行。

我们将会看到：在本书源码里，在大量的正常指令前面插入了 REX.Wrxb 宏。正如前面所说，这个宏是否起作用取决于__X64 符号是否定义。

更改寄存器

另外，我们可能还会看到少量利用 REX 系列宏来改变寄存器。

```
REX.wrxb      ; REX.B = 1, REX.W = 0
mov eax, 25 * 80 * 2 ; mov r8d, 25 * 80 * 2
```

在定义了__X64 符号的情况下，REX.wrxb 宏产生的值为 41H。如上面代码所示，将 REX.wrxb 宏插在指令“mov eax, 25 * 80 * 2”前面，它产生的指令机器码实际对应为“mov r8d, 25 * 80 * 2”指令。这是由于使用了 REX.B 来扩展 Opcode.reg 域。

1.2.3 64-bit 模式下其他指令处理

实际中，仅仅使用 REX 系列宏是不够的，还需要处理一些特定的指令情形。

处理 INC 与 DEC 指令

在 legacy 与 compatibility 模式下，opcode 从 40H 到 47H 属于 INC 指令，而 opcode 从 48H 到 4FH 属于 DEC 指令。但在 64-bit 模式下，它们改变为 REX prefix 字节。

当 64 位模块的 INC 与 DEC 指令放在 32 位下进行编译时，它们实际上仅仅是 REX prefix，并不是完整的 INC 与 DEC 指令。因此，我们需要使用 2 字节的 opcode 进行代替。

在 inc\cpu.inc 文件里定义了 INCv 与 DECv 宏，实现 2 字节 opcode 的 INC 与 DEC 指令。代码如下。

代码片段 1-3:

```
; -----
; DECv:
; 描述:
; 1) 这个宏使用 FF /1 opcode 的 dec 指令
; -----
%macro DECv 1
%if %1==eax
    DW 0C8FFh
%elif %1==ecx
    DW 0C9FFh
%elif %1==edx
    DW 0CAFFh
%elif %1==ebx
    DW 0CBFFh
%elif %1==esp
    DW 0CCFFh
%elif %1==ebp
    DW 0CDFFh
%elif %1==esi
    DW 0CEFFh
```



```

%elif %1==edi
    DW 0CFFh
%endif
%endmacro

; -----
; INCV:
; 描述:
;     1) 这个宏使用 FF /1 opcode 的 inc 指令
; -----
%macro INCv 1
%if %1==eax
    DW 0C0FFh
%elif %1==ecx
    DW 0C1FFh
%elif %1==edx
    DW 0C2FFh
%elif %1==ebx
    DW 0C3FFh
%elif %1==esp
    DW 0C4FFh
%elif %1==ebp
    DW 0C5FFh
%elif %1==esi
    DW 0C6FFh
%elif %1==edi
    DW 0C7FFh
%endif
%endmacro

```

注意：INCv 与 DECv 宏的输入参数只支持 eax 到 edi 寄存器。那是因为，我们要在 32 位下进行编译，不能使用 64 位的寄存器。

我们可以像下面示例一样使用 INCv 与 DECv 宏。

```

INCv ecx      ; 等价于 inc ecx
DECv ecx      ; 等价于 dec ecx

```

当需要 64 位操作数时，在指令前面插入 **REX.Wrb** 宏即可。

处理内存的直接寻址

当操作数使用内存直接寻址时，统一在 32 位下编译会遇到很大麻烦。下面我们以两个例子对内存直接寻址在 32 位与 64 位下的不同，以及解决混合编译时产生的麻烦进行说明。

- 例子一，如下面的两条指令所示：

```

mov eax, [4000h]      ; A1 00400000
mov ecx, 10           ; B9 0A000000

```

在 32 位下编译，这两条指令产生的指令编码分别为 A1 00 40 00 00 以及 B9 0A 00 00 00。此时，我们试图将它们放在 64-bit 模式下使用，那么，处理器将这两条指令编码解析为如下所示：

```

A1 00400000B90A0000      ; mov eax, [00000AB900004000h]
00                        ; 下一指令边界！

```

由于指令编译时使用的 opcode 码为 A1，在 64-bit 模式下它的 offset 是 64 位值。因此，处理器 fetch 指令时将读取 64 位的 offset 值，导致原本的指令编码被拆分，而下一条指令边界也是错误的，产生的这个结果不是我们需要的。

解决办法是：如果需要放在 32 位里进行编译，并且试图在 64-bit 模式下运行。那么就要避免使用直接内存寻址。

因此，我们通过间接内存寻址来解决这个麻烦。将前面的指令修改如下：

```
mov eax, 4000h           ; B8 00400000
mov eax, [eax]           ; 8B 00
mov ecx, 10              ; B9 0A000000
```

现在，它们就可以很正常地在 64-bit 模式下运行。机器码 8B 00 被解析为“mov eax, [rax]”指令，而其余的两条指令机器码在 32 位与 64 位环境是一样的。

- 例子二，假如我们需要在 64-bit 模式下执行指令“mov rbp, [gs: 0]”。那么在 32 位下编译，我们编写下面的代码，然后试图放在 64-bit 模式里运行：

```
REX.Wrxb                ; 48
mov ebp, [gs: 0]         ; 65 8B 2D 00000000
```

使用 32 位进行编译生成的指令编码序列为：48 65 8B 2D 00 00 00 00。但是，将它放在 64-bit 下运行将产生两个解析错误：

(1) 由于 prefix 而产生的错误解析。在指令编码序列里，REX prefix 必须在 legacy prefix 之后。也就是 48H 必须放在 65H 之后。否则将解析不到正确的指令。

(2) 由于内存的直接寻址 “[gs: 0]”而产生的错误解析，这条指令编码中的 ModRM 字节为 2DH（即 ModRM.mod = 00B 并且 ModRM.r/m = 101B）。如图 1-5 所示，在非 64-bit 模式与 64-bit 模式下有不同的解析，在 64-bit 模式下属于新增的“RIP-relative”寻址模式（RIP + disp32）。

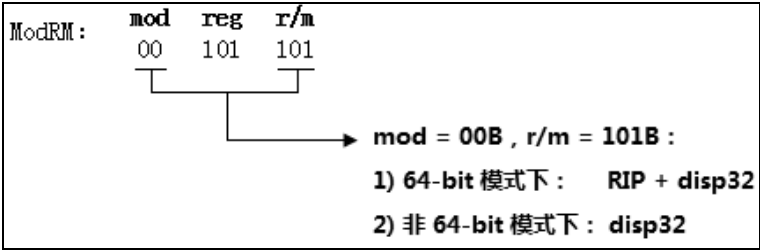


图 1-5

在 64-bit 模式下，这个操作数寻址变为 RIP + 00000000H，导致提取了错误的值。这两个方面的解析错误不是我们所要的。

遇到这个情况，我们需要手工地写入机器码，或者定义一个宏来解决。在 inc\cpu.inc 文件里，定义了下面 4 个宏。代码如下。

代码片段 1-4:

```
%define LoadFsBaseToRbp    DB 64h, 48h, 8Bh, 2Ch, 25h, 00h, 00h, 00h, 00h
```

```
%define LoadGsBaseToRbp    DB 65h, 48h, 8Bh, 2ch, 25h, 00h, 00h, 00h, 00h
%define LoadFsBaseToRbx    DB 64h, 48h, 8Bh, 1ch, 25h, 00h, 00h, 00h, 00h
%define LoadGsBaseToRbx    DB 65h, 48h, 8Bh, 1ch, 25h, 00h, 00h, 00h, 00h
```

它们在 64-bit 模式下，分别被解析为下面的指令：

```
mov rbp, [fs: 0]           ; 等价于 mov rbp, [fs: SDA.Base]
mov rbp, [gs: 0]           ; 等价于 mov rbp, [gs: PCB.Base]
mov rbx, [fs: 0]           ; 等价于 mov rbx, [fs: SDA.Base]
mov rbx, [gs: 0]           ; 等价于 mov rbx, [gs: PCB.Base]
```

这几条指令的目的是将 SDA.Base 或者 PCB.Base 值赋给 RBP 或 RBX 寄存器。在本书的源码里，常常会看到下面的使用方法：

```
%ifdef __X64
    LoadGsBaseToRbp
%else
    mov ebp, [gs: PCB.Base]
%endif
```

假如定义了 __X64 符号，则使用 LoadGsBaseToRbp 宏生成 “**mov rbp, [gs: PCB.Base]**” 指令。如果没有定义 __X64 符号，则为 32 位的 “**mov ebp, [gs: PCB.Base]**” 指令。

Natural-width 类指令

在 64-bit 模式下，有部分指令的默认操作数宽度为 64 位，如表 1-3 所示。

表 1-3

指 令	依赖于	描 述
CALL	RIP 与 RSP	near call
RET	RIP 与 RSP	near ret
JMP	RIP	near jmp
Jcc	RIP	条件 jmp
LOOP	RIP	循环
LOOPcc	RIP	条件循环
PUSH	RSP	压 stack
POP	RSP	出 stack
PUSHF/Q	RSP	压入 eflags/rflags
POPF/Q	RSP	弹出 eflags/rflags
PUSH FS	RSP	压入 FS
PUSH GS	RSP	压入 GS
POP FS	RSP	弹出 FS
POP GS	RSP	弹出 GS

我们可以称它们为 **natural-width**（自然宽度）类指令，这些指令的默认操作数宽度就取决于 CS.L 及 CS.D 标志位。

- 当 CS.L = 1, 并且 CS.D = 0 时 (64-bit 模式) 是 64 位。
- 当 CS.D = 1 时是 32 位。
- 当 CS.D = 0 时是 16 位。

当在 32 位下进行编译时, 这类指令不需要处理直接可用, 它们的指令编码在 32 位以及 64 位下是一样的。代码示例如下。

```

        push ebp                                ; push rbp
        push ebx                                ; push rbx
#ifdef __X64
        LoadGsBaseToRbp                        ; mov rbp, [gs: PCB.Base]
#else
        mov ebp, [gs: PCB.Base]
#endif
        ... ..

        REX.Wrb
        mov eax, [ebp + PCB.IpiRoutinePointer] ; mov rax, [rbp + IpiRoutinePointer]
        call eax                               ; call rax

        ... ..

        pop ebx                                ; pop rbx
        pop ebp                                ; pop rbp
        ret                                    ; ret

```

这段代码在 32 位下编译后, 既可以运行在 32 位环境下, 也可以运行在 64-bit 模式下。取决于在编译时 __X64 符号的定义。

1.2.4 函数重定义表

对于较为复杂的部分函数分别单独实现了 32 位与 64 位版本。例如, do_virtual_address_mapping32 函数进行 32 位下的虚拟地址映射, 而 do_virtual_address_mapping64 函数进行 64 位下的虚拟地址映射。

```

#ifdef __X64
    call do_virtual_address_mapping64          ; 调用 64 位版本
#else
    call do_virtual_address_mapping32          ; 调用 32 位版本
#endif

```

在调用不同版本的函数时, 我们需要根据 __X64 符号来选择调用相应的函数。这样让我们的编码变得烦琐起来。

因此, 为了方便代码的编写, 我们需要统一使用 do_virtual_address_mapping 作为函数名字进行调用。于是, 就产生了函数重定义。

inc\LibDef.inc 这个头文件定义了若干个重定义关系。代码如下。

```

;;
;;
;; 重定义函数名, 目的是保持一致
;;
#ifdef __STAGE1

```

```

;;
;; 定义在 stage1 阶段使用
;;
#define clear_4k_page          clear_4k_page32
#define clear_4k_buffer        clear_4k_buffer32
#define clear_4k_page_n        clear_4k_page_n32
#define clear_4k_buffer_n      clear_4k_buffer_n32
#define strlen                 strlen32
#define zero_memory            zero_memory32
#define memcpy                 memcpy32
#define delay_with_us          delay_with_us32
#define error_code_default_handler error_code_default_handler32
#define exception_default_handler exception_default_handler32
#define do_virtual_address_mapping do_virtual_address_mapping32
#define do_virtual_address_mapping_n do_virtual_address_mapping32_n
#define nmi_handler            nmi_handler32
#define exception_default_handler.@0 exception_default_handler32.@0

#elifdef __X64

;;
;; 定义在 X64 下
;;
#define clear_4k_page          clear_4k_page64
#define clear_4k_buffer        clear_4k_buffer64
#define clear_4k_page_n        clear_4k_page_n64
#define clear_4k_buffer_n      clear_4k_buffer_n64
#define strlen                 strlen64
#define zero_memory            zero_memory64
#define memcpy                 memcpy64
#define delay_with_us          delay_with_us64
#define error_code_default_handler error_code_default_handler64
#define exception_default_handler exception_default_handler64
#define install_kernel_interrupt_handler install_kernel_interrupt_handler64
#define install_user_interrupt_handler install_user_interrupt_handler64
#define read_idt_descriptor     read_idt_descriptor64
#define write_idt_descriptor    write_idt_descriptor64
#define read_gdt_descriptor     read_gdt_descriptor64
#define write_idt_descriptor64 write_idt_descriptor64
#define setup_sysenter          setup_sysenter64
#define do_virtual_address_mapping do_virtual_address_mapping64
#define do_virtual_address_mapping_n do_virtual_address_mapping64_n
#define do_guest_physical_address_mapping do_guest_physical_address_mapping64
#define do_guest_physical_address_mapping_n do_guest_physical_address_mapping64_n
#define dump_ept_paging_structure dump_ept_paging_structure64
#define nmi_handler            nmi_handler64
#define exception_default_handler.@0 exception_default_handler64.@0

#else

;;
;; 定义在 x86 下
;;
#define clear_4k_page          clear_4k_page32
#define clear_4k_buffer        clear_4k_buffer32
#define clear_4k_page_n        clear_4k_page_n32
#define clear_4k_buffer_n      clear_4k_buffer_n32
#define strlen                 strlen32
#define zero_memory            zero_memory32

```

```

#define memcpy          memcpy32
#define error_code_default_handler  error_code_default_handler32
#define exception_default_handler  exception_default_handler32
#define install_kernel_interrupt_handler  install_kernel_interrupt_handler32
#define install_user_interrupt_handler  install_user_interrupt_handler32
#define read_idt_descriptor  get_idt_descriptor
#define write_idt_descriptor  set_idt_descriptor
#define read_gdt_descriptor  get_gdt_descriptor
#define write_gdt_descriptor  set_gdt_descriptor
#define setup_sysenter  setup_sysenter32
#define do_virtual_address_mapping  do_virtual_address_mapping32
#define do_virtual_address_mapping_n  do_virtual_address_mapping32_n
#define do_guest_physical_address_mapping  do_guest_physical_address_mapping32
#define do_guest_physical_address_mapping_n  do_guest_physical_address_mapping32_n
#define nmi_handler  nmi_handler32
#define exception_default_handler.@0  exception_default_handler32.@0

%endif          ;; __STAGE1

```

经过函数名重定义后，我们在编写代码时，只需要直接调用 `do_virtual_address_mapping` 函数就可以了，而不需要理会是否定义了 `__X64` 符号。这个函数重定义表会帮我们自动转换这个关系。

1.3 地址空间

在本书源码里，我们假设机器上至少有 256MB 物理内存，在 vmware 建立新虚拟机时，分配这个虚拟机 256MB 的内存。

物理地址空间

我们划分的物理地址空间如表 1-4 所示。

表 1-4

序号	内存区域	大 小	描 述
1	0000_8000h - 0000_FFFFh	32KB	setup 模式使用
2	0001_0000h - 0001_FFFFh	64KB	保留未用
3	0002_0000h - 0002_FFFFh	64KB	protected/long 模块使用
4	0003_0000h - 000F_FFFFh	832KB	保留未用
5	0010_0000h - 0011_FFFFh	128KB	PCB pool
6	0012_0000h - 0014_FFFFh	192KB	SDA 区域
7	0020_0000h - 009F_FFFFh	8MB	legacy 模式下的 PT 区域
8	00A0_0000h - 00BF_FFFFh	2MB	EPT PDPT 区域
9	00C0_0000h - 00FF_FFFFh	4MB	保留未用
10	0100_0000h - 0100_FFFFh	64KB	保留未用
11	0101_0000h - 0103_FFFFh	192KB	user stack

续表

序号	内存区域	大 小	描 述
12	0104_0000h - 0105_FFFFh	128KB	kernel stack
13	0200_0000h - 021F_FFFFh	2MB	IA-32e 模式下的 PDPT 区域
14	0220_0000h - 02FF_FFFFh	14MB	IA-32e 模式下的 PT pool
15	0300_1000h - 031F_FFFFh	2MB	user pool
16	0320_0000h - 07FF_FFFFh	78MB	kernel pool
17	0800_0000h - 0FFF_FFFFh	128MB	VM domain pool

PCB pool 基址在物理地址 **100000h** 上共有 128KB，支持 16 个逻辑处理器。每个逻辑处理器拥有自己的 PCB 区域，每个 PCB 大小为 8KB。因此，CPU0 的 PCB 位于 100000h 上，而 CPU1 的 PCB 位于 102000h 上，依此类推。

VM domain pool 用于分配逻辑处理器的 VM domain，共 128MB。每个逻辑处理器对应一个 VM domain，大小为 8MB，因此共支持 16 个逻辑处理器。

在 inc\page.inc 文件里定义了物理地址相关的宏。代码如下。

代码片段 1-5:

```
;;
;; legacy 模式下的物理空间
;;
%define SYSTEM_DATA_PHYSICAL_AREA          100000h
%define USER_STACK_PHYSICAL_BASE           1010000h
%define KERNEL_STACK_PHYSICAL_BASE         1040000h
%define KERNEL_POOL_PHYSICAL_BASE          03200000h
%define USER_POOL_PHYSICAL_BASE            03001000h
... ..

;;
;; 定义 PAE 模式下 PT 和 PDT 的物理地址
;; 1) PT_PHYSICAL_BASE = 20_0000h
;; 2) PDT_PHYSICAL_BASE = PT_PHYSICAL_BASE + (PDT_BASE - PT_BASE) = 20_0000h +
60_0000h = 80_0000h
;; 3) PPT 表在 SDA 区域内

PT_PHYSICAL_BASE      EQU          200000h
PDT_PHYSICAL_BASE     EQU          800000h
PPT_PHYSICAL_BASE     EQU          803000h
... ..

;;
;; long-mode 模式下的物理空间
;;
%define SYSTEM_DATA_PHYSICAL_AREA64        100000h
%define USER_STACK_PHYSICAL_BASE64         1010000h
%define KERNEL_STACK_PHYSICAL_BASE64       1040000h
%define KERNEL_POOL_PHYSICAL_BASE64        03200000h
%define USER_POOL_PHYSICAL_BASE64         03001000h
... ..
```

```

;;
;; PPT 表物理地址:
;; 1) PPT_PHYSICAL_BASE64 = 200_0000h (32M 地址)
;;
;; PXT 表物理地址:
;; 1) PXT_PHYSICAL_BASE64 = PXT_BASE64 - PPT_BASE64 + PPT_PHYSICAL_BASE64 =
21e_d000h
;;

PPT_PHYSICAL_BASE64      EQU      2000000h
PXT_PHYSICAL_BASE64      EQU      21ed000h

;;
;; 页表 pool 地址
;; 说明:
;; 1) 位置在 PPT_PHYSICAL_BASE64 后面 (默认值为 220_0000h)
;; 2) PDT 和 PT 物理地址都从这个 pool 里分配
;; 3) PT_POOL_TOP64 = 300_0000h - 1 = 2ff_ffffh
;; 4) PT_POOL_SIZE 空间为 14M
;; 5) 虚拟地址 ffff_f800_8220_0000h 映射到 PT Pool 物理空间上
;;
PT_POOL_PHYSICAL_BASE64  EQU      (PPT_PHYSICAL_BASE64 + PPT_SIZE)
PT_POOL_PHYSICAL_TOP64  EQU      2FFFFFFh
PT_POOL_SIZE            EQU      (PT_POOL_PHYSICAL_TOP64 -
PT_POOL_PHYSICAL_BASE64 + 1)

;;
;; 页表备用 pool 地址
;; 说明:
;; 1) 当页表池 PT Pool 用完后, 使用备用的 PT pool 区域
;; 2) 备用的 PT Pool 是原 legacy 模式下的 PT 区域 (共 8M 空间: 20_0000h - 9f-ffffh)
;; 3) 虚拟地址 ffff_f800_8020_0000h 映射到备用 PT Pool 物理空间上
;;
PT_POOL2_PHYSICAL_BASE64 EQU      PT_PHYSICAL_BASE
PT_POOL2_PHYSICAL_TOP64 EQU      (PT_PHYSICAL_BASE + 800000h - 1)

```

在 legacy 与 IA-32e 模式下 SDA, user stack、kernel stack、user pool 以及 kernel pool 物理基址都是相同的。而页表相关的物理区域有所不同。

- 在 PAE 分页模式下, PT 的内存区域在 200000H 到 9FFFFFFH 的 8MB 空间内。PDT 物理基址在 800000H 位置上, 而 PDPT (PPT) 物理基址在 803000H 位置上。
- 在 IA-32e 模式下, PDPT 的内存区域在 2000000H 到 21FFFFFFH 之间的 2MB 空间内。PML4T 的内存区域在 21ED000H 到 21EDFFFH 的 4KB 空间内。

在 PAE 分页模式下, 所有的页表结构为 8MB (200000H 至 9FFFFFFH), 而在 IA-32e 模式下, 只定义了 2MB 的 PDPT 内存区域, 其余的 PDT 与 PT 需要从 PT pool 区域里进行动态分配。

PT pool 内存区域在从 2200000H 到 2FFFFFFH 的 14MB 空间里。如果这个空间分配完毕, 则从 8MB 的备用 PT pool (200000H 到 9FFFFFFH) 区域里分配。因此, IA-32e 分页模式下, 可分配的 PT pool 总共为 22MB。

虚拟地址空间

在 legacy 模式下，虚拟地址空间划分如表 1-5 所示。

表 1-5

序号	地址空间	大 小	描 述
1	0000_8000h - 0000_FFFFh	32KB	setup 模式使用
2	0001_0000h - 0001_FFFFh	64KB	保留未用
3	0002_0000h - 0002_FFFFh	64KB	protected/long 模块使用
4	8000_0000h - 8001_FFFFh	128KB	PCB pool
5	8002_0000h - 8004_FFFFh	192KB	SDA 区域
6	7FE0_0000h ~		user stack
7	FFE0_0000h ~		kernel stack
8	8320_0000h ~		kernel pool
9	7300_1000h ~		user pool
10	C000_0000h - C07F_FFFFh	8MB	PT 区域
11	C0A0_0000h - C0BF_FFFFh	2MB	EPT PDPT 区域
12	8800_0000h - 8FFF_FFFFh	128MB	VM domain

setup 与 protected 模块运行空间使用一对一映射，其他虚拟地址映射的物理地址可以对照表 1-4 所示的内存区域。

在 IA-32e 模式 (long-mode) 下，虚拟地址空间划分如表 1-6 所示。

表 1-6

序号	地址空间	大 小	描 述
1	0000_8000h - 0000_FFFFh	32KB	setup 模式使用
2	0001_0000h - 0001_FFFFh	64KB	保留未用
3	0002_0000h - 0002_FFFFh	64KB	protected/long 模块使用
4	FFFFF800_80000000h FFFFF800_8001FFFFh	- 128KB	PCB pool
5	FFFFF800_80020000h FFFFF800_8004_FFFFh	- 192KB	SDA 区域
6	00000000_7FE00000h ~		user stack
7	FFFFFFF8_FFE00000h ~		kernel stack
8	FFFFF800_83200000h ~		kernel pool
9	00000000_73001000h ~		user pool
10	FFFFF6FB_7DA00000h ~		PDPT 区域
11	FFFFF800_82200000h ~		PT pool
12	FFFFF800_80200000h ~		备用 PT pool

续表

序号	地址空间	大 小	描 述
13	FFFFF800_C0A00000h FFFFF800_C0BFFFFFh	- 2MB	EPT PDPT 区域
14	FFFFF800_88000000h FFFFF800_8FFFFFFFh	- 128MB	VM domain

在 legacy 模式下的虚拟地址空间划分基础下，32 位地址值扩展为 64 位。在 inc\page.inc 文件里定义了与虚拟地址相关的宏。代码如下。

代码片段 1-6:

```

;$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
;$    legacy 32-bit 环境下的系统参数    $$
;$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$

%define SYSTEM_DATA_AREA                80000000h
%define USER_STACK_BASE                 7FE00000h
%define KERNEL_STACK_BAS               0FFE00000h
%define KERNEL_POOL_BASE               83200000h
%define USER_POOL_BASE                 73001000h

;;
;; 说明:
;; 1) PT_BASE = c000_0000h
;; 2) PT_TOP  = PT_BASE + (4 * 512 * 512 * 8) - 1
;; 3) PDT_BASE = (PT_BASE >> 12 * 8) + PT_BASE = c060_0000h
;; 4) PDT_TOP  = (PT_TOP  >> 12 * 8) + PT_BASE = c060_3fffh
;; 5) PPT_BASE = (PDT_BASE >> 12 * 8) + PT_BASE = c060_3000h
;; 6) PPT_TOP  = (PDT_TOP  >> 12 * 8) + PT_BASE = c060_301Fh
;;

PT_BASE      EQU      0C0000000h
PT_TOP       EQU      0C07FFFFFFh
PDT_BASE     EQU      0C0600000h
PDT_TOP      EQU      0C0603FFFh
PPT_BASE     EQU      0C0603000h
PPT_TOP      EQU      0C060301Fh

PDT0_BASE    EQU      PDT_BASE
PDT1_BASE    EQU      (PDT_BASE + 1000h)
PDT2_BASE    EQU      (PDT_BASE + 2000h)
PDT3_BASE    EQU      (PDT_BASE + 3000h)

;$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
;$    longmode 64-bit 环境下的系统参数    $$
;$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$

```

```

#define SYSTEM_DATA_AREA64          0FFFFFF80000000000h
#define USER_STACK_BASE64           7FE00000h
#define KERNEL_STACK_BASE64         0FFFFFFF80FFE00000h
#define KERNEL_POOL_BASE64          0FFFFFFF800832000000h
#define USER_POOL_BASE64             73001000h

#define KERNEL_CODE_BASE64           0FFFFFFF8040000000h
#define USER_CODE_BASE64            0040000000h

#ifdef __X64
#define SYSTEM_DATA_SPACE_BASE      0FFFFFFF800800000000h
#else
#define SYSTEM_DATA_SPACE_BASE      800000000h
#endif

#define SYSTEM_DATA_SPACE_BASE32    800000000h

;;
;; PT 表区域
;;
PT_BASE64          EQU          0FFFFFFF680000000000h
PT_TOP64           EQU          0FFFFFFF6FFFFFFFFFh

;;
;; PDT 表区域
;;
PDT_BASE64         EQU          0FFFFFFF6FB400000000h
PDT_TOP64          EQU          0FFFFFFF6FB7FFFFFFFFh

;;
;; PPT 表区域
;;
PPT_BASE64         EQU          0FFFFFFF6FB7DA000000h
PPT_TOP64          EQU          0FFFFFFF6FB7DBFFFFFFFFh

;;
;; PXT 表区域
;;
PXT_BASE64         EQU          0FFFFFFF6FB7DBED0000h
PXT_TOP64          EQU          0FFFFFFF6FB7DBEDFFFFh

;;
;; 页表 pool 地址
;;
PT_POOL_BASE64     EQU          0FFFFFFF800822000000h

;;
;; 页表备用 pool 地址
;;
PT_POOL2_BASE64    EQU          0FFFFFFF800802000000h

```

在 64-bit 模式下，kernel 空间的虚拟地址值统一在原 32 位基础上，高 32 位为 FFFFFF800H，而 stack 地址的高 32 位则为 FFFFFFFF80H。至于 user 空间的虚拟地址与

legacy 模式是一致的。

但是，IA-32e 分页模式的页表地址与 PAE 分页模式则有很大的不同，并不是直接扩展高 32 位而来，这里借鉴了 Windows NT 内核的 IA-32e 分页模式的设计思想。

在 IA-32e 分页模式下，PDPT 区域是从 FFFFF6FB_7DA00000 到 FFFFF6FB_7DBFFFFH 之间的 2MB 空间。PML4T 区域是从 FFFFF6FB_7DBED000H 到 FFFFF6FB_7DBEDFFFH 之间的 4KB 空间。

1.4 数据结构

在系统平台里两个最重要的数据结构是：**PCB**（Processor Control Block，处理器控制块）以及 **SDA**（System Data Area，系统数据区域）。它们定义在 inc\system_manage_region.inc 文件里。

注意：PCB 与 SDA 结构内所有的地址值都是 64 位宽。这样的设计使得结构很容易适应于 32 位与 64 位环境。

1.4.1 PCB 结构

每个逻辑处理器对应一个 PCB 结构，也就是每个逻辑处理器有独立的 PCB 结构。用来维护管理与处理器相关的信息。下面将介绍 PCB 结构内主要的信息。

访问 GS 段

我们使用 PCB 结构基址作为 GS 段的基址。因此，代码总是使用 GS 段来访问 PCB 结构内的数据。在 common\protected.asm 模块里将 32 位的 PCB.Base 值写入 GS.base。代码如下。

代码片段 1-7：

```
mov edi, [gs: PCB.Base]
... ..

xor edx, edx
mov eax, edi
mov ecx, IA32_GS_BASE
wrmsr                      ; 将 PCB.Base 值写入 GS.base
```

在 common\long.asm 模块里将 64 位的 PCB.Base 值写入 GS.base，如下代码所示。

代码片段 1-8：

```
mov esi, [gs: PCB.Base]
mov edi, [gs: PCB.Base + 4]
... ..

mov eax, esi
mov edx, edi
```

```
mov ecx, IA32_GS_BASE
wrmsr                      ; 写入 64 位 PCB.Base 值
```

PCB.Base 存放 64 位的 PCB 基址，只能通过 WRMSR 指令写 IA32_GS_BASE 寄存器来达到写入 64 位的 GS.base 值。

在 **stage1** 阶段下（未分页保护模式），GS.base 存放 PCB 的物理基址。在进入 **stage2**（分页的保护模式）或者 **stage3** 阶段（64-bit 模式）前，GS.base 将写入线性地址。

引用的其他区域基址

在 PCB 结构内定义了若干个 64 位宽的地址值，用来保存所引用的其他区域基址。下面是 PCB 结构的部分定义。

代码片段 1-9:

```
Struc PCB

    .Base             RESQ      1      ; 保存 PCB 基址
    .PhysicalBase     RESQ      1      ; PCB 物理基址
    .Size             RESD      1      ; PCB size
    .SdaBase          RESQ      1      ; 指向系统数据区域
    .SdaPhysicalBase  RESQ      1      ; SDA 物理地址
    .SrtBase          RESQ      1      ; 指向系统服务例程表区域
    .SrtPhysicalBase  RESQ      1      ; SRT 物理地址
    .LsbBase          RESQ      1      ; 指向本地存储块
    .LsbPhysicalBase  RESQ      1      ; LSB 物理地址
    .ReturnStackPointer RESQ      1      ; 指向 ReturnStack
    ... ..
endstruc
```

PCB 结构偏移量为 0 处是 PCB.Base，它指向 PCB 块本身。因此，在代码里任何时候都可以使用下面指令读取 PCB 块基址。

```
%ifdef __x64
    LoadGsBaseToRbp
%else
    mov ebp, [gs: PCB.Base]
%endif
```

PCB.SdaBase 指向全局的 SDA（System Data Area）区域，在源码里，我们常常会像下面指令示例一样读取 SDA 基址。

```
%ifdef __x64
    LoadGsBaseToRbp
%else
    mov ebp, [gs: PCB.Base]
%endif

    REX.Wrxb                      ; 用于 x64
    Mov ebx, [ebp + PCB.SdaBase]  ; 读取 SDA 基址
```

PCB.Srt 指向 SDA 结构末尾 SRT（System Routine Table）区域，用来管理和维护系统服务例程表。PCB.LsbBase 指向每个处理器私有的 LSB（Local Store Block）区域。

注意：PCB.Base，PCB.SdaBase，PCB.SrtBase 以及 PCB.LsbBase 都是虚拟地址。当

运行在 64-bit 模式下时，使用全部的 64 位宽地址值；当运行在 32 位环境下时，只使用 64 位值的低 32 位。它们分别还有自己的物理地址值：PCB.PhysicalBase，PCB.SdaPhysicalBase，PCB.SrtPhysicalBase 及 PCB.LsbPhysicalBase。

描述符表管理记录

PCB 结构内部分管理记录与描述符表相关。代码如下。

代码片段 1-10:

```
struc PCB
    ... ..

    ;;
    ;; GDT 表 selector
    ;;
    ;; GS 用于管理 PCB 区域，FS 用于管理 SDA 区域
    ;;
    .GsSelector          RESW          1
    .TssSelector         RESW          1
    .LdtSelector         RESW          1

    ALIGNB 4

    ;;
    ;; Task Status Segment 管理数据
    ;;
    .TssBase             RESQ          1          ; 记录 TSS 块的基址
    .TssPhysicalBase     RESQ          1          ; 记录 TSS 块物理地址
    .TssLimit            RESD          1          ; Tss limit
    .IomapBase           RESQ          1          ; 记录 IOMAP 块地址
    .IomapPhysicalBase   RESQ          1          ; 记录 IOMAP 块物理地址

    ;;
    ;; Local Descriptor Table 管理数据
    ;;
    .LdtBase             RESQ          1          ; 记录 LDT 表基址
    .LdtLimit            RESD          1          ; Ldt 表 limit
    .LdtTop              RESQ          1          ; 记录 LDT 表的顶上地址
    .CurrentLdtBase      RESQ          1          ; 记录当前的 LDT 表地址

    ... ..
endstruc
```

GsSelector、TssSelector 以及 LdtSelector 分别记录 GS 段，TSS 块以及 LDT 表的 selector 值。下面较为重要的是：TssBase 记录当前 TSS 块的基址，TssPhysicalBase 记录 TSS 块的物理基址。在本书例子里，并没有使用 LDT。

处理器信息

PCB 结构有很大一部分值用来维护 and 记录与处理器相关的信息。代码如下。

代码片段 1-11:

```
struc PCB
    ... ..
```

```

Vendor                RESD                1                ; 处理器的厂商
;;
;; 记录处理器的频率,单位 MHz。即: 每次/ $\mu$ s (每  $\mu$ s 内 tick 次数) !
;; 1) ProcessorFrequency: 处理器显示出来的频率
;; 2) TicksFrequency: 测量出来的处理器频率
;; 注意:
;; 1) ProcessorFrequency 与 TicksFrequency 有些偏差, 由于乘除运算下而产生误差
;; 2) ProcessorFrequency 稍微比 TicksFrequency 大
;; 2.1) 在一台 2.4GHz 处理器上, ProcessorFrequency 的值为 2400
;; 2.2) 那么 TicksFrequency 的值大约为 2394
;;

.ProcessorFrequency RESD                1                ; 处理器 core 频率,
.TicksFrequency      RESD                1                ; 这时测量出来的 tick 频率数

.LapicTimerFrequency RESD                1                ; local timer 的计数频率, 单位
为 us

;;
;; 处理器最大 CPUID leaf 号
;;
.MaxBasicLeaf        RESD                1                ; 最大基本叶号
.MaxExtendedLeaf     RESD                1                ; 最大扩展叶号

;;
;; 处理器基本信息
;;
.CacheLineSize       RESD                1                ; cache line (单位为字节)

;;
;; 处理器型号信息
;;
.DisplayModel        RESB                1                ; 型号
.DisplayFamily       RESB                1                ; 家族号

ALIGNB 4

;;
;; CPUID.01H leaf
;;
.CpuidLeaf01Eax      RESD                1
.CpuidLeaf01Ebx      RESD                1
.CpuidLeaf01Ecx      RESD                1
.CpuidLeaf01Edx      RESD                1

;;
;; CPUID.02H leaf
;;
.CpuidLeaf02Eax      RESD                1
.CpuidLeaf02Ebx      RESD                1
.CpuidLeaf02Ecx      RESD                1
.CpuidLeaf02Edx      RESD                1

;;
;; CPUID.03H leaf
;;
.CpuidLeaf03Eax      RESD                1
.CpuidLeaf03Ebx      RESD                1

```

```

.CpuidLeaf03Ecx      RESD      1
.CpuidLeaf03Edx      RESD      1

;;
;; CPUID.04H leaf
;;
.CpuidLeaf04Eax      RESD      1
.CpuidLeaf04Ebx      RESD      1
.CpuidLeaf04Ecx      RESD      1
.CpuidLeaf04Edx      RESD      1

;;
;; CPUID.05H leaf
;;
.CpuidLeaf05Eax      RESD      1
.CpuidLeaf05Ebx      RESD      1
.CpuidLeaf05Ecx      RESD      1
.CpuidLeaf05Edx      RESD      1

;;
;; CPUID.06H leaf
;;
.CpuidLeaf06Eax      RESD      1
.CpuidLeaf06Ebx      RESD      1
.CpuidLeaf06Ecx      RESD      1
.CpuidLeaf06Edx      RESD      1

;;
;; CPUID.07H leaf
;;
.CpuidLeaf07Eax      RESD      1
.CpuidLeaf07Ebx      RESD      1
.CpuidLeaf07Ecx      RESD      1
.CpuidLeaf07Edx      RESD      1

;;
;; CPUID.08H leaf
;;
.CpuidLeaf08Eax      RESD      1
.CpuidLeaf08Ebx      RESD      1
.CpuidLeaf08Ecx      RESD      1
.CpuidLeaf08Edx      RESD      1

;;
;; CPUID.09H leaf
;;
.CpuidLeaf09Eax      RESD      1
.CpuidLeaf09Ebx      RESD      1
.CpuidLeaf09Ecx      RESD      1
.CpuidLeaf09Edx      RESD      1

;;
;; CPUID.0AH leaf
;;
.CpuidLeaf0AEax      RESD      1
.CpuidLeaf0AEbx      RESD      1
.CpuidLeaf0AECx      RESD      1
.CpuidLeaf0AEdx      RESD      1

;;

```



```

;; CPUID.0BH leaf
;;
.CpuidLeaf0BEax      RESD      1
.CpuidLeaf0BEbx      RESD      1
.CpuidLeaf0BECx      RESD      1
.CpuidLeaf0BEdx      RESD      1

#define PCB.FeatureEcX      PCB.CpuidLeaf01ECx
#define PCB.FeatureEdx      PCB.CpuidLeaf01Edx
#define PCB.FeatureAddition PCB.CpuidLeaf07Ebx

;;
;; 基本扩展信息
;;
.ExtendedFeatureEcX   RESD      1          ; CPUID.80000001H:ECX
.ExtendedFeatureEdx   RESD      1          ; CPUID.80000001H:EDX

.MaxPhysicalAddr      RESD      1          ; 最大的物理地址宽度
.MaxVirtualAddr       RESD      1          ; 最大的虚拟地址宽度

;;
;; SSE 指令支持 level, 分别为:
;; 1) 0000h - 不支持
;; 2) 0100h - SSE
;; 3) 0200h - SSE2
;; 4) 0300h - SSE3,    0301h - SSSE3
;; 5) 0401h - SSE4.1,  0402h - SSE4.2
;;
.SSELevel             RESD      1          ; 指示支持哪个级别的 SSE 指令

;;
;; MAXPHYADDR = 32 时: select mask = 00000000_FFFFFFFFh
;; MAXPHYADDR = 36 时: select mask = 0000000F_FFFFFFFFh
;; MAXPHYADDR = 40 时: select mask = 000000FF_FFFFFFFFh
;; MAXPHYADDR = 52 时: select mask = 000FFFFF_FFFFFFFFh
;;
.MaxPhyAddrSelectMask RESQ      1          ; 最大物理地址的 mask 位

;;
;; 处理器 ID, APIC 以及多线程相关信息
;;
.LapicVersion          RESD      1          ; local APIC 版本
.InitialApicId         RESD      1          ; 处理器初始 ID
.ApicId                RESD      1          ; 处理器 ID
.LogicalId              RESD      1          ; 处理器逻辑 ID
.MaxLogicalProcessor    RESD      1          ; package 内最大逻辑处理器数
.MaxProcessorCore       RESD      1          ; package 内最大 core 数
.ProcessorIndex         RESD      1          ; 处理器编号 #0, #1 ...
.LapicBase              RESQ      1          ; local APIC 虚拟映射基址
.LapicPhysicalBase      RESQ      1          ; local APIC 物理基址
.IapicBase              RESQ      1          ; IO APIC 虚拟映射基址
.IapicPhysicalBase      RESQ      1          ; IO APIC 物理基址

;;

```

```

;; 处理器的 package,core 以及 smt ID
;;
;;
processor
  .ProcessorTopology      RESB      TOPOLOGY_INFO_SIZE
  .LogicalProcessorCount  RESD      1          ; 处理器内总共多少个 local
core
  .ProcessorCoreCount     RESD      1          ; 处理器内总共多少个 processor
core

;;
;;
;; 处理器 cache 信息, 定义 4 个 cache 信息结构
;; 1) L1D(level-1 data):      level-1 数据 cache
;; 2) L1I(level-1 Instruction): level-1 指令 cache
;; 3) L2(level-2 unified):    level-2 统一的 cache
;; 4) L3(level-3 unified):    level-3 统一的 cache
;;
cache
  .CacheLevel            RESD      1          ; cache 层数

  .L1D                   RESB      CACHE_INFO_SIZE ; level-1 data cache 信息
  .L1I                   RESB      CACHE_INFO_SIZE ; level-1 instruction
  .MLC:                  ; MLC(Middle Level Cache)
  .L2                   RESB      CACHE_INFO_SIZE ; level-2 cache 信息
  .LLC:                  ; LLC(Last Level Cache)
  .L3                   RESB      CACHE_INFO_SIZE ; level-3 cache 信息

;;
;; 标志位
;;
  .IsMultiThreading      RESB      1          ; 处理器是否支持多线程
  .IsBsp                 RESB      1          ; 是否为 BSP 处理器
  .IsLapicEnable         RESB      1          ; 记录 Local APIC 是否开启
  .IsLx2ApicEnable       RESB      1          ; 是否为 x2APIC 模块
  .IsIapicEnable         RESB      1          ; 记录 IO APIC 是否开启

;;
;; 下面记录着处理器功能的开启位
;; [0] - PE 位
;; [1] - PG 位
;; [2] - PAE 位
;;
ALIGNB 4
  .ProcessorStatus       RESQ      1

;;
;; 下面记录着处理器指令可用位
;;
  .InstructionStatus     RESQ      1

;;
;; 处理器当前的活动状态
;;
  .ActivityState         RESD      1

;;
;; 维护处理器 debug 功能
;;
  .DebugCapabilities     RESQ      1          ; 记录处理器的 debug 功能

```

```

        .DebugStatus          RESQ          1          ; 记录处理器的 debug 开启项
        ;;
        ;; 维护处理器的 performance 功能
        ;;
        .PerfCapabilities      RESQ          1          ; 记录处理器的 performance 功能
        .PerfStatus           RESQ          1          ; 记录处理器的 performance 开
启项
        ... ..

endstruc

```

每个处理器运行在 stage1 阶段期间，将会调用 `update_processor_basic_info` 函数获得处理器信息并更新 PCB 结构的相关值，这个函数实现在 `lib\system_data_manage.asm` 文件里。

`TicksFrequency` 是测量出来的处理器频率，由 `update_processor_frequency` 函数得到，它的计算方法如下面的 C 伪码所示。

```

#define INTERVAL_IN_TICKS      18          // 8253 每秒约中断 18 次

update_processor_frequency()
{
    init_8253();                          // 初始化 8253 定时器
    enable_8259_timer();                  // 打开 IRQ0
    sti();

    TimerCount = GetTimerCount();          // 得到初始定时器计数值

    /*
     * 等待下一次定时来到
     */
    while (TimerCount == GetTimerCount())
        ;

    BeginTsc = rdtsc();                    // 读取初始 TSC 值

    TimerCount += INTERVAL_IN_TICKS + 1;    // 等待 19 次时间中断

    /*
     * 等待发生 19 次定时
     */
    while (TimerCount > GetTimerCount())
        ;

    EndTsc = rdtsc();                      // 读取截止 TSC 值
    TscCount = EndTsc - BeginTsc;           // 计算间隔 TSC 值

    /*
     * 计算频率: 54945 = (1/18.2) × 1000000, 100 万次中断需要 54 945s
     */
    TicksFrequency = TscCount / (54945 * INTERVAL_IN_TICKS);

    ... ..

    cli();
    disable_8259_timer();                  // 关闭 IRQ0
}

```

ProcessorFrequency 是在 TicksFrequency 基础上调整出来的值，属于处理器报告的频率，略大于 TicksFrequency 值。假如一个处理器的频率为 2.4GHz，那么 ProcessorFrequency 值为 2400，而 TicksFrequency 约为 2394。

update_processor_basic_info 函数接下来执行一系列的 CpuId 指令，用来获取处理器相关的信息，将它们保存在从 CpuIdLeaf01Eax 到 CpuIdLeaf0BEDx 的值里。

update_processor_basic_info 函数通过 CpuId 指令得到的信息来计算得到 DisplayModel, CacheLineSize, MaxPhyAddrSelectMask, SSELevel 等值。

update_processor_basic_info 函数接下来更新及初始化 local APIC 与 I/O APIC 单元信息。代码如下。

代码片段 1-12:

```
;;
;; APIC 基本信息
;;
mov ecx, IA32_APIC_BASE
rdmsr
mov ebx, eax
bt eax, 8 ; 检查是否为 BSP
setc BYTE [gs: PCB.IsBsp]
and eax, 0FFFFFF00h
mov [gs: PCB.LapicPhysicalBase], eax ; local APIC 物理基址
mov DWORD [gs: PCB.LapicBase], LAPIC_BASE ; local APIC 基址 (virtual
address)
mov ecx, [gs: PCB.ProcessorIndex]
mov edx, 01000000h
shl edx, cl ; 生成处理器逻辑 ID
mov [gs: PCB.LogicalId], edx
mov [eax + LAPIC_LDR], edx ; 设置 local APIC 的逻辑 ID

;;
;; 更新 local APIC 信息
;;
mov BYTE [gs: PCB.IsLapicEnable], 1
mov BYTE [gs: PCB.IsLx2ApicEnable], 0
mov ebx, [eax + LAPIC_ID]
mov [gs: PCB.ApicId], ebx ; 更新 Lapic ID
mov ebx, [eax + LAPIC_VERSION]
mov [gs: PCB.LapicVersion], ebx ; 更新 Lapic version

;;
;; 设置处理器的初始 TPR 值
;;
movzx ebx, BYTE [gs: PCB.CurrentTpr]
shl ebx, 4
mov [eax + LAPIC_TPR], ebx

;;
;; 初始化 local LVT 寄存器
;;
mov DWORD [eax + LVT_PERFMON], FIXED_DELIVERY | LAPIC_PERFMON_VECTOR
mov DWORD [eax + LVT_TIMER], TIMER_ONE_SHOT | LAPIC_TIMER_VECTOR | LVT_MASKED
mov DWORD [eax + TIMER_ICR], 0
mov DWORD [eax + LVT_ERROR], LAPIC_ERROR_VECTOR
```

```

mov DWORD [eax + LVT_THERMAL], SMI_DELIVERY
mov DWORD [eax + LVT_CMCI], LVT_MASKED
mov DWORD [eax + LVT_LINT0], EXTINT_DELIVERY
mov DWORD [eax + LVT_LINT1], NMI_DELIVERY

```

update_processor_basic_info.@3:

```

cmp BYTE [gs: PCB.IsBsp], 1
jne update_processor_basic_info.@4

;;
;; 初始化 IOAPIC
;;
call init_ioapic_unit

;;
;; 测量处理器频率
;;
call update_processor_frequency
jmp update_processor_basic_info.@5

```

update_processor_basic_info.@4:

```

;;
;; 屏蔽 APs LINT0 和 LINT1
;;
mov DWORD [eax + LVT_LINT0], LVT_MASKED | EXTINT_DELIVERY
mov DWORD [eax + LVT_LINT1], LVT_MASKED | NMI_DELIVERY

;;
;; 读取 BSP 的 PCB 块
;;
mov ebx, [fs: SDA.PcbPhysicalBase]

;;
;; 复制 BSP 的频率数据
;;
mov eax, [ebx + PCB.ProcessorFrequency]
mov [gs: PCB.ProcessorFrequency], eax
mov eax, [ebx + PCB.TicksFrequency]
mov [gs: PCB.TicksFrequency], eax

;;
;; 复制 BSP 的 lapic timer 计数频率
;;
mov eax, [ebx + PCB.LapicTimerFrequency]
mov [gs: PCB.LapicTimerFrequency], eax

```

update_processor_basic_info.@5:

```

;;
;; 设置 ioapic enable 状态
;;
mov BYTE [gs: PCB.IsIapicEnable], 1
mov DWORD [gs: PCB.IapicPhysicalBase], 0FEC00000h
mov DWORD [gs: PCB.IapicBase], IOAPIC_BASE

```

update_processor_basic_info 函数接下来主要的工作如下所示。

(1) 开启 PAE、XD 以及 SMEP 功能。

(2) 更新处理器标志位 ProcessorStatus。

(3) 检测处理器是否支持 VMX 架构，并调用 get_vmx_global_data 函数来获取 VMX 方面的能力信息。

处理器的 context 信息

这部分包括了通用寄存器 context 信息，以及 x87 FPU 与 XMM 单元的 image 信息。在本书代码里，这几部分信息并没有实际派上用处。

VMX 相关的管理记录

PCB 结构内包括了与 VMX 相关的管理记录信息，它们会经常使用。代码如下。

代码片段 1-13:

```

struct PCB
{
    ... ..

    ;;
    ;; 处理器内存 domain 范围
    ;;
    .Domain                RESQ        1            ; 内存 domain
    .DomainTop             RESQ        1            ; domain 顶部

    ... ..

    ;;
    ;; 记录 VMX 的 VMXON pointer
    ;;
    .VmxonPointer          RESQ        1
    .VmxonPhysicalPointer  RESQ        1

    ;;
    ;; 记录当前 VMCS region pointer
    ;;
    .VmcsPointer           RESQ        1
    .VmcsPhysicalPointer   RESQ        1

    ;;
    ;; 记录 4 个 VMCS 管理指针，指向 VMCS_MANAGE_BLOCK
    ;;
    .VmcsA                 RESQ        1
    .VmcsB                 RESQ        1
    .VmcsC                 RESQ        1
    .VmcsD                 RESQ        1
    .CurrentVmbPointer     RESQ        1

    ;;
    ;; guest 队列管理记录
    ;;
    .GuestRunningQueue     RESQ        1
    .GuestReadyQueue       RESQ        1
    .GuestRunningIndex     RESD        1
    .GuestRunningStatus    RESD        1
    .GuestReadyIndex       RESD        1
    .GuestReadyStatus      RESD        1
}
    
```

```

ALIGNB 8
;;
;; 每个 logical processor 支持 4 个 VMCS_MANAGE_BLOCK 结构
;;
.GuestA          RESB      VMCS_MANAGE_BLOCK_SIZE
.GuestB          RESB      VMCS_MANAGE_BLOCK_SIZE
.GuestC          RESB      VMCS_MANAGE_BLOCK_SIZE
.GuestD          RESB      VMCS_MANAGE_BLOCK_SIZE

ALIGNB 8
;;
;; VMM 管理记录
;;
.VmmStack        RESQ      1
.VmmMsrLoadAddress RESQ      1
.VmmMsrLoadPhyAddress RESQ    1

;;
;; Guest flag 值, 记录是否处于 Guest
;;
.EptEnableFlag   RESB      1

;;
;; ##### 下面是 VMCS buffer #####
;;
.GuestStateBuf    RESB      GUEST_STATE_SIZE
.HostStateBuf     RESB      HOST_STATE_SIZE
.ExecutionControlBuf RESB    EXECUTION_CONTROL_SIZE
.ExitControlBuf   RESB      EXIT_CONTROL_SIZE
.EntryControlBuf  RESB      ENTRY_CONTROL_SIZE
.ExitInfoBuf      RESB      EXIT_INFO_SIZE

;;
;; ##### 下面是 VM-exit 信息 buffer #####
;;
.GuestExitInfo    RESB      100h

ALIGNB 8
.InvDesc          RESB      INV_DESC_SIZE

... ..

;;
;; VMX capability information 记录
;;
.VmxGlobalData:
.VmxBasic         RESQ      1
.PinBasedCtls     RESQ      1
.ProcessorBasedCtls RESQ     1
.ExitCtls         RESQ      1
.EntryCtls        RESQ      1
.VmxMisc:
.Misc             RESQ      1
.Cr0Fixed0        RESQ      1
.Cr0Fixed1        RESQ      1
.Cr4Fixed0        RESQ      1
.Cr4Fixed1        RESQ      1

```

```
.VmcsEnum          RESQ      1
.ProcessorBasedCtls2 RESQ      1
.EptVpidCap         RESQ      1
.VmFunction         RESQ      1
```

```
;;
;; CR0 & CR4 fixed 1 mask (固定为 1 值)
;;
```

```
.Cr0FixedMask      RESQ      1
.Cr4FixedMask      RESQ      1
.VmcsMemoryType    RESD      1
.EptMemoryType     RESD      1
```

```
;;
;; 如果 bit55 of IA32_VMX_BASIC 为 1 , TrueFlag = 1
;;
.TrueFlag          RESB      1
```

```
... ..
```

```
endstruc
```

Domain 与 DomainTop 记录系统为每个处理器分配的 VM domain 区域范围, VmxonPointer 与 VmxonPhysicalPointer 分别记录处理器 VMM 模块当前对应的 VMXON pointer 虚拟地址及物理地址值。

在 lib\Vm\VmInit.asm 文件的 vmx_operation_enter 函数初始化 VMXON 区域, 将执行 VMXON 指令切入 VMX operation 模式。代码如下。

```
;;
;; 进入 VMX root operation 模式
;; 1) operand 是物理地址 pointer
;;
vmxon [ebp + PCB.VmxonPhysicalPointer]
```

VmxonPhysicalPointer 的 VMXON 区域物理指针被作为 VMXON 指针的操作数, 在调用 initialize_vmxon_region 函数初始化 VMXON 区域时, 调用 get_vmcs_access_pointer 函数分配空间。返回的虚拟地址与物理地址分别赋予 VmxonPointer 与 VmxonPhysicalPointer。

VmcsPointer 与 VmxonPhysicalPointer 用来记录当前 VMCS 指针值, 实际代码中并没有使用它们。而是使用 CurrentVmbPointer 来得到当前的 VMB (VM Manage Block) 区域。

VMB (VM manage block) 用来配置与维护 VM (Virtual Machine) 的运行环境。因此, 每个 VM 对应一个 VMB 区域, 也就是每个 guest 对应一个 VMB 结构。每个处理器支持 4 个 VMB 区域, 分别为 GuestA、GuestB、GuestC 以及 GuestD, 它们是 PCB 结构内嵌的 VMB 结构。VmcsA、VmcsB、VmcsC 以及 VmcsD 指针分别指向这几个 VMB 区域。

CurrentVmbPointer 记录当前的 VMB 区域, 这个指针值在每次执行 VMPTRLD 指令时进行切换。代码如下。

```
;;
;; 加载 VMCS pointer
;;
vmptrlld [R5 + PCB.GuestB] ; GuestB 为当前 VMB
```



```

    jc TargetCpuVmentry.Failure
    jz TargetCpuVmentry.Failure

    ;;
    ;; 更新当前 VMB 指针
    ;;
    mov R0, [R5 + PCB.VmcsB]
    mov [R5 + PCB.CurrentVmbPointer], R0      ; CurrentVmbPointer 指向 GuestB

```

PCB 结构内包括了下面几个 VMCS buffer:

- (1) GuestStateBuf, 对应 VMCS 的 guest-state 字段区域。
- (2) HostStateBuf, 对应 VMCS 的 host-state 字段区域。
- (3) ExecutionControlBuf, 对应 VMCS 的 VM-execution 控制字段区域。
- (4) ExitControlBuf, 对应 VMCS 的 VM-exit 控制字段区域。
- (5) EntryControlBuf, 对应 VMCS 的 VM-entry 控制字段区域。
- (6) ExitInfoBuf, 对应 VMCS 的 VM-exit 信息字段区域。

在初始化 VMCS 区域时, 并不直接写入 VMCS 的字段值, 而是写入各自对应的 VMCS buffer, 然后统一将 VMCS buffer 值刷新到 VMCS 区域。

initialize_vmcs_buffer 函数依次调用 init_vm_execution_control_fields, init_vm_exit_control_fields, init_vm_entry_control_fields, init_host_state_area 以及 init_guest_state_area 函数进行初始化对应的 VMCS buffer (参见第 3.11.3.3 节)。

在 VMM 使用 VMPTRLD 指令加载当前 VMCS 指针后, 最后由 setup_vmcs_region 函数分别调用 flush_execution_control, flush_exit_control, flush_entry_control, flush_host_state 以及 flush_guest_state 函数刷新到 VMCS。

PCB 内含一个 256 字节的 GuestExitInfo 区域。当 guest 产生 VM-exit 后, VMM 收集处理这个 VM-exit 所必要的相关信息, 保存在 GuestExitInfo 区域内。例如, guest 由于尝试任务切换而产生 VM-exit, VMM 将调用 GetTaskSwitchInfo 函数收集处理任务切换的相关信息。

在 update_processor_basic_info 函数里最后调用 get_vmx_global_data 函数来获取处理器关于 VMX 能力方面的信息, 将它们保存在 PCB 内。包括:

- (1) VmxBasic, 记录 VMX 的基本能力。
- (2) PinBasedCtls, ProcessorBasedCtls 以及 ProcessorBasedCtls2, 记录 VMX 的 VM-execution 控制能力。
- (3) ExitCtls, 记录 VMX 的 VM-exit 控制能力。
- (4) EntryCtls, 记录 VMX 的 VM-entry 控制能力。
- (5) VmxMisc, 记录 VMX 的杂项能力。

- (6) Cr0Fixed0 与 Cr0Fixed1, 记录 VMX 的 CR0 固定位设置信息。
- (7) Cr4Fixed0 与 Cr4Fixed1, 记录 VMX 的 CR4 固定位设置信息。
- (8) VmcsEnum, 记录 VMX 的 VMCS 字段可索引数。
- (9) EptVpidCap, 记录 VMX 在 EPT 以及 cache 方面的支持。
- (10) VmFunction, 记录 guest 内可执行 VMFUNC 指令方面的能力。

1.4.2 LSB 结构

PCB.LsbBase 指向一个被称为 LSB (Local Store Block, 本地存储块) 的结构, LSB 结构用来存储处理器对应的额外信息, 每个处理器拥有自己的 LSB 结构。它的定义如下所示。

代码片段 1-14:

```

struct LSB
{
    .Base                RESQ          1
    .PhysicalBase        RESQ          1

    ;;
    ;; local video buffer 管理记录
    ;;
    .LocalVideoBufferHead    RESQ          1
    .LocalVideoBufferPtr     RESQ          1
    .LocalVideoBufferLastChar RESD          1
    .LocalVideoBufferSize    RESD          1

    ;;
    ;; local keyboard buffer 管理记录
    ;;
    ALIGNB 8
    .LocalKeyBufferHead      RESQ          1
    .LocalKeyBufferPtr       RESQ          1
    .LocalKeyBufferSize      RESD          1

    ;;
    ;; local timer 管理记录
    ;;
    .LapicTimerRequestMask   RESD          1
    .LapicTimerRoutine       RESQ          1
    .LapicTimerCount         RESD          1

    ;;
    ;; 时间计数值
    ;;
    .Hour                    RESD          1
    .Minute                  RESD          1
    .Second                  RESD          1

    ;;
    ;; 本地 keyboard buffer 区域, 共 256 字节, 保存按键扫描码
    ;;
}
    
```

```

        .LocalKeyBuffer          RESB          256

        .Reserved                RESB          4096 - $

        ;;
        ;; 本地 video buffer 区域, 存放处理器的屏幕信息
        ;; 4K, 存放约 25 × 80 × 2 信息
        ;;
        .LocalVideoBuffer        RESB          (4096 * 2 - $)

        LOCAL_STORAGE_BLOCK_SIZE EQU          $
        LSB_SIZE                  EQU          $
endstruc

```

LSB 结构大小为 8KB，主要包括了两个存储区域：LocalKeyBuffer 与 LocalVideoBuffer。它们分别作为处理器对应的键盘输入缓冲区与 video 缓冲区（后 4KB 区域）。键盘输入缓冲区大小为 256B，video 缓冲区大小为 4KB，足够容纳下整个屏幕（25×80×2=4000）所需要数据。

每当切换处理器时，当前的键盘缓冲区与 video 缓冲区也会随着切换。每个处理器可以对应一个显示屏幕。例如，当焦点从 CPU0 切换到 CPU1 时，当前的屏幕内容会刷新为 CPU1 对应的屏幕内容。而从 CPU1 切换回 CPU0 时，当前的屏幕内容同样也会刷新为 CPU0 的屏幕内容。

键盘缓冲区的作用与 video 缓冲区是一致的，保证每个处理器拥有自己的输入数据，不被其他处理器干扰。

1.4.3 初始化 PCB

每个处理器在执行 common\setup.asm 模块时处于 stage1 阶段（也就是未分页的保护模式），它将调用 update_stage1_gs_segment 函数初始化 stage1 阶段的 PCB 结构。

update_stage1_gs_segment 函数实现在 lib\stage1.asm 文件里。代码如下。

代码片段 1-15:

```

;-----
; update_stage1_gs_segment()
; input:
;     none
; output:
;     none
; 描述:
;     1) 更新 stage1 的 GS 段, 对应处理器的 PCB 区域
;-----
update_stage1_gs_segment:
    push edx
    push ecx
    push ebx

    ;;
    ;; 分配 PCB 地址
    ;;

```

```

call alloc_pcb_base          ; edx:eax = VA:PA
mov ebx, eax                 ; ebx = PCB 物理地址
mov esi, edx                 ; esi = PCB 虚拟地址
xor edx, edx

;;
;; 将物理地址写入 GS base
;;
mov ecx, IA32_GS_BASE
wrmsr

;;
;; 更新 PCB 基本记录
;; 1) 地址的高 32 位使用在 64-bit 模式下
;;
mov edx, 0FFFFFF800h
mov [gs: PCB.PhysicalBase], ebx
mov [gs: PCB.Base], esi
mov [gs: PCB.Base + 4], edx
mov DWORD [gs: PCB.Size], PCB_SIZE
mov eax, [fs: SDA.Base]
mov [gs: PCB.SdaBase], eax          ; 指向 SDA 区域
mov [gs: PCB.SdaBase + 4], edx
add eax, SRT.Base                  ; SRT 区域基址 (位于 SDA 之后)
mov [gs: PCB.SrtBase], eax          ; 指向 System Service Routine

Table 区域
mov [gs: PCB.SrtBase + 4], edx
mov eax, [fs: SDA.PhysicalBase]
mov [gs: PCB.SdaPhysicalBase], eax
add eax, SRT.Base
mov [gs: PCB.SrtPhysicalBase], eax

;;
;; 更新 ReturnStackPointer
;;
lea eax, [esi + PCB.ReturnStack]
mov [gs: PCB.ReturnStackPointer], eax
mov [gs: PCB.ReturnStackPointer + 4], edx

;;
;; 默认的 TPR 级别为 3
;;
mov BYTE [gs: PCB.CurrentTp], INT_LEVEL_THRESHOLD
mov BYTE [gs: PCB.PrevTp], 0

;;
;; 设置 LDT 管理信息
;; 注意:
;; 1) LDT 此时为空, 这里使用虚拟地址
;; 2) 地址高 32 位使用在 64-bit 模式下
;;
mov DWORD [gs: PCB.LdtBase], SDA_BASE + SDA.Ldt
mov DWORD [gs: PCB.LdtBase + 4], 0FFFFFF800h
mov DWORD [gs: PCB.LdtTop], SDA_BASE + SDA.Ldt
mov DWORD [gs: PCB.LdtTop + 4], 0FFFFFF800h

;;

```

```

;; 更新 context 区域指针
;; 1) 在 stage1 使用物理地址
;;
lea eax, [ebx + PCB.Context]
mov [gs: PCB.ContextBase], eax
lea eax, [ebx + PCB.XMMStateImage]
mov [gs: PCB.XMMStateImageBase], eax

;;
;; 分配本地存储块
;;
mov esi, LSB_SIZE + 0FFFh
shr esi, 12
call alloc_stage1_kernel_pool_base           ; edx:eax = PA:VA
mov [gs: PCB.LsbBase], eax
mov DWORD [gs: PCB.LsbBase + 4], 0FFFFFF800h
mov [gs: PCB.LsbPhysicalBase], edx
mov DWORD [gs: PCB.LsbPhysicalBase + 4], 0
mov ecx, eax                                ; ecx = LSB

;;
;; 清空 LSB 块
;;
mov esi, LSB_SIZE
mov edi, edx
call zero_memory

;;
;; 更新 LSB 基本信息
;;
mov [edx + LSB.Base], ecx
mov DWORD [edx + LSB.Base + 4], 0FFFFFF800h           ; LSB.Base
mov [edx + LSB.PhysicalBase], edx
mov DWORD [edx + LSB.PhysicalBase + 4], 0             ; LSB.PhysicalBase

;;
;; local video buffer 记录
;;
lea esi, [ecx + LSB.LocalVideoBuffer]
mov [edx + LSB.LocalVideoBufferHead], esi
mov DWORD [edx + LSB.LocalVideoBufferHead + 4], 0FFFFFF800h ;
LSB.LocalVideoBufferHead
mov [edx + LSB.LocalVideoBufferPtr], esi
mov DWORD [edx + LSB.LocalVideoBufferPtr + 4], 0FFFFFF800h ;
LSB.LocalVideoBufferPtr

;;
;; local keyboard buffer 记录
;;
lea esi, [ecx + LSB.LocalKeyBuffer]
mov [edx + LSB.LocalKeyBufferHead], esi
mov DWORD [edx + LSB.LocalKeyBufferHead + 4], 0FFFFFF800h ;
LSB.LocalKeyBufferHead
mov [edx + LSB.LocalKeyBufferPtr], esi
mov DWORD [edx + LSB.LocalKeyBufferPtr + 4], 0FFFFFF800h ;
LSB.LocalKeyBufferPtr
mov DWORD [edx + LSB.LocalKeyBufferSize], 256 ;
LSB.LocalKeyBufferPtr = 256

```

```

;;
;; 更新 VMCS 管理指针 (虚拟指针)
;; 1) VmcsA 指向 GuestA
;; 2) VmcsB 指向 GuestB
;; 3) VmcsC 指向 GuestC
;; 4) VmcsD 指向 GuestD
;;
mov edx, 0FFFFFF800h
mov ecx, [gs: PCB.Base]
lea eax, [ecx + PCB.GuestA]
mov [gs: PCB.VmcsA], eax
mov [gs: PCB.VmcsA + 4], edx
lea eax, [ecx + PCB.GuestB]
mov [gs: PCB.VmcsB], eax
mov [gs: PCB.VmcsB + 4], edx
lea eax, [ecx + PCB.GuestC]
mov [gs: PCB.VmcsC], eax
mov [gs: PCB.VmcsC + 4], edx
lea eax, [ecx + PCB.GuestD]
mov [gs: PCB.VmcsD], eax
mov [gs: PCB.VmcsD + 4], edx

;;
;; 保存 GS selector
;;
mov WORD [gs: PCB.GsSelector], GsSelector

;;
;; 更新处理器状态信息
;;
mov eax, CPU_STATUS_PE
or DWORD [gs: PCB.ProcessorStatus], eax

pop ebx
pop ecx
pop edx
ret

```

update_stage1_gs_segment 首先调用 alloc_pcb_base 函数在 PCB pool (参见第 1.3 节) 里分配 PCB 区域基址, alloc_pcb_base 函数同时返回虚拟地址和物理地址。

注意: 接下来首先将物理地址写入 IA32_GS_BASE 寄存器。此时, GS.base 值就等于 PCB 物理地址, 然后通过 GS 段分别将 PCB 虚拟地址与物理地址写入 PCB.base 与 PCB.PhysicalBase 值里。PCB.base 是 64 位值, 高 32 位写入 FFFFFFF800H 值后, 无论后续运行在 stage2 还是 stage3 阶段, 都可以正确地引用 PCB 块。

update_stage1_gs_segment 函数将 PCB.SdaBase 初始化为 “[fs: SDA.Base]” 值。同样, 高 32 位写入 FFFFFFF800H 值。接下来设置 PCB.SrtBase 与 PCB.PhysicalBase 值, 初始化默认的 TPR 级别, LDT 管理记录, context 管理记录。

update_stage1_gs_segment 函数调用 alloc_stage1_kernel_pool_base 函数在 kernel pool 里分配 LSB 区域, 返回的虚拟地址与物理地址分别保存在 PCB.LsbBase 与 PCB.LsbPhysicalBase 值里, 然后初始化 LSB 结构的各项管理记录。

接下来初始化 VmcsA 到 VmcsD 指针值，分别指向 GuestA 到 GuestD。最后将当前的 GsSelector 保存在 PCB.Selector 里，更新处理器状态信息为 CPU_STATUS_PE，指示已经进入了保护模式。

1.4.4 SDA 结构

系统的全局数据信息记录在 SDA（System Data Area，系统数据区域）结构里，SDA 结构只有唯一的一份，由所有逻辑处理器共享。

访问 FS 段

我们使用 SDA 区域的基址作为 FS 段基址。因此，代码通过 FS 段来访问 SDA 区域数据。代码如下。

```
%ifdef __X64
    LoadFsBaseToRbp                ; mov rbp, [fs: SDA.Base]
%else
    mov ebp, [fs: SDA.Base]
%endif

    mov eax, [ebp + SDA.ProcessorCount]
```

或者，通过当前 GS 段来间接读取 SDA 区域。代码如下。

```
    REX.Wrb
    mov ebx, [ebp + PCB.SdaBase]    ; 通过 PCB.SdaBase 读取 SDA 基址

    mov eax, [ebx + SDA.ProcessorCount]
```

在 stage1 阶段，SDA 区域基址被设置为 SDA_PHYSICAL_BASE 值（120000H，参见表 1-4）。在 setup.asm 模块开头调用 protected_mode_enter 函数切换到未分页保护模式时设置。

protected_mode_enter 函数实现在 lib\crt16.asm 文件里，它执行在 16 位的实模式下。这个函数返回后，处理器将运行在未分页的保护模式环境里。因此，此时的 FS.base 基址是物理地址。

访问 PCB 区域

SDA 结构定义在 inc\system_manage_region.inc 文件里，下面是部分成员的定义。

代码片段 1-16:

```
struc SDA
    ;;
    ;; 系统数据区域，提供所有处理器共同使用的数据
    ;;
    .Base                RESQ    1        ; 此区域的地址 (virtual address)
    .PhysicalBase        RESQ    1        ; 此区域的物理地址
    .Size                RESD    1        ; 此区域的大小
    .PcbBase              RESQ    1        ; 指向 CPU0 的 PCB 块
    .PcbPhysicalBase     RESQ    1        ; 指向 CPU0 的 PCB 块
    ... ..
```

endstruc

SDA.Base 与 SDA.PhysicalBase 分别记录虚拟地址以及物理地址值，它们为 64 位宽，在 legacy 模式下使用低 32 位值。

SDA.PcbBase 与 SDA.PcbPhysicalBase 分别记录 CPU0 的 PCB 块虚拟地址及物理地址值，也就是 PCB pool 区域基址（参见表 1-4，表 1-5 及表 1-6）。通过 PcbBase 值可以获得所有处理器的 PCB 结构，lib\smp.asm 文件里的 get_processor_pcb 用来获取目标处理器的 PCB 基址。

代码片段 1-17:

```

;-----
; get_processor_pcb()
; input:
;     esi - 处理器 Index 值
; output:
;     eax - 该处理器的 PCB 基址
; 描述:
;     1) 根据提供的处理器 Index 值（从 0 开始），得到该处理器对应的 PCB 块
;     2) 出错时返回 0 值
;-----
get_processor_pcb:
    push ebp
    push edx

#ifdef __X64
    LoadFsBaseToRbp
#else
    mov ebp, [fs: SDA.Base]
#endif

    ;;
    ;; 检查提供的处理器 Index 值是否超限
    ;;
    mov eax, [ebp + SDA.ProcessorCount]
    cmp esi, eax
    setb al
    movzx eax, al
    jae get_processor_pcb.done
    ;;
    ;; 目标处理器 PCB base = PCB_BASE + (Index × PCB_SIZE)
    ;;
    mov eax, PCB_SIZE
    mul esi
    REX.Wrb
    add eax, [ebp + SDA.PcbBase]

get_processor_pcb.done:
    pop edx
    pop ebp
    ret

```

get_processor_pcb 函数需要提供目标处理器的 index 值，然后根据 index 值计算出目标处理器的 PCB 基址。Index 值从 0 开始计数，假如 index 大于等于 ProcessorCount 则无效，返回 0 值。

SMP 环境相关

在系统的 SMP 环境初始化期间，需要使用若干个锁来同步处理器的执行，它们放在 SDA 结构内。

代码片段 1-18:

```

struct SDA
    ... ..

    .ProcessorCount      RESD      1      ; 处理器计数
    .ApLockPointer       RESD      1      ; AP 处理器执行锁指针
    .ApLongmode          RESD      1      ; AP 处理器是否进入 long mode
    .ApStartupRoutineEntry RESD      1      ; AP 处理器的 Startup routine
入口地址
    .ApInitDoneCount     RESD      1      ; AP 处理器初始化完成计数

    ;;
    ;; AP 执行锁
    ;;
    .Stage1LockPointer   RESD      1      ; stage1 锁
    .Stage2LockPointer   RESD      1      ; stage2 锁
    .Stage3LockPointer   RESD      1      ; stage3 锁

    ... ..
endstruct

```

ProcessorCount 用于记录检测到的实际处理器计数。而 PCB 结构内的 LogicalProcessorCount 在调用 update_processor_topology_info 函数获取处理器拓扑信息时更新，它在迭代 Cpuuid 的 0B leaf 时计算出处理器所有逻辑处理器的数量。

代码片段 1-19:

```

update_processor_basic_info:
    push ebx
    push ecx
    push edx

    ;;
    ;; 设置处理器 count 与 index 值
    ;; 1) index 为 count 原值
    ;; 2) 增加 count 计数
    ;;
    mov eax, 1
    lock xadd [fs: SDA.ProcessorCount], eax      ; processor count
    mov [gs: PCB.ProcessorIndex], eax           ; index = count(初始值为 0)

    ... ..
    ret

```

如上代码所示，每个处理器在调用 update_processor_basic_info 函数更新 PCB 基本信息时，会递增 ProcessorCount 计数值。同时，也会根据 ProcessorCount 原值来更新处理器的 ProcessorIndex 值。

ApLockPointer 实际并没使用，而 Stage1LockPointer，Stage2LockPointer 及 Stage3LockPointer 是在系统初始化期间所使用的锁指针，分别指向 stage1、stage2 及

stage3 阶段的锁地址，以便在系统全局位置可以访问。

ApLongmode 是标志位，为 1 时指示系统需要进入 stage3 阶段（即 long-mode 模式）运行。这个值根据__X64 符号来设置。代码如下。

代码片段 1-20:

```
;;
;; 如果需要进入 longmode 则定义 __X64 符号
;; 1) SDA.ApLongmode = 1 时，所有处理器进入 longmode 模式
;; 2) SDA.ApLongmode = 0 时，使用 legacy 环境
;;
#ifdef __X64
    mov DWORD [fs: SDA.ApLongmode], 1
#else
    mov DWORD [fs: SDA.ApLongmode], 0
#endif
```

在后续的代码里，会根据 ApLongmode 选择入口点。另外，在 common\boot.asm 模块里，会根据__X64 符号来决定加载 protected 模块还是 long 模块。

在 stage1 阶段，BSP（Bootstrap Processor）从 ApStartupRoutineEntry 里取出启动例程（startup routine）的入口地址，然后广播 INIT-SIPI-SIPI 消息序列，唤醒所有 APs（Application Processors）执行启动例程进入初始化流程。

ApInitDoneCount 统计在某个阶段已经完成初始化的处理器计数。例如，在 stage1 阶段，BSP 需要利用 ApInitDoneCount 来等待 APs 完成初始化后，再进入下一阶段（stage2）。

IPI 相关

当我们需要切换处理器焦点时（例如，按<F2>键切换到 CPU1），BSP 会以 NMI delivery 方式发送 IPI（Inter-processor interrupt，处理器间中断）给目标处理器。下面的 SDA 结构成员与这个 NMI IPI 相关。

代码片段 1-21:

```
struct SDA
{
    ...

    ;;
    ;; 信号
    ;;
    .Signal                RESD    1           ; 内部使用的信号
    .SignalPointer         RESQ    1           ; 引用外部提供的信号

    ;;
    ;; NMI handler 使用的功能号，默认为 0 值，由硬件触发
    ;;
    .NmiIpiRequestMask     RESD    1
    .NmiIpiRoutine         RESQ    1

    ;;
    ;; 记录拥有焦点的 processor index 值
    ;;
    .InFocus               RESD    1
}
```

```

;;
;; 记录当前 video buffer 位置
;;
.VideoBufferHead      RESQ      1
.VideoBufferPtr        RESQ      1
.VideoBufferLastChar   RESD      1

;;
;; keyboard buffer 管理记录
;;
.KeyBufferHead         RESQ      1          ; 保存 LSB.LocalKeyBufferHead 值
.KeyBufferPtrPointer   RESQ      1          ; 指向 local KeyBufferPtr
.KeyBufferLength       RESD      1          ; 键盘缓冲区长度

... ..
endstruc

```

Signal 用于内部的等待同步，在 lib\smpt.asm 文件内的 dispatch_to_processor_with_waiting 例程将使用它等待目标处理器，直到完成目标任务后才返回。我们也可以向 SignalPointer 提供一个外部的信号量，它使用在 get_for_signal 函数里。

NmiIpiRoutine 指向目标的 NMI 例程，当以 NMI delivery 方式发送 IPI 时，目标处理器接到 NMI 后从 NmiIpiRoutine 取出目标例程的入口地址，执行目标任务。

典型的，当按<F2>键时，在 keyboard_8259_handler 里提供 switch_to_processor 例程作为入口地址，调用 force_dispatch_to_processor 例程向目标处理器发送 NMI IPI 消息。这个 switch_to_processor 入口地址将被写入 NmiIpiRoutine 值里。

NmiIpiRequestMask 是标志位，每一位对应一个处理器的 Index 值。例如，CPU0 对应 bit 0。当处理器接收 NMI 时，它需要判断 NMI 属于外部中断产生，还是通过 NMI IPI 产生。位为 1 时，表明通过 NMI IPI 方式产生，使用 IPI 方式处理。位为 0 时，使用原有方式处理。

InFocus 用来记录拥有焦点的处理器 index 值，它随着处理器的切换而切换。例如，当切换到 CPU2 时，InFocus 的值为 2 (ProcessorIndex = 2)。切换回 CPU0 时，InFocus 的值为 0。

VideoBufferHead 与 VideoBufferPtr 分别指向当前的 LSB 区域内的 VideoBuffer 头部及当前位置，KeyBufferHead 与 KeyBufferPtrPointer 分别指向当前 LSB 区域内的 KeyBuffer 头部及当前指针。它们随着处理器的切换而切换。

在初始化时，SDA.KeyBufferHead，SDA.KeyBufferPtrPointer 以及 SDA.KeyBufferLength 被设置为 BSP (CPU0) 的 LSB.LocalKeyBufferHead，LSB.LocalKeybufferPtr，以及 LSB.LocalKeyBufferSize 值。

VM domain pool

每个处理器都拥有自己的 VM domain 区域，在 PCB.Domain 里记录。在初始化时，VM domain pool (参见第 1.3 节) 的物理地址被设置为 DOMAIN_PHYSICAL_BASE 值

(08000000H)，虚拟地址被设置为 DOMAIN_BASE 值 (88000000H，64-bit 模式下为 FFFF800_88000000H)。

代码片段 1-22:

```

struct SDA
    ... ..

    ;;
    ;; 定义虚拟机 domain 管理记录
    ;;
    .DomainPhysicalBase    RESQ        1                ; domain pool 物理基址
    .DomainBase            RESQ        1                ; domain pool 虚拟基址

    ... ..
endstruct

```

在 initialize_vmcs_buffer 初始化 VMCS buffer 时，VMM 调用 vm_alloc_domain 函数在 VM domain pool 里分配处理器对应的 VM domian。这个分配顺序是按处理器初始化 VMCS buffer 的次序。

调试记录相关

如果我们在执行 Build 命令时，加入 DEBUG_RECORD_ENABLE 符号的定义，将会在代码里加入 debug record（调试记录）模块。

代码片段 1-23:

```

struct SDA
    ... ..

    ;;
    ;; 定义 DEBUG_RECORD_STRUCTURE 管理记录
    ;;
    %ifdef DEBUG_RECORD_ENABLE
        .DrsBase            RESQ        1
        .DrsTop             RESQ        1
        .DrsIndex           RESQ        1

        .DrsHeadPtr         RESQ        1
        .DrsTailPtr         RESQ        1

        .DrsMaxCount        RESD        1
        .DrsCount           RESD        1
    %endif

    ... ..

    %ifdef DEBUG_RECORD_ENABLE
        ;;
        ;; 下面是 DEBUG_RECORD_STRUCTURE 区域（默认为 32K）
        ;;
        .DrsBuffer:         RESB        (4096 * (9+16) + DRS_AREA_SIZE) - $
    %endif

    ... ..
endstruct

```

SDA 内关于调试记录的管理记录定义取决于 `DEBUG_RECORD_ENABLE` 符号。
`DrsBase` 存放 DRS (`DEBUG_RECORD_STRUCTURE`) 区域基址, `DrsTop` 指向 DRS 区域顶部, `DrsIndex` 指向下一条调试记录写入的位置。 `DrsHeadPtr` 存放 DRS 区域头部指针, `DrsTailPtr` 存放尾部指针, `DrsCount` 则记录调试记录的数量。

DRS 区域位于 SDA 结构偏移量 19000H (4096×25) 的位置。 `DRS_AREA_SIZE` 的值被定义为 8000H, 也就是 DRS 区域大小为 32KB, 可以容纳约 151 条记录。

外部中断转发相关

VMM 在转发外部中断时需使用 `EXTINT_RTE` (外部中断 routine table entry) 结构。
SDA 内定义了 `EXTINT_RTE` 区域管理记录。

代码片段 1-24:

```

struc SDA
    ... ..

    ;;
    ;; EXTINT_RTE 管理记录
    ;;
    .ExtIntRtePtr          RESQ          1
    .ExtIntRteIndex        RESQ          1
    .ExtIntRteCount        RESD          1

    ... ..
endstruc
```

`ExtIntRtePtr` 指向 `EXTINT_RTE` 列表区域基址, `ExtIntRteIndex` 指向下一条 `EXTINT_RTE` 记录写入的位置, `ExtIntRteCount` 记录 `EXTINT_RTE` 记录的数量。

描述符表管理记录

系统的 GDT、IDT 以及 TSS 都存放在 SDA 区域内, 供所有处理器访问。代码如下。

代码片段 1-25:

```

struc SDA
    ... ..

    ;;
    ;; GDT selector
    ;;
    .FsSelector            RESW          1
    .KernelCsSelector      RESW          1
    .KernelSsSelector      RESW          1
    .UserCsSelector        RESW          1
    .UserSsSelector        RESW          1
    .User64CsSelector      RESW          1
    .User64SsSelector      RESW          1
    .SysenterCsSelector    RESW          1
    .SyscallCsSelector     RESW          1
    .SysretCsSelector      RESW          1
endstruc
```

```

ALIGNB 4

;;
;; Global Descriptor Table 管理记录
;;
.GdtPointer:
.GdtLimit          RESW          1          ; GDT 表 limit 值
.GdtBase           RESQ          1          ; GDT 表 base 值

ALIGNB 4
.GdtTop            RESQ          1          ; 指向 GDT 表顶上描述符的地址

;;
;; Interrupt Descriptor Table 管理记录
;;
.IdtPointer:
.IdtLimit          RESW          1          ; IDT 表 limit 值
.IdtBase           RESQ          1          ; IDT 表 base 值

ALIGNB 4
.IdtTop            RESQ          1          ; IDT 表顶部

RESB              (4096 * 1) - $

;;
;; 定义 Global Descriptor Table 数据区域 (共 4K)
;;
.Gdt:
.NullDesc          RESQ          1
RESB              (4096 * 2) - $

;;
;; 下面是 IDT 所在区域 (共 4K)
;;
.Idt:              RESB          (4096 * 3) - $

;;
;; 下面是 TSS 区域 (共 4K)
;;
.Tss:              RESB          (4096 * 4) - $

;;
;; 下面是 IOMAP 区域 (共 8K)
;;
.Iomap:            RESB          (4096 * 6) - $

;;
;; 下面是 LDT 区域
.Ldt:              RESB          (4096 * 7) - $

... ..
endstruc

```

SDA 结构内定义了若干个 GDT selector 成员，用来保存相关的 selector 值（例如，CS 与 SS 选择子），系统的最终 GDT 与 IDT 指针值保存在 SDA 区域的 GdtPointer 与 IdtPointer 值内。在更新 GDTR 与 IDTR 寄存器时，通过 FS 段来加载 SDA 区域的 GDT 与 IDT 指针值。代码如下。

```
lgdt [fs: SDA.GdtPointer]      ; 更新 GDTR 寄存器
lidt [fs: SDA.IdtPointer]     ; 更新 IDTR 寄存器
```

GdtPointer 与 IdtPointer 定义为 80 位宽，低 16 位为描述符表 limit 值，高 64 位为描述符表 base 值。在 32 位环境下，base 使用低 32 位。

接下来分别定义 GDT，IDT，TSS，Iomap 以及 LDT 区域：

- GDT 区域在 SDA 区域偏移量 1000H 位置上，共 4KB 大小。
- IDT 区域在 SDA 区域偏移量 2000H 位置上，共 4KB 大小。
- TSS 区域在 SDA 区域偏移量 3000H 位置上，共 4KB 大小。
- Iomap 区域在 SDA 区域偏移量 4000H 位置上，共 8KB 大小。
- LDT 区域在 SDA 区域偏移量 6000H 位置上，共 4KB 大小。

当 BSP 运行在 stage1 阶段的 setup 模块初期，将会调用 init_system_data_area 函数对 GDT 与 IDT 的内容进行设置，调用 build_stage1_tssb 函数对 TSS 进行调用，LDT 保留未用。

GDT 与 IDT 的相关初始化如以下代码所示。

代码片段 1-26:

```
init_system_data_area:
    ... ..

    ;;
    ;; 设置基本 GDT 表项
    ;; 1) entry 0:      NULL descriptor
    ;; 2) entry 1,2:    64-bit kernel code/data 描述符
    ;; 3) entry 3,4:    32-bit user code/data 描述符
    ;; 4) entry 5,6:    64-bit user code/data 描述符
    ;; 5) entry 7,8:    32-bit kernel code/data 描述符
    ;; 6) entry 9,10:   fs/gs 段使用
    ;; 7) entry 11,12:  TSS 描述符动态增加
    ;;
    mov [fs: SDA.Gdt], ecx
    mov [fs: SDA.Gdt + 4], ecx

    ;;
    ;; 64-bit kernel CS/SS 描述符设置说明:
    ;; 1) 在 x64 体系下描述符可以设置为
    ;;      * CS = 00209800_00000000h (L=P=1, G=D=0, C=R=A=0)
    ;;      * SS = 00009200_00000000h (L=1, G=B=0, W=1, E=A=0)
    ;; 2) 在 VMX 架构下，在 VM-exit 返回 host 后会将描述符设置为
    ;;      * CS = 00AF9B00_0000FFFFh (G=L=P=1, D=0, C=0, R=A=1, limit=4G)
    ;;      * SS = 00CF9300_0000FFFFh (G=P=1, B=1, E=0, W=A=1, limit=4G)
    ;;
    ;; 3) 因此，为了与 host 的描述符达成一致，这里将描述符设为
    ;;      * CS = 00AF9A00_0000FFFFh (G=L=P=1, D=0, C=A=0, R=1, limit=4G)
```

```

;;      * SS = 00CF9200_0000FFFFh (G=P=1, B=1, E=A=0, W=1, limit=4G)
;;
%if 0
    mov [fs: SDA.Gdt + KernelCsSelector64], ecx
    mov DWORD [fs: SDA.Gdt + KernelCsSelector64 + 4], 00209800h
    mov [fs: SDA.Gdt + KernelSsSelector64], ecx
    mov DWORD [fs: SDA.Gdt + KernelSsSelector64 + 4], 00009200h
%endif

mov DWORD [fs: SDA.Gdt + KernelCsSelector64], 0000FFFFh
mov DWORD [fs: SDA.Gdt + KernelCsSelector64 + 4], 00AF9A00h
mov DWORD [fs: SDA.Gdt + KernelSsSelector64], 0000FFFFh
mov DWORD [fs: SDA.Gdt + KernelSsSelector64 + 4], 00CF9200h

;;
;; 32-bit User CS/SS 描述符
;;
mov DWORD [fs: SDA.Gdt + UserCsSelector32], 0000FFFFh
mov DWORD [fs: SDA.Gdt + UserCsSelector32 + 4], 00CFFA00h
mov DWORD [fs: SDA.Gdt + UserSsSelector32], 0000FFFFh
mov DWORD [fs: SDA.Gdt + UserSsSelector32 + 4], 00CFF200h
;;
;; 64-bit User CS/SS 描述符
;;
mov [fs: SDA.Gdt + UserCsSelector64], ecx
mov DWORD [fs: SDA.Gdt + UserCsSelector64 + 4], 0020F800h
mov [fs: SDA.Gdt + UserSsSelector64], ecx
mov DWORD [fs: SDA.Gdt + UserSsSelector64 + 4], 0000F200h
;;
;; 32-bit kernel CS/SS 描述符
;;
mov DWORD [fs: SDA.Gdt + KernelCsSelector32], 0000FFFFh
mov DWORD [fs: SDA.Gdt + KernelCsSelector32 + 4], 00CF9A00h
mov DWORD [fs: SDA.Gdt + KernelSsSelector32], 0000FFFFh
mov DWORD [fs: SDA.Gdt + KernelSsSelector32 + 4], 00CF9200h
;;
;; FS/GS 描述符
;; 1) FS base = 12_0000h, limit = 1M, DPL = 0
;; 2) GS base = 10_0000h, limit = 1M, DPL = 0
;;
mov DWORD [fs: SDA.Gdt + FsSelector], 0000FFFFh
mov DWORD [fs: SDA.Gdt + FsSelector + 4], 000F9212h
mov DWORD [fs: SDA.Gdt + GsSelector], 0000FFFFh
mov DWORD [fs: SDA.Gdt + GsSelector + 4], 000F9210h

;;
;; 更新 GDT selector
;;
mov WORD [fs: SDA.KernelCsSelector], KernelCsSelector32
mov WORD [fs: SDA.KernelSsSelector], KernelSsSelector32
mov WORD [fs: SDA.UserCsSelector], UserCsSelector32
mov WORD [fs: SDA.UserSsSelector], UserSsSelector32
mov WORD [fs: SDA.FsSelector], FsSelector
mov WORD [fs: SDA.SysenterCsSelector], KernelCsSelector32
mov WORD [fs: SDA.SyscallCsSelector], KernelCsSelector32
mov WORD [fs: SDA.SysretCsSelector], UserCsSelector32

;;
;; 更新 GDT pointer

```



```

;; 1) 此时 GDT base 使用物理地址
;;
mov DWORD [fs: SDA.GdtBase], SDA_PHYSICAL_BASE + SDA.Gdt
mov [fs: SDA.GdtBase + 4], edx
mov WORD [fs: SDA.GdtLimit], 11 * 8 - 1 ; 共 11 个
entry
mov DWORD [fs: SDA.GdtTop], SDA_PHYSICAL_BASE + SDA.Gdt + 10 * 8 ; 指向第 10
号 entry
mov [fs: SDA.GdtTop + 4], edx

;;
;; 更新 IDT pointer
;; 1) 此时 IDT base 使用物理地址
;;
mov DWORD [fs: SDA.IdtBase], SDA_PHYSICAL_BASE + SDA.Idt
mov [fs: SDA.IdtBase + 4], edx
mov WORD [fs: SDA.IdtLimit], 256 * 16 - 1 ; 255 个 vector
(longmode 下)
mov DWORD [fs: SDA.IdtTop], SDA_PHYSICAL_BASE + SDA.Idt ; top 指向 base
mov [fs: SDA.IdtTop + 4], edx

... ...
ret

```

init_system_data_area 函数设置 SDA.Gdt 区域的前 11 个描述符，分别如下：

- 第 0 号描述符为 NULL descriptor。
- 第 1, 2 号描述符是 64-bit 模式下 kernel 级别的 code 与 data 描述符。
- 第 3, 4 号描述符是 32 位 user 级别的 code 与 data 描述符。
- 第 5, 6 号描述符是 64-bit 模式下 user 级别的 code 与 data 描述符。
- 第 7, 8 号描述符是 32 位 kernel 级别的 code 与 data 描述符。
- 第 9, 10 号描述符是 FS 与 GS 段在 legacy 模式下使用。

这些描述符对应的 selector 分别保存在 SDA 区域的 GDT selector 成员里。由于在 stage1 阶段使用的是物理地址，因此，GDT 的 base 值设为 SDA.Gdt 区域的物理地址。GDT 的 limit 值设置为 $11 \times 8 - 1$ （容纳 11 个描述符）。

IDT 的 base 值设为 SDA.Idt 区域的物理地址，limit 的值为 $256 \times 16 - 1$ （容纳 256 个中断描述符）。

页表结构相关

SDA 区域定义了 legacy 模式下，以及 IA-32e 模式下的页表结构管理记录，如以下代码所示。

代码片段 1-27:

```

struc SDA
... ...

;;
;; paging 管理记录
;;

```

```

XD 功能
.XdValue          RESD      1          ; 保存 XD 中的值，这个值需要开启

.PtBase           RESD      1          ; PT 表基址
.PtTop            RESD      1          ; PT 表顶端
.PtPhysicalBase   RESD      1          ; PT 表物理基址
.PdtBase          RESD      1          ; PDT 表基址
.PdtTop           RESD      1          ; PDT 表顶端
.PdtPhysicalBase  RESD      1          ; PDT 表物理基址
.PptBase          RESD      1          ; PPT 表基址
.PptTop           RESD      1          ; PPT 表顶端
.PptPhysicalBase  RESD      1          ; PPT 表物理基址

;;
;; 下面是 legacy 模式下的 PPT 表
;;
ALIGNB 32
.Ppt              RESQ      4          ; PPT 表项

;;
;; long-mode 下的 paging 管理记录
;;
.PxtBase64        RESQ      1          ; PXT 基址
.PptBase64        RESQ      1          ; PPT 表基址 (PDPT)
.PdtBase64        RESQ      1          ; PDT 表基址
.PtBase64         RESQ      1          ; PT 表基址
.PxtTop64         RESQ      1          ; PXT 表顶端
.PtTop64          RESQ      1          ; PT 表顶端
.PdtTop64         RESQ      1          ; PDT 表顶端
.PptTop64         RESQ      1          ; PPT 表顶端
.PxtPhysicalBase64 RESQ      1          ; PXT 物理基址
.PptPhysicalBase64 RESQ      1          ; PPT 表物理基址

;;
;; 记录 PPT 表区域 (2M) 是否有效，有效时表明已经映射 PPT 表
;;
.PptValid         RESB      1          ;

... ..
endstruc

```

PAE 分页模式下的 PDPT 区域嵌在 SDA.Ppt 区域内，共 32 字节。SDA.PptPhysicalBase 存放 SDA.Ppt 区域的物理地址，SDA.PptBase 则存放 SDA.Ppt 的虚拟地址。在进入 stage2 阶段前，common\protected.asm 模块执行下面的指令加载 CR3 寄存器。

代码片段 1-28:

```

;;
;; 加载 PPT 表
;;
mov eax, [fs: SDA.PptPhysicalBase]
mov cr3, eax

```

SDA.PtBase 与 SDA.PtPhysicalBase 分别存放 PAE 分页模式下的 PT 区域虚拟地址及物理地址，它们的值分别为：**C0000000H**（参见表 1-5）与 **200000H**（参见表 1-4）。PAE 分页模式下的 PT 区域大小共 8MB，物理地址从 200000H 到 9FFFFFFH。

SDA.PptBase64 与 SDA.PptPhysicalBase64 分别存放 IA-32e 分页模式下的 PDPT 区域

虚拟地址及物理地址，它们的值分别为：FFFFF800_7DA00000H（参见表 1-6）与 2000000H（参见表 1-4）。IA-32e 分页模式下的 PDPT 区域大小共 2MB，物理地址从 2000000H 到 21FFFFFFH。

pool 相关

系统在运行期间会在内存 pool 里分配一些空间，这些 pool 的管理记录定义在 SDA 区域内。

代码片段 1-29:

```

struc SDA
    ... ..

    ;;
    ;; 下面记录栈空间分配
    ;;
    .UserStackBase          RESQ    1          ; 用户栈基址
    .UserStackPhysicalBase  RESQ    1
    .KernelStackBase       RESQ    1          ; kernel 栈基址
    .KernelStackPhysicalBase RESQ    1

    ;;
    ;; 下面记录 pool 空间分配
    ;;
    .UserPoolBase           RESQ      1          ; 用户 pool 基址
    .UserPoolPhysicalBase   RESQ      1
    .KernelPoolBase         RESQ      1          ; kernel pool 基址
    .KernelPoolPhysicalBase RESQ      1

    ;;
    ;; 下面记录 PCB pool 空间分配
    ;;
    .PcbPoolBase            RESQ      1          ; PCB pool 虚拟基址
    .PcbPoolTop             RESQ      1          ; PCB pool 顶部
    .PcbPoolPhysicalBase    RESQ      1          ; PCB pool 物理基址
    .PcbPoolPhysicalTop     RESQ      1          ; PCB pool 的顶部
    .PcbPoolSize            RESQ      1          ; PCB pool 的大小

    ;;
    ;; long-mode 下 paging 中的 PT pool 管理记录
    ;;
    .PtPoolPhysicalBase     RESQ      1          ; PT pool 物理基址
    .PtPoolPhysicalTop      RESQ      1          ; PT pool 顶部
    .PtPoolSize             RESQ      1          ; PT pool 大小
    .PtPoolBase             RESQ      1          ; PT pool 区域基址
    .PtPoolFree             RESB      1          ; PT pool 是否空闲可用

    ;;
    ;; 备用 PT pool 管理记录
    ;;
    .PtPool2PhysicalBase    RESQ      1
    .PtPool2PhysicalTop     RESQ      1
    .PtPool2Size            RESQ      1
    .PtPool2Base            RESQ      1

```

```

.PtPool2Free          RESB          1

ALIGNB 4
;;
;; TSS pool, 用来为每个处理器分配自己的 TSS 块
;; 注意:
;; 1) 每个 TSS 分配 size 为 100h 字节
;; 2) 共支持 16 个处理器
;;
.TssPoolBase          RESQ          1          ; TSS pool 基址
.TssPoolPhysicalBase  RESQ          1          ; TSS pool 物理基址
.TssPoolTop           RESQ          1          ; TSS pool 顶部
.TssPoolPhysicalTop   RESQ          1          ; TSS pool 物理顶部
.TssPoolGranularity   RESD          1          ; TSS pool 分配的粒度, 默
认为 100h 字节
... ..
endstruc

```

如上面代码所示, 这些 pool 包括:

(1) PCB pool, 为每个处理器分配私有的 PCB 块。在 stage1 阶段使用 alloc_pcb_base 函数进行分配。

- 物理地址区域: 从 100000H 到 11FFFFh。
- 虚拟地址区域: legacy 模式下从 80000000H 到 8001FFFFH, 64-bit 模式下从 FFFFF800_80000000H 到 FFFFF800_8001FFFFH。

(2) TSS pool (位于 SDA.Tss 区域的 4KB 空间), 用来动态分配新的 TSS 块。在 stage1 阶段的 build_stage1_tss 函数里分配。

(3) Stack pool, 用来分配 stack 空间。使用 get_user_stack_pointer 及 get_kernel_stack_pointer 函数进行分配。

- User stack, 物理地址从 1010000H 开始, 虚拟地址从 7FE00000H 开始。
- Kernel stack, 物理地址从 1040000H 开始, 虚拟地址从 FFE00000H 开始, 在 64-bit 模式下从 FFFFFFF80_FFE00000H 开始。

(4) Data pool, 用来动态分配空间。使用 alloc_user_pool 以及 alloc_kernel_pool 函数进行分配。

- User pool, 物理地址从 3001000H 开始, 虚拟地址从 73001000H 开始。
- Kernel pool, 物理地址从 3200000H 开始, 虚拟地址从 83200000H 开始, 在 64-bit 模式下从 FFFFF800_83200000H 开始。

(5) IA-32e 分页模式下的 PT pool, 在虚拟地址映射时动态分配 4K 页表, 调用 get_pt_physical_base32, get_pt_physical_base64, get_pt_virtual_base32 与 get_pt_virtual_base64 函数来分配。包括下面的两个 PT pool:

- 主要 PT pool, 物理地址从 2200000H 到 2FFFFFFH, 虚拟地址从 FFFFF800_82200000H 到 FFFFF800_82FFFFFFH。

- 备用 PT pool, 物理地址从 200000H 到 9FFFFFFH, 虚拟地址从 FFFFF800_8020000H 到 FFFFF800_809FFFFFFH。

(6) VM domain pool, 在初始化 VMCS buffer 时调用 alloc_vm_domain 函数进行分配。

系统在运行过程中分配的空间大多数来自 kernel pool, 如分配 VMXON 区域, VMCS 区域及 VMCS 区域内使用的各个区域 (如 I/O-bitmap, MSR-bitmap 等)。

EPT 页表相关

VMM 所使用的 EPT 页表结构管理记录也定义在 SDA 区域内。

代码片段 1-30:

```

struc SDA
    ... ..

    ;;
    ;; VMX ept (extended page table) 管理记录
    ;;
    .EptPxtBase64          RESQ          1          ; PXT 虚拟地址
    .EptPxtPhysicalBase64  RESQ          1          ; PXT 物理地址
    .EptPxtTop64           RESQ          1          ;
    .EptPxtPhysicalTop64   RESQ          1

    .EptPptBase64          RESQ          1          ; PPT 虚拟地址
    .EptPptPhysicalBase64  RESQ          1          ; PPT 物理地址
    .EptPptTop64           RESQ          1
    .EptPptPhysicalTop64   RESQ          1

    ... ..
endstruc

```

尽管系统为 EPT 页表结构预留了 2MB 空间作为 PML4T 与 PDPT 的固定区域, 实际上系统进行映射时所有页表结构都使用动态分配。在映射 GPA (guest-physical address) 时, 调用 get_ept_page 函数从 kernel pool 里进行分配。

因此, 上面代码片段中的 EPT 页表管理记录并没有在实际中使用。

1.4.5 初始化 SDA

在刚刚进入 common\setup.asm 模块运行时, 处理器处于实模式, 我们可以称之为“pre-stage1”阶段。在 pre-stage1 阶段里可以利用 BIOS 做一些收集信息的工作。

之后, 调用 protected_mode_enter 函数切换到未分页的保护模式, 返回后进入 stage1 阶段。此时, 最重要的工作是调用 init_system_data_area 函数初始化 SDA 结构。

init_system_data_area 函数实现在 lib\stage1.asm 文件里, 可自行查阅。init_system_data_area 函数主要的工作如下。

(1) 初始化 SDA 基本信息, 包含以下几项:

- SDA.Base 设置为 64 位的虚拟地址值 FFFFF800_8002000H, 在 32 位环境下只能

使用低 32 位，也就是 SDA_BASE 值（80020000H）。

- SDA.PhysicalBase 设置为 SDA_PHYSICAL_BASE 值（200000H）。
- SDA.PcbBase 与 SDA.PcbPhysicalBase 分别设置为 PCB_BASE（80000000H），高 32 位为 FFFFF800H，与 PCB_PHYSICAL_BASE 值（100000H）。
- SDA.VideoBufferHead 与 SDA.VideoBufferPtr 指向 B8000H。
- SDA.ApInitDoneCount 初始化为 1 值，指示 BSP 正在初始化。
- SDA.TimerCount ， SDA.LastStatusCode ， SDA.UsableProcessorMask ， SDA.ProcessorMask 与 SDA.NmiIpiRequestMask 初始化为 0 值。

(2) 更新 SDA.MemorySize 与 SDA.BootDriver 值，指示系统的物理内存及 boot 驱动器。

(3) 根据__X64 符号的定义，来设置 SDA.ApLongmode 标志位。

(4) 初始化多个 pool 管理记录，包括 PCB pool，TSS pool，IA-32e 模式下的 PT pool，stack 与 data pool，BTS 与 PEBS pool，以及 VM domain pool。

(5) 初始化描述符表相关的管理记录，包括 GDT selector，GDT 的描述符内容，GDT pointer 与 IDT pointer 值。

(6) 初始化 SRT（System Routine Table）区域管理记录。

(7) 初始化 legacy 模式与 IA-32e 模式下的相关页表结构管理记录。

(8) 初始化 EPT 页表结构管理记录与 GPA mapped list（GPA 映射列表）管理记录。

(9) 根据 DEBUG_RECORD_ENABLE 符号的定义，初始化 DRS（Debug Record Structure）区域管理记录。

(10) 初始化 BTS 与 PEBS 相关的管理记录。

(11) 初始化 EXTINT_RTE 区域管理记录。

(12) 调用 setup_pic8259 配置 8259A 中断控制器。

(13) 安装默认的中断与异常服务例程。

(14) 设置 AP 的 startup routine（启动例程）入口地址，以及 Stage1LockPointer，Stage3LockPointer 与 Stage3LockPointer 的值。

这些初始化数据有部分只适合在 stage1 阶段使用，在进入 stage2 或者 stage3 阶段后，这部分数据会得到更新。

1.4.6 DRS 结构

引入 DRS 结构是为了支持 debug record（调试记录）功能。它的定义在 inc\DebugRecord.inc 文件里，如下所示。

代码片段 1-31:

```

struct DRS
    .ProcessorIndex      RESD      1
    .RecordNumber        RESD      1

    ALIGNB 8
    .Rax                 RESQ      1
    .Rcx                 RESQ      1
    .Rdx                 RESQ      1
    .Rbx                 RESQ      1
    .Rsp                 RESQ      1
    .Rbp                 RESQ      1
    .Rsi                 RESQ      1
    .Rdi                 RESQ      1
    .R8                  RESQ      1
    .R9                  RESQ      1
    .R10                 RESQ      1
    .R11                 RESQ      1
    .R12                 RESQ      1
    .R13                 RESQ      1
    .R14                 RESQ      1
    .R15                 RESQ      1

    .Rflags              RESQ      1
    .Cs                  RESQ      1
    .Rip                 RESQ      1

    ;;
    ;; debug 点记录信息
    ;;
    .Line                RESD      1
    .FileName             RESQ      1

    ;;
    ;; 附加信息
    ;;
    .AppendMsg           RESQ      1

    ;;
    ;; 前缀与后缀信息
    ;;
    .PrefixMsg           RESQ      1
    .PostfixMsg          RESQ      1
    .Address             RESD      1

    ALIGNB 8
    ;;
    ;; 链表指针
    ;;
    .PrevDrs             RESQ      1
    .NextDrs             RESQ      1

    DRS_SIZE             EQU      $
    MAX_DRS_COUNT        EQU      (DRS_AREA_SIZE / DRS_SIZE)
endstruct

```

当我们在源代码里插入 `DEBUG_RECORD` 宏时, 这个宏将收集当前处理器的 context

信息，记录在 DRS 结构里。为了简便，所有的通用寄存器以及地址值都定义为 64 位。当然，我们也可以区分情况来定义：当定义了__X64 符号时，使用 64 位宽，否则使用 32 位宽。

ProcessorIndex 记录当前处理器的编号，RecordNumber 指示当前有多少条调试记录。接下来是 16 个通用寄存器，RFLAGS，CS 及 RIP 值。这里我们统一使用 64-bit 模式下的名字，而不区分 32 位。例如，不定义为 EAX，而是定义为 RAX。

Line 记录当前代码在源文件里的行数，FileName 则记录当前的源文件名字。这两个数据都是由 nasm 编译时产生的。AppendMsg，PrefixMsg 以及 PostfixMsg 分别指向附加信息，前缀信息及后缀信息。这是为了满足显示信息目的而设置的。

最后，由 PrevDrs 与 NextDrs 指针构成一个双向链表结构，方便我们动态添加记录及查阅记录。

1.5 系统启动

系统的启动分为三个阶段：boot 阶段、stage1 阶段、stage2 或者 stage3 阶段。Boot 阶段的主要工作是从磁盘上加载系统所需要的模块，对应的代码实现在 common\boot.asm 文件里。stage1 阶段进行第 1 阶段的初始化工作，对应的代码实现在 common\setup.asm 文件里。stage2 阶段开启分页机制后，并做最后的初始化工作。stage3 阶段将切换到 IA-32e 模式运行在 64 位环境里，对应的代码实现在 common\long.asm 文件里。

1.5.1 Boot 阶段

取决于映像文件系统类型，Boot 模块存放在映像文件（或 U 盘）的 0 号或者 63 号扇区。

- FAT 文件类型的映像文件 c.img 或者 U 盘：Boot 模块存放在 63 号扇区。
- Floppy 映像文件 demo.img：Boot 模块存放在 0 号扇区。

在 BIOS 运行的最后阶段，Boot 模块由 INT19 从磁盘里读入到内存 7C00H 位置上，转入执行。下面是 Boot 模块的执行流程。

代码片段 1-32：

```

;-----
; MAKE_LOAD_MODULE_ENTRY
; input:
;     %1 - 模块加载的内存段地址
;     %2 - 模块在磁盘的扇区号 (LBA)
; output:
;     none
; 描述:
;     1) 用来定义需要加载的模块表
;-----
%macro MAKE_LOAD_MODULE_ENTRY 2

```



```

        %%segment      DW      %1
        %%sector       DW      %2
%endmacro

LOAD_MODULE_ENTRY_SIZE      EQU      4

;
;
; bits 16
;
; ;
; ; 注意:
; ; 1) 现在处理器处于 real mode 下
; ; 2) int 19h 加载 boot 模块进入 BOOT_SEGMENT 段, BOOT_SEGMENT 定义为 7C00h
; ;
;
; org BOOT_SEGMENT
; jmp WORD Boot.Start
;
; ;
; ; 用于保存 int 13h/ax=08h 获得的 driver 参数
; ;
;
driver_parameters_table:
    driver_number      DB      0          ; driver number
    driver_type        DB      0          ; driver type
    cylinder_maximum   DW      0          ; 最大的 cylinder 号
    header_maximum     DW      0          ; 最大的 header 号
    sector_maximum     DW      0          ; 最大的 sector 号
    parameter_table    DW      0          ; address of parameter table
;
; ;
; ; 定义 int 13h 使用的 disk address packet, 用于 int 13h 读/写
; ;
;
disk_address_packet:
    size               DW      10h        ; size of packet
    read_sectors       DW      0          ; number of sectors
    buffer_offset      DW      0          ; buffer far pointer(16:16)
    buffer_selector    DW      0          ; 默认 buffer 为 0
    start_sector       DQ      0          ; start sector
;
; ;
; ;
; ; 加载模块表, 需要加载下面的模块 (符号定义在 ..\inc\support.inc 文件里)
; ; 1) setup 模块
; ; 2) 定义 x64 时加载 long 模块, 在 32 位下加载 protected 模块
; ; 3) guest 的 boot 模块
; ; 4) guest 的 kernel 模块
; ;
;
load_module_table:
    MAKE_LOAD_MODULE_ENTRY      (SETUP_SEGMENT >> 4), SETUP_SECTOR
;
; ;
; ; 在 64 位运行环境下, 需要加载 long 模块, 否则加载 proteccted 模块
; ;
; ;

```

```
%ifndef __X64
    MAKE_LOAD_MODULE_ENTRY                (LONG_SEGMENT >> 4), LONG_SECTOR
%else
    MAKE_LOAD_MODULE_ENTRY                (PROTECTED_SEGMENT >> 4), PROTECTED_SECTOR
%endif

;;
;; 假如定义了 GUEST_ENABLE 符号，则需要加载 GuestBoot 与 GuestKernel 模块
;;
%ifdef GUEST_ENABLE
    MAKE_LOAD_MODULE_ENTRY                (GUEST_BOOT_SEGMENT >> 4), GUEST_BOOT_SECTOR
    MAKE_LOAD_MODULE_ENTRY                (GUEST_KERNEL_SEGMENT >> 4), GUEST_KERNEL_SECTOR
%endif

load_module_table.end:

; ; ##### boot 模块入口 #####

Boot.Start:
    cli
    NMI_DISABLE                            ; 关闭 NMI
    FAST_A20_ENABLE                        ; 开启 A20 位

    ;;
    ;; set BOOT_SEG environment
    ;;
    mov ax, cs
    mov ds, ax
    mov ss, ax
    mov es, ax
    mov sp, BOOT_SEGMENT                  ; 设 stack 底为 BOOT_SEGMENT

    mov [driver_number], dl               ; 保存 boot driver
    mov WORD [buffer_selector], es       ; 读磁盘的 buffer segment 设置
为 es
    call get_driver_parameters           ; 读磁盘参数
    call clear_screen

    ;;
    ;; 下面进行模块加载工作
    ;;

    mov bx, load_module_table            ; 模块加载表
load_module.loop:
    mov ax, [bx]
    mov [buffer_selector], ax             ; segment 从模块加载表中读取
    mov WORD [buffer_offset], 0          ; selector = 0
    mov ax, [bx + 2]
    mov [start_sector], ax              ; sector 从模块加载表中读取
    call load_module
    add bx, LOAD_MODULE_ENTRY_SIZE
    cmp bx, load_module_table.end
    jnb load_module.loop
```

```
;;
;; 加载模块到内存后，转入 setup 模块入口点执行 (SETUP_SEGMENT + 4)
;;

    jmp SETUP_SEGMENT + 4

    ... ..
```

Boot 模块定义了一个 `load_module_table` 表，每个表项对应一个需要加载的模块，提供模块加载所需要的信息，包括加载到内存的位置，以及模块在磁盘上的扇区号。

使用 `MAKE_LOAD_MODULE_ENTRY` 宏来生成一个表项，若执行 Build 命令时定义了 `__X64` 与 `GUEST_ENABLE` 符号，则会生成下面的 4 个表项。

- (1) Setup 模块加载表项。
- (2) Long 模块加载表项。
- (3) GuestBoot 模块加载表项。
- (4) GuestKernel 模块加载表项。

没有定义 `__X64` 及 `GUEST_ENABLE` 符号时，只会生成下面的 2 个表项。

- (1) Setup 模块加载表项。
- (2) Protected 模块加载表项。

Boot 模块的工作很简单，首先将引导的驱动器号 (boot driver) 保存在 Boot 模块内的 `driver_number` 变量里，调用 `get_driver_parameters` 函数来读取驱动器参数。磁盘参数保存在模块前面定义的 `driver_parameters_table` 里。在 `read_sector` 函数内部会使用这些参数将 LBA 扇区号转化为 CHS 格式。

然后，根据 `load_module_table` 表多次调用 `load_module` 函数加载相应的模块。`load_module` 函数内部会调用 `read_sector` 函数读取磁盘扇区。

最后，加载模块完毕，Boot 模块直接转入 `setup` 模块继续执行。

1.5.2 stage1 阶段

从 Boot 模块转入 `setup` 模块后，处理器仍然运行在实模式下。在处理器切入保护模式之前，这个阶段称为“pre-stage1”阶段。`lib\stage1.asm` 文件内实现的函数是为了支持 stage1 阶段的运行。

在 pre-stage1 阶段里的工作是：调用 `get_system_memory` 函数通过 BIOS 来获取系统的可用物理内存。然后，调用 `protected_mode_enter` 函数直接切入保护模式，如下面代码所示。

代码片段 1-33:

```
;;
```

```

;; 说明:
;; 1) 模块开始点是 SETUP_SEGMENT
;; 2) 模块头存放的是“模块 size”
;; 3) load_module() 函数将模块加载到 SETUP_SEGMENT 位置上
;; 4) SETUP 模块的“入口点”是: SETUP_SEGMENT + 4

[SECTION .text]
org SETUP_SEGMENT

;
;; 在模块的开头 dword 大小的区域里存放模块的大小,
;; load_module 会根据这个 size 加载模块到内存
;;

        DD SETUP_LENGTH                ; 这个模块的 size

;;
;; 模块当前运行在 16 位实模式下
;;
        bits 16

SetupEntry:                                ; 这是模块代码的入口点

        cli
        NMI_DISABLE

        ;;
        ;; 在实模式下读取系统可用物理内存
        ;;
        call get_system_memory

        ;;
        ;; 调用 protected_mode_enter() 直接进入保护模式
        ;;
        call protected_mode_enter

```

get_system_memory 与 proected_mode_enter 函数实现在 lib\crt16.asm 文件里, protected_mode_enter 函数返回时, setup 模式进入 stage1 阶段(未分页保护模式)。

代码片段 1-34:

```

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; 下面是 32 位代码 ;;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

        bits 32

        ;;
        ;; 通过处理器的 long-mode 支持检查
        ;; 否则, 显示出错信息, 进入 HLT 状态
        ;;
        call pass_the_check_longmode

        ;;
        ;; 下面进行数据的初始化设置:

```

```

;; 1) 首先, 初始化 SDA (System Data Area) 区域数据
;; 2) 然后, 初始化 PCB (Processor Control Block) 区域数据
;;
;; 说明:
;; 1) SDA 数据是所有处理器共享, 所以必须先初始化
;; 2) PCB 数据是 logical processor 数据, 共支持 16 个 PCB 块
;; 3) PCB 数据是动态分配, 每个 PCB 块基址不同
;;
;;
;;
;; FS 段说明:
;; 1) FS 指向 SDA (System Data Area) 区域, 是所有 logical processor 共享的数据区域
;; 注意:
;; 1) 在支持 64 位的处理器上才能直接写 IA_FS_BASE 寄存器
;; 2) 否则, 需要开启保护模式来加载 FS 段基址
;; 3) GS 段基址在后续代码中更新
;;

```

```
call init_system_data_area
```

PcbInitEntry:

```

;;
;; 设置 PCB (Processor Control Block) 内容
;; 说明:
;; 1) 此处为 logical processor 的 PCB 初始化入口 (包括 BSP 与 AP)
;; 2) 每个 logical processor 都需要经过下面的 PCB 数据初始化
;;
;;
;; 调用 update_stage1_gs_segment() 更新 GS 段信息
;; 注意:
;; 1) 先获得 PCB 物理地址写入 GS 段
;; 2) 再分配 PCB 虚拟地址
;; 3) 最后映射 stage1 阶段 PCB
;;
call update_stage1_gs_segment

;;
;; 分配一个 stage1 阶段使用的 kernel stack, 此时使用物理地址
;; 1) 需将 stack pointer 调整到 4k base 值的顶部
;;
call alloc_stage1_kernel_stack_4k_physical_base
add eax, 0FF0h
mov esp, eax

;;
;; 加载 GDTR 与 IDTR
;;
lgdt [fs: SDA.GdtPointer]
lidt [fs: SDA.IdtPointer]

;;
;; 更新 selector
;;
call update_stage1_selector

;;
;; 构造 TSS 环境
;; 1) 此时 TSS 环境使用物理地址

```

```

;;
call build_stage1_tss

;;
;; 开启 local APIC
;;
call pass_the_enable_apic

;;
;; 更新处理器信息
;;
call update_processor_basic_info
call update_processor_topology_info
call update_debug_capabilities_info
call init_memory_type_manage
call init_perfmon_unit

... ..

```

protected_mode_enter 函数返回后进入 32 位的 stage1 阶段，接着调用 pass_the_check_longmode 检查处理器是否支持 long-mode，如果不支持则使处理器进入 HLT 状态。然后调用 init_system_data_area 函数初始化 SDA 区域（参见第 1.4.5 节）。pass_the_check_longmode 与 init_system_data_area 函数实现在 lib\system_data_manage.asm 文件里。

下面来到 PcbInitEntry 入口点，这是所有 APs（application processors）在 stage1 阶段的入口点。而 PcbInitEntry 之前的工作由 BSP（bootstrap processor）负责，也就是 init_system_data_area 函数只会由 BSP 进行调用，SDA 区域只会初始化一次。

所有处理器执行，下面的流程：

- (1) 调用 update_stage1_gs_segment 函数来初始化自己的 PCB 块（参见第 1.4.3 节）。
- (2) 调用 alloc_stage1_kernel_stack_4k_physical_base 函数从 kernel stack pool 里分配 4KB 的区域，返回的物理地址值调整到高端后赋给 ESP 寄存器。
- (3) 执行 LGDT 与 LIDT 指令从 SDA 区域里分别加载 GDT pointer 与 IDT pointer 到 GDTR 与 IDTR 寄存器。
- (4) 调用 update_stage1_selector 函数更新 stage1 阶段下的所有段寄存器。
- (5) 调用 pass_the_enable_apic 函数检查处理器是否支持 local APIC，并开启 local APIC。
- (6) 调用 update_processor_basic_info 函数更新处理器的基本信息。
- (7) 调用 update_processor_topology_info 函数更新处理器的拓扑信息，这个函数实现在 lib\apic.asm 文件里。
- (8) 调用 update_debug_capabilities_info 函数更新处理器 debug 方面的能力信息，这个函数实现在 lib\debug.asm 文件里。

(9) 调用 `init_memory_type_manage` 函数初始化 MTRR 寄存器管理的内存模块。

(10) 调用 `init_perfmon_unit` 函数初始化处理器的性能监控模块。

在本书平台里，`debug`、`MTRR` 及性能监控模块并没有实际应用。

1.5.2.1 stage1 阶段的多处理器初始化

系统在启动阶段实行处理器同步初始化策略（同时完成某个阶段的工作）。也就是说，当 BSP 完成 `stage1` 阶段工作后，**必须等待所有 APs 也完成 `stage1` 阶段工作，才进入下一阶段的工作。**

`stage1` 阶段多处理器同步初始化的处理逻辑，如下面伪代码所示。

```
/*
 * BSP 在 stage1 阶段的收尾工作
 */
BspStage1Entry:
    ... ..

    BROADCAST_IPI    INIT_SIPI_SIPI                // 广播 INIT-SIPI-SIPI 消息序列
    Stage1Lock = 0;    // 放开 stage1 阶段锁信号
    wait_for_stage1_ap_done();    // 等待所有 APs 完成 stage1 阶段工
作。
    ApInitDoneCount = 1;    // 重置 ApInitDoneCount 计数

    if (ApLongmode == 1)
    {
        goto BspLongEntry;    // 转入 BSP 的 stage3 阶段
    }
    else
    {
        goto BspProtectedEntry;    // 转入 BSP 的 stage2 阶段
    }

/*
 * AP 在 stage1 阶段的工作
 */
ApStage1Entry:
    /*
     * 等待获得 stage1 锁
     */
    while (Stage1Lock) ;    // 等待，直到 stage1Lock == 0
    Stage1Lock = 1;    // 锁上 stage1Lock

    ... ..

    /*
     * ### APs 的 stage1 收尾工作 ###
     */
    ApInitDoneCount++;    // 增加 ApInitDoneCount 计数值
    Stage1Lock = 0;    // 放开 stage1 阶段锁
```

```

/*
 * ### 下面根据 ApLongmode 值等待相应的锁，并转入相应的入口 ###
 */
if (ApLongmode == 1)
{
    /*
     * 继续等待 stage3 锁
     */
    while (Stage3Lock);           // 等待，直到 stage3Lock == 0
    Stage3Lock = 1;               // 锁上 stage3Lock
    goto ApLongEntry;             // 转入 stage3 阶段的 AP 入口
}
else
{
    /*
     * 继续等待 stage2 锁
     */
    while (Stage2Lock);           // 等待，直到 stage2Lock == 0
    Stage2Lock = 1;               // 锁上 stage2Lock
    goto ApProtectedEntry;        // 转入 stage2 阶段的 AP 入口
}

```

BSP 在 stage1 阶段的收尾工作

- (1) 广播 INIT-SIPI-SIPI 消息序列。
- (2) 开放 stage1 锁信号。
- (3) 等待所有 APs 完成 stage1 阶段工作。
- (4) 重置 ApInitDoneCount 计数值为 1，表明有一个处理器在初始化（即 BSP）。
- (5) 进入下一阶段：
 - ApLongmode = 1 时，进入 stage3 阶段。
 - ApLongmode = 0 时，进入 stage2 阶段。

APs 在 stage1 阶段的处理流程

- (1) 在进入 stage1 阶段前，等待获取 stage1 锁信号，直到 Stage1Lock = 0 时进入 stage1 阶段，并对 Stage1Lock 进行上锁。
- (2) APs 进行相关的 stage1 阶段初始化工作。
- (3) 增加 ApInitDoneCount 计数值。
- (4) 开放 stage1 锁信号（即设置 Stage1Lock = 0）。
- (5) 等待下一阶段锁信号：
 - ApLongmode = 1 时，等待获得 stage3 阶段锁信号，并锁上 Stage3Lock。
 - ApLongmode = 0 时，等待获得 stage2 阶段锁信号，并锁上 Stage2Lock。
- (6) 转入下一阶段的 APs 入口。

当 BSP 转入下一阶段后，说明所有的 APs 已经同步完成了 stage1 阶段的工作。APs 会继续等待，直至 BSP 完成下一阶段工作后开放下一阶段的锁信号。

1.5.2.2 BSP 的收尾工作

当处理器执行完 stage1 阶段的初始化工作后，接下来需要区分处理器属于 BSP 还是 AP。如果属于 BSP，则需要进行 BSP 在 stage1 阶段的收尾工作，如以下代码所示。

代码片段 1-35:

```
%ifndef DBG
;;
;; Stage1 阶段的最后工作，检查是否为 BSP
;; 1) 是，则发送 INIT-SIPI-SIPI 序列
;; 2) 否，则等待接收 SIPI
;;
cmp BYTE [gs: PCB.IsBsp], 1
jne ApStage1End

;;
;; 这是 BSP 第 1 阶段的最后工作：
;; 1) 发送 INIT-SIPI-SIPI 序列给 AP
;; 2) 等待所有 AP 第 1 阶段完成
;; 3) 转入下一阶段工作
;;
call wait_for_ap_stage1_done

;;
;; 置 ApInitDoneCount = 1，为下一阶段计数做准备
;;
mov DWORD [fs: SDA.ApInitDoneCount], 1

%endif

;;
;; 检查是否需要进入 longmode
;; 1) 是，跳过 stage2，进入 stage3 阶段 (longmode 模式)
;; 2) 否，进入 stage2 阶段
;;
cmp DWORD [fs: SDA.ApLongmode], 1
mov eax, [PROTECTED_SEGMENT + 4]
cmov eax, [LONG_SEGMENT + 4]

;;
;; 转入下阶段入口
;;
jmp eax
```

处理器通过 PCB.IsBsp 标志位来检查自己是否为 BSP。如果不属于 BSP 则进入 AP 的 stage1 阶段收尾工作，属于 BSP 则调用 wait_for_ap_stage1_done 函数广播 INIT-SIPI-SIPI 消息序列，并等待所有 APs 完成 stage1 阶段工作。

代码片段 1-36:

```
;-----
; wait_for_ap_stage1_done()
; input:
;   none
; output:
;   none
; 描述:
;   1) 发送 INIT-SIPI-SIPI 消息序列给 AP
;   2) 等待 AP 完成第 1 阶段工作
```

```

;-----
wait_for_ap_stage1_done:
    push ebx
    push edx

    ;;
    ;; local APIC 第 1 阶段使用物理地址
    ;;
    mov ebx, [gs: PCB.LapicPhysicalBase]

    ;;
    ;; 发送 IPI, 使用 INIT-SIPI-SIPI 序列
    ;; 1) 从 SDA.ApStartupRoutineEntry 提取 startup routine 地址
    ;;
    mov DWORD [ebx + ICR0], 000c4500h                ; 发送 INIT IPI, 使所有 APs 执行
INIT
    mov esi, 10 * 1000                                ; 延时 10ms
    call delay_with_us

    ;;
    ;; 下面发送两次 SIPI, 每次延时 200μs
    ;; 1) 提取 Ap Startup Routine 地址
    ;;
    mov edx, [fs: SDA.ApStartupRoutineEntry]
    shr edx, 12                                        ; 4K 边界
    and edx, 0FFh
    or edx, 000c4600h                                ; Start-up IPI
    ;;
    ;; 首次发送 SIPI
    ;;
    mov DWORD [ebx + ICR0], edx                        ; 发送 Start-up IPI
    mov esi, 200                                      ; 延时 200μs
    call delay_with_us

    ;;
    ;; 再次发送 SIPI
    ;;
    mov DWORD [ebx + ICR0], edx                        ; 再次发送 Start-up IPI
    mov esi, 200
    call delay_with_us

    ;;
    ;; 开放第 1 阶段 AP Lock
    ;;
    xor eax, eax
    mov ebx, [fs: SDA.Stage1LockPointer]
    xchg [ebx], eax

    ;;
    ;; 等待 AP 完成 stage1 工作:
    ;; 检查处理器计数 ProcessorCount 是否等于 LocalProcessorCount 值
    ;; 1) 是, 所有 AP 完成 stage1 工作
    ;; 2) 否, 在等待
    ;;
wait_for_ap_stage1_done.@0:
    mov eax, [fs: SDA.ApInitDoneCount]
    cmp eax, [gs: PCB.LogicalProcessorCount]
    jb wait_for_ap_stage1_done.@0

    pop edx

```

```
pop ebx
ret
```

wait_for_ap_stage1_done 函数实现在 lib\stage1.asm 文件里，工作流程如下。

- (1) 广播 INIT 消息，并延时 10ms。等待所有 APs 完成处理器内部的初始化。
- (2) 广播第 1 次 SIPI 消息，并延时 200μs，等待 AP 接收 SIPI 消息。广播第 2 次 SIPI 消息，并延时 200μs，等待某些可能遗漏的 AP 接收 SIPI 消息。
- (3) 开放 stage1 阶段锁信号：Stage1LockPointer 指向的值清 0。
- (4) 检查 SDA.ApInitDoneCount 计数值是否小于 PCB.LogicalProcessorCount 值。是则循环等待，否则函数返回。

wait_for_ap_stage1_done 函数通过检查 SDA.ApInitDoneCount 值，来确定所有 APs 是否已经完成 stage1 阶段的工作。因而，这个函数返回时，表明所有处理器已经完成 stage1 阶段的工作。

INIT-SIPI-SIPI 消息序列

BSP 广播 INIT 消息时，使用“**All Excluding Self**”（所有处理器除了自己）目标模式。所有 APs 接收到 INIT 消息后，进行处理器内部的初始化工作。

当所有处理器完成内部初始化工作后，处理器处于实模式，并进入“**wait-for-SIPI**”状态，等待接收 SIPI 消息。在 VMX 架构下，VMM 可以通过 VMCS 区域的 activity state 字段将虚拟处理器设置为“wait-for-SIPI”状态（参见第 3.8.5 节）。

BSP 接着广播 SIPI 消息，所有处于“wait-for-SIPI”状态的处理器接收到 SIPI 消息后，执行 SIPI 消息中的 startup routine（启动例程）。Intel 推荐发送两次 SIPI 消息，这是因为有些处理器可能会遗漏接收 SIPI 消息，所以发送第二次 SIPI 消息以确保所有处理器执行 startup routine。

SIPI 消息只有处理器在“wait-for-SIPI”状态下才会响应，在其他状态下会被阻塞（参见第 4.17.2 节），因此，当处理器执行 startup routine 时不会被 SIPI 消息所中断。

1.5.2.3 APs 的 stage1 阶段工作

BSP 广播的 SIPI 消息中，目标的 startup routine 入口地址存放在 SDA.ApStartupRoutineEntry 值里。在调用 init_system_data_area 函数初始化 SDA 的最后阶段将 ApStartupRoutineEntry 设为 ApStage1Entry。

代码片段 1-37:

```
;$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
;$      AP Stage1 Startup Routine      $
;$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
```

ApStage1Entry:

```
cli
```

```

;;
;; real mode 初始环境
;;
xor ax, ax
mov ds, ax
mov es, ax
mov ss, ax
mov sp, 7FF0h

;;
;; 检查 ApLock, 是否允许 AP 进入 startup routine 执行
;;
xor eax, eax
mov esi, 1

;;
;; 获得自旋锁
;;
AcquireApStage1Lock:
;;
;; 1) 使用 cmpxchg 指令
;;
lock cmpxchg [ApStage1Lock], esi
jz AcquireApStage1LockOk

;;
;; 2) 使用 bts 指令
;; lock bts DWORD [ApStage1Lock], 0
;; jnc AcquireApStage1LockOk
;;

CheckApStage1Lock:
mov eax, [ApStage1Lock]
test eax, eax
jz AcquireApStage1Lock
pause
jmp CheckApStage1Lock

AcquireApStage1LockOk:

;;
;; 进入保护模式
;;
call protected_mode_enter

bits 32

;;
;; 转入执行 PCB 初始化
;; 注意:
;; 此处使用绝对地址跳转, 因为 cs.base = 0
;;
mov eax, PcbInitEntry
jmp eax

```

APs 的 startup routine 实现在 common\setup.asm 模块的末尾。在初始化基本的段寄存

器后，等待 stage1 阶段的锁信号。这里使用了 CMPXCHG 指令，也可以使用 BTS 指令进行“test-and-set”的操作。这个 stage1 阶段定义在 setup.asm 模块的尾部。

代码片段 1-38:

```
[SECTION .data]

;;
;; 定义 AP 允许执行锁，初始状态为 1（已上锁）
;;
ApStage1Lock DD 1           ;; stage1 (setup) 阶段的锁
ApStage2Lock DD 1           ;; stage2 (protected) 阶段的锁
ApStage3Lock DD 1           ;; stage3 (long) 阶段的锁
```

同时，setup.asm 模块也定义了 stage2 与 stage3 阶段的锁。它们的初始值为 1，表明不可进入。

当 BSP 发送完 INIT-SIPI-SIPI 消息序列后，开放 ApStage1Lock 锁信号（ApStage1Lock = 0）。第一个抢先得得到锁的 AP 将进入“pre-stage1”阶段执行。

AP 接着调用 protected_mode_enter 函数切换到保护模式，返回后进入 stage1 阶段。然后转入 PcbInitEntry 执行 stage1 阶段的初始化工作（参见代码片段 1-34 所示）。

APs 的 stage1 阶段收尾工作

在前面的代码片段 1-35 里，当处理器检查发现自己属于 AP 时，跳转到下面代码片段 1-39 所示的 ApStage1End 处继续执行。

代码片段 1-39:

```
%ifndef DBG

;;
;; AP 第 1 阶段最后工作说明：
;; 1) 增加 ApInitDoneCount 计数值
;; 2) AP 等待第 2 阶段锁（等待 BSP 开放 stage2 锁）
;;

ApStage1End:

#ifdef TRACE
    mov esi, Stage1.Msg
    call puts
#endif

;;
;; 增加完成计数
;;
lock inc DWORD [fs: SDA.ApInitDoneCount]

;;
;; 开放第 1 阶段 AP Lock
;;
xor eax, eax
mov ebx, [fs: SDA.Stage1LockPointer]
xchg [ebx], eax
```

```

;;
;; 检查是否需要进入 longmode
;; 1) 是, 跳过 stage2, 等待 stage3 锁, 进入 stage3 阶段 (longmode 模式)
;; 2) 否, 等待 stage2 锁, 进入 stage2 阶段
;;
cmp DWORD [fs: SDA.ApLongmode], 1
je ApStage1End.WaitStage3

;;
;; 现在等待 stage2 的锁开放
;;
mov esi, [fs: SDA.Stage2LockPointer]
call get_spin_lock

;;
;; 进入 stage2
;;
jmp [PROTECTED_SEGMENT + 8]

ApStage1End.WaitStage3:
;;
;; 现在等待 stage3 锁开放
;;
mov esi, [fs: SDA.Stage3LockPointer]
call get_spin_lock

;;
;; 进入 stage3
;;
jmp [LONG_SEGMENT + 8]

```

APs 在 stage1 阶段的最后工作如下所示。

- (1) 增加 ApInitDoneCount 计数, 表明处理器已经完成这个阶段的初始化。
- (2) 开放 stage1 阶段的锁信号, 允许其他 APs 进入 pre-stage1 阶段执行。
- (3) 调用 get_spin_lock 函数等待下一阶段锁:
 - 当 ApLongmode = 1 时, 等待 stage3 阶段的锁信号。
 - 当 ApLongmode = 0 时, 等待 stage2 阶段的锁信号。

当 APs 获得下一阶段锁后, 转入下一阶段的 AP 入口继续执行。

1.5.3 stage2 阶段

stage2 阶段的代码实现在 common\protected.asm 文件里, 由 setup 模块转入。Protected 模块被 boot 加载到内存 20000h 位置上。在开启分页机制前, 我们可以称这个阶段为“pre-stage2”阶段, 直到分页机制开启后, 我们称之为“stage2”阶段。

在 protected 模块头定义了下面的标志值。

代码片段 1-40:

```
org PROTECTED_SEGMENT
```

DD	PROTECTED_LENGTH	; 保存模块长度
DD	BspProtectedEntry	; BSP 的 protected 模块入口
DD	ApStage2Routine	; AP 的 protected 模块入口

PROTECTED_SEGMENT 处存放 protected 模块的大小, PROTECTED_SEGMENT + 4 存放 BSP 在 protected 模块的入口地址, PROTECTED_SEGMENT + 8 存放 APs 在 protected 模块的入口地址。

因此, 当没有定义__X64 符号时, BSP 会转入 BspProtectedEntry 处执行, 而 APs 会转入 ApStage2Routine 处执行。

代码片段 1-41:

```

bits 32

BspProtectedEntry:
;;
;; 初始化 stage2 阶段的 paging 环境
;; 使用 PAE 分页结构
;;
call init_ppt_area
call init_pae_page

;;
;; 更新 stage2 的 GDT/IDT pointer 值
;;
call update_stage2_gdt_idt_pointer

ApProtectedEntry:
;;
;; 映射 stage2 阶段的 PCB 区域
;;
call map_stage2_pcb

;;
;; 更新 stage2 的 kernel stack
;; 1) 需要开启分页前及更新 FS 段前执行
;; 2) 将调整后的 kernel stack 保存在 PCB.KernelStack 内
;;
call update_stage2_kernel_stack

;;
;; 读 FS/GS base 值
;;
mov esi, [fs: SDA.Base]
mov edi, [gs: PCB.Base]

;;
;; 加载 PPT 表
;;
mov eax, [fs: SDA.PptPhysicalBase]
mov cr3, eax

;;
;; 开启 paging 管理
;;
mov eax, cr0
bts eax, 31

```

```

mov cr0, eax

;;
;; 更新 fs 与 gs 段为开启 paging 后的 base 值
;;
xor edx, edx
mov eax, esi
mov ecx, IA32_FS_BASE
wrmsr
mov eax, edi
mov ecx, IA32_GS_BASE
wrmsr

;;
;; 更新处理器状态
;;
or DWORD [gs: PCB.ProcessorStatus], CPU_STATUS_PG | CPU_STATUS_PE

;;
;; 更新 kernel stack
;;
mov esp, [gs: PCB.KernelStack]

;;
;; 重新加载 GDTR/IDTR 以及 TR
;;
lgdt [fs: SDA.GdtPointer]
lidt [fs: SDA.IdtPointer]
;;
;; 更新 TSS 环境, 这是 legacy 模式下最终的 TSS 环境
;;
call update_stage2_tss

;;
;; 设置 user stack pointer
;;
call get_user_stack_4k_pointer
mov [gs: PCB.UserStack], eax

;;
;; 配置 SYSENTER/SYSEXIT 使用环境
;;
call setup_sysenter

;;
;; 初始化处理器 debug store 单元
;;
call init_debug_store_unit

... ..

```

如上面代码所示, 在 stage2 阶段里 BSP 比 APs 多做了额外的工作: 初始化 PAE 页表结构以及更新 stage2 阶段的 GDT 与 IDT 信息, 因为这个工作只需由 BSP 做一次就可以了。

stage2 阶段的初始化流程如下:

- (1) BSP 调用 `init_ppt_area` 函数初始化 PAE 分页模式下的 PDPT 区域。
- (2) BSP 调用 `init_pae_page` 函数建立 stage2 阶段的页表映射。
- (3) BSP 调用 `update_stage2_gdt_idt_pointer` 函数更新 stage2 阶段的 GDT 与 IDT pointer 值。
- (4) 下面是所有处理器的共同工作。接下来调用 `map_stage2_pcb` 函数映射处理器的 PCB 区域。
- (5) 调用 `update_stage2_kernel_stack` 函数更新 stage2 阶段的 stack，目的是将 ESP 的物理地址转化为虚拟地址。
- (6) 开启分页机制，使用 PAE 分页模式。
- (7) 更新 FS 与 GS 段基址，目的是将物理地址替换为虚拟地址。
- (8) 更新处理器状态标志位 `ProcessorStatus`，指示已经开启分页机制。
- (9) 重新加载 GDTR 与 IDTR 寄存器。
- (10) 更新 stage2 阶段的 TSS 环境。
- (11) 设置 user stack。
- (12) 初始化 `SYSENTER/SYSEXIT` 指令使用环境。
- (13) 初始化 debug store 模块。
- (14) 进入 stage2 阶段最后初始化工作。

BSP 负责初始化 PAE 分页模式的页表转换环境，`init_ppt_area` 函数用来初始化 PDPT 区域。这个 PDPT 区域嵌入到 `SDA.Ppt` 区域里（参见第 1.4.4 节描述的页表结构相关信息）。

代码片段 1-42:

```

;-----
;-----
; init_pae_page(): 初始化设置 PAE-paging 模式的页转换表结构
; input:
;     none
; output:
;     none
;
; 系统页表结构:
;     * 0xc0000000-0xc07fffff (共 8M) 映射到物理页面 0x200000-0x9fffff 上, 使用 4K 页面
;
; 初始化区域描述:
;     1) 0x7000-0x1ffff 分别映射到 0x7000-0x1ffff 物理页面, 用于一般的运作
;     2) 0xb8000 - 0xb9fff 分别映射到 0xb8000-0xb9fff 物理地址, 使用 4K 页面, 用于 VGA 显
示区域
;     3) 0x80000000-0x8000ffff (共 64KB) 映射到物理地址 0x100000-0x10ffff 上, 用于系统数据
结构
;     4) 0x400000-0x400fff 映射到 1000000h page frame 使用 4K 页面, 用于 DS store 区域

```

```

;      5) 0x600000-0x7ffffff 映射到 0FEC00000h 物理页面上, 使用 2M 页面, 用于 LPC 控制器区域
(I/O APIC)
;      6) 0x800000-0x9ffffff 映射到 0FEE00000h 物理地址上, 使用 2M 页面, 用于 local APIC 区域
;      7) 0xb0000000 开始映射到物理地址 0x1100000 开始, 使用 4K 页面, 用于 VMX 数据空间

;
; 注, 下面是动态映射区域:
;      1) 0x7fe00000 开始映射到物理地址 0x1010000 开始, 使用 4K 页面, 用于 user 的 stack 空间
;      2) 0xffe00000 开始映射到物理地址 0x1020000 开始, 使用 4K 页面, 用于 kernel 的 stack 空
间
;-----
-----

init_pae_page:
    push ecx

    ;;
    ;; 清页转换表区域(8M)
    ;;
    mov esi, [fs: SDA.PtPhysicalBase]
    call clear_8m_for_pae

    ;;
    ;; 映射 8M 页转换表区域 (使用 4K page)
    ;;
    call map_pae_page_transition_table

    ;;
    ;; 0x7000-0x9000 分别映射到 0x7000-0x9000 物理页面, 使用 4K 页面
    ;;
    mov esi, 7000h
    mov edi, 7000h
    mov ecx, (10000h - 7000h) / 1000h
do_virtual_address_mapping.loop1:
    mov eax, PHY_ADDR | US | RW | P
    call do_virtual_address_mapping
    add esi, 1000h
    add edi, 1000h
    dec ecx
    jnz do_virtual_address_mapping.loop1

#ifdef GUEST_ENABLE
    mov esi, GUEST_BOOT_SEGMENT
    mov edi, GUEST_BOOT_SEGMENT
    mov eax, PHY_ADDR | US | RW | P
    call do_virtual_address_mapping

    mov esi, GUEST_KERNEL_SEGMENT
    mov edi, GUEST_KERNEL_SEGMENT
    mov eax, PHY_ADDR | US | RW | P
    mov ecx, [GUEST_KERNEL_SEGMENT]
    add ecx, 0FFFh
    shr ecx, 12
    call do_virtual_address_mapping_n
#endif

    ;;
    ;; 映射 protected 模块区域, 使用 4K 页
    ;;

```

```

    mov esi, PROTECTED_SEGMENT
    mov edi, PROTECTED_SEGMENT
    mov eax, PHY_ADDR | US | RW | P

#ifdef __STAGE2
    mov ecx, (PROTECTED_LENGTH + 0FFFh) / 1000h
#endif
    call do_virtual_address_mapping_n

    ;;
    ;; 0xb8000 - 0xb9fff 分别映射到 0xb8000-0xb9fff 物理地址, 使用 4K 页面
    ;;
    mov esi, 0B8000h
    mov edi, 0B8000h
    mov eax, XD | PHY_ADDR | US | RW | P
    call do_virtual_address_mapping
    mov esi, 0B9000h
    mov edi, 0B9000h
    mov eax, XD | PHY_ADDR | US | RW | P
    call do_virtual_address_mapping

    ;;
    ;; 映射 System Data Area 区域
    ;;
    mov esi, [fs: SDA.Base]                ; SDA virtual address
    mov edi, [fs: SDA.PhysicalBase]        ; SDA physical address
    mov ecx, [fs: SDA.Size]                ; SDA size
    add ecx, [fs: SRT.Size]                ; 映射区域大小 = SDA size + SRT
size
    add ecx, 0FFFh
    shr ecx, 12
do_virtual_address_mapping.loop2:
    mov eax, XD | PHY_ADDR | RW | P
    call do_virtual_address_mapping
    add esi, 1000h
    add edi, 1000h
    dec ecx
    jnz do_virtual_address_mapping.loop2

    ;;
    ;; 映射 System service routine table 区域 (4K)
    ;;
    mov esi, [fs: SRT.Base]
    mov edi, [fs: SRT.PhysicalBase]
    mov eax, XD | PHY_ADDR | RW | P
    call do_virtual_address_mapping

    ;;
    ;; 0x400000-0x400fff 映射到 1000000h page frame 使用 4K 页面
    ;;
    mov esi, 400000h
    mov edi, 1000000h
    mov eax, XD | PHY_ADDR | RW | P
    call do_virtual_address_mapping

    ;;
    ;; 0x600000-0x600fff 映射到 0FEC00000h 物理地址上, 使用 4K 页面
    ;;

```

```

mov esi, IOAPIC_BASE
mov edi, 0FEC0000h
mov eax, XD | PHY_ADDR | PCD | PWT | RW | P
call do_virtual_address_mapping

;;
;; 0x800000-0x800fff 映射到 0FEE00000h 物理地址上, 使用 4k 页面
;;
mov esi, LAPIC_BASE
mov edi, 0FEE0000h
mov eax, XD | PHY_ADDR | PCD | PWT | RW | P
call do_virtual_address_mapping

;;
;; 0xb0000000 开始映射到物理地址 0x1100000 开始, 使用 4K 页面
;;
mov esi, VMX_REGION_VIRTUAL_BASE                ; VMXON region virtual
address
mov edi, VMX_REGION_PHYSICAL_BASE                ; VMXON region physical
address
mov eax, XD | PHY_ADDR | RW | P
call do_virtual_address_mapping

pop ecx
ret

```

BSP 调用 `init_pae_page` 函数负责建立 stage2 阶段基本的虚拟地址映射关系。它首先调用 `map_pae_page_transition_table` 函数映射 8M 的 PDT 区域。接着调用 `do_virtual_address_mapping` 函数或者 `do_virtual_address_mapping_n` 函数逐一映射虚拟地址。

stage2 阶段的基本映射区域如下所示。

- (1) 虚拟地址 7000H 到 9FFFH 映射到相同的物理地址。
- (2) 假如定义了 `GUEST_ENABLE`, 则 `GuestBoot` 与 `GuestKernel` 模块的虚拟地址映射到相同的物理地址。
- (3) `Protected` 模块的虚拟地址映射到相同的物理地址。
- (4) 虚拟地址 B8000 到 B9FFF 映射到相同的物理地址。
- (5) 映射 `SDA` 区域, `SDA.Base` 的虚拟地址映射到 `SDA.PhysicalBase` 的物理地址, 大小为 `SDA.Size`。
- (6) 映射 `SRT` 区域。
- (7) 虚拟地址 400000H 到 400FFFH 映射到物理地址 1000000H 到 1000FFFH。
- (8) 映射 `IOAPIC_BASE` 区域到物理地址 `FEC0000H`。
- (9) 映射 `LAPIC_BASE` 区域到物理地址 `FEE0000H`。
- (10) 映射 `VMX_REGION_VIRTUAL_BASE` 区域到 `VMX_REGION_PHYSICAL_BASE` 物理地址, 但实际上并没使用。

init_ppt_area 与 init_pae_page 函数实现在 lib\page32.asm 文件里。Setup 模块内的其他大部分函数实现在 lib\stage2.asm 文件里。

1.5.3.1 BSP 在 stage2 最后处理

在完成前面所有处理器的通用处理后，接着检查 PCB.IsBsp 标志位，判断处理器是否属于 BSP，是则进行下面的最后初始化处理工作。

代码片段 1-43:

```
%ifndef DBG
;;
;; Stage2 阶段最后工作，检查是否为 BSP
;; 1) 是，等待所有 AP 完成 stage2 工作
;; 2) 否，转入 ApStage2End
;;
cmp BYTE [gs: PCB.IsBsp], 1
jne ApStage2End

call init_sys_service_call

;;
;; 等待所有 AP 第 2 阶段工作完成
;;
call wait_for_ap_stage2_done

;;
;; 将处理器切入到 VMX root 模式
;;
call vmx_operation_enter

%endif

;;
;; 当前处理器拥有焦点
;;
mov eax, [gs: PCB.ProcessorIndex]
mov [fs: SDA.InFocus], eax

;;
;; 更新 SDA.KeyBuffer 记录
;;
mov ebx, [gs: PCB.LsbBase]
mov eax, [ebx + LSB.LocalKeyBufferHead]
mov [fs: SDA.KeyBufferHead], eax
lea eax, [ebx + LSB.LocalKeyBufferPtr]
mov [fs: SDA.KeyBufferPtrPointer], eax
mov eax, [ebx + LSB.LocalKeyBufferSize]
mov [fs: SDA.KeyBufferLength], eax

;;
;; 打开键盘
;;
call enable_8259_keyboard
```

```

    sti
    NMI_ENABLE

    ;;
    ;; 更新系统状态
    ;;
    call update_system_status

;;=====;;
;;                               ;;
;;                               ;;
;;=====;;

    bits 32

    ;;
    ;; 嵌入实验例子代码, 在 ex.asm 文件里实现
    ;;
    %include "ex.asm"

    ... ..

```

BSP 接着调用 `init_sys_service_call` 函数初始化系统服务例程使用环境, 接下来调用 `wait_for_ap_stage2_done` 函数等待所有 APs 完成 stage2 阶段的工作。

所有 APs 完成 stage2 阶段工作后将转入 HLT (停机) 状态, BSP 进行下面的最后初始化。

- (1) 设置 BSP 拥有焦点 (`SDA.InFocus = PCB.ProcessorIndex`)。
- (2) 调用 `vmx_operation_enter` 函数切换到 VMX root-operation 模式。
- (3) 设置 BSP 的 LSB (Local Store Block) 内的 KeyBuffer 管理记录为当前 KeyBuffer 管理记录 (LSB 相关记录赋予 SDA 相关记录)。
- (4) 打开 8259 的 IRQ1, 允许响应键盘中断。
- (5) 开启中断许可。
- (6) 更新系统状态栏信息。
- (7) 转入 `ex.asm` 模块。这是每个例子的具体代码。

注意: `ex.asm` 文件嵌在 `setup.asm` 文件内, 用来完成相关的例子代码。每个例子所在的目录下都存在一个 `ex.asm` 文件, 对应一个实例代码。

1.5.3.2 APs 在 stage2 阶段收尾工作

当判断处理器属于 AP (Application Processor) 时, 处理器转至 `ApStage2End` 处执行收尾工作。

代码片段 1-44:

```

ApStage2End:

#ifdef TRACE
    mov esi, Stage2.Msg
    call puts

```

```
%endif
```

```
;;
;; 增加 ApInitDoneCount 计数
;;
lock inc DWORD [fs: SDA.ApInitDoneCount]

;;
;; 开放第 2 阶段 AP Lock
;;
xor eax, eax
mov ebx, [fs: SDA.Stage2LockPointer]
xchg [ebx], eax

;;
;; 设置 UsableProcessMask 值, 指示 logical processor 处于可用状态
;;
mov eax, [gs: PCB.ProcessorIndex]          ; 处理器 index
lock bts DWORD [fs: SDA.UsableProcessorMask], eax ; 设置 Mask 位

;;
;; 将处理器切到 VMX root operation 模式
;;
call vmx_operation_enter

;;
;; 更新系统状态
;;
call update_system_status

;;
;; 记录处理器的 HLT 状态
;;
mov DWORD [gs: PCB.ActivityState], CPU_STATE_HLT

;;
;; AP 第 2 阶段的最终工作是: 进入 HLT 状态
;;
sti
hlt
jmp $ - 1
```

APs 执行 stage2 阶段的收尾工作如下所示。

- (1) 增加 ApInitDoneCount 计数, 表明 AP 在 stage2 阶段已经完成。
- (2) 开放 stage2 阶段的锁信号, 允许其他 AP 进入 pre-stage2 阶段执行。
- (3) 设置处理器可用状态标志位 (PCB.UsableProcessorMask), 表明该处理器处于可用状态。
- (4) 同样调用 vmx_operation_enter 函数切换到 VMX root-operation 模式。
- (5) 更新 APs 的系统状态栏信息。
- (6) 接下来更新处理器活动状态 (PCB.ActivityState) 为 CPU_STATE_HLT 状态, 执

行 STI 指令开启中断许可，并执行 HLT 指令，将 APs 放入 HLT（停机）状态。

当 APs 进入 HLT 状态后，停止响应任何指令，直到下面的事件唤醒处理器（参见第 4.17.3 节）：

- 外部中断（external-interrupt）
- NMI
- SMI

注意：**这里不能被 INIT 消息唤醒**。因为，此时处理器运行在 VMX root-operation 模式内，将阻塞 INIT 消息。SIPI 消息也不能唤醒（只有在 wait-for-SIPI 状态下才允许响应）。

当处理器被事件唤醒而去处理事件，在处理完毕后执行“**jmp \$-1**”指令继续转入 HLT 状态。

1.5.4 stage3 阶段

使用 Build 命令时，定义__X64 符号将生成为 64 位的系统映像。Boot 在加载模块时，选择加载 long 模块，同样放入 20000H 位置上。在激活 IA-32e 模式之前，我们可以称之为“**pre-stage3**”阶段，直到 IA-32e 模式开启并被激活，系统进入 stage3 阶段运行。stage1 阶段结束转入 pre-stage3 阶段运行。stage3 阶段对应的代码在 common\long.asm 文件里实现。

在 long 模块头同样定义了下面几个标志值。

代码片段 1-45:

```
org LONG_SEGMENT

DD    LONG_LENGTH           ; long 模块长度
DD    BspLongEntry          ; BSP 入口地址
DD    ApStage3Routine        ; AP 入口地址
```

LONG_SEGMENT 存放 long 模块的长度，LONG_SEGMENT + 4 存放 BSP 在 long 模块的入口地址，LONG_SEGMENT + 8 存入 APs 在 long 模块的入口地址。

代码片段 1-46:

```
;;
;; 说明:
;; 1) 此时，处理器处于 stage1 阶段（即未分页保护模式）
;; 2) stage3 阶段将切换到 longmode
;;
bits 32

BspLongEntry:
;;
;; 进入 longmode 前准备工作，初始化 stage3 阶段的基本页表结构
;; 包括:
;; 1) compatibility 模式基本运行区域: LONG_SEGMENT
;; 2) setup 模块运行区域: SETUP_SEGMENT
;; 3) 64-bit 基本运行区域: ffff_ff80_4000_0000h
;; 4) video 区域: b_8000h
```



```

;;      5) SDA 区域映射到: ffff_f800_8002_0000h
;;      6) 映射页表池 PT Pool 和备用 PT Pool 区域
;;      7) LAPIC 与 IOAPIC 基址 (所有 logical processor 地址一致)
;;
call init_longmode_basic_page32

;;
;; 更新 stage3 阶段的 GDT/IDT pointer
;;
call update_stage3_gdt_idt_pointer

ApLongEntry:
;;
;; 映射 stage3 阶段的 PCB 区域
;;
call map_stage3_pcb

;;
;; 更新 stage3 的 kernel stack
;; 1) 需要开启分页前及更新 FS 段前执行
;; 2) 将调整后的 kernel stack 保存在 PCB.KernelStack 内
;;
call update_stage3_kernel_stack

;;
;; 读 GS base 值, 以备下一步更新
;;
mov esi, [gs: PCB.Base]
mov edi, [gs: PCB.Base + 4]

;;
;; 加载 longmode 下的 PXT 表
;;
mov eax, [fs: SDA.PxtPhysicalBase64]
mov cr3, eax

;;
;; 进入 long-mode 前先更新 GS.selector
;;
mov ax, [gs: PCB.GsSelector]
mov gs, ax

;;
;; 下面将切换到 longmode !
;; 说明:
;; 1) longmode_enter() 函数将切换到 64-bit 模式
;; 2) longmode_enter() 返回后处于 64-bit 的执行环境
;; 3) 更新 GS base
;; 4) 需要进一步进行后续的 longmode 环境初始化
;;

;;
;; 设置 EFER 寄存器, 开启 long mode
;;
mov ecx, IA32_EFER
rdmsr
bts eax, 8 ; EFER.LME = 1
wrmsr

;;

```

```

;; 激活 long mode, 进入 compatibility 模式
;;
mov eax, cr0
bts eax, 31
mov cr0, eax                                ; EFER.LMA = 1

;;
;; 转入 64-bit 模式
;;
jmp kernelCsSelector64 : ($ + 7)

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;;;;; 下面是 64-bit 代码 ;;;;;;;;;;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

bits 64

;;
;; 更新 FS/GS 段
;;
mov eax, SDA_BASE
mov edx, 0FFFFFF800h
mov ecx, IA32_FS_BASE
wrmsr
mov eax, esi
mov edx, edi
mov ecx, IA32_GS_BASE
wrmsr

;;
;; 刷新 segment selector 和 cache 部分
;;
mov ax, kernelSsSelector64
mov ds, ax
mov es, ax
mov ss, ax
mov rsp, [gs: PCB.KernelStack]

;;
;; 更新处理器状态
;;
or DWORD [gs: PCB.ProcessorStatus], CPU_STATUS_PG | CPU_STATUS_LONG |
CPU_STATUS_64

;;
;; 刷新 GDTR/IDTR
;;
lgdt [fs: SDA.GdtPointer]
lidt [fs: SDA.IdtPointer]

;;
;; 更新 TSS 环境
;;
call update_stage3_tss

```

```

;;
;; 安装默认中断处理程序
;;
call install_default_interrupt_handler

;;
;; 更新 GS 段信息
;;
call update_stage3_gs_segment

;;
;; 配置 SYSENTER/SYSEXIT 使用环境
;;
call setup_sysenter

... ..

```

BSP 在 pre-stage3 阶段需要负责初始化 IA-32e 模式下的页表结构，并进行基本的映射。也要更新 stage3 阶段最终的 GDT/IDT pointer 值。

处理器在 pre-stage3 与 stage3 阶段执行以下工作：

(1) BSP 调用 `init_longmode_basic_page32` 函数在 pre-stage3 阶段初始化 stage3 阶段的基本页表结构。

(2) BSP 调用 `update_stage3_gdt_idt_pointer` 函数更新 stage3 阶段的 GDT/IDT pointer。

(3) 在下面的第 (3) 到第 (14) 步流程需要所有处理器共同执行，接下来调用 `map_stage3_pcb` 函数映射 stage3 阶段的 PCB 区域。

(4) 加载 IA-32e 模式下的 PML4T 物理基址到 CR3 寄存器。

(5) 开启 IA-32e 模式，并激活。

(6) 转入 64-bit 模式，进入 stage3 阶段。

(7) 更新 stage3 阶段的 FS 与 GS 段基址。

(8) 更新段寄存器及 kernel stack。

(9) 更新处理器状态标志，指示开启了 IA-32e 模式，并进入 64-bit 模式。

(10) 重新加载 GDTR 与 IDTR 寄存器。

(11) 调用 `update_stage3_tss` 函数更新 stage3 阶段的 TSS 环境。

(12) 调用 `install_default_interrupt_handler` 函数安装 stage3 阶段的异常与中断服务例程。

(13) 调用 `update_stage3_gs_segment` 函数更新 stage3 阶段的 PCB 区域。

(14) 配置 SYSENTER/SYSEXIT 使用环境。

(15) 进入 stage3 阶段最后的收尾工作。

在 pre-stage3 阶段，处理器处于未分页保护模式，BSP 需调用 32 位的

init_longmode_basic_page32 函数来初始化 stage3 阶段的页转换表环境。这个函数实现在 lib\stage3.asm 文件里。

1.5.4.1 BSP 在 stage3 阶段的最后工作

stage3 阶段最后检查 PCB.IsBsp 标志位，判断处理器是否属于 BSP。判断是则进行下面的最后处理工作。

代码片段 1-47:

```
%ifndef DBG
;;
;; stage3 阶段最后工作，检查是否为 BSP
;; 1) 是，则等待 AP 完成 stage3 阶段工作（即等待所有 AP 完成切换到 long mode）
;; 2) 否，则转入 ApStage3End
;;
cmp BYTE [gs: PCB.IsBsp], 1
jne ApStage3Next

call init_sys_service_call

;;
;; 等待所有 AP 完成 stage3 阶段工作
;;
call wait_for_ap_stage3_done

;;
;; 将处理器切到 VMX root operation 模式
;;
call vmx_operation_enter

%endif

;;
;; 当前处理器拥有焦点
;;
mov eax, [gs: PCB.ProcessorIndex]
mov [fs: SDA.InFocus], eax

;;
;; 更新 SDA.KeyBuffer 记录
;;
mov rbx, [gs: PCB.LsbBase]
mov rax, [rbx + LSB.LocalKeyBufferHead]
mov [fs: SDA.KeyBufferHead], rax
lea rax, [rbx + LSB.LocalKeyBufferPtr]
mov [fs: SDA.KeyBufferPtrPointer], rax
mov eax, [rbx + LSB.LocalKeyBufferSize]
mov [fs: SDA.KeyBufferLength], eax

;;
;; 打开键盘
;;
call enable_8259_keyboard

sti
NMI_ENABLE
```

```

;;
;; 更新系统状态
;;
call update_system_status

;;=====;;
;;                               所有处理器初始化完成                               ;;
;;=====;;

bits 64

;;
;; 下面是实验例子的源代码
;;
#include "ex.asm"

;;
;; 等待重启
;;
call wait_esc_for_reset
... ..

```

BSP 调用 `init_sys_service_call` 配置 stage3 阶段的系统服务环境，然后调用 `wait_for_ap_stage3_done` 函数等待所有 APs 完成 stage3 阶段的工作。

当 `wait_for_ap_stage3_done` 函数返回时，表明所有 APs 已经完成了 stage3 阶段的最后工作。BSP 接下来完成剩余的处理。

(1) 调用 `vmx_operation_entry` 函数切换到 VMX root-operation 模式。

(2) 设置 BSP 拥有焦点 (`SDA.InFocus = PCB.ProcessorIndex`)。

(3) 设置 BSP 的 LSB (Local Store Block) 内的 KeyBuffer 管理记录为当前 KeyBuffer 管理记录 (SDA 相关的管理记录被初始化为 BSP 对应的 LSB 管理记录)。

(4) 打开 8259 的 IRQ1，接受键盘中断。

(5) 开启中断响应许可。

(6) 更新系统状态栏信息。

(7) 转入执行 `ex.asm` 模块。

实际上，BSP 在 stage2 与 stage3 阶段的最后工作是一样的 (参见第 1.5.3.1 节)。Ex.asm 文件同样被嵌入到 long.asm 文件里。也就是 ex.asm 模块既要能运行在 stage2 阶段，也要能运行在 stage3 阶段。

1.5.4.2 APs 在 stage3 阶段收尾工作

在 long 模块后半部分，当判断处理器属于 AP 时，转到 `ApStage3End` 执行 APs 的收尾工作。

代码片段 1-48:

```

;;
;; 下面是 APs 在 stage3 阶段的最后工作
;; 说明:

```

```

;      1) 增加处理器计数
;;      2) 开放 stage3 lock, 允许其他的 APs 进入执行
;;      3) 将 AP 放入 HLT 状态
;;

        bits 64

ApStage3End:

#ifdef TRACE
    mov esi, Stage3.Msg1
    call puts
#endif

;;
;; 增加完成计数
;;
lock inc DWORD [fs: SDA.ApInitDoneCount]

;;
;; 1) 开放 stage3 锁, 允许其他 AP 进入 stage3
;;
xor eax, eax
mov ebx, [fs: SDA.Stage3LockPointer]
xchg [rbx], eax

;;
;; 设置 UsableProcessMask 值, 指示 logical processor 处于可用状态
;;
mov eax, [gs: PCB.ProcessorIndex]                ; 处理器 index
lock bts DWORD [fs: SDA.UsableProcessorMask], eax ; 设置 Mask 位

;;
;; 进入 VMX operation 模式
;;
call vmx_operation_enter

;;
;; 更新系统状态
;;
call update_system_status

;;
;; 记录处理器的 HLT 状态
;;
mov DWORD [gs: PCB.ActivityState], CPU_STATE_HLT

;;
;; AP 进入 stage3 阶段最终状态: HLT 状态
;;
sti

hlt
jmp $-1

```

APs 执行 stage3 阶段收尾工作如下:

(1) 增加 ApInitDoneCount 计数, 表明 AP 在 stage3 阶段已经完成。

(2) 开放 stage3 阶段的锁信号，允许其他 AP 进入 pre-stage3 阶段执行。

(3) 设置处理器可用状态标志位 (PCB.UsableProcessorMask)，表明该处理器处于可用状态。

(4) 同样调用 vmx_operation_enter 函数切换到 VMX root-operation 模式。

(5) 更新 AP 的系统状态栏信息。

(6) 接下来更新处理器活动状态 (PCB.ActivityState) 为 CPU_STATE_HLT 状态，执行 STI 指令开启中断许可，并执行 HLT 指令将 AP 放入 HLT (停机) 状态。

APs 在 stage3 阶段最后同样转入 HLT 状态 (参见第 1.5.3.2 节)。

1.5.5 例子 1-1

例子 1-1 建立一个空项目测试基础平台

前面已经介绍了系统启动的 boot, stage1, stage2 及 stage3 阶段。现在建立一个空白项目，用来测试系统的运行。

例子的组成部分包括 common\目录下的 boot.asm, setup.asm, protected.asm 及 long.asm 文件，还有 chap01\ex1-1\ex.asm 文件。

ex.asm 是例子的主体代码源文件，在这个空白项目例子里只是输出一条简单信息，告诉我们系统运行在 stage3 阶段还是 stage2 阶段。

下面是 ex.asm 文件的代码。

代码片段 1-49:

```
;;
;; 例子 ex1-1: 这是一个空白项目的示例
;;

mov esi, Ex.Msg1
call puts
mov esi, [fs: SDA.ApLongmode]
add esi, 2
call print_dword_decimal
jmp $

Ex.Msg1      db      'example 1-1: run in stage', 0
```

ex 模块根据 SDA.ApLongmode 值打印出当前的 stage 阶段值。我们使用下面的两条命令进行编译:

- build
- build __X64

第 1 条命令生成 32 位系统映像，第 2 条命令生成 64 位系统映像。在 vmware 里使用软盘映像文件 demo.img 启动后，如图 1-6 所示。



图 1-6

ex 模块输出的信息指示当前运行在 stage2 阶段。现在，我们可以按功能键<Fn>来切换处理器屏幕具体取决于系统上有多少个 CPU。例如，按<F2>键切换到 CPU1，按<F3>键切换到 CPU2 等。这个系统只支持最多 16 个逻辑处理器。

1.6 系统机制

在这一节里，将着重了解这个平台一些特色机制的实现细节，包括分页机制、处理器间通信、处理器切换与调试记录。

1.6.1 分页机制

系统运行在 stage2 阶段使用 PAE 分页模式，运行在 stage3 阶段使用 IA-32e 分页模式。PAE 分页模式的页表结构很大程度上参考了 Windows NT 的设计。而 IA-32e 分页模式在映射时 PDT 与 PT 表采用了动态分配的策略。

1.6.1.1 PAE 分页模式实现

PAE 分页模式下存在 PDPT、PDT 及 PT 三级页表结构，在 inc\page.inc 文件里定义了 PAE 分页模式下的三个页表结构地址值：

- PT_BASE 与 PT_TOP，值分别为 C0000000H 与 C07FFFFFFH，这个 PT 区域共 8MB，是整个 PAE 分页模式下的页表结构区域。
- PT_PHYSICAL_BASE 与 PT_PHYSICAL_TOP，值分别为 200000H 与 9FFFFFFH。定义 PT 区域的物理地址。
- PDT_BASE 与 PDT_TOP，值分别为 C0600000H 与 C0603FFFFH，共 16KB。PDT 区域内嵌在 PT 区域内。
- PDT_PHYSICAL_BASE 与 PDT_PHYSICAL_TOP，值分别为 800000H 与 803FFFFH。定义 PDT 区域的物理地址。

在 Windows NT 设计里，PDPT_BASE 与 PDPT_TOP 值分别为 C0603000H 与 C060301FH，也嵌在 PDT 区域内。但是在本书平台里，将这个 32 字节的 PDPT 区域移到了 SDA.Ppt 区域内，并没有使用内嵌的 PDPT 区域。

SDA.Ppt 存放在 4 个 PDPTE 表项，在初始化时分别指向 4 个 PDT 区域，如图 1-7 所示。

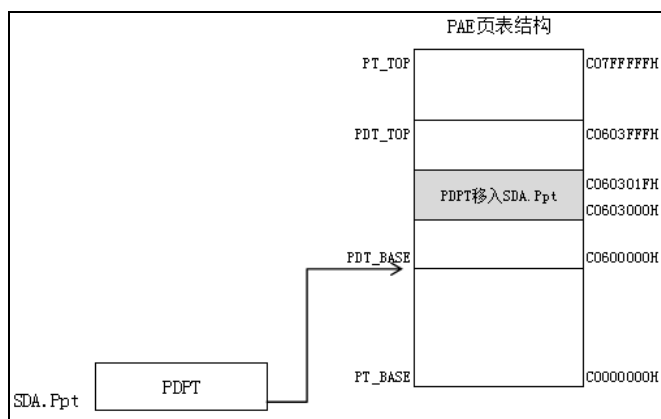


图 1-7

整个 PT 区域的物理地址区域是从 200000H 到 9FFFFFFH（共 8M），对应于虚拟地址 C0000000H 到 C07FFFFFFH。

初始化页表结构

在 pre-stage2 阶段（protected 模块头部，参见 1.5.3 节代码片段 1-41）调用 init_ppt_area 函数初始化 PDPT 区域。

代码片段 1-50:

```

;-----
; init_ppt_area()
; input:
;     none
; output:
;     none
; 描述:
;     1) 设置 stage2 阶段 PAE-paging 模式下的 PPT 表区域
;-----
init_ppt_area:
    push ebx
    push edx
    push ecx

    xor edx, edx
    xor ecx, ecx

    ;;
    ;; 在 PPT 表里写入 PDT 表基址
    ;;
    mov ebx, [fs: SDA.PptPhysicalBase]
    mov eax, [fs: SDA.PdtPhysicalBase]
    or eax, P

init_ppt_area.loop:
    mov [ebx + ecx * 8], eax

```

```

mov [ebx + ecx * 8 + 4], edx
inc ecx
add eax, 1000h
cmp ecx, 4
jb init_ppt_area.loop

pop ecx
pop edx
pop ebx
ret

```

init_ppt_area 函数所做的工作是：将 4 个 PDT 区域的物理地址写入 SDA.Ppt 区域内。形成下面 4 个 PDPTE 的设置：

- (1) PDPT[0] (地址为 SDA.Ppt) 写入值 800000H。
- (2) PDPT[1] (地址为 SDA.Ppt + 8) 写入值 801000H。
- (3) PDPT[2] (地址为 SDA.Ppt + 16) 写入值 802000H。
- (4) PDPT[3] (地址为 SDA.Ppt + 24) 写入值 803000H。

在 pre-stage2 阶段也调用 init_pae_page 函数建立基本的页映射关系（参见第 1.5.3 节代码片段 1-42）。init_pae_page 函数内部调用 map_pae_page_transition_table 函数初始化页表结构区域，并建立 PT_BASE 到 PT_TOP 区域（C0000000H 到 C07FFFFFFH）的映射关系。

代码片段 1-51:

```

;-----
; map_pae_page_transition_table()
; input:
;     esi - 页转换表虚拟地址
; output:
;     none
; 描述:
;     1) 映射页转换表结构区域
;     2) 使用 4K 页面
;-----
map_pae_page_transition_table:
    push ecx
    push ebx
    push edx

    ;;
    ;; 向 PDT_BASE - PDT_TOP 区域写入
    ;; 1) 物理地址区域 800000h - 803ffffh
    ;; 2) 值 200000h - 900000h
    ;;
    mov ebx, [fs: SDA.PdtPhysicalBase]
    mov eax, [fs: SDA.PtPhysicalBase]
    xor ecx, ecx
    mov esi, RW | P
    mov edx, US | RW | P
map_pae_page_transition_table.loop:
    and eax, 0FFFFFF00h
    or eax, edx
    mov [ebx + ecx * 8], eax

```

```

mov DWORD [ebx + ecx * 8 + 4], 0
inc ecx

;;
;; 在 8000_0000h - ffff_ffffh 区域具有 supervisor 权限
;;
cmp ecx, 2000h / 8
cmovae edx, esi
add eax, 1000h
cmp ecx, 4000h / 8
jb map_pae_page_transition_table.loop

;;
;; 修改 c000_0000 - c07f_ffffh 区域具有 XD 权限
;;
mov esi, [fs: SDA.PtBase]
call get_pde_physical_address
mov ebx, eax
mov ecx, 3
mov edx, [fs: SDA.XdValue]
map_pae_page_transition_table.loop2:
mov [ebx + ecx * 8 + 4], edx
dec ecx
jge map_pae_page_transition_table.loop2

pop edx
pop ebx
pop ecx
ret

```

map_pae_page_transition_table 函数的工作是：向 PDT 区域以物理地址方式写入 PT 区域的物理地址值（PT_PHYSICAL_BASE）。整个 PDT 区域共有 4×512 个表项需要填写。

在填写 PDT 表项时，也需要设置相应的访问权限。虚拟地址空间从 80000000H 到 FFFFFFFFH 需要 supervisor 访问权限。同时，C0000000 到 C07FFFFFFH 区域也设置 XD (Execute Disable, 不可执行) 权限。

实现虚拟地址映射

do_virtual_address_mapping32 函数（被重定义为 do_virtual_address_mapping）是虚拟地址映射到物理地址的实现函数，在 lib\page32.asm 文件里实现。

代码片段 1-52:

```

;-----
; do_virtual_address_mapping32(): 执行虚拟地址映射
; input:
;     esi - virtual address
;     edi - physical address
;     eax - attribute
; output:
;     0 - successful, 否则返回错误码
;
; description:

```

```

;     eax 传递过来的 attribute 由下面标志位组成:
;     [0] - P
;     [1] - R/W
;     [2] - U/S
;     [3] - PWT
;     [4] - PCD
;     [5] - A
;     [6] - D
;     [7] - PS
;     [8] - G
;     [12] - PAT
;     [28] - IGNORE
;     [29] - FORCE, 置位时, 强制进行映射
;     [30] - PHYSICAL, 置位时, 表示基于物理地址进行映射 (用于初始化时)
;     [31] - XD
;-----
do_virtual_address_mapping32:
    push ecx
    push ebx
    push edx
    push ebp

    ;;
    ;; 保证地址对齐在 4K 边界
    ;;
    and esi, 0FFFFFF00h
    and edi, 0FFFFFF00h

    push esi
    push edi

    ;;
    ;; PT_BASE - PT_TOP 区域已经初始化映射, 不能再进行映射
    ;; 假如映射到 PT_BASE - PT_TOP 区域内就失败返回
    ;;
    cmp esi, [fs: SDA.PtBase]
    jb do_virtual_address_mapping32.next
    cmp esi, [fs: SDA.PtTop]
    ja do_virtual_address_mapping32.next

    mov eax, MAPPING_ADDRESS_INVALID
    jmp do_virtual_address_mapping32.done

do_virtual_address_mapping32.next:
    ;;
    ;; 检查是否基于物理地址
    ;; ebx 存放 get_pde_XXX 函数
    ;; ebp 存入 get_pte_XXX 函数
    ;;
    test eax, PHY_ADDR                                ; physical address 标志位
    mov ebx, get_pde_virtual_address
    mov ecx, get_pde_physical_address
    cmovnz ebx, ecx
    mov ebp, get_pte_virtual_address
    mov ecx, get_pte_physical_address
    cmovnz ebp, ecx

    ;;
    ;; 获得输入的 XD 标志位

```

```

;; 最后, 合成 XD 标志, 取决于是否开启 XD 功能
;;
mov edx, eax
and edx, [fs: SDA.XdValue]          ; 是否开启 XD 功能
mov ecx, eax

;;
;; 读取 PDE 项地址
;;
call ebx
mov ebx, eax

test ecx, PS                        ; PS == 1 ? 使用 2M 页映射
jz do_virtual_address_mapping32.4k

;;
;; 下面使用 2M 页进行映射
;;
and edi, 0FFE0000h                  ; 调整到 2M 页物理地址
or di, cx                           ; page frame 属性
and edi, 0FFE011FFh                 ; 清 PDE 的 [11:9] 和 [20:13] 位

mov eax, [ebx]
test eax, P                          ; 检查是否已经在使用中
jz do_virtual_address_mapping32.2m.next ; 没有使用的话, 直接设置
test ecx, FORCE                       ; 检查 FORCE_PDE 位
jnz do_virtual_address_mapping32.2m.next

;;
;; 如果已经在使用, 检测内容是否一致, 不一致则返回错误码: MAPPING_USED
;;
xor eax, edi
test eax, ~(60h)                     ; 忽略 D 和 A 标志位
mov eax, MAPPING_USED
jnz do_virtual_address_mapping32.done
mov eax, [ebx + 4]                    ; PDE 高半部分
xor eax, edx                          ; 检查 XD 标志位
mov eax, MAPPING_USED
js do_virtual_address_mapping32.done  ; XD 标志位不符

do_virtual_address_mapping32.2m.next:
mov [ebx + 4], edx                    ; PDE 高 32 位
mov [ebx], edi                        ; PDE 低 32 位
mov eax, MAPPING_SUCCESS
jmp do_virtual_address_mapping32.done

;;
;; 下面使用 4K 页映射
;;
do_virtual_address_mapping32.4k:
btr ecx, 12                          ; 取 PAT 标志位
setc dl
shl dl, 7
or cl, dl                             ; 合成 PTE 的 PAT 标志位
xor dl, dl

;;
;; PDE.P = 0 时, 或者 FORCE = 1 时, 直接设置, 否则需要检查 PDE 内容
;;
mov eax, [ebx]

```

```

test eax, P
jz do_virtual_address_mapping32.4k.next
test ecx, FORCE
jnz do_virtual_address_mapping32.4k.next

;;
;; 检查 PDE 内容
;; 1) 如果 PDE.PS = 1, 返回出错码: MAPPING_PS_MISMATCH
;; 2) 如果 PDE.R/W = 0, 而输入的 R/W = 1, 返回出错码: MAPPING_RW_MISMATCH
;; 3) 如果 PDE.U/S = 0, 而输入的 U/S = 1, 返回出错码: MAPPING_US_MISMATCH
;; 4) 如果 PDE.XD = 1, 而输入的 XD = 0, 返回出错码: MAPPING_XD_MISMATCH
;;
test eax, PS
jz do_virtual_address_mapping32.4k.l1
mov eax, MAPPING_PS_MISMATCH
jmp do_virtual_address_mapping32.done
do_virtual_address_mapping32.4k.l1:
test eax, RW
jnz do_virtual_address_mapping32.4k.l2 ;如果 PDE.R/W = 1, 忽略
test ecx, RW
jz do_virtual_address_mapping32.4k.l2 ; 如果输入 PDE.R/W = 0, 则忽略
mov eax, MAPPING_RW_MISMATCH ; PDE.R/W = 0, 但 page frame 的 R/W =
1, 出错返回
jmp do_virtual_address_mapping32.done
do_virtual_address_mapping32.4k.l2:
test eax, US
jnz do_virtual_address_mapping32.4k.l3 ; 如果 PDE.U/S = 1, 则忽略
test ecx, US
jz do_virtual_address_mapping32.4k.l3 ; 如果输入 PDE.U/S = 0, 则忽略
mov eax, MAPPING_US_MISMATCH ; PDE.U/S = 0, 但 page frmae 的 U/S =
1, 出错返回
jmp do_virtual_address_mapping32.done
do_virtual_address_mapping32.4k.l3:
mov eax, [ebx + 4]
test eax, XD
jz do_virtual_address_mapping32.4k.next ; 如果 PDE.XD = 0, 则忽略
test edx, XD
jnz do_virtual_address_mapping32.4k.pte ; 如果输入 PDE.XD = 1, 则忽略
mov eax, MAPPING_XD_MISMATCH
jmp do_virtual_address_mapping32.done ; PDE.XD = 1, 但 page frame 的 XD =
0, 出错返回

do_virtual_address_mapping32.4k.next:
;;
;; 得到 PT 表物理地址
;;
call get_pte_physical_address
and eax, 0FFFFFF00h ; 调整到 4K 页物理地址
or al, cl
and eax, 0FFFFFF07h ; 清 PTE 的 [11:3] 位
; PCD 和 PWT 位不设置!
;;
;; 检查通过设置 PDE
mov DWORD [ebx + 4], 0 ; 清 PDE 高 32 位
mov [ebx], eax ; PDE 低 32 位

do_virtual_address_mapping32.4k.pte:
;;
;; 设置 PTE

```

```

    call ebp                                ; 得到 PTE 地址
    mov [eax + 4], edx
    or di, cx
    and edi, 0FFFFFFFh                    ; 清 PTE 的 [11:9] 位
    mov [eax], edi
    mov eax, MAPPING_SUCCESS
do_virtual_address_mapping32.done:
    pop edi
    pop esi
    pop ebp
    pop edx
    pop ebx
    pop ecx
    ret

```

在虚拟地址映射时使用页表 walk 方式。PDPT 已经设置好，只需从 PDT 开始 walk 直到 PT。当检查到表项属于 not-present 时进行写入相应的表项处理。

在后续代码里，我们可以使用 do_virtual_address_mapping 函数动态地映射一个新的虚拟地址。这个函数原型看起来如下：

```

STATUS_CODE
do_virtual_address_mapping(
    UINT    VirtualAddress,
    UINT    PhysicalAddress,
    UINT    PageAttribute
);

```

EAX 寄存器传递 PageAttribute（页属性）参数，包括页表项所支持的标志位（如 P、U/S 等）、FORCE、PHYSICAL、XD 等。

使用 FORCE 参数，则进行强制映射，无论虚拟地址是否已经被映射。使用 PHYSICAL 参数，则以物理地址方式设置页表结构（主要是在 pre-stage2 阶段使用）。使用 XD 标志，则为页表项加上 XD 访问权限。

1.6.1.2 IA-32e 分页模式实现

在 IA-32e 分页模式下，PML4T 与 PDPT 区域拥有固定区域，PDT 与 PT 表需要从 PT pool 里动态分配。inc\page.inc 头文件里定义了下面的值。

- PPT_BASE64 与 PPT_TOP64，值分别为 FFFFF6FB_7DA00000H 与 FFFFF6FB_7DBFFFFFFH，定义 PDPT 区域，共 2MB 大小。
- PPT_PHYSICAL_BASE64 与 PPT_PHYSICAL_TOP64，值分别为 2000000H 与 21FFFFFFH，定义 PDPT 区域的物理地址。
- PXT_BASE64 与 PXT_TOP64，值分别为 FFFFF6FB_7DBED000H 与 FFFFF6FB_7DBEDFFFFH，定义 PML4T 区域，共 4KB 大小。
- PXT_PHYSICAL_BASE64 与 PXT_PHYSICAL_TOP64，值分别为 21ED000H 与 21EDFFFFH，定义 PML4T 区域物理地址。

PDT 与 PT 在 PT pool（或备用 PT pool，参见表 1-6）里分配，如图 1-8 所示。

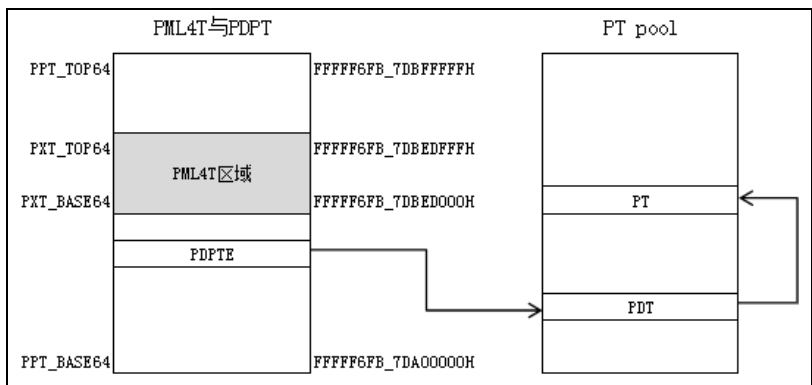


图 1-8

4K 大小的 PML4T 区域嵌入在 PDPT 区域内，其中部分 PML4TE 与 PDPTE 重叠在一起。

初始化页表结构

在 pre-stage3 阶段，BSP 调用 `init_longmode_basic_page32` 函数初始化页表结构，并建立基本的虚拟地址映射关系。该函数实现在 `lib\stage3.asm` 文件里。

`init_longmode_basic_page32` 函数首先调用 `map_longmode_page_transition_table32` 函数映射 PDPT 区域，也包括 PML4T 区域。如下代码所示。

代码片段 1-53:

```

;-----
; map_longmode_page_transition_table32()
; input:
;     none
; output:
;     none
; 描述:
;     1) 此函数用来映射 PXT, PPT, PDT 区域
;     2) init_longmode_page() 执行其他映射前调用
;     3) 此函数使用在 legacy 模式下
;-----
map_longmode_page_transition_table32:
    push ecx
    push ebx
    push edx

    ;;
    ;; 清 2M 的 PPT 表物理区域
    ;;
    call clear_2m_for_longmode_ppt

    ;;
    ;; 需要写入的 PPT 物理地址值 (200_0000h)
    ;;
    mov eax, [fs: SDA.PptPhysicalBase64]
    mov edx, [fs: SDA.PptPhysicalBase64 + 4]
    and eax, ~0FFFh
    and edx, [gs: PCB.MaxPhyAddrSelectMask + 4]

```



```

;;
;; 4K 的 PXT 表物理区域 (21ed000h - 21edfffh)
;;
mov ebx, [fs: SDA.PxtPhysicalBase64]

;;
;; 从高往下写入 21ff000h - 2000000h
;;
add eax, (200000h - 1000h)          ; 起始写入 21ff000h 值
mov ecx, (1000h - 8)                ; 从 21edfff8h 开始写
or eax, VALID_FLAGS | RW            ; Supervisor, Read/Write, Present,
PTE valid
mov esi, VALID_FLAGS | US | RW      ; User, Read/Write, Present, PTE
valid

map_longmode_page_transition_table32.loop:
    cmp ecx, 800h
    jae map_longmode_page_transition_table32.@1
    ;;
    ;; 21ed000h - 21ed7f8 之间写入 User 权限
    ;;
    or eax, esi

    ;;
    ;; 21ed800h - 21edff8 之间写入 Supervisor 权限
    ;;
map_longmode_page_transition_table32.@1:
    mov [ebx + ecx], eax
    mov [ebx + ecx + 4], edx
    sub eax, 1000h
    sub ecx, 8
    jns map_longmode_page_transition_table32.loop

    ;;
    ;; 更新 PPT 表区域管理标志为有效
    ;;
    mov BYTE [fs: SDA.PptValid], 1

    pop edx
    pop ebx
    pop ecx
    ret

```

map_longmode_page_transition_table32 函数的主要工作是：以物理地址方式向 PML4T 区域 (21ED000H) 写入 PDPT 基址值 (2000000H 开始)。形成下面的设置：

PML4T[0] (地址 21ED000H) 写入值 2000000H。

PML4T[1] (地址 21ED008H) 写入值 2001000H。

如此类推

.....

PML4T[510] (地址 21EDFF0H) 写入值 21FE000H。

PML4T[511] (地址 21EDFF8H) 写入值 21FF000H。

接着设置的访问权限是：PML4T[255:0]具有 user 访问权限，而 PML4T[511:256]需要有 supervisor 访问权限。

由于 `init_longmode_basic_page32` 函数运行在 pre-stage3 阶段（32 位未分页保护模式），它需要调用 `do_prev_stage3_virtual_address_mapping32` 函数进行 64 位的虚拟地址映射。EDI:ESI 传递 64 位的虚拟地址值，EDX:EAX 传递 64 位的物理地址值，ECX 传递 PageAttribute 参数。

`do_prev_stage3_virtual_address_mapping32` 函数实现在 `lib\page64.asm` 文件里，以 32 位进行编译。它的原型看起来像下面的声明：

```
STATUS_CODE
do_prev_stage3_virtual_address_mapping32(
    UINT64    VirtualAddress,
    UINT64    PhysicalAddress,
    UINT      PageAttribute
);
```

一旦完成页表初始化以及 stage3 阶段的基本映射，转入 64-bit 模式后，在后续的虚拟地址映射工作中系统使用 `do_virtual_address_mapping64`（重定义为 `do_virtual_address_mapping`）函数完成。

实现虚拟地址映射

`do_virtual_address_mapping64` 函数实现在 `lib\page64.asm` 文件里，以 64 位进行编译。函数比较长，这里不便列出，读者自行查阅。

`do_virtual_address_mapping64` 函数基本的工作原理如以下伪代码所示。

```
STATUS_CODE
do_virtual_address_mapping64(
    UINT64 VirtualAddress,
    UINT64 PhysicalAddress,
    UINT    PageAttribute
)
{
    /*
     * 从 PDPT 开始 walk
     */
    TableEntry = GetTableEntry(VirtualAddress);           // 读取相应的表项

    if (IsValid(TableEntry))                               // 检查页表项是否有效
    {
        /*
         * 有效：继续向下 walk，直到 PTE
         */
        ... ..

    }
    else
    {
        /*
         * 无效：
         * 1) 从 PT pool 里分配 4K 页面写入 TableEntry
         * 2) 将物理地址调整到虚拟地址，继续向下 walk，直至 PTE
         */
    }
}
```

```

        */
        ... ..
    }
    ... ..
}

```

`do_virtual_address_mapping64` 函数的 `PageAttribute` 参数新增了 `GET_PHY_PAGE_FRAME` 参数，用来返回虚拟地址对应页面的物理地址。

```

mov esi, 400000h
mov r8d, GET_PHY_PAGE_FRAME
call do_virtual_address_mapping64

```

函数将返回虚拟地址 400000h 所在物理页面的基址。

1.6.2 多处理器机制

第 1.5 节所述的系统启动完毕后，多处理器环境被开启，BSP 处于工作状态，而所有 APs 处于 HLT 状态，等待 BSP 分派任务执行。

1.6.2.1 调度任务

所有处理器都可以通过 IPI (Inter-processor interrupt, 处理器间中断) 消息形式调度目标处理器执行某个任务，也可以广播 IPI 给一组处理器共同执行某个程序。

在 `lib\smp.asm` 文件里实现了目标处理器执行任务的三个模式。

(1) `GOTO_ENTRY` 模式：让目标处理器跳转到某个地址执行，无须返回。

(2) `DISPATCH_TO_ROUTINE` 模式：让目标处理器执行提供的某个 routine (例程)，需要返回。

(3) `FORCE_DISPATCH_TO_ROUTINE` 模式：以 NMI 方式发送 IPI 给目标处理器，执行提供的某个 routine，需要返回。

这三个模式分别对应的调度函数是 `goto_processor`、`dispatch_to_processor` 以及 `force_dispatch_to_processor`。它们所使用的参数是一致的，原型看起来如下所示：

```

STATUS_CODE
dispatch_to_processor (
    UINT    ProcessorIndex,
    PVOID    Routine
);

```

它们实现在 `lib\smp.asm` 文件里，如下面代码所示。

代码片段 1-54:

```

;;
;; 定义 IPI 执行方式
;; 1) GOTO_ENTRY:                让处理器跳转到入口点
;; 2) DISPATCH_TO_ROUTINE:      让处理器执行一段 routine
;; 3) FORCE_DISPATCH_TO_ROUTINE: 让处理器执行 NMI handler

```

```

;;
#define GOTO_ENTRY                                FIXED_DELIVERY | IPI_ENTRY_VECTOR
#define DISPATCH_TO_ROUTINE                      FIXED_DELIVERY | IPI_VECTOR
#define FORCE_DISPATCH_TO_ROUTINE                 NMI_DELIVERY | 02h

;-----
; force_dispatch_to_processor()
; input:
;     esi - 处理器 index
;     edi - routine 入口
; output:
;     eax - status code
; 描述:
;     1) 使用 NMI delivery 方式发送 IPI
;     2) 将忽略目标处理器的 eflags.IF 标志位
;-----
force_dispatch_to_processor:
    mov eax, FORCE_DISPATCH_TO_ROUTINE
    jmp dispatch_to_processor.Entry

;-----
; goto_processor()
; input:
;     esi - 处理器 Index 号
;     edi - entry
; output:
;     eax - status code
; 描述:
;     1) 转到目标处理器的入口点执行
;     2) 输入参数 esi 提供目标处理器的 index 值 (从 0 开始)
;     3) 输入参数 edi 提供目标代码入口地址
;     4) 这个函数无须等待直接返回
;     5) 可以给自己调度执行!
;-----
goto_processor:
    mov eax, GOTO_ENTRY
    jmp dispatch_to_processor.Entry

;-----
; dispatch_to_processor()
; input:
;     esi - 处理器 Index 号
;     edi - routine 入口
; output:
;     eax - status code
; 描述:
;     1) 将一段 routine 调度到某个处理器执行
;     2) 输入参数 esi 提供目标处理器的 index 值 (从 0 开始)
;     3) 输入参数 edi 提供目标代码入口地址
;     4) 这个函数无须等待直接返回
;     5) 不能自己给自己调度任务执行!
;-----

```

```

dispatch_to_processor:
    mov eax, DISPATCH_TO_ROUTINE

dispatch_to_processor.Entry:
    push ebp
    push ebx
    push ecx
    push edx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    mov ecx, esi                                ; 处理器 index 值
    mov edx, eax

    ;;
    ;; 得到目标处理器的 PCB 块
    ;;
    call get_processor_pcb
    REX.Wrb
    mov ebx, eax

    ;;
    ;; 检查是否出错
    ;;
    mov eax, STATUS_PROCESSOR_INDEX_EXCEED
    REX.Wrb
    test ebx, ebx
    jz dispatch_to_processor.done

    ;;
    ;; 置处理器为 busy 状态 (去掉 usable processor 列表)
    ;;
    mov eax, SDA.UsableProcessorMask
    lock btr DWORD [fs: eax], ecx                ; 处理器为 unusable 状态

    cmp edx, FORCE_DISPATCH_TO_ROUTINE
    jne dispatch_to_processor.@0
    ;;
    ;; 如果是使用 NMI delivery 方式
    ;;
    REX.Wrb
    mov ebp, [ebp + PCB.SdaBase]                ; sda base
    REX.Wrb
    mov [ebp + SDA.NmiIpiRoutine], edi          ; 写入 SDA.NmiIpiRoutine
    lock bts DWORD [ebp + SDA.NmiIpiRequestMask], ecx ; 设置 NMI IPI routine
Mask 位
    jmp dispatch_to_processor.@1

dispatch_to_processor.@0:
    ;;
    ;; 写入 routine 入口地址到 PCB 中
    ;;
    REX.Wrb
    mov [ebx + PCB.IpiRoutinePointer], edi

```

```

dispatch_to_processor.@1:

    ;;
    ;; 使用物理 ID 方式，发送 IPI 到目标处理器
    ;;
    mov esi, [ebx + PCB.ApicId]                ; 处理器 ID 值
    SEND_IPI_TO_PROCESSOR esi, edx

    mov eax, STATUS_SUCCESS

dispatch_to_processor.done:
    pop edx
    pop ecx
    pop ebx
    pop ebp
    ret

```

goto_processor, dispatch_to_processor 以及 force_dispatch_to_processor 函数传入不同的模式参数到 EAX 寄存器，然后跳转到真正的入口 dispatch_to_processor.Entry 执行调度任务，进行下面的处理。

- (1) 调用 get_processor_pcb 函数根据 ProcessorIndex 值取得目标处理器的 PCB 块。
- (2) 将目标处理器设置为不可用状态（清 UsableProcessorMask 相应位）。
- (3) 检查调度模式是否属于 FORCE_DISPATCH_TO_ROUTINE。
 - 是，则将任务 routine 的入口地址写入 SDA.NmiIpiRoutine，并且设置 SDA.NmiIpiRequestMask 相应位，指示以 IPI 方式处理 NMI。继续下一步。
 - 否，则将任务 routine 的入口地址写入 PCB.IpiRoutinePointer。
- (4) 读取目标处理器 APIC ID 值，使用 SEND_IPI_TO_PROCESSOR 宏发送 IPI 消息。

注意：dispatch_to_processor.Entry 处理完毕后立即返回给发送 IPI 者，无须等待。lib\smp.asm 文件里也实现了一个等待版本的 dispatch_to_processor_with_waiting 函数，这个函数用来同步调度者与被调度者。需要等待目标处理器的任务执行完毕后才返回。

代码片段 1-55:

```

;-----
; dipatch_to_processor_with_waitting()
; input:
;     esi - 处理器 Index 号
;     edi - routine 入口
; output:
;     eax - status code
; 描述:
;     1) 将一段 routine 调度到某个处理器执行
;     2) 输入参数 esi 提供目标处理器的 index 值（从 0 开始）
;     3) 输入参数 edi 提供目标代码入口地址
;     4) 这个函数等 dispatch routine 完成后返回
;-----
dispatch_to_processor_with_waiting:
    ;;
    ;; 内部信号无效

```

```

;;
RELEASE_INTERNAL_SIGNAL

;;
;; 调度到处理器
;;
call dispatch_to_processor

;;
;; 等待信号有效，等待目标处理器执行完毕
;;
WAIT_FOR_INTERNAL_SIGNAL
ret

```

dispatch_to_processor_with_waitting 函数使用内部的信号控制同步，在调用 dispatch_to_processor 函数时使用 RELEASE_INTERNAL_SIGNAL 宏置无效信号，然后使用 WAIT_FOR_INTERNAL_SIGNAL 宏等待目标处理器任务执行完毕。

IPI routine

三种调度模式使用不同的中断向量号，目标处理器的 IPI routine 也不同。

(1) FORCE_DISPATCH_TO_ROUTINE 模式：使用 NMI 进行 deliver，中断向量号为 2。而 IPI routine 使用 NMI handler（实现在 lib\service.asm 与 lib\service64.asm 文件里）。

(2) DISPATCH_TO_ROUTINE 模式：中断向量号定义为 IPI_VECTOR（值为 E0H），IPI routine 使用 dispatch_routine 例程。

(3) GOTO_ENTRY 模式：中断向量号定义为 IPI_ENTRY_VECTOR（值为 E1H），IPI routine 使用 goto_entry 例程。

在 stage1 及 stage3 初始化阶段，系统会安装 IPI routine handler，如下代码所示。

代码片段 1-56:

```

;;
;; 安装 IPI 服务例程
;;
mov esi, IPI_VECTOR
mov rdi, dispatch_routine64
call install_kernel_interrupt_handler64

mov esi, IPI_ENTRY_VECTOR
mov rdi, goto_entry64
call install_kernel_interrupt_handler64

```

目标处理器收到 IPI 后，执行 IPI 例程（NMI handler，dispatch_routine 或者 goto_entry），然后转入执行目标任务。

代码片段 1-57:

```

;-----
; dispatch_routine()
; input:
;     none
; output:
;     none

```

```

; 描述:
; 1) 处理器的 IPI 服务例程
; -----
dispatch_routine:
    ;;
    ;; 保存被中断者 context
    ;;
    pusha

    ;;
    ;; 构造 BottomHalf 例程 0 级中断栈
    ;;
    push 02 | FLAGS_IF                ; eflags, 可中断
    push kernelCsSelector32          ; cs
    push dispatch_routine.BottomHalf ; eip

    ;;
    ;; 读 routine 入口地址
    ;;
    mov ebx, [gs: PCB.IpiRoutinePointer]

    ;;
    ;; IPI routine 返回, 目标任务由 BottomHalf 处理
    ;;
    LAPIC_EOI_COMMAND                ; 发送 EOI 命令
    iret                             ; 转入执行 BottomHalf 例程

; -----
; dispatch_routine.BottomHalf
; 描述:
; dispatch_routine 的下半部分处理
; -----
dispatch_routine.BottomHalf:
    ;;
    ;; 当前栈中数据:
    ;; 1) 8 个 GPR
    ;; 2) 被中断者的返回参数
    ;;

%define RETURN_EIP_OFFSET    (8 * 4)

    mov ebp, esp

    ;;
    ;; 将中断栈结构调整为 far pointer
    ;;
    mov eax, [ebp + RETURN_EIP_OFFSET] ; 读 eip
    mov esi, [ebp + RETURN_EIP_OFFSET + 4] ; 读 cs
    mov ecx, [ebp + RETURN_EIP_OFFSET + 8] ; 读 eflags
    mov [ebp + RETURN_EIP_OFFSET + 8], esi ; cs 写入原 eflags 位置
    mov [ebp + RETURN_EIP_OFFSET + 4], eax ; eip 写入原 cs 位置
    mov [ebp + RETURN_EIP_OFFSET], ecx    ; eflags 写入原 eip 位置

    ;;
    ;; 执行目标任务
    ;;
    test ebx, ebx

```



```

    jz dispatch_routine.BottomHalf.@1
    call ebx

    ;;
    ;; 写入状态值
    ;;
    mov [fs: SDA.LastStatusCode], eax

    ;;
    ;; 如果提供了 IPI routine 下半部分处理，则执行
    ;;
    mov eax, [gs: PCB.IpiRoutineBottomHalf]
    test eax, eax
    jz dispatch_routine.BottomHalf.@1
    call eax

dispatch_routine.BottomHalf.@1:
    ;;
    ;; 目标处理器已完成工作，置为 usable 状态
    ;;
    mov eax, [gs: PCB.ProcessorIndex]
    lock bts DWORD [fs: SDA.UsableProcessorMask], eax

    ;;
    ;; 置内部信号有效
    ;;
    SET_INTERNAL_SIGNAL

%undef RETURN_EIP_OFFSET

    ;;
    ;; 恢复 context 返回被中断者
    ;;
    popa                                ; 被中断者 context
    popf                                ; eflags
    retf                                ; 返回被中断者

```

由于 IPI routine 以中断方式响应，在执行期间它会关闭外部中断的响应。为了避免目标任务长期占有中断响应时间，dispatch_routine 例程实行分段处理。目标任务一律放在 BottomHalf（下半部）处理，上半部立即返回（允许中断响应）。

dispatch_routine 例程上半部构造一个 BottomHalf 中断栈，将 dispatch_routine.BottomHalf 入口地址主动压入当前栈中，然后执行中断返回。BottomHalf 执行完目标任务后，将目标处理器置为 usable（可用）状态，置内部信号有效。

代码片段 1-58:

```

;-----
; goto_entry()
; input:
;     none
; output:
;     none
; 描述:
;     1) 强制让处理器跳到入口点代码
;     2) 注意，这种方式不能返回
;-----

```

```

goto_entry:
    push eax
    push ebp
    mov ebp, esp

    add esp, 12                                ; 指向 CS

    ;;
    ;; 检查被中断者权限
    ;;
    test DWORD [esp], 03                      ; 检查 CS
    jz goto_entry.@0

    ;;
    ;; 属于非 0 级, 改写为 0 级中断栈
    ;;
    add esp, 16                                ; 指向未压入返回参数前
    push 02 | FLAGS_IF                       ; 压入 EFLAGS
    push kernelCsSelector32                  ; 压入 CS

goto_entry.@0:
    ;;
    ;; 写入目标地址
    ;;
    push DWORD [gs: PCB.IpiRoutinePointer]    ; 原返回地址<--- 目标地址

    ;;
    ;; 写 LAPIC EOI 命令
    ;;
    mov eax, [gs: PCB.LapicBase]
    mov DWORD [eax + EOI], 0
    mov eax, [ebp + 4]
    mov ebp, [ebp]
    iret                                     ; 转入目标地址

```

goto_entry 例程强制目标处理器跳转到目标地址执行, 实现原理是改写中断的返回地址。将目标任务地址覆盖原返回地址值, 当前被中断者如果为非 0 级, 则改为 0 级的中断栈。执行 IRET 指令后, 处理器便跳转到目标地址继续执行。

1.6.2.2 处理器切换

系统实现了处理器间的切换机制。我们可以按下功能键切换到目标处理器的屏幕, 便于观察目标处理器的输出信息。例如, 按<F1>键切换到 CPU0, <F2>键切换到 CPU1, <F3>键切换到 CPU2, 以此类推。

以按下<F4>键为例, 当按下<F4>键时, CPU0 响应键盘中断执行 8259 IRQ1 中断服务例程 keyboard_8259_handler。这个中断服务例程读取键盘扫描, 判断是否为按下功能键<F1>到<F10>。如果按下的键属于功能键, 则调用 force_dispatch_to_processor 函数向目标处理器发送 NMI IPI 消息, 并且提供的目标任务为 switch_to_processor 例程。

以 64 位版本为例, 目标处理器接收到 NMI IPI 消息, 调用 2 号中断向量的 nmi_handler64 例程, 其现在在 lib\service64.asm 文件里, 如下代码所示。

代码片段 1-59:

```

;-----
; nmi_handler64()
; input:
;     none
; output:
;     none
; 描述:
;     1) 由硬件或 IPI 调用
;-----
nmi_handler64:
    pusha64
    mov rbp, rsp
    ;;
    ;; 读取处理器 index, 判断 NMI handler 处理方式
    ;; 1) 当处理器 index 相应的 RequestMask 位为 1 时, 执行 IPI routine
    ;; 2) RequestMask 为 0 时, 执行默认 NMI 例程
    ;;
    mov ecx, [gs: PCB.ProcessorIndex]
    lock btr DWORD [fs: SDA.NmiIpiRequestMask], ecx
    jnc exception02                ; 转入执行默认 NMI 例程

    ;;
    ;; 下面调用 IPI routine
    ;;
    mov rax, [fs: SDA.NmiIpiRoutine]
    call rax

    ;;
    ;; 置内部信号有效
    ;;
    SET_INTERNAL_SIGNAL

    popa64
    iret64

```

nmi_handler64 例程检查 SDA.NmiIpiRequestMask 标志位, 如果 ProcessorIndex 对应的位为 1, 表明接收到 NMI IPI 而不是由硬件产生。nmi_handler64 例程从 SDA.NmiIpiRoutine 里读取目标任务的入口地址, 转而调用目标任务。

目标任务 switch_to_processor 例程执行处理器的切换工作, 如下代码所示。

代码片段 1-60:

```

;-----
; switch_to_processor()
; input:
;     none
; output:
;     none
; 描述:
;     1) 切换当前处理器
;-----
switch_to_processor:
    push ebp
    push ecx
    push ebx

```

```

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

;;
;; 切换焦点处理器
;;
mov ecx, [ebp + PCB.ProcessorIndex]
mov eax, SDA.InFocus
xchg [fs: eax], ecx

;;
;; 检查是否有 guese 环境存在需要切换
;;
mov esi, [ebp + PCB.ProcessorStatus]
test esi, CPU_STATUS_GUEST_EXIST
jz switch_to_processor.host

;;
;; 检查当前处理器是否已经拥有焦点
;; 1) 是: 则 XOR CPU_STATUS_GUEST 标志位
;; 2) 否: 则清 CPU_STATUS_GUEST 标志位
;;
mov edi, esi
and esi, ~CPU_STATUS_GUEST_FOCUS
xor edi, CPU_STATUS_GUEST_FOCUS
cmp ecx, [ebp + PCB.ProcessorIndex]
cmov esi, edi
mov [ebp + PCB.ProcessorStatus], esi

;;
;; 检查 host/guest 焦点
;;
test esi, CPU_STATUS_GUEST_FOCUS
jnz switch_to_processor.Guest

switch_to_processor.host:
;;
;; 切换 lcoal keyboard buffer
;;
REX.Wrxb
mov ebx, [ebp + PCB.LsbBase] ; ebx = LSB
REX.Wrxb
mov ebp, [ebp + PCB.SdaBase] ; ebp = SDA
REX.Wrxb
mov eax, [ebx + LSB.LocalKeyBufferHead]
REX.Wrxb
mov [ebp + SDA.KeyBufferHead], eax ; SDA.KeyBufferHead=LSB.LocalKeyBufferHead
REX.Wrxb
lea eax, [ebx + LSB.LocalKeyBufferPtr]
REX.Wrxb
mov [ebp + SDA.KeyBufferPtrPointer], eax ; KeyBufferPtrPointer=&LocalKeyBufferPtr
mov eax, [ebx + LSB.LocalKeyBufferSize]
mov [ebp + SDA.KeyBufferLength], eax ; KeyBufferLength = LocalKeyBufferSize

```

```

;;
;; 切换屏幕
;;
call flush_local_video_buffer          ; 刷新为当前处理器 local video buffer

jmp switch_to_processor.Done

switch_to_processor.Guest:
;;
;; 切换 VM keyboard buffer
;;
    REX.Wrxb
    mov ebx, [ebp + PCB.CurrentVmbPointer]
    REX.Wrxb
    mov ebx, [ebx + VMB.VsbBase]
    REX.Wrxb
    mov ebp, [ebp + PCB.SdaBase]        ; ebp = SDA
    REX.Wrxb
    mov eax, [ebx + VSB.VmKeyBufferHead]
    REX.Wrxb
    mov [ebp + SDA.KeyBufferHead], eax  ; SDA.KeyBufferHead =
VSB.VmKeyBufferHead
    REX.Wrxb
    lea eax, [ebx + VSB.VmKeyBufferPtr]
    REX.Wrxb
    mov [ebp + SDA.KeyBufferPtrPointer], eax ; SDA.KeyBufferPtrPointer =
&VmKeyBufferPtr
    mov eax, [ebx + VSB.VmKeyBufferSize]
    mov [ebp + SDA.KeyBufferLength], eax  ; SDA.KeyBufferLength =
VmKeyBufferSize

;;
;; 切换屏幕
;;
call flush_vm_video_buffer            ; 刷新为当前处理器 VM video buffer

switch_to_processor.Done:
    pop ebx
    pop ecx
    pop ebp
    ret

```

switch_to_processor 例程所做的工作如下：

- (1) 切换处理器焦点，将 SDA.InFocus 更新为当前处理器的 ProcessorIndex 值。
- (2) 检查是否存在 guest 环境。
 - 否则切换 host 端。
 - 是则检查 guest 当前是否拥有焦点。拥有焦点表明当前屏幕属于 guest，需要切换 guest 端。

在 host 端的切换中，将 LSB (Local Store Block, 本地存储块) 区域内的 LocalKeyBuffer 切换为当前的 KeyBuffer。当发生按键时，按下的键盘扫描码会存储在当前处理器的 LocalKeyBuffer 区域。接下来调用 flush_local_video_buffer 函数将 LocalVideoBuffer 数据刷新

到物理 VideoBuffer，完成显示屏幕的输出信息。

在 guest 端的切换中，将 VSB (VM Store Block，虚拟机存储块) 区域内的 VmKeyBuffer 切换为当前的 KeyBuffer。当发生按键时，按下的键盘扫描码会存储在当前 guest 对应的 VmKeyBuffer 区域。接下来调用 flush_vm_video_buffer 函数将 VmVideoBuffer 数据刷新到物理 VideoBuffer，完成显示屏幕的输出信息。

flush_local_video_buffer 函数与 flush_vm_video_buffer 函数都实现在 lib\LocalVideo.asm 文件里。flush_local_video_buffer 函数内部将 LocalVideoBuffer 切换为当前的 VideoBuffer。同样，flush_vm_video_buffer 函数内部将 VmVideoBuffer 切换为当前的 VideoBuffer。

1.6.3 调试记录机制

在代码的运行过程中难免会出现一些错误，我们需要快速找出错误的地方进行修正。系统实现了调试记录功能，可以很容易地观察到代码运行过程。

开启 debug record (调试记录) 功能需要在编译时提供 DEBUG_RECORD_ENABLE 符号的定义，如 build -DDEBUG_RECORD_ENABLE -D__X64。

为了支持 debug record 机制，SDA 区域定义了默认为 32K 大小的 DRS (Debug Record Structure) 区域 (参见第 1.4.4 节调试记录相关结构描述)，用来存放使用 DEBUG_RECORD 宏所产生的调试记录。

DRS 结构

在 inc\DebugRecord.inc 文件里定义了 DRS (Debug Record Structure) 结构，这个结构用来记录调试记录点当前的 context 信息。关于 DRS 结构内容参见第 1.4.6 节的描述。

DEBUG_RECORD 宏

将 DEBUG_RECORD 宏插入到代码中，它会为我们生成一条调试记录，并以链表的形式存放在 DRS 区域内。DEBUG_RECORD 宏需要提供一个参数，这个参数是我们提供的有说明意义的备注信息 (或附加信息)，目的是让我们可以更直观地了解当前的状态。

代码片段 1-61:

```

;-----
; DEBUG_RECORD
; input:
;     msg - 附注信息
; output:
;     none
; 描述:
;     1) 插入一条 debug 记录到 DRS 区域
;     2) %1 参数是附注信息
; 使用示例:
;     DEBUG_RECORD "hello, world"
;-----
%macro DEBUG_RECORD 1

```

```

#ifdef DEBUG_RECORD_ENABLE
    %%Line        EQU    __LINE__

    ;;
    ;; 下面在 64 位下编译
    ;;
    %if __BITS__ == 64
        pushf
        push rbp
        push rbx
        push rax
        push rcx
        mov rbp, [fs: SDA.Base]                ; rbp = SDA

        mov rbx, [rbp + SDA.DrsTailPtr]        ; rbx 指向 DRS 链表尾
        mov rax, [rbp + SDA.DrsIndex]         ; rax 指向 DRS 区域内的下一个节点

        ;;
        ;; index 超出了 DRS 区域顶部, 则不记录
        ;;
        cmp rax, [rbp + SDA.DrsTop]
        jae %%done

        ;;
        ;; 更新记录号
        ;;
        mov ecx, [rbx + DRS.RecordNumber]
        inc ecx
        mov [rax + DRS.RecordNumber], ecx

        ;;
        ;; 更新 DRS 链表节点
        ;;
        mov [rax + DRS.PrevDrs], rbx           ; DrsIndex->PrevDrs = DrsTailPtr
        mov [rbx + DRS.NextDrs], rax          ; DrsTailPtr->NextDrs = DrsIndex
        mov rbx, rax                         ; rbx 指向下一条 DRS 记录

        ;;
        ;; 更新 DrsIndex
        ;;
        mov eax, DRS_SIZE
        lock xadd [rbp + SDA.DrsIndex], rax

        ;;
        ;; 开始记录 debug record 信息
        ;;
        mov eax, [gs: PCB.ProcessorIndex]
        mov [rbx + DRS.ProcessorIndex], eax    ; 记录 processor index
        mov QWORD [rbx + DRS.Rip], %%IP        ; 记录 RIP
        mov rax, [rsp + 32]                   ; 读 rflags
        mov [rbx + DRS.Rflags], rax           ; 记录 Rflags

        ;;
        ;; 记录文件名和行数
        ;;
        mov DWORD [rbx + DRS.Line], %%Line
        mov QWORD [rbx + DRS.FileName], %%File
    %endif
#endif

```

```

;;
;; 附加信息
;;
mov QWORD [rbx + DRS.AppendMsg], %%Msg
mov QWORD [rbx + DRS.PrefixMsg], 0
mov QWORD [rbx + DRS.PostfixMsg], 0
mov DWORD [rbx + DRS.Address], 0

    jmp %%1

%%File DB    __FILE__, 0                ; 保存文件名
%%Msg  DB    %1, 0                    ; 保存附注信息

%%1:

;;
;; 更新 DRS count
;;
lock inc DWORD [rbp + SDA.DrsCount]

%else
;;
;; 在 32 位下编译
;;

pushf
push ebp
push ebx
push eax
push ecx

#ifdef __X64
LoadFsBaseToRbp
%else
mov ebp, [fs: SDA.Base]
#endif

    REX.Wrb
    mov ebx, [ebp + SDA.DrsTailPtr]        ; ebx 指向 DRS 链表尾
    REX.Wrb
    mov eax, [ebp + SDA.DrsIndex]         ; eax 指向下一个 DRS 节点

    REX.Wrb
    cmp eax, [ebp + SDA.DrsTop]
    jae %%done

;;
;; 更新记录号
;;
mov ecx, [ebx + DRS.RecordNumber]
INCv ecx
mov [eax + DRS.RecordNumber], ecx

;;
;; 更新 PrevDrs
;;

```



```

    REX.Wrxb
    mov [eax + DRS.PrevDrs], ebx                ; DrsIndex->PrevDrs = DrsTailPtr
    REX.Wrxb
    mov [ebx + DRS.NextDrs], eax                ; DrsTailPtr->NextDrs = DrsIndex
    REX.Wrxb
    mov ebx, eax                                ; rbx 指向下一条 DRS 记录

    ;;
    ;; 更新 DrsIndex
    ;;
    mov eax, DRS_SIZE
    PREFIX_LOCK
    REX.Wrxb
    xadd [ebp + SDA.DrsIndex], eax

    ;;
    ;; 开始记录 debug record 信息
    ;;
    mov eax, PCB.ProcessorIndex
    mov eax, [gs: eax]
    mov [ebx + DRS.ProcessorIndex], eax          ; 记录 processor index
    mov eax, %%IP
    REX.Wrxb
    mov [ebx + DRS.Rip], eax                    ; 记录 RIP

#ifdef __X64
    REX.Wrxb
    mov eax, [esp + 32]                        ; 读 rflags
#else
    mov eax, [esp + 16]                        ; 读 eflags
#endif
    REX.Wrxb
    mov [ebx + DRS.Rflags], eax                ; 记录 Rflags

    ;;
    ;; 记录文件名和行数
    ;;
    mov DWORD [ebx + DRS.Line], %%Line
    REX.Wrxb
    mov DWORD [ebx + DRS.FileName], %%file

    ;;
    ;; 附加信息
    ;;
    REX.Wrxb
    mov DWORD [ebx + DRS.AppendMsg], %%msg
    REX.Wrxb
    mov DWORD [ebx + DRS.PrefixMsg], 0
    REX.Wrxb
    mov DWORD [ebx + DRS.PostfixMsg], 0
    mov DWORD [ebx + DRS.Address], 0

    jmp %%1

%%file DB    __FILE__, 0                    ; 保存文件名
%%msg DB    %1, 0                          ; 保存附注信息

%%1:

```

```

;;
;; 记录 count
;;
lock inc DWORD [ebp + SDA.DrsCount]
%endif

;;
;; 记录 context
;;

%if __BITS__ == 64
    mov rax, [rsp + 8]                ; rax
    mov [rbx + DRS.Rax], rax
    mov rax, [rsp + 16]               ; rbx
    mov [rbx + DRS.Rbx], rax
    mov rax, [rsp + 24]               ; rbp
    mov [rbx + DRS.Rbp], rax
    lea rax, [rsp + 40]               ; rsp
    mov [rbx + DRS.Rsp], rax
    mov rax, [rsp]
    mov [rbx + DRS.Rcx], rax          ; rcx
    mov [rbx + DRS.Rdx], rdx          ; rdx
    mov [rbx + DRS.Rsi], rsi          ; rsi
    mov [rbx + DRS.Rdi], rdi          ; rdi
    mov [rbx + DRS.R8], r8            ; r8
    mov [rbx + DRS.R9], r9            ; r9
    mov [rbx + DRS.R10], r10          ; r10
    mov [rbx + DRS.R11], r11          ; r11
    mov [rbx + DRS.R12], r12          ; r12
    mov [rbx + DRS.R13], r13          ; r13
    mov [rbx + DRS.R14], r14          ; r14
    mov [rbx + DRS.R15], r15          ; r15
%else
%ifdef __X64
    REX.Wrxb
    mov eax, [esp + 8]                ; rax
    REX.Wrxb
    mov [ebx + DRS.Rax], eax
    REX.Wrxb
    mov eax, [esp + 16]               ; rbx
    REX.Wrxb
    mov [ebx + DRS.Rbx], eax
    REX.Wrxb
    mov eax, [esp + 24]               ; rbp
    REX.Wrxb
    mov [ebx + DRS.Rbp], eax
    REX.Wrxb
    lea eax, [esp + 40]               ; rsp
    REX.Wrxb
    mov [ebx + DRS.Rsp], eax
    REX.Wrxb
    mov eax, [esp]
    REX.Wrxb
    mov [ebx + DRS.Rcx], eax          ; rcx
    REX.Wrxb
    mov [ebx + DRS.Rdx], edx          ; rdx
    REX.Wrxb

```

```

        mov [ebx + DRS.Rsi], esi                ; rsi
        REX.Wrxb
        mov [ebx + DRS.Rdi], edi                ; rdi
        REX.WRxb
        mov [ebx + DRS.R8], eax                 ; r8
        REX.WRxb
        mov [ebx + DRS.R9], ecx                 ; r9
        REX.WRxb
        mov [ebx + DRS.R10], edx                ; r10
        REX.WRxb
        mov [ebx + DRS.R11], ebx                ; r11
        REX.WRxb
        mov [ebx + DRS.R12], esp                ; r12
        REX.WRxb
        mov [ebx + DRS.R13], ebp                ; r13
        REX.WRxb
        mov [ebx + DRS.R14], esi                ; r14
        REX.WRxb
        mov [ebx + DRS.R15], edi                ; r15
%else
        mov eax, [esp + 4]                      ; eax
        mov [ebx + DRS.Rax], eax
        mov eax, [esp + 8]                      ; ebx
        mov [ebx + DRS.Rbx], eax
        mov eax, [esp + 12]                     ; ebp
        mov [ebx + DRS.Rbp], eax
        lea eax, [esp + 20]                     ; esp
        mov [ebx + DRS.Rsp], eax
        mov eax, [esp]
        mov [ebx + DRS.Rcx], eax                ; ecx
        mov [ebx + DRS.Rdx], edx                ; edx
        mov [ebx + DRS.Rsi], esi                ; esi
        mov [ebx + DRS.Rdi], edi                ; edi

%endif                ;; // __X64

%endif                ;; // __BITS__

%%done:
;;
;; 更新 SDA.DrsTailPtr 记录和 DRS.NextDrs 指针
;;

%if __BITS__ == 64
        mov [rbp + SDA.DrsTailPtr], rbx
        mov QWORD [rbx + DRS.NextDrs], 0
%else
        REX.Wrxb
        mov [ebp + SDA.DrsTailPtr], ebx
        REX.Wrxb
        mov DWORD [ebx + DRS.NextDrs], 0
%endif

;;
;; 恢复 context
;;

```

```

%if __BITS__ == 64
    pop rcx
    pop rax
    pop rbx
    pop rbp
    popfq
%else
    pop ecx
    pop eax
    pop ebx
    pop ebp
    REX.wrbx
    popf
%endif

%%IP:

%endif          ;; DEBUG_RECORD_ENABLE
%endmacro

```

如上面代码所示，这个宏实现比较长。是为了能在 32 位与 64 位环境里都能够使用，并且能在 32 位以及 64 位下进行编译。

SDA.DrsHeadPtr 指向 SDA.Drs 区域的基址，SDA.DrsTailPtr 指向调试记录链的尾部，而 SDA.DrsIndex 则指向下一条记录将要写入的位置。在初始化 DRS 管理记录时，这三个指针是相等的。

每条 debug record 记录会存放在 SDA.DrsIndex 指向的位置。当插入第 1 条调试记录时，也就等于在 SDA.DrsHeadPtr 指向的头部位置存放 DRS 记录。插入一条记录后，SDA.DrsIndex 会指向下一条记录要写的位置，而 SDA.DrsTailPtr 则保持指向最后一条已写入的记录位置。

前一条记录的 DRS.NextDrs 指向本条记录，而本条记录的 DRS.PrevDrs 指针则指向由 SDA.DrsTailPtr 指向的记录。因此，形成双向链表结构。每次接入一条记录，SDA.DrsCount 值会被递增。如果之前的 SDA.DrsCount 为 0，则说明插入的是第 1 次记录。

显示调试记录

lib\DebugRecord.asm 文件里实现了两个模式的调试记录显示函数：

- (1) 调试记录显示模式：使用 dump_debug_record 函数。
- (2) msg-list（信息列表）显示模式：使用 dump_append_msg_list 函数。

dump_debug_reocrd 函数打印每条调试记录的明细信息，包括所有通用寄存器、CS:RIP、RFLAGS、文件名字、调试记录的插入点（行数），以及备注信息（DRS.AppendMsg）。而 dump_append_msg_list 函数则主要输出备注信息（DRS.AppendMsg）。

dump_debug_reocrd 与 dump_append_msg_list 函数这里不再一一列出，请读者朋友们自行在 lib\DebugRecord.asm 文件里查阅。

1.6.3.1 例子 1-2

示例 1-2: 使用调试记录功能观察信息

现在我们完成例子 1-2，我们编写简单的代码来练习使用调试记录观察信息。common\目录下的 boot.asm，setup.asm，protected.asm 以及 long.asm 文件是所有示例代码共用的，下面是例子 1-2 主体文件 chap01\ex1-2\ex.asm 的代码。

代码片段 1-62:

```

;;
;; 例子 ex1-2: 使用调试记录功能观察信息
;;

;;
;; 调度目标处理器执行 dump_debug_record() 函数，根据 CPU 数量来确定目标处理器!
;;
mov esi, [fs: SDA.ProcessorCount]
dec esi
mov edi, dump_debug_record
call dispatch_to_processor

;;
;; 输出信息
;;
mov esi, Ex.Msg1
call puts

;;
;; 插入三条调试记录
;;
mov ecx, 1
DEBUG_RECORD "debug record 1"
mov ecx, 2
DEBUG_RECORD "debug record 2"
mov ecx, 3
DEBUG_RECORD "debug record 3"

jmp $

Ex.Msg1      db      'example 1-2: test DEUBG_REOCD', 0

```

在 ex.asm 末尾插入了三条调试记录，备注信息分别为“debug record 1”、“debug record 2”，以及“debug record 3”，并且在每个记录点前给 ECX 寄存器设置不同的值。

使用下面的命令进行编译，并生成目标映像文件：

- build -DDEBUG_RECORD_ENABLE
- build -DDEBUG_RECORD_ENABLE -D__X64

这两条 Build 命令都需要定义 DEBUG_RECORD_ENABLE 符号，这样才能开启调试记录功能。

1.6.3.2 运行结果

我们使用生成的软盘映像文件 `demo.img` 来启动 vmware 运行后，按下<F4>键切换到 CPU3 对应的屏幕（如果只有 2 个处理器的话，需要按<F2>键切换到 CPU1 来观察；如果 4 个处理器则按<F4>键切换到 CPU3 来观察），如图 1-9 所示。

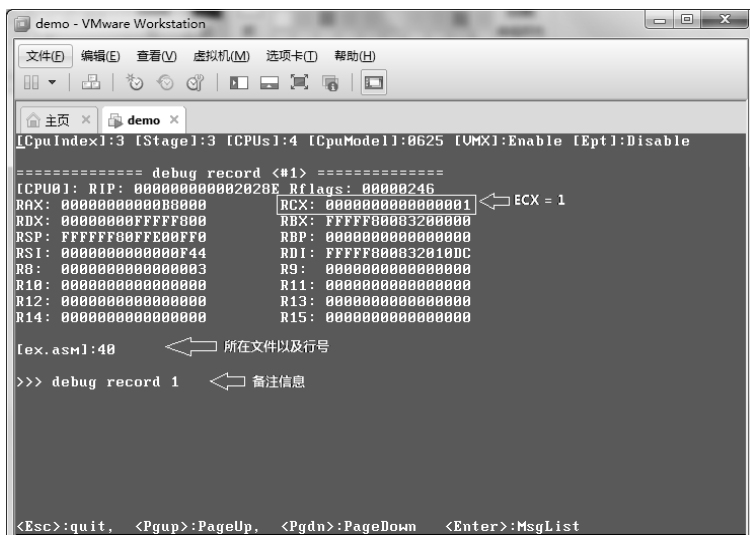


图 1-9

图中，我们看到的是第 1 条调试记录的明细信息。例如，ECX 寄存器的值为 1（第 1 条调试记录之前设置的值），备注信息为“debug record 1”。也可以观察到这个调试记录点位于 `ex.asm` 文件，行号为 40。

在调试记录显示模式里，我们可以按<PgUp>键向上翻记录，按<PgDn>键向下翻记录。接下来，我们按<Enter>键切换到 msg-list 消息列表显示模式，如图 1-10 所示。

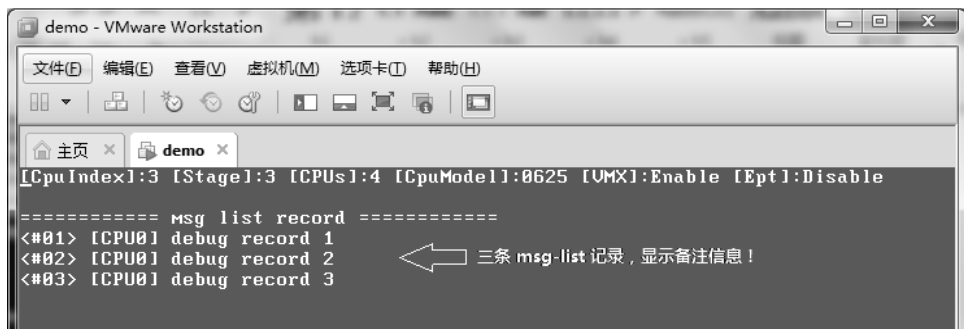
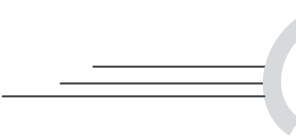


图 1-10

在图中的 msg-list 列表显示里，我们看到了插入到 `ex.asm` 文件里三条调试记录的备注信息，它们在 CPU0 里执行。我们按下<Enter>键可以切换回调试记录显示模式。



第 2 章

VMX 架构基础

为了支持虚拟机环境，在计算机结构体系中的 CPU 域和 PCI 总线域上，Intel 提供了三个层面的虚拟化技术（Intel Virtualization Technology）。

（1）基于处理器的虚拟化技术（Intel VT-x）：全称为 Virtualization Technology for x86，在处理器上实现了虚拟化技术，它的实现架构是 Virtual-Machine Extensions (VMX)。在 VMX 下引入了两种处理器模式，即 VMX root 与 VMX non-root，分别支持 host 与 guest 软件的运行，使 guest 软件能直接在处理器硬件上运行。

（2）基于 PCI 总线域设备实现的 I/O 虚拟化技术（Intel VT-d）：全称为 Virtualization Technology for Directed I/O，这个虚拟化技术实现在芯片组（Chipset）层面上，提供了诸如 DMA remapping 与 Interrupt remapping 等技术来支持外部设备 I/O 访问的直接虚拟化。

（3）基于网络的虚拟化技术（Intel VT-c）：全称为 Virtualization Technology for Connectivity，部署在 Intel 的网络设备上，这也是基于 PCI 总线域设备的虚拟化技术。

根据 Intel 的介绍，在所有 Intel 的 10 Gigabit Server Adapter 和部分 Gigabit Server Adapter 上，VT-c 提供了两个关键的技术：Virtual Machine Device Queues (VMDq) 和 Virtual Machine Direct Connect (VMDc)。Intel server adapter 上的 VMDq 可以做到为各个 guest OS 处理分类好的 packet，能明显地提高 I/O 吞吐量。而 VMDc 使用 PCI-SIG Single Root I/O (SR-IOV) 技术，允许虚拟机直接访问网络 I/O 硬件改善虚拟化性能。

在处理器虚拟化方面，Intel 也为 Itanium 平台提供了虚拟化，即 Intel VT-i。本书中所讨论的虚拟化技术主要是基于处理器虚拟化（Intel VT-x）中的 Virtual-Machine Extensions (VMX) 架构。关于 VT-d 技术读者可以下载《Intel Virtualization Technology for Directed I/O Architecture Specification》文档，它并不在本书讨论的范围。VT-d 技术在桌面平台上并不是每款处理器都提供，VT-c 技术提供在服务器平台上。

2.1 虚拟化概述

虚拟机的运用需要以处理器平台提供 Virtualization Technology (VT, 虚拟化技术) 作为前提, 是对资源的虚拟化管理的结果。在虚拟化技术出现之前, 软件运行在物理机器上, 对物理资源进行直接控制, 譬如设备的中断请求, guest 的内存访问, 设备的 I/O 访问等。软件更改这些资源的状态将直接反映在物理资源上, 或者设备的请求得到直接响应。

在 CPU 端的虚拟化里, 实现了 VMX (Virtual-Machine Extensions, 虚拟机扩展) 架构。在这个虚拟机架构里, 存在两种角色环境: VMM (Virtual Machine Monitor, 虚拟机监管者) 和 VM (Virtual Machine, 虚拟机)。host 端软件运行在 VMM 环境里, 可以仅仅作为 hypervisor 角色存在 (作为全职的虚拟机管理者), 或者包括 VMM (虚拟机监管者) 职能的 host OS。

guest 端软件运行在 VM 环境里。一个 VM 代表着一个虚拟机实例, 一个处理器平台可以有多个虚拟机实例存在, 由于 VM 里的资源被虚拟化, 每个 VM 是彼此独立的。

guest 软件访问的资源受到 VMM 的监管, guest 希望修改某些物理资源时, VMM 返回一个虚拟化后的结果给 guest。例如, guest 软件对 8259 中断控制器的状态进行修改, VMM 拦截这个修改, 进行虚拟化操作, 实施修改或者不修改 8259 中断控制器的物理状态, 反馈一个不真实的结果给 guest 软件。

图 2-1 展示了前面提及的虚拟机环境里的三种虚拟化管理:

- (1) 设备中断请求
- (2) guest 内存访问
- (3) 设备的 I/O 访问

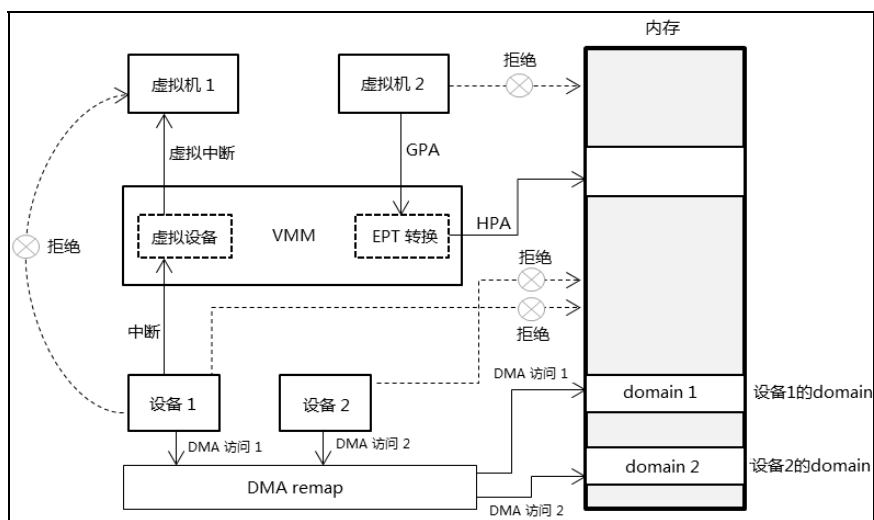


图 2-1

设备的中断请求经由 VMM 监管，模拟出虚拟设备反馈一个虚拟中断给 guest 执行，在这个模型里，设备中断请求不直接发给 guest 执行。而 guest 访问的物理地址也不是最终的物理地址，而是经过 EPT 进行转换才得到最终物理地址。设备 1 和设备 2 使用 DMA 访问时，它们最终的目标物理地址经过 VT-d 技术的 DMA 重新映射功能映射到属于自己的 domain（区域）。

2.1.1 虚拟设备

在设备发生中断请求时，这就产生了设备的 ISR（中断服务例程）是由 VMM（或 host OS）处理，还是直接由 VM（guest OS）处理（或由 VMM 转发给 VM）的问题，即 ISR 是运行在 host 还是 guest 端的问题。

依赖于 VMM 对外部设备所有权的设计产生了两种模型：

- 设备是属于 host 所有，物理设备的 ISR 由 VMM 来执行。
- 设备分配给 VM 使用，那么 ISR 将在 guest 环境里运行。

host 和 guest 都有自己的 IDT（中断描述符表），host vector 对应着物理设备 ISR 在 host IDT 的位置，而 guest vector 对应着 VMM 反馈给 guest 的中断请求在 guest IDT 上的位置。中断请求通过 VMM 给 VM 使用 injection event（注入事件）的手段来实现。

host vector 和 guest vector 并不相等，但在设备分配给 VM 使用的模型里，如果外部中断的控制权在 guest 手中，那么 guest vector 可以对应物理设备 ISR 在 guest IDT 的位置，也就是物理设备 ISR 完全由 guest 执行，而 VMM 并不监管或转发。

如果外部中断的控制权需要掌握在 VMM 手中，通过开启“external-interrupt exiting”功能，并且结合“acknowledge interrupt on exit”位的设置来实现。当发生外部中断时，VM 停止工作，处理器控制权切换回 VMM。

在设备所有权归 VMM 的模型里，虚拟设备应运而生，它是 VMM 在物理设备之上抽象出的虚拟设备概念。中断发生时，VMM 的 vector 是物理设备对应应在 host IDT 里，VMM 执行物理设备的 ISR。然后模拟 guest 的 ISR 处理流程，如发送 EOI 命令给中断控制器，更新中断控制器状态。根据对应的 guest vector 注入一个外部中断给 guest 在自己的 IDT 找到 ISR 执行，这个 guest ISR 可能并不能做实际的收尾工作，如发送 EOI 命令（被 VMM 接管）或更新中断控制器。

host vector 代表着来自平台的物理设备，而 guest vector 代表着来自虚拟设备，这两个 vector 值不一致，这个虚拟设备由 VMM 维护而代表着物理设备在 VM 环境中的名字。虚拟设备可能并不存在实体功能，只是逻辑表述上的抽象概念。

在设备分配给 VM 使用的模型里，guest vector 与 host vector 可能一致也可能不一致，但物理设备的 ISR 由 guest 去运行，代表着 ISR 的收尾工作由 guest 去执行。VMM 截取中断请求后根据 guest vector，同样使用事件注入手段转发外部中断给 guest 执行，而 VMM 可能并不做其他工作。这是在 host 夺取外部中断控制权的前提下，前面所述在

guest 掌控外部中断权时，设备 ISR 直接在 guest 里执行。

当然，VMM 也可以不拦截外部中断，这样外部中断就直接通过 guest-IDT 进行 deliver 执行，而不需要经过 VMM 转发。

2.1.2 地址转换

host 软件与 guest 软件都运行在物理平台上，需要 guest 不能干扰 VMM 的执行。譬如，guest 软件访问 100000h 物理地址，但这个物理地址可能属于 host 的私有空间，或者 host 也需要访问 100000h 物理地址。VMM 的设计需要 guest 不能访问到这个真实的物理地址，VMM 通过 EPT (Extend Page Table, 扩展页表) 来实现“**guest 端物理地址到 host 端物理地址**”的转换，使得 guest 访问到其他的物理区域。

也因为，物理平台上可能有多个 VM 在运行，例如虚拟机 1 与虚拟机 2 也可能会访问到同一个物理区域。为保证每个 VM 的物理地址空间隔离，保持独立性，也需要将 guest 的内存访问进行转换。内存虚拟化是一项很重要的工作，引进的 EPT (扩展页表) 机制是实现内存虚拟化的重要手段。EPT 的实现原理和分页机制里的转换页表一样，经过多级转换产生最终的物理地址。

在开启 EPT 机制的情况下，产生了两个地址概念：**GPA** (Guest Physical Address) 和 **HPA** (Host Physical Address)，HPA 是真正的物理地址。guest 软件访问的物理地址都属于 GPA，而 host 软件访问的物理地址则属于 HPA。而在没启用 EPT 机制的情况下，guest 软件访问的物理地址就是最终的物理地址。

这里还有另一个重要的概念：**guest paging-structure table** (guest 的页结构表)，也就是 guest 内保护模式分页机制下的线性地址到物理地址转换使用的页表。这个页表项内使用的物理地址是 GPA (例如 CR3 的页目录指针基址)，而 **EPT paging-structure table** (EPT 页表结构) 页表项使用的是 HPA。

2.1.3 设备的 I/O 访问

在由 CPU 发起访问外部设备时，需要通过 PCI/PCIe 总线的内存读写事务，I/O 读写事务或配置读写事务进行。当 guest 软件发起这样的访问时，VMM 可以通过内存虚拟化和 I/O 地址虚拟化达到虚拟化设备的目的。

当由设备主动发起访问内存时 (即 DMA 读写事务)，在 DMA 读写下 CPU 不参与其中执行过程，也就不能为这个读写内存提供地址转换，VMM 单纯依赖 CPU 来监控设备的访问是比较困难的。

于是，基于 PCI 总线域的虚拟化需要提供，VT-d 技术正是为了解决这些而提出的。VT-d 的其中一个重要功能是进行 DMA remapping (DMA 重新映射)，DMA remapping 机制在芯片组或 PCI 设备上实现地址转换功能，其原理和分页机制下的虚拟地址转换到物理地址是相似的。

DMA remapping 需要识别设备的 source-id，这个 source-id 代表着发起 DMA 访问设备（即 requester，请求者）的身份，实际上它就是 PCI 总线域 ID，由 bus、device 及 function 组成。根据 source-id 找到设备对应的页表结构，然后通过页表进行转换。

如图 2-2 所示，首先，一个被称为 Root-entry table 的结构需要在内存中构造，由 Root-entry 指出 Context-entry table，再由 context-entry 得到页表结构，最后经页表得到最终的物理地址。

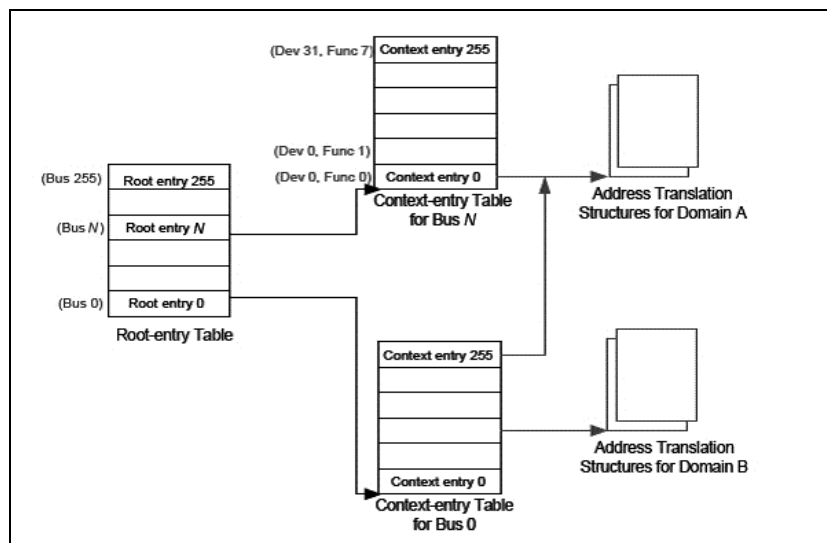


图 2-2

Root-entry table 的基址被提供在 MCH (Memory Control Hub) 部件，一般位于 Bus0、Device0、Function0 设备（内存控制器）扩展空间里的 MEREMAPBAR 寄存器，在这个寄存器提供一个 remap MMIO 空间，其中的 RTADDR_REG 寄存器提供 Root-entry table 指针值，但是对于不同的处理器，这个 remap MMIO 空间的基址也存放在不同的位置。

DMA remapping 提出了一个 domain（域）的概念，实际上就是为设备在内存中分配一个对应的区域。例如，为设备 1 分配 domain 1，为设备 2 分配 domain 2，它们访问各自独立的区域。这个 domain 值可能被用作标记，处理器使用它来标记内部的 cache。当然，不同的设备也可以访问同一个 domain，但必须使用同一个 domain 值。

2.2 VMX 架构

Intel 实现了 VMX 架构来支持 CPU 端的虚拟化技术 (Intel VT-x 技术)，引入了一种新的处理器操作模式，被称为 **VMX operation**。也加入了一系列的 VMX 指令支持 VMX operation 模式。

2.2.1 VMM 与 VM

在 VMX 架构下定义了两类软件的角色和环境：

- VMM (Virtual Machine Monitor, 虚拟机监管者)
- VM (Virtual Machine, 虚拟机)

VMM 代表一类在 VMX 架构下的管理者角色，它可以是以 hypervisor 软件形式独立存在，也可以是在 host OS 中集成了 VMM 组件，也就是在 host OS 中提供了虚拟机管理者的职能。软件运行在 VMM 环境下拥有物理平台的控制权，并且监控每个 VM 的运行。

VM 代表着虚拟机实例，VMM 在需要执行虚拟机实例代码时，需要进入 VM 环境。在一个 VMX 架构里可以有多个虚拟机实例。每个 VM 像是在一个独立的物理机器环境里，有自己的内存、中断甚至设备等。但 VM 本身并不会意识到自己运行在虚拟的环境里，VM 里的资源实际上掌控在 VMM 手中，由 VMM 仿真出一些资源反馈给 VM。

host 端软件可以是 VMM 或者 host OS，而 guest 端软件就是运行在 VM 实例环境里。guest OS 就像运行在普通的物理机器上，执行自己的应用程序、kernel 组件及 driver 组件等。

在虚拟化平台设计里，VM 不能影响到 VMM 软件的执行，并且每个 VM 之间也要确保独立互不干扰，这些工作需要 VMM 这个管理者对 VM 进行一些监控以及配置。VMM 软件监控着每个 VM 对资源的访问，并限制某些资源访问。典型的，VMM 可以允许或拒绝某个 VM 环境响应外部中断请求。又如，当 VM 里的 guest 软件发生#PF (Page Fault) 异常时，VMM 接管并分析#PF 异常产生的原因，进行或者不进行某些处理，然后反射回 guest 执行属于自己的#PF 异常处理。

2.2.2 VMXON 与 VMCS 区域

在 VMX 架构下，至少需要实现一个被称为“**VMXON region**”，以及一个被称为“**VMCS region**”的物理区域，VMXON 区域对应于 VMM，VMM 使用 VMXON 区域对一些数据进行记录和维护。而每个 VM 也需要有自己对应的 **VMCS** (Virtual Machine Control Structure, 虚拟机控制结构) 区域，VMM 使用 VMCS 区域来配置 VM 的运行环境，以及控制 VM 的运行。

在进入 VMX operation 模式前，必须先为 VMM 准备一份 VMXON 区域，同样在进入 VM 前也必须准备相应的 VMCS 区域，并配置 VM 的运行环境。一个 VM 对应一份 VMCS 区域，因此，VMX 平台上有多少份 VM 实例就需要维护多少份 VMCS 区域。

处理器进入 VMX operation 模式需要执行 VMXON 指令，一个指向 VMXON 区域的物理指针被作为 VMXON 指令的操作数提供。而进入 VM 之前必须先使用 VMPTRLD 指令加载 VMCS 指针，VMCS 指针指向需要进入的 VM 所对应的 VMCS 区域。

在 VMX operation 模式下，处理器会自动维护 4 个指针值：

- **VMXON pointer**，指向 VMXON 区域，有时也可以被称为 **VMX 指针**。
- **current-VMCS pointer**，这是当前所使用的 VMCS 区域指针，VMCS pointer 可以有多个，但在一个时刻里，只有唯一一个 current-VMCS pointer。
- **executive-VMCS pointer** 及 **SMM-transfer VMCS pointer**。

Executive-VMCS 指针与 SMM-transfer VMCS 指针使用在开启 SMM dual-monitor treatment (SMM 双重监控处理) 机制下。在这个机制下，executive-VMCS 指针既可以指向 VMXON 区域，也可以指向 VMCS 区域。SMM-transfer VMCS 指针指向切入 SMM 模式后使用的 VMCS 区域。

2.2.3 检测 VMX 支持

软件应通过检查 CPUID.01H:ECX[5].VMX 位来确定是否支持 VMX 架构，该位为 1 时表明处理器支持 VMX 架构。可以实现如代码片段 2-1 所示的独立的函数来检测。

代码片段 2-1:

```

;-----
; support_intel_vmx()
; input:
;     none
; output:
;     1 - support, 0 - unsupported
; 描述:
;     1) 检查是否支持 Intel VT-x 技术
;-----
support_intel_vmx:
    ;;
    ;; 检查 CPUID.01H:ECX[5].VMX 位
    ;;
    bt DWORD [gs: PCB.FeatureEcX], 5
    setc al
    movzx eax, al
    ret

```

或者在进入 VMX operation 模式前进行 VMX 支持的检查，第 2.5 节将介绍更多关于“VMX 支持能力检查”的描述。

2.2.4 开启 VMX 进入允许

要开启 VMX operation 模式，必须先开启 CR4.VMXE 控制位，该控制位也表明处理器允许使用 VMXON 指令，但其他的 VMX 指令则必须在进入 VMX operation 模式后才能使用。

代码片段 2-2:

```

;;
;; 检测是否支持 VMX
;;
bt DWORD [ebp + PCB.FeatureEcX], 5
mov eax, STATUS_UNSUCCESS

```

```

jnc vmx_operation_enter.done

;;
;; 开启 VMX operation 允许
;;
REX.Wrb
mov eax, cr4
REX.Wrb
bts eax, 13                ; CR4.VMEX = 1
REX.Wrb
mov cr4, eax

```

上面是实现进入 VMX operation 模式前的一段代码，在检查处理器支持 VMX 后，置 CR4.VMEX[13] 位将允许进入 VMX operation 模式。此时，可以执行 VMXON 指令。在 CR4.VMEX=0 时，执行 VMXON 指令将会产生 #UD 异常。

代码中的 REX.Wrb 定义为一个宏，在定义了 __X64 符号的情况下有效，它嵌入了一个 REX prefix 字节，也就是 48H 字节，在 64 位代码下使用 64 位的操作数（关于这方面的知识请参考第 1.2 节）。

2.3 VMX operation 模式

前面提及，在 VMX 架构下，处理器支持一种新的 VMX operation 模式。VMXON 指令只能在开启允许进入 VMX operation 模式后才能使用，其余的 VMX 指令只能在 VMX operation 模式下使用。VMX operation 模式里又分为两个操作环境，以支持 VMM 与 VM 软件的运行。

- VMX root operation
- VMX non-root operation

VMX 模式的 **root** 与 **non-root** 环境可以理解为：VMX 管理者与 guest 用户使用的环境。因此，VMM 运行在 VMX root operation 环境下，VM 则运行在 VMX non-root operation 环境下。

从 root 环境切换到 non-root 环境被称为“**VM-entry**”，反之从 non-root 环境切换回 root 环境被称为“**VM-exit**”。当软件运行在 VMX root operation 环境时，处理器的 CPL (Current Privilege Level) 必须为 0，拥有最高的权限，可以访问所有的资源，包括新引进的 VMX 指令。

在 VMX non-root operation 环境里，当前的 CPL 值不必为 0。根据 VMM 的相应设置，guest 软件的访问权限受到了限制，部分指令的行为也会发生改变。在 VMX non-root operation 模式下，guest 软件执行任何一条 VMX 指令（除 VMFUNC 指令外）都会导致被称为“**VM-exit**”的行为发生。另外，软件执行 MOV 指令对 CR0 寄存器进行设置时，写 CR0 寄存器的行为将发生改变，导致 CR0 寄存器的值允许写入或被拒绝修改。

2.3.1 进入 VMX operation 模式

如前面所述，在启用处理器的虚拟化机制前，必须先开启 VMX 模式的允许（开启 CR4.VMXE 位），表明允许执行 VMXON 指令进入 VMX operation 模式。

在执行 VMXON 指令切换到 VMX operation 模式之前，还需要做一系列的准备工作，下面的代码片段摘自 lib\Vmx\VmxInit.asm 文件里的 vmx_operation_enter 函数，这里简要地了解一下。

代码片段 2-3:

```
;;
;; 检查是否已经进入了 VMX root operation 模式
;;
test DWORD [ebp + PCB.ProcessorStatus], CPU_STATUS_VMXON
jnz vmx_operation_enter.done
;;
;; 检测是否支持 VMX
;;
bt DWORD [ebp + PCB.FeatureEcX], 5
mov eax, STATUS_UNSUCCESS
jnc vmx_operation_enter.done

;;
;; 开启 VMX operation 允许
;;
REX.Wrxb
mov eax, cr4
REX.Wrxb
bts eax, 13 ; CR4.VMEX = 1
REX.Wrxb
mov cr4, eax

;;
;; 更新指令状态，允许执行 VMX 指令
;;
or DWORD [ebp + PCB.InstructionStatus], INST_STATUS_VMX

;;
;; 初始化 VMXON 区域
;;
call initialize_vmxon_region
cmp eax, STATUS_SUCCESS
jne vmx_operation_enter.done

;;
;; 进入 VMX root operation 模式
;; 1) operand 是物理地址 pointer
;;
vmxon [ebp + PCB.VmxonPhysicalPointer]

;;
;; 检查 VMXON 指令是否执行成功
;; 1) 当 CF = 0 时，VMXON 执行成功
;; 1) 当 CF = 1 时，返回失败
;;
mov eax, STATUS_UNSUCCESS
```

```

jc vmx_operation_enter.done
jz vmx_operation_enter.done

```

上面代码的处理如下：

- (1) 首先检查处理器是否已经进入 VMX operation 模式。是则直接返回，否则进行下一步。处理器 PCB（处理器控制块）中的 ProcessorStatus 的值记录着当前处理器的状态。
- (2) 检测是否支持 VMX 架构，这里并不在其他地方使用独立的函数进行检查。
- (3) 开启 CR4.VMXE 控制位，表明允许进入 VMX operation 模式。详见第 2.2.4 节。
- (4) 其中重要的一步是调用 initialize_vmxon_region 函数来初始化 VMXON 区域。
- (5) 接着执行 VMXON 指令，提供一个 VMXON 区域的物理指针作为操作数，这个指针被称为 VMXON 指针，注意这个指针使用的是物理地址。
- (6) 紧接着对 VMXON 指令进行检查是否执行成功，分别检查 CF 与 ZF 标志位，当 CF=1 或 ZF=1 时，表明操作失败。

VMXON 指令执行成功后，表明处理器此时已经进入 VMX operation 模式，也就是已经处于 root 环境，行使 VMM 的管理职能。

2.3.2 进入 VMX operation 的制约

执行 VMXON 指令的基本要求是：需要开启 CR4.VMXE 位，不能在实模式、virtual-8086 以及兼容模式下执行，否则将产生#UD 异常；不能在非 0 级权限下执行，否则将产生#GP 异常。

2.3.2.1 IA32_FEATURE_CONTROL 寄存器

IA32_FEATURE_CONTROL 寄存器也影响着 VMXON 指令的执行，这个寄存器的 bit 0 为 lock 位，bit 1 与 bit 2 分别是 SMX 模式的 **inside** 与 **outside** 位。Inside 位指示允许在 SMX 内使用 VMX，outside 位指示允许在 SMX 外使用 VMX（这是开启 VMX 模式最基本的条件）。

(1) 当 lock 位为 0 时，执行 VMXON 指令将产生#GP 异常。因此，执行 VMXON 指令前必须确保 lock 位为 1 值，也就是锁上 IA32_FEATURE_CONTROL 寄存器。上锁后如果对 IA32_FEATURE_CONTROL 寄存器进行写操作，将产生#GP 异常。

(2) Enable VMX inside SMX 位 (bit 1)，指示当处理器处于 SMX (Safer Mode Extensions) 模式时，允许开启 VMX operation 模式。当 **inside=0 且处于 SMX 模式**时，执行 VMXON 指令将引发#GP 异常。

(3) Enable VMX outside SMX 位 (bit 2)，指示允许在 SMX 模式之外开启 VMX operation 模式。当 **outside=0 且不在 SMX 模式**时，执行 VMXON 指令将引发#GP 异常。

只有支持 VMX 与 SMX 模式时，才允许对 bit 1 置位（即 CPUID.01H:ECX[6:5]=11B）。在支持 VMX 模式时，才允许对 bit 2 置位（即

CPUID.01H:ECX[5]=1)。

注意：“Enable VMX outside SMX”位需要置位，这是运行 VMX 模式的基本条件。bit 1 与 bit 2 允许同时置位，此时表示无论是 SMX 模式内还是模式外都可以开启 VMX 模式。

下面的代码片段来自 \lib\system_data_manage.asm 里的 get_vmx_global_data 函数，用来收集和分析 VMX 的数据：

代码片段 2-4：

```
;;
;; 关于 IA32_FEATURE_CONTROL.lock 位:
;; 1) 当 lock = 0 时, 执行 VMXON 产生 #GP 异常
;; 2) 当 lock = 1 时, 写 IA32_FEATURE_CONTROL 寄存器产生 #GP 异常
;;

;;
;; 下面将检查 IA32_FEATURE_CONTROL 寄存器
;; 1) 当 lock 位为 0 时, 需要进行一些设置, 然后锁上 IA32_FEATURE_CONTROL
;;
mov ecx, IA32_FEATURE_CONTROL
rdmsr
bts eax, 0 ; 检查 lock 位, 并上锁
jc get_vmx_global_data.@7

;; lock 未上锁时:
;; 1) 对 lock 置位 (锁上 IA32_FEATURE_CONTROL 寄存器)
;; 2) 对 bit 2 置位 (启用 enable VMXON outside SMX)
;; 3) 如果支持 enable VMXON inside SMX, 对 bit 1 置位
;;
mov esi, 6 ; enable VMX outside SMX = 1, enable VMX inside SMX = 1
mov edi, 4 ; enable VMX outside SMX = 1, enable VMX inside SMX = 0

;;
;; 检查是否支持 SMX 模式
;;
test DWORD [gs: PCB.FeatureEcx], CPU_FLAGS_SMX
cmovz esi, edi
or eax, esi
wrmsr

get_vmx_global_data.@7:

;;
;; 假如使用 enable VMX inside SMX 功能,
;; 则根据 IA32_FEATURE_CONTROL[1] 来决定是否必须开启 CR4.SMXE
;; 1) 本书例子中没有开启 CR4.SMXE
;;
#ifdef ENABLE_VMX_INSIDE_SMX
;;
;; ### step 7: 设置 Cr4FixedMask 的 CR4.SMXE 位 ###
;;
;; 再次读取 IA32_FEATURE_CONTROL 寄存器
;; 1) 检查 enable VMX inside SMX 位 (bit1)
;; 1.1) 如果是 inside SMX (即 bit1 = 1), 则设置 CR4FixedMask 标志位的相应位
;;
```

```

rdmsr
and eax, 2                ; 取 enable VMX inside SMX 位的值 (bit1)
shl eax, 13               ; 对应 CR4 寄存器的 bit 14 位 (即 CR4.SMXE 位)
                          ; 在 Cr4FixedMask 里设置 enable VMX inside SMX 位的值
or DWORD [ebp + PCB.Cr4FixedMask], eax

%endif

```

这段代码检查 IA32_FEATURE_CONTROL 寄存器的 lock 位是否为 1。不为 1 时，需要上锁，并对“Enable VMX outside SMX”置位，同时根据 CPUID.01H:EDX[6].SMX 位来设置“Enable VMX inside SMX”位。

在定义了 ENABLE_VMX_INSIDE_SMX 符号时，接下来分析是否启用“Enable VMX inside SMX”，如果是，那么 Cr4FixedMask 的值需要添加 CR4.SMXE 位（需要为 1 值），也就是必须要开启 SMX 模式。本书中没有开启 CR4.SMXE 位。

2.3.2.2 CR0 与 CR4 固定位

进入 VMX operation 模式前，CR0 与 CR4 寄存器也必须要满足 VMX operation 模式运行的条件：

(1) CR0 寄存器需要满足 **PG** (bit 31)、**NE** (bit 5)，以及 **PE** (bit 0) 位为 1。也就是需要开启分页机制的保护模式，x87 FPU 数字异常的处理模式需要使用 native 模式。

(2) CR4 寄存器需要开启 VMXE 位 (bit 13)。

这些位在 IA32_VMX_CR0_FIXED0 寄存器里属于 Fixed to 1（固定为 1）位（参见第 2.5.10 节），否则执行 VMXON 指令将会产生 #GP 异常。由于 CR0.PG 与 CR0.PE 位必须为 1，因此 VMX operation 不能从实模式和非分页的保护模式里进入。如果需要在 SMX 模式里进入 VMX 模式，那么 CR0.SMXE 位 (bit 14) 也需要为 1 值。

在 VMCS 区域的字段设置里，也有很多类似这样的制约条件存在，某些位必须为 1 值，某些位必须为 0 值，某些位可以为 0 值也可以为 1 值。例如，上面的 CR0 与 CR4 寄存器列举的位必须为 1 值。因此，软件需要检测制约条件而进行相应的设置，这些设置也可能随着处理器架构的发展而有所改变。

上面所述的 CR0 与 CR4 寄存器固定位从第 1 代支持 VMX 的处理器开始就被确定了，后面我们将会看到使用 IA32_VMX_CR0_FIXED0 与 IA32_VMX_CR0_FIXED1 对 CR0 寄存器的固定位进行检测，使用 IA32_VMX_CR4_FIXED0 与 IA32_VMX_CR4_FIXED1 对 CR4 寄存器固定位进行检测。

下面一段代码来自 lib\Vmx\VmxInit.asm 里的 initialize_vmxon_region 函数，执行对 CR0 与 CR4 寄存器固定位的设置。

代码片段 2-5:

```

;;
;; 读 CR0 当前值
;;
REX.Wrb
mov ecx, cr0

```

```

mov ebx, ecx

;;
;; 检查 CR0.PE 与 CR0.PG 是否符合 fixed 位, 这里只检查低 32 位值
;; 1) 对比 Cr0FixedMask 值 (固定为 1 值), 不相同则返回错误码
;;
mov eax, STATUS_VMX_UNEXPECT          ; 错误码 (超出期望值)
xor ecx, [ebp + PCB.Cr0FixedMask]      ; 与 Cr0FixedMask 值异或, 检查是否相同
js initialize_vmxon_region.done        ; 检查 CR0.PG 位是否相等
test ecx, 1
jnz initialize_vmxon_region.done        ; 检查 CR0.PE 位是否相等

;;
;; 如果 CR0.PE 与 CR0.PG 位相符, 设置 CR0 其他位
;;
or ebx, [ebp + PCB.Cr0Fixed0]          ; 设置 Fixed 1 位
and ebx, [ebp + PCB.Cr0Fixed1]         ; 设置 Fixed 0 位
REX.Wrxb
mov cr0, ebx                          ; 写回 CR0
;;
;; 直接设置 CR4 fixed 1 位
;;
REX.W
mov ecx, cr4
or ecx, [ebp + PCB.Cr4FixedMask]       ; 设置 Fixed 1 位
and ecx, [ebp + PCB.Cr4Fixed1]         ; 设置 Fixed 0 位
REX.W
mov cr4, ecx

```

代码设置流程如下:

(1) 检查 CR0 寄存器的 PE 与 PG 位是否满足条件 (在开启分页的保护模式下), 通过与 Cr0FixedMask 值进行比较, 如果 PG 和 PE 位与 Cr0FixedMask 中的 PG 和 PE 值不同, 则失败返回。此错误不能在当前函数修复。

(2) 如果 CR0 满足上面的基本条件, 则直接设置 CR0 的 Fixed to 1 (固定 1) 位与 Fixed to 0 (固定 0) 位。这里的 Cr0Fixed0 与 Cr0Fixed1 的值来自于 IA32_VMX_CR0_FIXED0 与 IA32_VMX_CR0_FIXED1 寄存器。

(3) CR4 可以直接设置, 通过“或”上 Cr4FixedMask 值, 然后“与”上 Cr4Fixed1 值, 得到最终的 CR4 的值。

CR0 与 CR4 寄存器设置的算法是一样的, 通过“或”固定为 1 值, 然后“与”固定为 0 值, 得出最后需要的值。

根据 CR0 寄存器的限制, 处理器在 VMX operation 模式 (包括 **VMX root-operation** 与 **VMX non-root operation** 模式) 必须要开启分页保护模式, 那么 guest 软件必须运行在分页保护模式, 在进入 guest 端时, 处理器会检查是否符合这个要求。当不符合这个制约条件时, 将产生 VM-entry 失败。如果需要 guest 软件执行实模式代码, 那么可以选择进入 virtual-8086 模式。

后续的处理可能支持在 VMX non-root operation 模式下使用实模式, 软件需要检测是否支持 “**unrestricted guest**” (不受限制的 guest) 功能。在支持并开启 “unrestricted

guest”功能后，guest 软件允许运行在**实模式**或者**未分页的保护模式**。

在这种情况下，进入 guest 时，处理器将不会检查 CR0.PE 与 CR0.PG 控制位（参见第 4.5.1 节），而忽略 CR0 的 Fixed to 1（固定为 1）中的相应位。但是，如果 CR0.PG 为 1，则 CR0.PE 必须为 1。

2.3.2.3 A20M 模式

还有一个制约就是：当处理器处于 A20M 模式（即 A20 线 mask 模式），也不允许进入 VMX operation 模式，执行 VMXON 指令将引发 #GP 异常。A20M 模式会屏蔽 A20 地址线产生所谓的“wrapping”现象，从而模拟 8086 处理器的 1M 地址内访问行为。

例子的 boot 阶段就调用了 FAST_A20_ENABLE 宏来开启 A20 地址线（即 A20 线 unmask），这个宏实现在 inc\cpu.inc 文件里。

2.3.3 设置 VMXON 区域

在虚拟化平台中，可能只有一份 VMM 存在，但可以有多份 VM 实例存在。每个 VM 需要有对应的 VMCS（虚拟机控制结构）区域来控制，而 VMM 本身也需要一个 VMXON 区域来进行一些记录或者维护工作。

```
vmxon [ebp + PCB.VmxonPhysicalPointer]
```

如上面代码所示，执行 VMXON 指令，需要提供一个 VMXON 指针作为操作数，这个指针是物理地址，指向 VMXON 区域。

2.3.3.1 分配 VMXON 区域

在执行 VMXON 指令进入 VMX operation 模式前，需要分配一块物理内存区域作为 VMXON 区域。这块物理内存区域需要对齐在 4K 字节边界上。VMXON 区域的大小和内存 cache 类型可以通过检查 IA32_VMX_BASIC 寄存器来获得。

代码片段 2-6:

```
;;
;; 分配 VMXON region
;;
call get_vmcs_access_pointer          ; edx:eax = pa:va
REX.Wrxb
mov [ebp + PCB.VmxonPointer], eax
REX.Wrxb
mov [ebp + PCB.VmxonPhysicalPointer], edx
```

如上代码所示，在初始化 VMXON 区域阶段 initialize_vmxon_region 函数里调用 get_vmcs_access_pointer 来分配 VMXON 区域，返回的物理地址保存在 PCB.VmxonPhysicalPointer 值里，虚拟地址保存在 PCB.VmxonPointer 值里。

2.3.3.2 VMXON 区域初始设置

VMXON 区域的首 8 个字节结构与 VMCS 区域是一样的，首 4 个字节为 VMCS ID

值，下一个 DWORD 位置是 VMX-abort indicator 字段，存放 VMX-abort 发生后的 ID 值，如图 2-3 所示。

Byte Offset	Contents
0	Bits 30:0: VMCS revision identifier Bit 31: shadow-VMCS indicator
4	VMX-abort indicator
8	VMCS data (implementation-specific format)

图 2-3

这个 **VMCS ID** 值可以从 IA32_VMX_BASIC[31:0]来获得，执行 VMXON 指令前必须将这个 VMCS ID 值写入 VMXON 区域首 DWORD 位置。如果 VMCS ID 值与处理器当前 VMX 版本下的 VMCS ID 值不符，则产生 VMfailInvalid 失败（参见第 2.6.2 节），此时 CF=1，指示 VMCS 指针无效。

代码片段 2-7：

```
;;
;; 设置 VMCS region 信息
;;
REX.Wrb
mov ebx, [ebp + PCB.VmxonPointer]
mov eax, [ebp + PCB.VmxBasic]           ; 读取 VMCS revision identifier 值
mov [ebx], eax                          ; 写入 VMCS ID
```

上面代码来自 initialize_vmxon_region()函数，从 PCB.VmxBasic 里读取 32 位的 revision ID 值，然后写入 VMXON region 的首 4 个字节位置。PCB.VmxBasic 值在 stage1 阶段执行 get_vmx_global_data 函数时，从 IA32_VMX_BASIC 寄存器里读取。

注意：由于 VMXON 区域是物理内存区域，那么在设置 VMXON 区域之前，必须将 VMXON 区域映射到一个虚拟地址，然后通过虚拟地址进行写入操作。

代码中的 PCB.VmxonPointer 就是 VMXON 区域对应的虚拟地址指针，而 VmxonPhysicalPointer 则表示物理地址指针，这个物理指针使用在 VMXON 指令里。

2.3.4 退出 VMX operation 模式

处理器在 VMX operation 模式里不允许关闭 CR4.VMXE 位，只能在 VMX operation 模式外进行关闭，软件执行 VMXOFF 指令将退出 VMX operation 模式。

代码片段 2-8：

```
;;
;; 检查是否开启 VMX 模式
;;
test DWORD [ebp + PCB.ProcessorStatus], CPU_STATUS_VMXON
jz vmx_operation_exit.done
vmxoff
;;
;; 检查是否成功
```

```

;; 当 CF = 0 且 ZF = 0 时, VMXOFF 执行成功
;;
mov eax, STATUS_VMXOFF_UNSUCCESS
jc vmx_operation_exit.done
jz vmx_operation_exit.done

;;
;; 下面关闭 CR4.VMXE 标志位
;;
REX.Wrxb
mov eax, cr4
btr eax, 13
REX.Wrxb
mov cr4, eax

;;
;; 更新指令状态
;;
and DWORD [ebp + PCB.InstructionStatus], ~INST_STATUS_VMX
;;
;; 更新处理器状态
;;
and DWORD [ebp + PCB.ProcessorStatus], ~CPU_STATUS_VMXON

```

上面的代码片段来自 lib\VMX\VmxInit.asm 里的 vmx_operation_exit 函数，执行退出 VMX operation 工作：

- (1) 通过处理器状态标志检查当前是否处于 VMX operation 模式。
- (2) 执行 VMXOFF 指令退出 VMX operation 模式。
- (3) 关闭 CR4.VMXE 位。
- (4) 更新处理器状态标志位与指令状态标志位。

在执行 VMXOFF 指令后，必须检查指令是否成功。当 CF=1 时，指示当前的 VMCS 指针无效，当 ZF=1 时，指示 VMXOFF 指令执行遇到错误。只有当 CF 与 ZF 标志同时为 0 时，才表示 VMXOFF 是成功的！

在 VMX 开启 SMM dual-monitor treatment（SMM 双重监控处理）机制的情况下，必须先关闭 SMM 双重监控处理机制，才能使用 VMXOFF 指令关闭 VMX 模式。否则 VMXOFF 指令将会失败。

2.4 VMX operation 模式切换

进入 VMX operation 模式后，VMM 运行在 root 环境里，而 VM 需要运行在 non-root 环境里。虚拟化平台中经常会发生从 VMM 进入 VM，或者从 VM 返回到 VMM 的情况。有两个术语来描述它们之间的切换。

- **VM entry**（VM 进入）：从 VMX root operation 切换到 VMX non-root operation 就是 VM entry，表示从 VMM 进入到 VM 执行 guest 软件。
- **VM exit**（VM 退出）：从 VMX non-root operation 切换到 VMX root operation 就是

VM exit，表示从 VM 返回到 VMM。VMM 接管工作，VM 失去处理器控制权。

VM 的 entry 与 exit，顾名思义，是以 VM 的角度来定义。如果处理器要执行 guest 软件，那么 VMM 需要发起 VM-entry 操作，切换到 VMX non-root operation 模式执行。VM 会获得控制权，直到某些引发 VM exit 的事件发生。

VM-exit 发生后，处理器控制权重新回到 VMM。VMM 设置“VM exit”退出的条件是基于虚拟化处理器目的。因此，VMM 需要检查 VM 遇到了什么事件退出，从而虚拟化某些资源，返回一个虚拟化后的结果给 guest 软件，然后再次发起 VM-entry，切入 VM 让 guest 软件继续执行。

首次进入 VM 和退出后再次进入 VM 恢复执行，使用不同的指令进行。VMLAUNCH 指令发起首次进入 VM，VMRESUME 指令恢复被中断的 VM 执行。

图 2-4 摘自《Intel64 and IA-32 Architectures Software Developer's Manual》手册 Vol.3C23-2 页的图，很清晰地描述了 VMM 与 VM (guest) 之间的切换关系。利用 VM entry 与 VM exit 行为，VMM 在多个 VM 之间来回切换。这个过程类似于 OS 对进程的调度，而每个 VM 就像一个进程，VMM 其中的一个职能类似于 OS 进程调度器。

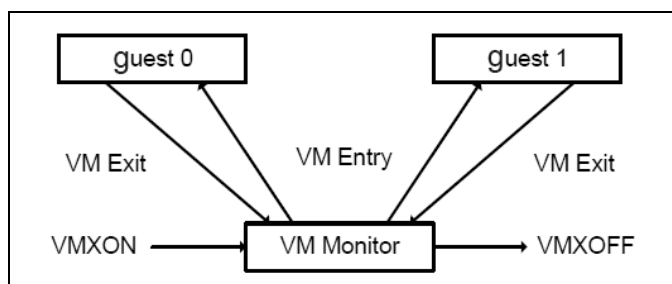


图 2-4

2.4.1 VM entry

首次进入 VM 环境，VMM 使用 VMLAUNCH 指令发起，VMLAUNCH 指令隐式地使用当前 VMCS 指针作为操作数。在发起 VM entry 操作前，VMM 需要对 VM 环境进行一些必要的配置工作，当前的 VMCS 指针（参见第 3.1.2 节）也必须先装载，下面是进入 VM 的简单流程。

(1) 分配一个物理内存区域作为 VMCS 区域，这个区域需要在 4K 字节边界上对齐，并且需要满足内存的 cache 类型。区域的大小以及支持的 cache 类型，需要在 IA32_VMX_BASIC 寄存器里查询获得。

(2) 写入 VMCS ID 值到这个 VMCS 区域的首 4 个字节里，这个 VMCS ID 值同样需要在 IA32_VMX_BASIC 寄存器里得到。

(3) 提供这个 VMCS 的物理指针作为操作数，执行 VMCLEAR 指令，这个操作将设置 VMCS 的 launch 状态为“clear”，处理器置当前 VMCS 指针为

FFFFFFFF_FFFFFFFFh 值。VMCS 的 launch 状态和当前指针值由处理器动态维护，《Intel 手册》没明确说明它们存放在哪里，但推断应该是存放在 VMXON 区域内（进入 VMX operation 模式时被 VMXON 指令使用）。

(4) 提供这个 VMCS 物理指针作为操作数，执行 VMPTRLD 指令，将装载这个 VMCS 指针作为当前的 VMCS 指针，处理器会维护这个 VMCS 指针。

(5) 对 VMCS 区域的初始化设置，执行一系列的 VMWRITE 指令，将必要的数写入当前 VMCS 指针指向的区域内（current-VMCS 区域）。VMWRITE 指令需要提供 VMCS 区域的一个 Index 值，这个 Index 值将用来指向 VMCS 区域的数据位置（参见第 3.3 节）。

(6) 在第 5 步初始化 VMCS 区域完成后，执行 VMLAUNCH 指令进入 VM 环境，成功后处理器将切换到 VMX non-root operation 模式运行。guest 软件执行的进入点由 VMCS 初始化时写入 guest state area（guest 状态区域）的 RIP 字段里（参见第 3.8 节）。

需要注意的是，每次执行 VMX 指令都需要检查 CF 与 ZF 标志，确定指令是否成功，除非你能确保指令是成功的。例如，执行一系列的 VMREAD 与 VMWRITE 指令来读写 VMCS 区域，需要确保当前 VMCS 指针是正确有效的，并且提供的 Index 值在 VMCS 区域是存在的，这样大可不必在每次读写执行后进行烦琐的检查。

当 VM exit 产生后，VMM 需要重新进入 VM 环境，那么需要使用 VMRESUME 指令来恢复 VM 的运行。VMRESUME 指令也是隐式使用当前 VMCS 指针作为操作数，但是 VMRESUME 指令执行的假定前提是由于 VM 退出后再次进入 VM。因此，VMCS 的 launch 状态必须为“**launched**”，区别于首次进入 VM 时的 launch 状态为“**clear**”（执行 VMCLEAR 指令后的结果）。

2.4.2 VM exit

在 VM 中，guest 软件无法知道自己是否处于 VM 之中（Intel 保证 guest 软件没有任何途径可以检测）。因此，guest 软件不可能主动放弃控制权进行“VM exit”操作。

只有 guest 软件遇到一些**无条件 VM exit 事件**或者**VMM 的设置引发 VM exit 的条件**发生，VM 才在不知不觉中失去了控制，VMM 将接管工作。guest 软件也无法知道自己什么时候发生了 VM exit 行为。

导致 VM exit 发生的三大类途径如下。

(1) 执行无条件引发 VM exit 的指令，包括 CPUID、GETSEC、INVD 与 XSETBV 指令，以及所有 VMX 指令（除了 VMFUNC 指令外）。

(2) 遇到无条件引发 VM exit 的**未被阻塞**的事件。例如，INIT 信号、SIPI 消息等。

(3) 遇到 VMM 设置引发 VM exit 的条件，包括执行某些指令或者遇到某些事件发生。譬如，VMM 设置了“HLT exiting”条件，而 guest 软件执行了 HLT 指令而引发 VM

exit。又如，VM 遇到了 external-interrupt 的请求，VMM 设置了“external-interrupt exiting”条件而导致 VM exit。

一些事件的发生能无条件地导致 VM exit 发生。例如，triple fault（三重 fault 异常）事件和接收到 INIT 信号与 SIPI 信号，以及在使用 EPT（扩展页表）机制的情况下，遇到了 EPT violation（EPT 违例）或 EPT misconfiguration（EPT 配置不当）发生也能引发 VM exit。

后面将讲到，INIT 信号与 SIPI 信号在 virtual processor（虚拟处理器）的一些状态下能被阻塞，这里所说的“**虚拟处理器**”是指进入 VM 环境后的处理器，因为它的状态及资源可以被 VMM 设置，而呈现出一个**虚拟**的概念，并非指虚拟的处理器。虚拟处理器在 VM entry 完成后，它的活动状态被加载为 VMCS 区域的“guest state area”内的 Activity state 字段设置的状态值（参见第 4.17 节）。

虚拟处理器在 wait-for-SIPI 状态下 INIT 信号将被阻塞，不会产生 VM-exit。而 SIPI 信号能导致 VM exit 也仅仅当虚拟处理器处于 wait-for-SIPI 状态之下时才有效。INIT 与 SIPI 信号在 VMX root-operation 模式下都被阻塞（也就是在 VMM 下），它们被忽略。

在其他状态下（包括 active、HLT 以及 shutdown 状态），虚拟处理器接收到 INIT 信号将无条件产生 VM-exit。当发生 triple fault 事件退出时，VMM 可以选择将虚拟处理器置为 shutdown 状态。

在 OS 里的进程执行中，当时间片用完，需要被切换出控制权。而在 VT-x 技术里，除了因 guest 遇到可引发 VM exit 产生的事件而导致 VM exit 外，类似地，VMM 也可以为每个 VM 设置一个“时间片”，时间片用完导致 VM exit 发生。

新近的 VMX 架构也引入了 **VMX-preemption timer** 机制，VMM 设置了启用“activate VMX-preemption timer”功能，提供一个“VMX-preemption timer value”作为计数值，在 VM entry 切换进行时，计数值就开始递减 1，当这个计数值减为 0 时引发 VM exit。

这个 preemption timer 计数值递减的步伐依赖于 TSC 与 IA32_VMX_MISC[4:0]值，我们将在后面探讨。

2.4.3 SMM 双重监控处理下

在 SMM dual-monitor treatment 机制下，VMX 定义了另外两类的 VM exit 与 VM entry，它们是“**SMM VM-exit**”与“**VM-entry that return from SMM**”（从 SMM 返回中进入）。

- **SMM VM-exit**，可以从 VMM（VMX root-operation）或者 VM（VMX non-root operation）中产生 VM 退出行为，然后进入 SMM 模式执行被称为“**SMM-transfer Monitor**”（切入 SMM 监控者）的代码。
- **VM-entry that return from SMM**，将从 SMM 模式退出，然后返回到原来的

VMM 或 VM 中继续执行。

这个 SMM 双重监控处理是使用 VMM 的两端代码：VMX 端以及 SMM 端。也就是说，SMM 模式下也有 VMM 代码在运行。当发生 SMI (System manage interrupt) 请求时，在 SMM 双重监控处理机制下，VMM 将从 **VMX 模式** 切入 **SMM 模式**，然后执行 SMM 模式里的代码。

VMM 在 VMX 端的代码被称为“**Executive monitor**”，在 SMM 端的代码被称为“**SMM-transfer monitor**”。执行在 VMX 端时使用的区域被叫作“**executive VMCS**”，而 SMM 端使用的是“**SMM-transfer VMCS**”。

2.5 VMX 能力的检测

VMX 架构中的许多功能可能在不同的处理器架构里有不同的支持，新加入的 VMX 特性也可能在新近的处理器才支持，允许使用较早前的处理器不支持的一些非常实用的功能，比如前面所说的 VMX-preemption timer 功能。因此，VMM 需要检测当前 VMX 架构下有什么能力，进行相应的设置。

提醒：本书在书写时主要参照的是《Intel 开发者手册》2012 年 8 月版本，order number 为：**325462-044US**。VMX 中的一些特性在笔者的电脑上并不支持，敬请读者注意。

2.5.1 检测是否支持 VMX

软件需要检测处理器是否支持 VMX 架构，使用 CPUID.01H:ECX.VMX[5]位来进行检测（详见第 2.2.3 节）。

2.5.2 通过 MSR 组检查 VMX 能力

VMX 架构提供了众多项目能力的检测，包括：VMX 的基本信息、杂项信息、VPID 与 EPT 能力，还有对 VMCS 内的 control fields（控制字段）允许设置的位。而控制字段的位允许被置为 1 时，代表着处理器拥有这个能力。

譬如，secondary processor-based control 字段的 bit 7 是“Unrestricted guest”位，当它允许被置 1 时，表明处理器支持 unrestricted guest（不受限制的 guest 端）功能。反之，则表示处理器不支持该功能。

VMX 的这些能力的检测提供在几组共 14 个 MSR (Model Specific Register) 里，如表 2-1 所示，除了这些，还有 4 个扩展了的 TRUE 系列寄存器，详见第 2.5.4 节和第 2.5.5 节。

表 2-1

序号	寄存器	描述
1	IA32_VMX_BASIC	提供 VMX 基本信息
2	IA32_VMX_PINBASED_CTLS	决定 Pin-based 控制字段
3	IA32_VMX_PROCBASED_CTLS	决定 Processor-based 控制字段
4	IA32_VMX_EXIT_CTLS	决定 VM-exit 控制字段
5	IA32_VMX_ENTRY_CTLS	决定 VM-entry 控制字段
6	IA32_VMX_MISC	提供 VMX 杂项信息
7	IA32_VMX_CR0_FIXED0	决定 CR0 的固定位
8	IA32_VMX_CR0_FIXED1	
9	IA32_VMX_CR4_FIXED0	决定 CR4 的固定位
10	IA32_VMX_CR4_FIXED1	
11	IA32_VMX_VMCS_ENUM	决定 VMCS Index 值
12	IA32_VMX_PROCBASED_CTLS2	决定 secondary Pin-based 控制字段
13	IA32_VMX_EPT_VPID_CAP	决定 EPT 及 VPID 能力
14	IA32_VMX_VMFUNC	决定 VM-function 控制字段

软件通过读取这些寄存器的值来确定，在本书例子中，对这些寄存器的读取在 stage1 阶段执行的 `get_vmx_global_data` 函数里完成。

代码片段 2-9:

```

-----
; get_vmx_global_data()
; input:
;     none
; output:
;     none
; 描述:
;     1) 读取 VMX 相关信息
;     2) 在 stage1 阶段调用
;-----
get_vmx_global_data:
    push ecx
    push edx

    ;;
    ;; vmxGlobalData 区域
    ;;
    mov edi, [gs: PCB.PhysicalBase]
    add edi, PCB.VmxGlobalData

    ;;
    ;; ### step 1: 读取 VMX MSR 值 ###
    ;; 1) 当 CPUID.01H:ECX[5]=1 时, IA32_VMX_BASIC 到 IA32_VMX_VMCS_ENUM 寄存器有效
    ;; 2) 首先读取 IA32_VMX_BASIC 到 IA32_VMX_VMCS_ENUM 寄存器值
    ;;
    mov esi, IA32_VMX_BASIC

```

```

get_vmx_global_data.@1:
    mov ecx, esi
    rdmsr
    mov [edi], eax
    mov [edi + 4], edx
    inc esi
    add edi, 8
    cmp esi, IA32_VMX_VMCS_ENUM
    jbe get_vmx_global_data.@1

;;
;; ### step 2: 接着读取 IA32_VMX_PROCBASED_CTLs2 ###
;; 1) 当 CPUID.01H:ECX[5]=1, 并且 IA32_VMX_PROCBASED_CTLs[63] = 1 时,
;; IA32_VMX_PROCBASED_CTLs2 寄存器有效
;;
test DWORD [gs: PCB.ProcessorBasedCtls + 4], ACTIVATE_SECONDARY_CONTROL
jz get_vmx_global_data.@5

mov ecx, IA32_VMX_PROCBASED_CTLs2
rdmsr
mov [gs: PCB.ProcessorBasedCtls2], eax
mov [gs: PCB.ProcessorBasedCtls2 + 4], edx

;;
;; ### step 3: 接着读取 IA32_VMX_EPT_VPID_CAP
;; 1) 当 CPUID.01H:ECX[5]=1, IA32_VMX_PROCBASED_CTLs[63]=1, 并且
;; IA32_VMX_PROCBASED_CTLs2[33]=1 时, IA32_VMX_EPT_VPID_CAP 寄存器有效
;;
test edx, ENABLE_EPT
jz get_vmx_global_data.@5

mov ecx, IA32_VMX_EPT_VPID_CAP
rdmsr
mov [gs: PCB.EptVpidCap], eax
mov [gs: PCB.EptVpidCap + 4], edx

;;
;; ### step 4: 读取 IA32_VMX_VMFUNC ###
;; 1) IA32_VMX_VMFUNC 寄存器仅在支持 "enable VM functions" 1-setting 时有效, 因此需要检测是否支持!
;; 2) 检查 IA32_VMX_PROCBASED_CTLs2[45] 是否为 1 值
;;
test DWORD [gs: PCB.ProcessorBasedCtls2 + 4], ENABLE_VM_FUNCTION
jz get_vmx_global_data.@5

mov ecx, IA32_VMX_VMFUNC
rdmsr
mov [gs: PCB.VmFunction], eax
mov [gs: PCB.VmFunction + 4], edx

get_vmx_global_data.@5:

;;
;; ### step 5: 读取 4 个 VMX TRUE capability 寄存器 ###
;;
;; 如果 bit55 of IA32_VMX_BASIC 为 1, 支持 4 个 capability 寄存器:
;; 1) IA32_VMX_TRUE_PINBASED_CTLs = 48Dh
;; 2) IA32_VMX_TRUE_PROCBASED_CTLs = 48Eh

```

```

;; 3) IA32_VMX_TRUE_EXIT_CTL5      = 48Fh
;; 4) IA32_VMX_TRUE_ENTRY_CTL5     = 490h
;;
bt DWORD [gs: PCB.VmxBasic + 4], 23
jnc get_vmx_global_data.@6

mov BYTE [gs: PCB.TrueFlag], 1                ; 设置 TrueFlag 标志位
;;
;; 如果支持 TRUE MSR, 那么就更新下面的 MSR:
;; 1) IA32_VMX_PINBASED_CTL5
;; 2) IA32_VMX_PROCBASED_CTL5
;; 3) IA32_VMX_EXIT_CTL5
;; 4) IA32_VMX_ENTRY_CTL5
;; 用 TRUE MSR 的值替代上面的 MSR!
;;
mov ecx, IA32_VMX_TRUE_PINBASED_CTL5
rdmsr
mov [gs: PCB.PinBasedCtl5], eax
mov [gs: PCB.PinBasedCtl5 + 4], edx
mov ecx, IA32_VMX_TRUE_PROCBASED_CTL5
rdmsr
mov [gs: PCB.ProcessorBasedCtl5], eax
mov [gs: PCB.ProcessorBasedCtl5 + 4], edx
mov ecx, IA32_VMX_TRUE_EXIT_CTL5
rdmsr
mov [gs: PCB.ExitCtl5], eax
mov [gs: PCB.ExitCtl5 + 4], edx
mov ecx, IA32_VMX_TRUE_ENTRY_CTL5
rdmsr
mov [gs: PCB.EntryCtl5], eax
mov [gs: PCB.EntryCtl5 + 4], edx

get_vmx_global_data.@6:
;;
;; ### step 6: 设置 CR0 与 CR4 的 mask 值 (固定为 1 值)
;; 1) Cr0FixedMask = Cr0Fixed0 & Cr0Fixed1
;; 2) Cr4FixedMask = Cr4Fixed0 & Cr4Fixed1
;;
mov eax, [gs: PCB.Cr0Fixed0]
mov edx, [gs: PCB.Cr0Fixed0 + 4]
and eax, [gs: PCB.Cr0Fixed1]
and edx, [gs: PCB.Cr0Fixed1 + 4]
mov [gs: PCB.Cr0FixedMask], eax                ; CR0 固定为 1 值
mov [gs: PCB.Cr0FixedMask + 4], edx
mov eax, [gs: PCB.Cr4Fixed0]
mov edx, [gs: PCB.Cr4Fixed0 + 4]
and eax, [gs: PCB.Cr4Fixed1]
and edx, [gs: PCB.Cr4Fixed1 + 4]
mov [gs: PCB.Cr4FixedMask], eax                ; CR4 固定为 1 值
mov [gs: PCB.Cr4FixedMask + 4], edx

;;
;; 关于 IA32_FEATURE_CONTROL.lock 位:
;; 1) 当 lock = 0 时, 执行 VMXON 产生 #GP 异常
;; 2) 当 lock = 1 时, 写 IA32_FEATURE_CONTROL 寄存器产生 #GP 异常
;;
;;
;;
;; 下面将检查 IA32_FEATURE_CONTROL 寄存器

```

```

;; 1) 当 lock 位为 0 时, 需要进行一些设置, 然后锁上 IA32_FEATURE_CONTROL
;; 2) 检查 IA32_FEATURE_CONTROL 的 bit1 与 bit2 位, 决定 Cr4FixedMask 值
;;
mov ecx, IA32_FEATURE_CONTROL
rdmsr
bts eax, 0 ; 检查 lock 位, 并上锁
jc get_vmx_global_data.@7

;; lock 未上锁时:
;; 1) 对 lock 置位 (锁上 IA32_FEATURE_CONTROL 寄存器)
;; 2) 对 bit 2 置位 (启用 enable VMXON outside SMX)
;; 3) 对 bit 1 清位 (关闭 enable VMXON inside SMX)
;; 因此, 此时 VMXON 指令需要在 outside SMX 下执行!
;;
bts eax, 2 ; 置 enable VMX outside SMX = 1
btr eax, 1 ; 清 enable VMX inside SMX = 0
wrmsr

get_vmx_global_data.@7:
;;
;; ### step 7: 设置 Cr4FixedMask 的 CR4.SMXE 位 ###
;;
;; 再次读取 IA32_FEATURE_CONTROL 寄存器
;; 1) 检查 enable VMX inside SMX 位 (bit1)
;; 2) 判断 VMX operation 是执行在 inside SMX 还是 outside SMX
;; 2.1) 如果是 inside SMX (即 bit1 = 1), 则设置 CR4FixedMask 位的相应位
;;
;; 如果 enable VMX inside SMX 位的值为 1, 则表明必须开启 CR4.SMXE 位 (即开启 SMX 模
式)
;;
rdmsr
and eax, 2 ; 取 enable VMX inside SMX 位的值 (bit1)
shl eax, 13 ; 对应在 CR4 寄存器的 bit 14 位 (即 CR4.SMXE 位)
or DWORD [ebp + PCB.Cr4FixedMask], eax ; 设置 enable VMX inside SMX 位的值

get_vmx_global_data.@8:
;;
;; ### step 8: 查询 vmcs 以及 access page 的内存 cache 类型 ###
;; 1) VMCS 区域内存类型
;; 2) VMCS 内的各种 bitmap 区域, access page 内存类型
;;
mov eax, [gs: PCB.VmxBasic + 4]
shr eax, 50-32 ; 读取 IA32_VMX_BASIC[53:50]
and eax, 0Fh
mov [gs: PCB.VmcsMemoryType], eax

get_vmx_global_data.@9:
;;
;; ### step 9: 检查 VMX 所支持的 EPT page memory attribute ###
;; 1) 如果支持 WB 类型则使用 WB, 否则使用 UC
;; 2) 在 EPT 设置 memory type 时, 直接或上 [gs: PCB.EptMemoryType]
;;
mov esi, MEM_TYPE_WB ; WB
mov eax, MEM_TYPE_UC ; UC
bt DWORD [gs: PCB.EptVpidCap], 14
cmovnc esi, eax
mov [gs: PCB.EptMemoryType], esi

```

```
get_vmx_global_data.done:
    pop edx
    pop ecx
    ret
```

这个 `get_vmx_global_data` 函数实现在 `lib\system_data_manage.asm` 文件里，在 `stage1` 阶段期间执行的 `update_processor_basic_info` 函数内部将调用这个函数来收集 VMX 的相关信息。

共分为 9 个步骤读取 VMX 的相关信息，其中一个额外的工作是设置前面第 2.3.2.1 节里提及的 `IA32_FEATURE_CONTROL` 寄存器。

2.5.3 例子 2-1

示例 2-1：列举出其中一个处理器 VMX 提供的能力信息

在继续讲解 VMX 能力之前我们先做个例子，将收集的 VMX 相关信息报告出来，例子主体代码在 `chap02\ex2-1\ex.asm` 文件里，如下所示。

代码片段 2-10：

```
    call get_usable_processor_index        ; 获取可用的处理器 index 值
    mov esi, eax                          ; 目标处理器为获取的处理器
    mov edi, TargetCpuVmxCapabilities     ; 目标代码
    mov eax, signal                       ; signal
    call dispatch_to_processor_with_waitting ; 调度到目标处理器执行

    ;;
    ;; 等待 CPU 重启
    ;;
    call wait_esc_for_reset

;-----
; TargetCpuVmxCapabilities()
; input:
;     none
; output:
;     none
; 描述:
;     1) 调度执行的目标代码
;-----
TargetCpuVmxCapabilities:
    call update_system_status             ; 更新系统状态
    call println

    ;;
    ;; 打印 VMX capabilities 信息
    ;;
    call dump_vmx_capabilities
    ret

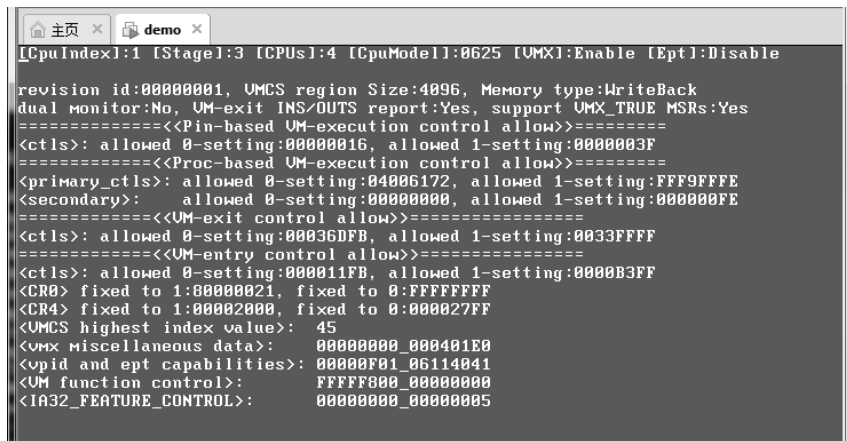
signal dd 1
```

这段代码找到一个可用的处理器 `Index` 值后，`get_usable_processor_index` 函数一般从

1 开始查询处理器是否可用。因此，一般会返回 1 值，也就是 CPU1。然后调度 TargetCpuVmxCapabilities 函数给这个处理器执行，TargetCpuVmxCapabilities 函数的工作只是简单地打印 CPU1 的 VMX 能力信息。dump_vmx_capabilities 函数代码实现在 lib\vmx\VmxDump.asm 文件里。

调度函数 dispatch_to_processor_with_waiting 需要提供一个信号指针，调用者需要等待目标处理器完成 TargetCpuVmxCapabilities 函数后才返回。它需要使用这个信号值进行等待。

使用命令 build -D __X64 编译该例子生成相应的映像文件后，在 VMware 里加载 demo.img（软盘映像）运行，该例子运行在 stage3 阶段。我们按下<F2>键切换到 CPU1，结果如图 2-5 所示。



```

[CpuIndex]:1 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable
revision id:00000001, VMCS region Size:4096, Memory type:WriteBack
dual monitor:No, VM-exit INS/OUTS report:Yes, support VMX_TRUE MSR:Yes
=====
<ctl>: allowed 0-setting:00000016, allowed 1-setting:0000003F
=====
<Proc-based VM-exit control allow>=====
<primary_ctl>: allowed 0-setting:04006172, allowed 1-setting:FFF9FFFE
<secondary>: allowed 0-setting:00000000, allowed 1-setting:000000FE
=====
<ctl>: allowed 0-setting:00036DFB, allowed 1-setting:0033FFFF
=====
<VM-exit control allow>=====
<ctl>: allowed 0-setting:000011FB, allowed 1-setting:0000B3FF
<CR0> fixed to 1:00000021, fixed to 0:FFFFFFFF
<CR4> fixed to 1:00002000, fixed to 0:000027FF
<VMCS highest index value>: 45
<VMX Miscellaneous data>: 00000000_000401E0
<vpid and ept capabilities>: 00000F01_06114041
<VM function control>: FFFF800_00000000
<IA32_FEATURE_CONTROL>: 00000000_00000005
  
```

图 2-5

如图 2-5 所示，当前运行的 CpuIndex 值为 1（表示运行的是 CPU1），运行在 Stage3 阶段，已经进入了 VMX 模式，有 4 个逻辑处理器。

注意：这些 VMX 能力信息（capability information）是 VMware 提供的，并不代表真实机器具有的 VMX 能力。在下面的 VMX 能力信息里，VMCS ID 为 00000001h，VMCS 区域的大小为 4K，支持 VMX TRUE 寄存器，接下来是各个控制位的 allowed 0-setting 与 allowed 1-setting 位值，CR0 与 CR4 寄存器的固定位及杂项寄存器的值。在笔者的电脑上，VMCS 区域的大小为 1K，VMCS ID 值为 0FH。

VMCS index 的最大值为 45，IA32_FEATURE_CONTROL 寄存器的值为 05H，表示 lock 位上锁，使用了“VMX in outside SMX”模式。

2.5.4 基本信息检测

IA32_VMX_BASIC 寄存器用来检测 VMX 的基本能力信息，如图 2-6 所示。

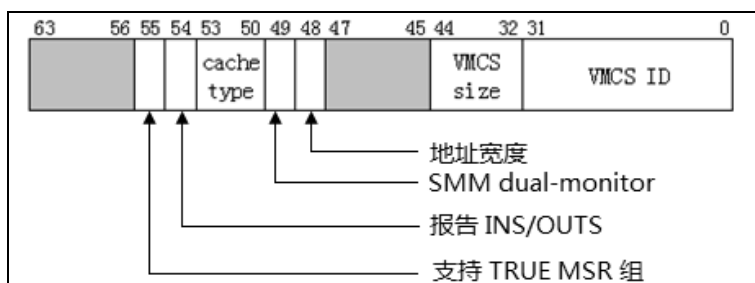


图 2-6

bits 31:0 为 VMCS ID 值，在初始化 VMXON 及 VMCS 区域时，需要用 VMCS ID 值来设置首 DWORD 位置。Bits 44:32 指示 VMCS 及 VMXON 区域的大小，这些区域大小以 1K 为单位，最高支持 4K 字节，并且地址需要在 4K 字节边界上。

bit 48 指示 VMXON 区域、VMCS 区域以及 VMCS 内所引用的区域（例如 I/O bitmap 区域、MSR bitmap 区域等）的宽度。为 1 时这些物理地址基址限定在 32 位内，为 0 时，物理地址宽度在 MAXPHYADDR 值内。在支持 64 位的平台上，bit 48 为 0 值。

bit 49 为 1 时，表明支持 SMM 及 SMI 的 dual-monitor treatment 功能。bit 54 为 1 时，表明支持当 VM-exit 是由于 INS 或 OUTS 指令而引发时，在 VMCS 的“VM-exit instruction information”字段里记录相应的信息。

bit 55 位为 1 时，表示支持 4 个 TRUE 寄存器，这 4 个 TRUE 寄存器将影响最终的 VMCS 中的某些控制位，如表 2-2 所示。

表 2-2

影响的 VMCS 字段	bit 55 = 0	bit 55 = 1
Pin-Based control	IA32_VMX_PINBASED_CTLS	IA32_VMX_TRUE_PINBASED_CTLS
Processor-Based control	IA32_VMX_PROCBASED_CTLS	IA32_VMX_TRUE_PROCBASED_CTLS
VM-exit control	IA32_VMX_EXIT_CTLS	IA32_VMX_TRUE_EXIT_CTLS
VM-entry control	IA32_VMX_ENTRY_CTLS	IA32_VMX_TRUE_ENTRY_CTLS

bit 55 的值决定由谁来控制这些 VMCS 字段固定位的设置。当 bit 55 为 0 时，这些 VMCS 字段的固定位值只需要分别受到 IA32_VMX_PINBASED_CTLS、IA32_VMX_PROCBASED_CTLS、IA32_VMX_EXIT_CTLS 及 IA32_VMX_ENTRY_CTLS 寄存器影响。为 1 时，则受对应的 TRUE 寄存器影响。

bit 53:50 这 4 位组成一个值，它指示 VMCS 区域以及 VMCS 区域内所引用的区域（例如 I/O bitmap，MSR bitmap 等）所支持的内存 cache 类型。

- 为 0 值时，支持 Uncacheable (UC) 类型
- 为 6 值时，支持 WriteBack (WB) 类型

VMX 目前只支持这两种 cache 类型，其他值（1~5 以及 7~15）都没使用（WC、WT、WP 及 UC-类型不支持）。在前面的 get_vmx_global_data 函数代码中读取了这个

cache 类型值，然后保存在 PCB.VmcsMemoryType 里。

IA32_VMX_BASIC 寄存器的其余位 (bits 47:45 以及 bits 63:56) 是保留位，为 0 值。

2.5.5 允许为 0 以及允许为 1 位

在表 2-2 里列举的两组寄存器（在 IA32_VMX_BASIC[55]=1 时使用 TRUE 寄存器），共 8 个寄存器，它们的结构和使用方法是一致的，结构如图 2-7 所示。

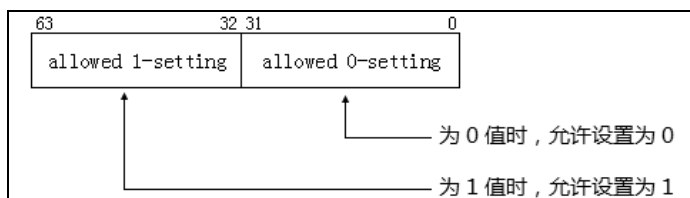


图 2-7

这些 64 位的寄存器低 32 位是 allowed 0-setting（允许设置为 0）位值，高 32 位是 allowed 1-setting（允许设置为 1）位值。这两组 32 位值对应一个 VMCS 区域的 control field（控制字段）值，这些控制字段是 32 位值，用法如下所示。

- bits 31:0 的用法：当其中的位为 0 值时，对应的控制字段相应的位允许为 0 值。
- bits 63:32 的用法：当其中的位为 1 值时，对应的控制字段相应的位允许为 1 值。

举个例子说明，在图 2-5 的运行结果里显示，IA32_VMX_TRUE_PINBASED_CTLs 的低 32 位值为 00000016h，高 32 位为 0000003Fh。它对应控制 Pin-based control 字段，那么表明：

(1) 00000016h，Pin-based control 字段除了 bit 1、bit 2 以及 bit 4（值为 16h）不能为 0 值外，其他位都可以设置为 0 值。

(2) 0000003Fh，Pin-based control 字段只允许 bit 0 到 bit 5 位可以设置 1 值，其余位必须为 0 值。

那么，形成 Pin-based control 字段的设置要求如图 2-8 所示。

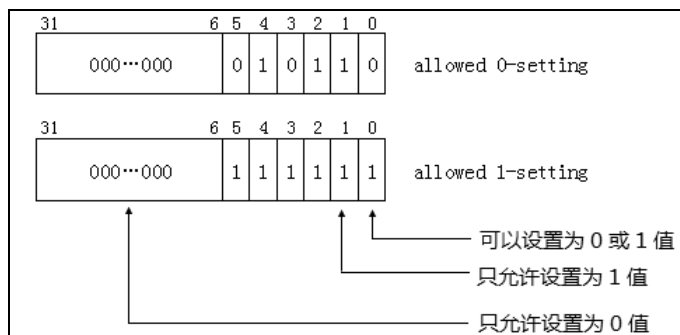


图 2-8

从上面的 allowed 0-setting 与 allowed 1-setting 的设置来看，bit 0 既可以为 0 值，也可以为 1 值。而 bit 1 只允许为 1 值。那么代表着 Pin-based control 字段的合法设置如下。

(1) bit 0、bit 3 以及 bit 5 可以设为 0 或 1 值。

(2) bit 1、bit 2 以及 bit 4 只能设为 1 值。

(3) bits 31:6 只能设为 0 值。

推广开来，表 2-2 中的 8 个寄存器，都使用相同的设置原理，不同的是设置的目标字段不一样。我们看到对于一个控制字段的设置：“某些位必须为 1，某些位必须为 0”（它们属于保留位）。

注意：这些必须为 0 值的保留位被称为“default0”位，必须为 1 值的保留位被称为“default1”位。这与通常接触到的数据结构中的“保留位必须为 0 值”有些区别（例如 PTE 中的保留位）。

当控制字段的保留位不符合这些 default0 和 default1 值，在 VM entry 操作时，字段的检查会失败，从而导致 VM entry 失败。

2.5.5.1 决定 VMX 支持的功能

前面所述，控制字段中必须为 0 及必须为 1 值的位都是保留位。当一个位可以设置为 1 值时，表明处理器将支持该项功能，也就是非保留位指示支持该位对应的功能。

以 Pin-based VM execution control 字段为例，它的 bit 6 是“activate VMX-preemption timer”功能，只有 bit 6 允许设置为 1 值时，才代表处理器支持 VMX-preemption timer 功能。在图 2-8 中，我们看到，bit 6 只能设置为 0 值。因此，这个处理器并不支持这项功能。

又如，该字段的 bit 0 为“external-interrupt exiting”功能位，而该位可以设置为 0 或 1 值，因而表明处理器支持该项功能。

2.5.5.2 控制字段设置算法

根据前面所述的 VMX 能力寄存器 allowed 0-setting 与 allowed 1-setting 位，对 VMCS 区域中相应的控制字段设置，可以分两步进行。首先根据自身的设计需求设置相应位，这是一个初始值。然后使用 allowed 0-setting 与 allowed 1-setting 位合成最终值。合成最终值的算法如下。

(1) Result1 = 初始值 “OR” allowed 0-setting 值

(2) Result2 = Result1 “AND” allowed 1-setting 值

这个 Result2 就是最终值，这个值确保满足在 VM-entry 时处理器对该控制字段的检查。下面的代码片段是对 Pin-based VM-execution control 字段的设置。

代码片段 2-11:

```
;;
;; 设置 Pin-based 控制域:
;; 1) [0] - external-interrupt exiting: Yes
```

```

;; 2) [3] - NMI exiting: Yes
;; 3) [5] - Virtual NMIs: No
;; 4) [6] - Activate VMX preemption timer: Yes
;; 5) [7] - process posted interrupts: No
;;
mov eax, EXTERNAL_INTERRUPT_EXITING | NMI_EXITING |
ACTIVATE_VMX_PREEMPTION_TIMER
or eax, [ebp + PCB.PinBasedCtls]           ; OR allowed 0-setting
and eax, [ebp + PCB.PinBasedCtls + 4]      ; AND allowed 1-setting

;;
;; 写入 Pin-based VM-execution control 值
;;
mov [ExecutionControlBufBase + EXECUTION_CONTROL.PinControl], eax

```

代码中对 Pin-based VM-execution control 字段的初始值设为 **49H**，然后依据上面的算法合成最终值。这是必须进行的一步，否则在执行 VM-entry 操作时，将由于检查 Pin-based VM-execution control 字段 default 位不满足要求而失败。

2.5.6 VM-execution 控制字段

在 VMCS 区域的三个 VM-execution control 字段需要检查 VMX 能力寄存器来确认 default0 与 default1 位。如下所示：

- Pin-based VM-execution control 字段
- primary processor-based VM-execution control 字段
- secondary processor-based VM-execution control 字段

在表 2-2 中列举了这些控制字段所支持的能力由其对应的寄存器提供。当 IA32_VMX_BASIC[55]为 1 时使用 TRUE 系列寄存器。此时，pin-based VM-execution control 字段的值由 IA32_VMX_TRUE_PINBASED_CTLs 寄存器来决定设置，primary processor-based VM-execution control 字段的值由 IA32_VMX_TRUE_PROCBASED_CTLs 寄存器来决定设置，但 secondary processor-based VM-execution control 字段没有对应的 TRUE 寄存器，它只由 IA32_VMX_PROCBASED_CTLs2 寄存器来决定设置。

2.5.6.1 Pin-based VM-execution control 字段

当 IA32_VMX_BASIC 的 bit 55 为 0 时，IA32_VMX_PINBASED_CTLs 寄存器决定 Pin-based VM-execution control 字段的设置。

当 IA32_VMX_BASIC 的 bit 55 为 1 时，忽略 IA32_VMX_PINBASED_CTLs 寄存器，Pin-based VM-execution control 字段由 IA32_VMX_TRUE_PINBASED_CTLs 寄存器来决定设置。

这两个 MSR 的用法是一样的，分为低 32 位与高 32 位。具体用法如下所示。

(1) bits 31:0 (allowed 0-setting)：某位为 0 时，Pin-based VM-execution control 字段相应的位允许设为 0 值。

(2) bits 63:32 (allowed 1-setting) : 某位为 1 时, Pin-based VM-execution control 字段相应的位允许设为 1 值。

Pin-based VM-execution control 字段属于 32 位值, 因此 MSR 的高 32 位 (allowed 1-setting) 也是对应 Pin-based VM-execution control 字段的 bits 31:0。

2.5.6.2 primary processor-based VM-execution control 字段

当 IA32_VMX_BASIC 的 bit 55 为 0 时, IA32_VMX_PROCBASED_CTLs 寄存器决定 primary processor-based VM-execution control 字段的设置。

当 IA32_VMX_BASIC 的 bit 55 为 1 时, 忽略 IA32_VMX_PROCBASED_CTLs 寄存器, primary processor-based VM-execution control 字段由 IA32_VMX_TRUE_PROCBASED_CTLs 寄存器来决定设置。

这两个 MSR 的用法是一样的, 分为低 32 位与高 32 位。具体用法如下所示。

(1) bits 31:0 (allowed 0-setting) : 某位为 0 时, primary processor-based VM-execution control 字段相应的位允许设为 0 值。

(2) bits 63:32 (allowed 1-setting) : 某位为 1 时, primary processor-based VM-execution control 字段相应的位允许设为 1 值。

primary processor-based VM-execution control 字段也是属于 32 位值, MSR 的高 32 位 (allowed 1-setting) 也是对应 primary processor-based VM-execution control 字段的 bits 31:0。

2.5.6.3 secondary processor-based VM-execution control 字段

由于 secondary processor-based VM-execution control 字段没有对应的 TRUE 寄存器。因此, 它只受到 IA32_VMX_PROCBASED_CTLs2 寄存器的影响。

(1) bits 31:0 (allowed 0-setting) : 某位为 0 时, secondary processor-based VM-execution control 字段相应的位允许设为 0 值。

(2) bits 63:32 (allowed 1-setting) : 某位为 1 时, secondary processor-based VM-execution control 字段相应的位允许设为 1 值。

secondary processor-based VM-execution control 字段也是属于 32 位值, 高 32 位的 allowed 1-setting 也是对应 secondary processor-based VM-execution control 字段的 bits 31:0。

2.5.7 VM-exit control 字段

当 IA32_VMX_BASIC 的 bit 55 为 0 时, IA32_VMX_EXIT_CTLs 寄存器决定 VM-exit control 字段的设置。

当 IA32_VMX_BASIC 的 bit 55 为 1 时, 忽略 IA32_VMX_EXIT_CTLs 寄存器, VM-exit control 字段由 IA32_VMX_TRUE_EXIT_CTLs 寄存器来决定设置。

这两个 MSR 的用法是一样的，分为低 32 位与高 32 位。具体用法如下所示。

(1) bits 31:0 (allowed 0-setting)：某位为 0 时，VM-exit control 字段相应的位允许设为 0 值。

(2) bits 63:32 (allowed 1-setting)：某位为 1 时，VM-exit control 字段相应的位允许设为 1 值。

VM-exit control 字段属于 32 位值，MSR 的高 32 位 (allowed 1-setting) 也是对应 VM-exit control 字段的 bits 31:0。

2.5.8 VM-entry control 字段

当 IA32_VMX_BASIC 的 bit 55 为 0 时，IA32_VMX_ENTRY_CTLs 寄存器决定 VM-entry control 字段的设置。

当 IA32_VMX_BASIC 的 bit 55 为 1 时，忽略 IA32_VMX_ENTRY_CTLs 寄存器。VM-entry control 字段由 IA32_VMX_TRUE_ENTRY_CTLs 寄存器来决定。

这两个 MSR 的用法是一样的，分为低 32 位与高 32 位。具体用法如下所示。

(1) bits 31:0 (allowed 0-setting)：某位为 0 时，VM-entry control 字段相应的位允许设为 0 值。

(2) bits 63:32 (allowed 1-setting)：某位为 1 时，VM-entry control 字段相应的位允许设为 1 值。

VM-entry control 字段属于 32 位值，MSR 的高 32 位 (allowed 1-setting) 也是对应 VM-entry control 字段的 bits 31:0。

2.5.9 VM-function control 字段

当 VMX 支持“enable VM functions”功能时，将提供 IA32_VMX_VMFUNC 寄存器来决定 VM-function control 字段哪些位可以置位。与其他控制字段不同，VM-function control 字段是 64 位值。

当下面的条件满足时，才支持 IA32_VMX_VMFUNC 寄存器：

(1) CPUID.01H:ECX[5]=1，表明支持 VMX 架构。

(2) IA32_VMX_PROCBASED_CTLs[63]=1，表明支持 IA32_VMX_PROCBASED_CTLs2 寄存器。

(3) IA32_VMX_PROCBASED_CTLs2[45]=1，表明支持“enable VM functions”功能。

IA32_VMX_VMFUNC 寄存器没有 allowed 0-setting 位，只有 allowed 1-setting 位，用法如下所示。

bits 63:0 (allowed 1-setting)：某位为 1 时，VM-function control 字段相应的位允许设为 1 值。

VM-function control 字段没有对应的 TRUE 寄存器，由 IA32_VMX_VMFUNC 寄存器最终决定设置。

2.5.10 CR0 与 CR4 的固定位

CR0 与 CR4 寄存器分别对应各自的 FIXED0 和 FIXED1 能力寄存器，如表 2-3 所示。

表 2-3

作 用	CR0 寄存器	CR4 寄存器
fixed to 1	IA32_VMX_CR0_FIXED0	IA32_VMX_CR4_FIXED0
fixed to 0	IA32_VMX_CR0_FIXED1	IA32_VMX_CR4_FIXED1

编号为 0 的 FIXED 寄存器指示固定为 1 值，编号为 1 的 FIXED 寄存器指示固定为 0 值。需要这两个寄存器 (FIXED0 和 FIXED1) 来控制，是由于在 64 位模式下，CR0 与 CR4 寄存器是 64 位的。因此，一个用来描述固定为 1 值，一个用来描述固定为 0 值。

- 当 FIXED0 寄存器的位为 1 值时，CR0 和 CR4 寄存器对应的位必须为 1 值。
- 当 FIXED1 寄存器的位为 0 值时，CR0 和 CR4 寄存器对应的位必须为 0 值。

VMX 架构还保证了另一种现象：当 **FIXED0 寄存器的位为 1 时，FIXED1 寄存器相应的位也必定返回 1 值。FIXED1 寄存器的位为 0 时，FIXED0 寄存器相应的位也必定为 0 值。**

举例来说：IA32_VMX_CR4_FIXED0 的 bit 13 为 1 值，那么 IA32_VMX_CR4_FIXED1 的 bit 13 也会保证为 1 值。IA32_VMX_CR4_FIXED1 的 bit 20 为 0 值，IA32_VMX_CR4_FIXED0 的 bit 20 也保证为 0 值。

另外注意：如果 FIXED0 寄存器的某位为 0，而 FIXED1 寄存器对应的该位为 1 时，表明这个位允许设为 0 或者 1 值。例如，FIXED0 的 bit 1 为 0，而 FIXED1 的 bit 1 为 1 时，则 CR0 或 CR4 寄存器的 bit 1 可以为 0，也可以为 1。实际上，它与前面的控制字段保留位为 default0 与 default1 值有相似之处。

以图 2-5 的运行结果为例，注意，结果里只输出了低 32 位，因此以低 32 位为例说明。CR0 寄存器对应的 IA32_VMX_CR0_FIXED0 的值为 80000021h，IA32_VMX_CR0_FIXED1 的值为 FFFFFFFFh。CR4 寄存器对应的 IA32_VMX_CR4_FIXED0 值为 00002000h，IA32_VMX_CR4_FIXED1 的值为 000027Fh。

那么说明以下几种现象：

(1) CR0 寄存器的 bit 0、bit 5 以及 bit 31 必须为 1 值 (FIXED0 与 FIXED1 寄存器的这些位都为 1)。也就是 CR0.PE、CR0.NE 以及 CR0.PG 位必须为 1，表示必须要开启分页的保护模式，并且使用 native 的 x87 FPU 浮点异常处理方法。

(2) CR4 寄存器的 bit 13 必须为 1 值 (FIXED0 与 FIXED1 寄存器的 bit 13 都为 1)。也就是 CR4.VMEX 位必须为 1, 表示必须开启 VMX 模式的许可。

(3) CR0 寄存器的其余位可为 0 或 1 值。

(4) CR4 寄存器的 bits 12:11 和 bits 31:14 必须为 0 值。

CR0 与 CR4 寄存器的固定位是 VMX 架构的制约条件之一, 详见第 2.3.2 节所述。

2.5.10.1 CR0 与 CR4 寄存器设置算法

CR0 与 CR4 寄存器的设置和前面所说的控制字段方法是一样的。

(1) Result1 = 初始值 “OR” FIXED0 寄存器

(2) Result2 = Result1 “AND” FIXED1 寄存器

这个 Result2 就是 CR0 与 CR4 寄存器的最终合成值, 也有下面的算法:

(1) FIXED0 “AND” FIXED1 可以找到哪些位必须为 1 值。

(2) FIXED0 “OR” FIXED1 可以找到哪些位必须为 0 值。

代码片段 2-12:

```
;;
;; 检查 CR0.PE 与 CR0.PG 是否符合 fixed 位, 这里只检查低 32 位值
;; 1) 对比 Cr0FixedMask 值 (固定为 1 值), 不相同则返回错误码
;;
mov eax, STATUS_VMX_UNEXPECT          ; 错误码 (超出期望值)
xor ecx, [ebp + PCB.Cr0FixedMask]     ; 与 Cr0FixedMask 值异或, 检查是否相同
js initialize_vmxon_region.done       ; 检查 CR0.PG 位是否相等
test ecx, 1
jnz initialize_vmxon_region.done       ; 检查 CR0.PE 位是否相等

;;
;; 如果 CR0.PE 与 CR0.PG 位相符, 设置 CR0 其他位
;;
or ebx, [ebp + PCB.Cr0Fixed0]          ; 设置 Fixed 1 位
and ebx, [ebp + PCB.Cr0Fixed1]         ; 设置 Fixed 0 位
REX.Wrxb
mov cr0, ebx                          ; 写回 CR0

;;
;; 直接设置 CR4 fixed 1 位
;;
REX.W
mov ecx, cr4
or ecx, [ebp + PCB.Cr4FixedMask]       ; 设置 Fixed 1 位
and ecx, [ebp + PCB.Cr4Fixed1]         ; 设置 Fixed 0 位
REX.W
mov cr4, ecx
```

上面的代码是在进入 VMX 模式前对 CR0 和 CR4 寄存器进行设置。首先, 检查 CR0 寄存器的初始值是否满足 CR0.PE=1 和 CR0.PG=1, 表示当前处于分页保护模式。然后 “或” FIXED0 寄存器, 以及 “与” FIXED1 寄存器值。

CR4 寄存器“或”Cr4FixedMask 值，这个值记录的是哪些位必须为 1，使用 Cr4FixedMask 而不是 FIXED0 寄存器是因为：Cr4FixedMask 也记录着是否需要开启 CR4.SMXE 位。

2.5.11 VMX 杂项信息

IA32_VMX_MISC 寄存器提供一些 VMX 的杂项信息，如图 2-9 所示。

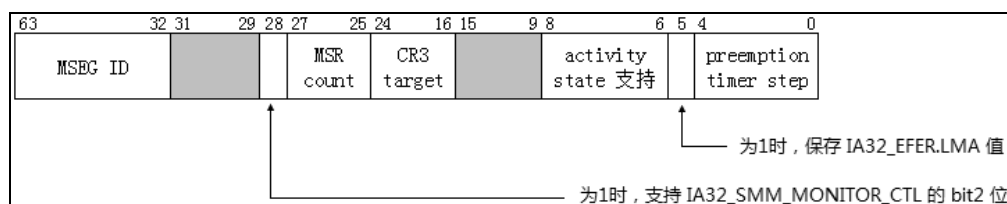


图 2-9

bits 4:0 提供一个 X 值，当 TSC 值的 bit X 改变时，VMX-preemption timer count 计数值将减 1。假如这个 X 值是 5，那么表示当 TSC 的 bit 5 发生改变（0 变 1 或 1 变 0）时，preemption timer count 计数值减 1。也就是说，TSC 计数 32 次时，preemption timer count 值减 1。

bit 5 为 1 时，表示当发生 VM-exit 行为时，将保存 IA32_EFER.LMA 的值在 VM-entry control 字段的“IA-32e mode guest”位里。只有当 VMX 支持“unrestricted guest”（不受限制的 guest）功能时，这个位才为 1 值。

bits 8:6 是一个 mask 位值，提供虚拟处理器 inactive 状态值的支持度，有下面的几个 inactive 状态值：

- bit 6 为 1 时，支持 HLT 状态
- bit 7 为 1 时，支持 shutdown 状态
- bit 8 为 1 时，支持 wait-for-SIPI 状态

只有这些 inactive 状态被支持时，才允许在 guest state 区域 activity state 字段设置相应的 inactive 状态值。bits 8:6 一般会返回 7（全部支持）。

bits 24:16 指示支持的 CR3-target 值的数量，一般会返回 4 值，表示支持 4 个 CR3 寄存器目标值。bits 27:25 返回一个 N 值，这个 N 值用来计算出在 MSR 列表（VM-exit MSR-load、VM-exit MSR-store 以及 VM-entry MSR-load 列表）里推荐的 MSR 最大个数。计算方法是：个数 = $(N+1) \times 512$ 。一般会返回 0 值，表示列表里推荐最多支持 512 个 MSR。

bit 28 为 1 时，表示支持 IA32_SMM_MONITOR_CTL 寄存器的 bit 2 位能被设为 1 值。这个 bit 2 置位时，表示执行 VMXOFF 指令时，SMI 能被解开阻塞。一般情况下 VMXOFF 指令的执行将阻塞 SMI 请求。

bits 63:32 提供一个 MSEG ID 值，这个 ID 值用于 SMM 双重监控处理机制下。在初始化 SMM-transfer Monitor 所使用的 MSEG 区域头部时需要使用该 ID 值。

2.5.12 VMCS 区域字段 index 值

IA32_VMX_VMCS_ENUM 寄存器提供 VMCS 区域内字段的最高 index 值。在 VMCS 区域内含有若干字段，每个字段都有相应的 encode 值。

访问 VMCS 的字段时，需要将 encode 值作为 VMREAD 和 VMWRITE 指令的操作数，然后执行 VMREAD 和 VMWRITE 指令，对相应的字段进行读写访问。

如图 2-10 所示，IA32_VMX_VMCS_ENUM 寄存器的 bits 9:1 提供 encode 内最高的 index 值，以图 2-5 运行结果为例，这个最高 index 值为 45。

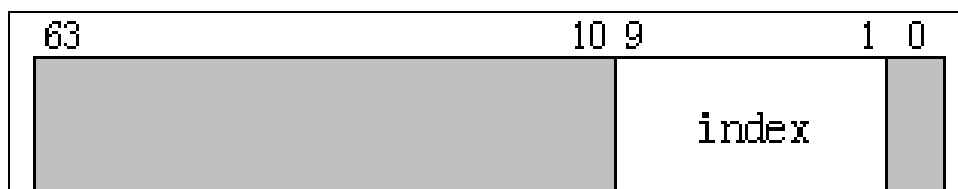


图 2-10

2.5.13 VPID 与 EPT 能力

IA32_VMX_EPT_VPID_CAP 寄存器提供两方面的能力检测，包括 EPT（扩展页表）所支持的能力，以及 EPT 页面 cache（TLBs 及 paging-structure cache）的能力。

当下面的条件满足时，才支持 IA32_VMX_EPT_VPID_CAP 寄存器。

- (1) CPUID.01H:ECX[5]=1，表明支持 VMX 架构。
- (2) IA32_VMX_PROCBASED_CTL[63]=1，表明支持 IA32_VMX_PROCBASED_CTL[63] 寄存器。
- (3) IA32_VMX_PROCBASED_CTL[33]=1，表明支持“enable EPT”位。

如图 2-11 所示是 IA32_VMX_EPT_VPID_CAP 寄存器的结构。

对于 EPT 能力，bit 0 为 1 时，允许在 EPT 页表项里的 bits 2:0 使用 **100b**（execute-only 页）属性。bit 6 为 1 时，表明支持 4 级页表结构。bit 16 为 1 时支持使用 2M 页，bit 17 为 1 时支持使用 1G 页。最后，bit 21 为 1 时支持在页表项里使用 dirty 标志。

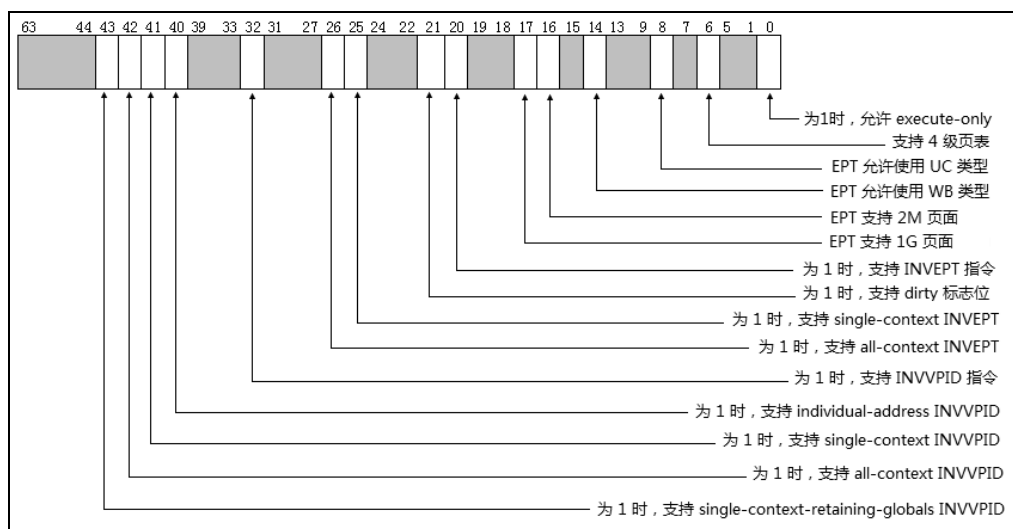


图 2-11

bit 8 为 1 时, 允许在 EPTP 字段的 bits 2:0 里设为 UC 类型 (值为 0), 而 bit 14 为 1 时, 允许在 EPTP 的 bits 2:0 里设置为 WB 类型 (值为 6)。参见第 4.4.1.3 节。

对于 EPT cache 的能力, bit 20 为 1 时支持 INVEPT 指令, bit 32 为 1 时支持 INVVPID 指令。INVEPT 指令支持的刷新类型由 bit 25 和 bit 26 检测。bit 25 为 1 时, 支持 single-context 刷新类型, bit 26 为 1 时, 支持 all-context 刷新类型。

INVVPID 指令支持的刷新类型由 bits 43:40 检测。bit 40 为 1 时, 支持 individual-address 类型 (type 值为 0)。bit 41 为 1 时, 支持 single-context 类型 (type 值为 1)。bit 42 为 1 时, 支持 all-context 类型 (type 值为 2)。bit 43 为 1 时, 支持 single-context-retaining-globals 类型 (type 值为 3)。

2.6 VMX 指令

VMX 架构提供 13 条 VMX 指令, 负责管理 4 个职能。如表 2-4 所示。

表 2-4

管理范围	指 令	描 述
VMCS 区域管理	VMPTRLD	加载一个 VMCS 指针作为 current-VMCS 指针
	VMPTRST	保存 current-VMCS 指针在内存里
	VMCLEAR	清 VMCS 状态, 并置 current-VMCS 指针为 FFFFFFFF_FFFFFFFFh 值
	VMREAD	读 current-VMCS 字段值
	VMWRITE	写 current-VMCS 字段值

续表

管理范围	指 令	描 述
VMX 模式管理	VMXON	进入 VMX operation 模式
	VMXOFF	退出 VMX operation 模式
	VMLAUNCH	发起 VM-entry 操作, 进入 VM
	VMRESUME	恢复 VM 执行
Cache 刷新	INVEPT	刷新 EPT 的 TLBs 和 paging-structure caches
	INVVPID	刷新 EPT 的 TLBs 和 paging-structure caches
调用服务例程	VMCALL	guest 调用 VMM 的服务例程, 并退出 VM
	VMFUNC	guest 调用为 VM 准备的服务例程, 不退出 VM

2.6.1 VMX 指令执行环境

VMX 架构对 CR0 和 CR4 寄存器的设置有基本的限制要求（详见第 2.3.2.2 节），即需要开启分页保护模式以及 CR4.VMX=1。下面是 VMX 指令执行的基本环境。

(1) 除了 VMXON 指令可以在进入 VMX operation 模式前执行，其他指令必须执行在 VMX operation 模式里。否则，将产生 #UD 异常。

(2) 不能在实模式、virtual-8086 模式以及 compatibility 模式下执行。否则，将产生 #UD 异常。除了在支持并开启“unrestricted guest”功能后，在 guest 的非分页或非保护模式环境里可以执行 VMFUNC 指令外。

(3) 所有 VMX 指令必须在 root 环境里执行（除了 VMFUNC 指令可以在 non-root 环境里执行外）。否则，将产生 VM-exit 行为。而 VMFUNC 指令执行在 root 环境里，将产生 #UD 异常。

(4) 除了 VMFUNC 指令外，所有 VMX 指令必须执行在 0 级权限里。否则，将产生 #GP 异常。

VMXON 指令是唯一在 VMX operation 模式外可执行的指令。VMXON 指令在 root 内执行会产生下面所说的 VMfailValid 失败。在 non-root 内执行则会产生 VM-exit 行为。而 VMFUNC 指令是唯一 non-root 内正常执行的指令，如果在 root 内执行则会产生 #UD 异常。

2.6.2 指令执行的状态

除了可能产生异常外（例如 #GP、#UD 异常等），每条 VMX 指令的执行还可能会产生 3 个状态，分别表示为 VMsucceed、VMfailInvalid 以及 VMfailValid：

(1) **VMsucceed**：指令执行成功。指令会清所有的 eflags 寄存器标志位，例如 CF 与 ZF 标志。

(2) **VMfailInvalid**：表示因 current-VMCS 指针无效而失败。当 current-VMCS 指针

或 VMCS 内的 ID 值是无效时，VMX 指令会置 CF 标志位，指示执行失败。

(3) **VMfailValid**：表示遇到某些原因而失败。例如，VMREAD 指令读一个 VMCS 区域内不存在的字段时，VMREAD 指令会置 ZF 标志位，指示执行失败。

在这样的一个情形下，例如，当 current-VMCS 指针为初始值 (FFFFFFFF_FFFFFFFh)，表明没有加载 VMCS 指针（使用 VMPTRLD 指令），执行 VMREAD 指令就会产生 VMfailInvalid 失败。

一般的情况下都需要检查 CF 与 ZF 标志位，以确定指令是否执行成功。VMfailInvalid 与 VMfailValid 失败的不同之处是：VMfailValid 失败会在当前 VMCS 内的 **VM-instruction error field**（指令错误字段）记录失败指令的编号。而 VMfailInvalid 是由于遇到无效的 VMCS 区域（典型地，VMCS ID 错误）或 current-VMCS 指针无效，因此不可能在 VMCS 内记录错误的信息。

2.6.3 VMfailValid 事件原因

VMX 指令产生的 VMfailValid 失败会对应一个编号，这个编号就记录在 VM-instruction error 字段里。这个字段是 VM-exit 信息区域内的其中一个 32 位字段。如表 2-5 所示。

表 2-5

错误编号	描 述
1	VMCALL 指令执行在 root 环境
2	VMCLEAR 指令操作数是无效的物理地址
3	VMCLEAR 指令操作数是 VMXON 指针
4	VMLAUNCH 执行时，current-VMCS 状态是非“clear”
5	VMRESULT 执行时，current-VMCS 状态是非“launched”
6	VMRESULT 在 VMXOFF 指令后
7	VM-entry 操作时，current-VMCS 含有无效的 VM-execution 控制字段
8	VM-entry 操作时，current-VMCS 含有无效的 host-state 字段
9	VMPTRLD 操作数是无效的物理地址
10	VMPTRLD 操作数是 VMXON 指针
11	VMPTRLD 执行时，VMCS 内的 VMCS ID 值不符
12	VMREAD/VMWRITE 指令读/写 current-VMCS 内不存在的字段
13	VMWRITE 指令试图写 current-VMCS 内的只读字段
14	
15	VMXON 指令执行在 VMX root operation 里
16	VM-entry 操作时，current-VMCS 内含有无效的 executive-VMCS 指针
17	VM-entry 操作时，current-VMCS 内使用了非“launched”状态的 executive-VMCS
18	VM-entry 操作时，current-VMCS 内的 executive-VMCS 指针不是 VMXON 指针

续表

错误编号	描 述
19	执行 VMCALL 指令切入 SMM-transfer monitor 时, VMCS 非 “clear” 状态
20	执行 VMCALL 指令切入 SMM-transfer monitor 时, VMCS 内的 VM-exit control 字段无效
21	
22	执行 VMCALL 指令切入 SMM-transfer monitor 时, MSEG 内的 MSEG ID 值无效
23	在 SMM 双重监控处理机制下执行 VMXOFF 指令
24	执行 VMCALL 指令切入 SMM-transfer monitor 时, MSEG 内使用了无效的 SMM monitor 特征
25	进行 “VM-entry that return from SMM” 操作时, 在 executive VMCS 中遇到了无效的 VM-execution control 字段
26	VM-entry 操作时, VMLAUNCH 或 VMRESUME 被 MOV-SS/POP-SS 阻塞
27	
28	INVEPT/INVVPID 指令使用了无效的操作数

表 2-5 列出了所有可能发生的 VMfailValid 原因, 编号 14, 21 以及 27 没有使用。其中有数个失败产生于 VM-entry 和 SMM 双重监控处理机制下的切入 SMM-transfer monitor 时。

2.6.4 指令异常优先级

如前面所述, 如果 VMX 指令非正常执行, 会出现下面三种情况之一。

- (1) 产生异常, 可能产生的异常为: #UD, #GP, #PF 或者 #SS。
- (2) 产生 VM-exit 行为。
- (3) 产生 VMfailInvalid 失败或者 VMfailValid 失败。

VMfailInvalid 与 VMfailValid 失败是在指令允许执行的前提下 (即执行指令不会产生异常及 VM-exit), 它会在 root 环境里。VMFUNC 指令执行在 non-root 环境不会产生失败, 只可能产生 #UD 异常或者 VM-exit。

在开启 “unrestricted guest” 功能后并进入实模式的 guest 软件里执行 VMX 指令时, 一个 #UD 异常的优先级高于 VM-exit 行为。或者一个由于 CPL 非 0 而引发的 #GP 异常, 优先级高于 VM-exit 行为 (参见第 5.6 节)。

2.6.5 VMCS 管理指令

有五条 VMX 指令涉及 VMCS 区域的管理, 如表 2-4 所示。下面将介绍它们的用法。

2.6.5.1 VMPTRLD 指令

VMPTRLD 指令从内存中加载一个 64 位的物理地址作为 **current-VMCS pointer**（当前 VMCS 指针），这个当前 VMCS 指针由处理器内部记录和维护，除了 VMXON、VMPTRLD 和 VMCLEAR 指令需要提供 VMXON 或 VMCS 指针作为操作数外，其他的指令都是在 current-VMCS 上操作。

```
vmptrlld [gs: PCB.GuestA]          ; 加载 64 位 VMCS 地址
```

注意：这条指令读取的是 64 位物理地址值，即使在 32 位环境下。如上面代码所示，在 PCB.GuestA 里保存着一个 VMCS 区域的物理地址值，执行这条指令将更新 **current-VMCS pointer** 值。在指令未执行成功时，current-VMCS pointer 将维持原有值。

处理器会根据提供的 VMCS 指针，进行下面的检查事项：

(1) 物理地址是否超过**物理地址宽度**，并且需要在 4K 地址边界上。当 IA32_VMX_BASIC[48]为 1 时，物理地址宽度在 32 位内，否则为 MAXPHYADDR 值（参见第 2.5.4 节）。

(2) VMCS 指针是否为 VMXON 指针。

(3) 目标 VMCS 区域首 DWORD 值是否符合 VMCS ID 值。

当上面检查不通过时，如果原来已经存在 current-VMCS pointer，则会产生 **VMfailValid** 失败，current-VMCS pointer 将维持不变，并且在 current-VMCS 的指令错误字段里记录错误号是 11（如表 2-5 所示）。指示：“执行 VMPTRLD 时，VMCS 内的 VMCS ID 值不符”。否则，产生 VMfailInvalid 失败。

2.6.5.2 VMPTRST 指令

VMPTRST 将 64 位的 current-VMCS pointer 保存在提供的内存中，如下面示例。

```
vmptrst [gs: PCB.VmcsPhysicalPointer] ; 保存 64 位 VMCS 地址
```

VMPTRST 指令不会产生 VMfailValid 或 VMfailInvalid 失败，除了产生异常或 VM-exit 外，它总是成功的。

2.6.5.3 VMCLEAR 指令

VMCLEAR 指令根据提供的 64 位 VMCS 指针，对目标 VMCS 区域进行一些初始化工作，并将目标 VMCS 的状态设置为“**clear**”。这个初始化工作，其中有一项是将处理器内部维护的关于 VMCS 的数据写入目标 VMCS 区域内。这部分 VMCS 数据可能与处理器相关，这是正确使用 VMCS 的前提。

在使用 VMLAUNCH 指令进入 VM 时，current-VMCS 的 launch 状态必须为“clear”。因此，当 VMCLEAR 指令对目标 VMCS 进行初始化后，使用 VMPTRLD 指令将目标 VMCS 加载为 current-VMCS，同时也更新 current-VMCS pointer 值。如下面代码示例。

```
vmclear [gs: PCB.GuestA]          ; 对 GuestA 进行初始化，置“clear”
jc @1
jz @1
```

```

vmptlrd [gs: PCB.GuestA]          ; 将 GuestA 加载为 current-VMCS
jc @1
jz @1

```

执行上述操作后，在后续工作里对 GuestA 区域进行配置，然后就可以使用 VMLAUNCH 指令进入 VM 执行。

VMCLEAR 指令也会对目标 VMCS 指针进行与 VMPTRLD 指令相同的检查（参见第 2.6.5.1 节）。如果在执行 VMCLEAR 指令前已经加载过当前 VMCS 指针，并且 VMCLEAR 指令的目标 VMCS 是 current-VMCS，则会设置 current-VMCS pointer 为 FFFFFFFF_FFFFFFFFh 值。

2.6.5.4 VMREAD 指令

VMREAD 指令需要提供一个 32 位的 VMCS 字段 ID 值放在寄存器里作为源操作数。目标操作数是寄存器或者内存操作数，将读取到的相应字段值放在目标寄存器或者内存地址中。

VMREAD 指令在 32 位模式下固定使用 32 位的操作数，在 64 位模式下固定使用 64 位操作数。而 VMCS 字段有不同的宽度，例如，存在 16 位字段和 **natural-width** 类型的字段。natural-width 类型的字段在支持 64 位架构的处理器上是 64 位，在不支持 64 位架构的处理器上是 32 位。

为了适应不同宽度的字段，VMREAD 指令使用下面的读取原则：

- 如果 VMCS 字段的 size 小于目标操作数 size，则 VMCS 字段值读入目标操作数后，目标操作数的高位部分清 0。
- 如果 VMCS 字段的 size 大于目标操作数 size，则 VMCS 字段的低位部分读入目标操作数，VMCS 字段的高位部分忽略。

由于 VMCS 字段 ID 值为 32 位，在 64 位模式下源操作数的 bits 63:32 必须为 0。否则会因为尝试读取一个不支持的 VMCS 字段而产生 VMfailValid 失败。

```

vmread [ebp + EXIT_INFO.ExitReason], eax
vmread ebx, eax

```

上面的代码中，VMCS 字段 ID 值放在源操作数 EAX 里，读到的值放在内存及 EBX 寄存器里。VMCS 的字段 ID 也被称为“**字段 encode**”，由几个部分组成，如图 2-12 所示。

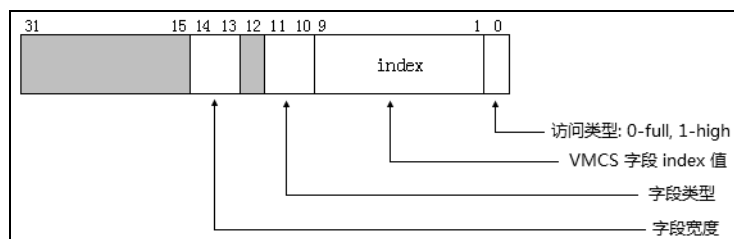


图 2-12

在前面第 2.5.12 节所提到的 IA32_VMX_VMCS_ENUM 寄存器（图 2-10）的 bits 9:1 提供最大的 VMCS 区域字段 index 值。这个最大 index 值就是指图 2-12 字段编码中的 index 最大值。关于字段编码的详细说明在第 3.3.1 节里描述。

在 inc\vmx.inc 文件里提供了两个宏，用来读取 VMCS 字段：**DoVmRead** 宏读取 VMCS 字段放在内存 buffer 中，**GetVmcsField** 宏读取 VMCS 字段值放在 EAX 寄存器中。如代码片段 2-13 所示。

代码片段 2-13:

```

;-----
; DoVmRead
; input:
;     %1 - vmcs ID
;     %2 - target value (memory)
; output:
;     %2 - target value
;-----
%macro DoVmRead 2
%if __BITS__ == 64
    mov rax, %1
    vmread %2, rax
%else
    mov eax, %1
    vmread %2, eax
%endif
%endmacro
;-----
; GetVmcsField
; input:
;     %1 - VMCS ID
; output:
;     eax - field value
;-----
%macro GetVmcsField 1
%if __BITS__ == 64
    mov rsi, %1
    vmread rax, rsi
%else
    mov esi, %1
    vmread eax, esi
%endif
%endmacro

```

DoVmRead 宏有两个参数，第 1 个参数是字段 ID 值，第 2 个是目标操作数。GetVmcsField 宏有 1 个参数，提供字段 ID 值，而输出固定使用 EAX 寄存器。

那么，就可以像下面的示例，使用 DoVmRead 或 GetVmcsField 宏来读取 VM-exit 信息区域里的 Exit-reason 字段。

DoVmRead	EXIT_REASON, [ebp + EXIT_INFO.ExitReason]
GetVmcsField	EXIT_REASON

VMREAD 指令操作在 current-VMCS 区域。因此，处理器会检查：（1）current-VMCS 指针是否无效（例如，为 FFFFFFFF_FFFFFFFFh 值）。（2）提供的字段 ID 值是

否有效，即 VMCS 中是否存在此字段。

在第 2.6.5.3 节里提到，当使用 VMCLEAR 指令对 current-VMCS 初始化时，当前 VMCS 指针就会变为 FFFFFFFF_FFFFFFFFh 值，此时执行 VMREAD 指令会产生 VMfailInvalid 失败。而当不存在 VMCS 字段时，则会产生 VMfailValid 失败。

2.6.5.5 VMWRITE 指令

VMWRITE 指令的源操作数是寄存器或者内存操作数，提供需要写入 VMCS 字段的值。目标操作数是寄存器，32 位的字段 ID 值提供在目标寄存器操作数里。

VMWRITE 指令在 32 位模式下固定使用 32 位的操作数，在 64 位模式下固定使用 64 位操作数。而 VMCS 字段有不同的宽度，例如存在 16 位字段和 natural-width 类型的字段。natural-width 类型的字段在支持 64 位架构的处理器上是 64 位，在不支持 64 位架构的处理器上是 32 位。

为了适合不同宽度的字段，VMWRITE 指令使用下面的写入原则：

- 如果写入值的 size 小于 VMCS 字段的 size，则值写入 VMCS 字段后，VMCS 字段的高位部分清 0。
- 如果写入值的 size 大于 VMCS 字段的 size，则忽略写入值的高位部分，低位部分写入 VMCS 字段里。

由于 VMCS 字段 ID 值为 32 位，在 64 位模式下目标操作数的 bits 63:32 必须为 0。否则会因为尝试写入一个不支持的 VMCS 字段而产生 VMfailValid 失败。

下面的代码中，写入数据放在 EBX 寄存器或者内存操作数，写入由 EAX 寄存器内的字段 ID 值指示的 VMCS 字段里。

```
vmwrite eax, ebx
vmwrite eax, [ebp + GUEST_STATUE.Rip]
```

与 VMREAD 指令稍微不同，VMWRITE 指令还会检查目标的 VMCS 字段是否可写。如果对一个 Read-Only 类型的字段进行写操作，将产生 VMfailValid 失败，错误编号为 13。

代码片段 2-14:

```
;-----
; DoVmWrite
; input:
;   %1 - vmcs ID
;   %2 - value (memory)
; output:
;   none
;-----
%macro DoVmWrite 2
%if __BITS__ == 64
    mov rax, %1
    vmwrite rax, %2
%else
    mov eax, %1
```

```

        vmwrite eax, %2
%endif
%endmacro
;-----
; SetVmcsField
; input:
;     %1 - VMCS ID
;     %2 - value
; output:
;     none
;-----
%macro SetVmcsField    2
%if __BITS__ == 64
    mov rsi, %1

%if %2 == eax
    mov rdi, rax
%elif %2 == ecx
    mov rdi, rcx
%elif %2 == edx
    mov rdi, rdx
%elif %2 == ebx
    mov rdi, rbx
%elif %2 == esp
    mov rdi, rsp
%elif %2 == ebp
    mov rdi, rbp
%elif %2 == esi
    mov rdi, rsi
%elif %2 == edi
    mov rdi, rdi
%else
    mov rdi, %2
%endif
    vmwrite rsi, rdi
%else
    mov esi, %1
    mov edi, %2
    vmwrite esi, edi
%endif
%endmacro

```

为了使用方便，在 inc\vmx.inc 文件里也实现了两个宏：**DoVmWrite** 与 **SetVmcsField**，用来写 VMCS 字段。DoVmWrite 宏与 DoVmRead 宏相对应，SetVmcsField 宏与 GetVmcsField 宏相对应。这里的 SetVmcsField 宏相对复杂一些，额外使用了 8 个条件判断，目的是当这个宏使用在 64 位模式下保证正确。

2.6.6 VMX 模式管理指令

此类指令共有 4 条，分别为：VMXON、VMXOFF、VMLAUNCH 以及 VMRESUME 指令。此类指令在启用 SMM 的“dual-monitor treatment”功能后，情况会变得复杂些。

2.6.6.1 VMXON 指令

执行 VMXON 指令成功后进入处理器的 VMX root-operation 模式，并且初始化 current-VMCS pointer 为 FFFFFFFF_FFFFFFFFh 值。在 root 环境下，INIT 与 SIPI 信号会无条件地被阻塞，也不允许进入 A20M 模式。因此，进入 VMX operation 前必须启用 A20 地址线，关闭地址的“wrapped”机制。

VMXON 指令的内存操作数里需要提供 64 位 VMXON 区域物理地址（另见第 2.3.3 节描述）。处理器会检查：（1）VMXON 指针是否超过物理地址宽度并且在 4K 边界上对齐。（2）VMXON 区域的头 DWORD 值是否符合 VMCS ID。这些检查不通过时产生 VMfailInvalid 类型失败。

当执行 VMXON 指令时，如果处理器已经进入到 VMX root operation 模式，将会产生 VMfailValid 失败，指令错误编号是表 2-5 上所列的 15（VMXON 指令执行在 VMX root operation 里）。指令执行在 non-root 环境里则会产生 VM-exit。

2.6.6.2 VMXOFF 指令

VMXOFF 指令无操作数，指令执行成功后将关闭处理器的 VMX operation 模式。如果开启了 SMM 双重监控处理机制，需要首先关闭 SMM 双重监控处理机制后再执行 VMXOFF 指令才能退出 VMX operation 模式，否则将产生 VMfailValid 类型失败。

关闭 SMM 双重监控处理机制做法是：在 root 模式里执行 VMCALL 指令切入 STM（SMM-transfer monitor）后，将当前 SMM-transfer VMCS 内 VM-entry 控制字段的“deactivate dual-monitor treatment”位置 1，再执行 VMRESUME 指令返回到 executive monitor。

2.6.6.3 VMLAUNCH 指令

VMLAUNCH 指令被使用于首次发起 VM-entry 操作。因此 current-VMCS 的状态必须为“clear”，否则产生 VMfailValid 类型失败。在发起 VM entry 操作前的一系列动作里，需要使用第 2.6.5.3 节里描述的 VMCLEAR 指令将目标 VMCS 状态置为“clear”状态，然后使用 VMPTRLD 指令加载该 VMCS 为 current-VMCS。

VM entry 操作是一个很复杂的过程，分为三个阶段进行，每个阶段会进行一些必要的检查。检查不通过时，这三个阶段的后续处理也不同，如下所示。

（1）执行基本检查。包括：

- 可能产生的指令异常（#UD 或者 #GP 异常）。
- current-VMCS pointer 是否有效。
- 前面所说的当前 VMCS 状态是否为“clear”。
- 执行的 VMLAUNCH 指令是否被“MOV-SS”指令阻塞。

（2）对 VMCS 内的 VM-execution、VM-exit、VM-entry 以及 host-state 区域的各个字段进行检查。

(3) 对 VMCS 的 guest-state 区域的各个字段进行检查。

在第 1 步里的检查失败后将执行 VMLAUNCH 指令的下一条指令。当 current-VMCS pointer 无效 (如 FFFFFFFF_FFFFFFFh) 时产生 VmfailInvalid 失败。当 current-VMCS 为非“clear”或者执行的 VMLAUNCH 指令被“MOV-SS”指令阻塞时, 产生 VmfailValid 失败。产生异常时转入执行异常处理例程。

这一步里, 有一个需要关注的“blocking by MOV-SS”阻塞状态, 在下面的情形里:

mov ss, ax	; 更新 SS 寄存器, 或者执行 pop ss 指令
vmLaunch	; vmLaunch 有“blocking by MOV-SS”阻塞状态

上述情形, 当 VMLAUNCH 或 VMRESUME 指令执行在 MOV-SS 或 POP-SS 指令后, 那么就会产生 VmfailValid 失败, 产生的指令错误号为 26, 指示“VMLAUNCH 或 VMRESUME 指令将被 MOV-SS 阻塞”。

在第 2 步的检查通不过时, 也产生 VmfailValid 失败。指令错误号为 7, 指示“VM-entry 操作时, current-VMCS 内含有无效的控制字段”。控制字段包括: VM-execution、VM-exit 以及 VM-entry 区域内的控制字段。也可能产生的指令错误号为 8, 指示“VM-entry 操作时, current-VMCS 内含有无效的 host-state 区域字段”(如表 2-5 所示)。失败后也执行 VMLAUNCH 指令的下一条指令。

在第 3 步里, 当检查通不过时会产生 VM-exit 行为。处理器在 Exit reason 字段里记录产生 VM-exit 原因的一个编号。它的编号是 33, 指示“由于无效的 guest-state 字段而产生 VM-entry 失败”。处理器会加载 host-state 的 context 信息, 然后转入执行 host 的入口点。

2.6.6.4 VMRESUME 指令

VMRESUME 指令执行与 VMLAUNCH 相同的动作, 但是 VMRESUME 指令使用于在 VM-exit 产生后再次进入 VM。因此, VMRESUME 指令检查 current-VMCS 的状态是否为“launched”, 否则产生 VmfailValid 失败。指令错误编号为 5, 指示“VM-entry 操作时, current-VMCS 状态为非 launched”。

2.6.6.5 返回到 executive monitor

在启用 SMM 双重监控处理机制的情况下, 有一种 VM-entry 操作叫做“VM-entry that return from SMM”, 直译为“从 SMM 退出进入 VM”。注意, 这种情形只会发生在 VMRESUME 指令里。

这是两个 monitor 之间的切换, 这种情形是从 SMM-transfer monitor 切换到 executive monitor。在这种情况下, 在前面第 2.6.6.3 节所述的第 2 步检查里, 处理器还会进行另一些检查 (这里的 current-VMCS 等于 SMM-transfer VMCS), 如下所示。

(1) 检查在 current-VMCS 内的 executive-VMCS pointer 字段的 executive-VMCS 指针是否有效。否则产生 VmfailValid 失败, 错误编号为 16, 指示: “current-VMCS 含有无效的 executive-VMCS 指针”。

(2) 如果 current-VMCS 内的 VM-entry 字段的 “deactivate dual-monitor treatment” 位为 1，检查 executive-VMCS 指针是否为 VMXON 指针。否则产生 VMfailValid 失败，错误编号为 18，指示：“current-VMCS 内的 executive-VMCS 指针不是 VMXON 指针”。

(3) 如果 executive-VMCS 指针不是 VMXON 指针，那么 executive-VMCS 指针在切换到 executive monitor 后，会被加载为新的 current-VMCS 指针。此时 executive-VMCS 的状态必须为 “**launched**”，否则产生 VMfailValid 失败，错误编号为 17，指示：“executive-VMCS 的状态为非 launched 状态”。

(4) 当 executive-VMCS 属于 VMXON region 时（即返回到 root），不会检查 VM-execution 控制字段。当 executive-VMCS 不是 VMXON region 时（即返回到 non-root），处理器对 executive-VMCS 的 VM-execution 控制字段执行所有检查（参见第 4.4.1 节）。检查不通过时产生 VMfailValid 失败，错误编号为 25，指示“executive-VMCS 内含有无效的 VM-execution 控制字段”。

(5) 当 executive-VMCS 属于 VMXON region 时，executive-VMCS 不能指示“注入一个事件”（除了 pending MTF VM-exit 事件外），参见第 3.6.3 节。

(6) 当 executive-VMCS 属于 VMXON region 时（返回 root），所有 guest-state 区域的字段检查基于 VM-execution 控制字段为 0 的情况下，activity state 字段不能指示为 wait-for-SIPI 状态。当 executive-VMCS 不是 VMXON 时（返回 non-root），guest-state 区域字段的检查基于 VM-execution 控制字段相应的设置。

上面的检查失败后，会执行 SMM-transfer monitor 中 VMRESUME 指令的下一条指令。

2.6.7 cache 刷新指令

VMX 架构下提供了两条用于刷新 cache 的指令：INVEPT 与 INVVPID 指令。它们都对 TLBs 与 paging-structure caches 进行刷新。关于 TLBs 与 paging-structure caches 的描述可以参考《x86/x64 体系探索及编程》第 11.6 节，或者《Intel 开发人员手册》Vol3A 第 4.10 节。

在 VMX 架构下实现了三类映射途径下的 TLB caches 与 paging-structure caches 信息，它们是：

(1) **linear mapping**，当 EPT 机制未启用时（或者在 VMX root operation 模式下），这类 cache 信息用来缓存 linear address 到 physical address 的转换（详见第 6.2.1 节）。

(2) **guest-physical mapping**，当 EPT 机制启用时，这类 cache 信息用来缓存 guest-physical address 到 host-physical address 的转换（详见第 6.2.2 节）。

(3) **combined mapping**，当 EPT 机制启用时，这类 cache 信息结合了 linear address 和 guest-physical address 到 host-physical address 的转换。也就是缓存了 linear address 到 host-physical address 的转换信息（详见第 6.2.3 节）。

这几类 cache 信息，我们将在后面的篇章里进行探讨（参见第 6.2 节）。INVEPT 与 INVVPID 指令的不同之处就是：刷新的 cache 信息，以及刷新的 cache 域。

2.6.7.1 INVEPT 指令

在启用 EPT 机制时，可以使用 INVEPT 指令对“GPA 转换 HPA”而产生的相关 cache 进行刷新。它根据提供的 EPTP 值（EPT pointer）来刷新 guest-physical mapping（guest-physical address 转换到 host-physical address）和 combined mapping（linear address 转换到 host-physical address）产生的 cache 信息。

`invept rax, [InveptDescriptor]` ; 刷新 TLBs 及 paging-structure caches

在上面的指令示例里，`rax` 寄存器提供 **INVEPT type**，指示使用何种刷新方式。而内存操作数里存放着 INVEPT 描述符，EPTP 值提供在这个 INVEPT 描述符里，如图 2-13 所示。

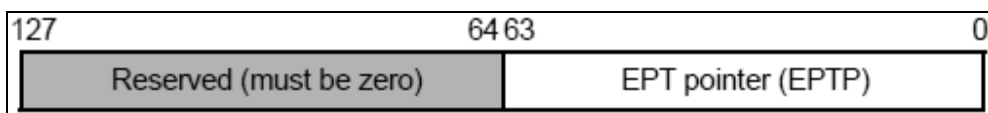


图 2-13

INVEPT 描述符的结构，共 16 个字节，bits 63:0 存放 EPTP 值。EPTP 值的 bits $N-1:12$ 指向 EPT 结构的 PML4T 表，INVEPT 指令将 $EPTP[N-1:12]$ 所引伸出来的层级转换表结构作为依据进行刷新 cache，这个 EPTP 字段 bits $N-1:12$ ($N=MAXPHYADDR$) 提供的值被称为“**EP4TA**”（EPT PML4T address，扩展页表的 PML4T 地址）。

INVEPT 指令支持两种刷新类型：single-context 与 all-context。详见第 6.2.6.4 节所述。

(1) 当 type 值为 1 时，使用 **single-context** 刷新方式。处理器刷新 INVEPT 描述符里提供的 EP4TA 值所对应的 guest-physical mapping 与 combined mapping 信息。

(2) 当 type 值为 2 时，使用 **all-context** 刷新方式。处理器刷新所有 EP4TA 值对应的 guest-physical mapping 与 combined mapping 信息。也就是说，此时将忽略 INVEPT 描述符。

另外需要特别注意的是：处理器也刷新**所有 VPID 与 PCID 值**所对应的 combined mappings 信息。软件应该要查询处理器的 INVEPT 指令是否支持上述的 type 值。

前面第 2.5.13 节描述了 INVEPT 指令支持的刷新类型，当使用不支持的类型时产生 VMfailValid 失败，错误编号为 28，指示“无效的 INVEPT/INVVPID 操作数”。

2.6.7.2 INVVPID 指令

INVVPID 指令依据提供的 VPID 值对 linear mapping 及 combined mapping 的 cache 信息进行刷新。也就是 INVVPID 指令可以刷新 EPT 机制**启用**或者**未启用**时的线性地址到物理地址转换而产生的 cache 信息。

`invvpid rax, [InvvpidDescriptor]` ; 刷新 TLBs 及 paging-structure caches

INVVPID 指令也需要在寄存器操作数里提供 INVVPID type 值，在内存操作数里提供 INVVPID 描述符。INVVPID 描述符结构如图 2-14 所示。

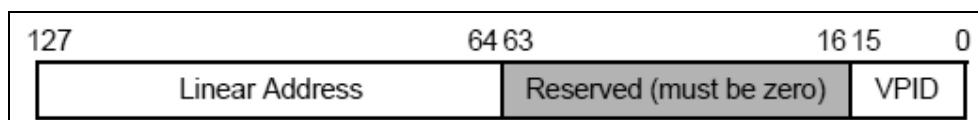


图 2-14

INVVPID 描述符的 bits 127:64 提供线性地址，bits 15:0 提供 VPID 值。INVVPID 指令依据这两个值进行刷新。INVVPID 指令支持 4 个刷新类型（详见第 6.2.6.3 节所述）。

- 当 type 值为 0 时，使用 **individual-address** 刷新方式。指令将刷新目标 VPID，所有 PCID 以及所有 EP4TA 域内与目标线性地址匹配的 cache 信息，具体为：① 匹配描述符内提供的目标线性址与目标 VPID 值。② 所有 PCID 域下对应的 linear mappings 与 combined mappings 信息。③ 所有 EP4TA 域下对应的 combined mappings 信息。
- 当 type 值为 1 时，使用 **single-context** 刷新方式。指令将刷新目标 VPID，所有 PCID 以及所有 EP4TA 域的 cache 信息，具体为：① 匹配描述符内提供目标 VPID 值。② 所有 PCID 域下对应的 linear mappings 与 combined mappings 信息。③ 所有 EP4TA 域下对应的 combined mappings 信息。
- 当 type 值为 2 时，使用 **all-context** 刷新方式。指令将刷新默认 VPID 值（0000H）外的所有 VPID，所有 PCID 以及所有 EP4TA 域的 cache 信息，具体为：① 所有 VPID 值（除了 0000H）。② 所有 PCID 域下对应的 linear mappings 与 combined mappings 信息。③ 所有 EP4TA 域下对应的 combined mappings 信息。
- 当 type 值为 3 时，使用 **single-context-retaining-global** 刷新方式。指令行为与类型 2 的 single-context 刷新方式相同，除了保留 global 的转换表外。

在所有的刷新方式里，都不能刷新 VPID 值为 0000H 的 cache 信息。否则产生 VMfailValid 失败，错误编号为 28，指示“无效的 INVEPT/INVVPID 操作数”。

软件也应该使用第 2.5.13 节里描述的方式，查询当前处理器支持哪种刷新类型。如果提供不支持的刷新类型，也同样产生编号为 28 的 VMfailValid 失败。

2.6.8 调用服务例程指令

VMX 架构提供了两个调用服务例程指令：VMCALL 与 VMFUNC 指令。它们服务的对象不同，VMCALL 指令使用在 VMM 里，而 VMFUNC 指令使用在 VM 里。

2.6.8.1 VMCALL 指令

利用 VMCALL 指令可以实现 SMM 的 dual-monitor treatment（SMM 双重监控处理）机制。VMCALL 指令在 non-root 里执行将会产生 VM-exit 行为，但在 root 环境里执行 VMCALL 指令，当满足检查条件时，在 VMM 里产生被称为“SMM VM-exit”的退出行

为，从而切换到 SMM 模式的 SMM-transfer monitor 里执行。这个 SMM-transfer monitor 入口地址提供在 MSEG 区域头部（由 IA32_SMM_MONITOR_CTL[31:12]提供）。

在 VMX root operation 里执行 VMCALL 指令，除了可能产生异常（#UD 或#GP）外，有两种可能：（1）指令失败（VMfailInvalid 或 VMfailValid）。（2）产生“SMM VM-exit”，激活 SMM 双重监控处理功能。

IA32_SMM_MONITOR_CTL 寄存器的 bit 0 为 valid 位。只有当 bit 0 为 1 时，才允许使用 VMCALL 指令通过切入 SMM-transfer monitor 执行来激活 SMM 双重监控处理机制。否则将产生 VMfailValid 失败，指示“VMCALL 指令执行在 VMX root operation 模式里”。

2.6.8.2 VMFUNC 指令

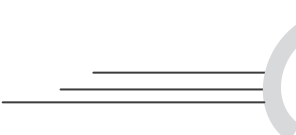
VMFUNC 指令是唯一能在 non-root 环境里使用的 VMX 指令。当允许并且设置 secondary processor-based control 字段的“enable VM functions”位为 1 时，允许 VM 里执行 VMFUNC 指令调用服务例程，否则将产生#UD 异常。注意，VMFUNC 指令在 VMX root operation 模式里执行也会产生#UD 异常。

执行 VMFUNC 指令前，在 eax 寄存器里放入功能号。然而，在执行某个功能号时，也需要在 VM-functions control 字段的相应位置位。VM-functions control 字段是一个 64 位值，因此 VMX 架构最多只支持 0 至 63 的功能号。如果提供的功能号大于 63，则会产生#UD 异常。

当启用“enable VM functions”功能，提供一个功能号给 eax 寄存器执行 VMFUNC 指令，但 VM-functions control 字段（参见第 3.5.20 节）相应位为 0 值，则会产生 VM-exit 行为。在成功执行 VMFUNC 指令的情况下不会产生 VM-exit 行为。

当前 VMX 架构下只实现了一个功能号 0，它是“EPTP switching”服务例程（参见第 6.1.11 节）。使用它则需要在 VM-functions control 字段的 bit 0 进行置位。软件使用前需要通过第 2.5.9 节描述的方法来检测是否支持该项功能。

为了支持 EPT switching 服务例程，VMX 架构添加了一个 EPTP-list address 字段（参见第 3.5.21 节），提供 512 个 EPT 值供切换。使用时需要在 EAX 寄存器放入 0 号功能，在 ECX 寄存器放入 EPT-list entry 的编号。当 ECX 的值大于 511 时将产生 VM-exit。



第 3 章

VMCS 结构

在 VMX 架构里，当发生 VMX operation 模式的 root 与 non-root 环境之间的切换时，VMCS (Virtual Machine Control Structure) 用来配置当前发生切换的逻辑处理器的状态及执行环境。在 VMCS 的配置下 logical processor (逻辑处理器) 可以视为 virtual processor (虚拟处理器)。

从 root 切换到 non-root 被称为 **VM-entry**，将进入 VM 环境 (虚拟处理器) 执行，逻辑处理器从 VMCS 的 guest-state 区域读取相应的字段来更新当前处理器上下文环境。从 non-root 切换到 root 被称为 **VM-exit**，意味着返回到 VMM 里。逻辑处理器从 VMCS 的 host-state 区域读取相应的字段来更新当前处理器上下文环境。

3.1 VMCS 状态

在一个存在多个虚拟机的平台里，每个虚拟处理器对应一个属于自己的 VMCS。在虚拟处理器之间的切换也意味着 VMCS 之间的切换。同一时刻，一个逻辑处理器只有一个 VMCS 是 **current-VMCS**。

根据 Intel 手册的描述，我们可以归纳出用来描述 VMCS 的三类属性状态，它们是：

- (1) activity 属性，包括 active (活动的) 及 inactive (非活动的) 状态。
- (2) current 属性，包括 current (当前的) 及 not current (非当前的) 状态。
- (3) launch 属性，包括 clear (干净的) 及 launched (已启动的) 状态。

在这些 VMCS 属性中，current 与 launch 属性是比较重要的。一个 VMCS 的某一类属性是什么状态，并不影响其余的属性状态。例如，VMCS 可以是“**active**”状态，但可能不属于“**current**”状态或“**clear**”与“**launched**”状态。

有一个例外，如果 VMCS 处于“**inactive**”状态，那么它必定是“**not current**”状态。另一种情况是：在刚进入 VMX operation 模式时，没有任何 VMCS 处于“**active**”状态，因此，也就不存在“**current**”及“**clear**”状态。

3.1.1 activity 属性

如果一个 VMCS 是 active 状态，表明它已经被加载过。在刚进入 VMX operation 模式后，一旦使用 VMPTRLD 指令加载过某一个 VMCS，那么这个 VMCS 就变成“active”状态。即使后续又使用 VMPTRLD 指令加载其他的 VMCS，原 VMCS 还是维持“active”状态，除非使用 VMCLEAR 指令强制初始化 VMCS。

```

vmptlrd [GuestA]      ; A 变成 active 状态
vmptlrd [GuestB]      ; 现在 A 与 B 都是 active 状态
vmclear [GuestA]      ; A 变成 inactive 状态

```

要将 VMCS 变为“inactive”状态，只能使用 VMCLEAR 指令初始化 VMCS。如上面的示例代码，VMCLEAR 指令执行前，A 和 B 都是“active”状态，VMCLEAR 指令执行后，A 变成“inactive”状态。

另外，在虚拟机平台迁移设计里，一个 active VMCS 不应该对应多个逻辑处理器。举例来说，VMCS A 在逻辑处理器 0 里已经是“active”状态，那么在逻辑处理器 1 上不应该直接使用 VMPTRLD 指令来加载 VMCS A，这样可能会造成 VMCS A 配置不适合逻辑处理器 1 使用。逻辑处理器 1 在加载 VMCS A 之前，必须使用 VMCLEAR 指令将 VMCS A 置为“inactive”状态，再使用 VMPTRLD 指令加载 VMCS A。

3.1.2 current 属性

在一个逻辑处理器中，一个时刻只能有一个 VMCS 处于“current”状态。这样的 VMCS 我们可以称为 **current-VMCS**，指向它的指针称为 **current-VMCS pointer**。处理器会自动记录和维护 current-VMCS pointer 值。

```

vmptlrd [GuestA]      ; 此时 A 为 current-VMCS
vmptlrd [GuestB]      ; 此时 B 为 current-VMCS, A 变为 not current 状态
vmclear [GuestB]      ; 此时 B 为 not current 状态

```

上面的示例代码中，每使用一次 VMPTRLD 指令来加载 VMCS，current-VMCS 就会更新一次。而原 current-VMCS 就会变为“not current”状态，current-VMCS 并不意味着它一定是“clear”或“launched”状态。一旦使用 VMCLEAR 指令初始化目标 VMCS，目标 VMCS 就变为“not current”状态以及“clear”状态。并且，如果目标 VMCS 是 current-VMCS，那么 current-VMCS pointer 的值会被初始化为 FFFFFFFF_FFFFFFFFh。

VMREAD, VMWRITE, VMLAUNCH, VMRESUME 及 VMPTRST 指令的操作都是实施在 current-VMCS 之上的。处理器内部使用和维护的 current-VMCS pointer 值就指向 current-VMCS。根据推测，current-VMCS pointer 要么存放在处理器内部寄存器中，要么存放在 VMXON 区域内。

3.1.3 launch 属性

处理器也记录着 VMCS 的 launch 属性状态。若一个逻辑处理器管理多个 VMCS，那

么可能存在多个“launched”或“clear”状态的 VMCS。

一旦使用 VMLAUNCH 指令成功进行 VM-entry 操作，current-VMCS 就会变为“launched”状态。VM-exit 后如果 current-VMCS 变更为另一个 VMCS，原 current-VMCS 还是维持“launched”状态，除非使用 VMCLEAR 指令对它显式地初始化。

vmptrld [GuestA]	; 此时 A 为 current-VMCS
vmlaunch	; A 为 launched 状态
.....	; 产生 VM-exit 后
vmptrld [GuestB]	; 此时 B 是 current-VMCS, A 维持 launched 状态
vmlaunch	; B 为 launched 状态
.....	; 产生 VM-exit 后
vmclear [GuestA]	; 此时 A 为 clear 状态

上面的示例代码中，A 经过成功执行 VMLAUNCH 指令后变为“launched”状态，后续发生了 VM-exit 回到 VMM，VMM 又加载了另一个 VMCS B，B 此时变成 current-VMCS。B 经过成功执行 VMLAUNCH 指令后也变为“launched”状态，但 A 还是维持“launched”状态，直至使用 VMCLEAR 指令对 VMCS A 进行初始化后，A 变为“clear”状态。

注意：VMLAUNCH 指令执行时需要 current-VMCS 为“clear”状态，而 VMRESUME 指令执行时需要 current-VMCS 为“launched”状态。

综合三节描述，VMCS 状态随着下面指令的执行而改变：

- (1) VMCLEAR 指令将目标 VMCS 置为“inactive”，“not current”及“clear”状态。
- (2) VMPTRLD 指令将目标 VMCS 置为“active”及“current”状态，并更新 current-VMCS 指针。
- (3) VMLAUNCH 指令将 current-VMCS 置为“launched”状态。

处理器会记录 VMCS 的这些状态信息。这些状态信息存放在目标 VMCS 的不可见字段里，这样处理器就很容易知道 VMCS 目前处于哪种状态。这些不可见字段并没有实现对应的字段 ID，因此，不能使用 VMREAD 与 VMWRITE 指令进行访问。

3.2 VMCS 区域

VMCS 结构存放在一个物理地址区域里，这个区域被称为“**VMCS region**”。VMCS 区域需要对齐在 4K 边界上。VMCS 区域的大小由 IA32_VMX_BASIC[44:32]域里得到（见第 2.5.4 节），以 KB 为单位，最高为 4KB。IA32_VMX_BASIC[53:50]报告了 VMCS 区域支持的 cache 类型，支持 UC 与 WB 类型。

执行 VMCLEAR 与 VMPTRLD 指令时，需要提供目标 VMCS 区域的物理指针作为操作数。而 VMREAD，VMWRITE，VMLAUNCH，VMRESUME 及 VMPTRST 指令隐式地使用 current-VMCS pointer 作为目标 VMCS 指针。

前面提到，使用 VMCLEAR 指令对 current-VMCS 指针进行初始化时，current-

VMCS 指针值变为 FFFFFFFF_FFFFFFFh。这时，执行上面隐式使用 current-VMCS pointer 的指令，会产生 VMfailInvalid 失败。

3.2.1 VMXON 区域

VMM 需要用一个被称为“**VMXON region**”的区域来管理整个 VMX operation 模式，VMXON 区域大小及所支持的 cache 类型与 VMCS 区域是一致的。指向 VMXON 区域的指针被称为“**VMXON 指针**”。进入 VMX operation 模式前，需要为 VMM 准备一份 VMXON 区域，VMXON 区域的物理指针需要作为操作数提供给 VMXON 指令。

一个 VMM 对应一个 VMXON 指针，除非在 VMM 里关闭 VMX operation 模式后，再使用另一个 VMXON 指针来重新开启 VMX operation 模式。否则，VMXON 指针是不会改变的。VMXOFF 指令也操作在这个 VMXON 指针上，用来关闭当前 VMXON 区域所管理的 VMX operation 模式。

3.2.2 Executive-VMCS 与 SMM-transfer VMCS

在 SMM 双重监控处理机制下，还有两类 VMCS，分别是：Executive-VMCS 与 SMM-transfer VMCS。Executive-VMCS 对应 VMM 在 VMX operation 模式（包括 root 与 non-root）下的 VMCS，而 SMM-transfer VMCS 对应 VMM 进入 SMM 模式时使用的 VMCS。

Executive-VMCS 指针并不是新加载的 VMCS 指针，它在 SMM VM-exit 发生后自动更新。它等于 VMXON 指针还是 current-VMCS 指针，取决于进入 SMM-transfer monitor 时处理器处于 VMX root operation 还是 VMX non-root operation。

- 当 SMM VM-exit 发生在 VMX root operation 模式时，Executive-VMCS 指针等于 VMXON 指针。
- 当 SMM VM-exit 发生在 VMX non-root operation 模式时，Executive-VMCS 指针等于 current-VMCS 指针。

在首次进入 SMM-transfer monitor 前（执行 VMCALL 指令），需要使用 VMPTRLD 指令加载另一个 VMCS 作为 SMM-transfer VMCS 指针，但这个 VMCS 不能是 current-VMCS。处理器会记录和维护这个 SMM-transfer VMCS 指针。在进入 SMM 模式处理 SMI 后，这个 SMM-transfer VMCS 指针就被视为 SMM 模式内的 current-VMCS 指针。

3.2.3 VMCS 区域格式

VMCS 区域寄存在物理地址空间，有一个 8 字节的头部和 VMCS 数据区域，如图 3-1 所示。

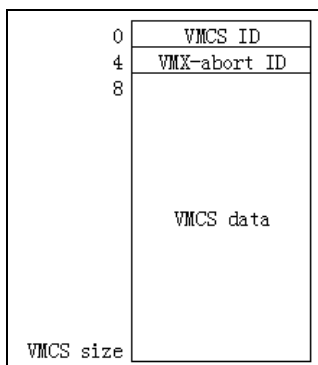


图 3-1

VMCS 的首 DWORD 值是 VMCS ID，这个 VMCS ID 值需要在初始化 VMCS 区域时写入，必须等于 IA32_VMX_BASIC[31:0]值（见第 2.5.4 节）。注意，VMXON 区域的首 DWORD 值也是 VMCS ID，也必须等于 IA32_VMX_BASIC[31:0]值。

接下来的 DWORD 值是 VMX-abort ID，当进行 VM-exit 操作遇到一个错误时，会产生“**VMX-abort**”。VMX-abort 导致处理器进入 shutdown 状态，处理器会写入一个非零值到 current-VMCS 的 VMX-abort ID 字段（详见第 5.17 节）。

VMCS 数据区包括 6 个区域，每个区域有若干字段，如下所示。

- (1) guest-state 区域：在 VM-entry 时，处理器的状态信息从 guest-state 区域中加载。在 VM-exit 时，处理器的当前状态信息保存在 guest-state 区域。
- (2) host-state 区域：在 VM-exit 时，处理器的状态信息从 host-state 区域中加载。
- (3) VM-execution 控制区域：在进入 VM 后，处理器的行为由 VM-execution 控制区域中的字段提供控制。例如，可以设置某些条件使得在 guest 执行中产生 VM-exit。
- (4) VM-exit 控制区域：控制处理器在处理 VM-exit 时的行为，也影响返回 VMM 后处理器的某些状态。
- (5) VM-entry 控制区域：控制处理器在处理 VM-entry 时的行为，也决定进入 VM 后处理器的某些状态。
- (6) VM-exit 信息区域：记录引起 VM-exit 事件的原因及相关的明细信息。也可以记录 VMX 指令执行失败后的错误编号。

3.3 访问 VMCS 字段

VMCS 数据区内的字段与处理器实现相关。在不同架构下的处理器中，这些 VMCS 字段位置可能会不一致。并且，在一些较早的处理器中也不支持 VMX 架构提出的新特性，导致在这些早期的处理器中某些字段是不存在的。

软件不能使用 MOV 指令来直接访问 VMCS 字段，原因是：

(1) 软件并不知道 VMCS 字段的位置，也无法查询或推测出具体位置。

(2) 使用 MOV 指令对 VMCS 区域进行读写，可能会读写到一些不正确的字段值，从而可能导致不可预测的结果发生。

软件必须通过 **VMREAD** 与 **VMWRITE** 指令来访问 VMCS 字段。每个字段定义一个唯一的 ID 值来对应，这个字段 ID 值提供给 VMREAD 与 VMWRITE 指令作为 index 值来访问相应的字段。

3.3.1 字段 ID 格式

字段 ID 值是 32 位的，它由几个域组成，如图 3-2 所示。

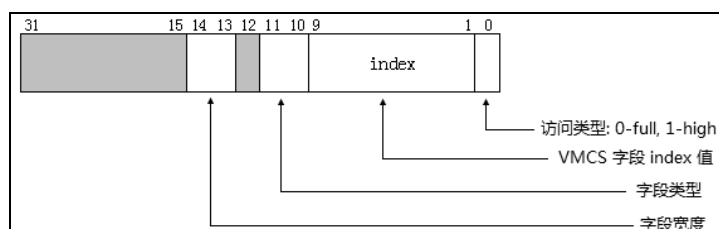


图 3-2

bit 0 指示访问的类型，为 0 时使用 full 类型，为 1 时使用 high 类型访问 64 位字段的高 32 位。除了 64 位字段外，其他宽度字段都是 full 类型。Bits 9:1 是 VMCS 字段的 index 值。

bits 11:10 指示字段的类型，包括下面的值。

- 为 0 值时，属于控制区域内的字段。包括 VM-execution 控制字段，VM-exit 控制字段及 VM-entry 控制字段。
- 为 1 值时，属于只读类型的字段。典型的 VM-exit 信息区域内的字段就属于这类。
- 为 2 值时，属于 guest-state 字段。
- 为 3 值时，属于 host-state 字段。

bits 14:13 指示字段的宽度，包括：

- 为 0 值时，属于 16 位字段。
- 为 1 值时，属于 64 位字段。
- 为 2 值时，属于 32 位字段。
- 为 3 值时，属于 natural-width 字段。

natural-width 字段在支持 64 位架构的处理器中为 64 位，在不支持 64 位架构的处理器中为 32 位。其他几类则属于固定宽度，不会改变。

注意：固定为 64 位的字段对应着两个字段 ID 值，分别是 **full** 类型字段与 **high** 类型字段。例如，guest-state 区域内的 IA32_EFER 字段是 64 位字段，它对应如下两个字段 ID：

- full 字段 ID 值为 00002806H。
- high 字段 ID 值为 00002807H。

在 32 位模式下写 64 位的字段，需要进行两次写入操作。先写入 full 字段（对应 64 位字段的低 32 位部分），再写入 high 字段（对应 64 位字段的高 32 位部分）。其余宽度的字段只有唯一的 ID 值。

3.3.2 不同宽度的字段处理

VMREAD 指令的源操作数提供字段 ID 值，目标操作数接收读取的 VMCS 字段值。VMWRITE 指令的目标操作数提供字段 ID 值，源操作数提供需要写入的值。对于不同宽度的字段有相应的读写原则。

16 位字段

- 在 32 位模式下，VMREAD 指令读取 VMCS 字段值放入目标操作数的 bits 15:0，目标操作数的 bits 31:16 清 0。VMWRITE 指令将源操作数的 bits 15:0 写入相应的 VMCS 字段，源操作数的 bits 31:16 忽略。
- 在 64 位模式下，VMREAD 指令读取 VMCS 字段值放入目标操作数 bits 15:0，目标操作数的 bits 63:16 清 0。VMWRITE 指令将源操作数的 bits 15:0 写入相应的 VMCS 字段，源操作数的 bits 63:16 忽略。

32 位字段

- 在 32 位模式下，VMREAD 指令读取 VMCS 字段值放入 32 位目标操作数。VMWRITE 指令将 32 位源操作数写入 VMCS 字段。
- 在 64 位模式下，VMREAD 指令读取 VMCS 字段值放入目标操作数的 bits 31:0，目标操作数的 bits 63:32 清 0。VMWRITE 指令将源操作数的 bits 31:0 写入相应 VMCS 字段，源操作数的 bits 63:32 忽略。

64 位字段

- 在 32 位模式下，完整的读取需要分两次进行：VMREAD 指令读 **full** 字段，得到 VMCS 字段的 bits 31:0 值放入目标操作数，VMCS 字段的 bits 63:32 忽略。VMREAD 指令读 **high** 字段，得到 VMCS 字段的 bits 63:32 值放入目标操作数，VMCS 字段的 bits 31:0 忽略。
- 在 32 位模式下，完整的写入需要分两次进行：VMWRITE 指令写 **full** 字段，源操作数写入 VMCS 字段的 bits 31:0，VMCS 字段的 bits 63:32 被清 0。VMWRITE 指令写 **high** 字段，源操作数写入 VMCS 字段的 bits 63:32，VMCS 字段的 bits 31:0 不变。
- 在 64 位模式下，直接读写 **full** 字段，对应完整的 64 位字段，无须读写 high 字段。
- 在 64 位模式下，如果读 **high** 字段，则 VMCS 字段的 bits 63:32 值放入目标操作数的 bits 31:0，目标操作数的 bits 63:32 清 0。如果写 **high** 字段，则源操作数的

bits 31:0 写入 VMCS 字段的 bits 63:32，源操作数的 bits 63:32 忽略，VMCS 字段的 bits 31:0 不变。

natural-width 字段

此类型的字段在支持 64 位架构的处理器上是 64 位，否则为 32 位。natural-width 字段只有一个字段 ID（full 字段，不存在 high 字段）。natural-width 字段的读写分为如下三个方面。

(1) 在 64 位架构处理器的 32 位模式下

- VMREAD 指令读 VMCS 字段的 bits 31:0 放入目标操作数，VMCS 字段的 bits 63:32 忽略。
- VMWRITE 指令将源操作数写入 VMCS 字段的 bits 31:0，VMCS 字段的 bits 63:32 清 0。

(2) 在 64 位架构处理器的 64 位模式下

- VMREAD 指令读 64 位 VMCS 字段值放入目标操作数。
- VMWRITE 指令将 64 位源操作数写入 64 位 VMCS 字段。

(3) 在 32 位架构处理器上，只有 32 位模式读写方式

- VMREAD 指令读 32 位 VMCS 字段值放入目标操作数。
- VMWRITE 指令将 32 位源操作数写入 32 位 VMCS 字段。

下面一段代码演示对 VMCS 字段进行写操作：

```
DoVmWrite GUEST_IA32_SYSENTER_CS, [ebp + GUEST_STATE.SysenterCsMsR]
DoVmWrite GUEST_IA32_SYSENTER_ESP, [ebp + GUEST_STATE.SysenterEspMsR]
DoVmWrite GUEST_IA32_SYSENTER_EIP, [ebp + GUEST_STATE.SysenterEipMsR]

DoVmWrite GUEST_IA32_DEBUGCTL_FULL, [ebp + GUEST_STATE.DebugCtlMsR]
DoVmWrite GUEST_IA32_PERF_GLOBAL_CTRL_FULL, [ebp + GUEST_STATE.PerfGlobalCtlMsR]
DoVmWrite GUEST_IA32_PAT_FULL, [ebp + GUEST_STATE.PatMsR]
DoVmWrite GUEST_IA32_EFER_FULL, [ebp + GUEST_STATE.EferMsR]

#ifdef __X64
    DoVmWrite GUEST_IA32_DEBUGCTL_HIGH, [ebp + GUEST_STATE.DebugCtlMsR + 4]
    DoVmWrite GUEST_IA32_PERF_GLOBAL_CTRL_HIGH, [ebp + GUEST_STATE.PerfGlobalCtlMsR
+ 4]
    DoVmWrite GUEST_IA32_PAT_HIGH, [ebp + GUEST_STATE.PatMsR + 4]
    DoVmWrite GUEST_IA32_EFER_HIGH, [ebp + GUEST_STATE.EferMsR + 4]
#endif
```

代码示例中，GUEST_IA32_SYSENTER_CS 是 32 位字段，GUEST_IA32_SYSENTER_ESP 与 GUEST_IA32_SYSENT_EIP 是 natural-width 字段，其余则是 64 位字段。32 位字段与 natural-width 字段只需直接进行写操作便可。

64 位字段需要区别对待，当定义了 __X64 符号时，说明代码当前运行在 64 位模式下，只需要直接写 64 位字段。当运行在 32 位模式下时分两次写入，必须先写 full 字段，再写 high 字段（如果先写 high 字段再写 full 字段，结果只能写 64 位字段的低 32 位）。

3.4 字段 ID 值

下面的内容将以字段宽度分类，列出所有 VMCS 字段 ID 值，分别有：

- (1) 16 位字段 ID
- (2) 64 位字段 ID
- (3) 32 位字段 ID
- (4) Natural-width 字段 ID

3.4.1 16 位字段 ID

16 位字段 ID 如表 3-1 所示。

表 3-1

类 型	字 段 名	ID 值
控制字段	VPID	00000000H
	Posted-interrupt notification vector	00000002H
guest-state 字段	guest ES selector	00000800H
	guest CS selector	00000802H
	guest SS selector	00000804H
	guest DS selector	00000806H
	guest FS selector	00000808H
	guest GS selector	0000080AH
	guest LDTR selector	0000080CH
	guest TR selector	0000080EH
	guest interrupt status	00000810H
host-state 字段	host ES selector	00000C00H
	host CS selector	00000C02H
	host SS selector	00000C04H
	host DS selector	00000C06H
	host FS selector	00000C08H
	host GS selector	00000C0AH
	host TR selector	00000C0CH

这些字段 ID 值的 bits 14:13 为 0 (16 位)。bits 11:10 是类型值，包含了三类 16 位字段：控制字段（类型为 0）、guest-state 字段（类型为 2）及 host-state 字段（类型为 3）。

3.4.2 64 位字段 ID

64 位字段 ID 如表 3-2 所示。

表 3-2

类 型	字 段 名	ID 值
控制字段	I/O bitmap A address (full)	00002000H
	I/O bitmap A address (high)	00002001H
	I/O bitmap B address (full)	00002002H
	I/O bitmap B address (high)	00002003H
	MSR bitmap address (full)	00002004H
	MSR bitmap address (high)	00002005H
	VM-exit MSR-store address (full)	00002006H
	VM-exit MSR-store address (high)	00002007H
	VM-exit MSR-load address (full)	00002008H
	VM-exit MSR-load address (high)	00002009H
	VM-entry MSR-load address (full)	0000200AH
	VM-entry MSR-load address (high)	0000200BH
	executive-VMCS pointer (full)	0000200CH
	executive-VMCS pointer (high)	0000200DH
	TSC-offset (full)	00002010H
	TSC-offset (high)	00002011H
	Virtual-APIC address (full)	00002012H
	Virtual-APIC address (high)	00002013H
	APIC-access address (full)	00002014H
	APIC-access address (high)	00002015H
	posted-interrupt descriptor address (full)	00002016H
	posted-interrupt descriptor address (high)	00002017H
	VM-function controls (full)	00002018H
	VM-function controls (high)	00002019H
	EPT pointer (full)	0000201AH
	EPT pointer (high)	0000201BH
	EOI-exit bitmap 0 (full)	0000201CH
	EOI-exit bitmap 0 (high)	0000201DH
	EOI-exit bitmap 1 (full)	0000201EH
	EOI-exit bitmap 1 (high)	0000201FH
	EOI-exit bitmap 2 (full)	00002020H
	EOI-exit bitmap 2 (high)	00002021H

续表

类 型	字 段 名	ID 值
	EOI-exit bitmap 3 (full)	00002022H
控制字段	EOI-exit bitmap 3 (high)	00002023H
	EPTP-list address (full)	00002024H
	EPTP-list address (high)	00002025H
只读字段	Guest-physical address (full)	00002400H
	Guest-physical address (high)	00002401H
guest-state 字段	VMCS link pointer (full)	00002800H
	VMCS link pointer (high)	00002801H
	guest IA32_DEBUGCTL (full)	00002802H
	guest IA32_DEBUGCTL (high)	00002803H
	guest IA32_PAT (full)	00002804H
	guest IA32_PAT (high)	00002805H
	guest IA32_EFER (full)	00002806H
	guest IA32_EFER (high)	00002807H
	guest IA32_PERF_GLOBAL_CTRL (full)	00002808H
	guest IA32_PERF_GLOBAL_CTRL (high)	00002809H
	guest PDPTE0 (full)	0000280AH
	guest PDPTE0 (high)	0000280BH
	guest PDPTE1 (full)	0000280CH
	guest PDPTE1 (high)	0000280DH
	guest PDPTE2 (full)	0000280EH
	guest PDPTE2 (high)	0000280FH
	guest PDPTE3 (full)	00002810H
	guest PDPTE3 (high)	00002811H
host-state 字段	host IA32_PAT (full)	00002C00H
	host IA32_PAT (high)	00002C01H
	host IA32_EFER (full)	00002C02H
	host IA32_EFER (high)	00002C03H
	host IA32_PERF_GLOBAL_CTRL (full)	00002C04H
	host IA32_PERF_GLOBAL_CTRL (high)	00002C05H

64 位字段 ID 值的 bits 14:13 值为 1, bits 11:10 的类型值包含 4 类: 控制字段 (类型为 0)、只读字段 (类型为 1)、guest-state 字段 (类型为 2) 及 host-state 字段 (类型为 3)。

3.4.3 32 位字段 ID

32 位字段 ID 如表 3-3 所示。

表 3-3

类 型	字 段 名	ID 值
控制字段	Pin-based VM-execution control	00004000H
	Primary processor-based VM-execution control	00004002H
	Exception bitmap	00004004H
	Page-fault error-code mask	00004006H
	Page-fault error-code match	00004008H
	CR3-target count	0000400AH
	VM-exit control	0000400CH
	VM-exit MSR-store count	0000400EH
	VM-exit MSR-load count	00004010H
	VM-entry control	00004012H
	VM-entry MSR-load count	00004014H
	VM-entry interruption-information field	00004016H
	VM-entry exception error code	00004018H
	VM-entry instruction length	0000401AH
	TPR threshold	0000401CH
	Secondary processor-based VM-execution control	0000401EH
	PLE_Gap	00004020H
	PLE_Window	00004022H
只读字段	VM-instruction error	00004400H
	Exit reason	00004402H
	VM-exit interrupt information	00004404H
	VM-exit interrupt error code	00004406H
	IDT-vectoring information field	00004408H
	IDT-vectoring error code	0000440AH
	VM-exit instruction length	0000440CH
	VM-exit instruction information	0000440EH
guest-state 字段	guest ES limit	00004800H
	guest CS limit	00004802H
	guest SS limit	00004804H
	guest DS limit	00004806H
	guest FS limit	00004808H
	guest GS limit	0000480AH

续表

类 型	字 段 名	ID 值
guest-state 字段	guest LDTR limit	0000480CH
	guest TR limit	0000480EH
	guest GDTR limit	00004810H
	guest IDTR limit	00004812H
	guest ES access right	00004814H
	guest CS access right	00004816H
	guest SS access right	00004818H
	guest DS access right	0000481AH
	guest FS access right	0000481CH
	guest GS access right	0000481EH
	guest LDTR access right	00004820H
	guest TR access right	00004822H
	guest interruptibility state	00004824H
	guest activity state	00004826H
	guest SMBASE	00004828H
	guest IA32_SYSENTER_CS	0000482AH
	VMX-preemption timer value	0000482EH
host-state 字段	host IA32_SYSENTER_CS	00004C00H

32 位字段 ID 值的 bits 14:13 值为 2, bits 11:10 的类型值包含 4 类: 控制字段 (类型为 0)、只读字段 (类型为 1)、guest-state 字段 (类型为 2) 及 host-state 字段 (类型为 3)。

3.4.4 natural-width 字段 ID

natural-width 字段 ID 如表 3-4 所示。

表 3-4

类 型	字 段 名	ID 值
控制字段	CR0 guest/host mask	00006000H
	CR4 guest/host mask	00006002H
	CR0 read shadow	00006004H
	CR4 read shadow	00006006H
	CR3-target value 0	00006008H
	CR3-target value 1	0000600AH

续表

类 型	字 段 名	ID 值
控制字段	CR3-target value 2	0000600CH
	CR3-target value 3	0000600EH
只读字段	Exit qualification	00006400H
	I/O RCX	00006402H
	I/O RSI	00006404H
	I/O RDI	00006406H
	I/O RIP	00006408H
	guest-linear address	0000640AH
guest-state 字段	guest CR0	00006800H
	guest CR3	00006802H
	guest CR4	00006804H
	guest ES base	00006806H
	guest CS base	00006808H
	guest SS base	0000680AH
	guest DS base	0000680CH
	guest FS base	0000680EH
	guest GS base	00006810H
	guest LDTR base	00006812H
	guest TR base	00006814H
	guest GDTR base	00006816H
	guest IDTR base	00006818H
	guest DR7	0000681AH
	guest RSP	0000681CH
	guest RIP	0000681EH
	guest RFLAGS	00006820H
	guest pending debug exception	00006822H
	guest IA32_SYSENTER_ESP	00006824H
	guest IA32_SYSENTER_EIP	00006826H
host-state 字段	host CR0	00006C00H
	host CR3	00006C02H
	host CR4	00006C04H
	host FS base	00006C06H
	host GS base	00006C08H
	host TR base	00006C0AH
	host GDTR base	00006C0CH

续表

类 型	字 段 名	ID 值
host-state 字段	host IDTR base	00006C0EH
	host IA32_SYSENTER_ESP	00006C10H
	host IA32_SYSENTER_EIP	00006C12H
	host RSP	00006C14H
	host RIP	00006C16H

natural-width 字段 ID 值的 bits 14:13 值为 3, bits 11:10 的类型值包含 4 类: 控制字段 (类型为 0)、只读字段 (类型为 1)、guest-state 字段 (类型为 2) 及 host-state 字段 (类型为 3)。在本书源码中, 以上的字段 ID 值定义在 inc\vmx.inc 文件中。

3.5 VM-execution 控制类字段

VM-execution 控制类字段主要控制处理器在 VMX non-root operation 模式下的行为能力, 典型地可以控制某些条件引发 VM-exit 事件, 也控制着 VMX 的某些虚拟化功能的开启, 例如 APIC 的虚拟化及 EPT 机制。

VM-execution 控制类字段包括下面所列举的 26 个项目:

- (1) Pin-based VM-execution control 字段
- (2) Processor-based VM-execution control 字段
- (3) Secondary processor-based VM-execution control 字段
- (4) Exception bitmap 字段, PFEC_MASK 及 PFEC_MATCH 字段
- (5) I/O bitmap A address 及 I/O bitmap B address 字段
- (6) TSC offset 字段
- (7) CR0 guest/host mask 字段
- (8) CR0 read shadow 字段
- (9) CR4 guest/host mask 字段
- (10) CR4 read shadow 字段
- (11) CR3-target count 字段
- (12) CR3-target value 0、CR3-target value 1、CR3-target value 2 及 CR3-target value 3 字段
- (13) APIC-access address 字段
- (14) Virtual-APIC address 字段
- (15) TPR threshold 字段

- (16) EOI-exit bitmap 0、EOI-exit bitmap 1、EOI-exit bitmap 2 及 EOI-exit bitmap 3 字段
- (17) Posted-interrupt notification vector 字段
- (18) Posted-interrupt descriptor address 字段
- (19) MSR bitmap address 字段
- (20) Executive-VMCS pointer 字段
- (21) EPTP 字段
- (22) Virtual-processor identifier 字段
- (23) PLE_Gap 字段
- (24) PLE_Window 字段
- (25) VM-function control 字段
- (26) EPTP-list address 字段

Pin-based VM-execution control 字段、Primary processor-based VM-execution control 字段及 Secondary processor-based VM-execution control 字段是 32 位向量值，每个字段提供最多 32 个功能控制。

在 VM-entry 时，处理器检查这些字段。如果检查不通过，产生 VMfailValid 失败。在 VM-exit 信息区域的 VM-instruction error 字段中保存指令错误码，接着执行 VMLAUNCH 或 VMRESUME 指令的下一条指令。

3.5.1 Pin-based VM-execution control 字段

Pin-based VM-execution control 字段提供基于处理器 Pin 接口的控制（INTR 与 NMI），也就是与外部中断和 NMI 相关的配置（包括一些特色功能），如表 3-5 所示。

表 3-5

位 域	控 制 名	配 置	描 述
0	external-interrupt exiting	0 或 1	为 1 时，发生外部中断则产生 VM-exit
2:1	保留位	1	固定为 1 值
3	NMI exiting	0 或 1	为 1 时，发生 NMI 则产生 VM-exit
4	保留位	1	固定为 1 值
5	virtual NMIs	0 或 1	为 1 时，定义 virtual NMI
6	activate VMX-preemption timer	0 或 1	为 1 时，启用 VMX-preemption 定时器
7	process posted-interrupts	0 或 1	为 1 时，启用 posted-interrupt processing 机制处理虚拟中断
31:8	保留位	0	固定为 0 值

在这个字段中，bits 2:1 及 bit 4 属于 default1 位（保留位为 1），bits 31:8 属于

default0 位（保留位为 0），软件需要根据 IA32_VMX_PINBASED_CTLs 与 IA32_VMX_TRUE_PINBASED_CTLs 寄存器来确定（参见 2.5.6.1 节）。

external-interrupt exiting

当“external-interrupt exiting”为 1 时，处理器接收到一个**未被阻塞**的 external-interrupt 请求时将产生 VM-exit。

注意：IF 标志位不能影响 VM-exit（即使 IF = 0），但是处理器的 shutdown 及 wait-for-SIPI 状态能阻塞外部中断产生 VM-exit（参见 4.17.2 节及 4.16.3 节）。

另外，local APIC 的本地中断能被 LVT mask 位（bit 16）及 TPR 寄存器屏蔽，而使用 Fixed delivery 模式的 IPI 能被 TPR 寄存器屏蔽。在这种情况下不能产生 VM-exit。

当“external-interrupt exiting”为 0 时，未被屏蔽与阻塞的外部中断则通过 guest-IDT 提交给处理器执行中断服务例程。

当 VM-exit control 字段中的“acknowledge interrupt on exit”也为 1 时，在退出 VM 时，处理器会响应中断控制器取得中断向量号，并在 VM-exit interruption information 字段中保存这个中断向量号及中断类型等相关信息（参见 3.7 节）。

VM-exit 将返回到 VMM，处理器会**自动清 EFLAGS 寄存器**（除了 bit 1 固定为 1 值外）。这个特性使得 VMM 能够夺取外部中断的 delivery。当“acknowledge interrupt on exit”为 0 时，VMM 使用 STI 指令重新打开 interrupt-window 时，处理器会响应这个外部中断并取得中断向量号，然后通过 host-IDT 来提交中断处理。

NMI exiting

当“NMI exiting”为 1，并且不存在“blocking by NMI”阻塞状态时，VMX non-root operation 内收到 NMI 将引发 VM-exit 产生。为 0 时，则通过 guest-IDT 的 vector 2 来提交 NMI 处理。

virtual NMIs

只有“NMI exiting”为 1 时，“virtual NMIs”位才能被置为 1。当“virtual NMIs”位为 1 时，产生了一个 virtual-NMI 的概念。interruptibility-state 字段的“blocking by NMI”位此时将视为“blocking by virtual-NMI”位，记录着 virtual-NMI 是否被阻塞（参见 3.8.6 节）。

virtual-NMI 的 delivery 是通过**事件注入**方式实现的。当注入一个 virtual-NMI 时，如果这个 virtual-NMI 在 delivery 期间遇到一个错误导致 VM-exit，此时“blocking by virtual-NMI”位被置位，指示 virtual-NMI 被阻塞。

“NMI exiting”位影响到 IRET 指令对 NMI 与 virtual-NMI 阻塞状态的解除作用：

(1) 当“NMI exiting”为 1 时，由于发生 NMI 会产生 VM-exit，IRET 指令不能被认为在 NMI 服务例程内执行。因此，IRET 指令的执行对 NMI 阻塞状态不产生影响。

(2) 当“NMI exiting”为 0 时，处理器认为 IRET 指令**可能会在 NMI 服务例程内执行**。因此，IRET 指令的执行将解除 NMI 阻塞状态。

(3) 另一方面，一个 virtual-NMI (“NMI exiting”与“virtual NMIs”同时为 1 时) 被提交处理器执行，IRET 指令的执行也将解除 virtual-NMI 的阻塞状态。

primary processor-based VM-execution control 字段的“NMI-window exiting”位也使用在 virtual-NMI 上，该控制位指示 virtual-NMI 的窗口区域。只有“virtual-NMIs”为 1 时，“NMI-window exiting”才能被置位。如果“NMI-window exiting”位为 1，那么在“blocking by NMI”为 0 时，完成 VM-entry 后将直接引发 VM-exit。

activate VMX-preemption timer

当“activate VMX-preemption timer”位为 1 时，启用 VMX 提供的定时器功能。VMM 需要在 VMX-preemption timer value 字段里为 VM 提供一个计数值。这个计数值在 VM-entry 操作开始时就进行递减，当减为 0 时产生 VM-exit。递减的步伐依赖于 TSC 及 IA32_VMX_MISC[4:0]值（见 2.5.11 节）。

process posted-interrupts

当“process posted-interrupts”位为 1 时，启用 posted-interrupts processing 机制处理 virtual-interrupt 的 delivery（参见 7.2.14 节）。

只有在下面的位都为 1 的前提下，“process posted-interrupts”位才能被置 1（参见 4.4.1.1 节）：

- “external-interrupt exiting”位。
- VM-exit control 字段的“acknowledge interrupt on exit”位。
- secondary processor-based VM-execution 字段的“virtual-interrupt delivery”位。

在一般情况下，由于“external-interrupt exiting”位为 1，那么收到一个外部中断将会产生 VM-exit。在 posted-interrupts processing 机制下，允许收到被称为“**通知中断**”的外部中断而不产生 VM-exit，继而进行虚拟中断 deliver（参见 3.5.13 节）的相关处理。这个通知中断向量号需要提供在 posted-interrupt notification vector 字段里。

3.5.2 processor-based VM-execution control 字段

processor-based 控制字段提供基于处理器层面上的控制，两个这样的控制字段如下：

- primary processor-based VM-execution control 字段
- secondary processor-based VM-execution control 字段

这两个字段是 32 位向量值，每一位可以对应一个功能控制。在进入 VMX non-root operation 模式后，它们控制着虚拟处理器的行为。primary processor-based VM-execution control 字段与一个 TRUE 寄存器对应，而 secondary processor-based VM-execution control 字段无须 TRUE 寄存器控制（参见 2.5.6 节）。

3.5.2.1 primary processor-based VM-execution control 字段

处理器 VMX non-root operation 模式下的主要行为由这个字段控制，它的 bit 31 也控制是否启用 secondary processor-based VM-execution control 字段。

这个字段部分保留位为 default1（固定为 1 值），部分保留位为 default0（固定为 0 值）。需要通过 IA32_VMX_PROCBASED_CTLs 或 IA32_VMX_TRUE_PROCBASED_CTLs 寄存器来决定（见 2.5.6 节）。

在表 3-6 中，bit 1，bits 6:4，bit 8，bits 14:13 及 bit 26 固定为 1 值，bit 0 及 bits 18:17 固定为 0 值。其余可设置为 0 或 1 值，即关闭或开启某项控制。

表 3-6

位域	控制名	配置	描述
0	保留位	0	固定为 0 值
1	保留位	1	固定为 1 值
2	interrupt-window exiting	0 或 1	为 1 时，在 IF=1 并且中断没被阻塞时，产生 VM-exit
3	Use TSC offsetting	0 或 1	为 1 时，读取 TSC 值时，返回的 TSC 值加上一个偏移值
6:4	保留位	1	固定为 1 值
7	HLT exiting	0 或 1	为 1 时，执行 HLT 指令产生 VM-exit
8	保留位	1	固定为 1 值
9	INVLPG exiting	0 或 1	为 1 时，执行 INVLPG 指令产生 VM-exit
10	MWAIT exiting	0 或 1	为 1 时，执行 MWAIT 指令产生 VM-exit
11	RDPMC exiting	0 或 1	为 1 时，执行 RDPMC 指令产生 VM-exit
12	RDTSC exiting	0 或 1	为 1 时，执行 RDTSC 指令产生 VM-exit
14:13	保留位	1	固定为 1 值
15	CR3-load exiting	0 或 1	为 1 时，写 CR3 寄存器产生 VM-exit
16	CR3-store exiting	0 或 1	为 1 时，读 CR3 寄存器产生 VM-exit
18:17	保留位	0	固定为 0 值
19	CR8-load exiting	0 或 1	为 1 时，写 CR8 寄存器产生 VM-exit
20	CR8-store exiting	0 或 1	为 1 时，读 CR8 寄存器产生 VM-exit
21	Use TPR shadow	0 或 1	为 1 时，启用“virtual-APIC page”页面来虚拟化 local APIC
22	NMI-window exiting	0 或 1	为 1 时，开 virtual-NMI window 时产生 VM-exit
23	MOV-DR exiting	0 或 1	为 1 时，读写 DR 寄存器产生 VM-exit
24	Unconditional I/O exiting	0 或 1	为 1 时，执行 IN/OUT 或 INS/OUTS 类指令产生 VM-exit
25	Use I/O bitmap	0 或 1	为 1 时，启用 I/O bitmap
26	保留位	1	固定为 1 值
27	Monitor trap flag	0 或 1	为 1 时，启用 MTF 调试功能
28	Use MSR bitmap	0 或 1	为 1 时，启用 MSR bitmap

续表

位域	控 制 名	配 置	描 述
29	MONITOR exiting	0 或 1	为 1 时, 执行 MONITOR 指令产生 VM-exit
30	PAUSE exiting	0 或 1	为 1 时, 执行 PAUSE 指令产生 VM-exit
31	activate secondary controls	0 或 1	为 1 时, secondary processor-based VM-execution control 字段有效

interrupt-window exiting

所谓的“interrupt-window”是指外部中断可以被响应的一个有效区间。也就是 eflags.IF=1 并且没有“blocking by MOV-SS”或“blocking by STI”这两类阻塞时。

有两个术语可以应用于 interrupt-window 上:

- **打开中断窗口**, 是指在 eflags.IF=0 时, 使用 STI 指令开启中断许可。那么, 中断窗口的有效区间是: 从 STI 指令的下一条指令执行完毕开始直到使用 CLI 或 POPF 指令关闭中断为止。
- **闭合中断窗口**, 是指使用 CLI 或 POPF 指令关闭中断许可。

```

... ... ; IF=0
sti ; 打开 interrupt-window
mov eax, 1 ; blocking by STI
... ... ; 此时, 产生 VM-exit, eax = 1

```

在上面的示例代码中, 假设 STI 指令之前 eflags.IF=0, STI 指令的下一条指令将存在“blocking by STI”阻塞状态。直到下一条指令执行完毕后, 中断窗口才算有效。在中断窗口区间内, 当“interrupt-window exiting”为 1 时就会产生 VM-exit。

另一种情形是: 当 IF = 1, “interrupt-window exiting”为 1 并且“blocking by STI”和“blocking by MOV-SS”为 0 时, VM-entry 完成后立即产生 VM-exit (参见 4.15.7 节)。如果此时“blocking by STI”或者“blocking by MOV-SS”为 1, 在 VM-entry 完成后执行 guest 的第一条指令后产生 VM-exit。

Use TSC offsetting

当“Use TSC offsetting”为 1 时, 在 VMX non-root operation 中使用 RDTSC, RDTSCP 或者 RDMSR 指令读取 TSC 值, 将返回 TSC 加上一个偏移值。这个偏移值在 TSC offset 字段中设置。

HLT exiting

当“HLT exiting”为 1 时, 在 VMX non-root operation 中执行 HLT 指令将产生 VM-exit。

INVLPG exiting

当“INVLPG exiting”为 1 时, 在 VMX non-root operation 中执行 INVLPG 指令将产生 VM-exit。

MWAIT exiting

当“MWAIT exiting”为 1 时，在 VMX non-root operation 中执行 MWAIT 指令将产生 VM-exit。

RDPMC exiting

当“RDPMC exiting”为 1 时，在 VMX non-root operation 中执行 RDPMC 指令将产生 VM-exit。

RDTSC exiting

当“RDTSC exiting”为 1 时，在 VMX non-root operation 中执行 RDTSC 指令将产生 VM-exit。

CR3-load exiting

当“CR3-load exiting”为 1 时，在 VMX non-root operation 中使用 MOV to CR3 指令来写 CR3 寄存器时，将根据 CR3-target value 与 CR3-target count 字段的值来决定是否产生 VM-exit。

当前 VMX 架构支持 4 个 CR3 target value 字段，CR3-target count 字段值指示前 N 个 CR3 target value 有效 ($N \leq 4$)，那么：

- 当写入 CR3 寄存器的值等于这 N 个 CR3-target value 字段的其中一个值时，不会产生 VM-exit。
- 如果写入 CR3 寄存器的值不匹配这 N 个 CR3-target value 字段中任何一个值或者 CR3-target count 字段值为 0 时 ($N=0$)，将产生 VM-exit。

当“CR3-load exiting”为 0 时，向 CR3 寄存器写入值不会产生 VM-exit。

CR3-store exiting

当“CR3-store exiting”为 1 时，在 VMX non-root operation 中使用 MOV from CR3 指令读 CR3 寄存器将产生 VM-exit。

CR8-load exiting

当“CR8-load exiting”为 1 时，在 VMX non-root operation 中使用 MOV to CR8 指令写 CR8 寄存器将产生 VM-exit。

CR8-store exiting

当“CR8-store exiting”为 1 时，在 VMX non-root operation 中使用 MOV from CR8 指令读 CR8 寄存器将产生 VM-exit。

Use TPR shadow

当“Use TPR shadow”为 1 时，将启用“virtual-APIC page”页面，在 virtual-APIC address 字段里提供一个物理地址作为 4K 的虚拟 local APIC 页面。

Virtual-APIC page 是为了虚拟化 local APIC 而存在，是物理平台上 local APIC 页面

的一个 shadow。在 virtual-APIC page 里存在 VTPR, VPPR, VEOI 等虚拟 local APIC 寄存器。

NMI-window exiting

只有在“NMI exiting”以及“virtual-NMIs”都为 1 时，“NMI-window exiting”才能被置位。这个位实际上被作为“**virtual-NMI window exiting**”控制位。NMI-window 是指在没有 virtual-NMI 被阻塞的情况下，即“blocking by NMI”清 0 时（见 3.5.1 节）。

当“NMI-window exiting”为 1 时，在没有 virtual-NMI 被阻塞的情况下，VM-entry 操作完成后将直接引发 VM-exit。

MOV-DR exiting

当“MOV-DR exiting”为 1 时，在 VMX non-root operation 中执行 MOV 指令对 DR 寄存器进行访问将产生 VM-exit（读或写）。

Unconditional I/O exiting

当“Unconditional I/O exiting”为 1 时，在 VMX non-root operation 中执行 IN/OUT（包括 INS/OUTS 类）指令将产生 VM-exit。当“Use I/O bitmap”位为 1 时，此控制位的作用被忽略。

Use I/O bitmap

当“Use I/O bitmap”为 1 时，需要在 I/O-bitmap A address 及 I/O-bitmap B address 这两个字段中提供物理地址作为 4K 的 I/O bitmap。I/O-bitmap A 对应端口 0000H 到 7FFFH，I/O bitmap B 对应端口 8000H 到 FFFFH。

当使用 I/O bitmap 时，将忽略“Unconditional I/O exiting”控制位的作用。I/O bitmap 的某个 bit 为 1 时，访问该位对应的端口将产生 VM-exit。

Monitor trap flag

当“Monitor trap flag”为 1 时，在 VM-entry 完成后将 pending MTF VM-exit 事件。在 guest 的第一条指令执行完毕后将产生一个 **MTF VM-exit** 事件，也就是由 MTF (Monitor trap flag) 引发的 VM-exit。

Use MSR bitmap

当“Use MSR bitmap”为 1 时，需要在 MSR bitmap address 字段中提供一个物理地址作为 4K 的 MSR bitmap 区域。MSR bitmap 的某位为 1 时，访问该位对应的 MSR 则产生 VM-exit。

MONITOR exiting

当“MONITOR exiting”为 1 时，在 VMX non-root operation 中执行 MONITOR 指令将产生 VM-exit。

PAUSE exiting

当“PAUSE exiting”为 1 时，在 VMX non-root operation 中执行 PAUSE 指令将产生 VM-exit。

activate secondary controls

当“activate secondary controls”为 1 时，将启用 secondary processor-based VM-execution control 字段，这个字段提供基于处理器层上的扩展控制措施。这个字段中的所有控制位在“activate secondary controls”为 0 时属于关闭状态。

3.5.2.2 secondary processor-based VM-execution control 字段

随着处理器架构的不断发展，一些新的 VMX 架构功能也可能被不断地加入。这个字段就用于提供这些扩展的控制功能，或者说新的 VMX 功能，如表 3-7 所示。只有在 primary processor-based VM-execution control 字段的“activate secondary controls”位为 1 时才有效。否则，全部控制位关闭。

表 3-7

位域	控 制 名	配置	描 述
0	virtualize APIC accesses	0 或 1	为 1 时，虚拟化访问 APIC-access page
1	enable EPT	0 或 1	为 1 时，启用 EPT
2	descriptor-table exiting	0 或 1	为 1 时，访问 GDTR, LDTR, IDTR 或者 TR 产生 VM-exit
3	enable RDTSCP	0 或 1	为 0 时，执行 RDTSCP 指令产生 #UD 异常
4	virtualize x2APIC mode	0 或 1	为 1 时，虚拟化访问 x2APIC MSR
5	enable VPID	0 或 1	为 1 时，启用 VPID 机制
6	WBINVD exiting	0 或 1	为 1 时，执行 WBINVD 指令产生 VM-exit
7	unrestricted guest	0 或 1	为 1 时，guest 可以使用非分页保护模式或者实模式
8	APIC-register virtualization	0 或 1	为 1 时，支持访问 virtual-APIC page 内的虚拟寄存器
9	virtual-interrupt delivery	0 或 1	为 1 时，支持虚拟中断的 delivery
10	PAUSE-loop exiting	0 或 1	为 1 时，决定 PASUE 指令是否产生 VM-exit
11	RDRAND exiting	0 或 1	为 1 时，执行 RDRAND 指令产生 VM-exit
12	enable INVPCID	0 或 1	为 0 时，执行 INVPCID 指令产生 #UD 异常
13	enable VM functions	0 或 1	为 1 时，VMX non-root operation 内可以执行 VMFUNC 指令
31:14	保留位	0	固定为 0 值

这个字段的保留位为 default0（固定为 0 值）。软件通过 IA32_VMX_PROCBASED_CTLSS2 寄存器来检查支持哪个控制位。当 primary processor-based VM-execution control 字段的“activate secondary control”控制位为 0 时，这个字段无效。

virtualize APIC accesses

当“virtualize APIC accesses”为 1 时，将实施 local APIC 访问的虚拟化。它引入了一个被称为“APIC-access page”的 4K 页面，这个 APIC-access page 的物理地址必须提供在 APIC-access address 字段里。

启用 local APIC 的访问虚拟化后，guest 软件访问 APIC-access page 页面将产生 VM-exit，或者访问到 virtual-APIC page 内的数据，这取决于“use TPR shadow”及“APIC-register virtualization”控制位。

这里需要清楚地理解“访问 APIC-access page”的三种方式（详见 7.2.3 节）。

(1) **线性地址访问**：线性地址转换的物理地址落在 APIC-access page 内。

(2) **guest-physical address 访问**：在启用 EPT 机制时，guest-physical address 转换的物理地址落在 APIC-access page 内，但这个 guest-physical address 并不是由线性地址转换而来的。

(3) **physical address 访问**：直接访问的物理地址落在 APIC-access page 内，这个物理地址并不是由线性地址转换而来的，也不是由 guest-physical address 转换而来的。

使用线性地址访问 APIC-access page 将实施虚拟化，guest-physical address 访问 APIC-access page 则产生 VM-exit。而 physical address 访问 APIC-access page 是一种不确定行为，可能引发 VM-exit，也可能访问到 virtual-APIC page 内的数据。

guest-physical address 访问 APIC-access page 可以来自下面的情形：

- 在 PAE 分页模式下，执行 MOV to CR3 指令更新 CR3 寄存器时会引起 PDPTEs 加载。CR3 寄存器存放的 guest-physical address 转换为物理地址后落入 APIC-access page 内。
- 在线性地址的转换过程中，由 guest paging-structure 结构内的 guest-physical address 基址访问了 APIC-access page 内的物理地址。例如，guest-PDE 内的 guest-PT 基址转换的物理地址落入了 APIC-access page 内。
- 处理器在更新 guest paging-structure 表项的 access 或 dirty 位时，访问了 APIC-access page 内的物理地址。例如，更新 guest-PTE 的 dirty 位时，guest-PTE 的地址属于 guest physical address，它转换为物理地址后落入 APIC-access page 内。

由 physical address 访问 APIC-access page 可以通过多种途径。例如下面的这些途径：

- 在启用 EPT 机制时，由 EPT paging-structure 结构访问了 APIC-access page，或者处理器更新 EPT paging-structure 表项的 access 或 dirty 位时访问了 APIC-access page。
- 未启用 EPT 机制时，PAE 分页模式下由执行 MOV to CR3 指令而引起 PDPTEs 加载访问了 APIC-access page。或者由 paging-structure 结构访问了 APIC-access page，或者处理器更新 paging-structure 表项的 access 或 dirty 位时访问了 APIC-access page。

- 由于访问了 VMCS 区域（即 VMCS 区域落入 APIC-access page 内），或者访问的 VMCS 区域内所引用的数据结构（例如 MSB bitmap, I/O bitmap 等）的物理地址落在 APIC-access page 内。

关于“guest paging-structure”与“EPT paging-structure”概念请参考下一段落的描述。

enable EPT

当“enable EPT”为 1 时，启用 EPT（Extended Page Table，扩展页表）机制。于是产生了 GPA（guest-physical address）和 HPA（host-physical address）两个概念。

- guest physical address，这是 guest 软件使用的物理地址，但这并不是真正的平台上的物理地址。在启用 EPT 机制后，VM 有自己独立的 guest-physical address 空间，每个 VM 之间的 GPA 空间互不干扰。在启用分页时，guest 软件的线性地址首先需要转换为 GPA，最后 GPA 必须通过 EPT 转换为最终的平台上的物理地址。
- host-physical address，这是物理平台上的地址。在未启用 EPT 机制时，guest 软件的物理地址就是 host-physical address。在启用 EPT 机制时，guest 软件的物理地址是 guest-physical address，而 host 软件的物理地址是 host-physical address。VMM 软件使用的是 host-physical address。

由于这两种物理地址的出现，在 guest 软件的线性地址转换过程中又产生了与这两种物理地址相对应的页转换表结构：

- guest paging-structure，这个页表结构存在于 guest 端，它是 x86/x64 体系开启分页机制下的产物，用来将线性地址转换为物理地址。但在 guest 中，线性地址通过 guest paging-structure 转换为 guest-physical address，而不是真正的平台物理地址。
- EPT paging-structure，这个页表结构只能由 VMM 进行设置，它的作用是将 guest-physical address 转换为 host-physical address。guest 软件并不知道它的存在，因此 guest 软件不能设置 EPT paging-structure。

guest-physical address 和 host-physical address 的产生是为了实现 CPU 的内存虚拟化管理。每个 VM 有自己独立的内存空间而不受 VMM 或其他 VM 的干扰（详见第 6 章）。

当“unrestricted guest”位为 1 时，“enable EPT”位必须为 1。表明如果 guest 使用物理地址，必须启用内存虚拟化。

descriptor-table exiting

当“descriptor-table exiting”为 1 时，VMX non-root operation 内使用 LGDT, LIDT, LLDT, LTR, SGDT, SIDT, SLDT 及 STR 指令来访问描述符表寄存器将产生 VM-exit。

enable RDTSCP

当“enable RDTSCP”为 1 时，VMX non-root operation 内允许使用 RDTSCP 指令。为 0 时，执行 RDTSCP 指令将产生 #UD 异常。

virtualize x2APIC mode

当“virtualize x2APIC mode”为 1 时，启用访问 x2APIC 模式的虚拟化。Guest 软件使用 RDMSR 和 WRMSR 指令访问 x2APIC MSR 时将产生 VM-exit，或者访问到 virtual-APIC page 页面内的数据。

“virtualize x2APIC mode”为 1 时，“virtualize APIC accesses”必须为 0。并且当“use TPR shadow”为 0 时，“virtualize x2APIC mode”必须为 0 值。

enable VPID

当“enable VPID”为 1 时，允许为线性地址转换 cache 提供一个 VPID (Virtual Processor Identifier, 虚拟处理器 ID) 值。线性地址转换 cache 能缓存两类信息：

- (1) EPT 机制未使用时，线性地址转换为物理地址的 linear mappings。
- (2) EPT 机制使用时，线性地址转换为 HPA 的 combined mappings (见 2.6.7 节描述)。

VPID 为每个虚拟处理器定义了一个“虚拟处理器域”的概念，即：每次 VM-entry 时，为虚拟处理器在 virtual-processor identifier 字段中提供一个 VPID 值。这个 VPID 值将对应该虚拟处理器下的所有线性地址转换 cache (参见 6.2.4 节)。

INVVPID 指令可以用来刷新 VPID 值对应的所有线性地址转换 cache，包括 linear mappings (线性映射) 及 combined mappings (合并映射)。

WBINVD exiting

当“WBINVD exiting”为 1 时，VMX non-root operation 内执行 WBINVD 指令将产生 VM-exit。

unrestricted guest

当“unrestricted guest”为 1 时，支持 guest 使用非分页的保护模式或者实模式。在 VM-entry 时，当“unrestricted guest”为 1 时，处理器忽略 CR0 字段 PE 与 PG 位的检查。但是，当 PG 位为 1 时，PE 位必须为 1。

注意：在“unrestricted guest”为 1 时，“enable EPT”也必须为 1。否则，在 VM-entry 时会产生 VMfailValid 失败，在 VM-instruction error 字段里指示“由于无效的 control 字段导致 VM-entry 失败”。如果“unrestricted guest”与 VM-entry control 字段的“IA-32e mode guest”位同时为 1 时，处理器会分别执行它们各自对应的检查。例如“enable EPT”和 guest-state 区域 CR0 字段的 PG 位必须为 1。

启用 unrestricted guest 功能 (“IA-32e mode guest”此时为 0) 后，guest 在运行过程中可能会切换到 IA-32e 模式后产生 VM-exit。基于这种考虑，VMX 架构支持在 VM-exit 时处理器自动将当前的 IA32_EFER.LMA 值写入 VM-entry control 字段的“IA-32e mode guest”控制位，用来记录 guest 是否运行在 IA-32e 模式里。

此时，会导致“IA-32e mode guest”与“unrestricted guest”位同时为 1 值。VM-exit control 字段的“save IA32_EFER”为 1 时，guest IA32_EFER 字段也会得到正确更新。下一次重新进入 VM 时，VMM 无须进行额外的检查和设置，就能够保证正确进入 guest 的 IA-32e 模式运行环境。

较早的处理器不支持 unrestricted guest，软件应该检查 IA32_VMX_PROCBASED_CTLDS2[7]位是否允许设置为 1 值。如果允许则表明支持该功能，否则不支持。

APIC-register virtualization

当“APIC-register virtualization”为 1 时，表明启用 local APIC 寄存器的虚拟化。软件使用线性地址访问 APIC-access page 时，会访问到 virtual-APIC page 内相应的虚拟 local APIC 寄存器。

注意：virtual-APIC page 是物理 local APIC 的一份 shadow 页面，包括 VPTR，VEOI，VISR，VIRR 等虚拟 local APIC 寄存器。

例如，guest 软件使用线性地址访问 APIC-access page 内偏移量 20H 的位置，将访问到 virtual-APIC page 内偏移量 20H 位置上的值（对应 APIC ID 寄存器），返回一个虚拟 local APIC 的 APIC ID 值。如果“APIC-register virtualization”为 0，则表明没有 local APIC 寄存器虚拟化（除了 TPR 外），此时访问 20H 偏移量的值，将会产生 VM-exit。

virtual-interrupt delivery

只有在“external-interrupt exiting”为 1 时，才允许设置“virtual-interrupt delivery”位。因此“virtual-interrupt delivery”为 1 时，一个外部中断请求将产生 VM-exit。可是当改写虚拟 local APIC 的状态时（包括 VPTR，VEOI 及 VICR），虚拟中断将被评估是否可以 delivery。若此时虚拟中断经评估后允许 delivery，则提交到处理器执行（参见 7.2.13 节）。

改写虚拟 local APIC 状态，也就是前面所述的通过“线性地址写 APIC-access page”内的值，最终将改写 virtual-APIC page 内的虚拟 local APIC 寄存器值。只有写 VTPR（virtual-TPR），VEOI（virtual-EOI）及 VICR-low（virtual-ICR 低 32 位）寄存器才可能发生“虚拟中断的评估与 delivery”。启用 posted-interrupt processing 机制也可能发生“虚拟中断的评估与 delivery”操作（参见 7.2.14 节）。

此外，当“virtual-interrupt delivery”为 1 时，在 VM-entry 时也会进行虚拟中断的评估与 delivery。虚拟中断 delivery 功能在较新的处理器上才支持。软件需要通过 IA32_VMX_PROCBASED_CTLDS[9]位检查是否支持该功能。

如果不支持该功能，或者“virtual-interrupt delivery”为 0，改写 VEOI 与 VICR 寄存器状态时最终将产生 VM-exit。而 VTPR 是个例外，改写 VPTR 值时将写入 VPTR。

PAUSE-loop exiting

在“PAUSE exiting”为 1 时，“PAUSE-loop exiting”的作用被忽略。此外，在 CPL 为非 0 的情况下也被忽略。

当“PAUSE exiting”为 0，“PAUSE-loop exiting”为 1，并且 CPL=0 时，处理器发现 PAUSE 指令在 **PAUSE-loop** 里执行的时间超过 VMM 设置的 **PLE_window** 值，就会产生 VM-exit。PLE_window 是 PAUSE 指令在一个循环（PAUSE-loop）里的执行时间的上限值。

PAUSE-loop 的检测由 **PLE_Gap** 值来决定。这个 PLE_Gap 值设置了两条 PAUSE 指令之间执行时间间隔的上限值，它用来检测 PAUSE 指令是否出现在一个循环里（见 3.5.19 节）。

RDRAND exiting

当“RDRAND exiting”为 1 时，在 VMX non-root operation 中执行 RDRAND 指令将产生 VM-exit。

enable INVPCID

当“enable INVPCID”为 1 时，在 VMX non-root operation 中允许执行 INVPCID 指令，否则将产生 #UD 异常。

enable VM functions

当“enable VM functions”为 1 时，允许在 VMX non-root operation 中执行 VMFUNC 指令，而不会产生 VM-exit。同时也指示“VM-function controls”字段有效。

注意：编写此书时，在最新的 VMX 版本里，secondary processor-based VM-execution control 字段加入了两个控制位“VMCS shadowing”和“EPT-violation #VE”，本书并不打算介绍这两项功能。

3.5.3 exception bitmap 字段

exception bitmap 字段是一个 32 位值，每个位对应一个异常向量。例如，bit 13 对应 #GP 异常（向量号为 13），bit 14 对应 #PF 异常（向量号为 14）。

在 VMX non-root operation 中，如果发生异常，处理器检查 exception bitmap 相应的位，该位为 1 时将产生 VM-exit，为 0 时则正常通过 guest-IDT 执行异常处理例程。当一个 **triple-fault** 发生时，将直接产生 VM-exit。

3.5.4 PFEC_MASK 与 PFEC_MATCH 字段

对于 #PF 异常的 VM-exit 处理有些特殊，允许根据引发 #PF 异常的条件进行筛选。引入了两个 32 位的字段 PFEC_MASK（page-fault error-code mask）与 PFEC_MATCH

(page-fault error-code match)，这两个字段决定如何进行筛选。

当#PF 发生时，产生的#PF 异常错误码 (PFEC) 格式如图 3-3 所示。

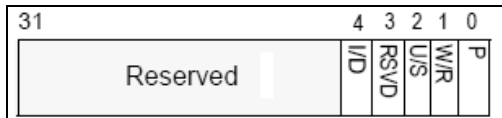


图 3-3

错误码的 bits 4:0 指示#PF 产生的原因，如下所示。

(1) P 位 (bit 0)：P=0 时，指示#PF 由 not-present 而引发。P=1 时，说明由其他条件引发。

(2) W/R 位 (bit 1)：为 0 时，表示#PF 发生时进行了读访问。为 1 时，表示#PF 发生时进行了写访问。

(3) U/S 位 (bit 2)：为 0 时，表示#PF 发生时当前运行在 supervisor 权限。为 1 时，表示#PF 发生时当前运行在 User 权限。

(4) RSVD 位 (bit 3)：为 0 时，保留位正常。为 1 时，指示保留位为 1。

(5) I/D 位 (bit 4)：为 0 时，说明#PF 发生在 fetch data。为 1 时，指示在 fetch instruction 时引发#PF 异常。

#PF 的错误码实际上描述了#PF 异常发生时的访问细节。举例来说，如果错误码的值为 03H，说明#PF 发生时的情况是：

- (1) P=1。
- (2) W/R=1，正在写访问。
- (3) U/S=0，处于 supervisor 权限。
- (4) RSVD=0，保留位正常。
- (5) I/D=0，非执行指令。

在 PFEC_MASK 与 PFEC_MATCH 的控制下，结合 PFEC 有下面两种情形：

- (1) PFEC & PFEC_MASK 等于 PFEC_MATCH 时，产生 VM-exit。
- (2) PFEC & PFEC_MASK 不等于 PFEC_MATCH 时，不会产生 VM-exit。

前面的例子（错误码为 03H）中，如果希望在 not-present 时产生 VM-exit，则设置 PFEC_MASK 为 01H 值，而 PFEC_MATCH 为 0 值。如果希望在读访问时产生 VM-exit，则设置 PFEC_MASK 为 02H，而 PFEC_MATCH 为 0H。

如果需要所有的#PF 异常都产生 VM-exit，可以设置 PFEC_MASK 与 PFEC_MATCH 都为 0 值。而如果希望所有的#PF 都不产生 VM-exit，可以设置 PFEC_MASK 为 0 值，PFEC_MATCH 为 FFFFFFFFH 值（任何一个非 0 值都可以）。

3.5.5 I/O bitmap address 字段

当“use I/O bitmap”控制位为 1 时，使用 I/O bitmap 来控制 I/O 指令对 I/O 地址的访问。I/O bitmap 的每个位对应一个 I/O 地址，当位为 1 时，访问相应的 I/O 地址将产生 VM-exit。此外，使用 I/O bitmap 将忽略“unconditional I/O exiting”控制位的作用（见 3.5.2.1 节）。

VMM 需要在 I/O bitmap address 字段中提供 I/O bitmap 物理地址。在 x86/x64 体系中共有 64K I/O 空间，地址从 0000H 至 FFFFh。那么，64K 个地址值需要有 8K 字节来对应。VMX 架构里提供了两个 I/O bitmap address 字段（A 与 B）。I/O bitmap A 对应 I/O 地址从 0000H 到 7FFFh，I/O bitmap B 对应 I/O 地址从 8000H 到 FFFFh。

3.5.6 TSC offset 字段

当“Use TSC offsetting”为 1 时，在 TSC offset 字段中提供一个 64 位的偏移值。在 VMX non-root operation 中执行 RDTSC，RDTSCP 或者 RDMSR 指令读取 TSC 时，返回的值为 TSC 加上 TSC offset。

前提条件是：

- (1) 使用 RDTSC 指令时，“RDTSC exiting”位为 0 值。
- (2) 使用 RDTSCP 指令时，“enable RDTSCP”位为 1 值。
- (3) 使用 RDMSR 指令时，MSR read bitmap 相应位为 0 值。

3.5.7 guest/host mask 与 read shadow 字段

有如下两组 guest/host mask 与 read shadow 字段：

- (1) CR0 寄存器的 guest/host mask 与 read shadow 字段。
- (2) CR4 寄存器的 guest/host mask 与 read shadow 字段。

这些字段是 natural-width 类型的字段，在 64 位架构处理器上是 64 位宽，否则为 32 位宽。guest/host mask 的每个位有如下两层意义：

- (1) 位为 1 时，表示该位权利属于 host 所有。
- (2) 位为 0 时，表示该位 guest 有权设置。

位属于 host

guest 读该位时，则返回 read shadow 相应位的值。写该位时，如果写入的值不等于 read shadow 相应位的值，则产生 VM-exit。

位 guest 有权设置

guest 读写该位反映在真实的寄存器上，不会产生 VM-exit。当 guest 读该位时，返回

CR0/CR4 寄存器该位的值。写该位时，CR0/CR4 寄存器该位的值将被写入。

举个例子来说明位的 host 与 guest 权利。假如 CR4 guest/host mask 为 00002021H 值，CR4 read shadow 为 00002020H 值，而 CR4 寄存器的值为 00002220H。此时，CR4 寄存器的 bit 0，bit 5 以及 bit 13 属于 host 权利，其余位则 guest 有权利。

```
mov eax, cr4           ; 返回 00002220h
mov eax, 2024h         ; bit 2 = 1
mov cr4, eax           ; cr4 被写入 00002024h 值
mov eax, 2021h
mov cr4, eax           ; 失败，产生 VM-exit
```

当读 CR4 寄存器时，将返回 00002220h 值。返回值的 bit 0，bit 5 及 bit 13 来自 read shadow 字段对应的 bit 0，bit 5 及 bit 13，而 bit 9 则来自 CR4 寄存器的 bit 9。

当向 CR4 寄存器写入 00002024h 值时，由于 bit 2 属于 guest 有权利，所以 bit 2 值能写入 CR4 寄存器。并且 bit 0，bit 5 及 bit 13 的值等于 read shadow 字段对应位的值，所以这个写入是成功的。

当向 CR4 寄存器写入 00002021h 值时，由于 bit 0 属于 host 权利，并且写入值的 bit 0 不等于 read shadow 字段的 bit 0 值，因此，这个写入失败将产生 VM-exit，CR4 寄存器的值不变。

3.5.8 CR3-target 字段

这是一组字段，包括 **CR3-target count** 字段与 4 个 **CR3-target value** 字段（从 CR3-target value 0 到 CR3-target value 3 字段）。当前 VMX 实现的版本最多支持 4 个 CR3-target value 值。软件可以查询 IA32_VMX_MISC[24:16]的值来获得支持 CR3-target value 的数量（见 2.5.11 节）。

若写入 CR3-target count 字段的值大于 4，VM-entry 时会产生 VMfailValid 失败。指令错误编号为 7，指示“由于无效的控制域字段导致 VM-exit 失败”。

在“CR3-load exiting”为 1 时，CR3-target count 与 CR3-target value 将决定写 CR3 寄存器是否会产生 VM-exit。以 CR3-target count 字段值作为 N ($N \leq 4$)：

- (1) 当向 CR3 写入的值等于这 N 个 CR3-target 值的其中一个时，不会产生 VM-exit。
- (2) 当向 CR3 写入的值不等于 N 个 CR3-target 值中的任何一个时，则产生 VM-exit。

N 值将决定有几个 CR3-target 值可以用来做对比。例如 CR3-target count 设为 3 时，表示前 3 个 CR3-target value 字段的值被用来做对比。

3.5.9 APIC-access address 字段

当“virtualize APIC accesses”为 1 时，APIC-access address 字段有效，需要提供一个物理地址作为 4K 的 **APIC-access page** 页面。

guest 软件线性访问这个区域内的地址时，将虚拟化这个访问行为。结果或者是产生 VM-exit，或者在 virtual-APIC page 内访问到虚拟 local APIC 的数据（参见 3.5.2.2 节）。

3.5.10 virtual-APIC address 字段

当“use TPR shadow”为 1 时，virtual-APIC address 字段有效，需要提供一个物理地址作为 4K 的 **virtual-APIC page** 页面。

virtual-APIC page 是物理平台上 local APIC 的一份 shadow 页面，VMM 可以初始化和维护 virtual-APIC page 内的数据。当“virtualize APIC access”与“use TPR shadow”都为 1 时，使用一个偏移值线性访问 APIC-access page 内的数据，将会访问到 virtual-APIC page 内偏移值相同的数据。也就是：处理器将对 APIC-access page 的访问转化为对 virtual-APIC page 的访问。

在“virtualize x2APIC mode”为 1 时，软件使用 RDMSR 与 WRMSR 指令来访问 local APIC，最终也会转化为对 virtual-APIC page 的访问。

在“APIC-registers virtualization”为 1 时，将虚拟化大部分的 local APIC 寄存器（参见 7.2.5 节及表 7-2）。此时，在 virtual-APIC page 内可以访问到这些虚拟 local APIC 寄存器，例如 VIRR, VISR 等。

3.5.11 TPR threshold 字段

TPR threshold 字段仅在“use TPR shadow”为 1 时有效。TPR threshold 字段的 bits 3:0 提供一个 VTPR 门坎值，bits 31:4 清为 0 值。当写 VPTR 值，而 VPTR[7:4]值低于 TPR threshold 的 bits 3:0 时将产生 VM-exit。

3.5.12 EOI-exit bitmap 字段

这是一组 64 位宽的字段，共有 4 个 EOI-exit bitmap，分别为：

- (1) EOI-exit bitmap 0，对应向量号从 0H 到 3FH。
- (2) EOI-exit bitmap 1，对应向量号从 40H 到 7FH。
- (3) EOI-exit bitmap 2，对应向量号从 80H 到 BFH。
- (4) EOI-exit bitmap 3，对应向量号从 C0H 到 FFH。

这些字段仅在“virtual-interrupt delivery”为 1 时有效，用于控制发送 EOI 命令时是否产生 VM-exit。当 EOI-exit bitmap 的位为 1 时，对应向量号的中断服务例程在发送 EOI 命令时，将产生 VM-exit。为 0 时，则进行“虚拟中断的评估及 delivery”，最终的结果可能是另一个虚拟中断被 deliver 执行。

3.5.13 posted-interrupt notification vector 字段

在“process posted interrupts”为 1 时启用 posted-interrupt processing 机制。“process posted-interrupts”可以被置位的前提是“external-interrupt exiting”与“virtual-interrupt delivery”都为 1 值。在一般情况下，guest 收到外部中断请求时，要么全部引发 VM-exit，要么全部 deliver 执行。

但是 posted-interrupt processing 机制允许处理器接收到一个“通知性质”的外部中断而不会产生 VM-exit，其他的外部中断仍然会产生 VM-exit。这个中断通知处理器进行下面的一些特殊的处理：

- 从一个被称为“posted-interrupt descriptor”的结构中读取预先设置好的中断向量号，并复制到 virtual-APIC page 内的 VIRR 寄存器里，从而构成 virtual-interrupt 请求。
- 对 virtual-interrupt 进行评估。通过评估后，**未屏蔽的虚拟中断**将由 guest-IDT 进行 deliver 执行（参见 7.2.13 节）。

这个**通知性质的外部中断向量号**提供在 posted-interrupt notification vector 字段，这个字段为 16 位宽。在使用 posted-interrupt processing 机制前，VMM 需要设置这个通知中断向量号以供处理器进行检查对比。

3.5.14 posted-interrupt descriptor address 字段

在 posted-interrupt processing 机制处理下，VMM 在一个被称为“posted-interrupt descriptor”的数据结构里预先设置需要给 guest 传递执行的中断向量号。Posted-interrupt descriptor address 字段提供这个数据结构的 64 位物理地址。关于 posted-interrupt descriptor 结构参见 7.2.14 节表 7-3。

3.5.15 MSR bitmap address 字段

在“use MSR bitmap”为 1 时，使用 MSR bitmap 来控制对 MSR 的访问。当 MSR bitmap 的某位为 1 时，访问该位所对应的 MSR 将产生 VM-exit。MSR bitmap address 字段提供 MSR bitmap 区域的 64 位物理地址。

MSR bitmap 区域为 4K 大小，分别对应低半部分和高半部分 MSR 的读及写访问：

(1) 低半部分 MSR read bitmap，对应 MSR 范围从 00000000H 到 00001FFFH，用来控制这些 MSR 的读访问。定位在 MSR bitmap 的首个 1K 区域里。

(2) 高半部分 MSR read bitmap，对应 MSR 范围从 C0000000H 到 C0001FFFH，用来控制这些 MSR 的读访问。定位在 MSR bitmap 的第 2 个 1K 区域里。

(3) 低半部分 MSR write bitmap，对应 MSR 范围从 00000000H 到 00001FFFH，用来控制这些 MSR 的写访问。定位在 MSR bitmap 的第 3 个 1K 区域里。

(4) 高半部分 MSR write bitmap，对应 MSR 范围从 C0000000H 到 C0001FFFH，用来控制这些 MSR 的写访问。定位在 MSR bitmap 的第 4 个 1K 区域里。

MSR bitmap 的某位为 0 时，访问该位所对应的 MSR 不会产生 VM-exit。

3.5.16 executive-VMCS pointer

这个字段用于 SMM dual-monitor treatment (SMM 双重监控处理) 机制下，当发生 SMM VM-exit 时，这个字段用来保存 executive-monitor 的 VMCS pointer (参见 2.6.6.5 节与 3.2.2 节)。

由于 SMM VM-exit 可能发生在 VMM 中，因此，executive-VMCS pointer 值可能等于 VMXON pointer，也可能等于进入 SMM-transfer monitor 前的 current-VMCS pointer。

处理器读取这个字段，用来决定从 SMM 是返回到 VMM 还是 VM。如果返回到 VMM，那么 guest-state 区域的 VMCS-link pointer 字段必须提供之前的 current-VMCS pointer。

3.5.17 EPTP 字段

在“enable EPT”为 1 时，支持 guest 端物理地址转换为 host 端的最终物理地址。Guest 端的物理地址被称为“guest-physical address”，而最终的物理地址被称为“host-physical address”。

EPT (Extended-Page Table, 扩展页表) 用于将 guest-physical address 转换为 host-physical address。EPT 的使用原理与保护模式的分页机制下的页表是一样的。VMX non-root operation 内的保护模式分页机制的页表结构被称为“guest paging-structure”，而 EPT 页表结构被称为“EPT paging-structure”。

64 位的 EPTP 字段提供 EPT PML4T 表的物理地址，与分页机制下的 CR3 寄存器作用类似。它的结构如图 3-4 所示。

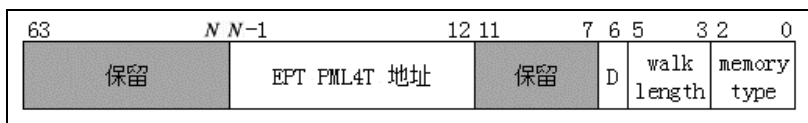


图 3-4

bits 2:0 指示 EPT paging-structure 的内存类型 (属于 VMM 管理的数据区域)，当前只支持 UC (值为 0) 与 WB (值为 6) 类型。bits 5:3 指示 EPT 页表结构的层级，这个值加上 1 才是真正级数。例如，它的值为 3 时表示有 4 级页表结构。

bit 6 为 1 时，指示 EPT 页表结构项里的 access 与 dirty 标志位有效 (EPT 表项的 bit 8 与 bit 9)。处理器会更新 EPT 表项的这两个标志位。

bits $N-1:12$ 提供 EPT PML4T 表的物理地址 ($N = \text{MAXPHYADDR}$)，这个值对齐在 4K 边界上。如果 MAXPHYADDR 为 36，则 bits 35:12 提供 36 位的物理地址的高 24 位。bits $N-1:12$ 的地址值也被称为“EP4TA”，作为一个域标识用于 cache 的维护操作。

bits 11:7 与 bits 63: N 是保留位，须为 0 值。如果 MAXPHYADDR 为 36，则 bits 63:36 是保留位。

3.5.18 virtual-processor identifier 字段

当“enable VPID”为 1 时，virtual-processor identifier 字段提供一个 16 位的 VPID 值。这个 VPID 用来标识虚拟处理器的 cache 域。处理器会在 cache 中维护 VPID 对应的一份 cache。如果每次 VM-entry 使用不同的 VPID 值，这样就为 VM 指定多个虚拟处理器 cache 域。

处理器维护每个 VPID 值对应的一份 cache 信息。另外，VMX 架构引入了 INVVPID 指令来刷新由 VPID 对应的 cache 信息（见 2.6.7.2 节）。

3.5.19 PLE_Gap 与 PLE_Window 字段

当“PAUSE-loop exiting”为 1，“PAUSE exiting”为 0 并且 CPL=0 时，可以控制在包含 PAUSE 指令的一个循环里，当执行的时间超过设置的上限值后产生 VM-exit。由 PLE_Gap 与 PLE_Window 这两个 32 位的字段来共同决定 PAUSE-loop exiting 行为。

“PAUSE-loop exiting”功能的引进主要是解决在一个长时间的 spin lock（自旋锁）等待循环里浪费处理器的时间。在这个自旋锁等待循环里处理器可以产生 VM-exit，从而去执行其他的任务。

```

acquire_lock:
    bts DWORD [spin_lock], 0                ; 尝试获取自旋锁，并上锁
    jnc acquire_lock_ok                    ; 成功获取，转入下一步

    ;;
    ;; 失败后，下面不断地测试锁，直到锁打开
    ;;
test_lock:
    bt DWORD [spin_lock], 0                ; 测试自旋锁是否打开
    jnc acquire_lock                       ; 如果锁已经放开，则再次尝试获取自旋锁
    pause                                   ;
    jmp test_lock                          ; 如果锁还是被锁上，则不断进行检测

    ;;
    ;; 后续工作
    ;;
acquire_lock_ok:
    ... ..

```

上面的代码示例是一个典型自旋锁的 test-test-and-set 指令序列，PAUSE 指令出现在测试循环里。那么，处理器是如何知道 PAUSE 指令出现在一个循环呢？这是“PAUSE-

loop exiting”功能的关键，PLE_Gap 字段就用来完成这项任务。

PLE_Gap 值

PLE_Gap 用来检测 PAUSE 指令是否出现在一个循环里，它是两条 PAUSE 指令执行的时间间隔的上限值。当本次执行 PAUSE 指令与前一次执行 PAUSE 指令之间的时间间隔超过 PLE_Gap 值时，处理器就认为本次执行的 PAUSE 指令出现在一个新的循环（即 PAUSE-loop）里。

如果这个被认为出现在新的循环里的 PAUSE 指令与下一次执行 PAUSE 指令的时间间隔又超过了 PLE_Gap 值，那么，处理器会认为下一次执行的 PAUSE 指令又出现在另一个新的循环里。换言之，之前认为的那个 PAUSE-loop 被作废！

周而复始，处理器使用这种方法不断地检测 PAUSE-loop 的存在。因此，也可能不断地刷新 PAUSE-loop 的位置（因为可能不断有新的 PAUSE-loop 被确认）。

当处理器已经确认了 PAUSE-loop 的存在后，就会记录 PAUSE-loop 里的第一条 PAUSE 指令执行时的时间值。这里，我们可以用一个变量 **first_pause_time** 来代表这个时间值。

```
/*
 * 如果当前 PAUSE 指令执行时间与上次 PAUSE 指令执行时间间隔大于 PLE_Gap 值，
 * 则认为出现了 PAUSE-loop
 */
if ((current_pause_time - last_pause_time) > PLE_Gap)
{
    /*
     * PAUSE-loop 内的第一条 PAUSE 指令执行时间为当前 PAUSE 指令执行时间
     */
    first_pause_time = current_pause_time;
}
```

在上面 C 代码的描述里，current_pause_time 是指本次 PAUSE 指令执行时的 TSC (Time-Stamp Counter) 时间值。last_pause_time 是指上一次 PAUSE 指令执行的 TSC 时间值。当然，PLE_Gap 也是一个 TSC 时间值。

注意：**PLE_Gap 应该是一个很小的值**。因为它是指一个循环里两条 PAUSE 指令执行时间间隔的最大值。而这个时间间隔是非常短的。如果设置一个比较大的值，就失去了意义和作用。

PLE_window 值

PLE_window 值用来决定什么时候产生 VM-exit，也就是说 guest 允许在 PAUSE-loop 中执行多长时间。它是从 **first_pause_time** 时间算起直到产生 VM-exit 的这段时间间隔的 TSC 值。

注意：PLE_window 应该是一个较大的值（相对 PLE_Gap 来说）。因为，它指定了 PAUSE-loop 执行时间的长短。

接前面所述，当 current_pause_time（本次 PAUSE 指令执行时间）与 last_pause_time

(上一次 PAUSE 指令执行时间)的时间间隔大于 PLE_Gap 时,处理器认为本次 PAUSE 指令出现在一个新的 PAUSE-loop 里。

但是,当这个时间间隔小于等于 PLE_Gap 时,处理器检查 current_pause_time 与 first_pause_time (PAUSE-loop 里第一条 PAUSE 指令执行时间)之间的时间间隔,如果大于 PLE_window 值,将导致 VM-exit 发生。如果小于等于 PLE_window 值,处理器继续执行 spin lock 等待循环,直到获得 spin lock 或者发生 VM-exit。

```
/*
 * 如果当前 PAUSE 指令执行时间与上次 PAUSE 指令执行时间间隔大于 PLE_Gap 值
 */
if ((current_pasue_time - last_pause_time) > PLE_Gap)
{
    /*
     * PAUSE-loop 内的第一条 PAUSE 指令执行时间为当前 PAUSE 指令执行时间
     */
    first_pause_time = current_pause_time;
}
else
{
    if ((current_pasue_time - first_pause_time) > PLE_window)
    {
        /*
         * 如果 PAUSE 当前执行时间与 first_pause_time 之间的间隔大于 PLE_window 值
         * 则产生 VM-exit
         */
    }
}

/*
 * 注意: last_pause_time 会被记录,更新为当前 PAUSE 指令执行时间
 */
last_pause_time = current_pasue_time;
```

上面的示例代码中,展示了处理器的“PAUSE-loop exiting”功能全貌。在 guest 中第一次执行 PAUSE 指令,它会满足 $(current_pause_time - last_pause_time) > PLE_Gap$ 的条件,找到 PAUSE-loop,然后记录 first_pause_time 时间值。

当进入 guest 后,处理器认为第一次 last_pause_time 时间就是在 VM-entry 后(即作为首次执行 PAUSE 指令时间值)。在随后的每次 PAUSE 指令执行时都会更新 last_pause_time 时间值。

3.5.20 VM-function control 字段

当 secondary processor-based VM-execution control 字段的“enable VM functions”为 1 时,这个 64 位的 VM-function control 字段有效。它提供对 VMFUNC 指令在 VMX non-root operation 内的控制。

VM-functions control 字段每个位对应一个 VM 功能号,最大功能号为 63。VMFUNC 指令执行时,VM 功能号提供在 EAX 寄存器中,当提供的功能号大于 63 时,将产生#UD 异常。

表 3-8

位域	控制名	配置	描 述
0	EPTP switching	0 或 1	为 1 时，可以调用 EPTP switching 功能
63:1	保留位	0	固定为 0 值

如表 3-8 所示，VMX 当前仅支持 0 号功能，它是“EPTP switching”功能。字段的“EPTP switching”位为 1 时，可以调用这个功能。

在 VMX non-root operation 内执行 VMFUNC 指令时，如果使用的功能号在 VM-functions control 字段相应位为 0，将产生 VM-exit。例如“EPTP switching”位为 0，在执行 VMFUNC 指令（提供 0 号功能）时，将产生 VM-exit（另见 2.6.8.2 节）。

软件应该检查 IA32_VMX_VMFUNC 寄存器，确定支持哪个 VM 功能号。IA32_VMX_VMFUNC 寄存器在“enable VM functions”位允许被置位下有效。

3.5.21 EPTP-list address 字段

在前面所述的 VM-functions control 字段“EPTP switching”位为 1 时，EPTP-list address 字段有效，提供 EPTP list 表的 64 位物理地址。EPTP list 表是 4KB 宽，EPTP list 的每个表项为 8 字节，故共有 512 个表项。每个表项就是一个 EPTP 值，如图 3-5 所示。

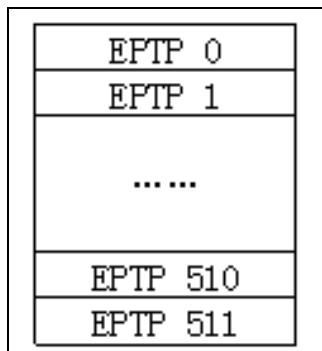


图 3-5

EPTP switching 服务例程根据表项号读取 EPTP 值，并检查 EPTP 值。如果 EPTP 值有效，则写入 VMCS 的 EPTP 字段，更新 cache 的 EP4TA 域。EPTP 值无效则产生 VM-exit。

在调用这个例程时，EPTP list 的表项号提供在 ECX 寄存器里，最大的表项号为 511。如果给 ECX 提供的表项号大于 511，则产生 VM-exit（关于 EPTP switching 服务例程参见 6.1.11 节）。

3.6 VM-entry 控制类字段

VM-entry 区域的控制字段包括下面这些：

- (1) VM-entry control 字段。
- (2) VM-entry MSR-load count 字段。
- (3) VM-entry MSR-load address 字段。
- (4) VM-entry interruption-information 字段。
- (5) VM-entry exception error code 字段。
- (6) VM-entry instruction length 字段。

这些字段用来控制在 VM-entry 时处理器的行为。在 VM-entry 时，处理器检查这些字段。如果检查不通过，产生 VMfailValid 失败，在 VM-instruction error 字段中保存错误号，接着执行 VMLAUNCH 或 VMRESUME 指令下面的指令。

3.6.1 VM-entry control 字段

VM-entry control 字段是 32 位宽，每个位对应一个控制功能，如表 3-9 所示。

表 3-9

位域	控 制 名	配置	描 述
1:0	保留位	1	固定为 1 值
2	load debug controls	0 或 1	为 1 时，加载 debug 寄存器
8:3	保留位	1	固定为 1 值
9	IA-32e mode guest	0 或 1	为 1 时，进入 IA-32e 模式
10	entry to SMM	0 或 1	为 1 时，进入 SMM 模式
11	deactivate dual-monitor treatment	0 或 1	为 1 时，返回 executive monitor，关闭 SMM 双重监控处理
12	保留位	1	固定为 1 值
13	load IA32_PERF_GLOBAL_CTRL	0 或 1	为 1 时，加载 IA32_PERF_GLOBAL_CTRL
14	load IA32_PAT	0 或 1	为 1 时，加载 IA32_PAT
15	load IA32_EFER	0 或 1	为 1 时，加载 IA32_EFER
31:16	保留位	0	固定为 0 值

设置时，软件应通过 IA32_VMX_ENTRY_CTLS 或 IA32_VMX_TRUE_ENTRY_CTLS 寄存器来检查当前 VMX 支持哪个功能，并且要确保保留位的设置是适当的。

如表 3-9 所示，bits 1:0，bits 8:3 以及 bit 12 是 default1 保留位，固定为 1 值。bits 31:16 是 default0 保留位，固定为 0 值。

load debug controls

当“load debug controls”为 1，在 VM-entry 时将从 guest-state 区域的相应字段中加载 DR7 和 IA32_DEBUGCTL 寄存器。否则不会加载。

IA-32e mode guest

当“IA-32e mode guest”为 1 时，表明进入 VM 后，guest 运行在 IA-32e 模式。由 CS.L 与 CS.D 指示 guest 处于 64 位模式还是 compatibility 模式。CS.L=1 并且 CS.D=0 时进入 64 位模式。CS.L=0 时进入 compatibility 模式。

只有发起 VM-entry 操作时，当前的 IA32_EFER.LMA 为 1（即此时 host 是 IA-32e 模式），才允许进入 IA-32e 模式的 guest。如果 host 是 legacy 模式，不可能进入 IA-32e 模式。

“IA-32e mode guest”与 secondary processor-based VM-execution control 字段的“unrestricted guest”位可能会同时为 1。在支持 unrestricted guest 功能的处理器中，也支持 IA32_VMX_MISC[5] 为 1 值（见 2.5.11 节）。在每次 VM-exit 操作时，处理器自动将当前的 IA32_EFER.LMA 值写入“IA-32e mode guest”控制位，用来记录 guest 是否运行在 IA-32e 模式。

下面是关于“IA-32e mode guest”与“unrestricted guest”位的两点描述：

(1) 当“unrestricted guest”为 1 时，允许 guest 运行在非分页保护模式或者实模式。如果 guest 在这两个模式的运行过程中切换到了 IA-32e 模式，有可能会在 64-bit 模式或者 compatibility 模式中产生 VM-exit。那么处理器会将“IA-32e mode guest”置位，并且更新 guest-state 区域的 CS access rights 字段的 CS.L 与 CS.D 位（指示处于 64 位模式还是 compatibility 模式）。此时“unrestricted guest”与“IA-32e mode guest”位同时为 1。

(2) 在 x64 体系上，IA32_EFER.LMA 等于 **CR0.PG & IA32_EFER.LME**。也就是在 VM-entry 完成后 CR0.PG 与 LME 位同时为 1 时，LMA 位才被置位（激活 IA-32e 模式）。当“IA-32e mode guest”与“unrestricted guest”同时为 1 时，guest-state 区域的字段一方面需要满足 unrestricted guest 条件，另一方面还要满足 IA-32e 模式对运行环境的设置要求。

实际上，在“IA-32e mode guest”为 1 时，guest CR0 字段的 PG 与 PE 必须为 1，IA32_EFER 字段的 LMA 与 LME 位也必须为 1。在 VM-entry 完成后，IA32_EFER.LMA 被更新为 CR0.PG & IA32_EFER.LME。

entry to SMM

当“entry to SMM”为 1 时，表明 VM-entry 后处理器将进入 SMM 模式。但是，只有处理器当前处于 SMM 模式，才能将“entry to SMM”置位发起 VM-entry 进入 SMM 模式。也就是需要从一个 SMM 切换到另一个 SMM 模式。如果处理器在非 SMM 模式下，将“entry to SMM”置位，然后发起 VM-entry，将会产生 VMfailValid 失败。

什么情况下会产生这种“从 SMM 模式进入另一个 SMM”呢？只有一种可能：在 SMM 双重监控处理机制下，当切换到 STM（SMM-Transfer Monitor）代码执行时，处理器是处于 SMM 模式。在 STM 代码中又使用 VMRESUME 指令发起一次 VM-entry 操作。这时“entry to SMM”可以被置为 1 值，这将退出 STM 而再一次进入目标 VMCS 的

SMM 模式执行。

实际上，这个“控制位置位发起 VM-entry 进入 SMM 模式”的作用并不重要，也没什么意义。只有当“entry to SMM”为 0 时，才有这个控制位存在的意义。

当有一个 SMI 请求发生时，处理器切换到 STM 执行，这表明处理器从 executive monitor 切换到 STM，即从非 SMM 模式进入 SMM 模式执行。STM 中最终都需要使用 VMRESUME 指令从 STM 切换回 executive monitor，即退出 SMM 模式，返回到原来的 root 或者 non-root 环境里。这种行为被称为“**VM-entry that return from SMM**”（参见 2.4.3 节及 2.6.6.5 节）。

这时，“entry to SMM”设为 0 值，表明需要返回到非 SMM 模式，从而 VMM 履行了一次对 SMI 的监控职能。也只有在“entry to SMM”为 0 的情况下，才能够返回 executive monitor。

deactivate dual-monitor treatment

在“deactivate dual-monitor treatment”为 1 时，表明处理器需要关闭 SMM 双重监控处理机制。此时，“entry to SMM”必须为 0 值。处理器是在 STM 内执行 VMRESUME 指令返回到 executive monitor，从而关闭 SMM 双重监控处理机制。

在非 SMM 模式下发起 VM-entry 操作，“deactivate dual-monitor treatment”必须为 0 值。这个位能被置位，必须是在 SMM 模式下发起 VM-entry（即实施“VM-entry that return from SMM”）。

load IA32_PERF_GLOBAL_CTRL

当“load IA32_PERF_GLOBAL_CTRL”为 1 时，表明在 VM-entry 时，需要从 guest-state 区域的 IA32_PERF_GLOBAL_CTRL 字段中读取值加载到 IA32_PERF_GLOBAL_CTRL 寄存器中。

load IA32_PAT

当“load IA32_PAT”为 1 时，表明在 VM-entry 时，需要从 guest-state 区域的 IA32_PAT 字段中读取值加载到 IA32_PAT 寄存器中。

load IA32_EFER

当“load IA32_EFER”为 1 时，表明在 VM-entry 时，需要将 guest-state 区域的 IA32_EFER 字段值加载到 IA32_EFER 寄存器中。Intel 推荐 64 位的 VMM 每次 VM-entry 时，“load IA32_EFER”设为 1 值，也就是每次都加载 IA32_EFER 寄存器。

当“load IA32_EFER”为 0 时，在 VM-entry 时，处理器会自动将“IA-32e mode guest”位的值写入 IA32_EFER.LMA 位。而 IA32_EFER.LME 位在 guest-state 区域 CR0 字段的 PG 位为 1 时，也被写入“IA-32e mode guest”的值（参见 4.5.6 节及 4.7.3 节）。

3.6.2 VM-entry MSR-load 字段

有两个字段用来控制 VM-entry 时 guest-MSR 列表的加载，它们是：

(1) VM-entry MSR-load count 字段，这个字段 32 位宽，提供需要加载 MSR 的数量值。

(2) VM-entry MSR-load address 字段，这个字段 64 位宽，提供 MSR 列表的物理地址。

当 VM-entry MSR-load count 字段为 0 值时，没有任何 MSR 需要加载。Intel 推荐这个 count 不要超过 512 值，count 推荐的最大值可以从 IA32_VMX_MISC 寄存器的 bits 27:25 里得到（见 2.5.11 节）。如果 count 值超过了推荐的最大值（当前为 512），可能会产生一些不可预测的错误。同时也需要保证最后一个 MSR 数据的上边界不能超过 MAXPHYADDR 规定的最大物理地址值。

VM-entry MSR-load address 字段提供 MSR 列表的物理地址，MSR 列表的每个表项为 16 字节。

MSR 列表的结构如图 3-6 所示，每个表项的 bits 31:0 是 MSR 的 index 值，提供 MSR 对应的地址值，低部分 MSR 从 00000000H 到 00001FFFH，高部分 MSR 从 C0000000H 到 C0001FFFH。bits 63:32 位是保留位，必须为 0 值。bits 127:64 存放需要加载的 MSR index 对应的 MSR 数据。

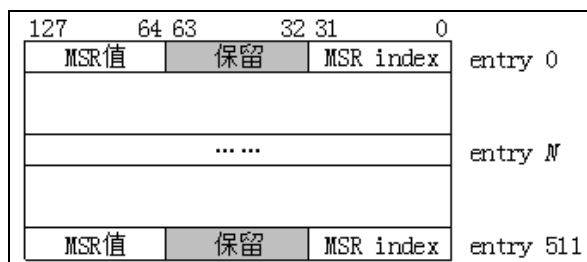


图 3-6

在 VM-entry 时，处理器根据表项的 MSR index 加载相应的 MSR 数据，这个过程相当于逐个使用 WRMSR 指令来写 MSR。写 MSR 的数量由 count 值决定。

在一般的应用中，我们可以设 VM-entry MSR-load 地址应该等于 VM-exit MSR-store 地址，这样保证它们在 VM-entry 与 VM-exit 时使用同一份 MSR 列表。

3.6.3 事件注入控制字段

事件注入是虚拟化平台中最关键的特性之一，是实现虚拟化管理的重要手段。允许在 VM-entry 完成后，执行任何 guest 指令前，处理器执行由 VMM 设置的注入事件。这个事件可以是一个中断或异常，甚至 pending MTF VM-exit 事件，它们被称为“**向量化事件**”，而含有注入事件的 VM-entry 被称为“**向量化的 VM-entry**”。

事件注入机制由如下三个字段来实现：

- (1) VM-entry interruption information 字段。
- (2) VM-entry exception error code 字段。
- (3) VM-entry instruction length 字段。

VMM 通过设置这三个字段来配置向量化事件。如果一个 VM-exit 是由于向量化事件而引起的，那么 VM-exit interruption information 和 VM-exit interruption error code 字段会记录这个向量化事件的信息。如果 INT3 或 INTO 指令产生的软件异常引发 VM-exit，或者由软件异常、软件中断、特权级软件中断 delivery 期间出错而引发 VM-exit，VM-exit instruction length 字段会记录引发 VM-exit 指令的长度。

VMM 可以直接复制 VM-exit interruption information，VM-exit interruption error code 及 VM-exit instruction length 字段的值到 VM-entry interruption information，VM-entry exception error code 及 VM-entry instruction length 字段来完成设置。VMM 也可以主动设置这几个字段来注入一个向量化事件给 guest 执行。

3.6.3.1 VM-entry interruption information 字段

这个字段是 32 位宽，用来设置注入事件的明细信息。包括：中断或异常的向量号、事件类型、错误码的 delivery 标志位及有效标志位。它的结构如图 3-7 所示。

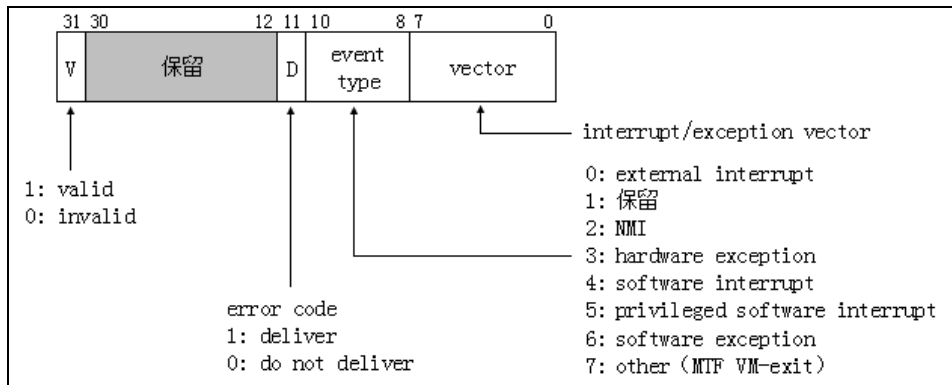


图 3-7

bits 7:0 设置中断或异常的向量号，当事件类型是 NMI 时，向量号必须为 2。事件类型为 other 时，向量号必须为 0 值。

bit 11 为 1 时，指示有错误码需要提交。在注入事件 delivery 时，错误码会被压入栈中。这个位只有在注入**硬件异常**事件时才能被置 1，否则会产生 VMfailValid 失败。能产生错误码的硬件异常是：#DF，#TS，#NP，#SS，#GP，#PF 及 #AC 这 7 类。注入其余的硬件异常不能将此位置 1。

bit 31 是有效位，为 1 时指示 VM-entry interruption information 字段有效，为 0 时无效。bits 10:8 设置事件类型，包括 7 个事件类型，如表 3-10 所示。

表 3-10

bits 10:8	类 型	描 述
0	external interrupt	外部硬件中断
1	保留	---
2	NMI	不可屏蔽的外部中断
3	hardware exception	指 fault 或 abort 事件，除#BP 及#OF 异常以外的所有异常，包括 BOUND 与 UD2 指令产生的异常
4	software interrupt	由 INT 指令产生的中断
5	privileged software exception	注入的事件是特权级软件中断，这个中断类型在评估 delivery 时不会检查权限
6	software exception	指由 INT3 或 INTO 指令产生的#BP 与#OF 异常，它们属于 trap 事件。
7	other event	注入 MTF VM-exit 事件

表 3-10 列出了可以注入的 7 种事件类型，对应 VM-entry interruption information 字段的 bits 10:8 位，类型值 1 保留未用。

外部中断（类型 0）

外部中断不能使用 0~15 号向量，也不应该使用 16~31 号向量，应使用 32~255 号向量。这个事件类型被期望用来注入一个中断控制器（local APIC，I/O APIC 或者 8259A）获得的中断请求。

NMI（类型 2）

NMI 必须使用 2 号向量。当 NMI 成功 deliver 执行后，处理器的“blocking by NMI”状态有效，表示存在 NMI 阻塞状态，直至 IRET 指令执行后解除这个阻塞（参见 3.8.6 节）。

当“virtual-NMIs”为 1 时，表示注入的是一个 virtual-NMI 事件，virtual-NMI delivery 后，就存在 virtual-NMI 阻塞状态，直至 IRET 指令执行后解除 virtual-NMI 阻塞。

当注入一个 virtual-NMI，而“NMI-window exiting”为 1，并且“blocking by NMI”为 0 时，则在 VM-entry 完成后直接产生 VM-exit。

硬件异常（类型 3）

硬件异常是指除了#BP 与#OF 异常以外的所有异常，它们属于 fault 或 abort 类型的异常。也包括由 BOUND 指令产生的#BR 异常及由 UD 指令产生的#UD 异常。所有类型的#DB 异常（#DB 异常也包括了 trap 类型）也属于硬件异常。硬件异常的向量号必须是 0~31。

尽管在 64 位模式下 BOUND 指令是无效的，但允许注入#BR 异常。

软件异常（类型 6）

软件异常指由 INT3 与 INTO 指令产生的#BP 与#OF 异常，对应的向量号必须为 3 与

4. 它们属于 trap 类型。尽管在 64 位模式下 INTO 指令是无效的，但允许注入 #OF 异常。

软件中断（类型 4）

软件中断指由 INT 指令执行的中断。软件中断在 delivery 期间需要进行权限的检查。对于一个 non-conforming 代码段， $CPL \leq \text{Gate 的 DPL 值}$ ，并且 $CPL \geq \text{Code 段的 DPL 值}$ 。如果在 delivery 期间发生权限不符将会产生 #GP 异常。如果 exception bitmap 字段的 bit 13 为 1，#GP 异常会导致 VM-exit。此时将在 VM-exit interruption information 字段中记录中断的向量号和事件类型等。在 VM-exit interruption error code 字段记录 #GP 异常错误码。

注入由 INT3 或 INTO 指令产生的软件异常和软件中断一样，在 delivery 期间同样需要进行权限的检查，处理手法与软件中断一样。

特权级软件中断（类型 5）

这类事件只能在注入时使用，不可能在执行中遇到。也就是说：在执行过程中不可能产生这类中断，只能通过注入事件的方式产生。根据笔者的测试，除了无须进行权限检查外，其余的处理手法与软件中断是一致的。由此推断出这类事件的特权由来。

其他事件（类型 7）

这类事件目前使用在 MTF (Monitor Trap Flag) 功能上。当事件类型为 7 时，向量号必须为 0。否则在 VM-entry 时会产生 VMfailValid 失败。注入这个类型事件时，将 pending 一个 MTF VM-exit 事件。当 VM-entry 完成后会产生 **MTF VM-exit**（参见 4.15.3 节）。

注意：这个 MTF VM-exit 事件不受“monitor trap flag”控制位的影响（另见 3.5.2.1 节），但必须在支持“monitor trap flag”功能时才能注入 MTF VM-exit。

3.6.3.2 VM-entry exception error code 字段

这个 32 位的字段只有在 VM-entry interruption information 字段的 bit 31 及 bit 11 为 1 时才有效，提供一个异常错误码在注入事件 delivery 期间压入栈中。

只有 #DF, #TS, #NP, #SS, #GP, #PF 及 #AC 异常才会产生错误码。因此，注入这几个硬件异常时需要提供错误码。对于其他硬件异常或者其他类型的事件，这个字段会被忽略。

3.6.3.3 VM-entry instruction length 字段

这个字段是 32 位宽。在事件注入中，它被期望 VM-exit 由下面的向量化事件引起时使用：

(1) 由于执行 INT3 或 INTO 指令产生的软件异常而引起 VM-exit。

(2) 在软件异常、软件中断或者特权级软件中断 delivery 期间间接引发 VM-exit。例如，delivery 期间产生了异常而导致 VM-exit 发生，或者由于 IDT 描述符使用了 task-gate 尝试任务切换而导致 VM-exit 发生。

如果由这些向量化事件引起 VM-exit，在 VM-exit instruction length 字段中会记录这些指令的长度。

当注入事件属于**软件异常**（类型 6）、**软件中断**（类型 4）以及**特权级软件中断**（类型 5）时，必须在这个字段中提供指令长度。指令长度在 1~15 之间，为 0 值会产生 VMfailValid 失败。

3.7 VM-exit 控制类字段

VM-exit 区域的控制字段相对较少，包括下面这些：

- (1) VM-exit control 字段。
- (2) VM-exit MSR-store count 与 VM-exit MSR-store address 字段。
- (3) VM-exit MSR-load count 与 VM-exit MSR-load address 字段。

这些字段用来控制发生 VM-exit 时的处理器行为，决定如何进行 VM-exit 操作。在 VM-entry 时，处理器会检查这些字段。如果检查不通过，产生 VMfailValid 失败，并且在 VM-instruction error 字段中保存错误码，然后接着执行 VMLAUNCH 或 VMRESUME 指令下面的指令。

3.7.1 VM-exit control 字段

VM-exit control 字段是 32 位宽，每个位对应一个控制功能，如表 3-11 所示。控制处理器在 VM-exit 时加载或保存某些寄存器值，或者返回 IA-32e mode 的 host 环境。

表 3-11

位域	控 制 名	配 置	描 述
1:0	保留位	1	固定为 1 值
2	save debug controls	0 或 1	为 1 时，保存 debug 寄存器
8:3	保留位	1	固定为 1 值
9	host address-space size	0 或 1	为 1 时，返回到 IA-32e 模式
11:10	保留位	1	固定为 1 值
12	load IA32_PERF_GLOBAL_CTRL	0 或 1	为 1 时，加载 IA32_PERF_GLOBAL_CTRL
14:13	保留位	1	固定为 1 值
15	acknowledge interrupt on exit	0 或 1	为 1 时，VM-exit 时处理器响应中断控制器，读取中断向量号
17:16	保留位	1	固定为 1 值
18	save IA32_PAT	0 或 1	为 1 时，保存 IA32_PAT
19	load IA32_PAT	0 或 1	为 1 时，加载 IA32_PAT
20	save IA32_EFER	0 或 1	为 1 时，保存 IA32_EFER

续表

位域	控制名	配置	描述
21	load IA32_EFER	0 或 1	为 1 时, 加载 IA32_EFER
22	save VMX-preemption timer value	0 或 1	为 1 时, VM-exit 时保存 VMX 定时器计数值
31:23	保留位	0	固定为 0 值

设置这个字段时, 软件应该查询 IA32_VMX_EXIT_CTLs 或 IA32_VMX_TRUE_EXIT_CTLs 寄存器来获得控制位支持度及保留位的默认值 (见 2.5.7 节)。

一般地, bits 1:0, bits 8:3, bits 11:10, bits 14:13 以及 bits 17:16 是 default1 位, 固定为 1 值。bits 31:23 为 default0, 固定为 0 值。其余位可正常设置 0 或 1 值。

save debug controls

当 “save debug controls” 为 1 值时, 在 VM-exit 时, 处理器在 guest-state 区域的相应字段中保存 DR7 与 IA32_DEBUGCTL 寄存器的值。

host address-space size

当 “host address-space size” 为 1 时, 将返回到 IA-32e 模式的 host 中。在发起 VM-entry 时, 当前的 IA32_EFER.LMA 为 1 时 (IA-32e 模式的 host), “host address-space size” 位必须设为 1 值。当 IA32_EFER.LMA 为 0 时 (legacy 模式的 host), “host address-space size” 必须为 0 值, 返回到 legacy 模式的 host 里。

在 64 位 VMM 管理下, guest 可以是 IA-32e 模式或 legacy 模式, 但 VM-exit 必须返回到 IA-32e 模式的 host 中。同样, 32 位的 VMM 下, VM-exit 必须返回到 legacy 模式。

在进行 VM-exit 操作时, “host address-space size” 的值也被写入 CS.L, IA32_EFER.LME 及 IA32_EFER.LMA 位。

load IA32_PERF_GLOBAL_CTRL

当 “load IA32_PERF_GLOBAL_CTRL” 为 1 值, 在 VM-exit 时, 处理器将在 host-state 区域的相应字段中加载 IA32_PERF_GLOBAL_CTRL 寄存器的值。

acknowledge interrupt on exit

当由于外部中断而产生 VM-exit 时, 设置 “acknowledge interrupt on exit” 为 1, 处理器将响应中断控制器的中断请求, 清外部中断对应的 **Request** 位。从中断控制器里获得中断向量号等信息, 保存在 VM-exit interruption information 字段里。

在下次重新 VM-entry 时, VMM 根据这个字段记录的信息来注入一个外部中断事件给 guest 执行中断服务例程。实际中, VMM 可以直接将 VM-exit interruption information 字段的值复制到 VM-entry interruption information 字段, 完成事件注入的设置。

当 “acknowledge interrupt on exit” 为 0 时, 处理器不响应中断控制器, 外部中断对应的 **Request** 位保持有效, 外部中断被悬挂着。返回到 host 后, VMM 可以使用 STI 指令重

新打开 interrupt window。此时，处理器将响应中断控制器并读取向量号。中断服务例程将通过 host-IDT deliver 执行。通过这个手段，VMM 可以夺取物理平台的外部中断控制权。

save IA32_PAT

当“save IA32_PAT”为 1 值时，在 VM-exit 时，处理器将在 guest-state 区域的相应字段中保存 IA32_PAT 寄存器值。

load IA32_PAT

当“load IA32_PAT”为 1 值时，在 VM-exit 时，处理器将在 host-state 区域的相应字段中加载 IA32_PAT 寄存器值。

save IA32_EFER

当“save IA32_EFER”为 1 值时，在 VM-exit 时，处理器将在 guest-state 区域的 IA32_EFER 字段中保存 IA32_EFER 寄存器。Intel 推荐每次 VM-exit 时都保存 IA32_EFER 值。

load IA32_EFER

当“load IA32_EFER”为 1 值时，在 VM-exit 时，处理器将在 host-state 区域的 IA32_EFER 字段中加载 IA32_EFER 寄存器。这个 host IA32_EFER 字段的 LME 与 LMA 位必须等于“host address-space size”值，在进行 VM-entry 时，处理器会检查 host-state 区域的 IA32_EFER 字段（参见 4.4.4.5 节）。

当“load IA32_EFER”为 0 时，处理器将“host address-space size”的值写入 IA32_EFER.LME 与 IA32_EFER.LMA 位。

save VMX-preemption timer value

当“save VMX-preemption timer value”为 1 时，在 VM-exit 时处理器将在 guest-state 区域的相应字段中保存这个 VMX 定时器计数值。假如 VM-exit 是由于定时器超时产生的，那么这个 timer value 为 0 值。

3.7.2 VM-exit MSR-store 与 MSR-load 字段

MSR-store 字段控制处理器 VM-exit 时如何保存 guest-MSR 列表，由如下两个字段组成：

- (1) VM-exit MSR-store count 字段。
- (2) VM-exit MSR-store address 字段。

MSR-load 字段控制处理器 VM-exit 时如何加载 host-MSR 列表，同样由如下两个字段组成：

- (1) VM-exit MSR-load count 字段。

(2) VM-exit MSR-load address 字段。

这 4 个字段的用法和 VM-entry MSR-load 字段是一样的（参见 3.6.2 节描述）。VM-exit MSR-store address 字段的值与 VM-entry MSR-load address 字段的值都是 guest-MSR 列表的地址，一般可以将它们设为相等，这样保证在进入和退出 VM 时读取同一份 guest-MSR 列表。

3.8 guest-state 区域字段

VMCS 的 guest-state 区域用来保存与 guest 运行环境相关的信息。包括两大类字段：寄存器类字段和非寄存器类字段。

(1) 寄存器类字段如表 3-12 所示。

表 3-12

类 型	字 段	宽 度	字段 ID
控制寄存器	CR0	natural-width	00006800H
	CR3		00006802H
	CR4		00006804H
调试寄存器	DR7		0000681AH
栈指针	RSP		0000681CH
指令指针	RIP		0000681EH
标志寄存器	RFLAGS		00006820H
ES	selector	16 位	00000800H
ES	base	natural-width	00006806H
	limit	32 位	00004800H
	access right		00004814H
CS	selector	16 位	00000802H
	base	natural-width	00006808H
CS	limit	32 位	00004802H
	access right		00004816H
SS	selector	16 位	00000804H
	base	natural-width	0000680AH
	limit	32 位	00004804H
	access right		00004818H
DS	selector	16 位	00000806H
	base	natural-width	0000680CH
	limit	32 位	00004806H
	access right		0000481AH

续表

类 型	字 段	宽 度	字段 ID
FS	selector	16 位	00000808H
	base	natural-width	0000680EH
	limit	32 位	00004808H
	access right		0000481CH
GS	selector	16 位	0000080AH
	base	natural-width	00006810H
	limit	32 位	0000480AH
	access right		0000481EH
LDTR	selector	16 位	0000080CH
	base	natural-width	00006812H
	limit	32 位	0000480CH
	access right		00004820H
TR	selector	16 位	0000080EH
	base	natural-width	00006814H
	limit	32 位	0000480EH
	access right		00004822H
GDTR	base	natural-width	00006816H
	limit	32 位	00004810H
IDTR	base	natural-width	00006818H
	limit	32 位	00004812H
MSR	IA32_DEBUGCTL	64 位	00002802H (full) 00002803H (high)
	IA32_SYSENTER_CS	32 位	0000482AH
	IA32_SYSENTER_ESP	natural-width	00006824H
	IA32_SYSENTER_EIP		00006826H
MSR	IA32_PERF_GLOBAL_CTRL	64 位	00002808H (full) 00002809H (high)
	IA32_PAT		00002804H (full) 00002805H (high)
	IA32_EFER		00002806H (full) 00002807H (high)
内部寄存器	SMBASE	32 位	00004828H

(2) 非寄存器类字段如表 3-13 所示。

表 3-13

字 段	宽 度	字段 ID
activity state	32 位	00004826H
interrupt state	32 位	00004824H
pending debug exceptions	natural-width	00006822H
VMCS link pointer	64 位	00002800H (full) 00002801H (high)
VMX-preemption timer value	32 位	0000482EH
PDPTE0	64 位	0000280AH (full) 0000280BH (high)
PDPTE1		0000280CH (full) 0000280DH (high)
PDPTE2		0000280EH (full) 0000280FH (high)
PDPTE3		00002810H (full) 00002811H (high)
guest interrupt status	16 位	00000810H

在 VM-entry 时，处理器会结合 VM-execution 控制字段、VM-entry 控制字段及 VM-exit 控制字段来检查以上 guest-state 区域的字段。当检查到不符合 guest 运行环境要求的设置时，会产生 VM-exit 行为，并在 VM-exit 信息区域的 exit reason 字段中记录退出原因编号为 33，指示“由于无效的 guest-state 而导致 VM-entry 失败”。检查通过后，处理器从 guest-state 区域中加载这些 context 信息。

control registers 字段

guest-state 区域有 3 个控制寄存器字段（CR0，CR3 以及 CR4）。它们是 natural-width 字段，在 64 位架构处理器上是 64 位，否则为 32 位。这些字段用来配置虚拟处理器的工作模式。CR0 与 CR4 寄存器需要满足 VMX operation 模式规定的运行条件。即 CR0.PE，CR0.NE 及 CR0.PG 必须为 1，CR4.VMXE 也必须为 1。

如果 VMX 支持 unrestricted guest 功能，并且 secondary processor-based VM-execution control 字段的“unrestricted guest”位为 1，允许 CR0.PE 与 CR0.PG 为 0 值。但是，如果 CR0.PG 为 1，CR0.PE 必须也为 1 值。

软件需要使用 2.5.10 节中描述的方法，通过 IA32_VMX_CR0_FIXED0 与 IA32_VMX_CR0_FIXED1 寄存器来检查 CR0 寄存器哪些位需要被置位，哪些位需要被清位。同样，也需要通过 IA32_VMX_CR4_FIXED0 与 IA32_VMX_CR4_FIXED1 寄存器来检查 CR4 寄存器哪些位需要被置位，哪些位需要被清位。

软件也需要通过检查 IA32_VMX_PROCBASED_CTLX2 或 IA32_VMX_TRUE_PROCBASE_CTLX2 寄存器（见 2.5.6.3 节）来确定是否支持 unrestricted guest 功能。

guest 入口信息

RSP, RIP 及 RFLAGS 字段设置 guest 的执行上下文环境。RIP 提供了 guest 代码的入口点, RSP 提供 guest 的栈指针, 而 RFLAGS 是标志位状态。它们都是 natural-width 类型字段, 在 64 位架构处理器上是 64 位宽, 否则为 32 位宽。在 64 位 guest 下, RIP 字段的地址值必须是 canonical 类型。

guest 环境并不需要提供通用寄存器组和 x87 FPU/XMM 寄存器组, host 的这些寄存器组的 context 环境会顺延至 guest 中。因此, VMM 也可以选择在 VMLAUNCH 与 VMRESUME 指令执行前, 对这些寄存器组进行清空或者设置相应的值来适合 guest 环境。

3.8.1 段寄存器字段

guest-state 区域涵盖了全部 6 个数据/代码段寄存器字段, 分别是 ES, CS, SS, DS, FS 及 GS 寄存器; 2 个系统段寄存器字段, 分别是 LDTR 与 TR 寄存器。

段寄存器由 **selector** (可视部分) 及 **cache** (不可视部分) 组成。内部的 cache 部分包括 base, limit 及 access right (又称 attribute) 三个子域 (关于段寄存器结构可参见《x86/x64 体系探索及编程》10.5.3 节描述)。

因此, 在 guest-state 区域中, 每个段寄存器有 4 个字段对应, 分别描述符段寄存器的各个域。

- (1) selector, 16 位字段。
- (2) base, natural-width 类型字段。在 64 位架构处理器上是 64 位, 否则为 32 位宽。
- (3) limit, 32 位字段。
- (4) access right, 32 位字段。

在 x64 体系中, ES.base, CS.base, SS.base 以及 DS.base 在 64 位模式下是无效的。并且它们仅支持 32 位的地址值。但在 VMCS 的 guest-state 区域 base 字段中, 由于是 natural-width 类型字段, 它们在 64 位处理器上是 64 位的。因此, ES.base, CS.base, SS.base 以及 DS.base 字段的 **bits 63:32** 必须为 0 值。否则, 在发起 VM-entry 时会引发“由于 guest-state 无效导致 VM-entry 失败”, 产生 VM-exit 行为。

3.8.1.1 access right 字段

在 guest-state 区域中, access right 字段为 32 位宽。在 x86/x64 平台中, 段寄存器内部 cache 的 attribute 域宽度是未知的 (或为 16 位或为 32 位)。

表 3-14 列出了 access right 字段的结构。除了新增了一个“**unusable**”位外, access right 字段与段寄存器 cache 内的属性部分是一一对应的, 在 VM-entry 时, access right 会被加载到段寄存器 cache 中。

表 3-14

位 域	属 性	描 述
3:0	type	段类型值
4	S	0=system, 1=code/data
6:5	DPL	段的访问权限
7	P	0=no present, 1=present
11:8	保留	---
12	AVL	系统软件可用
13	L	在 IA-32e 模式下为 L 标志, 在 legacy 下为保留位
14	D/B	默认操作数 size, 0=16 位, 1=32 位
15	G	段 limit 粒度, 0=1byte, 1=4KB
16	unusable	0=usable, 1=unusable
31:17	保留	---

type 位域

段类型有两大类, 分别为 system 段与 code/data 段, 由 access right 的 S 位来设置 (bit 4)。

- system 段: bit 4 为 0 时, 属于系统段。包括 LDT 与 TSS 段, 分别由 LDTR 与 TR 寄存器来引用。
- code/data 段: bit 4 为 1 时, 属于代码或数据段。包括代码段和所有数据段, 分别由 CS 与 ES, SS, DS, FS, GS 寄存器来引用。

access right 字段的 type 位域 (bits 3:0) 进一步设置段的类型。对于系统段来说, type 的值必须是表 3-15 所列出的类型值。

表 3-15

段寄存器	模 式	S	type	描 述
TR	legacy	0	0011B	16-bit busy TSS
			1011B	32-bit busy TSS
	IA-32e	0	1011B	64-bit busy TSS
LDTR	所有模式	0	0010B	LDT

TR 寄存器 access right 字段的 type 位域在 legacy 模式下必须为 **3 或 11** 值, 在 IA-32e 模式下必须为 **11** 值。LDTR 寄存器 access right 字段的 type 位域在所有模式下都必须为 **2** 值。

表 3-16 列出了 code/data 段寄存器 access right 字段的 type 位域支持的所有值, 这些段必须是已访问的。

表 3-16

段寄存器	模 式	S	type	描 述
CS	实模式, virtual-8086	1	0011B	可写/已访问, expand-up 数据段
	保护模式, IA-32e, 实模式	1	1001B	不可读/已访问, non-conforming 代码段
			1011B	可读/已访问, non-conforming 代码段
			1101B	不可读/已访问, conforming 代码段
			1111B	可读/已访问, conforming 代码段
SS	实模式, virtual-8086	1	0011B	可写/已访问, expand-up 数据段
	保护模, IA-32e, 实模式	1	0011B	可写/已访问, expand-up 数据段
		1	0111B	可写/已访问, expand-down 数据段
ES/DS/FS/GS	实模式, virtual-8086	1	0011B	可写/已访问, expand-up 数据段
	保护模式, IA-32e, 实模式	1	0xx1B	已访问, 数据段
		1	1x11B	可读/已访问, 代码段

- CS 寄存器

- 在 virtual-8086 模式下: type 值必须为 3, 指示“可写并且已访问的 expand-up 数据段”。
- 在实模式、保护模式或者 IA-32e 模式下: type 值可以为 9, 11, 13 或者 15。指示属于已访问的, 可读或不可读, conforming 或 non-conforming 类型代码段。在实模式下 type 值还可以为 3。

- SS 寄存器

- 在 virtual-8086 模式下: type 值必须为 3。
- 在实模式、保护模式或者 IA-32e 模式下: type 值可以为 3 或者 7, 表示可写并且已访问的 expand-up 或 expand-down 数据段。

- ES, DS, FS 以及 GS 寄存器

- 在 virtual-8086 模式下: type 值必须为 3。
- 在实模式, 保护模式或者 IA-32e 模式下: type 值为 **0xx1B** (x 为 0 或 1), 表示已访问的数据段, 或者为 **1x11B** (11 或 15), 表示可读已访问的代码段。

guest 要进入实模式运行, 必须是处理器支持 unrestricted guest 功能, 并且在 VM-entry 时设置 secondary processor-based VM-execution control 字段的“unrestricted guest”位为 1。而 virtual-8086 模式是在保护模式基础上模拟 8086 行为。

DPL 位域

access right 的 DPL 位域用来设置段的访问权限, 段在不同的 type 值下对应不同的 DPL 设置。表 3-17 描述了 VM-entry 时需要满足的 DPL 设置要求。

表 3-17

段寄存器	模 式	unrestricted guest	type	DPL	描 述
CS	virtual-8086	0/1	0011B	3	在 virtual-8086 模式下, DPL 必须为 3
	实模式	1	0011B	0	实模式下, DPL 必须为 0
	保护模式 IA-32e 模式	0/1	1001B	等于 SS.DPL 与 CS.RPL	属于 non-conforming 段, CS.DPL 必须等于 CS.RPL, 必须等于 SS.DPL 和 SS.RPL
		0/1	1011B		
		0/1	1101B	小于等于 SS.DPL 与 CS.RPL	属于 conforming 段, CS.DPL 必须小于等于 CS.RPL, 并且小于等于 SS.DPL 和 SS.RPL
		0/1	1111B		
SS	virtual-8086	0/1	0011B	3	在 virtual-8086 模式下, DPL 必须为 3
	实模式	1	0011B	0	实模式下, DPL 必须为 0
	保护模式	0	0011B	等于 SS.RPL 与 CS.RPL	必须等于 SS.RPL, 并且等于 CS.RPL
	IA-32e 模式	0	0111B		
ES, DS, FS 以及 GS	virtual-8086	0/1	0011B	3	在 virtual-8086 模式下, DPL 必须为 3
	实模式	1	0011B	忽略	实模式下, 其他数据段 DPL 忽略
	保护模式 IA-32e 模式	0	0xx1B	大于等于 RPL	属于数据段或者 non-conforming 段时, DPL 大于等于 RPL
		0	1011B		
		0	1111B	忽略	属于 conforming 段时, 忽略不检查

表 3-17 列出了各种模式下 VM-entry 时的检查需求。留意观察“unrestricted guest”位与 type 值的影响, 表中所列的保护模式对应的“unrestricted guest”位为 0 值, 当 guest 是保护模式并且“unrestricted guest”为 1 时(此时不能是 IA-32e 模式), 处理器的检查会有一些区别。对 DPL 的检查, 也由于段的类型不同而有区别。

在 x86/x64 体系中, SS.DPL 无论什么时候都代表着处理器的 CPL (当前权限级别)。关于 CS.RPL, CS.DPL, SS.RPL 及 SS.DPL, 有下面的描述:

(1) 在保护模式下, SS.DPL 与 SS.RPL 必须相等。CS.RPL, SS.RPL 及 SS.DPL 都表示处理器的 CPL (当前权限级别)。在使用 **non-conforming** 段时, CS.DPL 必须等于 CS.RPL。并且 CS.DPL 必须等于 SS.DPL 值, 进一步得出 CS.RPL 也必须等于 SS.RPL 值。

(2) 在使用 **conforming** 段的情况下, **CS.DPL 并不一定代表 CPL 值**。CS.DPL 需要小于或等于 CS.RPL 值, 允许在低权限级别执行高权限的目标代码段。

(3) 在实模式下, 不使用 CS.RPL 与 CS.RPL, 它们用来计算段基址。而 SS.DPL 必须为 0 值, 此时它表示处理器的 CPL 为 0。CS.DPL 的值同样由于 non-conforming 或 conforming 类型而要求不同, 使用 non-conforming 段时, CS.DPL 必须为 0 值。使用 conforming 段时, CS.DPL 小于等于 SS.DPL。

在 VMX 架构中, 进行 VM-entry 时对段寄存器的加载行为, 并不能完全等效于 x86/x64 体系保护模式下对段寄存器的加载。有下面的描述:

(1) 当 “unrestricted guest” 为 1 时, 在 VM-entry 时处理器**并不检查 SS.DPL 是否等于 SS.RPL**, 同样不检查 CS.DPL 是否等于 CS.RPL, 也不检查 CS.RPL 是否等于 SS.RPL。

(2) 在 “unrestricted guest” 为 0 的前提下, 处理器会检查 non-conforming 段的 CS.DPL, CS.RPL, SS.RPL 及 SS.DPL 这四者是否相等。

(3) ES, DS, FS 以及 GS 寄存器在 “unrestricted guest” 为 1 时, 也不检查 RPL 与 DPL。在 “unrestricted guest” 为 0 时, DPL 需要大于或等于 RPL 值。但如果使用 conforming 段作为数据段, 这个检查也将忽略。

(4) 在 virtual-8086 模式下, 所有段的 DPL 都必须为 3。

(5) VM-entry 时, 处理器并不检查是否有权限加载 ES, DS, FS 及 GS 寄存器。即使在 CPL=3 (SS.DPL = 3) 时, 也可以加载 DPL = 0 的 ES, DS, FS 及 GS 寄存器。在保护模式下这是不允许的。

VM-entry 时, DPL 位域是处理器对 access right 字段的检查之一。access right 字段通过检查后, 会加载到段寄存器的 cache 中。但并不代表这就是正确的 guest 运行环境设置。

例如, 当 CS.DPL 与 SS.DPL 都为 3 时, DS.DPL 却为 0 值。这符合 VM-entry 时对 DPL 的检查, 它们分别被加载到 CS, SS 及 DS 的 cache 中。

D/B 位与 L 位

D/B 位对 CS 和 SS 才有意义, 使用在 CS 上被作为 D 位, 使用在 SS 上被作为 B 位:

(1) CS.D 指示**默认的操作宽度** (default operation size)。CS.D = 1 时, 指示 32 位 address-size 及 32 位的 operand-size。CS.D = 0 时, 指示 16 位的 address-size 以及 operand-size。

(2) SS.B 指示**栈指针宽度**。SS.B = 1 时是 32 位, 栈指针为 ESP。SS.B = 0 时是 16 位, 栈指针为 SP。

L 位使用在 CS 上, 仅在 IA-32e 模式下有效。CS.L 为 1 时 CS.D 必须为 0, 表示 guest 进入 64-bit 模式。当 CS.L = 0 时, 表示 guest 进入 compatibility 模式, 此时由 CS.D 决定默认的操作宽度。

G 位

G 位指示段限的粒度, G = 1 时粒度为 4KB, G = 0 时粒度为 1B。所以, 在段限大于 0FFFH 时, G 必须为 1。段限在 0 至 0FFFH 之间时, G 必须为 0 值。

在 x64 体系的 64-bit 模式下, 除了 FS 与 GS 段外, 其余段的 limit 都是无效的。但在 VMX 架构下, 即使 guest 是 64-bit 模式, 在 VM-entry 时对 G 位的设置还是必须符合上述的要求。

unusable 位

access right 新增了一个 unusable 位 (bit 16)。为 1 时指示 access right 字段不可用，它的所有信息被忽略。处理器对 access right 字段有效性的检查建立在 unusable 位为 0 的前提下。

TR 段对应的 access right 字段必须是有效的 (**unusable 为 0**)。如果 guest 是 virtual-8086 模式，那么所有的段都必须是有有效的 (**unusable 为 0**)。

3.8.2 GDTR 与 IDTR 字段

这两个描述符寄存器由如下两个字段组成：

- (1) base 字段，提供描述符表基址。
- (2) limit 字段，提供描述符表的长度。

base 字段是 natural-width 类型，在 64 位架构处理器上是 64 位宽，否则为 32 位宽。limit 字段固定为 32 位，但仅低 16 位是有效的。

3.8.3 MSR 字段

guest-state 区域包括如下 7 个 MSR 字段：

- (1) IA32_DEBUGCTL (64 位)，当 “load debug controls” 为 1 时加载。
- (2) IA32_SYSENTER_CS (32 位)。
- (3) IA32_SYSENTER_ESP (natural-width)。
- (4) IA32_SYSENTER_EIP (natural-width)。
- (5) IA32_PERF_GLOBAL_CTRL (64 位)，当 “load IA32_PERF_GLOBAL_CTRL” 为 1 时加载。
- (6) IA32_PAT (64 位)，当 “load IA32_PAT” 为 1 时加载。
- (7) IA32_EFER (64 位)，当 “load IA32_EFER” 为 1 时加载。

IA32_SYSENTER_CS, IA32_SYSENTER_ESP 及 IA32_SYSENTER_EIP 寄存器在 VM-entry 时必须加载。其余的 MSR 是否加载由 VM-entry control 字段的控制位决定 (见 3.6.1 节)。

3.8.4 SMBASE 字段

这个字段使用在 SMM 双重监控处理机制下：

- (1) 当发生 “SMM VM-exit” 行为时，也就是从 executive-monitor 切换到 SMM-

transfer monitor 时，处理器在 SMBASE 字段中保存 SMBASE 寄存器的值。

(2) 当发生“VM-entry that return from SMM”行为时，也就是从 SMM-transfer monitor 切换回 Executive-monitor 时，处理器在 SMBASE 字段加载回原 SMBASE 寄存器的值。

3.8.5 activity state 字段

activity state 字段指示在 VM-entry 和 VM-exit 时，虚拟处理器的当前活动状态。包括 **active**（活动）与 **inactive**（非活动）两大类状态。目前 VMX 架构支持的状态如表 3-18 所示。

表 3-18

状态值	状 态	描 述
0	active	处于正常执行指令状态
1	HLT	指示由 HLT 或 mwait 指令而进入的停机状态
2	shutdown	指示处于 shutdown 状态
3	wait-for-SIPI	指示处于等待 SIPI 消息状态

当状态值为 1、2 或者 3 时，指示处理器处于 **inactive** 状态。guest 在执行过程中由于某个条件或事件而产生 VM-exit 后，VMM 在重新进入 VM 前，需要设置适当的 activity state 值来配置虚拟处理器的当前状态。软件需要查询当前 VMX 架构支持哪几种 inactive 状态，可以从 IA32_VMX_MISC 寄存器的 bits 8:6 中得到（见 2.5.11 节）。

在 guest 运行过程中，处理器的 **inactive** 状态会由于某些事件而被唤醒，从而转入 **active** 状态：

- (1) INIT 信号，外部中断，NMI 以及 SMI 都能唤醒 HLT 状态。
- (2) INIT 信号，NMI 及 SMI 能唤醒 shutdown 状态。
- (3) SIPI 消息能唤醒 wait-for-SIPI 状态。

当接收到的事件直接产生 VM-exit 行为时，处理器没有被真正唤醒。当前的状态会保存在 activity state 字段中，回到 host 后处理器才被唤醒。

如果是接收到 INIT 信号引发 VM-exit 的，VMM 在下次重新进入 VM 时，应清 activity state 字段值，指定虚拟处理器为 active 状态。

如果是接收到 SMI，只有在启用 SMM 双重监控处理机制下，才会产生 SMM VM-exit 而进入 SMM-transfer monitor 执行 SMM 模式代码。从 SMM-transfer monitor 返回到 executive-monitor 后，处理器会被唤醒为 active 状态。

HLT 状态

guest 执行 HLT 指令可以使处理器进入 HLT 状态。而 MWAIT 指令与 HLT 指令类

似，可以使处理器进入一个实现相关的节能优化状态，但并不反映在 activity state 字段上。

当“HLT exiting”为 1 时，guest 软件遇到 HLT 指令而直接产生 VM-exit，虚拟处理器并不在 HLT 状态。此时 activity state 字段中的值并不是 1（非 HLT 状态），原因是 HLT 指令没有得到执行。

在这种情形下，如果需要在下次重新进入 VM 后，将虚拟处理器保持在 HLT 状态。那么可以有下面的处理方式：

(1) 将“HLT exiting”清 0，由于 VM-exit 时 RIP 指向 guest 的 HLT 指令，直接使用 VMRSUME 指令重新进入 VM 后，HLT 指令得到执行，处理器进入 HLT 状态。

(2) 可以将 activity state 字段值设为 1（指示 HLT 状态）。重新进入 VM 后处理器就处于 HLT 状态。但是，guest 的 RIP 指向 HLT 指令是不正确的，VMM 需要将 RIP 指向 HLT 指令的下一条指令。当一个外部中断唤醒处理器后，处理完中断服务例程后继续执行 HLT 指令的下一条指令。

当“HLT exiting”为 0 时，guest 软件执行 HLT 指令而导致处理器进入 HLT 状态。收到 INIT 信号，外部中断，NMI 或者 SMI 可以唤醒处理器。假如这些事件**直接导致** VM-exit 发生（直接 VM-exit 向量事件），在这种情形下，处理器并不会被唤醒，还是处于 HLT 状态。

hlt	; guest 执行 HLT 指令进入 HLT 状态，停止执行指令
... ..	; 假如，此时发生 external-interrupt...

在上面的代码中，虚拟处理器由于执行 HLT 指令而进入 HLT 状态，RIP 指向下一条指令。此时，一个外部中断发生可以将虚拟处理器从 HLT 状态唤醒为 active 状态。

依赖于“external-interrupt exiting”的值，有下面两种情形：

(1) “external-interrupt exiting”为 0 时，处理器进入 active 状态，正常执行 HLT 指令下面的指令。

(2) “external-interrupt exiting”为 1 时，直接引发 VM-exit。此时 activity state 字段为 1，意味着处理器没被唤醒，还是处于 HLT 状态。

对第二种情形（即 VM-exit 时还是处于 HLT 状态），依赖于 VMM 对外部中断虚拟化的设计，VMM 也可以有两种处理逻辑：

(1) 假如 VM-exit control 字段的“acknowledge interrupt on exit”为 1，VMM 不截取这个外部中断的控制权。在下次重新进入 VM 时，VMM 可以直接注入这个外部中断事件反射给 guest 继续执行，activity state 字段的值保持不变（即 HLT 状态）。外部中断通过 guest-IDT 提交处理器执行，处理器从 HLT 状态唤醒为 active 状态。

(2) 假如 VM-exit control 字段的“acknowledge interrupt on exit”为 0，VMM 使用 STI 指令重新打开中断窗口，外部中断通过 host-IDT 提交处理器执行。在下次重新进入 VM 时，VMM 可以选择注入一个虚拟设备中断事件或者不注入事件。在不注入外部中断事件时，VMM 需要清除 activity state 字段的值，指示虚拟处理器为 active 状态，guest 继

续往下执行。

shutdown 状态

典型地，在 x86/x64 体系里，当遇到 **triple fault**（三重故障）异常或者其他某些错误时会使处理器进入 shutdown 状态。在 VMX non-root operation 模式里，triple fault 异常会直接导致 VM-exit 发生。VMM 可以在下一次重新进入 VM 时，将 activity state 字段设为 2 值，指示处理器处于 shutdown 状态。

INIT 信号、NMI 及 SMI 能唤醒 shutdown 状态。但 shutdown 状态会**阻塞外部中断请求及 SIPI 消息**，也能阻塞 MTF VM-exit 事件。当这些事件直接引发 VM-exit 时，处理器并不被唤醒，activity state 字段保存的值为 2。如果是由 NMI 引发 VM-exit 的，在下次进入 VM 时，VMM 注入一个 NMI 事件，处理器将被唤醒为 active 状态。

注意：在 shutdown 状态下，即使在 pin-based VM-execution control 字段的“external-interrupt exiting”为 1 时收到外部中断请求，这个外部中断也会由于被阻塞而不能产生 VM-exit。

wait-for-SIPI 状态

在多处理器环境的初始化阶段，BSP（Bootstrap Processor）需要发送 INIT-SIPI-SIPI 序列来初始化和配置 APs（Application Processors）。处理器收到 INIT 后进行内部初始化，最后会进入 **wait-for-SIPI** 状态。

处理器在 wait-for-SIPI 状态下会阻塞 INIT 信号、NMI、SMI 及外部中断。只有 SIPI 消息才能从 wait-for-SIPI 状态中唤醒处理器。但是，处理器在 active，HLT 或者 shutdown 状态下会阻塞 SIPI 消息。

基于这种原因，BSP 广播两次 SIPI 消息给 APs，目的是避免 system bus 上处理器接收 SIPI 消息有遗漏的情况。假如，一些处理器已经处于 active 状态，再次给它发送的 SIPI 消息会被忽略。而如果另一些处理器有遗漏，可以接收 BSP 再次发送的 SIPI 消息，从而能够保证 system bus 上的所有处理器都必须接收到 SIPI 消息。

然而，在 VMX non-root operation 环境下，处理器在 wait-for-SIPI 状态下接收到 SIPI 消息，会直接产生 VM-exit，只有在 VM-exit 操作完成后处理器才被唤醒为 active 状态。在 VM-exit 进行时，处理器还是处于 wait-for-SIPI 状态，activity state 字段里的值为 3（指示 wait-for-SIPI 状态）。

VMM 需要在下次进入 VM 时，将 activity state 字段的值清 0，指示为 active 状态。那么，进入 VM 后 guest 能正常执行。

3.8.6 interruptibility state 字段

32 位的 interruptibility state 字段指示当前虚拟处理器的**可中断性**（即是否可以响应中断），其中每一个位指示了当前的阻塞状态，如表 3-19 所示。

表 3-19

位 域	中断阻塞	描 述
0	blocking by STI	指示当前有 STI 阻塞状态
1	blocking by MOV-SS	指示当前有 MOV-SS 阻塞状态
2	blocking by SMI	指示当前有 SMI 阻塞状态
3	blocking by NMI	指示当前有 NMI 阻塞状态
31:4	保留	必须为 0 值

在 x86/x64 体系中有表 3-19 所列的 4 种中断阻塞状态。bits 3:0 其中的位为 1 时，表示当前的处理器有相应的阻塞状态。bits 31:4 是保留位，必须为 0 值，否则，在 VM-entry 时会失败。

blocking by STI

这种阻塞状态产生于刚刚打开“**interrupt-window**”时（参见 3.5.2 节）。仅当 eflags.IF = 0 时，执行 STI 指令打开中断许可，那么 STI 指令的下一条指令执行完毕前就存在“blocking by STI”阻塞状态。这个阻塞状态将阻塞外部中断。

如果在执行 STI 指令前 eflags.IF 已经为 1。那么，这条 STI 指令的下一条指令不会存在“blocking by STI”阻塞状态。

```
sti          ; 打开中断（假如之前 IF=0）
mov [eax], ebx ; blocking by STI 阻塞状态，直到指令执行完毕后清除
```

如上面代码所示，假设执行 STI 指令前 eflags.IF 为 0，那么，执行 STI 指令后，下一条 MOV 指令执行完毕前如果发生外部中断将被阻塞（某些处理器也可能会阻塞 NMI）。下一条 MOV 指令执行完毕后这个阻塞状态将被**自动解除**。

如果，执行 STI 指令下的 mov [eax], ebx 指令产生了 #PF 异常，而 exception bitmap 字段的 bit 14 为 1 直接导致 VM-exit 发生。那么，这个“blocking by STI”阻塞状态并不解除。在 VM-exit 时，interruptibility state 字段的值为 1，表示仍存在“blocking by STI”阻塞状态。

Intel 设计“blocking by STI”阻塞状态的初衷是用在下面的场合：

```
sti          ; 打开中断（假如之前 IF=0）
ret          ; 产生“blocking by STI”阻塞状态，避免在返回前产生中断
```

如以上代码所示，当 STI 指令最后是一条 RET 指令时，“blocking by STI”阻塞状态的存在避免了在返回前产生中断。

blocking by MOV-SS

这种阻塞状态和“blocking by STI”类似。使用 MOV 或者 POP 指令更新 SS 寄存器时，下一条指令执行完毕前就产生了“blocking by MOV-SS”阻塞状态。这个阻塞状态将阻塞**外部中断**、**NMI** 及 **#DB** 异常。

MOV 与 POP 指令都可以产生“blocking by MOV-SS”阻塞状态，如下面的代码所示：

```
mov ss, ax          ; 更新 SS 寄存器
mov [eax], ebx      ; blocking by MOV-SS 阻塞状态, 直到指令执行完毕后清除
```

或者

```
pop ss              ; 更新 SS 寄存器
mov [eax], ebx      ; blocking by MOV-SS 阻塞状态, 直到指令执行完毕后清除
```

在 MOV SS/POP SS 指令的下一条指令执行完毕后, “blocking by MOV-SS” 阻塞状态将被**自动解除**, 如果此时存在 #DB 异常的 pending, 这个 #DB 异常会得到 delivery。

Intel 设计 “blocking by MOV-SS” 阻塞状态的初衷是用在下面的场合:

```
mov ss, ax          ; 更新 SS 寄存器
mov esp, kernel_stack ; 更新 esp 栈指针 (0 级指针)
```

在更新 SS 栈段时, 需要保证栈指针 (典型为 0 级栈) 能够得到更新而不被中断打断 (外部中断, NMI 或者 #DB 断点异常), 避免在中断服务例程中使用了不适当的栈指针。

如果软件使用 LSS 指令来更新 SS 寄存器和 ESP 栈指针, 则不存在 “blocking by MOV-SS” 阻塞状态。因此, Intel 推荐使用 LSS 指令代替 MOV 或 POP 来更新 SS 寄存器。

```
mov ss, ax          ; 更新 SS 寄存器
mov ss, ax          ; 存在 “blocking by MOV-SS” 阻塞状态
mov [eax], 0        ; 正常, 没有阻塞状态
```

在上面的代码中, 紧接着两次进行 SS 寄存器更新。只有第一次更新 SS 寄存器时存在 “blocking by MOV-SS” 阻塞状态, 第二次并没有阻塞状态。

注意: 在一个指令流里, 不可能同时出现 “blocking by STI” 与 “blocking by MOV-SS” 两种阻塞状态。因此, 这两个位不能同时为 1 值。

blocking by SMI

SMI 阻塞状态发生在响应 SMI 请求时。处理器切换到 SMM 模式执行后, 新的 SMI 会被阻塞。直到执行 RSM 指令退出 SMM 模式后 “blocking by SMI” 阻塞状态被解除。

由于这个原因, VM-entry control 字段 “enter to SMM” 位为 1 时, 表示 VM-entry 将进入 SMM 模式, 此时 interruptibility state 字段的 “blocking by SMI” 位必须为 1。反之, VM-entry 进入非 SMM 模式时, interruptibility state 字段的 “blocking by SMI” 位必须为 0 值。

blocking by NMI

当处理器响应 NMI 请求时, NMI 服务例程**得到 delivery** 后将阻塞另一个 NMI 请求, 直到执行 IRET 指令后解除阻塞。因此, “blocking by NMI” 阻塞状态的存在依赖于 NMI 是否得到 delivery。

(1) 当 NMI 没有 delivery 时, 处理器 **VM-exit 当前**并不存在 “blocking by NMI” 阻塞状态。注意, 只有在 **VM-exit 完成后 (non-root 切回 root)** 处理器才存在 “blocking by NMI” 阻塞状态。

(2) 当 NMI 得到 delivery 执行时, 即使在 delivery 期间发生某些错误 (包括异常、

任务切换、APIC-access page、EPT violation 或者 EPT misconfiguration) 而产生 VM-exit。在进入 NMI handler 执行, 或者 VM-exit 操作前, 处理器就存在 “blocking by NMI” 阻塞状态。

当 “NMI exiting” 为 1 时, 一个 NMI 请求直接产生 VM-exit, NMI 并没有得到 delivery。处理器在 VM-exit 操作当前不存在 “blocking by NMI” 阻塞状态。此时, interruptibility state 字段的 “blocking by NMI” 位为 0 值。

另外, 当 VMM 注入一个 NMI 事件 (或 virtual-NMI 事件) 时, NMI 通过 guest-IDT 进行 deliver 执行。此时, 处理器也存在 “blocking by NMI” (或 “blocking by virtual-NMI”) 阻塞状态。

当 pin-based VM-execution control 字段的 “NMI exiting” 与 “virtual-NMIs” 位同时为 1 时, 产生 virtual-NMI 概念, “blocking by NMI” 此时被视为 “**blocking by virtual-NMI**” 位。当注入一个 virtual-NMI 事件 deliver 执行后, 处理器就存在 “blocking by virtual-NMI” 阻塞状态。当注入的 virtual-NMI 事件由于 “NMI-window exiting” 位为 1 而产生 VM-exit 时, 并不存在 “blocking by virtual-NMI” 阻塞状态。

如果 NMI (或 virtual-NMI) 在 delivery 期间发生某些错误而导致产生 VM-exit, 在 VM-exit 前处理器就已经存在 “blocking by NMI” (或 “blocking by virtual-NMI”) 阻塞状态。此时 interruptibility state 字段的 “blocking by NMI” 位为 1 值。

在 NMI 或 virtual-NMI 服务例程中, 执行 IRET 指令后, 这个 NMI 或 virtual-NMI 阻塞状态将解除。如果执行 IRET 指令而发生某些错误, 并不影响它解除 NMI 或 virtual-NMI 的阻塞状态。执行 IRET 指令发生一个异常而**直接导致 VM-exit** 时, 在 VM-exit interruption information 字段中的 “**NMI unblocking**” 位 (bit 12) 将为 1 值, 指示 NMI 或 virtual-NMI 阻塞状态已经被解除 (见 3.10.2.1 节)。

3.8.7 pending debug exceptions 字段

pending debug exceptions 字段用来记录和设置 guest 存在未处理而 pending (悬挂) 的 #DB 异常。属于 natural-width 类型字段, 在 64 位架构处理器上是 64 位, 否则为 32 位。

表 3-20 中, 在 pending debug exception 字段的 bits 3:0 中每一位对应相应的断点 (B3:B0)。这些断点由 DR7 寄存器对应的 LE (Local Breakpoint Enable) 或 GE (Global Breakpoint Enable) 位来开启, 并且在相应的 DR0 到 DR3 寄存器设置断点地址 (关于 DR7 寄存器及断点调试相关知识, 请参考《x86/x64 体系探索及编程》第 13 章 “断点调试”)。

表 3-20

位 域	状 态 位	描 述
0	B0	断点 0: 对应 DR0 寄存器, 为 1 时表示遇到了断点 0
1	B1	断点 1: 对应 DR1 寄存器, 为 1 时表示遇到了断点 1

续表

位 域	状 态 位	描 述
2	B2	断点 2：对应 DR2 寄存器，为 1 时表示遇到了断点 2
3	B3	断点 3：对应 DR3 寄存器，为 1 时表示遇到了断点 3
11:4	保留	---
12	enable breakpoint	为 1 时，表示遇到了断点
13	保留	---
14	BS	为 1 时，表示遇到了单步调试断点
63:15	保留	---

当 enable breakpoint 位 (bit 12) 为 1 时，表示遇到了断点。那么 B3:B0 相应位置位，指示遇到了哪个断点。当 BS 位 (bit 14) 为 1 时，表示遇到了单步调试。

pending debug exception 字段只支持记录 trap 类型 #DB 异常（由**数据断点**或者**I/O 断点**触发的 #DB 异常，以及由**单步调试**而触发的 #DB 异常），对于 fault 类型的 #DB 异常不能记录。

#DB 异常的组织

当遇到一个断点（包括指令断点、数据断点及 I/O 断点）或 single-step（单步调试）时，处理器将组织一个 #DB 异常，并提交处理器执行。

注意：当一个**指令断点**放在 MOV-SS/POP-SS 指令后面时，这个 #DB 异常将被抑制，或者说并没有被组织。而其他类型的 #DB 异常，遇到“blocking by MOV-SS”阻塞状态后将会被 pending（悬挂），等到“blocking by MOV-SS”状态解除后仍可 deliver 执行。

“blocking by MOV-SS”阻塞状态并不能阻塞由 DR 寄存器访问而产生的 #DB 异常。当 DR7.GD=1 时，遇到 MOV-DR 指令将直接提交 #DB 异常执行，或者产生 VM-exit 行为。

实际上，在 pending debug exception 字段中只能记录到由**数据或 I/O 断点**产生的 #DB 异常及由**单步调试**产生的 #DB 异常，单步调试分为**指令单步调试**及**分支单步调试**两种。

在 VM-exit 时存在 #DB 异常的 pending 状态，有如下两种可能。

(1) 一条指令产生 trap 类型的 VM-exit 事件，在 VM-exit 之前，断点已经由于这条指令而被触发。

(2) 在 VM-exit 前，已经存在 #DB 异常由于“blocking by MOV-SS”阻塞状态而被 pending。

pending debug exception 字段只存在“enable breakpoint”和 BS 位。对于指令断点的 #DB 异常和 DR 寄存器访问的 #DB 异常，它们不会被悬挂。在 VMX 架构下，并不支持在 non-root operation 环境中使用任务切换机制。一旦发生任务切换，将产生 VM-exit 行为。因此，在 pending debug exception 字段中并不支持记录任务切换时产生的 #DB 异常。

#DB 异常触发方式

在 x86/x64 体系上的 #DB（调试）异常使用 1 号向量，分为 fault 和 trap 两类，共有 5 个触发方式。fault 类型 #DB 异常有以下两类触发方式：

- (1) 由执行断点触发。
- (2) 由访问调试寄存器触发（DR7.GD=1 时）。

当断点对应的 DR7.R/W=0 时，使用执行断点，在断点寄存器 DR0~DR3 里设置断点。当 DR7.GD=1 时，使用 DR 寄存器访问触发方式。在后续的指令中访问任何一个 DR 寄存器将产生 #DB 异常。

trap 类型 #DB 异常有 4 类触发方式：

- (1) 由数据断点触发。
- (2) 由 I/O 断点触发。
- (3) 由单步调试触发。
- (4) 在任务切换时，由 TSS 的 T 标志为 1 时触发。

数据断点是访问的内存地址，I/O 断点是指令访问的 I/O 端口地址。单步调试在将 eflags.TF 置为 1 后，下一条指令执行完毕后触发（分支单步调试则在目标指令执行前触发）。任务切换的 #DB 异常在执行完任务切换后，处理器检测到 TSS.T 为 1 时触发。

注意：“blocking by MOV-SS”阻塞状态能够屏蔽指令断点的 #DB 异常。其他触发方式的 #DB 异常可能会造成 #DB 异常的 pending（悬挂）状态。

#DB 异常触发的位置

fault 类型与 trap 类型的 #DB 异常触发的位置是不同的。

- (1) 执行断点的 #DB 异常，在执行断点指令前触发。
- (2) DR 寄存器访问的 #DB 异常，在执行 MOV-DR 指令前触发。
- (3) 数据断点和 I/O 断点的 #DB 异常，在执行完访问数据后触发。
- (4) 指令单步调试的 #DB 异常，在执行完每一条指令后触发。
- (5) 分支单步调试的 #DB 异常，在执行每一个分支指令后触发。
- (6) 任务切换的 #DB 异常，在任务切换完毕后，新任务第一条指令执行前触发。

这些 #DB 异常的触发位置，将影响到 #DB 异常组织和 delivery 结果。在 VMX 的 non-root operation 环境下，决定了 pending debug exception 字段的记录。注意，pending debug exception 字段记录的是 trap 类型 #DB 异常，fault 类型 #DB 异常会直接执行或者丢失。

3.8.7.1 #DB 异常的处理

在 VM-exit 时，pending debug exception 字段会记录当前处理器是否有未处理而悬挂

的#DB 异常。

执行断点#DB 异常

遇到了执行断点的#DB 异常，pending debug exception 字段是不能记录到的。有下面两种情形：

```
SET_BREAKPOINT 0, BP_FETCH, Breakpoint          ; 设置执行断点 0
... ..
Breakpoint:
    cpuid                                          ; 这是一条 cpuid 指令
```

上面的代码中，断点处是一条 CPUID 指令，在一般情形下这会产生 VM-exit。但是，由于执行断点的#DB 异常在指令执行前已经触发了，而不会执行 CPUID 指令。因此，将执行#DB 异常服务例程，并不会产生 VM-exit。处理#DB 服务例程是 guest OS 的职责。在#DB 服务例程返回前，应将栈中的 eflags.RF 置位，返回到断点后能够继续执行指令。否则，将产生死循环。

```
SET_BREAKPOINT 0, BP_FETCH, Breakpoint          ; 设置执行断点 0
... ..
    mov ss, ax                                  ; 更新 SS
Breakpoint:
    cpuid                                          ; 存在“blocking by MOV-SS”阻塞
```

在上面的代码中，执行断点在 MOV-SS 指令后，此时，在断点中存在“blocking by MOV-SS”阻塞状态，#DB 异常不能被组织 deliver 执行。那么，这个#DB 异常将会丢失，从而执行 CPUID 指令产生 VM-exit。由于#DB 异常丢失了，在 pending debug exception 字段中并不会记录这个#DB 异常。

综上所述，遇到执行断点的#DB 异常，要么直接 deliver 执行，要么丢失，并不能在 pending debug exception 字段中记录。VMM 无须特别处理这类#DB 异常。

DR 寄存器访问的#DB 异常

和执行断点的#DB 异常情形处理不同，MOV-SS/POP-SS 指令不能阻塞由 DR 寄存器访问而触发的#DB 异常。这类#DB 异常要么产生 VM-exit，要么提交处理器执行。如下代码：

```
SET_DR_GENERAL_DETECT                            ; 启用 DR 寄存器访问#DB 异常
... ..
Breakpoint:
    mov eax, dr6                                ; 访问 DR6 寄存器
```

在 guest 里，这个#DB 异常是否能得到响应，取决于“MOV-DR exiting”的值（见 3.5.2.1 节）。当“MOV-DR exiting”为 1 时，执行这条指令将产生 VM-exit，而不会触发#DB 异常。

在这种情形下，对于这个 VM-exit 事件，VMM 可以检查 DR7 寄存器是否启用 DR 寄存器访问#DB 异常。如果启用了，那么在虚拟化 DR6 寄存器访问后，下次重新进入 VM 时，注入一个#DB 异常事件让 guest 继续执行。如果通过设置 pending debug exception 字段的 enable breakpoint 或 BS 位来达到提交#DB 异常给 guest。虽然可以实

现，但不太符合逻辑。这是由于 pending debug exception 字段并不记录 DR 寄存器访问的 #DB 异常。

当“MOV-DR exiting”为 0 时，MOV-DR 指令将触发 #DB 异常，从而执行 #DB 异常服务例程。

```

SET_DR_GENERAL_DETECT                                ; 启用 DR 寄存器访问 #DB 异常
... ..
mov ss, ax                                           ; 下面存在“blocking by MOV-SS”
Breakpoint:
mov eax, dr6                                         ; 访问 DR6 寄存器

```

注意：即使 MOV-DR 指令放在 MOV-SS/POP-SS 指令后面，这个“blocking by MOV-SS”阻塞状态也并不能阻塞这个 #DB 异常，#DB 异常会得到 delivery 执行。

综上所述，pending debug exception 字段并不能记录由 DR 寄存器访问而触发的 #DB 异常。

数据断点 #DB 异常

数据断点的 #DB 异常属于 trap 类型，在执行完访问断点地址后触发。“blocking by MOV-SS”阻塞状态可能会阻塞 #DB 异常，也可能不会阻塞 #DB 异常，取决于阻塞状态产生的位置。

当 VM-exit 时，如果由于“blocking by MOV-SS”而阻塞 #DB 异常，此时存在 #DB 异常的悬挂。在 pending debug exception 字段中就会记录这个悬挂的 #DB 异常。

```

SET_BREAKPOINT 0, BP_READ_WRITE4, 400000h          ; 设置一个读写断点 0
... ..
mov ss, ax                                           ; 下面存在“blocking by MOV-SS”
mov eax, [400000h]                                  ; #DB 异常在执行完指令后，被提交执行

```

上面的代码中，设置了一个读写断点 0，断点地址是 400000h。在 MOV-SS 指令后面的指令对断点地址进行读访问，在这个位置存在“blocking by MOV-SS”阻塞状态。但是，由于这个 #DB 异常在执行完 MOV 指令后触发，在 MOV 指令执行完毕后“blocking by MOV-SS”阻塞状态已经被解除。因此，在这种情形下，#DB 异常是正常可以被 deliver 执行的。

注意：trap 类型的 #DB 异常由前一条指令访问了数据或 I/O 断点而产生，或者由于指令单步调试产生。在下一条指令边界上，这个 trap 类型的 #DB 优先级别高于其他异常（例如 #PF 异常），也高于 NMI 与外部中断。

假如，在访问断点地址 400000h 时出现了 #PF 异常，处理器转为执行 #PF 异常服务例程。由于指令并没有成功访问到断点，不会产生 #DB 异常。如果由于 #PF 异常而直接产生 VM-exit，此时也就不存在 #DB 异常的 pending（悬挂）。

在 #PF handler 修复这个错误后，重新返回到访问数据断点的指令继续执行，指令执行完毕后，这个 #DB 异常还是会被组织 deliver。

```

SET_BREAKPOINT 0, BP_READ_WRITE4, 400000h          ; 设置一个读写断点 0
... ..

```

```
mov eax, [400000h] ; 执行完后将产生 #DB 异常
cpuid              ; 不会产生 VM-exit
```

在一般情况下, CPUID 指令会产生 VM-exit 行为。在上面的代码中, CPUID 指令被放在数据断点访问指令后。此时, 在 CPUID 指令执行前#DB 异常已经被组织 deliver, 转而执行#DB 异常服务例程, 并不会产生 VM-exit。

```
SET_BREAKPOINT 0, BP_READ_WRITE4, 400000h ; 设置一个读写断点 0
... ..
mov ss, [400000h] ; 更新 SS 寄存器, 但访问了数据断点
mov eax, 1 ; 存在“blocking by MOV-SS”阻塞状态
mov eax, 2 ; #DB 异常在此条指令前触发
mov eax, 3 ;
```

上面的代码展示了一个#DB 异常被悬挂的情形: 执行“**mov ss, [400000h]**”指令访问了数据断点, 处理器组织了一个#DB 异常 pending 在这条指令之后。但这条指令由于进行 SS 切换而同时产生了“blocking by MOV-SS”阻塞状态, #DB 异常继续保持 pending 状态。

当 MOV-SS 指令的下一条指令(即“**mov eax, 1**”)执行完毕后, “blocking by MOV-SS”阻塞状态自动被解除。此时, 悬挂的#DB 异常会被立即 deliver 执行。也就是在“**mov eax, 2**”指令执行前#DB 异常被 deliver 执行。此时, EAX 寄存器的值为 1 (执行了“**mov eax, 1**”指令)。

```
SET_BREAKPOINT 0, BP_READ_WRITE4, 400000h ; 设置一个读写断点 0
... ..
mov ss, [400000h] ; 下面存在“blocking by MOV-SS”
cpuid ; 产生 VM-exit 行为, #DB 异常被悬挂
mov eax, 1
mov eax, 2
```

假如在访问数据断点后面**直接产生**了 VM-exit (譬如 MOV-SS 指令后面放一条 CPUID 指令)。由于 CPUID 指令未执行而直接引发 VM-exit, 此时“blocking by MOV-SS”阻塞状态**还没被解除**。因此, 产生 VM-exit 后#DB 异常被阻塞而保持悬挂着。在 pending debug exception 字段中将会记录这个悬挂着的#DB 异常信息。

pending debug exception 字段的值为 **00001001H**。enable breakpoint (bit 12) 位为 1, 指示遇到了断点, 存在一个#DB 异常悬挂。B0 位为 1, 指示遇到了断点 0。interruptibility state 字段的值为 2, 指示存在“blocking by MOV-SS”阻塞状态。

这表明, 在 CPUID 指令边界前, 存在#DB 异常的 pending 及“blocking by MOV-SS”阻塞状态。VM-exit 时将保存这些状态信息。但是在下一指令边界前这些状态将被解除。

VMM 在下次重新进入 VM 时, 无须修改 pending debug exception 字段的值, 就可以在 VM-entry 后直接 deliver 一个#DB 异常给 guest 执行。

假如, 在 VM-entry 时, interruptibility state 字段的值保持为 2, 表明“blocking by MOV-SS”阻塞状态继续有效, #DB 异常保持 pending 状态。直到 VM-entry 后的第一条指令执行完毕后“blocking by MOV-SS”阻塞状态被解除, 悬挂的#DB 异常才被 deliver 执行。

但是，这样的处理不符合逻辑。因为按正常执行流程，CUID 指令执行完毕后（即下一条指令边界）悬挂的#DB 异常应该被 deliver 执行。由于 VMM 需要完成 CUID 指令的虚拟化，直接返回虚拟化 CUID 后的结果给 guest。此时，guest-RIP 必须指向 CUID 指令的下一条指令“**mov eax, 1**”，否则会不断尝试执行 CUID 指令而产生死循环。

如果“blocking by MOV-SS”继续保持有效，需要等“**mov eax, 1**”指令执行完毕后悬挂的#DB 异常才能被 deliver 执行。也就是说，#DB 异常的触发被延迟了一条指令。

Intel 也明确说明：在进入 VM-entry 时，如果还存在“blocking by MOV-SS”阻塞状态，这个悬挂的#DB 异常也可能会丢失。因此，VMM 应该在 VM-entry 时，将 interruptibility state 字段的 bit 1 清 0，去除“blocking by MOV-SS”阻塞状态，这是正确的处理方式。

I/O 断点的#DB 异常

在 guest 中，I/O 断点的#DB 异常能否 deliver 执行取决于 primary processor-based VM-execution 字段的“unconditional I/O exiting”和“use I/O bitmap”控制位，以及 I/O bitmap 的设置（见 3.5.2.1 节）。

执行 I/O 指令在下面两种设置下都会产生 VM-exit：

(1) 当“unconditional I/O exiting”为 1，并且“use I/O bitmap”为 0 时，执行所有 I/O 指令都会产生 VM-exit。

(2) 当“use I/O bitmap”为 1，并且访问的 I/O 端口地址在 I/O bitmap 中对应的位为 1 时，访问这个 I/O 端口将会产生 VM-exit。

排除上面的 VM-exit 情况，#DB 异常可能在一个 trap 类型的 VM-exit（**TPR below threshold**，**EOI 虚拟化**，**APIC-write VM-exit** 或者 **MTF VM-exit**）之前已经被 pending。那么 VM-exit 时就存在#DB 异常的悬挂。

```
SET_BREAKPOINT 0, BP_IO_READ_WRITE1, 60h          ; 设置一个 I/O 读写断点 0
mov dx, 60h
insb                                                ; 产生 trap 类型 VM-exit
```

上面的代码示例中，如果执行 INSB 指令引发 trap 类型 VM-exit，在 VM-exit 之前这个#DB 异常已经被触发，pending 在 INSB 指令之后。由于 trap 类型 VM-exit 保存的 RIP 为 INSB 指令之后，造成了#DB 异常的悬挂。

此时，pending debug exception 字段记录的值为 **00001001H**。“enable breakpoint”位与 B0 位为 1，指示遇到了 0 号 I/O 断点。

指令单步调试#DB 异常

单步调试产生的#DB 异常也属于 trap 类型。当将 eflags.TF 置位，并且 IA32_DEBUGCTL.BTF=0 时，在每一条指令执行完毕后，处理器将组织#DB 异常并提交执行，直到将 eflags.TF 位清 0 为止。

```

pushf
bts DWORD [esp], 8          ; TF = 1
popf
... ..
mov ss, ax                  ; 切换 SS
mov eax, 1                  ; 存在“blocking by MOV-SS”，#DB 异常被悬挂
mov eax, 2                  ; #DB 异常在指令执行前触发

```

在上面的代码中，POF 指令将 `eflags.TF` 置为 1。后续的一条 MOV-SS 指令执行产生“blocking by MOV-SS”阻塞状态，下一条“`mov eax, 1`”指令中存在 #DB 异常被阻塞而悬挂。

直到 `mov eax, 1` 指令执行完毕后，“blocking by MOV-SS”阻塞状态被解除。#DB 异常被提交执行，此时，EAX 寄存器的值为 1。

```

pushf
bts DWORD [esp], 8          ; TF = 1
popf
... ..
mov ss, ax                  ; 切换 SS
cpuid                       ; 存在“blocking by MOV-SS”，并且#DB 被悬挂
mov eax, 1                  ;
mov eax, 2                  ;

```

在上面的代码中，MOV-SS 指令后面直接产生 VM-exit（例如是一条 CPUID 指令），由于 #DB 异常被阻塞而执行 CPUID 指令产生 VM-exit。pending debug exception 字段将记录这个悬挂的 #DB 异常，它的值为 **00004000H**。BS 位为 1，指示遇到了单步调试 #DB 异常。同时 interruptibility state 字段的值为 2，表示存在“blocking by MOV-SS”阻塞状态。

同样，在指令单步调试中，如果执行一条指令引发了 trap 类型的 VM-exit，那么在 VM-exit 之前已经存在 #DB 异常的悬挂状态。

VMM 应该清除 interruptibility state 字段的“blocking by MOV-SS”阻塞状态，保持 pending debug exception 字段不变。下一次重新进入 VM 时，#DB 异常会得到 deliver 执行。

分支单步调试#DB 异常

当 `TF=1`，并且 `IA32_DEBUGCTL.BTF=1` 时，#DB 异常被 pending 在每条分支指令执行后的目标地址之前（分支单步调试）。

```

... ..                      ; TF=1, IA32_DEBUGCTL.BTF=1
jmp next
... ..
next:                          ; <--- #DB 异常 pending
mov eax, 1
... ..

```

上面的代码示例启用了分支单步调试，执行 JMP 指令跳转后，#DB 异常被 pending 在目标地址前。此时，由于 primary processor-based VM-execution control 字段的“monitor trap flag”位为 1 而产生 MTF VM-exit，在这种情况下，VM-exit 时就存在分支单步调试 #DB 异常的悬挂状态。pending debug exception 字段的值为 **00004000H**，指示遇

到了单步调试#DB 异常。

注意：在分支单步调试中不会存在“blocking by MOV-SS”阻塞状态。因此在 VM-entry 时，处理器不存在对 pending debug exception 字段 BS 位进行检查的这个步骤（参见 4.5.9 节）。

3.8.8 VMCS link pointer 字段

这个字段是 64 位宽，目前仅用在 SMM 双重监控处理机制下。当 executive-VMCS pointer 字段的值为 VMXON pointer 时，VMCS link pointer 字段保存着在切换到 STM (SMM-transfer monitor) 前的 current-VMCS pointer。当从 STM 切换回 executive monitor 时，处理器将从 VMCS link pointer 字段中读取回原来的 current-VMCS pointer 值。

Intel 推荐这个字段的初始值设为 FFFFFFFF_FFFFFFFFh，这样在非 SMM 双重监控处理机制下的 VM-entry 可以不检查这个字段，避免由于这个字段的检查无效而造成 VM-entry 失败。

3.8.9 VMX-preemption timer value 字段

这个字段为 32 位宽，当设置 pin-based VM-execution control 字段的“activate VMX-preemption timer”位为 1 时（见 3.5.1 节），这个字段提供一个 32 位的定时器初始计数值。在 VM-entry 开始时这个值开始递减，当减为 0 时产生 VM-exit。递减的步伐取决于 TSC 及 IA32_VMX_MISC[4:0]值（见 2.5.11 节）。

3.8.10 PDPTEs 字段

PDPTEs (Page-Directory Pointer Table Entries, 页目录指针表项) 字段共有 4 个，分别为 PDPTE0, PDPTE1, PDPTE2 及 PDPTE3，每个字段固定为 64 位宽。这些字段在支持 EPT 功能时有效。

当 secondary processor-based VM-execution control 字段的“enable EPT”位为 1 时（见 3.5.2.2 节），如果 guest OS 使用 PAE 分页模式，在 VM-entry 时 4 个 PDPTEs 字段中的值将会被加载到处理器内部的 PDPTE 寄存器。此时，CR3 寄存器被忽略。

当“enable EPT”为 0 时，如果 guest OS 使用 PAE 分页模式，VM-entry 时，CR3 寄存器所指向的 4 个 PDPTEs 会被加载到处理器内部的 PDPTE 寄存器，VMCS 内的 PDPTEs 字段会被忽略。

PAE 分页模式是指：CR4.PAE=1，CR0.PG=1，并且 IA32_EFER.LME 为 0 时，使用的是非 IA-32e 模式下的分页模式。

3.8.11 guest interrupt status 字段

在开启“virtual-interrupt delivery”功能的情况下，guest interrupt status 字段被作为虚拟 local APIC 的状态值，处理器使用这个字段来维护虚拟中断的状态。

16 位的 guest interrupt status 字段包括两部分：**RVI** (Requesting Virtual Interrupt) 及 **SVI** (Servicing Virtual Interrupt)。它们是 8 位宽，保存虚拟中断的向量号。

- RVI 记录处理器当前优先级最高的 virtual-interrupt 请求的向量号。
- SVI 记录处理器当前正在执行的 virtual-interrupt 服务例程的向量号。

这两个字段与 virtual-APIC page 页面内的 **VIRR** (Virtual Interrupt Request Register，虚拟的中断请求寄存器) 以及 **VISR** (Virtual In-Service Register，虚拟的中断服务寄存器) 相互作用。

当执行 **EOI 虚拟化**，**Self-IPI 虚拟化** 或者 **虚拟中断 delivery** 操作时，处理器会更新 guest interrupt status 字段的值。

EOI 虚拟化

在虚拟中断 delivery 后，SVI 记录处理器当前正在服务的 virtual-interrupt 向量号。中断服务例程在返回前将进行 EOI 虚拟化操作，将 SVI 值所对应在 VISR 的位清 0，然后更新 SVI 值。

如果当前有虚拟中断服务例程的嵌套，也就是上一个虚拟中断服务例程被优先级更高的虚拟中断请求打断，当虚拟中断服务例程**发送 EOI 命令**后将返回到之前**被中断的服务例程**继续执行。那么，新的 SVI 值就等于之前被中断服务例程的向量号。

```
if (VISR != 0)                                /* VISR 不为 0，表明存在中断服务例程的嵌套 */
{
    SVI = VISR 中为 1 的最高位 index 值;      /* 选择一个优先级别最高的服务例程 */
}
else
    SVI = 0;                                  /* 虚拟中断 In-service 列表为空 */
```

SVI 值等于 VISR 中“为 1 的最高位”所对应的向量号。也就是从**虚拟中断的 In-service 列表**中选择一个优先级最高的中断服务例程的向量号。如果 VISR 为 0，表示虚拟中断 In-service 列表为空，则 SVI 的值为 0。关于“EOI 虚拟化”详见 7.2.12.3 节描述。

Self-IPI 虚拟化

Self-IPI 虚拟化将更新 RVI 的值，处理器从 VIRR 中取出“为 1 的最高位 index”作为 RVI 值。也就是从当前的虚拟中断请求列表中选择优先级别最高的虚拟中断。

```
VIRR[vector] = 1;                            /* 置虚拟中断请求位 */
RVI = VIRR 中为 1 的最高位 index 值;         /* 从中断请求中选择一个优先级别最高的 */
```

VIRR 相当于虚拟中断请求列表，记录着所有发出的虚拟中断请求。当前 RVI 值就是这个列表优先级别最高的向量号。当不断有更高优先级别的虚拟中断请求发出时，RVI 就会不断地被更新。而低优先级别的虚拟中断请求则不影响 RVI 值。关于“Self-IPI 虚拟

化” 详见 7.2.12.4 节描述。

虚拟中断 delivery

EOI 虚拟化和 Self-IPI 虚拟化最后的结果可能会产生虚拟中断的 delivery 操作。虚拟中断 delivery 操作会更新 RVI 与 SVI 值。

```

SVI = RVI;                                /* SVI 等于 RVI */
VIRR[RVI] = 0;                            /* 清 RVI 对应的虚拟中断请求位 */
VISR[SVI] = 1;                            /* 置 SVI 对应的虚拟中断服务位 */
If (VIRR != 0)
{
    RVI = VIRR 中为 1 的最高位 index 值;    /* 选择下一个优先级最高的中断请求 */
}
Else
    RVI = 0;
    
```

在虚拟中断 delivery 操作中，SVI 被赋于当前的 RVI 值。VIRR 对应的位被清 0，而 VISR 对应的位被置 1。RVI 被更新为 VIRR 为 1 的最高位 index 值，也就是选择下一个优先级最高的中断请求向量号。当 VIRR 列表为空时，RVI 为 0 值。关于“虚拟中断 delivery” 详见 7.2.13 节描述。

3.9 host-state 区域字段

在 VM-exit 发生时，处理器将从 host-state 区域读取 host（VMM）的环境信息，加载到处理器内，转入 host-state 区域内提供的入口点继续执行。

当进行 VM-entry 操作时，如果由于 guest-state 区域字段检查或加载而引起 VM-entry 失败，处理器也会从 host-state 区域里加载 host 环境信息转入 host 入口点继续执行。

如表 3-21 所示，host-state 区域内的字段比 guest-state 字段少了许多，那是因为当返回到 host（VMM）时，有许多处理器的状态值是强制设置的。因此，这此状态值不必提供在 host-state 区域中。

表 3-21

类 型	字 段	宽 度	字段 ID
控制寄存器	CR0	natural-width	00006C00H
	CR3		00006C02H
	CR4		00006C04H
栈指针	RSP	natural-width	00006C14H
指令指针	RIP		00006C16H
段选择子	ES selector	16 位	00000C00H
	CS selector		00000C02H
	SS selector		00000C04H
	DS selector		00000C06H

续表

类 型	字 段	宽 度	字段 ID
	FS selector	16 位	00000C08H
	GS selector		00000C0AH
	TR selector		00000C0CH
段基址	FS base	natural-width	00006C06H
	GS base		00006C08H
	TR base		00006C0AH
	GDTR base		00006C0CH
	IDTR base		00006C0EH
MSR	IA32_SYSENTER_CS	32 位	00004C00H
	IA32_SYSENTER_ESP	natural-width	00006C10H
	IA32_SYSENTER_EIP		00006C12H
	IA32_PERF_GLOBAL_CTRL	64 位	00002C04H （full） 00002C05H （high）
	IA32_PAT		00002C00H （full） 00002C01H （high）
	IA32_EFER		00002C02H （full）
			00002C03H （high）

控制寄存器字段

CR0 与 CR4 寄存器必须符合 VMX 架构对进入 VMX root-operation 模式的要求（见 2.3.2.2 节与 2.5.10.1 节），host 或 VMM 必须运行在分页的保护模式下。因此，CR0.PE 和 CR0.PG 必须为 1，CR0.NE 也必须为 1。CR4.VMXE 必须为 1，表明允许进入 VMX 模式。

在设置 host-state 区域的控制寄存器时，一个比较好的主意是：在初始化 VMCS 区域时，复制 VMM 当前的 CR0、CR3 与 CR4 寄存器的值到这些控制寄存器字段中，这必定是正确的。

host-RSP 与 host-RIP 字段

这两个字段属于 natural-width 类型。在发生 VM-exit 后，处理器从 host-RIP 字段读取 VMM 的入口地址，执行 VMM 的管理例程。host-RSP 字段提供 VMM 使用的栈指针。

段 selector 字段

host-state 区域有 6 个 code/data 段寄存器和一个 TR 寄存器 selector 字段，固定为 16 位宽。注意：host-state 区域里不提供 LDTR 寄存器的 selector 字段。

段基址字段

host-state 区域的段基址字段只有 5 个：FS、GS、TR、GDTR 及 IDTR 寄存器基址字

段。其他的 code/data 段基址字段都不提供。在发生 VM-exit 后，CS.base 强制设置为 0 值，而 ES，SS 及 DS 段寄存器在内部 access right 的 unusable 位为 0（表示可用的）时，base 值设置为 0 值，否则为未定义值。

MSR 字段

host-state 区域提供 6 个 MSR 字段。IA32_SYSENTER_CS 字段为 32 位，IA32_SYSENTER_ESP 及 IA32_SYSENTER_EIP 字段属于 natural-width 类型（在 64 位处理器上是 64 位，否则为 32 位）。

IA32_PERF_GLOBAL_CTRL，IA32_PAT 及 IA32_EFER 字段是固定 64 位。假如它们在 VM-exit control 字段中相应的“load IA32_xxx”位为 1，发生 VM-exit 后，处理器相应地加载这几个 MSR 字段。假如“load IA32_EFER”为 1，IA32_EFER 字段 LME 和 LMA 位的值必须等于“Host address-space size”位的值。

host-state 并不提供 IA32_DEBUGCTL 和 DR7 寄存器字段。在发生 VM-exit 后，IA32_DEBUGCTL 寄存器被强制写为 0 值，DR7 寄存器强制设为 00000400H 值。

3.10 VM-exit 信息类字段

VMCS 的 VM-exit 信息类字段用来保存发生 VM-exit 事件的原因及明细信息，VMM 利用这些信息来决定如何管理和控制 VM。VM-exit 信息类字段的 VM-instruction error 字段也保存着 VMX 指令发生 VMfailValid 失败后的原因值（见 2.6.3 节）。

表 3-22

类 型	字 段 名	宽 度	字 段 ID
基本信息类	exit reason	32 位	00004402H
	exit qualification	natural-width	00006400H
	guest-linear address		0000640AH
	guest-physical address	64 位	00002400H (full) 00002401H (high)
直接向量事件类	VM-exit interruption information	32 位	00004404H
	VM-exit interruption error code		00004406H
间接向量事件类	IDT-vectoring information	32 位	00004408H
	IDT-vectoring error code		0000440AH
指令信息类	VM-exit instruction length	32 位	0000440CH
	VM-exit instruction information		0000440EH
I/O SMI 信息类	I/O RCX	natural-width	00006402H
	I/O RSI		00006404H
	I/O RDI		00006406H

续表

类 型	字 段 名	宽 度	字段 ID
I/O SMI 信息类	I/O RIP	natural-width	00006408H
指令失败类	VM-instruction error	32 位	00004400H

如表 3-22 所示，VM-exit 信息类字段分为 6 个类别。如前面所述，VM-instruction error 字段保存着执行 VMX 指令发生 **VMfailValid** 失败时的错误原因值。当 VMX 指令发生 **VMfailInvalid** 失败时，不会产生错误原因值。

3.10.1 基本信息类字段

包括如下 4 个字段：

(1) exit reason 字段，32 位宽，保存导致 VM-exit 的原因值。

(2) exit qualification 字段，属于 natural-width 类型字段，进一步提供导致 VM-exit 的某些事件的明细信息。

(3) guest-linear address 字段，属于 natural-width 类型字段，保存导致 VM-exit 的某些事件的线性地址值。

(4) guest-physical address 字段，64 位宽，保存由于 EPT violation 或者 EPT misconfiguration 故障引起 VM-exit 时的 GPA (Guest Physical Address) 值。

每类导致 VM-exit 的事件都对应一个原因编号，VMM 根据这个原因编号值来确定由什么事件导致 VM-exit 发生，再结合其余字段进一步提供的明细信息做出相应处理。

3.10.1.1 Exit reason 字段

exit reason 字段由几个位域组成，如表 3-23 所示。

表 3-23

位 域	描 述
15:0	保存 VM 退出原因值
27:16	保留位 (为 0)
28	为 1 时，指示 SMM VM-exit 时，存在 pending MTF VM-exit 事件
29	为 1 时，指示 SMM VM-exit 从 VMX root-operation 中产生
30	保留位 (为 0)
31	为 1 时，表明是在 VM-entry 过程中引发了 VM-exit

bits 15:0 保存 16 位的退出原因码，bit 28 和 bit 29 都使用在 SMM 双重监控处理机制下。bit 28 为 1 时，指示一个 SMM VM-exit 发生后，pending MTF VM-exit 事件在悬挂中。这种情况发生在同时收到 SMI 及 pending MTF VM-exit 事件时，由于 SMI 的优先级别高于 pending MTF VM-exit，处理器将响应 SMI，而导致 pending MTF VM-exit 事件被

悬挂。

bit 29 为 1 时，指示一个 SMM VM-exit 发生在 VMX root-operation 环境中（见 2.4.3 节）。为 0 时，表明 SMM VM-exit 发生在 VMX non-root operation 环境中。

bit 31 为 1 时，表明是在 VM-entry 过程中，由于 guest-state 的检查或者信息的加载失败而产生 VM-exit。为 0 时，表明属于在 VMX non-root operation 模式中的 VM-exit 行为。

3.10.1.2 VM-exit 原因

当前实现的 VMX 架构里共有 57 个退出原因码，如表 3-24 所示。

表 3-24

退出原因码	描 述
0	VM-exit 由异常或 NMI 引发
1	VM-exit 由外部中断引发
2	VM-exit 由 triple-fault 引发
3	由 INIT 信号导致 VM-exit
4	由 SIPI 导致 VM-exit
5	I/O SMI：指接到 SMI 而发生 SMM VM-exit，SMI 发生在 I/O 指令后
6	Other SMI：指接收 SMI 而发生 SMM VM-exit，SMI 不发生在 I/O 指令后
7	VM-exit 由“interrupt-window exiting”为 1 时引发
8	VM-exit 由“NMI-window exiting”为 1 时引发
9	尝试进行任务切换而导致 VM-exit
10	尝试执行 CPUID 指令导致 VM-exit
11	尝试执行 GETSEC 指令导致 VM-exit
12	VM-exit 由尝试执行 HLT 指令引发
13	尝试执行 INVD 指令导致 VM-exit
14	VM-exit 由尝试执行 INVLPG 指令引发
15	VM-exit 由尝试执行 RDPMC 指令引发
16	VM-exit 由尝试执行 RDTSC 指令引发
17	VM-exit 由尝试执行 RSM 指令引发
18	尝试执行 VMCALL 指令导致 VM-exit
19	尝试执行 VMCLEAR 指令导致 VM-exit
20	尝试执行 VMLAUNCH 指令导致 VM-exit
21	尝试执行 VMPTRLD 指令导致 VM-exit
22	尝试执行 VMPTRST 指令导致 VM-exit
23	尝试执行 VMREAD 指令导致 VM-exit
24	尝试执行 VMRESUME 指令导致 VM-exit
25	尝试执行 VMWRITE 指令导致 VM-exit

续表

退出原因码	描 述
26	尝试执行 VMXOFF 指令导致 VM-exit
27	尝试执行 VMXON 指令导致 VM-exit
28	VM-exit 由尝试访问控制寄存器引发
29	VM-exit 由尝试执行 MOV-DR 指令引发
30	VM-exit 由尝试执行 I/O 指令引发
31	VM-exit 由尝试执行 RDMSR 指令引发
32	VM-exit 由尝试执行 WRMSR 指令引发
33	在 VM-entry 时，由于无效 guest-state 字段而导致 VM-exit
34	在 VM-entry 时，由于 MSR 加载失败而导致 VM-exit
35	---
36	VM-exit 由尝试执行 MWAIT 指令引发
37	在 VM-entry 后，由 MTF (monitor trap flag) 导致 VM-exit
38	---
39	VM-exit 由尝试执行 MONITOR 指令引发
40	VM-exit 由尝试执行 PAUSE 指令引发
41	在 VM-entry 时，由于 machine-check 而导致 VM-exit
42	---
43	由于 VTPR 低于 TPR threshold 值而导致 VM-exit
44	访问 APIC-access page 页面导致 VM-exit
45	执行 EOI 虚拟化而导致 VM-exit
46	VM-exit 由尝试访问 GDTR 或 IDTR 寄存器引发
47	VM-exit 由尝试访问 LDTR 或 TR 寄存器引发
48	由于 EPT violation 而导致 VM-exit
49	由于 EPT misconfiguration 而导致 VM-exit
50	VM-exit 由尝试执行 INVEPT 指令引发
51	VM-exit 由尝试执行 RDTSCP 指令引发
52	由于 VMX-preemption timer 超时而导致 VM-exit
53	尝试执行 INVVPID 指令而导致 VM-exit
54	VM-exit 由尝试执行 WBINVD 指令引发
55	尝试执行 XSETBV 指令而导致 VM-exit
56	VM-exit 由尝试向 APIC-access page 写入而引发
57	VM-exit 由尝试执行 RDRAND 指令而引发
58	VM-exit 由尝试执行 INVPCID 指令而引发
59	VM-exit 由执行 VMFUNC 指令而引发

当执行一条 VMX 指令（譬如 VMLAUNCH 指令）时可能会发生 VM-exit，也可能是指令执行失败。VM-exit 原因码 0 值是有效的，它指示为“由异常或 NMI 导致 VM-exit”。而 VM-exit instruction error 字段的值为 0 时，表明 VMX 指令执行成功。

如果我们需要输出 VM-exit 信息区域所有字段的信息，当根据 exit reason 字段值输出相应的 VM-exit 原因信息时，需要先检查 VM-exit instruction error 字段，确定 VMX 指令是成功的，然后再输出具体的 VM-exit 原因信息，如下面的伪码所示。

```
if (vmexit_instruction_error == 0)
{
    /*
     * 输出 VM-exit 原因信息
     */
    msg = ExitReasonInfoTable[exit_reason];
    printf("%s\n", msg);
}
else
{
    /*
     * 否则打印 exit reason 字段值
     */
    printf("%s%d", exit_reason);
}
```

导致 VM-exit 发生的原因中，主要分无条件 VM-exit 与有条件 VM-exit 两大类。另外在 VMX non-root operation 模式中收到某些事件也会直接产生 VM-exit（例如 INIT，SMI，以及 SIPI 消息）。当 guest 发生 triple fault 时，也会无条件地产生 VM-exit。

无条件导致 VM-exit 的指令

在 VMX non-root operation 模式中，执行所有 VMX 指令（除 VMFUNC 指令外），CPUID，GETSEC，INVD 及 XSETBV 指令将无条件地导致 VM-exit 发生。

但是，在启用 unrestricted guest 功能的前提下，如果 **guest 运行在实模式**（CR0.PE = 0），执行所有 VMX 指令（除 VMFUNC 指令外）将产生 #UD 异常，而不是 VM-exit。

有条件的导致 VM-exit 的指令

取决于 primary processor-based VM-execution control 和 secondary processor-based VM-execution control 字段（见 3.5.2 节）的设置，执行某些指令能引发 VM-exit：

- (1) 当“HLT exiting”为 1 时，尝试执行 HLT 指令将导致 VM-exit。
- (2) 当“INVLPG exiting”为 1 时，尝试执行 INVLPG 指令将导致 VM-exit。
- (3) 当“enable INVPCID”和“INVLPG exiting”为 1 时，尝试执行 INVPCID 指令将导致 VM-exit。
- (4) 当“RDPMC exiting”为 1 时，尝试执行 RDPMC 指令将导致 VM-exit。
- (5) 当“RDTSC exiting”为 1 时，尝试执行 RDTSC 指令将导致 VM-exit。
- (6) 当“enable RDTSCP”和“RDTSC exiting”为 1 时，尝试执行 RDTSCP 指令将

导致 VM-exit。

(7) 当在 STM (SMM-transfer monitor) 内时, 尝试执行 RSM 指令将导致 VM-exit。

(8) 当 “CR3-load exiting” 为 1, 并且写入的值不等于 CR3-target 值或者 CR3-count 值为 0 时, 尝试执行 MOV to CR3 指令将导致 VM-exit (见 3.5.8 节)。

(9) 当 “CR3-store exiting” 为 1 时, 尝试执行 MOV from CR3 指令将导致 VM-exit。

(10) 当 “CR8-load exiting” 为 1 时, 尝试执行 MOV to CR8 指令将导致 VM-exit。

(11) 当 “CR8-store exiting” 为 1 时, 尝试执行 MOV from CR8 指令将导致 VM-exit。

(12) 当 CR0 的 guest/host mask 字段的某位为 1 时, 尝试执行 MOV to CR0 指令, 若写入该位的值不等于 CR0 的 read shadows 字段对应的位时, 将导致 VM-exit (见 3.5.7 节)。

(13) 当 CR4 的 guest/host mask 字段的某位为 1 时, 尝试执行 MOV to CR4 指令, 若写入该位的值不等于 CR4 的 read shadows 字段对应的位时, 将导致 VM-exit (见 3.5.7 节)。

(14) 当 CR0 的 guest/host mask 与 read shadows 字段的 bit 3 都为 1 时, 尝试执行 CLTS 指令将导致 VM-exit。

(15) 当 CR0 的 guest/host mask 字段 bits 3:1 中的某位为 1 时, 尝试执行 LMSW 指令, 若写入值 bits 3:1 中的该位不等于 CR0 read shadows 字段 bits 3:1 中对应的位, 将导致 VM-exit。

(16) 当 CR0 的 guest/host mask 字段 bit 0 (CR0.PE) 为 1 时, 尝试执行 LMSW 指令, 若写入值的 bit 0 为 1, 并且 CR0 read shadows 字段的 bit 0 为 0 值, 将导致 VM-exit。

(17) 当 “MOV-DR exiting” 为 1 时, 尝试执行 MOV 指令读/写 DR 寄存器将导致 VM-exit。

(18) 当 “unconditional I/O exiting” 为 1, 并且 “use I/O bitmap” 为 0 时, 尝试执行 IN/OUT 类与 INS/OUTS 类指令访问 I/O 端口地址, 将导致 VM-exit。

(19) 当 “use I/O bitmap” 为 1, 并且 I/O bitmap 的某位为 1 时, 尝试执行 IN/OUT 类与 INS/OUTS 类指令访问该位对应的 I/O 端口地址, 将导致 VM-exit。

(20) 当 “use MSR bitmap” 为 0 时, 尝试执行 RDMSR 指令将导致 VM-exit。

(21) 当 “use MSR bitmap” 为 1 时, 使用 RDMSR 指令读 MSR, 但 ECX 寄存器提供的 MSR 地址值不在 00000000H~00001FFFH 或者 C0000000H~C0001FFFH 范围内, 将导致 VM-exit。

(22) 当 “use MSR bitmap” 为 1 时, 使用 RDMSR 指令读 MSR, 但 ECX 寄存器提供的 MSR 地址值对应应在 MSR read bitmap 的位为 1, 将导致 VM-exit (见 3.5.15 节)。

(23) 当 “use MSR bitmap” 为 0 时, 尝试执行 WRMSR 指令将导致 VM-exit。

(24) 当 “use MSR bitmap” 为 1 时，使用 WRMSR 指令写 MSR，但 ECX 寄存器提供的 MSR 地址值不在 00000000H~00001FFFH 或者 C0000000H~C0001FFFH 范围内，将导致 VM-exit。

(25) 当 “use MSR bitmap” 为 1 时，使用 WRMSR 指令写 MSR，但 ECX 寄存器提供的 MSR 地址值对应在 MSR write bitmap 的位为 1，将导致 VM-exit（见 3.5.15 节）。

(26) 当 “MWAIT exiting” 为 1 时，尝试执行 MWAIT 指令将导致 VM-exit。

(27) 当 “MONITOR exiting” 为 1 时，尝试执行 MONITOR 指令将导致 VM-exit。

(28) 当 “PAUSE exiting” 为 1 时，尝试执行 PAUSE 指令将导致 VM-exit。

(29) 当 “PAUSE exiting” 为 0，“PAUSE-loop exiting” 为 1，并且 CPL=0 时，PAUSE 指令在一个循环中执行的时间超过设置的 PLE_window 值，将导致 VM-exit（见 3.5.2.2 节和 3.5.19 节）。

(30) 当 “descriptor-table exiting” 为 1 时，尝试执行 LGDT，SGDT，LIDT，SIDT，LLDT，SLDT，LTR 或者 STR 指令将导致 VM-exit。

(31) 当 “RDRAND exiting” 为 1 时，尝试执行 RDRAND 指令将导致 VM-exit。

(32) 当 “WBINVD exiting” 为 1 时，尝试执行 WBINVD 指令将导致 VM-exit。

(33) 当 “enable VM functions” 为 1 时，尝试执行 VMFUNC 指令，若 EAX 寄存器提供的功能号在 VM-function control 字段相应的位为 0 将导致 VM-exit。

(34) 当 “enable VM functions” 为 1 时，尝试执行 VMFUNC 指令，若 EAX 寄存器提供的功能号为 0，并且 ECX 寄存器提供的 EPT entry 索引值大于 511 将导致 VM-exit。

引发 VM-exit 的事件

下面的事件发生能导致 VM-exit：

(1) 当发生异常（包括 INT3 和 INTO 指令引起的软件异常，BOUND 和 UD2 指令引起的硬件异常），异常向量号在 exception bitmap 对应的位为 1 时引发 VM-exit。对于 #PF 异常有些特殊，当 exception bitmap 的 bit 14 为 1，并且 PFEC & PFEC_MASK = PFEC_MATCH 时，#PF 异常会导致 VM-exit（见 3.5.3 节和 3.5.4 节）。

(2) 当 “NMI exiting” 为 1，并且不存在 “blocking by NMI” 及 “blocking by MOV-SS” 阻塞状态时（某些处理器可能也需不存在 “blocking by STI” 阻塞状态），接收到一个 NMI 请求将引发 VM-exit。

(3) 当 “NMI exiting”，“virtual NMIs” 及 “NMI-window exiting” 都为 1，并且不存在 “blocking by virtual-NMI” 及 “blocking by MOV-SS” 阻塞状态时（某些处理器可能也需要不存在 “blocking by STI” 阻塞状态），VM-entry 后直接引发 VM-exit。

(4) 当 “external-interrupt exiting” 为 1 时，并且不存在 “blocking by STI” 及 “blocking by MOV-SS” 阻塞状态时，接收到一个外部中断请求将引发 VM-exit。

(5) 在“interrupt-window exiting”为 1 的前提下，当 `eflags.IF=1` 并且不存在“blocking by STI”及“blocking by MOV-SS”阻塞状态时，将引发 VM-exit。

(6) 当发生 **triple fault**（三重故障）异常时，将引发 VM-exit。

(7) 当接收到一个 INIT 信号，并且处理器不在 wait-for-SIPI 状态时，将引发 VM-exit。

(8) 当接收到一个 SIPI 消息，并且处理器处于 wait-for-SIPI 状态时，将引发 VM-exit。

(9) 当开启了 SMM 双重监控处理机制，接收到一个 SMI 请求，并且不存在“blocking by SMI”阻塞状态时（即当前不处于 SMM 模式），将引发 SMM VM-exit。在 VMX root-operation 模式执行 VMCALL 指令时也可以主动发起 SMM VM-exit 进入 SMM 里执行。因此，SMM VM-exit 分三种形式：I/O SMI，other SMI 及使用 VMCALL 指令主动发起的 SMM VM-exit。

(10) 在 VMX non-root operation 内尝试进行任务切换（使用 TSS 机制）将引发 VM-exit。

(11) 当“monitor trap flag”为 1 时，在 VM-entry 之后将 pending 一个 MTF VM-exit 在 guest 第 1 条指令后，guest 第 1 条指令执行完毕引发 VM-exit。

(12) 当注入一个 pending MTF VM-exit 事件（中断类型为 other，向量号为 0）时，在 VM-entry 之后，guest 第 1 条指令执行之前引发 VM-exit。

(13) 当“activate VMX-preemption timer”为 1 时，guest-state 区域内的 VMX-preemption timer 值减少为 0 时引发 VM-exit。

(14) 当“use TPR shadow”为 1 时，在 VM-entry 或者 TPR 虚拟化时，如果 `VPTR[7:4]` 低于 TPR threshold 字段的 bits 3:0 值将引发 VM-exit。

(15) 当“use TPR shadow”与“virtualize APIC accesses”为 1 时，执行 EOI 虚拟化时，由于虚拟中断向量号在 EOI-exit bitmap 中对应的位为 1 而导致 VM-exit。

(16) 在“virtualize APIC accesses”为 1 的前提下。当“use TPR shadow”为 0 时，尝试线性读或写 virtual-APIC page 内的值将引发 VM-exit（见 3.5.2 节和 3.5.9 节）。当“use TPR shadow”为 1，并且“APIC-register virtualization”为 0 时，读 virtual-APIC page 内除 VTPR 外其他的寄存器，将引发 VM-exit。当“use TPR shadow”为 1，并且“APIC-register virtualization”也为 1 时，读除了 virtual-APIC page 内的 APIC ID，APIC Version，VTPR，VEOI，VLDR，VDFR，VSVR，VISR，VTMR，VIRR，VESR，VICR，LVT，Timer initial count 及 Timer divide configuration 寄存器外其他的寄存器，将产生 VM-exit。

(17) 在“virtualize APIC accesses”为 1，并且“use TPR shadow”也为 1 的前提下。当“APIC-register virtualization”和“virtual-interrupt delivery”都为 0 时，写 virtual-APIC page 内除 VPTR 外其他的寄存器，将引发 VM-exit。当“virtual-interrupt delivery”为 1

时，写 virtual-APIC page 内除 VPTR，VEOI 或者 VICR-low 外其他寄存器将产生 VM-exit。当“APIC-register virtualization”为 1 时，写 virtual-APIC page 内除了 VPTR，VEOI 或者 VICR-low 外其他的寄存器，将产生 APIC-write VM-exit。

(18) 当“enable EPT”为 1 时，违例（指无权限）访问 guest-physical address 将引发 VM-exit。这种造成 VM-exit 的原因被称为 **EPT violation**。

(19) 当“enable EPT”为 1 时，访问 guest-physical address，由于 EPT 页表结构中的表项配置不当而引发 VM-exit。这种造成 VM-exit 的原因被称为 **EPT misconfiguration**。

由 VM-entry 失败而导致的 VM-exit

除了前面所述的在 VMX non-root operation 内引发 VM-exit 的条件外，在 VM-entry 失败时也可能引发 VM-exit。

- 在检查 guest-state 区域的字段时，由于无效的 guest-state 字段而导致 VM-exit。
- 在加载 guest-state 区域的 MSR 时失败而导致 VM-exit。
- 在 VM-entry 期间可能由于遇到 machine-check 事件而失败导致 VM-exit。

3.10.1.3 Exit qualification 字段

exit qualification 字段属于 natural-width 类型（在 64 位架构处理器上是 64 位，否则为 32 位）。在 VM-exit 时，这个字段记录由于下面的原因导致 VM-exit 的明细信息：

(1) 由某些指令而引发的 VM-exit，包括：INVEPT，INVLPG，INVPCID，INVVPID，LGDT，SGDT，LIDT，SIDT，LLDT，SLDT，LTR，STR，VMCLEAR，VMPTRLD，VMPTRST，VMREAD，VMWRITE，VMXON，MWAIT 指令。

- (2) 由#DB 异常而引发的 VM-exit。
- (3) 由#PF 异常而引发的 VM-exit。
- (4) 由于接收到 SIPI 消息而引发的 VM-exit。
- (5) 由于接收到 SMI 请求而引发的 VM-exit。
- (6) 由于进行任务切换而引发的 VM-exit。
- (7) 由访问控制寄存器而引发的 VM-exit。
- (8) 由于执行 MOV-DR 指令而引发的 VM-exit。
- (9) 由于访问 I/O 地址而引发的 VM-exit。
- (10) 由于访问 APIC-access page 而引发的 VM-exit。
- (11) 由于 EPT violation 而引发的 VM-exit。
- (12) 由 EOI 虚拟化而导致的 VM-exit。
- (13) 由 APIC-write VM-exit 而导致的 VM-exit。

对于其他原因导致的 VM-exit, exit qualification 字段将清为 0 值。根据不同的 VM-exit 原因, exit qualification 字段记录的信息格式也不同。下面分别描述上面介绍的 13 种 VM-exit 明细信息。

3.10.1.4 由某些指令引发的 VM-exit

exit qualification 字段只记录部分指令产生 VM-exit 的明细信息, 这些指令包括: INVEPT, INVLPG, INVPCID, INVVPID, LGDT, SGDT, LIDT, SIDT, LLDT, SLDT, LTR, STR, VMCLEAR, VMPTRLD, VMPTRST, VMREAD, VMWRITE, VMXON, MWAIT 指令。

对于 INVLPG 指令, exit qualification 字段保存操作数的**线性地址值**, 如以下代码所示:

```
invlpg [rax] ; 刷新 rax 指向地址所在页的 TLB 和 paging-structure cache
```

这条指令的目的是刷新当前 PCID 下由 RAX 所指向地址所在页的 TLB 以及 paging-structure cache 信息。在 guest 里尝试执行它产生 VM-exit 时, exit qualification 字段记录 rax 的值 (线性地址)。

对于 MWAIT 指令来说, exit qualification 字段的值为 0 或者 1, 用来指示地址监控是否准备就绪。

- 值为 0 时, 指示地址监控未准备好。
- 值为 1 时, 指示地址监控已经准备就绪。

对于其他指令, exit qualification 字段记录指令**内存操作数中的偏移量** (displacement), 如果不存在偏移量, 则 exit qualification 字段被清 0。

```
invept rax, [rbx + 0Ch]
```

如上面这条指令, 在 guest 中尝试执行它将产生 VM-exit, 在 exit qualification 字段中保存的值是 0CH。如果内存操作数是[rbx], 则 exit qualification 字段的值为 0。

在 x64 体系中支持 **RIP-relative** 寻址, 当在 64 位模式下使用这种寻址时, exit qualification 字段保存 **RIP 加上偏移量**后的值。

```
invept rax, [rip + 0Ch]
```

在上面这条指令中, 如果 RIP 的值是 400000H, 那么 exit qualification 字段的值是 40000CH。

注意: 在指令内存操作数寻址里, address-size 可能会被 address-size override prefix 改写。例如, 在 64-bit 模式下, 使用 67H prefix 改写为 32 位 address-size。那么会在 VM-exit instruction information 字段中记录实际的 address-size。

3.10.1.5 由#DB 异常引发的 VM-exit

如果 VM-exit 由 #DB 异常而引发 (exception bitmap 字段 bit 1 为 1), exit qualification 字段保存的信息格式如表 3-25 所示。

表 3-25

位 域	名 字	描 述
3:0	B3~B0	对应于断点 3~断点 0。为 1 时，表示遇到了某个断点
12:4	保留	为 0
13	BD	为 1 时，表示#DB 异常由“访问 DR 寄存器”而引发
14	BS	为 1 时，表示遇到了单步调试
63:15	保留	为 0

注意：exit qualification 字段记录的 debug 异常信息与 pending debug exception 字段记录的 debug 异常信息是不同的。它们的用途也不同：

- exit qualification 字段记录的是由#DB 异常引发 VM-exit 时的 debug 异常信息。
- pending debug exception 字段记录的是由其他原因导致 VM-exit，而#DB 异常未处理而被 pending（悬挂）的信息。

在 3.8.7.1 节的描述里，执行断点的#DB 异常和访问 DR 寄存器的#DB 异常是会产生悬挂的。它们要么得到执行，要么产生 VM-exit 行为。当它们由于 exception bitmap 字段 bit 1 为 1 而产生 VM-exit 时，可以在 exit qualification 字段记录#DB 异常的相关信息。

因此，在 exit qualification 字段中存在 BD 位（bit 13），当 BD 为 1 时，表示#DB 异常是由访问 DR 寄存器而引发（前提是 DR7.GD=1）。注意，由“MOV-DR exiting”而引发 VM-exit 的这种情形并不属于这里描述的情形。它们的 VM-exit 引发原因不同，exit qualification 字段记录的内容也不同。

当 B3~B0 其中一位为 1 时，表示遇到了断点 3 到断点 0 其中的一个。断点的地址在相应的 DR3~DR0 寄存器设置。

当 BS 为 1 时，表示遇到的#DB 异常是单步调试。单步调试有如下两种模式：

- 基于指令的单步调试，eflags.TF=1 并且 IA32_DEBUGCTL.BTF=0。
- 基于跳转分支的单步调试，eflags.TF=1 并且 IA32_DEBUGCTL.BTF=1。

3.10.1.6 由#PF 异常引发的 VM-exit

由#PF 异常而引发 VM-exit，exit qualification 字段将记录产生#PF 异常的线性地址值。这个值等于 CR2 寄存器的值。

3.10.1.7 由 SIPI 引发的 VM-exit

当虚拟处理器处于“wait-for-SIPI”状态，接收到 SIPI 消息而产生 VM-exit 时，exit qualification 字段将记录这个 SIPI 消息所使用的向量号（字段的 bits 7:0 为向量号，bits 63:8 为 0）。

3.10.1.8 由 I/O SMI 引发的 VM-exit

当 VMX 启用 SMM 双重监控处理功能后，处理器接收到 SMI 将引发 SMM VM-exit，或者在 VMX root-operation 中执行 VMCALL 指令也可以产生 SMM VM-exit。

在 exit qualification 字段中仅记录 I/O SMI 形式的明细信息。I/O SMI 是指在执行完 I/O 指令后紧接着收到 SMI 而导致 SMM VM-exit。

在 64 位架构处理器下，exit qualification 字段为 64 位宽（否则为 32 位宽），那么 bits 63:32 是保留位须为 0 值。如表 3-26 所示，bits 2:0 记录 I/O 指令的操作数大小：

- 为 0 时，是 1 字节，例如：in al, 92h。
- 为 1 时，是 2 字节，例如：in ax, 92h。
- 为 3 时，是 4 字节，例如：in eax, 92h。

bit 3 指示 I/O 指令访问的方向，为 1 是 IN，为 0 属于 OUT。当 bit 4 为 1 时，表明是 INS/OUTS 类 I/O 串操作指令。bit 5 为 1 时，表明在一个串操作指令里存在 REP 前缀。bit 6 指示 I/O 端口的寻址，为 1 时，表明使用立即数寻址；为 0 时，表明使用 DX 寄存器寻址。

最后，位域 bits 31:16 为 16 位宽，可以完整地记录 x86/x64 体系中的 16 位 I/O 端口地址。如果属于 other SMI 和 VMCALL 指令发起的 SMM VM-exit，此时 exit qualification 字段为 0 值。

表 3-26

位 域	名 字	描 述
2:0	size	I/O 指令的 operand size
3	direction	为 1 时，是 IN 操作，为 0 是 OUT 操作
4	string	为 1 时，是字符串操作
5	REP prefix	为 1 时，存在 REP prefix
6	immed address	为 1 时，I/O 端口是立即数，否则为 DX 寄存器
15:7	保留	为 0
31:16	I/O port	记录 I/O 端口地址值

3.10.1.9 由任务切换引发的 VM-exit

在 guest 中以任何形式尝试执行 task switch（任务切换）操作，都会引发 VM-exit。exit qualification 字段记录信息的格式如表 3-27 所示。

表 3-27

位 域	名 字	描 述
15:0	selector	记录 TSS selector
29:16	保留	为 0
31:30	source	任务切换的发起源：
		0 - call 指令
		1 - iret 指令
		2 - jmp 指令
		3 - 由访问 task gate 发起

字段的 bits 15:0 记录任务切换中使用的 TSS selector 值。bits 31:30 的值指示由什么发起任务切换操作，分为 4 种情况：为 0 时由 CALL 指令发起，为 1 时由 IRET 指令发起，为 2 时由 JMP 指令发起，为 3 时由访问 IDT 表中的 task gate（任务门）而发起。

在第 4 种情况下，可以使用 INT 指令发起或者由发生的一个中断 / 异常（或注入事件）发起。中断或异常向量号对应在 IDT 表内的描述符属于 task gate 描述符。

3.10.1.10 访问控制寄存器引发的 VM-exit

访问控制寄存器除了使用 MOV-CR 类指令，还包括 CLTS 和 LMSW 指令。因此，exit qualification 字段记录的信息需要区分这些情况，如表 3-28 所示。

表 3-28

位 域	名 字	描 述
3:0	CRn	记录访问的控制寄存器
5:4	Access type	访问类型
6	LMSW operand	LMSW 指令的操作数类型
7	保留	为 0
11:8	GPR	记录使用的通用寄存器
15:12	保留	为 0
31:16	LMSW source	LMSW 指令的源操作数值

字段的 bits 3:0 记录指令访问的控制寄存器，值从 0 到 15，分别对应 CR0 到 CR15 寄存器。在不支持 x64 的体系中，只有 CR0 到 CR7 寄存器。对于 CLTS 和 LMSW 指令来说，bits 3:0 固定为 0 值，也就是固定为 CR0 寄存器。

bits 5:4 指示指令访问的类型如下：

- 为 0 时，是 MOV to CR，例如：mov cr0, eax。
- 为 1 时，是 MOV from CR，例如：mov eax, cr0。
- 为 2 时，是执行 CLTS 指令。
- 为 3 时，是执行 LMSW 指令。

bit 6 仅用于 LMSW 指令上，指示 LMSW 指令的操作数类型。为 0 时，表明是寄存器操作数。为 1 时，表明是内存操作数。对于 CLTS 和 MOV-CR 指令来说，这个位为 0 值。

bits 11:8 指示 MOV-CR 指令中使用的 GPR（通用寄存器）。共 16 个寄存器，值从 0 到 7 分别对应 EAX（RAX），ECX（RCX），EDX（RDX），EBX（RBX），ESP（RSP），EBP（RBP），ESI（RSI），EDI（RSI）。在 x64 平台上，值从 8 到 15 分别对应 R8 到 R15 寄存器。对于 CLTS 和 LMSW 指令来说，这个值为 0（即 EAX 或 RAX 寄存器）。

bits 31:16 仅用于 LMSW 指令上，记录 LMSW 指令源操作数的值。例如：lmsw ax，那么 bits 31:16 记录 AX 寄存器的值。

3.10.1.11 由 MOV-DR 指令引发的 VM-exit

访问调试寄存器只能使用 MOV 指令，相对于访问控制寄存器更简单。exit qualification 字段记录的格式如表 3-29 所示。

表 3-29

位 域	名 字	描 述
2:0	DR n	记录访问的调试寄存器
3	保留	为 0
4	direction	访问的方向：
		0 - MOV to DR
		1 - MOV from DR
7:5	保留	为 0
11:8	GPR	记录使用的通用寄存器
31:12	保留	为 0

字段的 bits 2:0 共 3 位。值从 0 到 7，分别对应 DR0 到 DR7。bit 4 指示指令访问的方向，为 0 时表明写 DR 寄存器，为 1 时表明读 DR 寄存器。

bits 11:8 记录指令使用的通用寄存器。共 16 个寄存器，值从 0 到 7 分别对应 EAX (RAX)，ECX (RCX)，EDX (RDX)，EBX (RBX)，ESP (RSP)，EBP (RBP)，ESI (RSI)，EDI (RSI)。在 x64 平台上，值从 8 到 15 分别对应 R8 到 R15 寄存器。

3.10.1.12 由 I/O 指令引发的 VM-exit

I/O 指令有两类，分为串 (string) 指令 INS/OUTS 系列和非串指令 IN/OUT。INS/OUTS 系列指令还可以使用 REP 前缀来实现重复执行 I/O 指令。如表 3-30 所示。

表 3-30

位 域	名 字	描 述
2:0	size	I/O 指令 operand size
3	direction	访问方向：0 - OUT, 1 - IN
4	string	为 1 时，属于串指令
5	REP	为 1 时，存在 REP prefix
6	Imme address	为 1 时，使用立即数，否则使用 DX 寄存器
15:7	保留	为 0
31:16	port	记录 I/O 端口地址

exit qualification 字段的 bits 2:0 记录 I/O 指令访问的数据大小：

- 为 0 时，1 个字节，例如：in al, 92h。
- 为 1 时，2 个字节，例如：in ax, 92h。
- 为 3 时，4 个字节，例如：in eax, 92h。

bit 3 指示 I/O 指令的访问方向，为 0 时是输出（OUT），为 1 时是输入（IN）。bit 4 指示 I/O 指令是否是串指令，为 1 时属于串指令。bit 5 指示 I/O 指令是否存在 REP 前缀，为 1 时表明使用了 REP 前缀。bit 6 指示 I/O 端口地址寻址方式，为 1 时使用立即数寻址，为 0 时使用 DX 寄存器寻址。bits 31:16 记录 I/O 端口地址值。

3.10.1.13 由于访问 APIC-access page 引发的 VM-exit

访问 APIC-access page 引发的 VM-exit 可以是来自 guest 软件的线性访问或者 guest-physical 访问（参见 7.2.3 节）。而 guest-physical 访问发生在事件的 delivery 期间，或者某条指令执行过程中。如表 3-31 所示。

表 3-31

位 域	名 字	描 述
11:0	offset	在线性访问中，APIC-access page 内的偏移值
15:12	type	访问的类型：
		0 - 线性读访问
		1 - 线性写访问
		2 - 线性执行访问
		3 - 在事件的 delivery 过程中，发生了线性访问
		10 - 在事件的 delivery 过程中，发生了 guest-physical 访问
		15 - guest-physical 访问
31:16	保留	为 0

exit qualification 字段的 bits 11:0 记录在线性访问中所访问的 APIC-access page 4K 页面内的偏移值，如以下代码所示：

```
mov eax, [APIC_BASE + LAPIC_TPR] ; 读 TPR 值
```

如果这条指令发生了线性访问 APIC-access page，那么 exit qualification 字段的 bits 11:0 值就是 80H，也就是 TPR 寄存器在 APIC-access page 内的偏移量。

bits 15:12 的值指示 APIC-access page 访问的类型，表 3-31 中列出了 6 种可能的 APIC-access page 访问类型。其中线性访问单独列出三种常规访问类型：读访问（值为 0），写访问（值为 1）及执行访问（值为 2）。而 guest-physical 访问将三种访问类型合并在一起（值为 15）。

产生读写访问的指令除了常用的 MOV 指令外，也包括所有含有内存操作数的指令。比如下面所说的指令：

(1) CLFLUSH 指令，执行刷新处理器的 cache line 操作。例如：**clflush [eax]**。在这里 CFLUSH 指令刷新内存地址 eax 所在的 cache line 数据，这属于读访问。

(2) ENTER 指令，执行开辟一块栈空间作为局部栈帧。例如：**enter 32, 0**。在这里 ENTER 指令将写入原 ebp 值到栈中。因此，属于写访问。

(3) MASKMOVQ 指令，执行根据 mask 位来选择写入 DS:EDI 指向的内存。它属于写访问。

(4) MONITOR 指令，执行对目标地址进入准备监控操作。它属于读访问。

在事件的 delivery 期间也可能会发生对 APIC-access page 的线性访问或者 guest-physical 访问，这个事件指**异常或中断**（也包括 **NMI**，以及**注入事件**），它们的服务例程被提交到处理器执行。

如果是由于 **physical** 访问（注意区别于 **guest-physical** 访问）引起的 VM-exit，exit qualification 字段的值是 undefined 的，也就是无意义的数。

3.10.1.14 由 EPT violation 引发的 VM-exit

EPT violation 是指不允许访问或者没权限访问 guest-physical address，类似在保护模式下线性地址通过页转换表结构的转换过程中，软件对线性地址的访问被拒绝。exit qualification 字段记录的格式如表 3-32 所示。

表 3-32

位 域	名 字	描 述
0	read	为 1 时，指示对 GPA 进行读访问
1	write	为 1 时，指示对 GPA 进行写访问
2	execute	为 1 时，指示对 GPA 进行执行访问
3	readable	为 1 时，表明 GPA 具有可读权限
4	writable	为 1 时，表明 GPA 具有可写权限
5	executable	为 1 时，表明 GPA 具有可执行权限
6	保留	为 0
7	valid	为 1 时，表明存在一个 guest-linear address
8	translation	为 1 时，表明 EPT violation 发生在 GPA 转换 HPA 的过程。
		为 0 时，表明 EPT violation 发生在对 guest paging-structure 表项访问的环节上
11:9	保留	为 0
12	NMI unblocking	为 1 时，表明执行了 IRET 指令，并且 NMI 阻塞已经被解除
31:13	保留	为 0

bits 2:0 指示产生 EPT violation 时对 GPA (guest-physical address) 进行哪些访问：

- bit 0 为 1 时，当时正在对 GPA 进行读访问。
- bit 1 为 1 时，当时正在对 GPA 进行写访问。
- bit 2 为 1 时，表明正在尝试执行 GPA 地址上的指令（fetch 指令）。

通常对 GPA 的访问来自于 guest 通过 guest-linear address 访问内存。譬如，guest 对一个 guest-linear address 进行读访问，而这个 guest-linear address 将转换为 guest-physical

address, 从而转变为对这个 guest-physical address 的读访问。

在启用 EPT 页表项的 accessed 与 dirty 标志位后, 对 guest paging-structure 表项的访问都被视为“写访问”。当在 guest paging-structure 表项的访问时发生 EPT violation, 此时 exit qualification 字段的 bit 0 与 bit 1 都被置位。

在进行 read-modify-write 类操作时发生 EPT violation (譬如执行“INC DWORD [eax]”这类指令), 此时 bit 0 与 bit 1 可能都被置位 (取决于处理器架构的实现)。

bits 5:3 指示访问的目标 GPA 具有什么权限:

- bit 3 为 1 时, 表明 GPA 具有可读权限。
- bit 4 为 1 时, 表明 GPA 具有可写权限。
- bit 5 为 1 时, 表明 GPA 具有可执行权限。

它们的值来自于各级 EPT paging-structure 表项相应权限位 AND 操作的结果 (即各级页表项的 bits 2:0 位“AND”后的结果)。如果任何一级页表项出现 not present 现象, bits 5:3 的值为 0。

当 bit 7 为 1 时, 表明存在一个 guest-linear address, 引起 EPT violation 的起源来自于对一个 guest-linear address 的访问 (也就是说, 对这个 guest-linear address 访问最终产生的结果是 EPT violation)。

注意: bit 7 为 1, 并不代表 guest-linear address 可以成功转换为 guest-physical address。因为在 guest-linear address 转换 guest-physical address 的过程中, 访问任何一级 guest paging-structure 表项时都可能会产生 EPT violation 故障。

当 bit 7 为 0 时, 表明引起 EPT violation 的起源并不是对 guest-linear address 的访问。一个典型的例子是: 在使用 PAE 分页模式时, 执行 MOV to CR3 指令会引起加载 PDPTE, 最终导致 EPT violation。由于 CR3 寄存器存放的是 guest-physical address, 因此属于直接访问 guest-physical address, 并不存在对 guest-linear address 的访问。

当 bit 7 为 0 时, bit 8 属于保留位。因此, 在 bit 7 为 1 的前提下, bit 8 有下面的意义:

(1) bit 8 为 1 时, 表明 guest-linear address 成功转换为 guest-physical address。EPT violation 来自于将这个 guest-physical address 转换到 host-physical address 阶段。

(2) bit 8 为 0 时, 表明 guest-linear address 转换 guest-physical address 的过程失败。EPT violation 来自于 guest paging-structure 页表项访问环节上 (也就是在 guest paging-structure 各级页表的 walk 过程中, 参见 6.1.1.1 节及图 6-2)。

因此, 结合 bit 7 与 bit 8 位可以帮助 VMM 判断 EPT violation 发生在哪个环节上。

“NMI unblocking”位 (bit 12) 被认为可能在 NMI 或 virtual-NMI 的服务例程内执行 IRET 指令才有意义。也就是下面的情形:

(1) “NMI exiting”为 0 时, 这时候 IRET 指令被认为可能在 NMI 服务例程内执行。

(2) “NMI exiting”与“virtual-NMIs”都为 1 时，这时候 IRET 指令被认为可能在 virtual-NMI 服务例程内执行。

bit 12 在下面的情形下属于未定义。

(1) “NMI exiting”为 1，并且“virtual-NMIs”为 0 时。

(2) IDT-vectoring information 字段的 bit 31 为 1 时，也就是由向量事件间接导致 VM-exit。

IRET 指令的执行会对“blocking by NMI”或者“blocking by virtual-NMI”阻塞状态进行解除，即使 IRET 指令引起了 fault，EPT violation 或 EPT misconfiguration 等，也无碍它对 NMI 和 virtual-NMI 阻塞状态的解除。

如果 IRET 指令的执行引起了异常，而这个异常在 delivery 期间又发生了另一个异常，那么，这时“NMI unblocking”位也属于 undefined（未定义）值（参见 3.10.2.1 节）。

因此，当 bit 12 为 1 时，实际上表明 EPT violation 是由于执行 IRET 指令而引起的。例如，IRET 指令所引用的 stack 空间无效。也就是说，ESP 指向的线性地址在转换物理地址时失败，发生了 EPT violation 故障。

3.10.1.15 由 EOI 虚拟化引发的 VM-exit

当“virtual-interrupt delivery”为 1 时，对 APIC-access page 内的 VEOI（偏移值为 B0H）进行写操作将引发 EOI 虚拟化。

```
mov DWORD [ApicAccessPage + EOI], 0 ; 将引发 EOI 虚拟化
```

上面的代码假设在一个虚拟中断服务例程中执行发送 EOI 命令，它访问了 VEOI 而引发 EOI 虚拟化操作。如果中断服务例程的向量号在“EOI-exit bitmap”字段中对应的位为 1，将引发 VM-exit（见 3.5.12 节）。

此时，exit qualification 字段的 bits 7:0 将记录这个中断的向量号，而其余位（即 bits 63:8）被清 0。

3.10.1.16 由 APIC-write 引发的 VM-exit

在“APIC-register virtualization”为 1 的前提下，对 APIC-access page 内的虚拟 local APIC 寄存器进行写访问时，除 VPTR，VEOI 以及 VICR-low 寄存器外，其余的所有寄存器最终都将产生 APIC-write VM-exit。例如，向 VESR（virtual-ESR）寄存器进行写，在写完 VESR 后产生 VM-exit。

此时，exit qualification 字段的 bits 11:0 将记录虚拟 local APIC 寄存器 4K 页面内的偏移值，其余位被清 0。

3.10.1.17 guest-linear address 字段

guest-linear address 字段属于 natural-width 类型，在 64 位架构处理器上为 64 位宽，否则为 32 位宽。在一些 VM-exit 情形下，这个字段记录了线性地址值。

(1) 由 LMSW 指令引发的 VM-exit，如果它的操作数是内存，guest-linear address 字段记录内存操作数的线性地址值。

(2) 由 INS 或 OUTS 系列串指令引发的 VM-exit，guest-linear address 字段将记录目标地址 **ES:EDI** 或源地址 **DS:ESI** 的线性地址值。

(3) 由 EPT violation 引发的 VM-exit，当 exit qualification 字段的 bit 7 为 1 时（表明线性地址有效），guest-linear address 字段将记录这个线性地址值。

对于其他情况，guest-linear address 字段是未定义值。

3.10.1.18 guest-physical address 字段

如果 VM-exit 是由于 EPT violation 或 EPT misconfiguration 引起的，64 位的 guest-physical address 字段将记录这个引起 EPT violation 或 EPT misconfiguration 的 guest-physical address 值。对于其他情况，这个字段是未定义值。

3.10.2 直接向量事件类信息字段

VM-exit 的直接向量事件信息类字段有两个：VM-exit interruption information 和 VM-exit interruption error code 字段。

所谓的“**直接向量事件**”是指**直接引发 VM-exit** 的向量事件。包括下面 4 类。

(1) 硬件异常：由于异常的向量号在 exception bitmap 对应的位为 1 而直接导致 VM-exit。

(2) 软件异常（#BP 与 #OF）：由于异常的向量号在 exception bitmap 对应的位为 1 而直接导致 VM-exit。

(3) 外部中断：发生外部中断请求时，由于“external-interrupt exiting”为 1 而直接导致 VM-exit。

(4) NMI：发生 NMI 请求时，由于“NMI exiting”为 1 而直接导致 VM-exit。

直接向量事件不包括软件中断及特权级软件中断（但它们可以属于间接向量事件，见 3.10.3 节）。

3.10.2.1 VM-exit interruption information 字段

在 guest 中当发生异常或者外部中断请求时，对于异常（包括 INT3、INTO、BOUND 及 UD2 指令产生的软/硬件异常），如果向量号在 exception bitmap 对应的位为 1，将产生 VM-exit。对于外部中断，如果“external interrupt exiting”为 1，将产生 VM-exit。对于 NMI，如果“NMI exiting”为 1，将产生 VM-exit。这些向量事件能直接引发 VM-exit。

32 位宽的 VM-exit interruption information 字段记录向量事件的 delivery 信息，格式如表 3-33 所示。

表 3-33

位 域	名 字	描 述
7:0	vector	记录异常或中断的向量号
10:8	type	中断类型:
		0 - 外部中断
		1 - 保留
		2 - NMI
		3 - 硬件异常
		4 - 保留
		5 - 保留
		6 - 软件异常
		7 - 保留
11	error code	为 1 时, 有错误码
12	NMI unblocking	为 1 时, 表示 “blocking by NMI” 被解除
30:13	保留	为 0
31	valid	为 1 时, VM-exit interruption information 字段有效

如果是由于外部中断而引发的 VM-exit, 在 VM-exit control 字段的 “acknowledge interrupt on exit” 为 1 时, 在 VM-exit 时处理器会发送响应周期给中断控制器, 并且从中断控制器读取中断的 delivery 信息 (见 3.7.1 节)。否则, VM-exit interruption information 字段不会记录外部中断的 delivery 信息。

如表 3-33 所示, 字段的 bits 7:0 记录异常或中断的向量号。Bits 10:8 指示中断的类型, 在 VM-exit interruption information 字段中仅支持 4 种类型: **外部中断**、**NMI**、**硬件异常**及**软件异常**。这是因为在 guest 中不会由于特权级软件中断及软件中断而直接引发 VM-exit。因此, 这个字段的中断类型并不包括特权级软件中断和软件中断 (参见 3.6.3 节)。

当 bit 11 为 1 时, 表示存在错误码。只有以下几个硬件异常事件中才会有错误码存在, 它们是: #DF 异常 (向量号为 8)、#TS 异常 (向量号为 10)、#NP 异常 (向量号为 11)、#SS 异常 (向量号为 12)、#GP 异常 (向量号为 13)、#PF 异常 (向量号为 14) 和 #AC 异常 (向量号为 17)。外部中断、NMI 及软件异常并不存在错误码。

如果执行 IRET 指令时引发了一个异常而导致 VM-exit (**直接或间接**), 在 **VM-exit 开始前**, 处理器的 “blocking by NMI” 或者 “blocking by virtual-NMI” 阻塞状态已经被解除。

但是, **只有在异常直接导致 VM-exit 发生时**, bit 12 才被置位, 表示 IRET 指令执行之前可能存在 “blocking by NMI” 或者 “blocking by virtual-NMI” 阻塞状态, 而这个阻塞状态已经被解除 (由于 IRET 指令而解除)。

bit 12 在下面的情形中是**未定义值**:

- Pin-based VM-execution control 字段的 “NMI exiting” 为 1, 并且 “virtual NMIs”

为 0 时。表示“一个 NMI 已经产生了 VM-exit”，因此，IRET 指令对“blocking by NMI”阻塞状态的解除不产生影响。

- 执行 IRET 指令时由于**间接向量事件**而导致 VM-exit（参见 3.10.3 节），即执行 IRET 指令产生一个异常，但这个异常不直接引发 VM-exit，而在这个异常 delivery 期间引发另一个异常而导致 VM-exit 发生。此时，对“NMI unblocking”位无影响。也就是在 IDT-vectoring information 字段的 **bit 31 为 1** 时，bit 12 为未定义。
- 同上，由于执行 IRET 指令产生连串异常，转变为 #DF 异常导致 VM-exit。此时，对“NMI unblocking”位无影响。也就是 VM-exit interruption information 字段的 bits 7:0 为 8（指示 #DF 异常）。

也就是 IRET 指令被认为在 NMI handler（或者 virtual-NMI handler）内执行，才会对“blocking by NMI”（或者“blocking by virtual-NMI”）阻塞状态进行解除。如果 NMI 直接引发 VM-exit，NMI 不被 deliver 执行。

另外，如果 IRET 指令产生了一个异常，但这个异常并不直接引发 VM-exit，此时**“blocking by NMI”阻塞状态已经被解除**。而后来由于异常 delivery 期间导致 VM-exit（间接引发），bit 12 已经没有任何意义，它属于未定义值。也就是 IDT-vectoring information 字段的 bit 31 为 1 时，这个“NMI unblocking”位没有意义。

在前面 3.10.1.14 节的“EPT violation 引发 VM-exit”的描述里，“NMI unblocking”位仅在 IRET 指令可能被认为在 NMI 服务例程里执行才有意义，其他情况下属于未定义值。

注意：VM-exit interruption information 字段记录直接产生 VM-exit 的向量事件。对于其他原因产生的 VM-exit 事件（其中也包括“acknowledge interrupt on exit”位为 0 时的情况），VM-exit interruption information 字段的 **bit 31 为 0**，其余位为未定义（undefined）。bit 31 为 1 时，VM-exit interruption information 字段才有效。当 bit 31 为 0 时，这个字段中的所有值是未定义值。

3.10.2.2 VM-exit interruption error code 字段

如果 VM-exit 由硬件异常引发，当 VM-exit interruption information 字段的 bit 11 为 1 时，VM-exit interruption error code 字段记录异常的错误码。错误码仅存在于前面所述的 7 种硬件异常里，包括：#DF，#TS，#NP，#SS，#GP，#PF 及 #AC 异常。其他类型的事件这个字段清为 0 值。

3.10.3 间接向量事件类信息字段

记录间接导致 VM-exit 向量事件的字段有两个：IDT-vectoring information 与 IDT-vectoring error code 字段。

所谓的“**间接向量事件**”是指：当一个向量事件发生后它不直接产生 VM-exit，在通过 guest-IDT delivery 期间由于遇到了某些错误而导致 VM-exit。

因此，这个向量事件间接导致 VM-exit 发生。间接导致 VM-exit 的向量事件可以

为：硬件异常、软件异常、软件中断、特权级软件中断、外部中断及 NMI。

例如，在 guest 内执行一个软件中断调用，像下面的示例：

```
int 60h ; 执行软件中断调用
```

这个中断调用不会直接产生 VM-exit。如果处理器在 guest-IDT 查找 60h 的中断描述符时由于描述符类型不符而产生了 #GP 异常，而这个 #GP 异常导致 VM-exit 发生，那么，这种情况就属于“**VM-exit 由向量事件在 delivery 期间出错而引发**”，也就是间接导致 VM-exit。

下面的几种情况属于“VM-exit 由于向量事件 delivery 期间出错而引发”：

(1) 在向量事件 delivery 期间引发了一个异常，这个异常由于向量号在 exception bitmap 字段对应的位为 1 而导致 VM-exit（包括转变为 #DF 异常）。

(2) 在向量事件 delivery 期间引发连串异常，最终转变为 **triple fault** 而导致 VM-exit。

(3) 向量事件 delivery 期间由于向量号在 guest-IDT 内对应的描述符为 task-gate 描述符，尝试进行任务切换而导致 VM-exit。

(4) 向量事件 delivery 期间访问了 APIC-access page 页面（包括线性访问和 guest-physical address 访问）而导致 VM-exit（在启用“virtualize APIC-accesses”功能时）。

(5) 向量事件在 delivery 期间发生了 EPT violation 或者 EPT misconfiguration 而导致 VM-exit（在启用“enable EPT”功能时）。

3.10.3.1 IDT-vectoring information 字段

如果 VM-exit 是由于在一个向量事件 delivery 期间产生了一个异常而导致的，那么就会出现两个向量事件：一个**间接向量事件**（即原始的向量事件，例如前面代码中的那个软件中断调用），另一个是**直接向量事件**（即后面直接引发 VM-exit 的向量事件）。

如果 VM-exit 是由于一个向量事件 delivery 期间遇到任务切换、EPT violation、EPT misconfiguration 或者访问了 APIC-access page 页面而导致的，那么这种情况下只存在一个间接向量事件（原始事件）。

IDT-vectoring information 字段只记录间接向量事件（原始向量事件）的信息。如果存在直接向量事件（引发 VM-exit 的向量事件），则由前面 3.10.2.1 节描述的 VM-exit interruption information 字段来记录。

表 3-34 是 IDT-vectoring information 字段的信息格式，它和 VM-exit interruption information 字段极为相似。正如前面所举的软件中断调用的例子，原始向量事件可以是**软件中断**或者**注入的事件**。因此，这个字段的中断类型也可以为软件中断或者特权级软件异常（由事件注入来实现）。

表 3-34

位 域	名 字	描 述
7:0	vector	记录异常或中断的向量号
10:8	type	中断类型:
		0 - 外部中断
		1 - 保留
		2 - NMI
		3 - 硬件异常
		4 - 软件中断
		5 - 特权级软件异常
		6 - 软件异常
		7 - 保留
11	error code	为 1 时, 有错误码
12	undefined	未定义位
30:13	保留	为 0
31	valid	为 1 时, IDT-vectoring information 字段有效

bit 12 是未定义位, 它是无意义的。由于原始向量事件没有成功 deliver 执行, 不可能产生 NMI 阻塞状态的解除动作, 也就没有“NMI unblocking”位存在的意义。

同样, bit 11 为 1 时, 表明原始向量事件存在错误码, 同样也只有在前面 3.10.2.1 节所说的 7 种硬件异常事件下才会产生错误码。

3.10.3.2 IDT-vectoring error code 字段

这个字段用来记录原始事件的错误码, 它的作用和 VM-exit interruption error code 字段是一样的, 所不同的是记录的向量事件对象不同。

3.10.4 指令类信息字段

这类字段有 6 个, 包括: VM-exit instruction length 和 VM-exit instruction information 字段, 以及由 I/O SMI 引起 VM-exit 所使用的 I/O RCX、I/O RSI、I/O RDI、I/O RIP 字段。

3.10.4.1 VM-exit instruction length 字段

在 guest 中产生 VM-exit 情形可以归纳为两大类: 由指令引起 VM-exit, 以及由事件引起 VM-exit。32 位宽的 VM-exit instruction length 字段记录引起 VM-exit 指令的长度, 指令长度在 1 到 15 之间。如果由指令产生的向量事件导致了 VM-exit (例如 INT3, INTO 指令), VM-exit instruction length 字段也能记录这个指令的长度。

注意: 如果软件异常, 软件中断或者特权级软件中断是通过事件注入方式 deliver, 它们间接导致了 VM-exit 发生, 那么, VM-exit instruction length 字段记录的指令长度等

于 VM-entry instruction length 字段的值。下面关于“某些向量事件引起的 VM-exit”的段落将阐述原因。

由指令引起的 VM-exit

在 3.10.1.2 节的描述里，能引起 VM-exit 的指令分为：**无条件**引发 VM-exit 的指令及**有条件地**引发 VM-exit 的指令。下面包括了无条件和有条件引发 VM-exit 的指令。

(1) 无条件指令（共 16 条）：CPUID，GETSEC，INVD，XSETBV 及所有 VMX 指令（除 VMFUNC 外）。但是，如果 guest 运行在实模式下，执行 VMX 指令（除 VMFUNC 外）将产生#UD 异常，而不是产生 VM-exit。

(2) 有条件指令（共 33 条）：HLT，INVLPG，INVPCID，RDPMC，RDTSC，RDTSCP，RSM，MOV from CR3，MOV to CR3，MOV from CR8，MOV to CR8，CLTS，LMSW，MOV to CR0，MOV to CR4，MOV-DR，IN/OUT，INS/OUTS，RDMSR，WRMSR，MWAIT，MONITOR，PAUSE，LGDT，LIDT，LLDT，LTR，SGDT，SIDT，SLDT，STR，RDRAND 及 WBINVD 指令。

上面的 49 条指令引发 VM-exit 时，VM-exit instruction length 字段将记录它们的指令长度。某些指令的操作数可以是寄存器，也可以是内存。因此，同一条指令可能存在由操作数不同而指令长度不同的情况。

上述指令企图在 guest 里访问系统资源，这在处理器虚拟化架构下是不允许或被限制的。VMX 架构下，这些指令的执行将产生 VM-exit 作为响应。

图 3-8 中，在 VM 里执行一条 CPUID 指令产生了 VM-exit，guest 中的 CPUID 的执行被 VMM 拦截，VMM 进行了虚拟化后，反射一个虚拟化的结果给 guest。

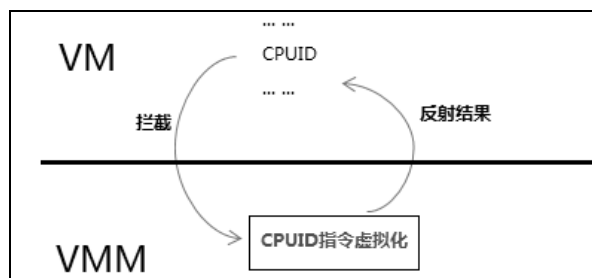


图 3-8

那么，VMM 重新进入 VM 后，guest 应该从哪里开始执行呢？肯定不是指令本身。guest 应该从引起 VM-exit 的指令的下一条指令开始执行。这就是 VM-exit instruction length 字段的作用。

如前面所述，所有指令引起的 VM-exit 都是 fault 类型。也就是说：guest-state 区域的 **guest-RIP** 字段是指向引起 VM-exit 的指令的地址。VM-exit instruction length 字段保存这条指令的长度，VMM 在重新进入 VM 前，根据 VM-exit instruction length 字段的值来计算 guest 下一条指令的地址，然后设置 guest-RIP 字段的值。

也就是说，新的 guest-RIP 字段的值应该等于原 guest-RIP 加上 VM-exit instruction length 字段的值。这样在重新进入 VM 后 guest 能够正确执行下去。

某些向量事件引起的 VM-exit

VM-exit instruction length 字段也记录部分由某些指令产生的向量事件，而这些向量事件引起了 VM-exit，包括：

(1) 由 INT3 或 INTO 指令产生的软件异常，属于**直接 VM-exit 向量事件**。

(2) 由软件异常（INT3 或 INTO 指令产生）、软件中断（INT 指令产生）或者特权级软件中断（通过注入事件）在 delivery 期间出现错误。它们属于**间接 VM-exit 向量事件**（参见 3.10.3 节），包括出现下面的错误：

- 在 delivery 期间引发了一个异常，这个异常由于向量号对应 exception bitmap 字段的位为 1 而导致 VM-exit（包括 #DF 异常）。
- 在 delivery 期间引发了连串异常，最终转变为 triple fault 而导致 VM-exit。
- 在 delivery 期间由于向量号在 guest-IDT 内对应的描述符为 task-gate 描述符，尝试进行任务切换而导致 VM-exit。
- 在 delivery 期间访问了 APIC-access page 页面（包括线性访问和 guest-physical address 访问）而导致 VM-exit。
- 在 delivery 期间发生了 EPT violation 或者 EPT misconfiguration 而导致 VM-exit。

VM-exit instruction length 字段记录这些指令长度。VMM 在下次重新进入 VM 时，需要注入这些向量事件来帮助 guest 完成处理，并且将 VM-exit instruction length 字段的值复制到 VM-entry instruction length 字段。

注意：在上面的第 (2) 点描述里。如果这些向量事件的 delivery 是通过**事件注入方式**实现（特权级软件中断必须通过事件注入来实现）的，有下面的描述：

- VMM 在注入事件前，必须在 VM-entry instruction length 字段里提供指令长度。
- 这些向量事件在 delivery 期间发生错误而引发 VM-exit 时，VM-exit instruction length 字段记录的值就等于 VM-entry instruction length 字段中的值。

关于 VM-entry instruction length 字段的设置参见 3.6.3.3 节描述。

任务切换引起的 VM-exit

VM-exit instruction length 字段记录包括下面的途径产生任务切换时的指令长度：

- 执行 CALL, JMP 及 IRET 指令尝试进行任务切换。
- 上面第 (2) 点描述的**软件中断或软件异常** delivery 期间（例如执行 INT, INT3, INTO 指令，或者事件注入方式）尝试进行任务切换。

exit qualification 字段也会记录这个任务切换的信息（见 3.10.1.9 节），其中 bits 31:30 位域指示任务切换的发起源。

由特权级软件中断 delivery 期间尝试任务切换引起 VM-exit 时，VM-exit instruction

length 字段记录的值等于 VM-entry instruction length 字段中的值。

由 VMFUNC 指令引起的 VM-exit

在“enable VM function”为 1 的前提下，执行 VMFUNC 指令时由于下面的原因而导致 VM-exit：

- (1) EAX 的值在 VM-function control 字段相应的位为 0。
- (2) EAX 的值为 0，并且 ECX 的值大于 511。

当前 VMX 架构下只实现了功能号 0（即“EPT switching”功能，参见 6.1.11 节），调用其他的功能号将产生 VM-exit。或者调用 0 号功能但 ECX 的值大于 511 时产生 VM-exit。VM-exit instruction length 字段将记录 VMFUNC 指令的长度。

由除前面所述之外的原因引起 VM-exit 时，VM-exit instruction length 字段的值属于未定义。

3.10.4.2 VM-exit instruction information 字段

如果 VM-exit 是由下面的指令引起的：INS，OUTS，LGDT，LIDT，LLDT，LTR，SGDT，SIDT，SLDT，STR，RDRAND，INVPID，INVEPT，INVVPID，VMCLEAR，VMPTRLD，VMPTRST，VMREAD，VMWRITE 以及 VMXON 指令，此时，VM-exit qualification 字段记录这些指令（除 INS/OUTS 和 RDRAND 指令外）内存操作数的偏移量（参见 3.10.1.4 节），32 位宽的 VM-exit instruction information 字段进一步补充指令的明细信息。只有结合这两个字段才能够完整地分析指令信息，给 VMM 提供管理帮助。

由 INS/OUTS 指令引发的 VM-exit

INS 和 OUTS 指令属于串操作指令。INS 指令默认使用 ES:EDI 作为目标 buffer 指针，OUTS 指令默认使用 DS:ESI 作为源 buffer 指针。可以通过 segment prefix（段前缀）方式来改变默认的数据段。

表 3-35

位 域	名 字	描 述
6:0	---	未定义
9:7	address size	记录地址大小：
		0 - 16 位
		1 - 32 位
		2 - 64 位
14:10	---	未定义
17:15	segment	记录所使用源或目标数据段：
17:15	segment	0 - ES
		1 - CS

续表

位 域	名 字	描 述
17:15	segment	2 - SS
		3 - DS
		4 - FS
		5 - GS
31:18	---	未定义

对于 INS/OUTS 指令来说，VM-exit qualification 字段会记录 I/O 指令的明细信息（如 3.10.1.12 节的表 3-30 所示）。VM-exit instruction information 字段会记录补充的额外信息。

如表 3-35 所示，它的 bits 9:7 指示源或目标 buffer 的地址大小，64 位的地址仅在 64 位模式下有效。bits 17:15 指示所使用的数据段。由于可以使用段前缀来修改默认的 ES 和 DS 段，因此，这个域必须给出明确的最终的数据段。其他的位域属于未定义，它的数据是无意义的。

由 INVEPT, INVPCID, INVVPID 指令引发的 VM-exit

这三条指令有两个操作数，第 1 个操作数是寄存器，第 2 个操作数是内存操作数。下面以 INVEPT 指令为例，如图 3-9 所示，这条指令寄存器操作数是 RAX。在内存操作数中使用了 FS 段前缀，base 寄存器是 RBX，index 寄存器是 RSI，scale 值为 8，偏移量为 0CH。

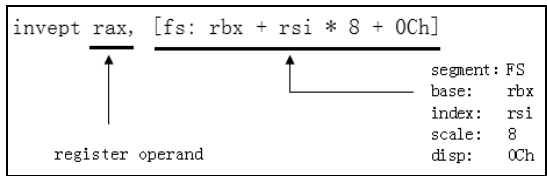


图 3-9

VM-exit qualification 字段会记录指令内存操作数中的偏移量（参见 3.10.1.4 节），也就是 0CH 值。而 VM-exit instruction information 字段记录的信息格式如表 3-36 所示。

表 3-36

位 域	名 字	描 述
1:0	scale	记录内存寻址中的 scale 值:
		0 - 没有 scale
		1 - scale 为 2
		2 - scale 为 4
		3 - scale 为 8
6:2	---	未定义
9:7	address size	记录内存地址大小:

续表

位 域	名 字	描 述
9:7	address size	0 - 16 位
		1 - 32 位
		2 - 64 位
10	保留	固定为 0
14:11	---	未定义
17:15	segment	记录段寄存器:
		0 - ES
		1 - CS
		2 - SS
		3 - DS
		4 - FS
		5 - GS
21:18	index	记录 index 寄存器
22	index invalid	为 1 时, 表示 index 寄存器无效, 0 为有效
26:23	base	记录 base 寄存器
27	base invalid	为 1 时, 表示 base 寄存器无效, 0 为有效
31:28	register operand	记录指令的寄存器操作数

字段的 bits 1:0 记录内存操作数寻址的 scale 值。bits 9:7 记录地址大小, 64 位的地址值仅在 64 位模式下有效。bits 17:15 记录段寄存器值, 在本例中段寄存器为 FS。

bits 21:18 记录 index 寄存器, 当 bit 22 为 1 时, bits 21:18 记录的 index 寄存器无效, 表明内存寻址中不存在 index 寄存器。bits 26:23 记录 base 寄存器, 当 bit 27 为 1 时, bits 26:23 记录的 base 寄存器无效, 表明内存寻址中不存在 base 寄存器。bits 31:28 记录指令所使用的寄存器操作数, 即第 1 个操作数。

bits 21:18, bits 26:23 及 bits 31:28 都记录寄存器编码值, 它们的格式是一样的。值从 0 到 7 分别对应 EAX (RAX), ECX (RCX), EDX (RDX), EBX (RBX), ESP (RSP), EBP (RBP), ESI (RSI) 以及 EDI (RDI) 寄存器。在 64 位模式下, 值从 8 到 15 分别对应 R8 到 R15 寄存器。

由 LGDT, LIDT, SGDT 及 SIDT 指令引发的 VM-exit

LGDT 和 SGDT 指令访问全局描述符表寄存器, LIDT 和 SIDT 指令访问中断描述符表寄存器。它们的操作数是内存操作数。因此 VM-exit instruction information 字段需要记录内存寻址信息。

表 3-37 的结构与表 3-36 相似, 字段的 bits 27:0 用来记录内存操作数寻址完整信息。由于这 4 条指令的操作数是一样的, 因此 bits 29:28 用来指示具体的指令。

表 3-37

位 域	名 字	描 述
1:0	scale	记录内存寻址中的 scale 值:
		0 - 没有 scale
		1 - scale 为 2
		2 - scale 为 4
		3 - scale 为 8
6:2	---	未定义
9:7	address size	记录内存地址大小:
		0 - 16 位
		1 - 32 位
		2 - 64 位
10	保留	固定为 0
11	operand size	记录操作数大小
		0 - 16 位
		1 - 32 位
14:12	---	未定义
17:15	segment	记录段寄存器:
		0 - ES
		1 - CS
		2 - SS
		3 - DS
		4 - FS
		5 - GS
21:18	index	记录 index 寄存器
22	index invalid	为 1 时, 表示 index 寄存器无效, 0 为有效
26:23	base	记录 base 寄存器
27	base invalid	为 1 时, 表示 base 寄存器无效, 0 为有效
29:28	indentify	指示具体的指令:
		0 - SGDT
		1 - SIDT
		2 - LGDT
		3 - LIDT
31:30	---	未定义

同样, VM-exit qualification 字段会记录内存操作数寻址中的偏移量 (参见 3.10.1.4 节)。

由 LLDT, SLDT, LTR 及 STR 引发的 VM-exit

LLDT 和 SLDT 指令访问局部描述符表寄存器, LTR 和 STR 指令访问 TSS 段寄存器。由于它们需要首先访问系统段描述符, 它们的操作数提供系统段描述符在 GDT 表中对应的 selector。

与 LGDT, SGDT, LIDT 及 SIDT 指令不同的是, 它们的操作数可以是寄存器, 也可以是内存操作数。VM-exit instruction information 字段必须区分这两种情况, 如表 3-38 所示。

表 3-38

位 域	名 字	描 述
1:0	scale	记录内存寻址中的 scale 值:
		0 - 没有 scale
		1 - scale 为 2
		2 - scale 为 4
		3 - scale 为 8
2	---	未定义
6:3	register operand	记录指令的寄存器操作数
9:7	address size	记录内存地址大小:
		0 - 16 位
		1 - 32 位
		2 - 64 位
10	mem/reg	1 - 寄存器操作数, 0 - 内存操作数
14:11	---	未定义
17:15	segment	记录段寄存器:
		0 - ES
		1 - CS
		2 - SS
		3 - DS
		4 - FS
		5 - GS
21:18	index	记录 index 寄存器
22	index invalid	1 - index 寄存器无效, 0 - 有效
26:23	base	记录 base 寄存器
27	base invalid	1 - base 寄存器无效, 0 - 有效
29:28	identity	指示具体的指令:
		0 - SLDT
		1 - STR

续表

位 域	名 字	描 述
29:28	identity	2 - LLDT
		3 - LTR
31:30	---	未定义

字段的 bits 1:0 指示内存寻址中的 scale 值。bits 6:3 记录寄存器操作数，如果 bit 10 为 1，表明指令使用寄存器操作数，那么 bits 6:3 域有效。bit 10 为 0 时，bits 6:3 域为未定义值。

bits 9:7 指示内存地址大小。Bits 17:15 记录内存操作数使用的段寄存器。bits 21:18 记录内存寻址中的 index 寄存器。当 bit 22 为 1 时，bits 21:18 有效，否则为未定义值。bits 26:23 记录内存寻址中的 base 寄存器。当 bit 27 为 1 时，bits 26:23 有效，否则为未定义值。

bits 29:28 用来识别具体的指令：0 为 SLDT 指令，1 为 STR 指令，2 为 LLDT 指令，3 为 LTR 指令。同样，VM-exit qualification 字段会记录内存操作数寻址中的偏移量（参见 3.10.1.4 节）。

由 RDRAND 指令引发的 VM-exit

RDRAND 指令用来读取由硬件产生的随机数，它的操作数只能为寄存器。

如表 3-39 所示，VM-exit instruction information 字段仅有两个位域有效，bits 6:3 记录寄存器操作数，bits 12:11 指示操作数大小。

表 3-39

位 域	名 字	描 述
2:0	---	未定义
6:3	register	记录寄存器操作数：
		0 - EAX, 1 - ECX
		2 - EDX, 3 - EBX
		4 - ESP, 5 - EBP
		6 - ESI, 7 - EDI
		8 到 15 - R8 到 R15
10:7	---	未定义
12:11	operand size	操作数大小：
		0 - 16 位
		1 - 32 位
		2 - 64 位
31:13	---	未定义

由 VMCLEAR, VMPTRLD, VMPTRST 及 VMXON 引发的 VM-exit

这几条指令的操作数是 VMCS 指针，必须为内存操作数。因此，VM-exit instruction

information 字段需要记录内存寻址信息，如表 3-40 所示。

表 3-40

位 域	名 字	描 述
1:0	scale	记录内存寻址中的 scale 值：
		0 - 没有 scale
		1 - scale 为 2
		2 - scale 为 4
		3 - scale 为 8
6:2	---	未定义
9:7	adderss size	内存地址大小：
		0 - 16 位
		1 - 32 位
		2 - 64 位
10	保留	为 0
14:11	---	未定义
17:15	segment	记录段寄存器：
		0 - ES
		1 - CS
		2 - SS
		3 - DS
		4 - FS
		5 - GS
21:18	index	记录 index 寄存器
22	index invalid	1 - index 寄存器无效，0 - 有效
26:23	base	记录 base 寄存器
27	base invalid	1 - base 寄存器无效，0 - 有效
31:28	---	未定义

字段的 bits 27:0 记录一个内存操作数寻址完整的信息，分别为 scale，base 与 index 寄存器，段寄存器及内存地址大小。这里并不需要记录操作数大小，VMCS 指针固定为 64 位。

由 VMREAD 与 VMWRITE 指令引发的 VM-exit

对于这两条指令，VM-exit instruction information 所记录的信息最为复杂。它们有两个操作数，VMREAD 指令的目标操作数可以是寄存器或内存操作数，VMWRITE 指令的源操作数也可以是寄存器或内存操作数，如表 3-41 所示。

表 3-41

位 域	名 字	描 述
1:0	scale	记录 scale 值:
		0 - 没有 scale
		1 - scale 为 2
		2 - scale 为 4
		3 - scale 为 8
2	---	未定义
6:3	reg1	记录目标寄存器操作数 (第 1 个操作数)
9:7	address size	内存地址大小:
		0 - 16 位
		1 - 32 位
		2 - 64 位
10	mem/reg	1 = 寄存器, 0 - 内存操作数
14:11	---	未定义
17:15	segment	记录段寄存器:
		0 - ES
		1 - CS
		2 - SS
		3 - DS
		4 - FS
		5 - GS
21:18	index	记录 index 寄存器
22	index invalid	1 - index 寄存器无效, 0 - 有效
26:23	base	记录 base 寄存器
27	base invalid	1 - base 寄存器无效, 0 - 有效
31:28	reg2	记录源寄存器操作数 (第 2 个操作数)

字段的 bits 1:0 记录内存寻址中的 scale 值, bits 9:7 指示内存地址大小, bits 17:15 记录内存操作数使用的段寄存器, bits 21:18 和 bits 26:23 分别记录内存寻址中的 index 和 base 寄存器。

bits 6:3 和 bits 31:28 记录 VMREAD 或 VMWRITE 指令的寄存器操作数, bits 6:3 是目标寄存器操作数 (即第 1 个操作数), bits 31:28 是源寄存器操作数 (即第 2 个操作数)。

bits 10 指示指令的操作数 (指由 ModRM.mod 与 ModRM.r/m 联合寻址的操作数) 是内存还是寄存器操作数。为 1 时, 表明是寄存器操作数。为 0 时, 表明是内存操作数。

以下面 4 个指令用法为例:

1) vmread eax, ebx ; ModRM.mod=11B & ModRM.r/m=eax, ModRM.reg=ebx

2) vmread [eax], ebx	; ModRM.mod=00B, ModRM.reg=ebx
3) vmwrite eax, ebx	; ModRM.mod=11B & ModRM.r/m=ebx, ModRM.reg=eax
4) vmwrite eax, [ebx]	; ModRM.mod=00B, ModRM.reg=eax

在第 1 条和第 3 条指令里, bit 10 为 1, bits 6:3 记录 EAX 寄存器, bits 31:28 记录 EBX 寄存器。在第 2 条指令里, bit 10 为 0, 那么 bits 6:3 为未定义值, bits 31:28 记录 EBX 寄存器。在第 4 条指令里, bit 10 为 0, 那么 bits 31:28 为未定义值, bits 6:3 记录 EAX 寄存器。

3.10.5 I/O SMI 信息类字段

在启用 SMM 双重监控处理机制下, 如果 SMM VM-exit 属于 I/O SMI 类型 (exit reason 值为 5, 见 3.10.1.1 节), 那么下面几个字段被使用:

- I/O RCX
- I/O RSI
- I/O RDI
- I/O RIP

这些字段属于 natural-width 类型, 在 64 位架构处理器上是 64 位, 否则为 32 位。当在 INS/OUTS 类指令后面发生 SMM VM-exit, 它们分别记录 RCX, RSI, RDI 及 RIP 值。

3.10.6 指令错误类字段

当执行一条 VMX 指令时, 如果发生 VMfailValid 失败, 32 位的 VM-instruction error 字段将记录指令的错误码。注意: 如果 VMX 指令执行时产生异常, 处理器会执行异常处理。此时, 不会存在 VMfailValid 失败。

在 2.6.3 节表 2-5 中, 列举了所有可能产生 VMfailValid 失败的原因, 每个原因有唯一的指令错误码。

3.11 VMM 初始化实例

在系统平台上, PCB (Processor Control Block) 结构中内嵌了 4 个 VMB (VM Manage Block) 结构, 分别为 GuestA, GuestB, GuestC 以及 GuestD。每个 VMB 结构对应一个 VM 实例, 用来管理和维护 VM, 其中一个功能是初始化每个 VM 对应的 VMCS 区域。

PCB 结构中也包含了若干个 VMX 相关的管理记录, 其中 VmxonPointer 指向 VMXON 区域, CurrentVmbPointer 指向当前的 VMB 块, 还有若干个对应的 VMCS buffer。

如以下代码所示 (另参见 1.4.1 节)。

代码片段 3-1:

```

struct PCB
    ... ..

    ;;
    ;; 记录 4 个 VMCS 管理指针, 指向 VMCS_MANAGE_BLOCK
    ;;
    .VmcsA            RESQ            1
    .VmcsB            RESQ            1
    .VmcsC            RESQ            1
    .VmcsD            RESQ            1

    ;;
    ;; 每个 logical processor 支持 4 个 VMCS 块
    ;;
    .GuestA            RESB            VMCS_MANAGE_BLOCK_SIZE
    .GuestB            RESB            VMCS_MANAGE_BLOCK_SIZE
    .GuestC            RESB            VMCS_MANAGE_BLOCK_SIZE
    .GuestD            RESB            VMCS_MANAGE_BLOCK_SIZE

    ... ..

    ;;
    ;; ##### 下面是 VMCS buffer #####
    ;;
    .GuestStateBuf     RESB            GUEST_STATE_SIZE
    .HostStateBuf      RESB            HOST_STATE_SIZE
    .ExecutionControlBuf RESB          EXECUTION_CONTROL_SIZE
    .ExitControlBuf    RESB            EXIT_CONTROL_SIZE
    .EntryControlBuf   RESB            ENTRY_CONTROL_SIZE
    .ExitInfoBuf       RESB            EXIT_INFO_SIZE

    ... ..
endstruct
```

在本节的内容中, 我们将根据前面获得的知识来实现 VMM 相关设置及 VMCS 初始化。

3.11.1 VMCS 相关的数据结构

在 inc\vmcs.inc 文件中定义了 VMB, 以及 VMCS 区域对应的 6 个 VMCS buffer 结构, 分别为 GUEST_STATE、HOST_STATE、EXECUTION_CONTROL、EXIT_CONTROL、ENTRY_CONTROL 及 EXIT_INFO 结构。

PCB 结构的 GuestStateBuf, HostStateBuf, ExecutionControlBuf, ExitControlBuf, EntryControlBuf 及 ExitInfoBuf 分别对应这些 VMCS buffer 结构。

3.11.1.1 VMB 结构

下面是 VMB 结构部分成员的定义。

代码片段 3-2:

```

struct VMB
    .PhysicalBase      RESQ    1          ; VMCS region 物理地址
```

```

.Base                RESQ    1          ; VMCS region 虚拟地址
.VsbBase             RESQ    1          ; VSB 基址
.VsbPhysicalBase     RESQ    1          ; VSB 物理基址
.DomainBase          RESQ    1          ; domain 虚拟基址
.DomainPhysicalBase  RESQ    1          ; domain 物理基址
.DomainPhysicalTop   RESQ    1          ; domain 物理地址顶部

;;
;; Guest 状态
;;
.GuestStatus         RESD     1

ALIGNB 8

;;
;; guest 环境入口点
;;
.GuestEntry          RESQ    1
.GuestStack          RESQ    1          ; guest 使用的栈

;;
;; VMM 入口
;;
.HostEntry           RESQ    1
.HostStack           RESQ    1

;;
;; 每个 VM 的 EP4TA
;;
.Ep4taBase           RESQ    1          ; 指向 EPT PML4T 虚拟地址
.Ep4taPhysicalBase   RESQ    1          ; 指向 EPT PML4T 物理地址

;;
;; 每个 VM 有自己的 VMX-preemption timer value 值
;;
.VmxTimerValue       RESD     1

;;
;; 每个 logical processor 的每个 VMCS 的 VPID
;;
.Vpid                RESW     1

ALIGNB 4
.GuestFlags          RESD     1

;;
;; 某些情况下, 传给 VMM do_xxx 函数的参数
;;
.DoProcessParam       RESD     1
.VmmDumpVmcsMask     RESQ     1

;;
;; APIC-access address
;;
.ApicAccessAddress    RESQ     1

```

```

.ApicAccessPhyAddress    RESQ    1

;;
;; ##### 下面的区域需要在初始化时分配页面 #####
;; 1) IoBitmap A page
;; 2) IoBitmap B page
;; 3) Virtual-access page
;; 4) MSR-Bitmap page
;; 5) VM-entry/VM-exit MSR store page
;; 6) VM-exit MSR load page
;; 7) IoVteBuffer page
;; 8) MsrVteBuffer page
;; 9) GpaHteBuffer page
;;

;;
;; I/O-Bitmap Address A & B
;;
.IoBitmapAddressA        RESQ    1
.IoBitmapPhyAddressA     RESQ    1
.IoBitmapAddressB        RESQ    1
.IoBitmapPhyAddressB     RESQ    1

;;
;; Virtual-APIC address
;;
.VirtualApicAddress       RESQ    1
.VirtualApicPhyAddress    RESQ    1

;;
;; MSR-Bitmap Address
;;
.MsrBitmapAddress         RESQ    1
.MsrBitmapPhyAddress      RESQ    1

;;
;; VM-exit MSR-store Address
;;
.VmEntryMsrLoadAddress:
.VmExitMsrStoreAddress    RESQ    1
.VmEntryMsrLoadPhyAddress:
.VmExitMsrStorePhyAddress RESQ    1

;;
;; VM-exit MSR-load Address
;;
.VmExitMsrLoadAddress     RESQ    1
.VmExitMsrLoadPhyAddress  RESQ    1

... ..

VMB_SIZE                  EQU     $
VMCS_MANAGE_BLOCK_SIZE    EQU     $
endstruc

```

PhysicalBase 是 VMCS 区域的物理指针，Base 是虚拟指针。当使用 VMPTRLD 指令加载 GuestA 为 current-VMCS 时，就可以像下面两种代码形式：


```

vmptlrd [ebp + PCB.GuestA]
vmptlrd [ebp + PCB.GuestA + VMB.PhysicalBase]

```

VsbBase 指向 VM 对应的 VSB (VM Store Block) 结构, 而 DomainBase 指向 VM 所使用的内存 domain 基址 (参见 6.3.1 节)。GuestEntry 和 HostEntry 分别指向 guest 和 host 的入口地址。每个虚拟机的 GuestEntry 值一般情况下是不同的, HostEntry 值可以一样, 表示 VMM 管理多个虚拟机。

Vpid 用来设置每个 VMCS 属于自己的 VPID 值, VmxTimerValue 是每个 VM 所使用的定时器初始计数值。GuestFlags 用来设置 guest 的运行模式及其他标志位。

代码片段 3-3:

```

#define GUEST_FLAG_PE          1
#define GUEST_FLAG_PG          80000000h
#define GUEST_FLAG_IA32E       4
#define GUEST_FLAG_V8086       8
#define GUEST_FLAG_UNRESTRICTED 10h
#define GUEST_FLAG_EPT         20h
#define GUEST_FLAG_USER        40h

```

inc\vmcs.inc 中定义了 7 个 GUEST_FLAG 值。在对 VMCS 区域初始化时, 可以使用这些标志位来设置目标 guest 的运行环境。

接下来, 为每个 VMCS 区域指定自己所引用的访问区域, 包括 I/O bitmap A 和 B 区域、Virtual-APIC page shadow 页面、MSR bitmap 区域、VM-exit MSR store 及 VM-exit MSR load 区域。VM-entry MSR load 区域和 VM-exit MSR store 区域重叠, 而 APIC-access page 地址应该设置为 IA32_APIC_BASE 寄存器中 APIC 物理基址值。

3.11.1.2 VSB 结构

VSB 结构同样定义在 inc\vmcs.inc 文件中, 它的定义如下。

代码片段 3-4:

```

struct VSB
{
    .Base                RESQ        1
    .PhysicalBase        RESQ        1

    ;;
    ;; VM video buffer 管理记录
    ;;
    .VmVideoBufferHead    RESQ        1
    .VmVideoBufferPtr     RESQ        1
    .VmVideoBufferLastChar RESD        1
    .VmVideoBufferSize    RESD        1

    ;;
    ;; VM keyboard buffer 管理记录
    ;;
    ALIGNB 8
    .VmKeyBufferHead      RESQ        1
    .VmKeyBufferPtr       RESQ        1
    .VmKeyBufferSize      RESD        1
}

```

```

;;
;; guest 的 context 信息
;;
.Context:
.Rax                RESQ        1
.Rcx                RESQ        1
.Rdx                RESQ        1
.Rbx                RESQ        1
.Rsp                RESQ        1
.Rbp                RESQ        1
.Rsi                RESQ        1
.Rdi                RESQ        1
.R8                 RESQ        1
.R9                 RESQ        1
.R10                RESQ        1
.R11                RESQ        1
.R12                RESQ        1
.R13                RESQ        1
.R14                RESQ        1
.R15                RESQ        1
.Rip                RESQ        1
.RFlags             RESQ        1

;;
;; FPU 单元 context
;;
ALIGNB 16
.FpuStateImage:
.FpuEnvironmentImage:  RESB        28
.FpuStackImage:        RESB        80

;;
;; XMM image 区域, 使用于 FXSAVE/FXRSTOR 指令 (512 字节)
;;
ALIGNB 16
.XMMStateImage:        RESB        512

;;
;; VM keyboard buffer 区域, 共 256 个字节, 保存按键扫描码
;;
.VmKeyBuffer          RESB        256
.Reserve               RESB        4096 - $

;;
;; VM video buffer 区域, 存放处理器的屏幕信息
;; 4K, 存放约 25 × 80 × 2 信息
;;
.VmVideoBuffer         RESB        (4096 * 2 - $)

VM_STORAGE_BLOCK_SIZE EQU    $
VSB_SIZE               EQU    $
endstruc

```

VSX 结构定义了三类存储区域, 分别为:

(1) VmKeyBuffer, 每个 VM 对应的键盘缓冲区。

(2) VmVideoBuffer, 每个 VM 对应的 video 缓冲区。

(3) context 区域, 包括 15 个通用寄存器, RIP 与 RFLAGS, x87 FPU/MMX state 区域以及 XMM state 区域。

每个 VM 有自己独立的键盘与 video 缓冲区, 这样可以与 host 使用的 LSB (Local Store Block) 区域内的键盘与 video 缓冲区分离开, 避免互相干扰 (参见 1.4.2 节)。在发生处理器的切换时, 会检查 guest 当前是否拥有焦点, 从而也需要进行 VSB 区域的键盘与 video 缓冲区的切换。

guest 的 context 信息

在 VSB 区域中添加每个 VM 的 context 区域, 是为了将每个 guest 的 context 独立出来。当 VMM 同时管理多个 VM 时 (即同时运行多个 guest), 就显得尤为重要。

当 guest 产生 VM-exit 时, 进入 VMM 后首要任务是调用 store_guest_context 函数在 guest 私有的 VSB.Context 中保存上下文环境信息。在执行 VMRESUME 指令恢复 guest 运行前, VMM 调用 restore_guest_context 函数恢复 guest 原来的上下文环境信息。

store_guest_context 与 restore_guest_context 函数实现在 lib\Vm\VmLib.asm 文件中, 也实现了 get_guest_register_value 与 set_guest_register_value 函数来读写 guest 的某个寄存器值。代码如下:

代码片段 3-5:

```
;-----
; get_guest_register_value()
; input:
;     esi - register ID
; output:
;     eax - base address
; 描述:
;     1) 根据提供的寄存器 ID 来读取 guest 寄存器值
;-----
get_guest_register_value:
    push ebp
#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif
    REX.Wrb
    mov ebp, [ebp + PCB.CurrentVmbPointer]
    REX.Wrb
    mov ebp, [ebp + VMB.VsbBase]
    and esi, 0Fh
    REX.Wrb
    mov eax, [ebp + VSB.Context + esi * 8]
    pop ebp
    ret
```

get_guest_register_value 函数根据寄存器 ID 值从 VSB.Context 中读取相应的寄存器, 这个函数在 VMM 对某些指令的分析时非常有用。set_guest_register_value 函数实现的原理一样, 根据寄存器 ID 值写 VSB.Context 的寄存器值。

3.11.1.3 VMCS buffer 结构

VMM 在初始化 VMCS 区域时，并不直接向 VMCS 字段写入值，而是选择将值写入 VMCS buffer 对应的值中，然后在某个时刻统一将 VMCS buffer 数据刷新到 VMCS 区域。

PCB 结构中的 6 个 VMCS buffer 分别对应 VMCS 的 6 个区域，下面是 EXECUTION_CONTROL 结构的定义。

代码片段 3-6:

```

struc EXECUTION_CONTROL
    .PinControl1                RESD    1
    .ProcessorControl1          RESD    1
    .ProcessorControl2          RESD    1
    .ExceptionBitmap            RESD    1
    .PfErrorCodeMask            RESD    1
    .PfErrorCodeMatch           RESD    1
    .IoBitmapAddressA           RESQ    1
    .IoBitmapAddressB           RESQ    1
    .TscOffset                   RESQ    1
    .Cr0GuestHostMask           RESQ    1
    .Cr0ReadShadow              RESQ    1
    .Cr4GuestHostMask           RESQ    1
    .Cr4ReadShadow              RESQ    1
    .Cr3TargetCount             RESD    1
    .Cr3Target0                 RESQ    1
    .Cr3Target1                 RESQ    1
    .Cr3Target2                 RESQ    1
    .Cr3Target3                 RESQ    1

    ;;
    ;; APIC virtualization 有关域
    ;;
    .ApicAccessAddress           RESQ    1
    .VirtualApicAddress          RESQ    1
    .TprThreshold                RESD    1
    .EoiBitmap0                  RESQ    1
    .EoiBitmap1                  RESQ    1
    .EoiBitmap2                  RESQ    1
    .EoiBitmap3                  RESQ    1
    .PostedInterruptVector       RESW    1
    ALIGNB 4
    .PostedInterruptDescriptorAddr RESQ    1

    .MsrBitmapAddress            RESQ    1
    .ExecutiveVmcsPointer        RESQ    1
    .EptPointer                  RESQ    1
    .Vpid                        RESW    1

    ALIGNB 4
    .PleGap                      RESW    1
    .Plewindow                   RESW    1

    ALIGNB 4
    .VmFunctionControl1          RESQ    1
    .EptpListAddress             RESQ    1

    EXECUTION_CONTROL_SIZE      EQU     $

```

endstruc

EXECUTION_CONTROL 结构中的每个成员与 VMCS 的 VM-execution control 区域字段是保持一致的。同样，GUEST_STATE，HOST_STATE，EXIT_CONTROL，ENTRY_CONTROL 及 EXIT_INFO 结构内的成员都与 VMCS 区域字段一一对应。

3.11.2 初始化 VMXON 区域

处理器在 stage2 或者 stage3 阶段初始化末尾将调用 vmx_operation_enter 函数进入 VMX operation 模式（参见 1.5.2 节与 1.5.3 节）。vmx_operation_enter 函数内在执行 VMXON 指令之前，将调用 initialize_vmxon_region 函数初始化 VMXON 区域。

代码片段 3-7:

```

;-----
; initialize_vmxon_region()
; input:
;     none
; output:
;     0 - successful
;     otherwise - 错误码
; 描述:
;     1) 初始化 VMM (host) 的 vmxon region
;-----
initialize_vmxon_region:
    push ebx
    push ecx
    push ebp

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

;;
;; 读 CR0 当前值
;;
    REX.Wrb
    mov ecx, cr0
    mov ebx, ecx

;;
;; 检查 CR0.PE 与 CR0.PG 是否符合 fixed 位，这里只检查低 32 位值
;; 1) 对比 Cr0FixedMask 值（固定为 1 值），不相同则返回错误码
;;
    mov eax, STATUS_VMX_UNEXPECT          ; 错误码（超出期望值）
    xor ecx, [ebp + PCB.Cr0FixedMask]     ; 与 Cr0FixedMask 值异或，检查是否相同
    js initialize_vmxon_region.done       ; 检查 CR0.PG 位是否相等
    test ecx, 1
    jnz initialize_vmxon_region.done      ; 检查 CR0.PE 位是否相等

;;
;; 如果 CR0.PE 与 CR0.PG 位相符，设置 CR0 其他位
;;
    or ebx, [ebp + PCB.Cr0Fixed0]         ; 设置 Fixed 1 位

```

```

    and ebx, [ebp + PCB.Cr0Fixed1]          ; 设置 Fixed 0 位
    REX.Wrb
    mov cr0, ebx                            ; 写回 CR0

    ;;
    ;; 直接设置 CR4 fixed 1 位
    ;;
    REX.W
    mov ecx, cr4
    or ecx, [ebp + PCB.Cr4FixedMask]        ; 设置 Fixed 1 位
    and ecx, [ebp + PCB.Cr4Fixed1]         ; 设置 Fixed 0 位
    REX.W
    mov cr4, ecx

    ;;
    ;; 分配 VMXON region
    ;;
    call get_vmcs_access_pointer            ; edx:eax = pa:va
    REX.Wrb
    mov [ebp + PCB.VmxonPointer], eax
    REX.Wrb
    mov [ebp + PCB.VmxonPhysicalPointer], edx

    ;;
    ;; 设置 VMCS region 信息
    ;;
    REX.Wrb
    mov ebx, [ebp + PCB.VmxonPointer]
    mov eax, [ebp + PCB.VmxBasic]           ; 读取 VMCS revision identifier 值
    mov [ebx], eax                         ; 写入 VMCS ID

    mov eax, STATUS_SUCCESS

initialize_vmxon_region.done:
    pop ebp
    pop ecx
    pop ebx
    ret

```

initialize_vmxon_region 函数内调用 get_vmcs_access_pointer 函数分配 4K 的 VMXON 区域。然后将 IA32_VMX_BASIC 寄存器 bits 31:0 所代表的 VMCS ID 值写入 VMXON 区域首 DWORD 位置。

initialize_vmxon_region 函数也负责设置 CR0 与 CR4 寄存器。CR0 = (CR0 | Cr0Fixed0) & Cr0Fixed1, CR4 = (CR4 | Cr4FixedMask) & Cr4Fixed1, 这使得 CR0 与 CR4 寄存器的固定位满足 VMX 要求。

3.11.3 初始化 VMCS 区域

VMM 使用 initialize_vmcs_buffer 函数根据目标 VMB 来初始化 VMCS buffer, 后续再将 VMCS buffer 数据刷新到 VMCS 区域。initialize_vmcs_buffer 函数的原型看起来像下面这样:

```

STATUS_CODE
initialize_vmcs_buffer(

```

```
PVMB VmbPointer
);
```

VmbPointer 提供目标 VMB 块指针。因此，这个函数可以在 VMM 执行 VMPTRLD 指令加载 current-VMCS 之前调用。当 VMM 加载 current-VMCS 之后，可以调用 setup_vmcs_region 函数将 VMCS buffer 刷新到 current-VMCS 区域中，如以下代码所示。

代码片段 3-8:

```
-----
; setup_vmcs_region():
; input:
;     none
; output:
;     none
;-----
setup_vmcs_region:
    ;;
    ;; 下面将 VMCS buffer 数据刷新到 VMCS 中
    ;;
    call flush_execution_control
    call flush_exit_control
    call flush_entry_control
    call flush_host_state
    call flush_guest_state
    ret
```

setup_vmcs_region 函数只是简单地调用每个 VMCS buffer 对应的刷新函数写入 current-VMCS 的各个区域中。

3.11.3.1 分配 VMCS 区域

在系统的 stage2 及 stage3 阶段，初始化末尾会调用 vmx_operation_enter 函数进入 VMX operation 模式（参见 1.5.3 节与 1.5.4 节），这个函数将分配 4 个 VMCS 区域保存在 GuestA, GuestB, GuestC 及 GuestD 结构的 VMCS 指针中（参见 3.11.1.1 节）。

代码片段 3-9:

```
;;
;; 分配 VMCS A 区域，并作为默认的 VMCS 区域
;;
call get_vmcs_pointer
RAX.WRxb
mov [ebp + PCB.GuestA + 8], eax          ; VMCS A 虚拟地址
RAX.WRxb
mov [ebp + PCB.GuestA], edx             ; VMCS A 物理地址
mov ax, cx
or ax, 1
mov [ebp + PCB.GuestA + VMB.Vpid], ax   ; VMCS A 的 VPID

;;
;; 分配 VMCS B 区域
;;
call get_vmcs_pointer
RAX.WRxb
mov [ebp + PCB.GuestB + 8], eax          ; VMCS B 虚拟地址
RAX.WRxb
mov [ebp + PCB.GuestB], edx             ; VMCS B 物理地址
```

```

mov ax, cx
or ax, 2
mov [ebp + PCB.GuestB + VMB.Vpid], ax          ; VMCS B 的 VPID

;;
;; 分配 VMCS C 区域
;;
call get_vmcs_pointer
REX.Wrxb
mov [ebp + PCB.GuestC + 8], eax                ; VMCS C 虚拟地址
REX.Wrxb
mov [ebp + PCB.GuestC], edx                    ; VMCS C 物理地址
mov ax, cx
or ax, 3
mov [ebp + PCB.GuestC + VMB.Vpid], ax

;;
;; 分配 VMCS D 区域
;;
call get_vmcs_pointer
REX.Wrxb
mov [ebp + PCB.GuestD + 8], eax                ; VMCS D 虚拟地址
REX.Wrxb
mov [ebp + PCB.GuestD], edx                    ; VMCS D 物理地址
mov ax, cx
or ax, 4
mov [ebp + PCB.GuestC + VMB.Vpid], ax
... ..

```

上面 `vmx_operation_enter` 函数部分代码，VMB 结构偏移量 0 位置是 VMCS 的物理地址，偏移量 8 位置是 VMCS 的虚拟地址。它也负责为每个 VMCS 分配一个 VPID 值。

3.11.3.2 VMCS 初始化模式

取决于 guest 运行环境的不同，VMCS 有如下两种初始化模式：

(1) guest 使用与 host 相同的环境。因此，VMCS 的 `guest-state` 区域字段值设置为 host 环境对应寄存器的值，也使用 host 环境的分页模式与同一份页表结构。

(2) guest 使用独立的环境。在这种模式下，VMM 支持完整独立的 guest OS 运行。

本书的第 4 章与第 5 章的例子使用“guest 与 host 相同环境”模式。第 6 章与第 7 章的例子使用“guest 独立环境”模式，支持 guest OS 从实模式启动运行。guest 使用 host 相同环境是非常有用的，当 host OS 中加入 VMM 组件时，可以监控 host OS 的运行。

在调用 `initialize_vmcs_buffer` 函数之前，需要给目标 VMB 设置初始标志值，如下面的代码所示。

代码片段 3-10：

```

mov R5, [gs: PCB.Base]

;;
;; CR0.PG = CR.PE = 1
;;
mov eax, GUEST_FLAG_PE | GUEST_FLAG_PG
#ifdef __X64

```



```

    or eax, GUEST_FLAG_IA32E
%endif
    mov DWORD [R5 + PCB.GuestA + VMB.GuestFlags], eax

    ;;
    ;; 初始化 VMCS region
    ;;
    mov DWORD [R5 + PCB.GuestA + VMB.GuestEntry], guest_entry
    mov DWORD [R5 + PCB.GuestA + VMB.HostEntry], VmmEntry

    ;;
    ;; 分配 guest stack
    ;;
    mov R7, get_user_stack_pointer
    mov R6, get_kernel_stack_pointer
    test eax, GUEST_FLAG_USER
    cmovnz R6, R7
    call R6
    mov [R5 + PCB.GuestA + VMB.GuestStack], R0

    ;;
    ;; 初始化 VMCS buffer
    ;;
    mov R6, [R5 + PCB.VmcsA]
    call initialize_vmcs_buffer

```

上面代码中的寄存器 **R5**、**R6** 及 **R7** 名字定义在 `ex.asm` 文件同一目录下的 `ex.inc` 文件里，是为了能在 32 位及 64 位下通用编译而定义。`initialize_vmcs_buffer` 函数在配置 `GuestStateBuf` 结构时，会根据 `VMB.GuestFlags` 这个标志位设置 `guest` 进入的运行模式。`VMB.GuestFlags` 可以有下面的设置。

- `GuestFlags = GUEST_FLAG_PE | GUEST_FLAG_PG` 时，表明 `guest` 需要进入分页的保护模式。
- `GuestFlags = GUEST_FLAG_PE | GUEST_FLAG_PG | GUEST_FLAG_IA32E` 时，表明 `guest` 需要进入 IA-32e 模式。
- `GuestFlags = GUEST_FLAG_UNRESTRICTED | GUEST_FLAG_EPT` 时，表明 `guest` 需要进入实模式。
- `GuestFlags = GUEST_FLAG_UNRESTRICTED | GUEST_FLAG_EPT | GUEST_FLAG_PE` 时，表明 `guest` 进入未分页的保护模式。

在本书例子的初始化里，当 `guest` 进入保护模式时，使用与 `host` 相同的环境，其他情况下则使用 `guest` 独立环境。`initialize_vmcs_buffer` 函数检查 `VMB.GuestFlags` 的 `GUEST_FLAG_PE` 标志位，为 1 时调用 `init_guest_state_area` 函数设置 `GuestStateBuf` 结构。为 0 时则调用 `init_realmode_guest_state` 函数设置 `GuestStateBuf` 结构。

`VMB` 块内的 `GuestEntry` 与 `HostEntry` 值也需要 `VMM` 提前设置。`VMM` 的入口 `VmmEntry` 例程实现在 `lib\Vm\Vm\VMM.asm` 文件中，是由所有原因而导致 `guest` 产生 `VM-exit` 后 `VMM` 处理的入口。而 `guest` 的入口 `guest_entry` 函数代码则实现在例子目录下的 `ex.asm` 文件中。

当 guest 使用与 host 相同的环境时，需要为 guest 分配独立的 stack 空间。当 guest 进入 3 级权限时，调用 `get_user_stack_pointer` 分配 user 层的栈空间，而进入 0 级权限时，调用 `get_kernel_stack_pointer` 分配 kernel 层的栈空间。

3.11.3.3 VMCS buffer 初始化

`initialize_vmcs_buffer` 函数实现在 `lib\Vm\VmInit.asm` 文件中，如下面的代码所示。

代码片段 3-11:

```

;-----
; initialize_vmcs_buffer()
; input:
;     esi - VM_MANAGE_BLOCK pointer
; output:
;     0 - successful
;     otherwise - 错误码
; 描述:
;     1) 初始化提供的 vmcs buffer (由 VM 管理块指针提供)
;-----
initialize_vmcs_buffer:
    push ebp
    push ecx
    push ebx
    push edx

    ;;
    ;; PCB 基址
    ;;
%ifdef __X64
    LoadGsBaseToRbp
%else
    mov ebp, [gs: PCB.Base]
%endif

    push esi                                ; 保存 VMCS 管理块指针

    REX.Wrxb
    mov ebx, esi

    ;;
    ;; 写入 VMCS region 的 Identifier 值
    ;;
    mov eax, [ebp + PCB.VmxBasic]
    REX.Wrxb
    mov edi, [ebx + VMB.Base]                ; VMCS region 虚拟地址
    mov [edi], eax

    ;;
    ;; 写入 VMM 管理记录
    ;;
    REX.Wrxb
    mov eax, [ebp + PCB.VmmStack]
    REX.Wrxb
    mov [ebx + VMB.HostStack], eax
    REX.Wrxb
    mov eax, [ebp + PCB.VmmMsrLoadAddress]
    REX.Wrxb

```

```

mov [ebx + VMB.VmExitMsrLoadAddress], eax
REX.Wrxb
mov eax, [ebp + PCB.VmmMsrLoadPhyAddress]
REX.Wrxb
mov [ebx + VMB.VmExitMsrLoadPhyAddress], eax

;;
;; 初始化 VM 的 VSB 区域
;;
REX.Wrxb
mov esi, ebx
call init_vm_storage_block

;;
;; 初始化 VM domain
;;
call vm_alloc_domain
REX.Wrxb
mov [ebx + VMB.DomainBase], eax
REX.Wrxb
mov [ebx + VMB.DomainPhysicalBase], edx
REX.Wrxb
add edx, (DOMAIN_SIZE - 1)
mov [ebx + VMB.DomainPhysicalTop], edx

;;
;; 初始化 EP4TA
;;
call get_vmcs_access_pointer
REX.Wrxb
mov [ebx + VMB.Ep4taBase], eax
REX.Wrxb
mov [ebx + VMB.Ep4taPhysicalBase], edx

;;
;; 下面为 VMCS region 分配相关的 access page, 包括:
;; 1) IoBitmap A page
;; 2) IoBitmap B page
;; 3) Virtual-access page
;; 4) MSR-Bitmap page
;; 5) VM-entry/VM-exit MSR store page
;; 6) VM-exit MSR load page
;; 7) IoVteBuffer page
;; 8) MsrVteBuffer page
;; 9) GpaHteBuffer page
;;

mov ecx, 9 ; 共 9 个 access page
REX.Wrxb
lea ebx, [ebx + VMB.IoBitmapAddressA] ; VMB.IoBitmapAddressA 地址

;;
;; 使用 get_vmcs_access_pointer() 分配一个 access page
;; 1) edx:eax 返回对应的 physical address 与 virtual address
;; 2) 在 X64 下返回对应的 64 位地址
;; 3) 注意: 这里不检查 get_vmcs_access_pointer() 的返回值,
;; 作为演示, 并没设计当超出内存资源时的情形!
;;

```

```

initialize_vmcs_buffer.loop:
    call get_vmcs_access_pointer
    REX.Wrxb
    mov [ebx], eax                ; 写入 access page 虚拟地址
    REX.Wrxb
    mov [ebx + 8], edx            ; 写入 access page 物理地址
    REX.Wrxb
    add ebx, 16                   ; 指向下一条记录
    DECv ecx
    jnz initialize_vmcs_buffer.loop

    pop ebx                      ; ebx - VMCS 管理块指针
    xor eax, eax

    ;;
    ;; 初始化 IO & MSR table entry 计数值
    ;;
    mov [ebx + VMB.IoVteCount], eax
    mov [ebx + VMB.MsrVteCount], eax
    mov [ebx + VMB.GpaHteCount], eax

    ;;
    ;; 初始化 MSR-store/MSR-load 列表计数值
    ;;
    mov [ebx + VMB.VmExitMsrStoreCount], eax
    mov [ebx + VMB.VmExitMsrLoadCount], eax

    ;;
    ;; 初始化 IO VTE, MSR VTE, GPA HTE, EXTINT ITE 指针
    ;;
    REX.Wrxb
    mov eax, [ebx + VMB.IoVteBuffer]
    REX.Wrxb
    mov [ebx + VMB.IoVteIndex], eax
    REX.Wrxb
    mov eax, [ebx + VMB.MsrVteBuffer]
    REX.Wrxb
    mov [ebx + VMB.MsrVteIndex], eax
    REX.Wrxb
    mov eax, [ebx + VMB.GpaHteBuffer]
    REX.Wrxb
    mov [ebx + VMB.GpaHteIndex], eax

    ;;
    ;; I/O 操作标志位
    ;;
    mov DWORD [ebx + VMB.IoOperationFlags], 0

    ;;
    ;; 初始化 guest-status
    ;;
    mov DWORD [ebx + VMB.GuestSmb + GSMB.ProcessorStatus], 0
    mov DWORD [ebx + VMB.GuestSmb + GSMB.InstructionStatus], 0

    ;;

```

```

;; 清空 VMCS buffer
;;
mov esi, EXECUTION_CONTROL_SIZE
REX.Wrxb
lea edi, [ebp + PCB.ExecutionControlBuf]
call zero_memory
mov esi, ENTRY_CONTROL_SIZE
REX.Wrxb
lea edi, [ebp + PCB.EntryControlBuf]
call zero_memory
mov esi, EXIT_CONTROL_SIZE
REX.Wrxb
lea edi, [ebp + PCB.ExitControlBuf]
call zero_memory
mov esi, HOST_STATE_SIZE
REX.Wrxb
lea edi, [ebp + PCB.HostStateBuf]
call zero_memory
mov esi, GUEST_STATE_SIZE
REX.Wrxb
lea edi, [ebp + PCB.GuestStateBuf]
call zero_memory

;;
;; 下面分别初始化各个 VMCS 域, 包括:
;; 1) VM execution control fields
;; 2) VM-exit control fields
;; 3) VM-entry control fields
;; 4) VM host state fields
;; 5) VM guest state fields
;;
REX.Wrxb
mov esi, ebx
call init_vm_execution_control_fields
REX.Wrxb
mov esi, ebx
call init_vm_exit_control_fields
REX.Wrxb
mov esi, ebx
call init_vm_entry_control_fields
REX.Wrxb
mov esi, ebx
call init_host_state_area

REX.Wrxb
mov esi, ebx

;;
;; 如果 guest 为实模式, 则调用 init_realmode_guest_sate
;;
mov eax, init_guest_state_area
mov ebx, init_realmode_guest_state
test DWORD [esi + VMB.GuestFlags], GUEST_FLAG_PE
cmovz eax, ebx
call eax

pop edx
pop ebx
pop ecx

```

```
pop ebp
ret
```

initialize_vmcs_buffer 函数所做的主要工作如下：

- (1) 向 VMCS 写入 VMCS ID 值。
- (2) 调用 init_vm_storage_block 函数初始化 VSB 结构。
- (3) 调用 vm_alloc_domain 函数为每个 VM 分配内存 domain。
- (4) 分配 EPT 页表的 PML4T 区域，作为 EP4TA (EPT PML4T address) 值，保存在 Ep4taBase 中。
- (5) 分配 VMCS 所使用的各个区域。包括：I/O bitmap, virtual-APIC page, MSR-bitmap, VM-entry 与 VM-exit MSR-store 列表区域, VM-exit MSR-load 列表区域。也包括 VMM 其他使用区域。
- (6) 初始化这些区域的管理记录。
- (7) 最后清空 VMCS buffer，然后调用一系列的 init_vm_xxx_fields 函数来初始化 VMCS buffer。

lib\Vm\VmInit.asm 文件中包括了这些 VMCS buffer 初始化函数，实现的代码比较长，这里不再贴出，请自行查阅。这里先初始化几个控制区域：VM-execution, VM-exit 及 VM-entry，再初始化 host-state 区域，最后初始化 guest-state 区域。host-state 与 guest-state 区域的字段可能会因为控制区域字段而有不同的设置。

3.11.4 例子 3-1

示例 3-1：使用 guest 与 host 相同的环境，测试 guest

在这个例子中，guest 使用与 host 相同的运行环境。也就是：当 host 运行在 stage2 阶段时，guest 也运行在 stage2 阶段。host 运行在 stage3 阶段时，guest 同样运行在 stage3 阶段。

例子的主体代码实现在 ex.asm 文件里，我们加入了一个命令选择器 do_command 函数，由 CPU0 执行供用户选择相应的命令。当用户按下数字键<1>时将调度 CPU1 执行 TargetCpuVmentry 例程，让 CPU1 切入 guest 运行。

代码片段 3-12：

```
;-----
; TargetCpuVmentry()
; input:
;     none
; output:
;     none
; 描述:
;     1) 调度执行的目标代码
;     2) 此函数让处理器执行 VM-entry 操作
;-----
```

```

TargetCpuVmentry:
    push R5
    mov R5, [gs: PCB.Base]

    ;;
    ;; CR0.PG = CR.PE = 1
    ;;
    mov eax, GUEST_FLAG_PE | GUEST_FLAG_PG
#ifdef __X64
    or eax, GUEST_FLAG_IA32E
#endif
    mov DWORD [R5 + PCB.GuestA + VMB.GuestFlags], eax

    ;;
    ;; 初始化 VMCS region
    ;;
    mov DWORD [R5 + PCB.GuestA + VMB.GuestEntry], guest_entry
    mov DWORD [R5 + PCB.GuestA + VMB.HostEntry], VmmEntry

    ;;
    ;; 分配 guest stack
    ;;
    mov R7, get_user_stack_pointer
    mov R6, get_kernel_stack_pointer
    test eax, GUEST_FLAG_USER
    cmovnz R6, R7
    call R6
    mov [R5 + PCB.GuestA + VMB.GuestStack], R0

    ;;
    ;; 初始化 VMCS buffer
    ;;
    mov R6, [R5 + PCB.VmcsA]
    call initialize_vmcs_buffer

    ;;
    ;; 执行 VMCLEAR 操作
    ;;
    vmclear [R5 + PCB.GuestA]
    jc @1
    jz @1

    ;;
    ;; 加载 VMCS pointer
    ;;
    vmptlrd [R5 + PCB.GuestA]
    jc @1
    jz @1

    ;;
    ;; 更新当前 VMB 指针
    ;;
    mov R0, [R5 + PCB.VmcsA]
    mov [R5 + PCB.CurrentVmbPointer], R0

    ;;
    ;; 配置 VMCS
    ;;
    call setup_vmcs_region

```

```

    call update_system_status

    ;;
    ;; 进入 guest 环境
    ;;
    call reset_guest_context
    or DWORD [gs: PCB.ProcessorStatus], CPU_STATUS_GUEST
    vmlaunch

@1:
    call dump_vmcs
    call wait_esc_for_reset
    pop R5
    ret

```

TargetCpuVmentry 例程的工作是初始化目标处理器的 VMCS，然后执行 VMLAUNCH 指令发起 VM-entry 操作进入 guest 环境执行。

Guest 代码

guest 的入口为 guest_entry 函数，如下面的代码所示。

代码片段 3-13:

```

guest_entry:

    DEBUG_RECORD    "[VM-entry]: switch to guest !"        ; 插入 debug 记录点

    call dump_guest_env                                    ; 打印环境信息

    hlt
    jmp $ - 1
    ret

```

在 guest 入口处插入了一个调试记录点，以便于观察。实际的工作只是调用 dump_guest_env 函数来打印 guest 的环境信息，然后转入 HLT 状态。

编译及运行

我们需要打开调试记录功能，使用下面的命令进行编译并生成相应映像文件。

- build -DDEBUG_RECORD_ENABLE，编译为 32 位环境。
- build -DDEBUG_RECORD_ENABLE -D_X64，编译为 64 位环境。

在 VMware 中加载 demo.img 映像文件运行后，在 CPU0 屏幕的命令选择器中，按下数字<1>键后让 CPU1 进入 guest 运行，然后按功能键<F2>切换到 CPU1 屏幕，如图 3-10 所示。

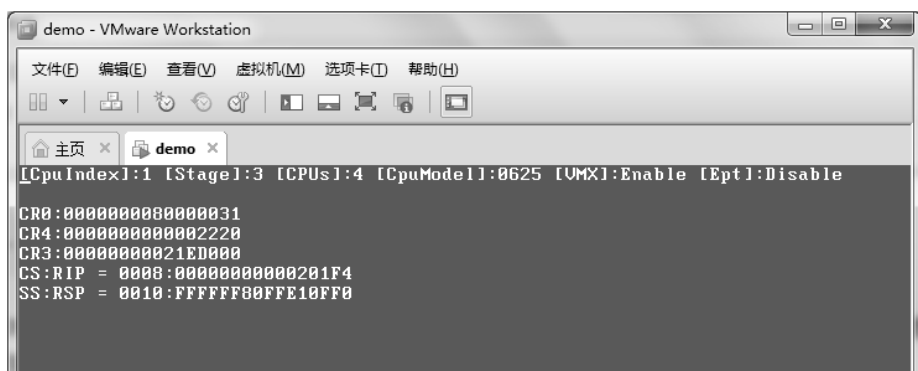


图 3-10

dump_guest_env 函数分别打印出 guest 环境中的 CR0, CR4 及 CR3 寄存器值, 它们和 host 环境中的值是一致的。两个段寄存器 CS 与 SS 的值也等于 host 环境中 CS 与 SS 寄存器的值。注意, 这时候 EPT 机制是关闭的, 由于 CR3 寄存器的值一样, 表明 guest 中也使用 host 的页表结构。

下面按下<F4>键切换到 CPU3, 再按下<Enter>键切换到 msg-list 列表显示屏, 如图 3-11 所示。

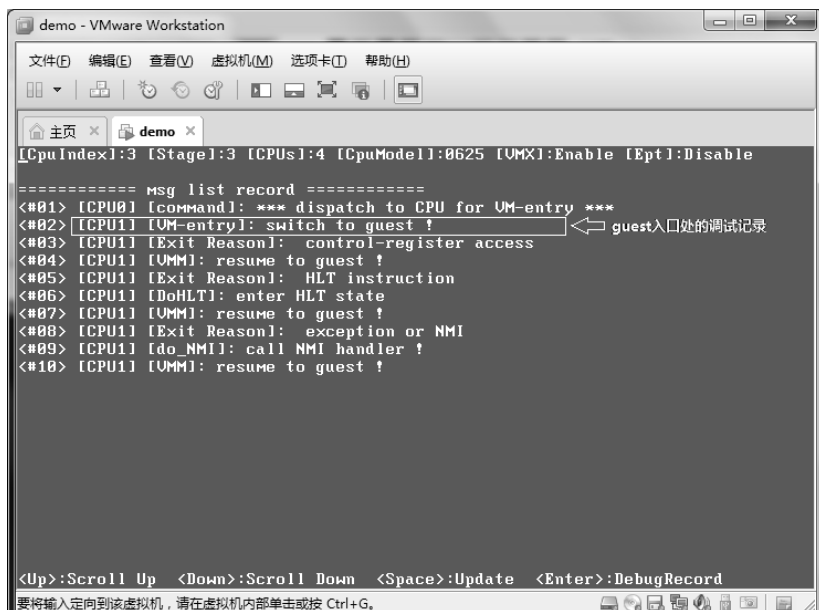


图 3-11

在图 3-11 中的 msg-list 列表里, 第 1 条记录显示 CPU0 调度了 CPU1 进行 VM-entry 操作。第 2 条记录显示 CPU1 切换到 guest 运行, 这条记录是我们在 guest 入口处插入的那条调试记录。后续信息显示 guest 曾经访问过 CR 寄存器, 并且执行 HLT 指令进入 HLT 状态。

第 4 章

VM-entry 处理

处理器在 VMX operation 模式下有两种执行环境：**root** 与 **non-root**。VMM (Virtual Machine Monitor) 软件运行在 root 环境，VM (Virtual Machine) 运行在 non-root 环境。VM 代表着一个虚拟机实例，guest 软件运行在这个虚拟机里。

在一个虚拟机平台里可以存在多个虚拟机实例，同时运行多个 guest 软件。每当需要执行某个 guest 软件时，处理器必须从 root 模式切换到 non-root 模式执行，这个切换过程叫做“**VM-entry**”。

逻辑处理器

一个逻辑处理器 (logical processor) 是指处理器的一个执行单元。在 Intel 处理器上可以实现 Hyper-Threading (超线程) 与 multi-core (物理多核心) 技术来支持 multi-threading (处理器的多线程) 架构。一个 SMT (simultaneous multithreading) 是物理处理器里的一个线程单元，也是物理处理器的执行单元，也代表着一个逻辑处理器。在不支持 multi-threading 的情况下，一个处理器的物理核心就代表着一个逻辑处理器。

注意：本书所有章节中所提到的“**处理器**”一词都可以认为是“**逻辑处理器**”。为了书写简便，大多时候省去了“逻辑”二字，敬请留意。

虚拟处理器

由于存在虚拟机的概念 (或环境)，所以**虚拟处理器**表述为“运行在虚拟机环境里的逻辑处理器”，也就是说当逻辑处理器运行在 VMX non-root operation 模式里时可以被称为“虚拟处理器”。这个虚拟处理器并非虚拟的，而是物理的逻辑处理器，只是在概念上被描述为“虚拟”来对应虚拟机环境，这也是因为虚拟机环境内处理器的可配置性。

在 VM-entry 后，虚拟处理器的状态信息和运行环境由 VMCS 内的 guest-state 区域字段进行配置。在 VMX non-root operation 模式内，处理器的行为也由若干个 VM-execution 字段进行控制。

4.1 发起 VM-entry 操作

进入 VMX operation 模式后，下面的代码展示了进入 VM 的一系列工作：

```

mov esi, [VmcsA]
call initialize_vmcs_buffer

.....

;;
;; 初始化 VMCS
;;
vmclear [GuestA]
jc VmFailInvalid
jz VmFailValid

;;
;; 加载 VMCS
;;
vmptrld [GuestA]
jc VmFailInvalid
jz VmFailValid

... ..

;;
;; 配置 VMCS
;;
call setup_vmcs_region

;;
;; 发起 VM-entry
;;
call reset_guest_context
or DWORD [gs: PCB.ProcessorStatus], CPU_STATUS_GUEST
vmlaunch
jc VmFailInvalid
jz VmFailValid

```

使用 `initialize_vmcs_buffer` 初始化 VMCS buffer 后，后续需要使用 `setup_vmcs_region` 函数把 VMCS buffer 数据刷新到 VMCS 区域（参见 3.11.3 节）。

`VMCLEAR` 指令用来初始化提供的目标 VMCS，将处理器相关的数据写入目标 VMCS 区域不可见的字段里，同时将目标 VMCS 置为“**clear**”状态。这是首次发起 VM-entry 必要的工作。

`VMPTRLD` 指令将目标 VMCS 加载为 current-VMCS，并更新 current-VMCS pointer 值。成功执行完这条指令后使得当前 VMCS 指针有效（即存在当前 VMCS 指针），在这个基础上 VMM 执行一系列的 `VMWRITE` 指令对 VMCS 区域的各个字段逐一设置。上面代码示例中的 `setup_vmcs_region` 函数就是使用 `VMWRITE` 指令将 VMCS buffer 内的数据刷新到 VMCS 区域的。

最后，VMM 使用 `VMLAUNCH` 指令发起首次 VM-entry 操作，另一个指令

VMRESUME 也用来发起 VM-entry，但这两条指令在不同的场合下使用：

(1) VMLAUNCH 指令使用在首次进入 VM 时，current-VMCS 必须是“clear”状态。

(2) VMRESUME 指令使用在 VM 产生 VM-exit 后，VMM 重新发起 VM-entry 恢复 VM 的运行。因此，current-VMCS 必须是“launched”状态。

当执行 VMLAUNCH 或 VMRESUME 指令时，处理器会进行一系列复杂的检查，当所有检查通过后才允许切换到 VMX non-root operation 模式执行 guest 软件。

4.2 VM-entry 执行流程

进行 VM-entry 操作时处理器会执行严格的检查，可以分为以下 3 个阶段。

第 1 阶段：对 VMLAUNCH 或 VMRESUME 指令的执行进行基本检查。

第 2 阶段：对当前 VMCS 内的 VM-execution、VM-exit 以及 VM-entry 控制区域和 host-state 区域进行检查。

第 3 阶段：对当前 VMCS 内的 guest-state 区域进行检查，并加载 MSR。

在所有的检查都通过后，处理器从 guest-state 区域里加载处理器状态和执行环境信息。如果设置需要加载 MSR，则接着从 VM-entry MSR-load MSR 列表区域里加载 MSR。

VM-entry 操作附加动作会清由执行 MONITOR 指令而产生的地址监控，这个措施可以防止 guest 软件检测到自己处于虚拟机内。

在成功完成 VM-entry 后，如果注入了一个向量事件，则通过 guest-IDT 立即 deliver 这个向量事件执行。如果存在 pending debug exception 事件，则在注入事件完成后 deliver 一个 #DB 异常执行。

注意：在整个 VM-entry 操作流程里，如果 VM-entry 失败可能产生以下三种结果：

(1) VMLAUNCH 和 VMRESUME 指令产生异常，从而执行相应的异常服务例程。

(2) VMLAUNCH 和 VMRESUME 指令产生 VMfailInvalid 或 VMfailValid 类型失败，处理器接着执行下一条指令。

(3) VMLAUNCH 和 VMRESUME 指令的执行由于检查 guest-state 区域不通过，或者在加载 MSR 阶段失败而产生 VM-exit，从而转入 host-RIP 的入口点执行。

在前面所述的第 1 阶段检查里，可能会产生第 1 种或第 2 种结果。在第 2 阶段检查里，可能会产生第 2 种结果。在第 3 阶段检查里可能会产生第 3 种结果。

4.3 指令执行的基本检查

这是 VM-entry 操作流程里的第 1 阶段检查，涉及下面 5 个检查点：

(1) 是否允许执行 VMLAUNCH 和 VMRESUME 指令。

(2) 是否有权限执行 VMLAUNCH 和 VMRESUME 指令。

(3) current-VMCS 是否有效。

(4) 执行 VMLAUNCH 和 VMRESUME 指令时, 是否存在 “blocking by MOV-SS” 阻塞状态。

(5) current-VMCS 的状态是否符合 VMLAUNCH 和 VMRESUME 指令的要求。

如前面 4.2 节所述, 这个阶段的检查可能会产生异常, VMfailInvalid 或者 VMfailValid 失败。

1. 产生异常

当不允许执行 VMLAUNCH 或 VMRESUME 指令时, 会产生 #UD 异常; 当不够权限执行时, 会产生 #GP 异常。发生异常后转入执行相应的异常服务例程。

- 除了 VMXON 指令外, 其余所有 VMX 指令必须在 VMX 模式内执行。进入 VMX 模式时 CR0.PG 和 CR0.PE 必须为 1。因此 VMLAUNCH 和 VMRESUME 指令不能在**实模式**或者**未分页的保护模式**下执行, 也不能在 **virtual-8086** 或 **IA-32e 的兼容模式**下执行, 否则将产生 #UD 异常。
- 执行 VMLAUNCH 和 VMRESUME 指令必须在 0 级权限里, CPL 不为 0 时将产生 #GP 异常。

注意: VMLAUNCH 与 VMRESUME 指令仅能产生 #UD 或 #GP 这两类异常。

2. 产生 VMfailInvalid 失败

当 current-VMCS 无效时, 执行 VMLAUNCH 或 VMRESUME 指令将产生 VMfailInvalid 类型的失败。eflags.CF 被置位, 指示 current-VMCS 无效。处理器接着执行下一条指令。

current-VMCS 无效, 可能包括下面两种情形:

(1) 没有使用 VMPTRLD 指令加载 VMCS, 这时候不存在 current-VMCS。

(2) 在加载 VMCS 后, 执行 VMCLEAR 指令初始化 VMCS, 而提供的目标 VMCS 是 current-VMCS, 这时 current-VMCS pointer 会被置为 FFFFFFFF_FFFFFFFFH 值。

由于 current-VMCS 无效, 因此并没有指令错误码产生。

3. 产生 VMfailValid 失败

当 current-VMCS 有效时, 处理器按顺序检查, 遇到下面的情形会产生 VMfailValid 类型失败:

(1) 发起 VM-entry 时存在 “blocking by MOV-SS” 阻塞状态, 也就是说 VMLAUNCH 或 VMRESUME 指令在 MOV-SS 或 POP-SS 指令后面。

(2) 执行 VMLAUNCH 指令时, current-VMCS 为“非 clear”状态。

(3) 执行 VMRESUME 指令时, current-VMCS 为“非 launched”状态。

当产生 VMfailValid 失败后, 会在 VM-instruction error 字段里记录指令错误码 (参见 2.6.3 节)。并且 eflags.ZF 被置位, 处理器接着执行下一条指令。

因此, 在 VMLAUNCH 和 VMRESUME 指令后面需要检查指令是否执行成功。

vmlaunch	; 只有失败才会执行下面的指令
jc vmfailInvalid	; CF=1 时, 产生 vmfailInvalid 类型失败
jz vmfailValid	; ZF=1 时, 产生 vmfailValid 类型失败

类似上面的代码, VMLAUNCH 指令后面分别检查 eflags.CF 与 eflags.ZF 标志, 用来确定是否产生 VMfailInvalid 或 VMfailValid 失败。

4.4 检查控制区域及 host-state 区域

在第 2 阶段的检查里, 会分别检查 VM-execution、VM-exit、VM-entry 控制区域和 host-state 区域。当检查不通过时会产生 VMfailValid 失败并记录错误码, 处理器接着执行下一条指令。

注意: 在这个阶段不是按顺序检查的, 有可能是并行检查。host-state 区域检查不通过, 并不代表控制区域的检查通过。例如, 当检查到 VM-exit control 字段的“host address-space size”位时, 可能会一并检查 host-state 区域的 IA32_EFER 字段。发现 IA32_EFER 字段的 LME 和 LMA 位不等于“host address-space size”值时, 产生 VMfailValid 失败, 记录错误码为 8, 指示“进行 VM-entry 时, host-state 区域有无效字段”。

将 host-state 区域与控制区域放在一起检查, 是为了首先确保 host 执行环境是正确的, 当在第 3 阶段对 guest-state 区域检查不通过时, 可以产生 VM-exit 从而转入 host-RIP 入口执行。

4.4.1 VM-execution 控制区域检查

VM-execution 控制区域里有 3 个重要的控制字段:

- pin-based VM-execution control 字段
- primary processor-based VM-execution control 字段
- secondary processor-based VM-execution control 字段

大部分的检查都基于这 3 个控制字段。当 primary processor-based VM-execution control 字段的“activate secondary controls”位 (bit 31) 为 1 时, secondary processor-based VM-execution control 字段才有效。当“activate secondary controls”为 0 时, 所有 secondary processor-based VM-execution control 字段的控制位都被视为 0 值。

4.4.1.1 检查 pin-based VM-execution control 字段

- pin-based VM-execution control 字段的控制位设置必须符合 2.5.6.1 节所述，另外参见 3.5.1 节的表 3-5 所示。
- 当 secondary processor-based VM-execution control 字段的“virtual-interrupt delivery”位为 1 时，“external-interrupt exiting”位必须为 1。
- 当“NMI exiting”为 0 时，“virtual NMIs”位必须为 0。也就是“virtual NMIs”允许设为 1 值的前提是“NMI exiting”位必须为 1。
- 当“process posted interrupts”为 1 时，必须满足下面的所有条件。
 - secondary processor-based VM-execution control 字段的“virtual-interrupt delivery”必须为 1。因此，连带“external-interrupt exiting”必须为 1。
 - VM-exit control 字段的“acknowledge interrupt on exit”必须为 1。
 - posted-interrupt notification vector 字段提供一个中断向量号，从 0 到 255。
 - posted-interrupt descriptor address 字段提供的物理地址 bits 5:0 必须为 0，也就是必须对齐在 64 字节边界上。
 - posted-interrupt descriptor address 字段提供的物理地址宽度不能超过 MAXPHYADDR 值。例如，当 MAXPHYADDR 为 36 时，地址值的 bits 63:36 必须为 0。

4.4.1.2 检查 primary processor-based VM-execution control 字段

- primary processor-based VM-execution control 字段的控制位设置必须符合 2.5.6.2 节所述，另外参见 3.5.2.1 节的表 3-6 所示。
- 当“use TPR shadow”为 1 时，需要在 virtual-APIC address 字段里提供 virtual-APIC page 的物理地址。这个物理地址值 bits 11:0 必须为 0（即对齐在 4K 字节边界上），并且地址宽度不能超过 MAXPHYADDR 值。例如，当 MAXPHYADDR 为 36 时，地址值的 bits 63:36 必须为 0。
- 当“use TPR shadow”为 1，并且 secondary processor-based VM-execution control 字段的“virtual-interrupt delivery”为 0 时，TPR threshold 字段的 bits 31:4 必须为 0。
- 当“use TPR shadow”为 1，并且 secondary processor-based VM-execution control 字段的“virtualize APIC accesses”和“virtual-interrupt delivery”位都为 0 时，VPTR[7:4]必须不低于（大于或等于）TPR threshold 字段的 bits 3:0 值。
- 当“use TPR shadow”为 0 时，secondary processor-based VM-execution control 字段的“virtualize x2APIC mode”、“APIC-register virtualization”以及“virtual-interrupt delivery”都必须为 0。
- “NMI-window exiting”允许设为 1 值的前提是 pin-based VM-execution control 字段的“virtual NMIs”必须为 1，从而“NMI exiting”也必须为 1。
- 当“use I/O bitmaps”为 1 时，需要在 I/O-bitmap address 字段 A 和 B 里提供两个 4K 的物理页面地址。地址值的 bits 11:0 必须为 0（对齐在 4K 字节边界），并且地址的宽度不能超过 MAXPHYADDR 值。例如，当 MAXPHYADDR 为 36 时，地

址值的 bits 63:36 必须为 0。

- 当“use MSR bitmaps”为 1 时，需要在 MSR-bitmap address 字段里提供 4K 的物理页面地址。地址值的 bits 11:0 必须为 0（对齐在 4K 字节边界），并且地址的宽度不能超过 MAXPHYADDR 值。例如，当 MAXPHYADDR 为 36 时，地址值的 bits 63:36 必须为 0。

4.4.1.3 检查 secondary processor-based VM-execution control 字段

- secondary processor-based VM-execution control 字段的控制位设置必须符合 2.5.6.3 节所述。另外参见 3.5.2.2 节的表 3-7 所示。
- 当“virtualize APIC-accesses”为 1 时，需要在 APIC-access address 字段里提供 APIC-access page 页面物理地址。地址值的 bits 11:0 必须为 0（对齐在 4K 字节边界），并且地址的宽度不能超过 MAXPHYADDR 值。例如，当 MAXPHYADDR 为 36 时，地址值的 bits 63:36 必须为 0。
- 当“enable EPT”为 1 时，需要在 EPTP 字段里提供一个 EPTP（extended-page table pointer）值，这个 EPTP 值必须满足下面的条件。
 - Bits 2:0 设置的内存类型值必须是 VMX 支持的，可由 IA32_VMX_EPT_VPID_CAP 寄存器查询（参见 2.5.13 节）。当前 VMX 架构支持 0（Uncacheable）和 6（WriteBack）值。
 - Bits 5:3 的值必须是 3，表明使用 4 级 EPT paging-structure 结构。
 - Bit 6 的值在不支持 EPT 页表项的 accessed 和 dirty 标志位时必须为 0。可由 IA32_VMX_EPT_VPID_CAP 寄存器的 bit 21 查询（参见 2.5.13 节）。
 - Bits 11:7 以及 bits 63:*N* 保留位的值必须为 0。这里的 *N* 值等于 MAXPHYADDR 值，例如，MAXPHYADDR 为 36 时，*N* 的值为 36。
- 当“virtualize x2APIC mode”为 1 时，“virtualize APIC-accesses”必须为 0。
- 当“enable VPID”为 1 时，VPID 字段的值不能为 0。
- 当“unrestricted guest”为 1 时，“enable EPT”必须为 1，从而 EPTP 字段必须提供一个 EPTP 值（参见上面所述）。
- 在 primary processor-based VM-execution control 字段的“use TPR shadow”为 0 时，“virtualize x2APIC mode”、“APIC-register virtualization”以及“virtual-interrupt delivery”都必须为 0。
- 当“enable VM functions”为 1 时，VM-function control 字段保留位必须为 0，允许设置为 1 的位必须符合 2.5.9 节所述。当“enable VM functions”为 0 时，不检查 VM-function control 字段。
- VM-function control 字段的“EPTP switching”位允许设置为 1，前提是“enable VM functions”和 primary processor-based VM-execution control 字段的“activate secondary controls”都为 1。
- 当 VM-function control 字段“EPTP switching”为 1 时，需要在 EPTP-list address 字段里提供 4K 页面物理地址。地址值的 bits 11:0 必须为 0（对齐在 4K 字节边

界)，并且地址的宽度不能超过 MAXPHYADDR 值。例如，当 MAXPHYADDR 为 36 时，地址值的 bits 63:36 必须为 0。

4.4.1.4 检查 CR3-target 字段

CR3-target count 字段的值不能大于 4，当前 VMX 实现下仅支持最多 4 个 CR3-target value 字段。这个最大值可由 IA32_VMX_MISC 寄存器的 bits 24:16 查询获得（见 2.5.11 节）。

4.4.2 VM-exit 控制区域检查

这个区域的检查分两个方面：VM-exit control 字段检查，MSR-store 与 MSR-load 相关字段的检查。

4.4.2.1 VM-exit control 字段的检查

- VM-exit control 字段的控制位设置必须符合 2.5.7 节所述，另外参见 3.7.1 节表 3-11 所示。
- “host address-space size” 的值必须等于当前 IA32_EFER.LMA 的值。也就是“host address-space size” 的值取决于发起 VM-entry 时当前处理器是否运行在 IA-32e 模式。
- 在 pin-based VM-execution control 字段的“process posted-interrupts” 为 1 时，“acknowledge interrupt on exit” 位必须为 1。
- 在 pin-based VM-execution control 字段的“activate VMX-preemption timer” 为 0 时，“save VMX-preemption timer value” 必须为 0。

4.4.2.2 MSR-store 与 MSR-load 相关字段的检查

- 当 VM-exit MSR-store count 字段值不为 0 时，必须满足以下条件：
 - VM-exit MSR-store address 字段里的物理地址 Bits 3:0 必须为 0（对齐在 16 字节边界上），并且地址宽度不能超过 MAXPHYADDR 值（bits 63:N 必须为 0，N=MAXPHYADDR）。
 - VM-exit MSR-store 列表区域的最后一个字节的地址不能超过 MAXPHYADDR 值。最后一个字节的地址值=address + count×16 - 1。
- 当 VM-exit MSR-load count 字段值不为 0 时，必须满足下面的条件：
 - VM-exit MSR-load address 字段里的物理地址 Bits 3:0 必须为 0（对齐在 16 字节边界上），并且地址宽度不能超过 MAXPHYADDR 值（bits 63:N 必须为 0，N=MAXPHYADDR）。
 - VM-exit MSR-load 列表区域的最后一个字节的地址不能超过 MAXPHYADDR 值。最后一个字节的地址值=address + count×16 - 1。

注意：处理器并不检查 MSR-store count 与 MSR-load count 字段的值是否超过 IA32_VMX_MISC 寄存器 bits 27:25 位域推荐的 count 数量（见 2.5.11 节）。但是，如果超过了可能会产生不可预测的结果。

4.4.3 VM-entry 控制区域检查

这个区域的检查包括三个方面：VM-entry control 字段的检查、MSR-load 相关字段的检查，以及事件注入相关字段的检查。

4.4.3.1 VM-entry control 字段的检查

- VM-entry control 字段的控制位设置必须符合 2.5.8 节所述，另参见 3.6.1 节表 3-9 所示。
- 发起 VM-entry 时，若当前 **IA32_EFER.LMA** 为 0，“IA-32e mode guest”位必须为 0。
- 发起 VM-entry 时，若当前处理器不处于 SMM 模式，“entry to SMM”和“deactivate dual-monitor treatment”位必须为 0。
- “entry to SMM”与“deactivate dual-monitor treatment”位不能同时为 1。

4.4.3.2 MSR-load 相关字段的检查

- 当 VM-entry MSR-load count 字段值不为 0 时，必须满足以下条件：
 - VM-entry MSR-load address 字段里的物理地址 Bits 3:0 必须为 0（对齐在 16 字节边界上），并且地址宽度不能超过 MAXPHYADDR 值（bits 63:N 必须为 0，N=MAXPHYADDR）。
 - VM-entry MSR-load 列表区域的最后一个字节的地址不能超过 MAXPHYADDR 值。最后一个字节的地址值=address + count×16 - 1。

注意：处理器并不检查 MSR-load count 字段的值是否超过 IA32_VMX_MISC 寄存器 bits 27:25 位域推荐的 count 数量（见 2.5.11 节）。但是，如果超过了可能会产生不可预测的结果。

4.4.3.3 事件注入相关字段的检查

- 当 VM-entry interruption-information 字段的 bit 31 为 1 时表示有效，这个字段的设置必须满足以下条件：
 - Bits 10:8（中断类型）的值不能为 1（保留值）。
 - 当 primary processor-based VM-execution control 字段的“monitor trap flag”位不允许设为 1 值时，bits 10:8 的值不能为 7（other event）。
 - 当 bits 10:8（中断类型）为 **NMI** 时，bits 7:0（中断向量号）的值必须为 2。
 - 当 bits 10:8（中断类型）为**硬件异常**时，bits 7:0（中断向量号）的值最大为 31。
 - 当 bits 10:8（中断类型）为**其他事件**时，bits 7:0（中断向量号）的值必须为 0。
 - 当 guest 运行在实模式（即 secondary processor-based VM-execution control 字段的“unrestricted guest”为 1，并且 guest-CR0 字段的 PE 位为 0）时，bit 11（error code）必须为 0。
 - 当中断类型不是硬件异常时，bit 11（error code）必须为 0。
 - 当中断类型为硬件异常，并且向量号为 8（#DF）、10（#TS）、11（#NP）、

12 (#SS)、13 (#GP)、14 (#PF) 以及 17 (#AC) 时, bit 11 (error code) 必须为 1。反之, 向量号为其他值时, bit 11 必须为 0。

➤ Bits 30:12 (保留位) 必须为 0。

- 当 VM-entry interruption-information 字段的 bit 11 (error code) 为 1 时, VM-entry exception error code 字段的 **bits 31:16** 必须为 0 (只允许 **16 位的错误码**)。
- 当 VM-entry interruption-information 字段 bits 10:8 的值为 4 (软件中断)、5 (特权级软件中断) 或者 6 (软件异常) 时, 必须提供指令长度。VM-entry instruction length 字段的值必须在 1 到 15 之间 (它们属于**同步事件**, 参见 4.12 节描述)。

4.4.4 Host-state 区域的检查

Host-state 区域的字段分为 5 个部分: 控制寄存器字段、指针字段、段选择子字段、段基址字段以及 MSR 字段 (见 3.9 节表 3-21)。在发生 VM-exit 后这些字段值将被加载到处理器相应的寄存器里, 因此, 处理器需要检查这些字段值是否适合 host 的运行环境。

4.4.4.1 Host 控制寄存器字段的检查

- 在 64 位架构处理器上 CR0 与 CR4 字段为 64 位宽, CR0 与 CR4 字段的 bits 63:32 位必须为 0。
- CR0 字段的值必须符合 VMX 架构对 CR0 寄存器的设置要求 (见 2.5.10 节所述)。当前 VMX 架构下需要 CR0.PE、CR0.NE 以及 CR0.PG 都必须为 1。
- CR4 字段的值必须符合 VMX 架构对 CR4 寄存器的设置要求 (见 2.5.10 节所述)。当前 VMX 架构下需要 CR4.VMXE 必须为 1, bits 12:11 以及 bits 31:14 必须为 0。
- 当 VM-exit control 字段的 “host address-space size” 为 0 时, CR4 字段的 bit 17 (CR4.PCIDE) 必须为 0。
- 当 VM-exit control 字段的 “host address-space size” 为 1 时, CR4 字段的 bit 5 (CR4.PAE) 必须为 1。
- CR3 字段在 64 位架构处理器上是 64 位宽, 因此, CR3 字段的 bits 63:*N* 位必须为 0, *N* 值等于 MAXPHYADDR。如果 MAXPHYADDR 为 36, 则 CR3 字段的 bits 63:36 必须为 0。

4.4.4.2 Host-RIP 的检查

在 64 位架构处理器上 (RIP 字段为 64 位宽) 需要满足下面的条件:

- 当 VM-exit control 字段的 “host address-space size” 为 0 时, RIP 字段的 bits 63:32 必须为 0。
- 当 VM-exit control 字段的 “host address-space size” 为 1 时, RIP 字段的地址值必须是 canonical 形式 (即地址值的 bits 63:48 必须是 bit 47 的符号扩展)。

注意: 处理器并不对 host-RSP 字段进行检查。

4.4.4.3 段 selector 字段的检查

VMM (host) 必须运行在 0 级权限里，段 selector 字段需要满足下面的条件：

- ES、CS、SS、DS、FS、GS 以及 TR 寄存器的 selector 字段 bits 2:0 必须为 0（即 TI 与 RPL 位必须为 0）。
- CS 与 TR 寄存器的 selector 字段 bits 15:3 不能为 0（即 index 域不能为 0），即 CS 与 TR 寄存器的 selector 字段不能为 0 值。
- 当 VM-exit control 字段的“host address-space size”为 0 时，SS selector 字段的 bits 15:3 不能为 0（index 域不能为 0），也就是 SS selector 字段的值不能为 0。

注意：当 VM-exit control 字段的“host address-space size”为 1 时，SS selector 字段的值允许为 0。即返回到 IA-32e 模式的 host 时，SS selector 可以为“NULL-selector”。

4.4.4.4 段基址字段的检查

在 64 位架构处理器上，FS、GS、TR、GDTR 以及 IDTR 字段的地址值必须为 canonical 形式。即地址值的 bits 63:48 必须是 bit 47 的符号扩展。

4.4.4.5 MSR 字段的检查

- 在 64 位架构处理器上，IA32_SYSENTER_ESP 与 IA32_SYSENTER_EIP 字段的地址值必须为 canonical 形式。即地址值的 bits 63:48 必须是 bit 47 的符号扩展。
- 当 VM-exit control 字段的“load IA32_PERF_GLOBAL_CTRL”为 1 时，IA32_PERF_GLOBAL_CTRL 字段的保留位必须为 0（即 bits 31:N 与 bits 63:35 必须为 0，这个 N 值等于 IA32_PMCx 计数器的个数）。
- 当 VM-exit control 字段的“load IA32_PAT”为 1 时，IA32_PAT 字段的 bits 7:0 不能为保留值（即不能为 02H、03H 以及 08H 到 FFH）。
- 当 VM-exit control 字段的“load IA32_EFER”为 1 时，对 IA32_EFER 字段进行下面的检查：
 - 保留位（bits 7:1，bit 9 以及 bits 63:12）必须为 0。
 - LMA 位（bit 10）与 LME 位（bit 8）必须等于 VM-exit control 字段“host address-space size”位的值。

4.5 检查 guest-state 区域

当第 2 阶段对控制区域及 host-state 区域检查通过后，处理器接着检查第 3 阶段的 guest-state 区域。如果 guest-state 区域检查失败将产生 VM-exit，处理器将从 host-state 区域加载处理器的 host 执行环境，并转入执行 host-RIP 提供的入口代码。

当 guest-state 区域检查通过后，处理器将从 guest-state 区域加载 guest 执行环境并转入 guest-RIP 提供的入口代码。另外处理器将清由执行 MONITOR 指令而产生的地址监控。

4.5.1 检查控制寄存器字段

处理器对 guest-state 区域的 CR3 与 CR4 字段的检查与 host-state 区域一致。而 CR0 字段的检查受到 secondary processor-based VM-execution control 字段的“**unrestricted guest**”位和 VM-entry control 字段的“**IA-32e mode guest**”位影响。

- 处理器执行下面与“unrestricted guest”位对应的检查：
 - “unrestricted guest”为 0 时，CR0 字段的值必须符合 VMX 架构对 CR0 寄存器的设置要求（见 2.5.10 节所述）。即 CR0 字段的 PE、PG 和 NE 位都必须为 1。
 - “unrestricted guest”为 1 时，处理器忽略 CR0 字段的 PE 与 PG 位检查。但 PG 位为 1 时，PE 位必须为 1。
- 处理器执行下面与“IA-32e mode guest”位对应的检查：
 - “IA-32e mode guest”为 1 时，CR0 字段的 PE 与 PG 位必须为 1。
 - “IA-32e mode guest”为 1 时，CR4 字段的 PAE 位必须为 1。
 - “IA-32e mode guest”为 0 时，CR4 字段的 PCIDE 位必须为 0。
- 当“unrestricted guest”与“IA-32e mode guest”同时为 1 时，处理器将执行它们各自对应的检查。CR0 字段的 PE 与 PG 位必须为 1，可以有下面的理解（并不矛盾）：
 - 在执行“unrestricted guest”对应的检查时，PE 与 PG 位忽略。
 - 在执行“IA-32e mode guest”对应的检查时，PE 与 PG 位必须为 1。
- 处理器从不检查 CR0 字段的 NW 与 CD 位。在 VM-entry 加载 guest CR0 字段时，这些位被忽略，即 CR0.NW 与 CR0.CD 保持不变。
- CR4 字段的值必须符合 VMX 架构对 CR4 寄存器的设置要求（见 2.5.10 节所述）。当前 VMX 架构下需要 CR4.VMXE 必须为 1，bits 12:11 以及 bits 31:14 必须为 0。
- CR3 字段在 64 位架构处理器上是 64 位宽，因此 CR3 字段的 bits 63:*N* 位必须为 0，这个 *N* 值等于 MAXPHYADDR。例如 MAXPHYADDR 为 36，则 CR3 字段的 bits 63:36 必须为 0。

注意：在上面所列的其中一种情况：当“unrestricted guest”与“IA-32e mode guest”同时为 1 时，代表 guest 的执行环境是 IA-32e 模式。因此，处理器检查 guest 字段必须符合 IA-32e 模式的设置要求。

4.5.2 检查 RIP 与 RFLAGS 字段

在 64 位架构处理器上 RIP 与 RFLAGS 字段为 64 位，否则为 32 位。在 64 位架构处理器上有下面的检查：

- 当“IA-32e mode guest”为 0，或者 CS access right 字段的 L 位为 0 时（对应 CS.L 为 0 时进入 compatibility 模式），RIP 字段的 bits 63:32 必须为 0，即 guest 处于非 64 位模式时 bits 63:32 必须为 0。

- RIP 字段的地址值必须是 canonical 形式，即地址值的 bits 63:48 是 bit 47 的符号扩展。
- RFLAGS 字段的 bits 63:22 (32 位处理器上是 bits 31:22)，bit3、bit5 以及 bit15 必须为 0，而 bit1 必须为 1。
- 当“IA-32e mode guest”为 1，或者 CR0 字段对应的 CR0.PE 为 0 时，RFLAGS 字段的 VM 位必须为 0（因为 virtual-8086 模式必须运行在 legacy 保护模式内）。
- 当 VM-entry interruption information 字段的 bit 31 为 1，并且 bits 10:8 为 0（表明注入一个外部中断事件）时，RFLAGS 字段的 IF 位必须为 1（允许响应中断请求）。

4.5.3 检查 DR7 与 IA32_DEBUGCTL 字段

- 在 64 位架构处理器上，当 VM-entry control 字段的“load debug controls”为 1 时，DR7 字段的 bits 63:32 必须为 0。
- 当 VM-entry control 字段的“load debug controls”为 1 时，IA32_DEBUGCTL 字段对应的保留位 (bits 63:15 与 bits 5:2) 必须为 0。

4.5.4 检查段寄存器字段

这里检查段寄存器相关的字段包括 ES、CS、SS、DS、FS、GS、TR 以及 LDTR 寄存器的 selector 字段、base 字段、limit 字段和 access rights 字段。

这些字段的检查受到 secondary processor-based VM-execution 字段的“unrestricted guest”位，VM-entry control 字段的“IA-32e mode guest”位，以及 guest-state 区域的 CR0 字段与 RFLAGS 字段影响。由于存在“unrestricted guest”与“IA-32e mode guest”位可能同时为 1，所以 guest 处于哪个模式取决于下面的设置。

(1) **Virtual-8086 模式**：在“IA-32e mode guest”为 0，并且 CR0 字段的 PE 位为 1 的前提下，当 RFLAGS 字段的 bit 17 (VM 标志位) 为 1 时，表示进入 virtual-8086 模式 guest。处理器必须运行在 3 级权限的 legacy 保护模式下。

(2) **IA-32e 模式**：当“IA-32e mode guest”为 1 时，表示进入 IA-32e 模式 guest。

(3) **实模式**：当“IA-32e mode guest”为 0，“unrestricted guest”为 1，并且 CR0 字段的 PE 为 0 时，表示进入实模式 guest。

(4) **legacy 保护模式**：当“IA-32e mode guest”为 0，并且 CR0 字段的 PE 为 1 时，表示进入 legacy 保护模式 guest。

但是，上面列举的第 3 点（实模式）与第 4 点（legacy 保护模式）不作为检查的依据。处理器主要依赖“unrestricted guest”与“IA-32e mode guest”位，以及 CS access right 字段中的 type (bits 3:0) 位作为检查准则。

处理器对段寄存器相关字段的检查，还取决于该段寄存器是否为 usable（可用）。

- **usable**: 段寄存器的 access right 字段 unusable 位 (bit 16) 为 0 时, 表示可用。
- **unusable**: 段寄存器的 access right 字段 unusable 位 (bit 16) 为 1 时, 表示不可用。当段寄存器属于 unusable 时, 相当于段寄存器在 guest 里被加载一个 NULL selector。
- **TR** 寄存器必须是 usable。

在 x64 架构体系的 64 位模式里, 除 CS 寄存器外 NULL selector 允许被加载 SS (必须是非 3 级), ES、DS、FS 以及 GS 寄存器。但在 VMX 架构下, 进入 guest 后 CS、SS、ES、DS、FS 以及 GS 寄存器都可以被加载为 unusable (不可用)。

TR 寄存器必须是可用的, LDTR 寄存器允许为不可用的。如果段寄存器属于不可用, 该段寄存器的某些字段和位不会检查, 但某些相关字段和位是必须检查的 (见第 4.5.4.1、4.5.4.2 及 4.5.4.3 节所述)。

4.5.4.1 virtual-8086 模式下的检查

1. selector 字段的检查

- TR selector 字段的 TI 位 (bit 2) 必须为 0, 表明必须使用 GDT。
- 如果 LDTR 寄存器是可用的, selector 字段的 TI 位 (bit 2) 必须为 0。

注意: 在 virtual-8086 模式下, 其余段寄存器的 selector 字段不检查。

2. base 字段的检查

- ES、CS、SS、DS、FS 以及 GS 寄存器的 base 字段值必须等于 $\text{selector} \ll 4$ 。
- 在 64 位架构处理器上, 对 TR 与 LDTR 寄存器进行下面的检查:
 - TR base 字段的地址值必须为 canonical 形式 (bits 63:48 是 bit 47 的符号扩展)。
 - 如果 LDTR 寄存器是可用的, LDTR base 字段的地址值必须是 canonical 形式。

3. limit 字段的检查

- ES、CS、SS、DS、FS 以及 GS 寄存器的 limit 字段值必须为 0000FFFFh (64K 段限)。
- 对 TR 的 limit 字段和 LDTR 寄存器在可用时 (unusable 位为 0) 的 limit 字段进行下面的检查:
 - 段寄存器 access right 字段的 G 位 (bit 15) 为 0 时, 该段寄存器 limit 字段值必须小于等于 000FFFFh 值。
 - 段寄存器 access right 字段的 G 位 (bit 15) 为 1 时, 该段寄存器 limit 字段值必须大于等于 00000FFFh 值。

4. access right 字段的检查

- 在 virtual-8086 模式下, ES、CS、SS、DS、FS 以及 GS 寄存器必须是可用的。所有的 access right 字段的值固定为 000000F3h, 它的各个位域值为:
 - Type 值 (bits 3:0) 必须为 3。

- S 位 (bit 4) 必须为 1。
- DPL 值 (bits 6:5) 必须为 3。
- P 位 (bit 7) 必须为 1。
- unusable 位 (bit 16) 必须为 0。
- AVL 位 (bit 12)、L 位 (bit 13)、D/B 位 (bit 14)，以及 G 位 (bit 15) 必须为 0。
- 保留位 (bits 11:8 与 bits 31:17) 必须为 0。
- TR 寄存器必须是可用的，对它进行下面的检查：
 - Type 值 (bits 3:0) 必须为 3 或 11。
 - S 位 (bit 4) 必须为 0。
 - P 位 (bit 7) 必须为 1。
 - 当 limit 字段值小于 00000FFFh 时，G 位 (bit 15) 必须为 0。
 - 当 limit 字段值大于 000FFFFFFh 时，G 位 (bit 15) 必须为 1。
 - unusable 位 (bit 16) 必须为 0。
 - 保留位 (bits 11:8 与 bits 31:17) 必须为 0。
- LDTR 寄存器在 unusable 位 (bit 16) 为 0 时 (可用的)，对它进行下面的检查：
 - Type 值 (bits 3:0) 必须为 2。
 - S 位 (bit 4) 必须为 0。
 - P 位 (bit 7) 必须为 1。
 - 当 limit 字段值小于 00000FFFh 时，G 位 (bit 15) 必须为 0。
 - 当 limit 字段值大于 000FFFFFFh 时，G 位 (bit 15) 必须为 1。
 - 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

4.5.4.2 unrestricted guest 位为 0 时的检查

“unrestricted guest”为 0 时，包括了 IA-32e 模式与 legacy 保护模式。某些位的检查也取决于 guest 是否为 IA-32e 模式。

1. selector 字段的检查

- TR selector 字段的 TI 位 (bit 2) 必须为 0。
- 如果 LDTR 是可用的，selector 字段的 TI 位 (bit 2) 必须为 0。
- SS selector 字段的 RPL 值 (bits 1:0) 必须等于 CS selector 字段的 RPL 值。
- 如果 ES、DS、FS 以及 GS 寄存器是可用的，selector 字段的 RPL 值 (bits 1:0) 必须小于等于它们的 access right 字段 DPL 值 (bits 6:5)。

2. base 字段的检查

在 VMX 架构里，支持 64 位架构处理器的 base 字段是 64 位宽。而在 x64 体系中，ES、CS、SS 以及 DS 段寄存器的 base 域无效，不能加载 64 位的地址值。

因此，在 64 位架构处理器上，有下面的检查 (不管是否为 IA-32e 模式 guest)：

- CS base 字段的 bits 63:32 必须为 0（不管是否为可用的）。
- 如果 ES、SS 和 DS 寄存器是可用的，base 字段的 bits 63:32 必须为 0。
- TR base 字段的地址值必须是 canonical 形式（即地址值的 bits 63:48 是 bit 47 的符号扩展）。
- 如果 LDTR 寄存器是可用的，base 字段的地址值必须是 canonical 形式。
- FS 与 GS 寄存器 base 字段的地址值必须是 canonical 形式（不管是否为可用的）。

3. limit 字段的检查

由于段寄存器内 attribute 域的 G 位指示了段限的粒度，G 为 1 时是 4K 字节粒度，G 为 0 时是 1 字节粒度，因此 G 位与 limit 是相互影响的。

对于 CS 与 TR 寄存器，以及 ES、SS、DS、FS、GS、LDTR 寄存器为可用时，有下面的检查（CS 不管是否为可用）：

- 段寄存器 access right 字段的 G 位（bit 15）为 0 时，该段寄存器 limit 字段值必须小于等于 000FFFFFFh 值。
- 段寄存器 access right 字段的 G 位（bit 15）为 1 时，该段寄存器 limit 字段值必须大于等于 00000FFFh 值。

4. access right 字段的检查

对这些段寄存器 access right 字段的检查需要区分具体的段寄存器，TR 寄存器必须是可用的（unusable 位为 0），其余的 ES、CS、SS、DS、FS、GS 以及 LDTR 寄存器 unusable 位可以为 1。处理器执行下面几种情况的检查。

(1) CS 寄存器

- Type 值（bits 3:0）必须是 9、11、13 或者 15 其中之一。
- S 位（bit 4）必须为 1。
- 当 type 值为 9 或 11 时，DPL（bits 6:5）必须等于 SS access right 字段的 DPL 值。
- 当 type 值为 13 或 15 时，DPL（bits 6:5）必须小于等于 SS access right 字段的 DPL 值。
- P 位（bit 7）必须为 1。
- 当 guest 运行在 IA-32e 模式里，并且 L 位（bit 13）为 1 时，D/B 位（bit 14）必须为 0。
- 当 limit 字段值小于 00000FFFh 时，G 位（bit 15）必须为 0。
- 当 limit 字段值大于 000FFFFFFh 时，G 位（bit 15）必须为 1。
- 保留位（bits 11:8 与 bits 31:17）必须为 0。

(2) SS 寄存器

- DPL（bits 6:5）必须等于 SS selector 字段的 RPL 值。
- 如果 unusable 位（bit 16）为 0（SS 寄存器是可用的），将进行下面的检查，否则无须检查（unusable 位为 1 时）：

- Type 值 (bits 3:0) 必须为 3 或 7。
- S 位 (bit 4) 必须为 1。
- P 位 (bit 7) 必须为 1。
- 当 limit 字段值小于 00000FFFh 时, G 位 (bit 15) 必须为 0。
- 当 limit 字段值大于 000FFFFFFh 时, G 位 (bit 15) 必须为 1。
- 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

(3) ES、DS、FS 及 GS 寄存器

- 当 unusable 位 (bit 16) 为 0 时 (ES、DS、FS 及 GS 寄存器是可用的), 将进行下面的检查, 否则无须检查:
 - Type (bits 3:0) 这 4 位值必须是 **0xx1B** 或者 **1x11B** (二进制数, 其中 x 为 0 或 1) 其中之一, 即 bit 0 必须为 1 (已访问)。当 Bit 3 为 1 时 (代码段), bit 1 也必须为 1 (可读的)。
 - S 位 (bit 4) 必须为 1。
 - 如果 type 是 **0xx1B** 或 **1011B** 其中之一 (不属于 **conforming** 代码段), DPL (bits 6:5) 必须大于等于它们的 selector 字段 RPL 值。
 - P 位 (bit 7) 必须为 1。
 - 当 limit 字段值小于 00000FFFh 时, G 位 (bit 15) 必须为 0。
 - 当 limit 字段值大于 000FFFFFFh 时, G 位 (bit 15) 必须为 1。
 - 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

(4) TR 寄存器

- guest 运行在 IA-32e 模式时, type 值 (bits 3:0) 必须为 11。否则可以为 3 或 11。
- S 位 (bit 4) 必须为 0。
- P 位 (bit 7) 必须为 1。
- 当 limit 字段值小于 00000FFFh 时, G 位 (bit 15) 必须为 0。
- 当 limit 字段值大于 000FFFFFFh 时, G 位 (bit 15) 必须为 1。
- unusable 位 (bit 16) 必须为 0。
- 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

(5) LDTR 寄存器

- 当 unusable 位 (bit 16) 为 0 时 (LDTR 寄存器是可用的), 将进行下面的检查, 否则无须检查:
 - Type 值 (bits 3:0) 必须为 2。
 - S 位 (bit 4) 必须为 0。
 - P 位 (bit 7) 必须为 1。
 - 当 limit 字段值小于 00000FFFh 时, G 位 (bit 15) 必须为 0。
 - 当 limit 字段值大于 000FFFFFFh 时, G 位 (bit 15) 必须为 1。
 - 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

4.5.4.3 unrestricted guest 位为 1 时的检查

当“unrestricted guest”为 1 时，表明 guest 允许进入实模式。guest 具体运行在什么模式取决于“IA-32e mode guest”位，以及 guest-state 的 CR0 与 RFLAGS 字段（参见 4.5.4 节所述）。

1. selector 字段的检查

- TR selector 字段的 TI 位 (bit 2) 必须为 0。
- 如果 LDTR 是可用的，selector 字段的 TI 位 (bit 2) 必须为 0。

注意：其余的 ES、CS、SS、DS、FS 以及 GS 寄存器的 selector 不需要检查（不管是否可用）。

2. base 字段的检查

在 64 位架构处理器上，base 字段为 64 位宽。有下面的检查（不管是否为 IA-32e 模式 guest）：

- CS base 字段的 bits 63:32 必须为 0。
- 如果 ES、SS 以及 DS 寄存器是可用的，base 字段的 bits 63:32 必须为 0。
- TR base 字段的地址值必须是 canonical 形式（即地址值的 bits 63:48 是 bit 47 的符号扩展）。
- 如果 LDTR 寄存器是可用的，base 字段的地址值必须是 canonical 形式。
- FS 与 GS 寄存器 base 字段的地址值必须是 canonical 形式（不管是否为可用的）。

3. limit 字段的检查

由于段寄存器内 attribute 域的 G 位指示了段限的粒度，G 为 1 时是 4K 字节粒度，G 为 0 时是 1 字节粒度。因此 G 位与 limit 是相互影响的。

对于 CS 与 TR 寄存器，以及 ES、SS、DS、FS、GS，LDTR 寄存器在属于可用时，有下面的检查（CS 不管是否为可用）：

- 段寄存器 access right 字段的 G 位 (bit 15) 为 0 时，该段寄存器 limit 字段值必须小于等于 000FFFFh 值。
- 段寄存器 access right 字段的 G 位 (bit 15) 为 1 时，该段寄存器 limit 字段值必须大于等于 00000FFFh 值。

4. access right 字段的检查

对这些段寄存器 access right 字段的检查需要区分具体的段寄存器，TR 寄存器必须是可用的 (unusable 位为 0)，其余的 ES、CS、SS、DS、FS、GS 以及 LDTR 寄存器 unusable 位允许为 1。处理器执行下面几种情况的检查。

(1) CS 寄存器

- Type 值 (bits 3:0) 必须是 3、9、11、13 或者 15 其中之一。
- S 位 (bit 4) 必须为 1。

- 当 type 值为 3 时, DPL (bits 6:5) 必须为 0 值。
- 当 type 值为 9 或 11 时, DPL (bits 6:5) 必须等于 SS access right 字段的 DPL 值。
- 当 type 值为 13 或 15 时, DPL (bits 6:5) 必须小于等于 SS access right 字段的 DPL 值。
- P 位 (bit 7) 必须为 1。
- 当 guest 运行在 IA-32e 模式里, 并且 L 位 (bit 13) 为 1 时, D/B 位 (bit 14) 必须为 0。
- 当 limit 字段值小于 00000FFFh 时, G 位 (bit 15) 必须为 0。
- 当 limit 字段值大于 000FFFFFFh 时, G 位 (bit 15) 必须为 1。
- 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

(2) SS 寄存器

- 如果 CS access right 字段的 type 值为 3, DPL 值 (bits 6:5) 必须为 0。
- 如果 guest-CR0 字段的 PE 位为 0 (实模式), DPL 值 (bits 6:5) 必须为 0。
- 如果 unusable 位 (bit 16) 为 0 (SS 寄存器是可用的), 将进行下面的检查。否则无须检查 (unusable 位为 1) :
 - Type (bits 3:0) 必须为 3 或 7。
 - S 位 (bit 4) 必须为 1。
 - P 位 (bit 7) 必须为 1。
 - 当 limit 字段值小于 00000FFFh 时, G 位 (bit 15) 必须为 0。
 - 当 limit 字段值大于 000FFFFFFh 时, G 位 (bit 15) 必须为 1。
 - 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

(3) ES、DS、FS 及 GS 寄存器

- 如果 unusable 位 (bit 16) 为 0 (ES、DS、FS 及 GS 寄存器是可用的), 将进行下面的检查, 否则无须检查:
 - Type (bits 3:0) 这 4 位值必须是 **0xx1B** 或者 **1x11B** (二进制数, 其中 x 为 0 或 1) 其中之一。即 bit 0 必须为 1 (已访问)。当 Bit 3 为 1 时 (代码段), bit 1 也必须为 1 (可读的)。
 - S 位 (bit 4) 必须为 1。
 - P 位 (bit 7) 必须为 1。
 - 当 limit 字段值小于 00000FFFh 时, G 位 (bit 15) 必须为 0。
 - 当 limit 字段值大于 000FFFFFFh 时, G 位 (bit 15) 必须为 1。
 - 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

注意: 这里不需要检查 DPL 值 (bits 6:5)。

(4) TR 寄存器

- guest 运行在 IA-32e 模式时, Type 值 (bits 3:0) 必须为 11。否则可以为 3 或 11。
- S 位 (bit 4) 必须为 0。

- P 位 (bit 7) 必须为 1。
- 当 limit 字段值小于 00000FFFh 时, G 位 (bit 15) 必须为 0。
- 当 limit 字段值大于 000FFFFFFh 时, G 位 (bit 15) 必须为 1。
- unusable 位 (bit 16) 必须为 0。
- 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

(5) LDTR 寄存器

- 当 unusable 位 (bit 16) 为 0 时 (LDTR 寄存器是可用的), 将进行下面的检查, 否则无须检查:
 - Type 值 (bits 3:0) 必须为 2。
 - S 位 (bit 4) 必须为 0。
 - P 位 (bit 7) 必须为 1。
 - 当 limit 字段值小于 00000FFFh 时, G 位 (bit 15) 必须为 0。
 - 当 limit 字段值大于 000FFFFFFh 时, G 位 (bit 15) 必须为 1。
 - 保留位 (bits 11:8 与 bits 31:17) 必须为 0。

4.5.5 检查 GDTR 与 IDTR 字段

- 在 64 位架构处理器上, GDTR 与 IDTR 寄存器的 base 字段为 64 位宽。因此, 地址值必须是 canonical 形式。
- GDTR 与 IDTR 寄存器的 limit 字段 bits 31:16 必须为 0 (只有 16 位的段限)。

4.5.6 检查 MSR 字段

除了 IA32_EFER 字段的 LMA 与 LME 位外, guest-state 的 MSR 字段与 host-state 的 MSR 字段检查基本一致。下面是 MSR 字段的检查。

- 在 64 位架构处理器上, IA32_SYSENTER_ESP 与 IA32_SYSENTER_EIP 字段的地址值必须为 canonical 形式 (即地址值的 bits 63:48 必须是 bit 47 的符号扩展)。
- 当 VM-entry control 字段的 “load IA32_PERF_GLOBAL_CTRL” 为 1 时, IA32_PERF_GLOBAL_CTRL 字段的保留位必须为 0 (即 bits 31:N 与 bits 63:35 必须为 0, 这个 N 值等于 IA32_PMCx 计数器的个数)。
- 当 VM-entry control 字段的 “load IA32_PAT” 为 1 时, IA32_PAT 字段的 bits 7:0 不能为保留值 (即不能为 02H、03H 以及 08H 到 FFH)。
- 当 VM-entry control 字段的 “load IA32_EFER” 为 1 时, 对 IA32_EFER 字段进行下面的检查:
 - 保留位 (bits 7:1, bit 9 以及 bits 63:12) 必须为 0。
 - LMA 位 (bit 10) 必须等于 VM-entry control 字段 “IA-32e mode guest” 位。
 - 当 CR0 字段 PG 位为 1 时, LME 位 (bit 8) 必须等于 “IA-32e mode guest” 位。

在上面的 LME 位检查里，当 VM-entry control 字段“IA-32e mode guest”为 0，CR0 字段的 PG 位也为 0 时，LME 允许为 1 值（注意，此时 LME 位并不等于“IA-32e mode guest”）。

4.5.7 检查 activity state 字段

当前 VMX 架构下支持 4 种处理器状态值（见 3.8.5 节的表 3-18 所示），处理器对 activity state 字段进行下面的检查。

- Activity state 字段的值必须是 0 到 3，处理器具体所支持的 inactive 状态值需要查询 IA32_VMX_MISC 寄存器 bits 8:6 得到（见 2.5.11 节）。
- 当 SS access right 字段的 DPL 值不为 0 时，activity state 字段必须**不能为 1**（HLT 状态）。
- 当 interruptibility state 字段的 bit 0 为 1（存在“blocking by STI”阻塞状态）或者 bit 1 为 1（存在“blocking by MOV-SS”阻塞状态）时，activity state 字段值**必须为 0**（active 状态）。
- 当存在事件注入时（VM-entry interruption information 字段的 bit 31 为 1），取决于注入事件的中断类型与向量号（VM-entry interruption information 字段的 bits 10:8 与 bits 7:0），有下面的检查：
 - 注入事件属于软件中断，特权级软件中断或者软件异常（中断类型值为 4、5 或者 6）时，activity state 字段值**不能为 1**（HLT 状态）。
 - 注入的事件属于**硬件异常**时（除了#DB 和#MC 异常外，对应向量号为 1 和 18），activity state 字段值**不能为 1**（HLT 状态）。
 - 注入 **NMI**（中断类型值为 2）以及**#MC 异常**（硬件异常并且向量号为 18）以外的其他事件，activity state 字段值都不**允许为 2**（shutdown 状态）。
 - 当有事件注入时，activity state 字段值**不允许为 3**（wait-for-SIPI 状态）。
- 当 VM-entry control 字段的“entry to SMM”为 1 时，activity state 字段值**不允许为 3**（wait-for-SIPI 状态）。

4.5.8 检查 interruptibility state 字段

interruptibility state 字段中只有 4 个阻塞状态位（bits 3:0），处理器对 interruptibility state 字段有下面的检查。

- 保留位（bits 31:4）必须为 0。
- “blocking by STI”与“blocking by MOV-SS”位不能同时为 1。
- guest-state 区域 RFLAGS 字段的 IF 位为 0 时，“blocking by STI”位必须为 0。
- 存在事件注入时（VM-entry interruption information 字段 bit 31 为 1），注入的事件不能被阻塞。取决于事件的中断类型（VM-entry interruption information 字段 bits 10:8），有下面的检查：

- 当注入**外部中断**（中断类型值为 0）时，“blocking by STI”与“blocking by MOV-SS”位必须为 0。
- 当注入 **NMI**（中断类型值为 2）时，“blocking by MOV-SS”位必须为 0。
- 当注入 **NMI**（中断类型值为 2）时，“blocking by STI”位可能需要为 0。由于 NMI 是不可屏蔽的（不能被 IF 标志位阻塞），因此，“blocking by STI”是否要求为 0，依赖于处理器架构的实现。在一般情况下“blocking by STI”并不需要为 0。
- 当注入 **virtual-NMI**（中断类型值为 2，并且 pin-based VM-execution control 字段的“NMI exiting”与“virtual NMIs”都为 1）时，“blocking by NMI”位必须为 0。
- 发起 VM-entry 时当前处理器**不在 SMM 模式**，“blocking by SMI”位必须为 0。
- 发起 VM-entry 时当前处理器**处于 SMM 模式**，并且 VM-entry 字段的“entry to SMM”为 1，“blocking by SMI”位必须为 1。

4.5.9 检查 pending debug exception 字段

在 64 位架构处理器上，pending debug exception 字段为 64 位宽，否则为 32 位。这个字段的检查也受 activity state 或 interruptibility state 字段影响，有下面的检查。

- 保留位（bits 11:4、bit 13，以及 bits 63:15，在 32 位处理器上是 bits 32:15）必须为 0。
- 当 interruptibility state 字段的“blocking by STI”或“blocking by MOV-SS”位为 1 时，或者 activity state 字段指示属于 HLT 状态时，有下面的检查：
 - 当 guest RFLAGS 字段的 TF 位（bit 8）为 1，并且 guest IA32_DEBUGCTL 字段的 BTF 位（bit 1）为 0 时，BS 位（bit 14）必须为 1。
 - 当 guest RFLAGS 字段的 TF 位（bit 8）为 0，或者 guest IA32_DEBUGCTL 字段的 BTF 位（bit 1）为 1 时，BS 位（bit 14）必须为 0。

从另一个角度看，**分支单步调试**异常不可能由于“blocking by MOV-SS”阻塞状态而被悬挂，但**指令单步调试**异常可以（参见 4.13.1 节）。在 3.8.7.1 节里描述了可由“monitor trap flag”位而引起的 MTF VM-exit 产生悬挂。

另一方面，即使不存在“blocking by STI”或“blocking by MOV-SS”阻塞状态，单步调试#DB 异常也可能会由于 trap 类型的 VM-exit（包括 **TPR below threshod**、**EOI 虚拟化**、**APIC-write VM-exit** 或者 **“monitor trap flag” VM-exit**）而产生悬挂。此时，pending debug exception 字段的 BS 位（bit 14）为 1 值，不在上述的检查条件之内。

4.5.10 检查 VMCS link pointer 字段

VMCS link pointer 字段当前使用在 SMM 双重监控处理机制下，提供一个 VMCS 物理指针值。当字段值不为 FFFFFFFF_FFFFFFFFh 时，进行以下检查：

- Bits 11:0 必须为 0（对齐在 4K 字节边界上）。
- 地址值的 bits 63: N 必须为 0（ N 等于 MAXPHYADDR 值）。例如 MAXPHYADDR 为 36 时，bits 63:36 必须为 0。
- 物理地址指向的 VMCS 区域首 DWORD 字节必须是有效的 VMCS ID 值（见 3.2.3 节图 3-1）。
- 当前处理器不在 SMM 模式，或者 VM-entry control 字段的“entry to SMM”为 1 时，这个字段不能是原 guest VMCS 的 current-VMCS pointer 值。
- 当前处理器处于 SMM 模式，或者 VM-entry control 字段的“entry to SMM”为 0 时，这个字段不能是 VMXON pointer 值。

4.5.11 检查 PDPTE 字段

guest-state 区域中包含 4 个 PDPTE 字段（从 PDPTE0 到 PDPTE3），它们仅在 guest 使用 PAE 分页模式（IA32_EFER.LMA 为 0，CR0.PG 与 CR4.PAE 为 1）时加载，也就是 VM-entry control 字段的“IA-32e mode guest”为 0，并且 guest CR0 字段的 PG 位与 guest-CR4 字段的 PAE 位为 1 时。

当 guest 使用 PAE 分页模式时，取决于 secondary processor-based VM-execution control 字段的“enable EPT”位，处理器执行以下检查。

(1) 当“enable EPT”为 1 时，4 个 PDPTE 字段会有如下检查：

- PDPTE 字段的 P 位 (bit 0) 为 0 时，检查通过 (bits 63:1 被忽略)。
- PDPTE 字段的 P 位 (bit 0) 为 1 时，PDPTE 字段必须符合 PDPTE 表项的设置，如下所示。
 - 保留位 (bits 2:1 以及 bits 8:5) 必须为 0。
 - Bits 63: N 必须为 0（ N 等于 MAXPHYADDR 值），即不能超过最大物理地址值。例如，MAXPHYADDR 为 36 时，bits 63:36 必须为 0。

(2) 当“enable EPT”为 0 时，PDPTE 字段无须检查（参考下面 4.5.11.1 节的描述）。

4.5.11.1 由加载 CR3 引发的 PDPTE 检查

尽管在“enable EPT”为 0 时，不执行 guest-state 区域 PDPTE 字段的检查，但是下面的情况将引起 VM-entry 操作时对“CR3 寄存器所指向的 PDPTE 表项”进行检查：

- 发起 VM-entry 时 host 不使用 PAE 分页模式，而 guest 使用 PAE 分页模式。
- 进行 VM-entry 操作会改变 CR3 寄存器的值（即 guest 使用不同的 CR3 值）。

处理器对“CR3 寄存器指向的 PDPTE 表项”进行以下检查。

- PDPTE 表项的 P 位 (bit 0) 为 0 时，检查通过 (bits 63:1 被忽略)。
- PDPTE 表项的 P 位 (bit 0) 为 1 时，PDPTE 表项的设置为：
 - 保留位 (bits 2:1 以及 bits 8:5) 必须为 0。

- Bits 63: N 必须为 0 (N 等于 MAXPHYADDR 值)，即不能超过最大物理地址值。例如，MAXPHYADDR 为 36 时，bits 63:36 必须为 0。

当上面的检查不通过时，VM-entry 失败产生 VM-exit 行为。

4.6 检查 guest state 引起的 VM-entry 失败

在第 4.5 节描述的 guest-state 区域检查过程中，如果检查不通过将引起 VM-entry 失败，从而导致 VM-exit 发生。处理器将从 host-state 区域加载 host 的执行环境信息，转入 host-RIP 指向的入口执行。

处理器在 VM-exit 信息区域的 exit reason 字段里记录这个 VM-exit 信息：

- Bits 15:0 记录的原因码为 33，指示“由无效的 guest state 而引起 VM-entry 失败”（见 3.10.1.2 节表 3-24）。
- Bits 30:16 清 0。
- Bit 31 被置位，指示由 VM-entry 失败而引起 VM-exit。

由无效 guest-state 而引起 VM-entry 失败时，exit qualification 字段大多数情况下被清 0，除了下面几种情况：

- 在检查 PDPTE 字段时失败（见 4.5.11 节），exit qualification 的值为 2。
- 在检查 interruptibility state 字段时，如果由于注入 NMI 事件但“blocking by STI”位为 1 而引起 VM-entry 失败（见 4.5.8 节），exit qualification 的值为 3。
- 在检查 VMCS link pointer 字段时失败（见 4.5.10 节），exit qualification 的值为 4。
- Exit qualification 值为 1 时，表示 Not used（属于保留值）。

除了上面的情况外，其他 VM-exit 信息类字段不改变。

注意：“由 VM-entry 失败而导致的 VM-exit”与“在 guest 里产生的 VM-exit”有下面的不同之处：

- VM-entry interruption information 字段的 valid 位 (bit 31) 不会清 0，保持有效。
- Guest-state 区域内的字段不会被更新。
- 没有任何一个 MSR 保存在 VM-exit MSR-store 列表里。

4.7 加载 guest 环境信息

当 4.3 节描述的第 1 阶段检查，4.4 节描述的第 2 阶段检查，以及 4.5 节描述的第 3 阶段检查都通过后，处理器执行加载 guest-state 区域字段到相应的寄存器及更新处理器状态，作为 guest 的运行环境。

4.7.1 加载控制寄存器

- CR0 寄存器从 guest CR0 字段里加载，除了以下位域：
 - CR0 寄存器的保留位 (bits 15:6, bit 7 以及 bits 28:19) 总是为 0 值。
 - **CR0.ET** (bit 4) 总是为 1 值。
 - **CR0.NW** (bit 29) 以及 **CR0.CD** (bit 30) 保持不变 (VM-entry 不改变 CR0 寄存器的这些位)。
- CR3 字段与 CR4 字段分别直接加载到 CR3 与 CR4 寄存器。

4.7.2 加载 DR7 与 IA32_DEBUGCTL

当 VM-entry control 字段 “load debug controls” 为 1 时，执行下面的加载，否则忽略。

- DR7 字段加载到 DR7 寄存器，除了以下位域：
 - Bit 12 与 bits 15:14 忽略，加载到 DR7 寄存器总是为 0 值。
 - Bit 10 忽略，加载到 DR7 寄存器总是为 1 值。
- IA32_DEBUGCTL 字段直接加载到 IA32_DEBUGCTL 寄存器。

4.7.3 加载 MSR

部分 MSR 的加载受到 VM-entry control 字段影响，处理器执行下面的加载操作。

(1) 32 位的 IA32_SYSENTER_CS 字段加载到 IA32_SYSENTER_CS 寄存器的 bits 31:0, bits 63:32 清 0。

(2) IA32_SYSENTER_EIP 与 IA32_SYSENTER_ESP 字段的加载，有以下情形：

- 在 64 位架构处理器上这两个字段为 64 位宽，分别直接加载到 IA32_SYSENTER_EIP 与 IA32_SYSENTER_ESP 寄存器。
- 在 32 位架构处理器上，分别加载到 IA32_SYSENTER_EIP 与 IA32_SYSENTER_ESP 寄存器 bits 31:0, bits 63:32 清 0。

(3) 对 FS 与 GS 段寄存器 base 字段的 MSR 加载处理：

- 在 64 位架构处理器上，base 字段分别加载到 IA32_FS_BASE 与 IA32_GS_BASE 寄存器。
- 在 32 位架构处理器上，不存在 IA32_FS_BASE 与 IA32_GS_BASE 寄存器。

(4) “load IA32_PERF_GLOBAL_CTRL” 位为 1 时，IA32_PERF_GLOBAL_CTRL 字段加载到 IA32_PERF_GLOBAL_CTRL 寄存器。

(5) “load IA32_PAT” 位为 1 时，IA32_PAT 字段加载到 IA32_PAT 寄存器。

(6) 对于 IA32_EFER 寄存器，处理器进行下面的处理：

- 当 “load IA32_EFER” 位为 1 时，IA32_EFER 字段会被加载到 IA32_EFER 寄存器里。

- 当“load IA32_EFER”位为 0 时，有以下设置。
 - IA32_EFER.LMA 等于“IA-32e mode guest”位的值。
 - 当 CR0.PG 位被加载为 1 时，IA32_EFER.LME 也等于“IA-32e mode guest”位的值。否则，IA32_EFER.LME 的值不改变。

在上面的 IA32_EFER 加载里，当“IA-32e mode guest”为 0，并且 CR0.PG 位也被加载为 0 时，LME 位的值不改变，也就是**可能为 1**值（注意，此时 LME 位并不等于“IA-32e mode guest”），而 LMA 位为 0 值。

出现这种情况的前提是：host 运行在 IA-32e 模式，但 guest 运行在**非分页的保护模式**（“IA-32e mode guest”为 0，并且“unrestricted guest”为 1）。host 的 IA32_EFER.LME 位延伸至 guest 环境不改变，导致 LME 位为 1，而 LMA 位为 0（另见 4.5.6 节对 LME 的检查处理）。

4.7.4 SMBASE 字段处理

guest-state 区域的 SMBASE 字段只有在 SMM 双重监控处理机制下的“**VM-entry that return from SMM**”发生时，才会被加载到处理器内部 SMBASE 寄存器里，否则此字段忽略。

4.7.5 加载段寄存器与描述符表寄存器

(1) TR 寄存器的 selector、base、limit 以及 access right 字段都得到加载。

(2) ES、CS、SS、DS、FS、GS 以及 LDTR 寄存器 access right 字段 unusable 位 (bit 16) 为 0 时，它的 selector、base、limit 以及 access right 字段都得到加载。

(3) ES、CS、SS、DS、FS、GS 以及 LDTR 寄存器的 access right 字段 unusable 位为 1 时，base、limit 以及 access right 字段忽略不加载。在 VM-entry 完成后，段寄存器内部的 base、limit 及 access right 域是未定义值，除了下面进行的强制设置外，如下所示。

- CS 寄存器
 - selector、base、limit 字段以及 access right 字段的 L 位、D 位与 G 位都得到加载。
 - access right 字段的其余位被忽略。CS 寄存器内部的属性保持不变。
- SS 寄存器
 - selector 字段得到加载。
 - SS.base 的 bits 3:0 被清 0（并且在 64 位架构处理器上 bits 63:32 也被清 0）。
 - SS.DPL 加载为 access right 字段的 DPL 值 (bits 6:5)。
 - SS.B 设为 1 值。
- ES 与 DS 寄存器
 - selector 字段得到加载。
 - 在 64 位架构处理器上，ES.base 与 DS.base 的 bits 63:32 被清 0。

- FS 与 GS 寄存器
 - selector 字段得到加载。
 - FS.base 与 GS.base 分别从各自的 base 字段里加载。在 64 位架构处理器上，FS.base 与 GS.base 值也等于加载到 IA32_FS_BASE 与 IA32_GS_BASE 寄存器里。
 - 在 32 位架构处理器上，FS.base 与 GS.base 分别从各自的 base 字段里加载。
- LDTR 寄存器
 - Selector 字段得到加载。
 - 在 64 位架构处理器上，LDTR.base 被设置为 canonical 形式的未定义值。

4.7.5.1 unusable 段寄存器

在前面 4.7.5 节的描述里，VM-entry 完成后段寄存器属于不可用的，相当于加载一个 NULL selector 到段寄存器里，guest 软件执行时有下面的影响。

(1) 对于 ES、CS、SS、DS、FS 以及 GS 寄存器：

- guest 在非 64 位模式里运行时，使用 unusable 段寄存器访问将会产生#GP 或#SS 异常。
- guest 运行在 64 位模式时，使用 unusable 段寄存器访问不会产生异常。

(2) 使用 unusable 的 LDTR 寄存器访问时，总会产生#GP 异常。

4.7.5.2 加载 GDTR 与 IDTR

GDTR 与 IDTR 寄存器的 base 和 limit 字段直接加载到 GDTR 与 IDTR 寄存器里。

4.7.6 加载 RIP、RSP 和 RFLAGS

RIP、RSP 和 RFLAGS 字段分别直接加载到 RIP、RSP 和 RFLAGS 寄存器里。在 64 位架构处理器上，这些字段和寄存器都是 64 位宽。在字段值加载到寄存器后，如果 guest 运行在非 64 位模式下，寄存器内的 64 位值使用时有下面的处理：

- RSP 寄存器的 bits 63:32 忽略，属于未定义数据，guest 软件只使用低 32 位作为栈指针。
- RIP 与 RFLAGS 寄存器的 bits 63:32 为 0 值。由于在第 3 阶段的检查里，假如 guest 属于非 64 位模式，那么这两个寄存器对应的字段 bits 63:32 必须为 0 值（见 4.5.2 节描述）。

在 32 位架构处理器上，字段和寄存器都是 32 位宽，不存在上述的使用问题。

4.7.7 加载 PDPTE 表项

在两个地方的 PDPTE 表项可能需要加载处理器内部的 4 个 PDPTE 寄存器，即 PDPTE 字段里存放的 PDPTE 表项或者由 CR3 字段指向的 PDPTE 表项。

只有在 guest 使用 PAE 分页模式（即 VM-entry control 字段“IA-32e mode guest”位为 0，并且 CR0 字段的 PG 位和 CR4 字段的 PAE 位都为 1）时，才会发生 PDPTE 表项的加载。

当 guest 使用 PAE 分页模式时，取决于 secondary processor-based VM-execution control 字段“enable EPT”位的设置，有下面的加载处理：

- “enable EPT”为 1 时，4 个 PDPTE 字段值被加载到处理器内部相应的 PDPTE 寄存器里。此时，这些 PDPTE 表项内的地址值是 guest-physical address。
- “enable EPT”为 0 时，在 4.5.11.1 节描述的情况下，由 CR3 字段寄存器指向的 PDPTE 表项被加载到处理器内部相应的 PDPTE 寄存器里。此时，这些 PDPTE 表项内的地址值是 host-physical address。

4.8 刷新处理器 cache

在 VM-entry 时，取决于 secondary processor-based VM-execution control 字段“enable VPID”位的设置，处理器还会刷新 TLB 与 paging-structure cache 缓存。

- 当“enable VPID”为 1 时，不需要刷新这三种信息数据，包括 linear mapping、combined mapping 以及 guest-physical mapping 产生的 cache 信息（参见 6.2.1 节、6.2.2 节以及 6.2.3 节）。这些 cache 信息在进入 VM 后保持有效。
- 当“enable VPID”为 0 时，进入 VM 后将关闭 VPID（虚拟处理器 ID）机制。处理器将进行下面的 cache 刷新处理（参见 6.2 节）：
 - 刷新默认 VPID 值（0000H）下所有 PCID 值对应的 linear mapping 相关 cache 信息。
 - 刷新默认 VPID 值（0000H）下所有 PCID 与 EP4TA 值对应的 combined mapping 相关 cache 信息。

处理器的这个刷新 TLB 与 paging-structure cache 的动作相当于使用 single-context 方式执行了一次 INVVPID 指令（见 2.6.7.2 节），而提供的目标 VPID 值是 0000H，这个 VPID 值是在关闭 VPID 功能情形下的默认 VPID 值。

4.9 更新 Vritual-APIC 状态

在 VM-entry 阶段，处理器还进行两项 local APIC 虚拟化操作来更新 virtual-APIC 状态，包括 PPR 虚拟化（参见 7.2.12.2 节）以及虚拟中断的评估与 delivery（参见 7.2.13 节）。

当 secondary processor-based VM-execution control 字段的“virtual-interrupt delivery”为 1 时，将从 guest interrupt status 字段里加载 RVI 与 SVI 值，然后执行下面的操作。

4.9.1 PPR 虚拟化

只有当“virtual-interrupt delivery”为 1 时才会发生 PPR 虚拟化操作。处理器根据 SVI 值与 VTPR 值（虚拟 TPR、virtual-APIC page 页面内偏移量 80H 位置）来更新 VPPR 值（虚拟 PPR、virtual-APIC page 页面内偏移量 A0H 位置）。

```
if (VPTR[7:4] >= SVI[7:4])
{
    VPPR = VPTR & 0FFh
}
else
{
    VPPR = SVI & 0F0h
}
```

SVI 值记录了当前处理器的虚拟中断服务列表中的最大向量号。将 SVI 与 VTPR 做比较，选择较大值来更新 VPPR 值。如上面代码所示，当 VTPR[7:4] 大于等于 SVI[7:4] 时，VPPR 更新为 VTPR 值，否则更新为 SVI 值。VPPR 的 bits 31:8 总是被清 0。

4.9.2 虚拟中断评估与 delivery

也只有当“virtual-interrupt delivery”为 1 时才会发生虚拟中断评估与 delivery 操作。处理器根据 RVI 值与 VPPR 值，以及 primary processor-based VM-execution control 字段的“interrupt-window exiting”位来决定是否允许组织一个虚拟中断进行 deliver。

```
if ("interrupt-window exiting" == 0)
{
    if (RVI[7:4] > VPPR[7:4])
    {
        /* 组织虚拟中断的 pending */
    }
}
```

RVI 值记录着当前虚拟中断请求列表中的最大向量号，RVI[7:4] 大于 VPPR[7:4] 表示虚拟中断请求能通过仲裁。当“interrupt-window exiting”为 0，并且通过仲裁后才允许进行虚拟中断 pending 操作。

虚拟中断一旦成功 pending，在完成 VM-entry 的所有工作后（包括注入事件 handler 执行完毕），处理器将这个 pending 的虚拟中断 deliver 执行（详情参见 7.2.13 节）。

4.10 加载 MSR-load 列表

在加载 guest-state 环境信息的最后阶段是：当 VM-entry MSR-load count 字段不为 0 时，处理器将从 VM-entry MSR-load 列表里加载一系列的 MSR（见 3.6.2 节及图 3-6）。VM-entry MSR-load 列表的每一个表项是 128 位宽，bits 31:0 提供 MSR index，bits 127:64 提供 MSR data。

下面的 C 伪码描述了 VM-entry MSR-load 列表的加载操作。

```

for (i = MsrCount; i; i--)          // 根据 MSR count 字段值决定加载 MSR 的数量
{
    eax = MsrData[31:0];             // 读取 MSR data 低 32 位
    edx = MsrData[63:32];           // 读取 MSR data 高 32 位
    ecx = MsrIndex;                 // 从 MSR-store 表项的 bits 31:0 提供 MSR 编号
    wrmsr();                        // 执行 WRMSR 指令写 MSR
}

```

处理器从 VM-entry MSR-load 列表的 MSR data (bits 127:64) 里读取需要加载的 MSR 数据。然后写入由 MSR index (bits 31:0) 提供的 MSR 编号所对应的 MSR 里（相当于执行了 WRMSR 指令）。

VM-entry MSR-load 列表项的设置属于下面情况时，加载 MSR 将会失败。

- Bits 63:32 不为 0 值。
- Bits 31:0 提供保留或者未实现的 MSR。
- Bits 31:0 值为 **C0000100H** 或者 **C0000101H**，即在 MSR-load 列表里不能加载 IA32_FS_BASE 与 IA32_GS_BASE 寄存器。
- Bits 31:0 值是 **00000800H 到 000008FFH**，即在 MSR-load 列表里不能加载 x2APIC 模式下使用的 local APIC 寄存器。
- Bits 31:0 提供了只能在 SMM 模式下写的 MSR，但当前处理器不在 SMM 模式下（当前处理器模式已经被切换为 guest 环境）。IA32_SMM_MONITOR_CTL 寄存器只能在 SMM 模式下写。
- Bits 31:0 提供的 MSR 在写时发生 #GP 异常（不是由权限不足引起）。例如，当写 IA32_EFER 寄存器时，MSR data (bits 127:64) 的 LME 位为 0，但 CR0.PG 为 1，那么写这个 MSR 时会引起 #GP 异常（开启分页下不能关闭 IA-32e 模式）。
- Bits 31:0 提供的某些 MSR 可能会由于处理器架构实现的制约，而在 VM-entry 操作时不能在 MSR-load 列表里加载（尽管可以正常使用 WRMSR 指令对该 MSR 进行写）。在 VMX 当前架构下，有两个 MSR 不支持出现在 VM-entry MSR-load 列表里：IA32_BIOS_UPDT_TRIG 和 IA32_BIOS_SIGN_ID（编号分别为 79H 和 8BH），它们不允许在 VM-entry 时加载。

当加载 VM-entry MSR-load 列表出错时，会由于 VM-entry 失败导致 VM-exit 发生。处理器从 host-state 区域加载 host 的运行环境信息，执行 host-RIP 字段提供的 host 入口代码，并在 exit reason 字段里记录 VM-exit 原因码（这个值为 34，指示“由 MSR 加载而导致 VM-entry 失败”）。

4.10.1 IA32_EFER 的加载处理

IA32_EFER 寄存器的加载分为两个阶段：

- (1) 第 1 阶段取决于 VM-entry control 字段的“load IA32_EFER”位的设置。
- (2) 第 2 阶段取决于 MSR-load 列表设置。

在第 1 阶段（见 4.7.3 节）完成后，IA32_EFER 寄存器的 LME 位就已经被建立。

1. IA32_EFER 加载的第 1 阶段

- 当“load IA32_EFER”为 1 时，直接将 IA32_EFER 字段值加载到 IA32_EFER 寄存器。
- 当“load IA32_EFER”为 0 时，处理器进行下面的强制设置：
 - LMA 设为“IA-32e mode guest”的值。
 - CR0.PG 位被加载为 1 时，LME 也设为“IA-32e mode guest”的值，否则 LME 值不改变。
- 当“IA-32e mode guest”与 CR0.PG 都为 0 时，允许 LME 位加载为 1 值。

2. IA32_EFER 加载的第 2 阶段

在第 1 阶段里，IA32_EFER.LME 的值已经被建立。当 VM-entry MSR-load 列表里存在 IA32_EFER 寄存器需要加载时（VM-entry MSR-load 列表的其中一个表项 bits 31:0 值为 C0000080H），有下面的处理：

- 当 CR0.PG 位被加载为 1 时，IA32_EFER 映像（bits 127:64 的 MSR 数据）的 LME 位必须等于“IA-32e mode guest”位。如果不相等，则尝试修改 LME 位会引发 VM-entry 失败。
- 当 CR0.PG 位被加载为 0 时，IA32_EFER 映像的 LME 位允许不等于“IA-32e mode guest”位，即允许修改 LME 位而不会失败。

4.10.2 其他 MSR 字段的加载处理

如果下面几个 MSR 出现在 VM-entry MSR-load 列表里：

- IA32_DEBUGCTL
- IA32_SYSENTER_CS、IA32_SYSENTER_ESP 和 IA32_SYSENTER_EIP
- IA32_PERF_GLOBAL_CTRL
- IA32_PAT

那么，在 VM-entry MSR-load 列表加载时，这些 MSR 在之前加载的值将被覆盖。

4.11 由加载 guest state 引起的 VM-entry 失败

在加载 guest state 阶段里，对 MSR 的加载（见 4.7.3 节及 4.10 节）引起 VM-entry 失败时，处理器将从 host-state 区域加载 host 的执行环境信息，转入 host-RIP 指向的入口执行。

exit reason 字段 bits 15:0 记录的 VM-exit 原因码为 34，指示“由 MSR 的加载而引起 VM-entry 失败”。Bits 30:16 被清为 0，bit 31 被置位指示由 VM-entry 失败而导致 VM-exit 发生。

如果是由 VM-entry MSR-load 列表加载引起的 VM-entry 失败，那么 exit qualification 字段将记录 VM-entry MSR-load 列表中加载错误的表项序号值。例如，VM-entry MSR-

load 列表第 2 个表项发生错误，则 exit qualification 字段的值为 2。其余 VM-exit 信息类字段不改变。

注意：“由 VM-entry 失败而导致的 VM-exit”与“在 guest 里产生的 VM-exit”有下面的不同之处。

- VM-entry interruption information 字段的 valid 位 (bit 31) 不会清 0。
- guest-state 区域内的字段不会被更新。
- 没有任何一个 MSR 保存在 VM-exit MSR-store 列表里。

4.12 事件注入

MSR-load 列表成功加载完毕（如果需要加载），代表着在整个 VM-entry 操作流程里，处理器在 host 端的工作已经完成，当前处理器已经成功转入 guest 端的运行环境。

如果存在“事件注入”或者“pending debug exception”，它们将是 VM-entry 完成后处理器在 guest 环境里的第 1 个需要执行的任务。大多数情况下，事件注入或者 pending debug exception 是虚拟化 guest 端产生事件的一种手段，但也可以是 host 主动让 guest 执行额外的工作。

VM-entry interruption information 字段的 bit 31 为 1 时，取决于该字段的设置（参见第 3.6.3 节与 4.4.3.3 节描述），有下面的事件注入：

- 外部中断（类型为 0），NMI（类型为 2），硬件异常（类型为 3），软件中断（类型为 4），特权级软件中断（类型为 5）以及软件异常（类型为 6）。
- Pending MTF VM-exit 事件：中断类型为 7，并且向量号为 0。

这些注入的事件属于“向量事件”，当 VM-entry 伴随着一个事件注入时，这样的 VM-entry 被称为“向量化的 VM-entry”。由于事件注入是 VM-entry 完成后 guest 第 1 个需要执行的任务，因此具有很高的优先级别。guest 环境下当前的 IDTR 寄存器已经被加载（见 4.7.5.2 节），表示 guest 端的 IDT 已经建立。这个向量事件将通过 guest IDT 进行 deliver 执行。

1. pending MTF VM-exit 事件

VMX 架构允许注入一个不执行任何代码的事件。中断类型为 7 并且向量号为 0 时，注入一个 MTF VM-exit 事件 pending 在 guest 的第 1 条指令前，VM-entry 操作完切换到 guest 环境后立即产生 VM-exit。注入的 pending MTF VM-exit 事件不受 processor-based VM-execution control 字段的“monitor trap flag”位影响，即使“monitor trap flag”为 0。

2. 向量事件的注入点

在 guest 执行注入的事件前 guest 的运行环境已经被建立。当前的 CS:RIP（指令指针）、SS:RSP（栈指针）以及 RFLAGS 值已经从 guest-state 区域相应的字段里加载。

RIP 指向 VM-entry 后 guest 端第 1 条指令的位置，向量事件被 pending 在 guest 第 1

条指令之前。因此，注入事件在 guest 第 1 条指令之前被 deliver 执行。

如图 4-1 所示，guest 执行 INT3 指令由于 exception bitmap 字段 bit 3 为 1 而产生 VM-exit，VMM 注入#BP 异常恢复 guest 执行（必须在 VM-entry instruction length 字段里提供指令长度）。#BP 异常在 guest 第 1 条指令（即 INT3）前被 deliver 执行。处理器在 deliver #BP 异常期间压入的返回值等于 guest-RIP 加上指令长度（也就是#BP handler 返回会跳过 INT3 指令）。

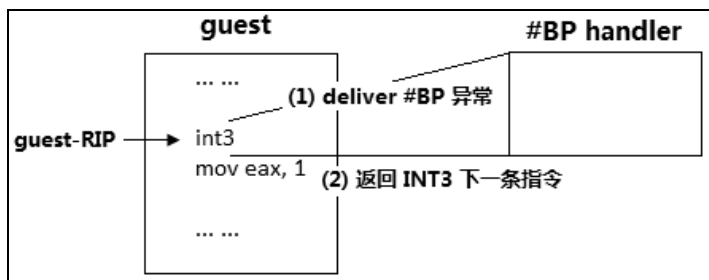


图 4-1

假如开启分支记录功能来监控注入事件的 delivery 情况，我们可以看到事件 delivery 的源地址是 guest 第 1 条指令地址，目标地址是注入事件的服务例程入口地址。IRET 指令返回的目标地址是 guest 第 1 条指令地址（在图中的例子是 INT3 指令的下一条指令）。

从 guest 执行流程角度来看，注入的事件本质上相当于“VM-entry 后 guest 第 1 条指令前产生了一个向量事件”。取决于事件的类型有下面的几种情况：

- 注入硬件异常事件时，相当于引发了一个异常。
- 注入外部中断或 NMI 时，相当于遇到了一个外部中断或 NMI 请求。
- 注入软件中断或者特权级软件中断时，相当于插入了一条 INT 指令。
- 注入软件异常时，相当于插入了一条 INT3 或 INTO 指令。

3. fault 与 trap 类型事件

在 guest 里，一个向量事件直接或者间接引发 VM-exit 后，VMM 需要在 VM-entry 时通过注入事件的方式让 guest 完成向量事件的处理。

引用对异常的分类方法，按照事件处理后恢复的执行点不同，我们也可以将向量事件分为以下两大类。

- **fault 类型**：指硬件异常（不包括由单步调试产生的#DB 异常）、外部中断和 NMI。
- **trap 类型**：指软件异常（#BP 与#OF、执行 INT3 与 INTO 指令产生）、软件中断（执行 INT 指令）和特权级软件中断（事件注入方式）。

注意：在 64 位模式下 INTO 指令无效，但允许注入#OF 异常。特权级软件中断需要在 VM-entry 时通过事件注入的方式产生。软件中断与特权级软件中断只能产生间接 VM-exit，不可能直接引发 VM-exit。

fault 与 trap 类型的向量事件决定了在注入事件 delivery 期间如何压入返回地址。下面的代码为例。

```
... ... ; 在此处发生了一个异常，外部中断请求或者 NMI 而导致 VM-exit
mov eax, 1
```

假设，“mov eax, 1”指令前触发一个外部中断或者 NMI 导致 VM-exit，那么 guest-RIP 字段值指向“mov eax, 1”这条指令。如果触发的是异常，取决于异常的类型（fault 还是 trap）：属于 fault 类型异常时，guest-RIP 字段指向引发异常的指令；属于 trap 类型异常时，guest-RIP 字段值指向引发异常指令的下一条指令。

```
int 40h ; 此处执行一条 INT 指令，或者 INT3、INTO 指令而间接导致 VM-exit
mov eax, 1
```

假设，“mov eax, 1”指令前执行 INT 指令，或者 INT3、INTO 指令（在 64 位模式下无效）时，由于 delivery 期间遇到错误而间接导致 VM-exit（见 3.10.3 节）。那么 guest-RIP 字段值指向间接导致 VM-exit 的指令（即上面代码中的“int 40h”指令），同时在 VM-exit instruction length 字段里记录这条指令的长度（参见 3.10.4.1 节）。

如果注入一个**软件异常**、**软件中断**或者**特权级软件中断**（trap 类型）事件，那么必须在 VM-entry instruction length 字段里提供指令长度。处理器在注入的事件 delivery 期间压入的返回值是 guest-RIP 值加上指令长度值。

4. 特权级软件中断

在一个特殊的应用场合里，如果 guest 执行软件中断由于权限不足产生 #GP 异常而导致 VM-exit，VMM 选择注入“**特权级软件中断**”来让 guest 绕过权限的检查，成功执行软件中断。

```
do_GP()
{
    ... ..

    if (IdtVectoringInfo.Type == INTERRUPT_TYPE_SOFTWARE_EXCEPTION)
    {
        /*
         * 属于软件中断
         */
        if (GuestCS.Rpl > IdtDesc.Dpl)
        {
            /*
             * 属于权限检查不通过，使用同样的向量号注入特权级软件中断
             */
            do_inject_event(vecotr,
INTERRUPT_TYPE_PRIVILEGED_SOFTWARE_EXCEPTION);
        }
    }

    ... ..
}
```

注意：这只是某个特殊的情况。VMM 在 #GP 异常的处理里，检查发现属于软件中断，并且 INT 指令没有足够的权限执行。VMM 使用了一个“取巧”手段来让 guest 的软

件中断得以执行。例如，当 guest 运行在 3 级权限，而对应的 IDT 描述符 DPL 为 0 级时，注入这个特权级软件中断可以绕过权限检查。

5. 事件的注入与 VM-exit

注入的事件不会直接产生 VM-exit，但可能会由于 delivery 期间遇到错误而间接产生 VM-exit（见 3.10.3 节描述）。当遇到下面的注入事件时（属于直接向量事件，见 3.10.2 节描述）：

- 注入硬件异常或者软件异常（#BP 与 #OF）时，向量号在 exception bitmap 字段中对应的位为 1。
- 注入外部中断时，Pin-based VM-exectuion control 字段的 “external-interrupt exiting” 位为 1。
- 注入 NMI 时，Pin-based VM-execution control 字段的 “NMI exiting” 位为 1。

即使满足上面的条件，注入的事件也不会直接产生 VM-exit。作为例外，注入一个 pending MTF VM-exit 事件会在 VM-entry 完成后，guest 第 1 条指令之前，直接产生 VM-exit。

4.12.1 注入事件的 delivery

如前面所述，事件注入相当于在 VM-entry 后的 guest 第 1 条指令前触发一个向量事件（中断或异常），在转入 guest 环境后，注入的事件通过 guest-IDT 进行 deliver。因此，x86/x64 体系里的中断或异常的 delivery 流程完全适用于注入事件。

基本的 delivery 流程如下：

- (1) 在 guest IDT 里读取相应的描述符表项并进行检查（包括类型权限等）。
- (2) 在 guest 栈里压入 RFLAGS、CS 以及 RIP 值，如果有错误码则压入错误码。在 IA-32e 模式里，会无条件压入 SS 与 RSP 值。
- (3) 转入执行中断服务例程。

然而，取决于 guest 的运行模式和注入事件类型，执行的细节有所不同。

4.12.1.1 保护模式下的事件注入

下面描述保护模式下的 delivery 流程（包括 IA-32e 模式）。在 IA-32e 模式里，不管是否发生权限切换，中断或异常 delivery 期间都会无条件压入 SS 与 RSP。在允许向栈里压入被中断者的返回信息前，处理器需要进行一系列的检查。

1. 常规检查

处理器根据注入事件的向量号，在 IDT 里找到对应的描述符表项。

- 描述符是否超出 IDT limit。
- 描述符是否为 present (P=1)。

- 描述符是否为 interrupt-gate、trap-gate 或者 task-gate。
- 接着对描述符内的目标代码段 selector 和目标代码段描述符进行常规检查（也可以参考《x86/x64 体系探索及编程》10.5.4.4 节第 185 页描述）。
- 如果发生权限切换（栈切换），也会引发对 TSS 段、对 TSS 段内的 SS selector 及目标栈段描述符进行常规检查（详见 7.1.3 节）。

2. 权限检查

- 当注入软件异常（#BP 与 #OF）或者软件中断时，需要满足下面的权限要求。
 - $CPL \leq$ 中断描述符的 DPL。
 - $CPL \geq$ 目标代码段的 DPL（中断调用中忽略 conforming 段权限检查）。
- 当注入硬件异常、外部中断、NMI 以及特权级软件中断时，无须检查权限。

3. 保存被中断者返回信息

前面的一系列检查通过后，处理器在 SS:RSP 指向的当前栈里压入被中断者的返回信息。SS 与 RSP 当前的值从 guest-state 区域的 SS 与 RSP 字段加载而来。如果发生权限改变，SS 与 RSP 的值将从 TSS 段对应的权限级别栈指针里加载（在 IA-32e 模式下，SS 被加载为 NULL selector，RSP 也可以使用 IST 指针）。在压入返回信息前处理器也会根据 SS 段限检查栈是否有足够空间容纳返回信息。

- 在 legacy 保护模式里，需要压入 EFLAGS、CS 以及 EIP 值。当发生权限改变时，首先压入 SS 与 ESP，然后是 EFLAGS、CS 和 EIP 值。
- 在 IA-32e 模式下固定依次压入 SS、RSP、RFLAGS、CS 和 RIP。

SS、RSP、RFLAGS、CS 和 RIP 的值分别从 guest-state 区域相应字段加载而来。但是，取决于注入事件的类型，压入栈中的返回值（RIP）需要进行一些修正处理。

当 VM-exit 由软件中断、软件异常及特权级软件中断间接引发时，处理器会在 VM-exit instruction length 字段里记录引发 VM-exit 的指令的长度（参见 3.10.4.1 节）。

- 注入**软件中断、软件异常及特权级软件中断**时，压入栈中的返回值等于 guest RIP 字段值加上 VM-exit instruction length 字段的值。
- 注入**硬件异常、外部中断及 NMI**时，压入栈中的返回值就等于 guest RIP 字段值。

由于注入的软件中断或者软件异常被用来虚拟化处理 guest 内执行 INT 或者 INT3 与 INTO 指令（64 位模式下无效），而它们属于 trap 类型，因此，注入事件执行完后需要执行下一条指令。压入栈中的返回值需要被修正为指向 guest 的下一条指令。

4. 错误码的 delivery

当 VM-entry interruption information 字段 bit 11 为 1 时，在注入事件 delivery 期间需要压入 16 位的错误码（参见 4.4.3.3 节），错误码从 VM-entry exception error code 字段里读取。

如果注入事件在 delivery 期间引发一个内嵌的异常，但这个内嵌的异常并不导致 VM-exit，那么在这个内嵌的异常 delivery 期间压入的错误码有下面的处理（错误码的

bits 15:3 为 selector 域, bit 2 为 TI 位, bit 1 为 IDT 位, bit 0 为 EXT 位) :

- 如果注入事件属于硬件异常、外部中断、NMI 或者特权级软件中断, 那么这个**内嵌的异常**压入栈中的错误码 EXT 位 (bit 0) 被置位。举例来说, 注入的一个#DB 异常在 delivery 期间引发了#GP 异常。在这个#GP 异常 delivery 时压入栈中的错误码为 **000BH** (即 selector 为 1, TI 位为 0, IDT 位为 1, EXT 位为 1)。
- 如果注入事件属于软件异常 (#BP 与 #OF) 或者软件中断, 那么这个内嵌的异常压入栈中的错误码 EXT 位 (bit 0) 被清 0。举例来说, 注入的一个软件中断 (向量号为 60H) 在 delivery 期间引发了#GP 异常。在这个#GP 异常 delivery 时压入栈中的错误码为 **0302H** (即 selector 为 60H, TI 位为 0, IDT 位为 1, EXT 位为 0)。
- 接上所述, 注入事件 delivery 期间所引发的异常, 在 delivery 期间引发了第 2 个异常, 但没有产生#DF 异常 (见下面 4.12.2 节所述)。第 2 个异常 delivery 时压入栈中的错误码 EXT 位 (bit 0) 被置位。接上例, 在#GP 异常 delivery 时引发了第 2 个#PF 异常, 这个#PF 异常不会产生#DF 异常, 那么#PF 异常 delivery 时压入的错误码为 **000BH** (即 selector 为 1, TI 位为 0, IDT 位为 1, EXT 位为 1)。
- 接上所述, 注入事件 delivery 期间所引发的异常, 在 delivery 期间引发了第 2 个异常, 继而转变为#DF 异常 (见下面 4.12.2 节所述)。#DF 异常在 delivery 时压入栈中的错误码为 **0000H** (即 selector 为 0, TI 位为 0, IDT 位为 0, EXT 位为 0)。接前面的例子, #GP 异常 delivery 时引发了第 2 个#SS 异常从而转变为#DF 异常。

5. VM-entry 后处理器状态的更新

取决于注入事件的类型, 处理器有下面的更新情形:

- 注入**#DB 异常**时, 处理器响应这个#DB 注入事件不会修改 DR6、DR7 和 IA32_DEBUGCTL 寄存器的值 (正常情况下, #DB 异常的 delivery 会更新 DR6 寄存器状态值, DR7.GD 被清位, IA32_DEBUGCTL 寄存器的 LBR 与 BTF 位也会被清位)。
- 注入一个 **virtual-NMI** 事件 (即 Pin-based VM-execution control 字段的 “NMI exiting” 与 “virtual NMIs” 位都为 1) 时, 这个 virtual-NMI 一旦被 delivery 后就存在 “blocking virtual-NMI” 阻塞状态 (即使在 delivery 期间发生错误而导致 VM-exit)。
- 在 VM-entry 完成后, 当 IA32_DEBUGCTL 寄存器的 **LBR** 位被加载为 1 时, 处理器会在 LBR 栈寄存器组里记录 guest 环境最近发生的分支记录 (LBR, Last-Branch Record)。那么, 注入事件的 delivery 将是 guest 环境里的第 1 条分支记录。分支的源地址就是 guest-RIP 指向的入口地址, 目标地址是注入事件的例程入口。可是, 当注入事件在 delivery 期间发生错误而导致 VM-exit 时, 则不会有 LBR 记录产生。
- 同上, 当 IA32_DEBUGCTL 寄存器的 **LBR** 位为 1 时, 处理器也可能在 LER (Last Exception Record) 寄存器组里记录最后一次发生的异常或中断前的最后分支记录。LER 记录与处理器架构实现相关, 因此, 当注入事件 delivery 期间发生错误而导致 VM-exit 时, LER 记录可能会产生, 也可能不会产生。

4.12.1.2 实模式下的事件注入

当 CR0 寄存器 PE 位被加载为 0 时, guest 进入实模式环境。由于实模式 guest 运行在 0 级权限里, 因此注入事件的 delivery 不存在权限检查环节 (见 4.5.4.3 节)。

实模式下的 IDT 被作为 IVT (Interrupt Vector Table), IDTR.base 指向 IVT 基址, IDTR.limit 指出 IVT 表限。由于 IVT 表项直接存放目标代码的 32 位 far pointer (即 CS:IP, 长度为 16:16), 因此不存在描述符类型检查环节, 但仍会执行 IVT limit 的检查。

在实模式下, VM-entry interruption information 字段 bit 11 必须为 0, 指示不需要压入错误码。处理器依次压入 16 位的 EFLAGS[15:0]、CS 以及 IP 后, 从 IVT 表项里加载 CS 与 IP 直接执行。

4.12.1.3 virtual-8086 模式下的事件注入

当 CR0 寄存器的 PE 位被加载为 1, 并且 RFLAGS 寄存器的 VM 位为 1 时, guest 进入 virtual-8086 模式环境。实际上, virtual-8086 模式相当于保护模式下的一个子模式, 运行在 3 级权限里。但不支持在 IA-32e 模式下开启 virtual-8086 模式。

在 virtual-8086 模式下, 发生软件中断时, 取决于 RFLAGS.IOPL 与 CR4.VME 位有不同的处理。当 CR4.VME 为 1 时, 启用增强的 virtual-8086 模式, 引入了 **software interrupt redirection** (软件中断重定向) 机制。在 TSS 段里开辟一块 32 字节的 redirection bitmap 区域。

Redirection bitmap 里的每个位对应一个中断向量号, 用来决定对应的软件中断最终使用哪种处理方式。结合 RFLAGS.IOPL、CR4.VME 和 redirection bitmap 有下面的处理方式。

(1) 当 RFLAGS.IOPL = 3 时, 有下面的软件中断重定向。

- CR4.VME=0 或者 CR4.VME=1, 并且 redirection bitmap 相应位为 1 时, 软件中断使用下面的保护模式中断处理手法:
 - 切换到 0 级权限里, 使用 0 级权限的栈。
 - 在栈里依次压入 GS、FS、DS、ES、SS、ESP、EFLAGS、CS 以及 EIP 值。
 - 清 EFLAGS 的 VM、NT、RF 以及 TF 标志位 (属于 interrupt-gate 时, 也清 IF 位)。
 - 清 GS、FS、DS 以及 ES 寄存器为 0。
 - 从 gate 描述符找到 CS 与 EIP 进行加载, 执行中断服务例程。
- CR4.VME=1, 并且 redirection bitmap 相应位为 0 时, 软件中断使用下面的实模式的中断处理手法:
 - 依次压入 16 位的 EFLAGS、CS 和 IP 值。
 - 清 EFLAGS 的 TF 与 IF 标志位。
 - 从 IVT 里加载 CS:IP, 执行中断服务例程。

(2) 当 $RFLAGS.IOPL < 3$ 时，有下面的软件中断重定向：

- CR4.VME=0 时，引发保护模式的#GP 异常处理（与前面的保护模式中断处理手法相同）。
- CR4.VME=1，并且 redirection bitmap 相应位为 1 时，引发保护模式下的#GP 异常处理（与前面的保护模式中断处理手法相同），但引入 VIF 与 VIP 标志位使用在中断的屏蔽处理上。只有当 EFLAGS 的 IF、VIF 和 VIP 都为 1 时，才可以响应可屏蔽中断请求。
- CR4.VME=1，并且 redirection bitmap 相应位为 0 时，引入 VIF 与 VIP 标志位使用在中断的屏蔽处理上。软件中断使用下面的实模式中断处理手法：
 - 压入 EFLAGS 前，IOPL 设为 3，VIF 等于 IF 位的值。
 - 依次压入 CS 与 IP 值。
 - 清 EFLAGS 的 VIF 与 TF 标志位。
 - 从 IVT 里加载 CS:IP，执行中断服务例程。

1. 注入的软件中断处理

在普通情况下，当 EFLAGS.IOPL 小于 3 时，CR4.VME 为 0 或者 CR4.VME 为 1，并且 redirection bitmap 相应位为 1 时，将引发#GP 异常。

下面是 VMX 架构下，在 **IOPL 小于 3** 时注入软件中断的处理。

- 注入一个软件中断不会因为 IOPL 小于 3 而引发#GP 异常，因此 VMM 在注入事件前，需要检查 EFLAGS.IOPL 的值是否小于 3。当 IOPL 小于 3 并且前面的条件成立时，VMM 需要注入一个**#GP 异常**代替软件中断。
- CR4.VME=1 并且 redirection bitmap 相应位为 0 时，注入的软件中断在 delivery 时压入的 EFLAGS 被修改：IOPL 设为 3 值，并且 VIF 设为 IF 位的值。

2. 其他注入事件的处理

注入一个特权级软件中断与注入软件中断处理一致。当注入硬件异常、软件异常以及 NMI 时，使用前面所述的保护模式中断处理手法。

当注入外部中断时，也使用保护模式中断处理手法。但是引入了 VIF 与 VIP 标志位控制可屏蔽中断的请求。当 EFLAGS 的 IF、VIF 以及 VIP 标志位同时为 1 时才允许响应外部中断。

4.12.2 注入事件的间接 VM-exit

注入的事件不会直接产生 VM-exit，但在下面的情形里可以间接地产生 VM-exit。

- 注入的事件在 delivery 期间引发了一个异常，这个异常的向量号在 exception bitmap 字段相应的位为 1，从而导致 VM-exit。
- 注入的事件在 delivery 期间由于连串异常而引发了**#DF** 异常（double fault，双重错误），这个#DF 异常并不导致 VM-exit。在#DF 异常 delivery 期间又引发了另一个

异常，继而转变为 **triple fault**（三重错误）直接导致 VM-exit。

- 注入的事件是一个 **#DF** 异常。在这个事件 delivery 期间引发了一个异常，继而转变为 **triple fault** 直接导致 VM-exit。
- 注入事件的向量号对应的 IDT 描述符属于 task-gate，继而尝试进行任务切换而导致 VM-exit。
- 注入事件在 delivery 期间访问了 APIC-access page 页面（包括线性访问和 guest-physical address 访问）而导致 VM-exit。
- 注入事件在 delivery 期间发生了 EPT violation 或者 EPT misconfiguration 而导致 VM-exit。

1. #DF 异常与 triple fault 引发的条件

在 x86/x64 体系里，一条指令的执行或者一个向量事件的 delivery 期间可能会引发一连串异常而转变为 #DF 异常，最终也可能会产生 triple fault。

如表 4-1 所示，概括了 #DF 异常和 triple fault 发生的条件：

表 4-1

第 1 个异常	第 2 个异常	转 变	第 3 个异常转变
#DE、#TS、#NP、#SS、#GP	#DE	#DF	triple fault
	#TS		
	#NP		
	#SS		
	#GP		
#PF	#DE		
	#TS		
	#NP		
	#SS		
	#GP		
	#PF		

- 遇到的第 1 个和第 2 个异常属于 #DE、#TS、#NP、#SS 或者 #GP 时，将转变为 #DF 异常。
- 遇到的第 1 个异常是 #PF，第 2 个异常属于 #DE、#TS、#NP、#SS、#GP 或者 #PF 时，将转变为 #DF 异常。
- Triple fault 触发的情形和 #DF 异常一致，在 #DF 异常 delivery 期间引发了上述的两类异常时将转变为 triple fault。

实际上，在异常 delivery 期间也只能遇到 #TS、#NP、#SS、#GP 以及 #PF 异常。

2. 注入事件 delivery 期间的 #DF 与 triple fault

在注入事件的 delivery 期间，#DF 异常和 triple fault 发生的条件取决于事件类型和发

生异常的向量号对应应在 exception bitmap 字段的位：

- 注入的事件属于#DE、#TS、#NP、#SS 或者#GP 异常，在 delivery 期间引发了#DE、#TS、#NP、#SS 或者#GP 异常，但这些异常并不导致 VM-exit，继而转变为#DF 异常。
- 注入的事件是#PF 异常，在 delivery 期间引发了#DE、#TS、#NP、#SS、#GP 或者#PF 异常，但这些异常并不导致 VM-exit 继而转变为#DF 异常。
- 注入事件属于上述两种事件以外，在注入事件 delivery 期间引发了第 1 个异常（上述的两类异常之一，如#GP 异常），#GP 异常并不导致 VM-exit。继而在#GP 异常 delivery 期间又引发了第 2 个异常（上述的两类异常之一，如#SS 异常），这个#SS 异常也不导致 VM-exit，继而转变为#DF 异常。
- 注入事件 delivery 期间，在上述的情况下产生#DF 异常。但#DF 异常并不导致 VM-exit，继而在#DF 异常 delivery 期间又引发了另一个异常（上述的两类异常之一），继而转变为 triple fault 直接导致 VM-exit。
- 注入事件是#DF 异常，在事件 delivery 期间引发了一个异常（上述的两类异常之一），继而转变为 triple fault 直接导致 VM-exit。

4.13 执行 pending debug exception

VMM 发起 VM-entry 操作切换到 guest 环境后，如果伴随着注入事件，则会首先 deliver 这个注入事件。另一方面，当 pending debug exception 字段的 enabled breakpoint 位 (bit 12) 或者 BS 位 (bit 14) 为 1 时（并属于有效），表明处理器当前存在一个**悬挂的#DB 异常**。那么，当这个#DB 的 delivery 条件允许时，处理器会紧接着 deliver 这个悬挂的#DB 异常。

当 exception bitmap 字段的 bit 1（对应#DB 异常）为 1 时，VM-entry 完成后悬挂#DB 异常的 delivery 会直接引发 VM-exit。

pending debug exception 字段记录 **trap 类型**的#DB 异常悬挂情况。BS 位 (bit 14) 为 1 时，表示存在指令或者分支单步调试异常悬挂。“enabled breakpoint”位 (bit 12) 为 1 时，表示存在数据断点或者 I/O 断点调试异常悬挂。

悬挂#DB 异常的 delivery 分为以下两种情形：

- (1) VM-entry 伴随着注入事件。
- (2) VM-entry 不含注入事件。

这两种情况的处理手法有较大的不同，例如，“blocking by MOV-SS”阻塞状态影响着**非注入事件**下悬挂#DB 异常的 delivery，而对**注入事件**下的悬挂#DB 异常 delivery 并没有实际的影响。下面的内容将对这两种情况分别进行描述。

4.13.1 注入事件下的 #DB 异常 delivery

在 VM-entry 含有注入事件时，悬挂的 #DB 异常被认为出于下面两种情况而产生。

(1) 指令单步调试中，在“MOV-SS”指令后面由于执行 INT3 或 INTO 指令而引发 VM-exit。

(2) 执行 INT3 或 INTO 指令引发 VM-exit 之前，已经存在由数据断点触发的 #DB 异常的悬挂，并且存在“blocking by MOV-SS”阻塞状态。

思考第 1 种情况，在 MOV-SS 指令后面发生了软件异常（#BP 或 #OF 异常，由 INT3 或 INTO 指令执行产生）。如下面的代码示例。

```

... ... ; TF=1, 启用单步调试
mov ss, ax ; 切换 SS
int3 ; 由于 MOV-SS 阻塞状态 #DB 被悬挂, int3 产生 VM-exit

```

上面的代码展示了其中一种 #DB 异常悬挂情况的产生由来。正常情况下，开启了指令单步调试而非分支单步调试（`rflags.TF=1`，并且 `IA32_DEBUGCTL.BTF=0`），将在每条指令执行完毕后组织 #DB 异常进行 deliver。然而在上面的代码里，执行完“`mov ss, ax`”指令后产生了“blocking by MOV-SS”阻塞状态，使得指令单步调试 #DB 异常被悬挂而未能 deliver。

MOV-SS 指令后面是一条 INT3 指令，执行这条指令产生的 #BP 软件异常属于**直接向量事件**（见 3.10.2 节），由于 exception bitmap 字段的 bit 3 为 1 而直接引发 VM-exit。

在 VM-exit 开始前，处理器当前就已经存在“blocking by MOV-SS”阻塞状态和 #DB 异常的悬挂，处理器在下面的字段里记录这些信息。

- VM-exit interruption information 字段的值为 **80000603H**（类型为软件异常，向量号为 3）。
- VM-exit instruction length 字段的值为 **1**（INT3 指令长度）。
- interruptibility state 字段的值为 **00000002H**（“blocking by MOV-SS”位为 1）。
- pending debug exception 字段的值为 **00004000H**（BS 位为 1，指示存在指令单步调试的 #DB 异常悬挂）。

在这个情况下，就存在一个**有效的 pending debug exception**。那么，VMM 在下次进行 VM-entry 操作时，应该注入一个 #BP 异常事件让 guest 接着完成 INT3 指令的工作，并且维持 pending debug exception 字段不变。而“blocking by MOV-SS”阻塞状态可以保持不变，在 VM-entry 含有注入事件时，对悬挂的 #DB 异常 delivery 并没有影响。

思考第 2 种情况，同样是由于执行了 INT3 或者 INTO 指令直接引发 VM-exit。在 VM-exit 之前已经存在 #DB 异常的悬挂和“blocking by MOV-SS”阻塞状态。如下面的代码示例。

```

SET_BREAKPOINT 0, BP_READ_WRITE2, rsp ; 在当前栈中存在一个数据断点 0
mov ss, [rsp] ; 触发数据断点, 并且产生 “blocking by
MOV-SS”
int3 ; #BP 异常引发 VM-exit

```

在上面的代码里，由于将 RSP 设为数据断点，在执行 MOV-SS 指令时产生了“blocking by MOV-SS”阻塞状态，同时也触发了数据断点，因而造成了数据断点#DB 异常的悬挂。

处理器在下面的字段里记录这些信息。

- VM-exit interruption information 字段的值为 **80000603H**（类型为软件异常，向量号为 3）。
- VM-exit instruction length 字段的值为 **1**（INT3 指令长度）。
- interruptibility state 字段的值为 **00000002H**（“blocking by MOV-SS”位为 1）。
- pending debug exception 字段的值为 **00001001H**（“enabled breakpoint”位与 B0 位为 1，指示遇到了 0 号数据断点的#DB 异常悬挂）。

同样，VMM 需要注入#BP 异常帮助 guest 完成 INT3 指令的执行，并且维持 pending debug exception 字段值不变。“blocking by MOV-SS”阻塞状态可以保持不变，在 VM-entry 含有注入事件时，对悬挂的#DB 异常 delivery 并没有影响。

1. 分支单步调试

在 rflags.TF=1，并且 IA32_DEBUGCTL.BTF=1 时，启用分支单步调试。注意，**控制权跳转到目标地址**，这一过程不可能产生“blocking by MOV-SS”阻塞状态。因此，也就不存在由于“blocking by MOV-SS”阻塞状态而导致#DB 异常悬挂。

我们看看下面的代码示例。

```

    jmp @1          ;; eflags.TF = 1, 并且 IA32_DEBUGCTL.BTF = 1
    ...
@1:
    mov ss, ax      ;; 在执行这条 MOV-SS 指令前#DB 异常已经被 deliver, 不存在#DB 异常悬挂
    int3            ;; 不存在#DB 异常

```

存在“blocking by MOV-SS”阻塞状态的情况下，pending debug exception 字段记录的 BS 位（bit 14）为 1，只可能是由于指令单步调试引起。另参见 4.5.9 节关于 pending debug exception 字段检查的描述。

另一方面，trap 类型的 VM-exit 可以使分支单步调试异常产生悬挂。例如，由于“monitor trap flag”位而产生的 MTF VM-exit。

2. 有效的 pending debug exception

从前面所述的两种情况来看，当 VM-entry 操作含有事件注入时，需要满足下面两个条件，pending debug exception 才可能是有效的。

- 注入事件的类型属于**软件异常**（类型为 6）、**软件中断**（类型为 4）或者**特权级软件中断**（类型为 5）。
- VM-entry 后存在“blocking by MOV-SS”阻塞状态。

当上面的条件满足后，pending debug exception 字段的 enabled breakpoint 位（bit 12）

或者 BS 位 (bit 14) 为 1 时, 就存在有效的 pending debug exception。

当注入软件异常, 但是向量号不是 3 (#BP 异常) 或者 4 (#OF 异常) 时, 这个 pending debug exception 可能会丢失, 但也有可能成功 deliver。

在《Intel 手册》Vol.3C 26-22 页里 (26.6.3 节) 描述的一段话, 其中有一句讲述的意思是:

“当注入事件类型为外部中断、NMI、硬件异常或者特权级软件中断时, 不存在 pending debug exception。”

然而, 在笔者的 Westmere 平台测试里, 注入**特权级软件中断**时, 悬挂的 #DB 异常依然可以 deliver。在此, 笔者认为注入事件属于特权级软件中断时 pending debug exception 也是有效的。

无法理解“对于注入事件属于软件中断这种情况”。并不能在 guest 中**重现**由执行 INT 指令 (间接产生 VM-exit) 而造成 #DB 异常悬挂的现象, 因为, INT 指令并不能直接引发 VM-exit, 间接引发 VM-exit 时 (即 IDT-vectoring information 字段的 bit 31 为 1), 在 VM-exit 开始前已经不存在“blocking by MOV-SS”阻塞状态 (参见 5.10.2 节)。

但是, 可以由 VMM 主动注入一个软件中断事件来模拟实现“注入软件中断伴随着 #DB 异常的悬挂”。由此, 笔者理解为“注入软件中断及特权级软件中断事件并且伴随着 #DB 异常悬挂时, 被视作 INT3 与 INTO 指令的模拟”。

关于这一点, 《Intel 手册》里并没有做进一步的说明, 只阐述了当存在“blocking by MOV-SS”阻塞状态, 以及 #DB 异常的悬挂时, 被认为在 MOV-SS 指令后执行了 INT3 或 INTO 指令。

3. 检查 pending debug exception 字段

在 VM-entry 时, 不管 pending debug exception 字段的值是由 guest 反射而来, 还是由 VMM 主动设置 (并不是由 guest 反射而来), 这个字段必须要符合 VM-entry 时的检查要求 (参见 4.5.9 节)。即 VM-entry 后如果存在“blocking by MOV-SS”或“blocking by STI”阻塞状态, 或者处于 HLT 状态, 那么必须满足下面的条件:

- Guest RFLAGS 字段的 TF 位 (bit 8) 为 1, 并且 IA32_DEBUGCTL 字段的 BTF 位 (bit 1) 为 0 时, pending debug exception 字段的 BS 位 (bit 14) 必须为 1。
- Guest RFLAGS 字段的 TF 位 (bit 8) 为 0, 或者 IA32_DEBUGCTL 字段的 BTF 位 (bit 1) 为 1 时, pending debug exception 字段的 BS 位 (bit 14) 必须为 0。

这要求 pending debug exception 字段的 **BS** 位 (bit 14) 必须是基于**指令**的单步而不是分支单步。如果 #DB 异常不是由于指令单步调试而造成悬挂, pending debug exception 字段的“enable breakpoint”位为 1, 表示遇到了数据断点或者 I/O 断点。

4. #DB 异常的 delivery 明细

当 pending debug exception 有效时, 在 VM-entry 后, 首先会 deliver 注入的事件, 接

着再 deliver #DB 异常。注意，如果在 VM-entry 时存在“blocking by MOV-SS”阻塞状态，那么这个阻塞状态由于注入事件的 delivery 被自动解除（另参见 5.10.2 节对间接 VM-exit 状态更新的描述）。

基于这个原因，“blocking by MOV-SS”阻塞状态对悬挂#DB 异常的 delivery 没有实际影响。

在 VM-entry 含有注入事件时，指令单步调试#DB 异常与数据或 I/O 断点#DB 异常的 delivery 处理手法是不同的。我们看看下面这段 guest 内的代码示例。

```

... .. ; TF = 1

mov ss, ax ; 产生“blocking by MOV-SS”状态
int3       ; 产生 VM-exit
mov eax, 3
mov eax, 5
mov eax, 6

```

guest 执行 INT3 指令产生了 VM-exit，VMM 注入#BP 异常给 guest 执行，并且此时存在 pending debug exception。处理器的整个处理流程如图 4-2 所示。

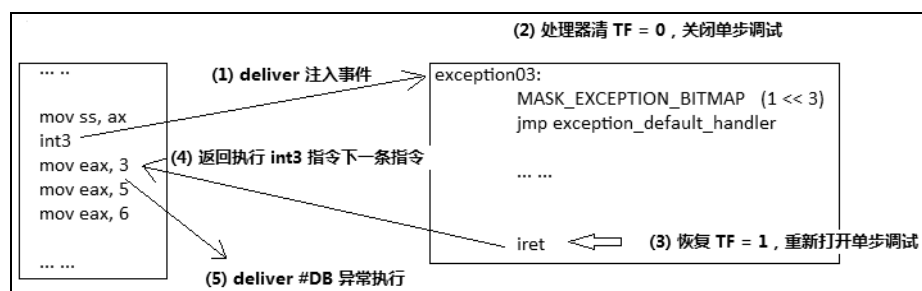


图 4-2

如图 4-2 所示，处理器的 5 个处理流程为：

(1) 在 VM-entry 后，首先 deliver 注入的事件，也就是#BP 异常。

(2) 处理器在#BP 异常的 delivery 期间清 eflags.TF 位为 0，关闭了单步调试功能。由于悬挂的#DB 异常是单步调试异常，**导致不能继续 deliver #DB 异常**。

(3) 执行 IRET 指令返回，并且恢复 eflags.TF 位为原来的 1 值，这一步将**重新打开单步调试**。

(4) 执行 IRET 指令的下一条指令，也就是“mov eax, 3”。

(5) 产生#DB 异常，处理器接着 deliver #DB 异常执行。

我们看到，由于在注入事件的 delivery 期间将 TF 标志位清 0，导致 pending debug exception 不能被 deliver 执行。因此，pending debug exception 只有在注入事件执行完毕后返回，并且执行完产生软件异常的下一条指令后才被 deliver 执行。

但是，如果在#BP 异常例程将栈中 EFLAGS 寄存器映像 bit 8 (TF 位) 清 0，则返回

后会关闭单步调试功能，此时就不可能产生#DB 异常。另外，如果在紧接着 IRET 指令之前先打开 TF=1，则在 IRET 指令返回后立即 deliver #DB 异常执行。

```

... ..
mov ss, [R4]                ; 触发数据断点与 “blocking by MOV-SS”
int3                        ; 产生 VM-exit
mov eax, 3
mov eax, 5
mov eax, 6

```

在上面这段代码示例里，将数据断点设置为 RSP（代码中的 R4）。执行 MOV-SS 指令产生了“blocking by MOV-SS”阻塞状态，并且触发了数据断点。同样由后面的 INT3 指令产生 VM-exit，从而造成#DB 异常的悬挂。

如图 4-3 所示，处理器处理注入事件与数据断点#DB 异常的 delivery 流程分为以下几点：

- (1) 处理器首先 deliver 注入事件（#BP 异常）。
- (2) 处理器检查到当前存在悬挂的#DB 异常，由于属于数据断点引起，不受 TF 标志位影响，并且 trap 类型的#DB 异常优先级高于#BP 异常。处理器紧接着 deliver #DB 异常，导致#BP 异常被抢先，处理器执行#DB 异常例程。
- (3) #DB 异常例程执行完毕后，返回到#BP 异常例程的第 1 条指令继续执行。
- (4) #BP 异常例程执行完毕后，返回到 INT3 的下一条指令执行（guest-RIP 加上指令长度）。

我们看到，#BP 异常在第 1 条指令处被#DB 异常抢先。#BP 异常在 delivery 期间压入的返回地址等于 guest-RIP 加上指令长度，因此，#BP 异常处理完毕返回到 INT3 指令的下一条指令执行，如图 4-3 所示。

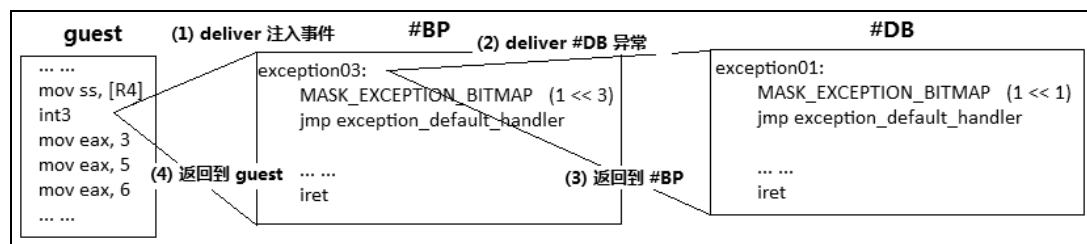


图 4-3

4.13.2 例子 4-1

示例 4-1：观察注入事件下，单步调试与数据断点#DB 的 delivery

现在，我们通过例子 4-1 来观察#DB 异常的 delivery。选取前面所说的单步调试和数据断点#DB 两种情况。这个例子里，我们将开启两个虚拟机分别在 CPU1 和 CPU2 里运行来观察这两种情况。

- 在 guest1 中生成单步调试#DB 的 pending。
- 在 guest2 中生成数据断点#DB 的 pending。

每个 guest 由于#BP 异常而引发 VM-exit，然后在 VMM 中注入#BP 异常事件，并且存在 pending debug exception。

1. guest 端代码

这个例子的 guest 使用与 host 相同的环境，guest 具备 host 系统的所有机制。guest1 的入口为 guest_entry1，guest2 的入口为 guest_entry2，下面是它们的代码。

代码片段 4-1:

```
guest_entry1:
    DEBUG_RECORD    "[VM-entry]: switch to guest 1 !"        ; 插入 debug 记录点

    mov ax, ss

    ;;
    ;; 打开单步调试
    ;;
    pushf
    bts DWORD [R4], 8                ; TF=1
    popf

    mov ss, ax                      ; 产生 MOV-SS 阻塞状态
    int3                            ; 产生 #BP 异常导致 VM-exit
    mov eax, 3
    mov eax, 5
    mov eax, 6

    hlt
    jmp $-1
    ret
```

代码片段 4-1 是 guest1 的代码，在 chap04\ex4-1\ex.asm 文件里。前面将 SS 保存在 AX 寄存器里，是为了后面使用 MOV 指令来产生“blocking by MOV-SS”阻塞状态。然后，使用 POPF 指令来置 TF 位，开启指令单步调试功能。MOV-SS 指令后面是 INT3 指令，用来产生软件异常并直接退出 VM-exit。

后面安排“mov eax, 5”与“mov eax, 6”指令，是为了观察#DB 异常在哪个地方 deliver。

代码片段 4-2:

```
guest_entry2:
    DEBUG_RECORD    "[VM-entry]: switch to guest 2 !"        ; 插入 debug 记录点

    ;;
    ;; 设置断点
    ;;
    mov ax, ss
    mov [R4], ax
    SET_BREAKPOINT 0, BP_READ_WRITE2, R4
```



```

mov ss, [R4]                ; 触发数据断点与 “blocking by MOV-SS”
int3                        ; 产生 VM-exit
mov eax, 3
mov eax, 5
mov eax, 6

hlt
jmp $ - 1
ret

```

代码片段 4-2 是 guest2 的代码，这里使用 SET_BREAKPOINT 宏来设置断点 0 在当前栈里。接着 MOV-SS 指令在触发数据断点的同时也产生 “blocking by MOV-SS” 阻塞状态，使得#DB 异常被悬挂。后面同样是一条 INT3 指令，用来产生#BP 异常，并直接引发 VM-exit。

CPU0 执行 do_command 函数让用户按下数字键<1>时，调度 CPU1 执行 TargetCpuVmentry1 函数进入 guest_entry1 执行。按下数字键<2>时，调度 CPU2 执行 TargetCpuVmentry2 函数进入 guest_entry2 执行。在 CPU0 执行 do_command 函数之前，设置示例的#BP 与#DB 异常例程。如代码 4-3 所示。

代码片段 4-3:

```

... ..
mov esi, BP_VECTOR
mov edi, foo
call install_kernel_interrupt_handler      ; 设置新的 #BP handler

mov esi, DB_VECTOR
mov edi, bar
call install_kernel_interrupt_handler      ; 设置新的 #DB handler

SET_EXCEPTION_BITMAP    BP_VECTOR        ; 让 #BP 直接引发 VM-exit
CLEAR_EXCEPTION_BITMAP  DB_VECTOR        ; #DB 不能直接引发 VM-exit
... ..

```

CPU0 将#BP 异常与#DB 异常的 handler 分别设为 foo 与 bar，用于测试与观察。并设置由#BP 异常直接产生 VM-exit，而#DB 异常将被 deliver 执行。

代码片段 4-4:

```

foo:
    DEBUG_RECORD    "[#BP]: enter #BP handler !"
    mov eax, 1
    mov eax, 2
    REX.Wrxb
    iret

bar:
    DEBUG_RECORD    "[#DB]: enter #DB handler !"

%if __BITS__ == 64
    mov rcx, rax
    mov esi, Ex.Msg0
    call puts
    mov rsi, rcx
    call print_qword_value64

```

```

    call println
    mov esi, Ex.Msg1
    call puts
    mov rsi, rsp
    call print_qword_value64

    lock btr DWORD [rsp + 16], 8           ;; 清 TF
%else
    mov ecx, eax
    mov esi, Ex.Msg0
    call puts
    mov esi, ecx
    call print_dword_value
    call println
    mov esi, Ex.Msg1
    call puts
    mov esi, esp
    call print_dword_value

    lock btr DWORD [esp + 8], 8           ;; 清 TF
%endif

    REX.Wrxb
    iret

```

foo 与 bar 分别作为 #BP 与 #DB 新的处理例程，只是简单地在开头插入一条 DEBUG_RECORD 记录点，用来观察信息。在 bar (#DB handler) 里打印 RAX 寄存器以及当前 RSP 寄存器的值，并且将 TF 清 0，防止在 #DB 异常返回后继续进行单步调试。

2. 编译及运行

使用下面的命令编译及生成映像文件。

- 生成 32 位代码：build -DDEBUG_RECORD_ENABLE。
- 生成 64 位代码：build -D__X64 -DDEBUG_RECORD_ENABLE。

定义了 DEBUG_RECORD_ENABLE 符号用来开启 DEBUG_RECORD 宏的记录功能，否则 DEBUG_RECORD 是空。

在 VMware 上运行后，在 CPU0 的命令选择界面上分别按数字 1 键和 2 键，让 CPU1 与 CPU2 执行自己的 VM-entry 操作（即 TargetCpuVmentry1 与 TargetCpuVmentry2）。

按下 <F2> 键切换到 CPU1，观察 guest1 的运行结果。如图 4-4 所示。



图 4-4

在#DB handler 内打印的 RAX 寄存器的值为 3，这是#BP handler 返回后，执行 INT3 的下一条指令“mov eax, 3”后的结果。

按<F3>键切换到 CPU2 上，观察 guest2 的运行结果。如图 4-5 所示。



图 4-5

#DB handler 打印的 RAX 值是使用 SET_BREAKPOINT 宏设置断点时遗留下来的，它是 guest2 内的 RSP 指针值。注意，RSP 寄存器指示当前栈中已经压入了 10 个值（对比与图 4-4 的不同）。

guest 内 RSP 初始值为 FFFFFFFF80_FFE11FF0H，经过#BP 与#DB 的 delivery 后，#DB handler 打印的值为 FFFFFFFF80_FFE11F98H，stack 中保存的值如图 4-6 所示。

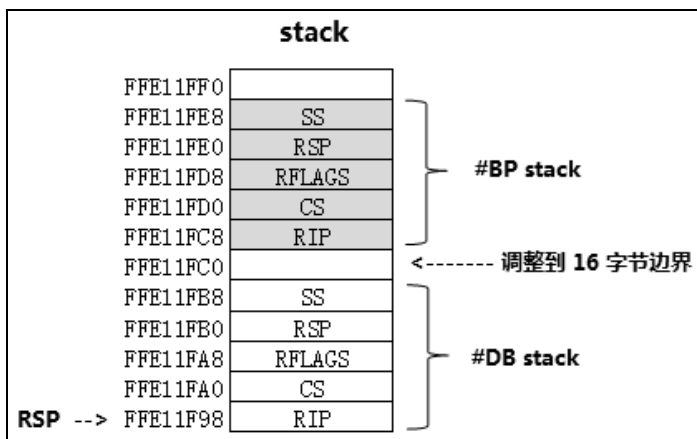


图 4-6

从图 4-6 中可以清楚地看到处理器在 deliver #BP 与#DB 异常时的处理过程。首先在 deliver #BP 异常时已经压入了 guest 的返回信息，然后再 deliver #DB 异常，接着压入#BP 的返回信息。#DB handler 抢在#BP handler 第 1 条指令前执行。

最后，我们按下<F4>键切换到 CPU3，再按下<Enter>键切换到 msg-list 列表，如图 4-7 所示。

```

Home x demo x
[CpuIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable

===== msg list record =====
<#01> [CPU0] [command]: *** dispatch to CPU for VM-entry ***
<#02> [CPU1] [VM-entry]: switch to guest 1 !
<#03> [CPU1] [Exit Reason]: exception or NMI
<#04> [CPU1] [do_BP]: inject a #BP event !
<#05> [CPU1] [UMM]: resume to guest !
<#06> [CPU1] [#BP]: enter #BP handler !
<#07> [CPU1] [#DB]: enter #DB handler !
<#08> [CPU1] [Exit Reason]: HLT instruction
<#09> [CPU1] [DoHLT]: enter HLT state
<#10> [CPU1] [UMM]: resume to guest !
<#11> [CPU0] [command]: *** dispatch to CPU for VM-entry ***
<#12> [CPU2] [VM-entry]: switch to guest 2 !
<#13> [CPU2] [Exit Reason]: exception or NMI
<#14> [CPU2] [do_BP]: inject a #BP event !
<#15> [CPU2] [UMM]: resume to guest !
<#16> [CPU2] [#DB]: enter #DB handler !
<#17> [CPU2] [#BP]: enter #BP handler !
<#18> [CPU2] [Exit Reason]: HLT instruction
<#19> [CPU2] [DoHLT]: enter HLT state
<#20> [CPU2] [UMM]: resume to guest !

产生 VM-exit 后, 注入 #BP 事件, 并且 #DB 异常悬挂, guest1 先执行 #BP, 再执行 #DB
产生 VM-exit 后, 注入 #BP 事件, 并且 #DB 异常悬挂, guest2 先执行 #DB, 再执行 #BP

```

图 4-7

分析图 4-7, 得到的信息如下:

(1) guest1 由于#BP 异常而引发 VM-exit, 在 VMM 的 do_BP 函数里注入一个#BP 异常事件后返回到 guest1 继续执行。guest1 先进入#BP handler 执行, 然后再进入#DB handler 内执行。

(2) guest2 同样由于#BP 异常而引发 VM-exit, 在 VMM 的 do_BP 函数里注入一个#BP 异常事件后返回到 guest2 继续执行, 但在 deliver #BP 期间, 由于被#DB 异常抢断了, 处理器先执行#DB handler, 然后再返回执行#BP handler。

CPU1 的执行流程是: VM-entry → 由#BP 异常而产生 VM-exit → 注入#BP 事件 → resume 回到 guest1 → 进入 #BP handler → 返回到 guest1 → 执行 INT3 下一条指令 → deliver #DB 执行。

CPU2 的执行流程是: VM-entry → 由#BP 异常而产生 VM-exit → 注入#BP 事件 → resume 回到 guest2 → deliver #BP → deliver #DB → 进入 #DB handler 执行 → 返回 #BP handler 执行。

这个例子, 主要是用来验证在 VM-entry 含有注入事件时, 单步调试#DB 与数据断点#DB 异常之间的 delivery 差异。

4.13.3 非注入事件下的#DB 异常 delivery

在 VM-entry 不含有注入事件时, 有效的 pending debug exception 在 VM-entry 后直接 deliver 给 guest 执行。如果 exception bitmap 字段的 bit 1 (对应#DB 异常) 为 1, 在 VM-entry 后由于#DB 异常的 delivery 而立即产生 VM-exit。

在非注入事件下, 造成#DB 异常的悬挂可能是下面的原因:

- (1) 在 VM-exit 之前#DB 异常已经被组织 (生成) 等待 deliver。
- (2) 在 VM-exit 之前已经存在由于“blocking by MOV-SS”阻塞状态而造成#DB 异常

的悬挂。

无论是指令单步调试#DB 异常，还是数据断点或 I/O 断点#DB 异常，它们都属于 **trap 类型**（即执行一条指令后才被 deliver），这种特性使得#DB 异常可以被悬挂。

思考下面的指令单步调试情形。

```
... ... ; TF=1, 启用单步调试
mov rax, [rbx] ;
mov DWORD [rax], 0 ; 执行这条指令如果产生 trap 类型的 VM-exit, #DB 被悬挂
```

在上面的代码示例里，开启了指令的单步调试。当执行到“**mov DWORD [rax], 0**”指令时，在正常情况下指令执行完毕后将立即 deliver #DB 异常。可是，执行这条指令时产生了 **trap 类型**的 VM-exit，那么将会有有一个#DB 异常处于 pending 状态。

trap 类型的 VM-exit 引发的原因，典型地是 TPR below threshold、EOI 虚拟化或者 APIC-write VM-exit，也会由于 MTF VM-exit（primary processor-based VM-execution control 字段“monitor trap flag”为 1）而造成。

同样，trap 类型的 VM-exit 也可以造成数据断点或者 I/O 断点#DB 异常的悬挂。下面我们来看看两个没有“blocking by MOV-SS”阻塞状态，由 trap 类型 VM-exit 而产生#DB 异常悬挂的情形。

```
mov rax, [rbx] ;
SET_BREAKPOINT 0, BP_READ_WRITE4, rax ; RAX 寄存器内的地址上
mov DWORD [rax], 0 ; 如果产生 trap 类型的 VM-exit, #DB 被悬挂
```

一个写内存操作触发了数据断点，如果这条指令产生了上面所说的 trap 类型 VM-exit 事件，则数据断点的#DB 异常被悬挂。

```
mov edx, 60h
SET_BREAKPOINT 0, BP_IO_READ_WRITE4, edx ; I/O 断点设置在端口 dx 上
insb ; 如果产生 trap 类型的 VM-exit, #DB 被悬挂
```

执行上面的 INSB 指令读 I/O 端口时触发了 I/O 断点，如果这条指令产生了 trap 类型的 VM-exit 事件，则 I/O 断点的#DB 异常悬挂。

下面再来看看两个由于“blocking MOV-SS”阻塞状态而产生#DB 异常悬挂的例子。

```
mov ss, [Stack] ; 切换 SS, 产生“blocking by MOV-SS”阻塞状态
cpuid ; 执行这条指令直接产生 VM-exit, #DB 被悬挂
```

上面的代码示例，在 CPUID 指令产生 VM-exit 前已经存在#DB 异常的悬挂（由于 MOV-SS 阻塞状态）。也可以由其他事件产生 VM-exit，如下面的代码。

```
mov ss, [Stack] ; 切换 SS, 产生“blocking by MOV-SS”阻塞状态
mov rax, [rcx] ; 执行这条指令产生 VM-exit, #DB 被悬挂
```

上面代码的“MOV rax, [rcx]”指令执行时，可能会由于遇到 APIC-access、EPT violation 或者 EPT misconfiguration 等而产生 VM-exit，那么，这些 VM-exit 事件属于 fault 类型。

处理器在 VM-exit 时保存的 pending debug exception 字段反映了处理器当前是否存在#DB 异常悬挂。Pending debug exception 字段值为 **00004000H** 时，表示遇到了指令单步调

试#DB；为 00001001H 时，表示遇到了一个 0 号的数据断点或者 I/O 断点。

1. 有效的 pending debug exception

如果 VM-entry 时不存在事件注入，当 activity state 字段的值不为 2 或 3（即不是 shutdown 或者 wait-for-SIPI 状态）时，下面的情况就存在有效的 pending debug exception。

- pending debug exception 字段的 bit 12 (enabled breakpoint 位) 为 1 时，表示 VM-entry 后存在数据断点或 I/O 断点#DB 异常，忽略 DR7 与 DR0~DR3 寄存器的设置（即使 DR7 寄存器没有设置）。
- pending debug exception 字段的 bit 14 (BS 位) 为 1 时，表示 VM-entry 后存在指令单步调试#DB 异常。即使在 TF=0 时也有效，除非此时存在“blocking by MOV-SS”阻塞状态，那么 VM-entry 后的那个#DB 异常被阻塞（效果等于 pending debug exception 被丢失）。

因此，DR7 与 RFLAGS.TF 不影响 VM-entry 后的 pending debug exception 的有效性。但是，如果存在“blocking by MOV-SS”阻塞状态，则可能在阻塞状态解除后 deliver，也可能会丢失。

2. 检查 pending debug exception

如果 guest 开启了指令单步调试，而由于 MOV-SS 指令而导致#DB 异常悬挂，VM-exit 时保存的 interruptibility state 字段值为 2，pending debug exception 字段的“enabled breakpoint”或者 BS 位为 1，表示遇到了指令单步调试、数据断点或者 I/O 断点。

因此，在 VM-entry 不含注入事件时，VM-entry 后存在“blocking by MOV-SS”或“blocking by STI”阻塞状态，或者处理器处于 HLT 状态，同样需要检查 pending debug exception 字段。BS 位 (bit 14) 需要与指令的单步调试功能保持同步（参见前面的 4.13.1 节）。

3. #DB 异常的 delivery

当 pending debug exception 有效时，有下面的情形。

- 不存在“blocking by MOV-SS”阻塞状态时，VM-entry 后直接 deliver #DB 异常给 guest 执行，#DB 异常在 guest 第 1 条指令前执行。
- 存在“blocking by MOV-SS”阻塞状态时，pending debug exception 在 VM-entry 后会保持 pending 状态，直到 guest 第 1 条指令执行完毕后解除“blocking by MOV-SS”阻塞状态，悬挂的#DB 异常才被 deliver 执行。

也就是，存在“blocking by MOV-SS”阻塞状态时，#DB 异常被 pending 在 guest 第 1 条指令后。但是，在某些处理器上也可能会丢失这个 pending debug exception。

当 exception bitmap 字段的 bit 1（对应#DB 异常）为 1 时，#DB 异常的 delivery 会导致 VM-exit。

4.14 使用 MTF VM-exit 功能

VMX 架构下提供了 MTF VM-exit 功能，允许 VMM 在 VM-entry 时 pending 一个 MTF VM-exit 事件让 guest 产生 VM-exit。MTF VM-exit 功能为 VMM 提供了强有力的调试手段。

MTF VM-exit 有下面两种 pending 方式。

(1) 通过**事件注入**方式，注入一个中断类型为 7 (other) 并且向量号为 0 的事件。MTF VM-exit 在 VM-entry 后被直接 pending 在 guest 第 1 条指令前。

(2) 通过**设置 MTF 位**方式，VMM 将 primary processor-based VM-execution control 字段的“monitor trap flag”位设为 1。MTF VM-exit 被 pending 在 guest 第 1 条指令之后，它属于 **trap 类型** 的 VM-exit。

由此可见，这两种方式 pending 点有所不同，注入一个 MTF VM-exit 事件时将 pending 在 guest 第 1 条指令前。也就是，VM-entry 完成后在 guest 第 1 条指令执行前立即产生 VM-exit (详情参见 4.15.3 节)。

而由“**monitor trap flag**”位引起的 MTF VM-exit 属于 trap 类型，它被 pending 在一条指令之后，或者一个**向量事件** (异常、中断、NMI、注入事件或者 pending debug exception) **deliver** 之后的第 1 条指令前。

本节主要介绍由“**monitor trap flag**”位产生 pending MTF VM-exit 的不同情形。

4.14.1 注入事件下的 MTF VM-exit

在 VM-entry 含有注入事件的前提下，有下面的情形。

- 在注入的向量事件完成 delivery 后，MTF VM-exit 将会 pending 在向量事件 handler 内的第 1 条指令之前。
- 假如 VM-entry 同时注入一个 pending MTF VM-exit 事件 (中断类型为 7，向量号为 0)，MTF VM-exit 会 pending 在 guest 第 1 条指令前。此时会忽略“**monitor trap flag**”位的作用，也就是使用注入事件的 pending 方式。

如图 4-8 所示，MTF VM-exit 将插在事件 handler 内第 1 条指令“mov eax, 1”前。

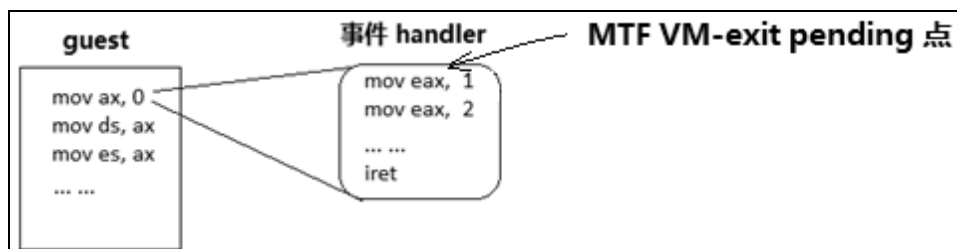


图 4-8

4.14.2 非注入事件下的 MTF VM-exit

如果 VM-entry 不含有注入事件，但存在有效的 pending debug exception，取决于是否存在“blocking by MOV-SS”阻塞状态，MTF VM-exit 的 pending 有下面的情形。

1. pending debug exception 下的 MTF VM-exit

- 不存在“blocking by MOV-SS”阻塞状态时，pending debug exception 被直接 deliver，而 MTF VM-exit 被 pending 在 #DB handler 的第 1 条指令前，如图 4-9 所示。

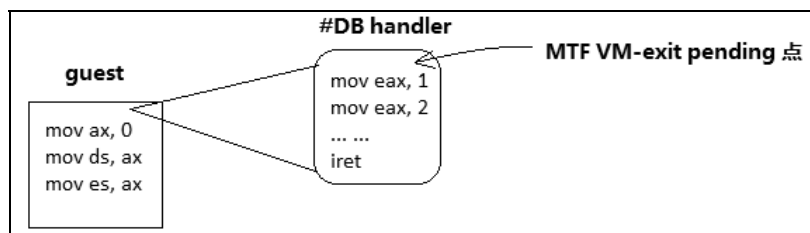


图 4-9

- 存在“blocking by MOV-SS”阻塞状态时，执行完 guest 第 1 条指令后解除“blocking by MOV-SS”阻塞状态，pending debug exception 与 MTF VM-exit 并发。但由于 MTF VM-exit 优先级更高，抢在 #DB deliver 之前引发 VM-exit，因此，#DB 异常并没有被 deliver。

2. 非 pending debug exception 下的 MTF VM-exit

在 VM-entry 后也不存在 pending debug exception 时，有下面的情形。

- MTF VM-exit 被 pending 在 guest 第 1 条指令之后。
 - 如果 guest 第 1 条指令引发了异常，那么 MTF VM-exit 被 pending 在这个异常 delivery 后第 1 条指令（即异常 handler 的第 1 条指令前）。
 - 如果 guest 第 1 条指令是 INT、INT3 或者 INTO，那么 MTF VM-exit 被 pending 在这些中断 delivery 后第 1 条指令（即中断 handler 的第 1 条指令前）。
 - 如果 guest 第 1 条指令是 HLT，那么 MTF VM-exit 被 pending 在 HLT 指令之后产生 VM-exit，并且唤醒为 active 状态。
- 如果 guest 第 1 条指令是含有 REP 前缀的串指令（譬如 MOVSB，INS 等）：
 - MTF VM-exit 被 pending 在 REP 循环里的第 1 次迭代之后。
 - 假如在执行 REP 循环里的第 1 次迭代时引发了异常，那么 MTF VM-exit 被 pending 在这个异常 delivery 后第 1 条指令前（即异常 handler 的第 1 条指令前）。

4.14.3 MTF VM-exit 与其他 VM-exit

MTF VM-exit 被 pending 在一条指令之后（guest），但尝试执行这条指令时引发了

VM-exit，即 VM-exit 并不是由 MTF VM-exit 造成的。

同样，MTF VM-exit 被 pending 在向量事件的 delivery 之后，但是向量事件直接引发了 VM-exit，即 VM-exit 并不是由于 MTF VM-exit 造成的。

例如，执行 INT3 指令产生的 #BP 异常，由于 exception bitmap 字段的 bit 3 为 1 而直接引发了 VM-exit，或者 pending debug exception 在 deliver 时，由于 exception bitmap 字段的 bit 1 为 1 而直接引发了 VM-exit 或者执行一条指令时发生了异常，而这个异常直接引发了 VM-exit。

4.14.4 MTF VM-exit 的优先级别

由“monitor trap flag”位而产生的 VM-exit 与注入 pending MTF VM-exit 事件具有同样的优先级别（参见 4.15.3 节）。

也就是，低于 TPR threshold VM-exit、INIT 和 SMI 事件，而高于 pending debug exception、VMX-preemption timer、NMI、外部中断等。

4.14.5 例子 4-2

示例 4-2: VMM 利用 MTF 对 guest 进行单步调试

实现 VMM 对 guest 的指令单步调试功能，可以使用下面的方法。

- (1) guest RFLAGS.TF = 1 时捕捉 #DB 异常的执行（由 #DB 异常直接产生 VM-exit）。
- (2) 利用 trap 类型的 MTF VM-exit 机制捕捉 guest 指令的执行。

MTF VM-exit 是 VMX 架构引入的一种调试手段，具有捕捉 #DB 异常方法无法比拟的优势。首先，MTF VM-exit 不会被“blocking by MOV-SS”阻塞状态阻塞，而当 guest 的指令单步调试（TF=1）出现在 MOV-SS 指令后面时会被阻塞，以致于 MOV-SS 指令会被错过。其次，MTF VM-exit 的优先级别高于 trap 类型的 #DB 异常，即使 guest 在开启 TF = 1 时也能对 guest 进行调试。也就是说 guest 的 #DB 异常可以被 MTF 监控。最后，MTF VM-exit 使用非常方便，仅仅需要设置“monitor trap flag”位即可。

在这个实验里，我们只开启一个虚拟机 guest1 进行测试。为了直观地查看效果，加入了对几条指令的简单反汇编功能，lib\Decode\Decode.asm 模块是反汇编的入口，具体的 decode 功能实现在 lib\Decode\DoOpcode.asm 模块里。

代码片段 4-5:

```
... ..
mov esi, 60h
mov edi, foo
call install_kernel_interrupt_handler

SET_PRIMARY_PROCBASED_CTL5    MONITOR_TRAP_FLAG    ; 启用 MTF 调试功能
```

```

%if __BITS__ == 64
    mov rax, [fs: SDA.DmbBase]
    mov DWORD [rax + DMB.DecodeEntry], guest_entry1 ; 起始 decode 点设在 guest 第 1 条
指令
%else
    mov eax, [fs: SDA.DmbBase]
    mov DWORD [eax + DMB.DecodeEntry], guest_entry1
%endif

;;
;; 需要 VMM 进行解码
;;
mov DWORD [R5 + PCB.GuestA + VMB.DoProcessParam], DO_PROCESS_DECODE
... ..

```

这是 chap04\ex4-2\ex.asm 文件里的 TargetCpuVmentry1 函数代码片段，是 guest1 的设置代码。通过 SET_PRIMARY_PROCBASED_CTL5 宏来设置 primary processor-based VM-execution control 字段的“monitor trap flag”位。

这段代码的额外工作是为 60H 设置一个中断例程，并且 guest_entry1 作为 decode 的开始地址，赋给 SDA 结构内 **DMB** (Decode Manage Block) 内的 DecodeEntry 值，并且给 DoMTF 例程传递一个处理参数 DO_PROCESS_DECODE，指示需要进行解码工作。

代码片段 4-6:

```

foo:
    mov eax, 11
    REX.Wrxb
    iret

```

foo 是 60H 中断的服务例程，作为演示目的，仅有两条指令。

代码片段 4-7:

```

guest_entry1:
    mov eax, 1
    mov eax, 2
    mov eax, 3
    int 60h
    mov eax, 4
    mov eax, 5
    mov eax, 6

    jmp $

```

代码片段 4-7 是 guest1 的代码，作为演示目的，也只是简单地执行几条 MOV 指令，其中一条是 60H 中断的调用指令 INT，目的是捕捉中断的执行。

代码片段 4-8:

```

;-----
; DoMTF()
; input:
;     none
; output:
;     none
; 描述:
;     1) 处理由 pending MTF VM-exit 引发的 VM-exit

```

```

;-----
DoMTF:
    push ebp
    push ebx
    push edx
    push ecx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

;;
;; 如果不需要 decode, 则跳过
;;
cmp esi, DO_PROCESS_DECODE
jne DoMTF.done

;;
;; 分析 guest 环境
;;
mov eax, [ebp + PCB.EntryControlBuf + ENTRY_CONTROL.VmEntryControl]
mov edx, [ebp + PCB.GuestStateBuf + GUEST_STATE.CsAccessRight]
test eax, IA32E_MODE_GUEST
jz DoMTF.@1
test edx, SEG_L
jz DoMTF.@1
mov eax, TARGET_CODE64
jmp DoMTF.@2
DoMTF.@1:
    mov eax, TARGET_CODE32
    test edx, SEG_D
    jnz DoMTF.@2
    mov eax, TARGET_CODE16
DoMTF.@2:

    REX.Wrxb
    mov ebp, [ebp + PCB.SdaBase]
    REX.Wrxb
    mov ebx, [ebp + SDA.DmbBase]

    mov [ebx + DMB.TargetCpuMode], eax

;;
;; 对 GuestRip 处进行 decode
;;
mov esi, [ebx + DMB.DecodeEntry]
REX.Wrxb
mov edi, [ebx + DMB.DecodeBufferPtr]
call Decode
test eax, DECODE_STATUS_FAILURE
jnz DoMTF.done

    REX.Wrxb
    mov edx, edi

```

```

;;
;; 更新 debug record 信息
;;
mov eax, [ebx + DMB.DecodeEntry]
xor edi, edi
REX.Wrxb
mov esi, [ebx + DMB.DecodeBufferPtr]
call update_append_msg

REX.Wrxb
mov [ebx + DMB.DecodeBufferPtr], edx

;;
;; 指向 guest 下一条指令
;;
GetVmcsField    GUEST_RIP
mov [ebx + DMB.DecodeEntry], eax

mov ecx, VMM_PROCESS_RESUME
DoMTF.done:
mov eax, ecx
pop ecx
pop edx
pop ebx
pop ebp
ret

```

这段代码位于\lib\VMX\VmxFMM.asm 文件里，当产生 MTF VM-exit 时，VMM 调用 DoMTF 例程进行处理。它的主要工作是对 DecodeEntry 地址里的指令进行反汇编输出，DoMTF 例程最后需要更新解码地址，指向 guest 的下一条指令。

编译及运行

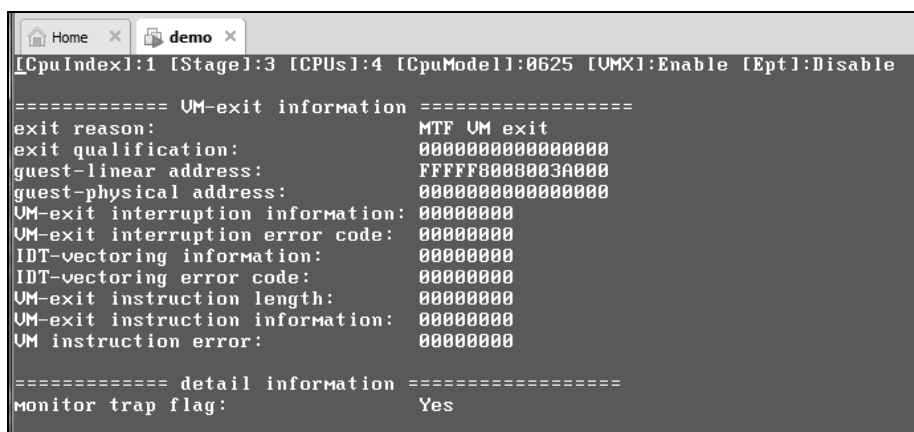
使用下面的命令进行编译及生成映像文件。

- 编译为 64 位代码：build -D__X64 -DDEBUG_RECORD_ENABLE。
- 编译为 32 位代码：build -DDEBUG_RECORD_ENABLE。

这里需要定义符号 DEBUG_RECORD_ENABLE 开启 debug record 记录功能，用来观察信息输出。

在 VMware 里加载映像文件 demo.img 运行，在 CPU0 屏幕里的命令选择器中按下数字<1>键，让 CPU1 进入 guest1 执行，然后按下<F2>键切换到 CPU1 屏幕里。

如图 4-10 所示，CPU1 由于“monitor trap flag”位而产生了 MTF VM-exit。



```

Home x demo x
[CpuIndex]:1 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable

===== UM-exit information =====
exit reason:                MTF UM exit
exit qualification:         0000000000000000
guest-linear address:       FFFFFFFF0000003A000
guest-physical address:     0000000000000000
UM-exit interruption information: 00000000
UM-exit interruption error code: 00000000
IDT-vectoring information:  00000000
IDT-vectoring error code:   00000000
UM-exit instruction length: 00000000
UM-exit instruction information: 00000000
UM instruction error:       00000000

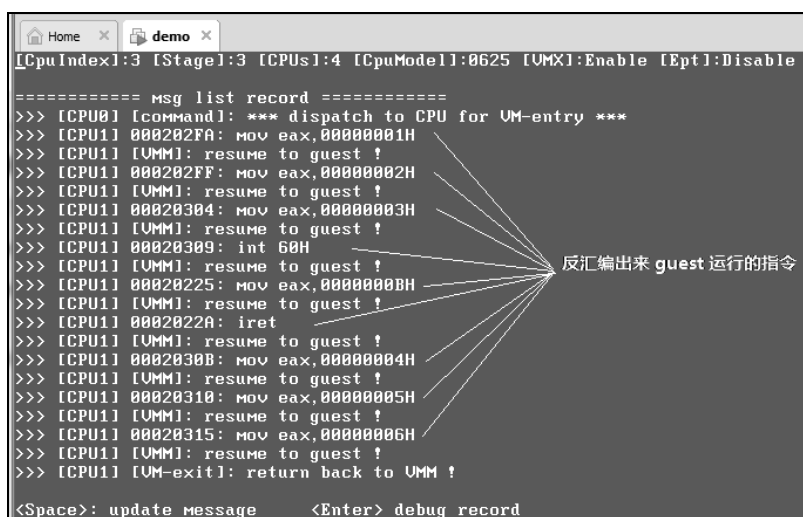
===== detail information =====
monitor trap flag:         Yes

```

图 4-10

接着按<F4>键切换到 CPU3 屏幕观察 debug record 信息列表，然后按下<Enter>键切换到 msg list record 显示结果上。

如图 4-11 所示，在 debug record 列表里显示了反汇编出来的几条指令（对比前面的代码片段 4-6 和 4-7，看是否正确）。反汇编指令的信息中，也包括了指令地址（guest 的 RIP 值）。



```

Home x demo x
[CpuIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable

===== msg list record =====
>>> [CPU0] [command]: *** dispatch to CPU for UM-entry ***
>>> [CPU1] 000202FA: mov eax,0000001H
>>> [CPU1] [VMM]: resume to guest !
>>> [CPU1] 000202FF: mov eax,0000002H
>>> [CPU1] [VMM]: resume to guest !
>>> [CPU1] 00020304: mov eax,0000003H
>>> [CPU1] [VMM]: resume to guest !
>>> [CPU1] 00020309: int 60H
>>> [CPU1] [VMM]: resume to guest !
>>> [CPU1] 00020225: mov eax,0000000BH
>>> [CPU1] [VMM]: resume to guest !
>>> [CPU1] 0002022A: iret
>>> [CPU1] [VMM]: resume to guest !
>>> [CPU1] 0002030B: mov eax,0000004H
>>> [CPU1] [VMM]: resume to guest !
>>> [CPU1] 00020310: mov eax,0000005H
>>> [CPU1] [VMM]: resume to guest !
>>> [CPU1] 00020315: mov eax,0000006H
>>> [CPU1] [VMM]: resume to guest !
>>> [CPU1] [UM-exit]: return back to UMM !

<Space>: update message    <Enter> debug record

```

反汇编出来 guest 运行的指令

图 4-11

注意：作为实验演示目的，Decode 模块远远没有实现完整，仅仅能反汇编这几条指令。因此，**不要试图反汇编其他指令**。Decode 模块将在后续有需要时再一步步实现。

每条反汇编信息后面是一条 “[VMM]: resume to guest” 信息，如果将这些信息关闭后重新编译一次运行（在 lib\VMX\VmxFMM.asm 文件的 VmmEntry 函数里注释掉！）结果如图 4-12 所示。

```

[CPUIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable

===== msg list record =====
>>> [CPU0] [command]: *** dispatch to CPU for VM-entry ***
>>> [CPU1] 000202FA: mov eax,00000001H
>>> [CPU1] 000202FF: mov eax,00000002H
>>> [CPU1] 00020304: mov eax,00000003H
>>> [CPU1] 00020309: int 60H
>>> [CPU1] 00020225: mov eax,0000000BH
>>> [CPU1] 0002022A: iret
>>> [CPU1] 0002030B: mov eax,00000004H
>>> [CPU1] 00020310: mov eax,00000005H
>>> [CPU1] 00020315: mov eax,00000006H
>>> [CPU1] [VM-exit]: return back to UMM !

```

图 4-12

去掉一些额外的 debug record 信息，图 4-12 的显示效果更为直观了。CPU1 的运行流程是：VM-entry → 执行 guest 第 1 条指令 → MTF VM-exit → DoMTF → resume → ... MTF VM-exit → DoMTF → resume → VM-exit。

CPU1 不断重复 MTF VM-exit，然后由 DoMTF 函数执行反汇编功能，再由 VMM 进行 resume 操作，guest 执行下一条指令后又一次产生 MTF VM-exit，直到 DoMTF 函数遇到不可识别的指令后停止反汇编，停留在 VMM 代码里。

目前，jmp 指令没有加入反汇编识别。因此，DoMTF 函数遇到 jmp 指令时停止反汇编，即 guest 里“mov eax, 6”指令的下一条指令（参见代码片段 4-7）。

从图 4-12 里，我们可以看到：guest 在执行完“mov eax, 3”指令后，执行了 INT 中断调用，然后转入执行中断例程里的“mov eax, 11”指令，执行 IRET 指令返回，接着执行 MOV 指令。

最后，我们按下<Enter>键切换回 debug record 明细信息，按<PgDn>键选取一条记录察看。

如图 4-13 所示的明细信息里，我们看到 guest 执行完“mov eax, 6”指令后，RAX 寄存器的值为 6。

```

[CPUIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable

===== debug record <#10> =====
[CPU1]: RIP: 0002A061-Rflags: 00000002
RAX: 0000000000000006 RCX: 00000000C000101
RDX: 0000000000000000 RBX: 000000000000000B
RSP: FFFFFFFF80FFE03FE8 RBP: FFFFFFFF8000002000
RSI: 0000000000004002 RDI: 00000000BE21E7F2
R8: FFFFFFFF80000022600 R9: 0000000000000000
R10: 0000000000000000 R11: 0000000000000000
R12: 0000000000000000 R13: 0000000000000000
R14: 0000000000000000 R15: 0000000000000000

[...lib\Umx\UmxUMM.asm]:38
>>> mov eax,00000006H

```

图 4-13

4.15 VM-entry 后直接导致 VM-exit 的事件

在 VM-entry 完成后某些事件可以直接导致 VM-exit 发生，包括下面的事件：

- (1) 由 VTPR[7:4]值低于 TPR threshold 的 bits 3:0 而导致 VM-exit。
- (2) 由 pending MTF VM-exit 导致 VM-exit。
- (3) 在 pending debug exception 的 delivery 中，由于 exception bitmap 字段的 bit 1 为 1 而导致 VM-exit。
- (4) 由 VMX-preemption timer 超时而导致 VM-exit。
- (5) 由 primary processor-based VM-execution control 字段的 “NMI-window exiting” 位为 1 而导致 VM-exit。
- (6) 由 primary processor-based VM-execution control 字段的 “interrupt-window exiting” 位为 1 而导致 VM-exit。

在 VM-entry 后如果检测到存在上面 6 种事件，会在 guest 第 1 条指令执行前立即直接导致 VM-exit。当有多个事件同时存在时，会按事件的优先级别进行处理。

注意：因接收到 INIT、SMI 或者 SIPI 而导致 VM-exit 的事件并不在本节所讨论的事件里，它们属于异步事件。

4.15.1 VM-exit 事件的优先级别

在上面 6 种 VM-entry 后能直接导致 VM-exit 的事件里，优先级别由高到低排列分别是：**TPR below threshold > pending MTF VM-exit > pending debug exception > VMX-preemption timer > NMI-window exiting > interrupt-window exiting**。

以大于号形式进行排列，在这 6 种事件里 TPR threshold 事件拥有最高的优先级，最低级别是 interrupt-window exiting 事件。

如果算上 SMI、INIT、NMI 和 external-interrupt 则有下面的排列：**TPR below threshold > SMI, INIT > pending MTF VM-exit > pending debug exception > VMX-preemption timer > NMI-window exiting > NMI exiting > NMI > interrupt-window exiting > external-interrupt exiting > external-interrupt**。

NMI exiting 的优先级大于 NMI 事件。因为，收到 NMI 请求会产生 exiting 而不是 deliver NMI 执行（前提是设置了 NMI exiting）。同理，external-interrupt exiting 优先级大于 external-interrupt 事件。因为，收到 external-interrupt 后会产生 VM-exit 而不是 deliver 外部中断执行（前提是设置了 external-interrupt exiting）。

4.15.2 TPR below threshold VM-exit

当 primary processor-based VM-execution control 字段的 “use TPR shadow” 为 1 时，表示启用 virtual-APIC page 页面以及 TPR threshold 值。

在启用 TPR threshold 值的前提下，决定 VM-entry 操作行为需要区分以下几种情况：

(1) 当 secondary processor-based VM-execution control 字段的 “virtualize APIC accesses” 与 “virtual-interrupt delivery” 都为 0 时，TPR threshold 字段 bits 3:0 值不能大于 VTPR[7:4] 值（小于等于），否则会引起 VM-entry 失败。

(2) 当 “virtual-interrupt delivery” 为 1 时，在 VM-entry 后将进行 “pending 的虚拟中断评估” 操作，如果评估通过，那么将 deliver 一个虚拟中断执行。

(3) 当 “virtual APIC accesses” 为 1，“virtual-interrupt delivery” 为 0 时，如果在 VM-entry 后检测到 VTPR[7:4] 值低于 TPR threshold 字段 bits 3:0 值，则立即产生 VM-exit。

由于 TPR threshold 值引发 VM-exit 的原因码为 43，指示 “由 TPR 低于 threshold 而导致 VM-exit”。

1. TPR below threshold VM-exit 优先级

VM-entry 后由于 TPR threshold 而直接引发的 VM-exit 具有非常高的优先级别，高于 pending MTF VM-exit 事件。在 guest 中，当对 VTPR 进行写时，写入的 VTPR[7:4] 低于 TPR threshold 的 bits 3:0 时引发的 VM-exit，它的优先级别高于 SMI 与 INIT 事件。

2. TPR below threshold VM-exit 特性

- TPR below threshold VM-exit 不能被 eflags.IF 标志阻塞（即使 IF = 0）。
- TPR below threshold VM-exit 不能被 interruptibility state 字段里设置的阻塞状态阻塞。
- TPR below threshold VM-exit 不能被 HLT 状态阻塞，但可以被 shutdown 或 wait-for-SIPI 状态阻塞。假如 VM-entry 后处理器处于 HLT 状态，TPR below threshold VM-exit 可以唤醒处理器产生 VM-exit，但不能唤醒 shutdown 和 wait-for-SIPI 状态，也就是 VM-entry 后处理器处于 shutdown 或者 wait-for-SIPI 状态时，VTPR[7:4] 低于 TPR threshold 字段的 bits 3:0 值并不能产生 VM-exit。
- 假如 VM-entry 后处理器处于 shutdown 状态，一个 NMI 事件能唤醒处理器的 shutdown 状态。当 Pin-based processor VM-execution control 字段的 “NMI exiting” 为 0 时，NMI 得到 delivery，处理器被唤醒为 active 状态维持在 VMX non-root operation 模式里。此时，如果 VTPR[7:4] 低于 TPR threshold 字段的 bits 3:0 则产生 VM-exit。

可是在笔者的 westmere 平台上测试，不知何故 activity-state 字段设为 shutdown 状态，在 VM-entry 后直接产生 VM-exit（原因是接收到 INIT 信号），这与《Intel 手册》的描述大相径庭。因此，姑且认为 westmere 平台的 VMX 实现并不完善。

4.15.3 pending MTF VM-exit

以下两种方式可以 pending 一个 MTF VM-exit 事件：

(1) 通过事件注入方式，设置的注入事件类型为 7 (other)，并且向量号为 0 时，将注入 pending MTF VM-exit 事件。在 VM-entry 后直接引发 VM-exit。

(2) 设置 primary processor-based VM-execution control 字段的“monitor trap flag”位为 1 时，将 MTF VM-exit 事件 pending 在 guest 第 1 条指令之后（参见 4.14 节）。

注意：使用“monitor trap flag”位进行 pending MTF VM-exit（属于 trap 类型）并不能在 VM-entry 后立即引发 VM-exit，在 guest 执行完第 1 条指令后才引发 VM-exit。因此，它不属于 VM-entry 后直接引发 VM-exit 事件。

1. pending MTF VM-exit 优先级

由注入的 pending MTF VM-exit 事件直接引发的 VM-exit 具有很高的优先级别，高于 pending debug exception，但低于 TPR below threshold VM-exit、SMI 与 INIT 事件。

2. pending MTF VM-exit 特性

当 VM-entry 后处理器处于 HLT 状态，pending MTF VM-exit 事件能直接引发 VM-exit 并唤醒处理器进入 active 状态，但不能唤醒 shutdown 或者 wait-for-SIPI 状态，在这些状态下 pending MTF VM-exit 将被阻塞不能产生 VM-exit。

假如处理器处于 shutdown 状态时，一个事件唤醒为 active 状态（例如，收到 NMI 但并不产生 VM-exit，而是 deliver 执行），MTF VM-exit 将在事件的 delivery 后产生。

4.15.4 由 pending debug exception 引发的 VM-exit

当存在有效的 pending debug exception 时（参见前面 4.13 节），VM-entry 的注入事件（如果存在）deliver 后紧接着 deliver #DB 异常。如果此时 exception bitmap 字段的 bit 1（对应 #DB 异常）为 1，这个 #DB 异常的 delivery 会直接导致 VM-exit。

1. pending debug exception 优先级

由 pending #DB 引发的 VM-exit 优先级比 VMX-preemption timer 高，但比 TPR below threshold VM-exit、SMI、INIT 以及 pending MTF VM-exit 要低。

2. pending debug exception 特性

pending debug exception 引起的 VM-exit 可以唤醒 HLT 状态，但不能唤醒 shutdown 和 wait-for-SIPI 状态（在这些状态下被阻塞）。

4.15.5 VMX-preemption timer

VMX 架构下支持抢占式的定时器（VMX-preemption timer），允许 VMM 分配每个 VM 固定的执行时间片（基于 TSC 值），当 VM 的时间片用完后产生 VM-exit。

实现原理是：当 Pin-based VM-execution control 字段的“activate VMX-preemption timer”位为 1 时，在 VM-entry 开始的那一刻起就开启这个抢占式定时器，将初始计数值进行递减，直至减为 0 时停止计数值并产生 VM-exit。而 guest-state 区域的 VMX-preemption timer value 字段提供一个 32 位无符号初始计数值。

1. VMX-preemption timer 优先级

由 VMX-preemption timer 超时引发的 VM-exit 事件优先级高于由“NMI-window exiting”引发的 VM-exit、NMI，由“interrupt-window exiting”引发的 VM-exit 以及外部中断请求。

但低于 TPR threshold VM-exit、SMI、INIT、pending MTF VM-exit 以及 pending debug exception。

2. VMX-preemption timer 特性

由 VMX-preemption timer 超时引发的 VM-exit 可以唤醒 HLT 和 shutdown 状态，即在 HLT 或 shutdown 状态下不能阻塞 VMX-preemption timer VM-exit。

但不能唤醒 wait-for-SIPI 状态，也就是 wait-for-SIPI 状态下会阻塞 VMX-preemption timer VM-exit 事件。

3. VMX 定时器计数速率

VMX-preemption timer 计数值递减的速率取决于 IA32_VMX_MISC 寄存器 bits 4:0 值（参见 2.5.11 节）。bits 4:0 提供一个 X 值，当 64 位 TSC 值的位 X 发生改变时 VMX-preemption timer 计数值减 1。举例来说，假设 X 值为 5，TSC[5] 值改变时（由 0 变 1 或由 1 变 0，也就是 TSC 的值每增加 32），VMX-preemption timer 计数值减 1。

VMX-preemption timer 允许在处理器 C-states 的 C0、C1 以及 C2 状态下计数（不能在超过 C2 的深度睡眠状态下计数），也允许在处理器 shutdown 或者 wait-for-SIPI 状态下计数。但是处理器在 wait-for-SIPI 状态下 VMX-preemption timer 计数值减为 0 时不会产生 VM-exit，在其他状态下计数值减为 0 时将产生 VM-exit。

4. 测量 VM-entry 所需的时间

假如在 VM-entry 操作进行时 VMX-preemption timer 计数值就已经减为 0 值，那么在 VM-entry 完成后 guest 第 1 条指令执行前会产生 VM-exit。

在 VM-entry 期间需要耗费一些处理时间，我们可以使用 VMX-preemption timer 来测量 VM-entry 所需要的时间（TSC 值），做如下设置：

- pin-based VM-execution control 字段“activate VMX-preemption timer”为 1。
- VM-exit control 字段的“save VMX-preemption timer value”为 1。
- 在 VMX-preemption timer value 字段里提供一个较大的值，譬如设为 0FFFFFFFFh 值。
- 注入一个 pending MTF VM-exit 事件。

由于注入的 pending MTF VM-exit 事件令 VM-entry 完成后直接产生 VM-exit，因而 VMX-preemption timer 计数器的值保存在 guest-state 区域的 VMX-preemption timer value 字段里。通过查看这个字段值可以计算出 VM-entry 已经消耗了多少时间。

在笔者测试平台里，VM-entry 操作过程中计数器消耗了 67 个值，也就是约为 $67 \times 32 = 2144$ 个 TSC 时间值（按每 32 个 TSC 值递减 1 次计算）。在 2.2GHz 的机器上大约等于 $1\mu\text{s}$ （微秒）的时间。

5. SMM 模式下的 VMX-preemption timer

在 VMX 模式下（包括 root 与 non-root），当收到 SMI 请求时会产生下面两种处理。

(1) SMM 的默认处理：收到 SMI 后处理器会保存当前 VMX 模式必要的相关信息，然后退出 VMX 模式切换到 SMM 模式执行 SMI 例程。直到执行 RSM 指令退出 SMM 模式，然后加载保存的 VMX 模式信息重新进入 VMX 模式。

(2) SMM 双重监控处理：在启用“SMM 双重监控处理”机制后，收到 SMI 会产生 SMM VM-exit 行为，处理器保持在 VMX 模式下切入到 SMM 模式里执行 SMM-transfer monitor 例程代码。直到执行 VMRESUME 指令产生 VM-entry that return from SMM 行为，从 SMM-transfer monitor 切换回 executive monitor 执行。

在第 (1) 种 SMM 默认处理中，在切换 SMM 执行到 RSM 指令返回到 VMX non-root operation 模式这段期间，VMX-preemption timer 会保持计数。如果在执行 SMI 例程期间发生计数器减为 0 值，只有在返回到 VMX non-root operation 模式后才会产生 VM-exit。

在第 (2) 种 SMM 双重监控处理机制下，切入和退出 SMM 模式也被相应地称为 VM-exit 与 VM-entry。VMX-preemption timer 超时的处理和普通情况下一致（即和 VMX non-root operation 环境超时产生 VM-exit 行为一致）。也就是由 SMI 请求产生的 SMM VM-exit 当作普通的 VM-exit 行为，计数器不会进行计数。

4.15.6 NMI-window exiting

当 pin-based VM-execution control 字段的“NMI exiting”与“virtual-NMIs”位都为 1 时，存在“**virtual-NMI window**”（即打开虚拟 NMI 窗口）。并且 primary processor-based VM-execution control 字段的“**NMI-window exiting**”位为 1 时，取决于 guest-state 区域的 interruptibility state 字段，满足下面的条件时将会在 VM-entry 后直接产生 VM-exit：

- “blocking by NMI” 位为 0（不存在 virtual-NMI 的阻塞状态）。
- “blocking by MOV-SS” 位为 0（不存在 MOV-SS 的阻塞状态）。

在某些处理器里，NMI-window exiting 也可能由于“blocking by STI”阻塞状态而被阻塞（依赖于处理器的实现）。

1. NMI-window exiting 优先级

由 NMI-window（即 virtual-NMI window）引发的 VM-exit 事件优先级高于 NMI（即

virtual-NMI)，高于由“interrupt-window exiting”引发的 VM-exit 以及外部中断请求。

低于 TPR below threshold VM-exit、SMI、INIT、pending MTF VM-exit、pending debug exception，以及由 VMX-preemption timer 超时引发的 VM-exit。

2. NMI-window exiting 特性

由 NMI window 引发的 VM-exit 事件可以唤醒 HLT 和 shutdown 状态（即在 HLT 或 shutdown 状态下不能阻塞 NMI-window 引发的 VM-exit），但不能唤醒 wait-for-SIPI 状态（即在 wait-for-SIPI 状态下会阻塞 NMI-window 引发的 VM-exit）。

4.15.7 interrupt-window exiting

当 primary processor-based VM-execution control 字段的“interrupt-window exiting”位为 1 时，打开中断窗口，并且没有“blocking by STI”与“blocking by MOV-SS”阻塞状态，将立即引发 VM-exit。

打开中断窗口是指：当前 eflags.IF 为 0 时，使用 STI、POPF 或者 IRET 指令将 IF 置位允许响应外部中断请求。

在 VM-entry 时 guest-RFLAGS 字段的 IF 位为 1，interruptibility state 字段的 bit 0 与 bit 1 为 0，并且“interrupt-window exiting”为 1 时，VM-entry 后将直接产生 VM-exit。

1. interrupt-window exiting 优先级

由 interrupt-window 引发的 VM-exit 事件优先级高于外部中断请求，也就等同于高于由 pin-based VM-execution control 字段的“external-interrupt exiting”位引发的 VM-exit。低于前面所说的所有 VM-exit 事件，包括 TPR below threshold VM-exit、SMI、INIT、pending MTF VM-exit、pending debug exception、VMX-preemption timer 超时引发的 VM-exit，以及 NMI-window 引发的 VM-exit。

2. interrupt-window exiting 特性

由 interrupt-window 引发的 VM-exit 事件能唤醒处理器的 HLT 状态，但在 shutdown 或 wait-for-SIPI 状态下被阻塞。

4.16 处理器的可中断状态

在 VM-entry 后会根据 guest-state 区域的 interruptibility state 字段值来更新处理器当前的 interruptibility（可中断性）状态信息（即中断是否允许响应）。这个 interruptibility 状态由处理器内部进行维护。

4.16.1 中断的阻塞状态

interruptibility state 字段的 4 个位（bits 3:0）指示 VM-entry 后虚拟处理器是否存在下

面 4 种中断的阻塞状态（参见 3.8.6 节）：

- Bit 0 为 1 时，指示存在“blocking by STI”阻塞状态。
- Bit 1 为 1 时，指示存在“blocking by MOV-SS”阻塞状态。
- Bit 2 为 1 时，指示存在“blocking by SMI”阻塞状态。
- Bit 3 为 1 时，指示存在“blocking by NMI”阻塞状态，或“blocking by virtual-NMI”阻塞状态。

interruptibility state 字段的值可能是在产生 VM-exit 后更新（由 guest 反射而来），也可能由 VMM 主动设置。当属于 VMM 主动设置时，必须满足 VM-entry 时对 interruptibility state 字段的检查（参见 4.5.8 节）。

4.16.2 阻塞状态的解除

中断的阻塞状态会随着处理器的执行而被解除，不同的阻塞状态有不同的解除方法：

- “blocking by STI”阻塞状态随着 STI 指令的下一条指令执行完毕自动解除。

```
... ... ; IF = 0 (中断窗口是关闭的)
sti      ; 打开中断窗口 (interrupt-window)
ret      ; 此处存在“blocking by STI”阻塞状态
          ; 直到这条指令执行完毕，阻塞状态被解除。
```

注意：只有在中断窗口关闭时（即 IF=0），打开中断窗口（即 IF 置位）才会产生“blocking by STI”阻塞状态。如上面的例子所示，“blocking by STI”阻塞状态的意图是“让 STI 指令后的 RET 指令得以顺利执行，避免代码返回前产生中断”。

- “blocking by MOV-SS”阻塞状态随着 MOV-SS（包括 POP SS）指令的下一条指令执行完毕自动解除。

```
... ... ;
mov ss, ax ; 切换栈段
mov rsp, kernel_stack ; 更新栈指针，此处存在“blocking by MOV-SS”阻塞状态！
                      ; 直到这条指令执行完毕，阻塞状态被解除。
```

如上面的例子所示，“blocking by MOV-SS”阻塞状态的意图是“让 MOV-SS 指令后的 RSP 栈指针得到顺利更新，避免栈指针更新前产生中断引用了错误的栈”。

- “blocking by SMI”阻塞状态随着 RSM 指令的执行而解除。这个阻塞状态在 SMI 成功 deliver 进入 SMI 服务例程（切换到 SMM 模式）后产生。因此，在 SMI 服务例程内执行 RSM 指令退出 SMM 模式后解除这个阻塞状态。
- “blocking by NMI”阻塞状态随着 IRET 指令的执行而解除。这个阻塞状态被认为在 NMI 成功 deliver 进入 NMI 服务例程后产生。因此，在 NMI 服务例程内执行 IRET 指令返回被中断者后，解除这个阻塞状态。

在 VMX 架构下，“blocking by NMI”阻塞状态的解除受到 pin-based VM-execution control 字段的“NMI exiting”以及“virtual-NMIs”位的影响：

- 当“NMI exiting”为 1 时，IRET 指令的执行不会对“blocking by NMI”阻塞状态

产生影响。这是由于一个 NMI 的 delivery 直接产生 VM-exit，也就不会存在“blocking by NMI”阻塞状态。

- 在“NMI exiting”为 0 时，执行 IRET 指令会解除“blocking by NMI”阻塞状态，即使这条 IRET 指令执行时产生了某些错误（譬如引发 #GP 异常，或者 EPT violation 等）。
- 当“NMI exiting”与“virtual-NMIs”都为 1 时，interruptibility state 字段的 bit 3 被视为“blocking by virtual-NMI”位，执行 IRET 指令会解除“**blocking by virtual-NMI**”阻塞状态，即使这条 IRET 指令执行时产生了某些错误。

“NMI exiting”位为 0 时，“virtual-NMIs”必须为 0（参见 4.4.1.1 节）。这两个位也影响着 primary processor-based VM-execution control 字段“NMI-window exiting”位（见 4.4.1.2 与 4.15.6 节）。

4.16.3 中断的阻塞

取决于 pin-based VM-execution control 字段的设置，**未被阻塞**的外部中断（external-interrupt）以及 NMI 都可以直接产生 VM-exit。

- “external-interrupt exiting”为 1 时，guest 接收到外部中断请求直接产生 VM-exit。这个外部中断是指由中断控制器（例如 8259 中断控制器，local APIC 或者 I/O APIC 中断控制器）发出的可屏蔽中断请求。
- “NMI exiting”为 1 时，guest 接收到 NMI 请求直接产生 VM-exit。

处理器响应收到的 SMI 是否产生 VM-exit，取决于是否使用 SMM 双重监控处理机制：

- 在 SMM 双重监控处理下，VMX 模式（root 或 non-root）里响应 SMI 将产生 SMM VM-exit，切入到 SMM 执行 SMM-transfer monitor 例程。
- 在 SMM 默认处理下，VMX 模式（root 或 non-root）里响应 SMI 将退出 VMX 模式，切入 SMM 模式执行 SMI 服务例程。

当收到**外部中断**与 **NMI** 不直接产生 VM-exit（排除前面所说的情况）以及 **SMI** 时，处理器当前的阻塞状态影响着这些中断的响应。

- 存在“blocking by STI”或者“blocking by MOV-SS”阻塞状态时，**外部中断**不能被响应。
- 存在“blocking by MOV-SS”阻塞状态时，**NMI** 和 **virtual-NMI**（“virtual-NMIs”为 1 时）不能被响应。依赖于处理器架构的实现，某些处理器上“blocking by STI”状态也能阻塞 NMI 及 virtual-NMI。
- 由于存在 **virtual-NMI** 的概念，“blocking by NMI”阻塞状态与 virtual-NMI 交互作用如下所示：
 - 当 pin-based VM-execution control 字段的“virtual-NMIs”为 1，存在“blocking by NMI”阻塞状态时，将阻塞 **virtual-NMI**（虚拟 NMI）。但是并不阻塞

NMI，接收到 NMI 后会产生 VM-exit（由于“NMI-exiting”为 1）。

- 当“virtual-NMIs”为 0，存在“blocking by NMI”阻塞状态时，NMI 不能被响应。
- 存在“blocking by SMI”阻塞状态时，SMI 不能被响应。NMI 也能被阻塞，直到执行 RSM 指令解除阻塞状态。

只有在处理器的阻塞状态解除后（参见前面 4.16.2 节），guest 收到中断请求才允许响应。另一个情况是：在 SMI 服务例程内收到 NMI 时，这个 NMI 将被锁存，直到执行了 IRET 或 RSM 指令后 deliver。“blocking by MOV-SS”阻塞状态也能阻塞#DB 异常。

4.16.4 VM-entry 后的可中断状态

interruptibility state 字段指示 VM-entry 后的处理器可中断状态。但是当 VM-entry 含有注入事件时，interruptibility state 字段的设置必须符合要求（参见 4.5.8 节），也就是：

- 注入外部中断时，不能存在“blocking by STI”或者“blocking by MOV-SS”阻塞状态。
- 注入 NMI 时，不能存在“blocking by MOV-SS”阻塞状态。在某些处理器上也可能不允许存在“blocking by STI”阻塞状态。
- 注入 virtual-NMI 时，不能存在“blocking by NMI”阻塞状态。

对于其他类型的注入事件（如**硬件异常、软件中断事件**），并没有限制 interruptibility state 字段的设置。在 VM-entry 注入事件 deliver 后，不会存在“blocking by STI”与“blocking by MOV-SS”阻塞状态，即使 interruptibility state 字段的 bit 0 与 bit 1 为 1 值（参见 4.16.2 节）。

当 VM-entry 不含有注入事件，存在“blocking by STI”阻塞状态（除了 RFLAGS 字段的 IF=0 时），“blocking by MOV-SS”阻塞状态（但这两个阻塞状态不能同时存在），或者“blocking by NMI”阻塞状态时，这些阻塞状态在 VM-entry 后的解除如下（参见 4.16.2 节）：

- “blocking by STI”与“blocking by MOV-SS”阻塞状态在 guest 第一条指令执行完毕后解除，或者由于 guest 第 1 条指令引发了异常而解除。
- “blocking by NMI”和“blocking by virtual-NMI”由于执行了 IRET 指令而解除。

当 VM-entry 后执行在 SMM 模式时，处理器存在“blocking by SMI”阻塞状态。interruptibility state 字段的 bit 2 必须为 1（参见 4.5.8 节）。VM-entry 后执行在非 SMM 模式时，处理器不存在“blocking by SMI”阻塞状态，“blocking by SMI”位必须为 0。

4.17 处理器的活动状态

同样，在 VM-entry 后会根据 guest-state 区域 activity state 字段值来更新虚拟处理器当

前的 activity（活动性）状态信息（即处理器执行单元的运行状态），更详细的描述参见 3.8.5 节。

4.17.1 active 与 inactive 状态

activity state 字段值为 0 时，VM-entry 后处理器执行单元（逻辑处理器）处于正常的执行状态，可以执行指令以及响应事件。activity state 字段值为非 0 时，VM-entry 后处理器执行单元（逻辑处理器）处于下面的 inactive 状态。

- HLT 状态（值为 1）：处理器执行单元停止执行指令，由执行 HLT 或 MONITOR 指令产生。
- shutdown 状态（值为 2）：处理器执行单元停止执行指令，由 triple fault 或其他原因引起。
- Wait-for-SIPI 状态（值为 3）：处理器执行单元停止执行指令，可由接收到 INIT 信号而引起。

activity state 字段的值可能是在 VM-exit 产生后更新（由 guest 反射而来），也可能由 VMM 主动设置。当属于 VMM 主动设置时，必须满足 VM-entry 时对 activity state 字段的检查（参见 4.5.7 节）。

注意：每当处理器从 active 状态转变为 inactive 状态时，处理器都会产生一个特殊的 bus 时钟周期来提示发生了“从 active 状态切换到 inactive 状态”。从 inactive 状态转变为 active 状态时，同样会产生这个 bus 周期来提示“从 inactive 状态切换到 active 状态”。

4.17.2 事件的阻塞

在 VM-entry 后，处理器的 active 与 inactive 状态都能阻塞某些事件，如下所示：

- SIPI 消息在 active、HLT 以及 shutdown 状态都被阻塞（也能在 VMX root operation 模式下被阻塞）。
- 外部中断在 shutdown 以及 wait-for-SIPI 状态下被阻塞。
- 外部中断、NMI、SMI 以及 INIT 信号在 wait-for-SIPI 状态下被阻塞。

当收到外部中断请求而被阻塞时（如在 shutdown 状态下），即使 pin-based VM-execution control 字段的“external-interrupt exiting”位为 1 也不能产生 VM-exit。同理，当收到 NMI 请求而被阻塞时（wait-for-SIPI 状态下），即使用“NMI exiting”位为 1 也不能产生 VM-exit。

除了上述的事件外，在前面 4.15 节描述的 6 种事件在某些状态下也能被阻塞。例如，由于 VMX-preemption timer 超时而产生的 VM-exit 在 wait-for-SIPI 状态下被阻塞。由低于 TPR threshold 值而产生的 VM-exit 在 shutdown 及 wait-for-SIPI 状态下被阻塞。

4.17.3 inactive 状态的唤醒

当处理器处于 inactive 状态时，不能被阻塞的事件可以唤醒 inactive 状态。一些事件唤醒后进入 active 状态继续执行，一些事件收到后会直接产生 VM-exit（如 INIT 信号），处理器的 inactive 状态在 VM-exit 完成后才被唤醒（即 VM-exit 当时处理器仍处于 inactive 状态）。

1. 唤醒进入 active 状态

取决于 pin-based VM-execution control 字段的设置，外部中断或 NMI 能唤醒处理器进入 active 状态：

- HLT 状态下，当“external-interrupt exiting”或者“NMI-exiting”为 0 时，收到外部中断或者 NMI 后处理器被唤醒进入 active 状态继续执行。
- shutdown 状态下，“NMI-exiting”为 0 时，收到 NMI 处理器被唤醒进入 active 状态继续执行。

注意：当这些事件唤醒 inactive 状态，但是在事件的 delivery 期间引发了 VM-exit，处理器也已经成功转为 active 状态，也就是在 VM-exit 前处理器处于 active 状态。

2. 唤醒产生 VM-exit

- HLT 状态下，能被外部中断、NMI、SMI 以及 INIT 信号唤醒。
- shutdown 状态下，能被 NMI、SMI 以及 INIT 信号唤醒。
- Wait-for-SIPI 状态下，能被 SIPI 消息唤醒。

注意：在唤醒直接产生 VM-exit 这种情况下，只有在 VM-exit 完成后处理器才会从 inactive 状态转入 active 状态。也就是“从 non-root 切换回 root 后，处理器才处于 active 状态”，在 VM-exit 当时处理器还处于 inactive 状态。在 guest-state 区域保存的 activity state 字段仍指示为 inactive 状态（在 Intel 的文档里，明确说明转入 active 状态是在完成 VM-exit 后。因此，在 VM-exit 当时处理器仍处于 inactive 状态）。

同样，每当处理器从 inactive 状态转为 active 状态时，处理器都会产生特殊的 bus 时钟周期以提示发生了“从 inactive 状态转入 active 状态”。包括在 VM-exit 前转入 active 状态，以及在 VM-exit 完成后转入 active 状态。

在前面 4.15 节描述的 6 种事件在产生 VM-exit 时也能唤醒处理器的 inactive 状态。例如，由于 VMX-preemption timer 超时而引发的 VM-exit 可以唤醒 HLT 与 shutdown 状态。

4.17.4 VM-entry 后的活动状态

activity state 字段指示 VM-entry 后的处理器活动状态，这个字段的设置必须要符合要求（参见 4.5.7 节）。

1. 向量化的 VM-entry

当 VM-entry 含有注入事件时，activity state 字段的设置必须符合下面的要求：

- VM-entry 后不能为 wait-for-SIPI 状态。
- 注入软件中断、特权级软件中断或者软件异常时，activity state 不能为 HLT 状态。
- 注入 #DB 与 #MC 异常以外的**硬件异常**时，activity state 不能为 HLT 状态。
- 注入 NMI 与 #MC 异常以外的事件时，activity state 不能为 shutdown 状态。

除了上述的限制外允许设为 inactive 状态。例如，注入 #DB 异常 activity state 字段允许设为 HLT 状态，注入 NMI 事件 activity state 字段允许设为 shutdown 状态。

在 VM-entry 后处理器将处于 **active 状态**，即使 activity state 字段指示为 inactive 状态（前面所述的允许 inactive 状态）。

2. 非向量化 VM-entry

当 VM-entry 不含有注入事件时，处理器被更新为 activity state 字段指示的状态。如果 VM-entry 后处理器处于 inactive 状态，将产生特殊的 bus 时钟周期提示处理器“**从 active 状态转入 inactive 状态**”。处理器在 inactive 状态下停止执行指令，直到 inactive 状态被唤醒（见前面 4.17.3 节）。

4.18 VM-entry 的机器检查事件

假设在 VM-entry 期间发生了 machine-check event（机器检查事件），取决于这个事件发生在 VM-entry 的哪个阶段，将有不同的处理方式。也取决于 CR4.MCE 位，这个位控制着是否允许产生 #MC 异常，用以修正机器检查错误。

1. 发生在 VM-entry 前

- 当前的 CR4.MCE 为 0 时，处理器当前处于 SMX 模式时将产生 TXT shutdown（错误码为 000CH，指示“不可修复的机器检查错误”）。处理器当前处于非 SMX 模式时产生 shutdown 状态。
- 当前的 CR4.MCE 为 1 时，产生 #MC 异常，通过 host-IDT 进行 deliver。

2. 发生在 VM-entry 期间

机器检查事件发生在 VM-entry 期间将引发 VM-exit，退出原因码为 41，指示“在 VM-entry 期间发生了机器检查事件”。

3. 发生在 VM-entry 完成后

- VM-entry 后的 CR4.MCE 为 0 时，处理器处于 SMX 模式时将产生 TXT shutdown（错误码为 000CH，指示“不可修复的机器检查错误”）。处理器处于非 SMX 模式时产生 shutdown 状态。
- VM-entry 后的 CR4.MCE 为 1 时，#MC 异常的 delivery 取决于 exception bitmap 字段的 bit 18（对应 #MC 异常）。Bit 18 为 1 时，#MC 异常将产生 VM-exit。Bit 18 为 0 时，#MC 异常通过 guest-IDT 进行 deliver。

第 5 章

VM-exit 处理

逻辑处理器在 VMX non-root operation 模式的执行过程中，由于尝试执行某些指令或遇到某些事件后被迫切换回 VMX root-operation 模式，这一行为称为“VM-exit”。另外，在 SMM 双重监控处理机制下，VMM 还可以主动产生 *SMM VM-exit* 行为。

需要注意的是，由于在 VMX non-root operation 环境里被迫发生 VM-exit，guest 软件并不知道自己何时发生过 VM-exit，也不可能检测到自己何时发生过 VM-exit。出于安全性目的，Intel 保证 guest 没有任何途径能检测自己是否处于虚拟机环境里（VMX non-root operation 环境）。

出于保护及虚拟化物理资源的目的，在 non-root 环境里尝试执行某些涉及物理资源访问的指令或者接收到某些事件时会产生 VM-exit。部分指令能无条件地产生 VM-exit，部分指令由于 VMM（virtual machine monitor）设置了触发条件而引发 VM-exit。

5.1 无条件引发 VM-exit 的指令

在 VMX non-root operation 模式下尝试执行下面的指令，将无条件直接引发 VM-exit：

- CPUID，INVD，GETSEC 以及 XSETBV 指令。
- VMX 系列指令，包括：INVEPT，INVVPID，VMCALL，VMCLEAR，VMLAUNCH，VMPTRLD，VMPTRST，VMREAD，VMRESUME，VMWRITE，VMXOFF 以及 VMXON。

VMX 指令集中的 VMFUNC 指令是个例外，它允许在 VMX non-root operation 环境里执行而不产生 VM-exit。前提条件是：secondary processor-based VM-execution control 字段的“enable VM functions”位为 1，EAX 寄存器提供的功能号不大于 63，并且在 VM-function control 字段对应的位为 1 时允许执行。

另外，当“unrestricted guest”为 1 时，如果 guest 进入实模式运行，执行 VMX 系列指令产生 #UD 异常而不是直接引发 VM-exit。此时 exception bitmap 字段 bit 6 为 1 时，由 #UD 而产生 VM-exit。

5.2 有条件引发 VM-exit 的指令

取决于 pin-based VM-execution control 字段，primary processor-based VM-execution control 字段以及 secondary processor-based VM-execution control 字段的设置，在 VMX non-root operation 模式下尝试执行下面的指令将有条件地直接引发 VM-exit。

- **HLT**，当“HLT exiting”为 1 时，尝试执行 HLT 指令将导致 VM-exit。
- **INVLPG**，当“INVLPG exiting”为 1 时，尝试执行 INVLPG 指令将导致 VM-exit。
- **INVPCID**，当“enable INVPCID”和“INVLPG exiting”为 1 时，尝试执行 INVPCID 指令将导致 VM-exit。
- **RDPMP**，当“RDPMP exiting”为 1 时，尝试执行 RDPMP 指令将导致 VM-exit。
- **RDTSC**，当“RDTSC exiting”为 1 时，尝试执行 RDTSC 指令将导致 VM-exit。
- **RDTSCP**，当“enable RDTSCP”和“RDTSC exiting”为 1 时，尝试执行 RDTSCP 指令将导致 VM-exit。
- **RSM**，当在 SMM 双重监控机制下的 SMM 模式内，尝试执行 RSM 指令将导致 VM-exit。
- **MOV to CR3**，当“CR3-load exiting”为 1，并且写入的值不等于其中一个 CR3-target 值或者 CR3-count 值为 0 时，尝试执行 MOV to CR3 指令将导致 VM-exit（参见 3.5.8 节）。
- **MOV from CR3**，当“CR3-store exiting”为 1 时，尝试执行 MOV from CR3 指令将导致 VM-exit。
- **MOV to CR8**，当“CR8-load exiting”为 1 时，尝试执行 MOV to CR8 指令将导致 VM-exit。
- **MOV from CR8**，当“CR8-store exiting”为 1 时，尝试执行 MOV from CR8 指令将导致 VM-exit。
- **MOV to CR0**，当 CR0 的 guest/host mask 字段的某位为 1 时，尝试执行 MOV to CR0 指令，而写入该位的值不等于 CR0 的 read shadows 字段对应位的值时，将导致 VM-exit（见 3.5.7 节）。
- **MOV to CR4**，当 CR4 的 guest/host mask 字段的某位为 1 时，尝试执行 MOV to CR4 指令，而写入该位的值不等于 CR4 的 read shadows 字段对应位的值时，将导致 VM-exit（见 3.5.7 节）。
- **CLTS**，当 CR0 的 guest/host mask 与 read shadows 字段的 bit 3（对应 TS 位）都为 1 时，尝试执行 CLTS 指令将导致 VM-exit。

- **LMSW**

- 当 CR0 的 guest/host mask 字段 bits 3:1 中的某位为 1 时，尝试执行 LMSW 指令，而写入值 bits 3:1 中的该位不等于 CR0 read shadows 字段 bits 3:1 中对应位的值时，将导致 VM-exit（注意，LMSW 指令从不清 CR0.PE 位）。
- 当 CR0 的 guest/host mask 字段 bit 0 (CR0.PE) 为 1 时，尝试执行 LMSW 指令，而写入值的 bit 0 为 1，但 CR0 read shadows 字段的 bit 0 为 0 值时，将导致 VM-exit。

- **MOV-DR**，当 “MOV-DR exiting” 为 1 时，尝试执行 MOV 指令读/写 DR 寄存器将导致 VM-exit。

- **IN/OUT**

- 当 “unconditional I/O exiting” 为 1，并且 “use I/O bitmap” 为 0 时，尝试执行 IN/OUT 类（包括 INS/OUTS）指令访问 I/O 端口地址，将导致 VM-exit。
- 当 “use I/O bitmap” 为 1，并且 I/O bitmap 的某位为 1 时，尝试执行 IN/OUT 类（包括 INS/OUTS）指令访问该位对应的 I/O 端口地址，将导致 VM-exit。

- **RDMSR**

- 当 “use MSR bitmap” 为 0 时，尝试执行 RDMSR 指令将导致 VM-exit。
- 当 “use MSR bitmap” 为 1 时，使用 RDMSR 指令读 MSR，但 ECX 寄存器提供的 MSR 地址值不在 00000000H – 00001FFFH 或者 C0000000H – C0001FFFH 范围内，将导致 VM-exit。
- 当 “use MSR bitmap” 为 1 时，使用 RDMSR 指令读 MSR，但 ECX 寄存器提供的 MSR 地址值对应在 MSR read bitmap 的位为 1，将导致 VM-exit（见 3.5.15 节）。

- **WRMSR**

- 当 “use MSR bitmap” 为 0 时，尝试执行 WRMSR 指令将导致 VM-exit。
- 当 “use MSR bitmap” 为 1 时，使用 WRMSR 指令写 MSR，但 ECX 寄存器提供的 MSR 地址值不在 00000000H – 00001FFFH 或者 C0000000H – C0001FFFH 范围内，将导致 VM-exit。
- 当 “use MSR bitmap” 为 1 时，使用 WRMSR 指令写 MSR，但 ECX 寄存器提供的 MSR 地址值对应在 MSR write bitmap 的位为 1，将导致 VM-exit（见 3.5.15 节）。

- **MWAIT**，当 “MWAIT exiting” 为 1 时，尝试执行 MWAIT 指令将导致 VM-exit。

- **MONITOR**，当 “MONITOR exiting” 为 1 时，尝试执行 MONITOR 指令将导致 VM-exit。

- **PAUSE**

- 当 “PAUSE exiting” 为 1 时，尝试执行 PAUSE 指令将导致 VM-exit。
- 当 “PAUSE exiting” 为 0，“PAUSE-loop exiting” 为 1，并且 CPL=0 时，PAUSE 指令在一个循环里执行的时间超过设置的 PLE_window 值时，将导致 VM-exit（见 3.5.2 节和 3.5.19 节）。

- **LGDT, SGDT, LIDT, SIDT, LLDT, SLDT, LTR, STR**: 当 “descriptor-table exiting” 为 1 时, 尝试执行这些指令访问描述符表寄存器将导致 VM-exit。
- **RDRAND**, 当 “RDRAND exiting” 为 1 时, 尝试执行 RDRAND 指令将导致 VM-exit。
- **WBINVD**, 当 “WBINVD exiting” 为 1 时, 尝试执行 WBINVD 指令将导致 VM-exit。
- **VMFUNC**
 - 当 “enable VM functions” 为 1 时, 尝试执行 VMFUNC 指令, EAX 寄存器提供的功能号在 VM-function control 字段相应的位为 0 时, 将导致 VM-exit。
 - 当 “enable VM functions” 为 1 时, 尝试执行 VMFUNC 指令, EAX 寄存器提供的功能号为 0, 并且 ECX 寄存器提供的 EPT entry 索引值大于 511 时, 将导致 VM-exit。

在 VMX non-root operation 模式里, **INVPCID** 与 **RDTSCP** 指令需要开启才可执行。当 secondary processor-based VM-execution control 字段的 “enable INVPCID” 或 “enable RDTSCP” 位为 0 时, 执行 INVPCID 或 RDTSCP 指令会产生 #UD 异常。

LMSW 指令允许对 CR0.PE 进行置位。但是, 即使源操作数的 bit 0 为 0, LMSW 指令也不会执行 CR0.PE 清位工作。因此, 当 CR0 的 guest/host mask 字段 bit 0 为 1 时, 尝试执行 LMSW 指令, 即使写入值的 bit 0 为 0, 并且 CR0 shadow 值的 bit 0 为 1, 也不会产生 VM-exit (LMSW 不清 bit 0 为 0)。

5.3 引发 VM-exit 的事件

在 VMX non-root operation 里, 当接收到某些事件时可以产生 VM-exit。一些事件也会由于处理器的不可中断状态或者非活动状态而被阻塞 (参见 4.16.3 节及 4.17.2 节)。

收到下面的事件将直接产生 VM-exit (假设不存在 activity state 或者 interruptibility state 的阻塞, 以及中断控制器的屏蔽):

- **INIT**, 当处理器不在 wait-for-SIPI 状态时, 收到 INIT 信号将引发 VM-exit。
- **SIPI**, 当处理器处于 wait-for-SIPI 状态时, 收到 SIPI 消息将引发 VM-exit。
- **SMI**, 在 SMM 双重监控处理机制下, 收到 SMI 将引发 SMM VM-exit。
- **NMI**, 当 pin-based VM-execution control 字段的 “NMI exiting” 为 1 时, 收到 NMI 将引发 VM-exit。
- **外部中断**, 当 pin-based VM-execution control 字段的 “external-interrupt exiting” 为 1 时, 收到外部中断请求将产生 VM-exit。
- **异常 (硬件异常与软件异常)**
 - 发生 #PF 以外的异常时 (包括硬件与软件异常), 当异常向量号在 exception bitmap 字段对应的位为 1 时引发 VM-exit。
 - 发生 #PF 异常时, 当 exception bitmap 的 bit 14 为 1 并且 PFEC & PFEC_MASK

字段 = PFEC_MATCH 字段时, 这个#PF 异常导致 VM-exit (参见 3.5.3 节和 3.5.4 节)。

- **NMI-window exiting**, 当 pin-based VM-execution control 字段的 “NMI exiting” 与 “virtual-NMIs” 为 1, 并且 Primary processor-based VM-execution control 字段的 “NMI-window exiting” 为 1 时, 在 VM-entry 后立即产生 VM-exit (参见 4.15.6 节)。
- **Interrupt-window exiting**, 当 Primary processor-based VM-execution control 字段的 “interrupt-window exiting” 为 1 并且 eflags.IF=1 时, 在 VM-entry 后立即产生 VM-exit (参见 4.15.7 节)。
- **VMX-preemption timer**, 在 pin-based VM-execution control 字段的 “activate VMX-preemption timer” 为 1 时, 由于 VMX-preemption timer 计数器减为 0 而引发 VM-exit (参见 4.15.5 节)。
- **Pending MTF VM-exit**
 - 注入一个 pending MTF VM-exit 事件, 在 VM-entry 后直接产生 VM-exit (参见 4.15.3 节)。
 - 当 Primary processor-based VM-execution control 字段的 “monitor trap flag” 为 1 时, 在 VM-entry 后 guest 第 1 条指令执行完毕后产生 VM-exit。
- **Pending debug exception**, 在 VM-entry 时存在悬挂的#DB 异常, #DB 异常的 delivery 由于 exception bitmap 字段的 bit 1 为 1 而导致 VM-exit (参见 4.15.4 节)。
- **Triple fault**
 - 执行一条指令产生了一连串异常, 而这些异常并不直接引发 VM-exit, 继而最终转换为 triple fault 导致 VM-exit。
 - 一个向量事件的 delivery 期间最终引发了 triple fault 而导致 VM-exit。或者#DF 异常的 delivery 期间引发了另一个异常而导致 VM-exit (参见 4.12.2 节)。
- **任务切换**
 - 执行 CALL, JMP 或者 IRET 指令尝试进行任务切换时引发 VM-exit。
 - 一个向量事件的 delivery 期间尝试进行任务切换时引发 VM-exit。
- **EPT violation**, 当 secondary processor-based VM-execution control 字段的 “enable EPT” 为 1 时, 指令的执行或者向量事件 delivery 期间引发了 EPT violation 而导致 VM-exit。
- **EPT misconfiguration**, 当 “enable EPT” 为 1 时, 指令的执行或者向量事件 delivery 期间引发了 EPT misconfiguration 而导致 VM-exit。
- **TPR below threshold**, 当 primary processor-based VM-execution control 字段的 “use TPR shadow” 与 secondary processor-based VM-execution control 字段的 “virtualize APIC accesses” 为 1 时, 有下面的情况。
 - 在 VM-entry 后, 检测到 VTPR[7:4]值低于 TPR threshold 字段的 bits 3:0 值将立即引发 VM-exit (参见 4.15.2 节)。

- 在更新 VTPR 的操作里 (**TPR 虚拟化**)，写入的 VPTR[7:4]值低于 TPR threshold 字段的 bits 3:0 时引发 VM-exit。
- **APIC-access VM-exit**，在 secondary processor-based VM-execution control 字段的“virtualize APIC-accesses”为 1 的前提下，取决于 Primary processor-based VM-execution control 与 secondary processor-based VM-execution control 字段相关位的设置，有下面的情况。
 - 当“use TPR shadow”为 0 时，尝试**线性读写** APIC-access page 内的值将引发 VM-exit（参见 3.5.2 节和 3.5.9 节）。
 - 当“use TPR shadow”为 1，并且“APIC-register virtualization”为 0 时，尝试**线性读** APIC-access page 内除 VTPR 以外其他的寄存器将引发 VM-exit。
 - 当“use TPR shadow”为 1，并且“APIC-register virtualization”也为 1 时，尝试**线性读** APIC-access page 内除 APIC ID，APIC Version，VTPR，VEOI，VLDR，VDFR，VSVR，VISR，VTMR，VIRR，VESR，VICR，LVT 表项，Timer initial count 以及 Timer divide configuration 寄存器以外的其他值将产生 VM-exit。
 - 当“use TPR shadow”为 1，并且“APIC-register virtualization”也为 1 时，尝试**线性写** APIC-access page 内除 APIC ID，VTPR，VEOI，VLDR，VDFR，VSVR，VESR，VICR，LVT 表项，Timer initial count 以及 Timer divide configuration 寄存器以外的其他值将产生 VM-exit。
 - 当“use TPR shadow”为 1，并且“APIC-register virtualization”与“virtual-interrupt delivery”都为 0 时，尝试**线性写** APIC-access page 内除 VTPR 以外的其他寄存器将产生 VM-exit。
 - 当“use TPR shadow”为 1，并且“APIC-register virtualization”为 0，“virtual-interrupt delivery”为 1 时，尝试**线性写** APIC-access page 内除 VTPR，VEOI 以及 VICR（低 32 位）寄存器以外的其他寄存器将产生 VM-exit。
 - 尝试以 guest-physical address 方式访问 APIC-access page 内的任何值将产生 VM-exit。
 - 尝试以 physical address 方式访问 APIC-access page 内的任何值**可能会**产生 VM-exit。
- **APIC-write VM-exit**，在 secondary processor-based VM-execution control 字段的“virtualize APIC-access”为 1 的前提下，取决于 Primary processor-based VM-execution control 与 secondary processor-based VM-execution control 字段相关位的设置，有下面的情况。
 - 当“use TPR shadow”为 1，并且“APIC-register virtualization”为 1 时，进行**线性写** APIC ID，VLDR，VDFR，VSVR，VESR，LVT 表项，Timer initial count 以及 Timer divide configuration 寄存器**最终结果**将产生 APIC-write VM-exit。
 - 当“use TPR shadow”为 1，并且“APIC-register virtualization”为 1，“virtual-

interrupt delivery”为 0 时，进行**线性写** VEOI 或者 VICR（低 32 位）寄存器**最终结果将产生 APIC-write VM-exit**。

- 当“use TPR shadow”为 1，并且“APIC-register virtualization”与“virtual-interrupt delivery”都为 1 时，写入 VICR（低 32 位）寄存器的值需要满足下面的所有条件。否则，最终结果将产生 APIC-write VM-exit。
 - 保留位（bits 31:20, bits 17:16, bit 13）以及 bit 12 必须为 0。
 - Bits 19:18 值必须为 1（也就是 **self**）。
 - Bit 15 必须为 0（edge 触发模式）。
 - Bits 10:8 值必须为 0（delivery 模式为 Fixed）。
 - Bits 7:4 值不能为 0（即 vector 必须大于 **10H**）。
- **EOI 虚拟化**，当“use TPR shadow”为 1，并且“virtualize APIC accesses”与“virtual-interrupt delivery”都为 1 时，进行线性写 VEOI 寄存器将执行 EOI 虚拟化。由于虚拟中断向量号在 EOI-exit bitmap 字段中对应的位为 1 而导致 VM-exit。

注意：APIC-access VM-exit 与 APIC-write VM-exit 使用同一个 VM-exit 原因码（44 号）。其中由于 **TPR below threshold**，**EOI 虚拟化**，**APIC-write VM-exit** 以及由“**monitor trap flag**”位而引起的 VM-exit 属于 trap 类型，也就是保存的 guest RIP 字段将指向下一条指令边界上。

5.4 由于 VM-entry 失败导致的 VM-exit

在进行 VM-entry 时由于遇到下面事件而失败导致 VM-exit：

- 在检查 guest-state 区域字段时，由于无效的 guest-state 字段 VM-entry 失败而导致 VM-exit。
- 在加载 guest-state 区域 MSR 时 VM-entry 失败而导致 VM-exit。
- 在 VM-entry 期间可能由于遇到 machine-check 事件而失败导致 VM-exit（参见 4.18 节）。

5.5 例子 5-1

示例 5-1：测试无条件与有条件产生 VM-exit 的 CPUID 与 RDTSC 指令

作为展示 VM-exit 的例子，我们只是简单地打开两个虚拟机运行（分别为 guest1 与 guest2），guest1 执行一条 CPUID 指令后，由 VMM 虚拟化 CPUID 指令。guest2 用来报告 RDTSC 指令引发的 VM-exit。

代码片段 5-1：

```

;-----
; guest_entry1():

```

```

; input:
;     none
; output:
;     none
; 描述:
;     1) 这是 guest1 的入口点
;-----
guest_entry1:

    DEBUG_RECORD    "[VM-entry]: switch to guest1 !"        ; 插入 debug 记录点
    DEBUG_RECORD    "[guest]: execute CPUID !"              ; 插入 debug 记录点

    ;;
    ;; guest 尝试执行 CPUID.01H
    ;;
    mov eax, 01h
cupid                                ; 此处将引发 VM-exit

    ;;
    ;; 输出 guest CPU 模型
    ;;
    mov esi, eax
    call get_display_family_model
    mov ebx, eax
    mov esi, GuestCpuMode
    call puts
    mov esi, ebx
    call print_word_value

    hlt
    jmp $ - 1
    ret

```

上面的代码片段位于 chap05\ex5-1\ex.asm 文件里，属于 guest1 的代码。为了方便观察执行过程，在 guest 代码里插入了两个 debug 记录点（使用 DEBUG_RECORD 宏），这两个 debug 记录点用来记录处理器的 context 及附加的信息。

Guest1 尝试查询 CPUID.01H (EAX=01H) 信息将会引发 VM-exit，由 VMM 接管 CPUID 指令的执行，下面是 VMM 对 CPUID 指令的虚拟化处理。

代码片段 5-2:

```

;-----
; DoCPUID()
; input:
;     none
; output:
;     none
; 描述:
;     1) 处理尝试执行 CPUID 指令引发的 VM-exit
;-----
DoCPUID:
    push ebp
    push ebx
    push ecx
    push edx

#ifdef __x64

```

```

LoadGsBaseToRbp
%else
    mov ebp, [gs: PCB.Base]
%endif

;;
;; 由 VMM 反射一个 CPUID 虚拟化结果给 guest
;;
mov eax, [ebp + PCB.Rax]           ; 读取 CPUID 功能号
cpuid                             ; 执行 CPUID 指令
mov eax, 633h                     ; 修改 guest CPU 的型号

;;
;; 将 CPUID 结果反射给 guest
;;
REX.Wrxb
mov [ebp + PCB.Rax], eax
REX.Wrxb
mov [ebp + PCB.Rbx], ebx
REX.Wrxb
mov [ebp + PCB.Rcx], ecx
REX.Wrxb
mov [ebp + PCB.Rdx], edx

;;
;; 调整 guest-RIP
;;
call update_guest_rip

mov eax, VMM_PROCESS_RESUME       ; 通知 VMM 进行 RESUME 操作

pop edx
pop ecx
pop ebx
pop ebp
ret

```

这个代码片段位于 lib\VMX\VmxBMM.asm 文件里，DoCPUID 函数将由 VmmEntry 函数调用（同一文件内），而 VmmEntry 函数是 VM-exit 发生后的 VMM 进入点。

DoCPUID 函数执行的虚拟化处理如下所示：

(1) 从 EAX 寄存器里读取 guest 给 CPUID 指令提供的功能号，EAX 寄存器的值在 VmmEntry 函数内使用 STORE_CONTEXT 宏保存在 PCB 的 context 区域内。

(2) VMM 执行 CPUID 指令读取 01H 功能信息。这里只修改了处理器的型号信息，其他信息保留不变。EAX 的值修改为 0633H。

(3) VMM 将 CPUID.01H 信息保存在 context 区域里，用来给 guest 传递信息。

(4) 调用 update_guest_rip 函数来更新 guest 的 RIP 值，让 RIP 指向 CPUID 的下一条指令，否则将会产生死循环（不断地发生 VM-exit 与 VM-entry）。

(5) 最后，通过 VmmEntry 函数进行 RESUME 操作。

代码片段 5-3:

```

;-----
; guest_entry2():
; input:
;     none
; output:
;     none
; 描述:
;     1) 这是 guest 2 的入口点
;-----
guest_entry2:

    DEBUG_RECORD    "[VM-entry]: switch to guest2 !"        ; 插入 debug 记录点
    DEBUG_RECORD    "[guest]: execute RDTSC !"              ; 插入 debug 记录点

    rdtsc

    hlt
    jmp $ - 1
    ret

```

上面代码片段位于 chap05\ex5-1\ex.asm 文件里, 属于 guest2 的代码。这个 guest 只是简单地执行一条 RDTSC 指令, 同样也插入了两个 debug 记录点, 用来追踪观察。

代码片段 5-4:

```

TargetCpuVmentry2:
    ... ..

    ;;
    ;; "rdtsc exiting" = 1
    ;;
    SET_PRIMARY_PROCBASED_CTLDS    RDTSC_EXITING

    mov esi, IA32_TIME_STAMP_COUNTER
    xor eax, eax
    xor edx, edx
    call append_vmentry_msr_load_entry

    ... ..

```

在 guest2 对应的 TargetCpuVmentry2 函数里, 使用 SET_PRIMARY_PROCBASED_CTLDS 宏来设置“RDTSC exiting”位为 1, 使 guest2 执行 RDTSC 指令能产生 VM-exit。

编译及运行

我们使用下面的命令进行编译及生成映像文件。

- 64 位代码, build -D__X64 -DDEBUG_RECORD_ENABLE
- 32 位代码, build -DDEBUG_RECORD_ENABLE

在 VMware 里加载映像文件 demo.img 运行, 在 CPU0 的命令选择器里分别按数字 <1>和<2>键, 让 CPU1 与 CPU2 执行 VM-entry 操作。接着按下<F4>键切换到 CPU3 屏幕, 再按下<Enter>键切换到信息列表查看。

如图 5-1 所示的运行图是在 CPU3 里运行 dump_debug_record 函数的结果, 打印出所

有 debug 记录点的信息，从这个列表里，我们可以看到下面的信息。

```

Home x demo x
[CpuIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable

===== msg list record =====
<#01> [CPU0] [command]: *** dispatch to CPU for VM-entry ***
<#02> [CPU1] [VM-entry]: switch to guest1 !
<#03> [CPU1] [guest]: execute CPUID !
<#04> [CPU1] [Exit Reason]: CPUID instruction ← guest 1 执行 CPUID 指令产生 VM-exit,
<#05> [CPU1] [DoCPUID]: virtualize CPUID!      最后由 VMM 返回虚拟化结果给 guest 1
<#06> [CPU1] [VMM]: resume to guest !
<#07> [CPU1] [Exit Reason]: HLT instruction
<#08> [CPU1] [DoHLT]: enter HLT state
<#09> [CPU1] [VMM]: resume to guest !
<#10> [CPU0] [command]: *** dispatch to CPU for VM-entry ***
<#11> [CPU2] [VM-entry]: switch to guest2 !
<#12> [CPU2] [guest]: execute RDTSC !
<#13> [CPU2] [Exit Reason]: RDTSC instruction ← guest 2 执行 RDTSC 指令产生 VM-exit,
<#14> [CPU2] [DoRDTSC]: processing RDTSC      最后由 VMM 打印出 VM-exit 明细信息
<#15> [CPU2] [VMM]: dump VMCS !
  
```

图 5-1

- (1) 由 CPU0 发送两次 IPI，分别让 CPU1 和 CPU2 执行 VM-entry 操作。
- (2) 在 CPU1 里的执行流程是：VM-entry → Guest 执行 CPUID 指令 → 产生 VM-exit → RESUME 重新进入 guest。
- (3) 在 CPU2 里的执行流程是：VM-entry → Guest 执行 RDTSC 指令 → 产生 VM-exit。

这些信息全部是由所插入的 debug 记录点所记录的。注意，在 CPU2 里（guest 2）停留在 VMM 里，并没有 RESUME 重新进入 guest。

我们也可以按<ENTER>键在信息列表与 debug 记录里来回切换，如图 5-2 所示是 debug 记录信息。

```

Home x demo x
[CpuIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable

===== debug record <#2> =====
[CPU1]: RIP: 00020308 Rflags: 00000202
RAX: 0000000000000000 RCX: 00000000C0000101
RDX: 0000000000000000 RBX: 0000000000000000
RSP: FFFFFFFF00000000 RBP: FFFFFFFF00000000
RSI: FFFFFFFF00000000 RDI: 0000000021ED0000
R8: 0000000000000000 R9: 0000000000000000
R10: 0000000000000000 R11: 0000000000000000
R12: 0000000000000000 R13: 0000000000000000
R14: 0000000000000000 R15: 0000000000000000

[ex.asm]:264
>>> [VM-entry]: switch to guest1 !
  
```

图 5-2

图 5-2 显示了 debug 记录的明显信息，包括处理器寄存器值、文件、行号以及附加信息，方便追踪观察 debug 记录当时的处理器状态。

接着，我们再按下<F2>键切换到 CPU1 查看运行结果，如图 5-3 所示。



图 5-3

上图是 CPU1 运行的 CPUID 指令虚拟化后的结果图，显示 guest 的处理器型号为 0603H（由 VMM 设置的 0633H 值而来），而处理器真实的型号为 0625H。

通过对 CPUID 指令的虚拟化，可以控制 guest 处理器拥有什么样的功能和特性。例如，在返回的 CPUID.01H 信息中将 ECX 寄存器的 bit 25（对应为 AES 指令支持位）清 0，表示 guest 处理器不支持 AES 指令。将 ECX 寄存器的 bit 5 清 0，表示 guest 处理器不支持 VMX 架构。

下面，我们再按<F3>键切换到 CPU2 查看运行结果，如图 5-4 所示。

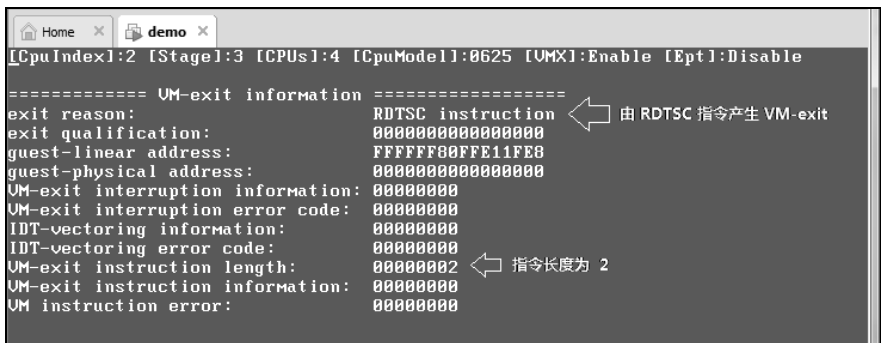


图 5-4

上图是由 CPU2 运行的 RDTSC 指令产生的 VM-exit 结果，由 VMM 执行 dump_vmcs 函数输出，可以使用<PgUp>与<PgDn>键进行上下翻页，可以很容易地观察到产生 VM-exit 后 VMCS 区域内记录的所有数据。VMM 并没有执行 VMRESUME 指令恢复 guest2 执行，只是在 VMM 里观察 VMCS 数据。

5.6 指令引发的异常与 VM-exit

在 VMX non-root operation 模式下，第 5.1 节与第 5.2 节所提到的指令都能直接产生 VM-exit。当尝试这些指令时可能会引发某些异常（譬如#UD 异常），某些异常的优先级高于 VM-exit，此时处理器会 deliver 异常而不是产生 VM-exit。因为这些异常在指令的 fetch 阶段引发。

5.6.1 优先级高于 VM-exit 的异常

当指令引发下面的异常，优先级将高于 VM-exit 事件。

(1) **#UD 异常**，引发#UD 异常可能是由于下列情况：

- 在指令前加上了**错误的指令前缀**（譬如 LOCK 前缀）。例如，执行“**lock cpuid**”指令将引发#UD 异常。
- 指令使用了不正确的操作数。例如，执行“**mov cr1, rax**”指令将引发#UD 异常。
- 在 compatibility, virtual-8086 或者实模式执行不支持的指令。例如，在 compatibility 模式下执行任何一条 VMX 指令将引发#UD 异常（除了 VMFUNC 指令）。
- 执行处理器**不支持或者没有开启**的指令。例如，在 CR4.OSXSAVE = 0 时，执行 XSETBV 指令将引发#UD 异常。

注意：不包括 UD2 指令产生的#UD 异常。

(2) **#GP 异常**，由于下面情况引发的#GP 异常：

- 基于**权限不足**而引起的#GP 异常（譬如在 virtual-8086 模式下执行任何特权级指令）。例如，在 CPL = 3 时，执行“**mov rax, cr3**”指令将引发#GP 异常。例如，在 virtual-8086 模式下执行 LGDT 指令将产生#GP 异常。
- 基于 TSS 内的 **I/O permission bitmap** (I/O 许可位图) 而引发的#GP 异常。又如，在 CPL > rflags.IOPL 时，执行“**in al, 60h**”指令但该 I/O 端口在 I/O permission bitmap 对应的位为 1 而引发#GP 异常。

(3) 在**提取指令操作数时产生的异常高于由于访问操作数而产生的 VM-exit**（除 INS/OUTS 指令外），典型地指令的操作数是内存操作数。例如，下面执行 MOV 指令或者 LMSW 指令可能产生 VM-exit 以及相关异常。

<code>mov eax, [ebx + LAPIC_ID]</code>	；可能产生 APIC-access VM-exit
<code>lmsw ss:[eax]</code>	；可能由于 CR0 guest/mask 及 CR0 shadow 产生 VM-exit

在提取指令操作数时可以产生下面的异常：

- **#GP 异常**，例如：可能由于 DS 内含一个 NULL selector，或者地址 offset 值超出 DS.limit 值而产生。
- **#PF 异常**，由于无权限访问线性地址，或者线性地址为 not-present 等情况而产生。
- **#SS 异常**，当使用一个 SS 段前缀时（例如“**lmsw ss:[eax]**”），由于地址 offset 超出 SS.limit 值产生，或者由于 SS 段为 not-present 而产生。
- **#NP 异常**，由于 DS, ES, FS 或者 GS 段属于 not-present 而产生。
- **#AC 异常**，由于开启了 AC 机制（CR0.AM = 1，且 eflags.AC = 1），在 CPL=3 时内存操作数地址不在对齐边界上而产生。

(4) 指令引发的异常高于 **trap 类型的 VM-exit**（包括 TPR below threshold, EOI 虚拟化, APIC-write VM-exit 以及由“monitor trap flag”引起的 VM-exit）。

上面的这些异常事件发生后，处理器将响应异常的 delivery，而不是产生 VM-exit。但是，如果这些异常的向量号在 exception bitmap 字段对应位为 1，将由这些异常事件引发 VM-exit。

5.6.2 VM-exit 优先级高于指令的异常

由执行 INS/OUTS 指令而产生的 VM-exit（由于“unconditional I/O exiting”或者 I/O bitmap），这个 VM-exit 的优先级高于下面的异常：

- **#GP 异常**，执行 INS/OUTS 指令可能会由于 ES 或 DS 含有一个 NULL selector 而引发 #GP 异常，或者由于地址 offset 值超出段 limit，或者执行 INS 指令时 ES 不是可写的段。但不包括由于权限而产生的 #GP 异常。
- **#PF 异常**，由于无权限访问线性地址，或者线性地址为 not-present 等情况而产生。
- **#AC 异常**，由于开启了 AC 机制（CR0.AM = 1，且 eflags.AC = 1），在 CPL=3 时内存操作数地址不在对齐边界上而产生。

除了前面所说的异常事件优先级别高于 VM-exit 外，其余的 fault 类型的 VM-exit 优先级都高于指令产生的异常。

5.6.3 例子 5-2

示例 5-2：测试优先级高于 VM-exit 的异常

这里，我们选取两类异常（#UD 与 #GP 异常）作为测试对象，用来验证与 VM-exit 之间的优先级别关系。这两个异常的产生如下所示。

(1) #UD 异常由于使用了错误的指令前缀而引起。例如，尝试执行“lock cupid”指令。

(2) #GP 异常由于 CPL=3 时执行了“mov rax, cr3”指令而产生。

在 guest1 的 TargetCpuVmentry1 函数里，将 exception bitmap 字段的 bit 6 清 0（使用 CLEAR_EXCEPTION_BITMAP 宏实现），允许发生 #UD 异常后在 guest 里执行 #UD 异常处理例程。如下面代码所示。

```
CLEAR_EXCEPTION_BITMAP UD_VECTOR ; 清 bit 6 位，允许执行 #UD 例程
```

在 guest2 的 TargetCpuVmentry2 函数里，将 primary processor-based VM-execution control 字段的“cr3-store exiting”置为 1（使用 SET_PRIMARY_PROCBASED_CTLDS 宏实现），目的是让 guest 尝试执行“mov rax, cr3”指令时产生 VM-exit。

```
SET_PRIMARY_PROCBASED_CTLDS CR3_STORE_EXITING ; 执行 MOV from CR3 指令将产生 VM-exit
SET_EXCEPTION_BITMAP GP_VECOTR ; 置 bit 13 位，允许 #GP 异常产生 VM-exit
```

注意：guest2 的 CPL 被设置为 3 级。因此，当执行“mov rax, cr3”指令时将引发 #GP 异常。下面是 guest2 的代码，在 chap05\ex5-2\ex.asm 文件里。

代码片段 5-5:

```

;-----
; guest_entry2():
; input:
;     none
; output:
;     none
; 描述:
;     1) 这是 guest 的入口点
;-----
guest_entry2:

    DEBUG_RECORD_USER_S    "[VM-entry]: switch to guest1 !"    ; 插入 debug 记录点
    DEBUG_RECORD_USER_S    "[guest]: MOV RAX, CR3 !"

    mov R0, cr3
    ret

```

注意：在上面代码里使用了 `DEBUG_RECORD_USER_S` 宏来插入两个 debug 记录点。这是由于 `guest2` 执行在 `user` 权限下（3 级），`DEBUG_RECORD` 宏不能完成插入 debug 记录点的工作。`DEBUG_RECORD_USER_S` 是 `DEBUG_RECORD` 宏的扩展版本，用来在 `user` 权限插入 debug 记录点。

编译及运行

我们使用下面的命令进行编译及生成映像文件。

- 64 位代码，`build -D__X64 -DDEBUG_RECORD_ENABLE`
- 32 位代码，`build -DDEBUG_RECORD_ENABLE`

在 VMware 中加载映像文件 `demo.img` 运行，在 CPU0 的命令选择器里分别按下数字 <1>和<2>键，让 CPU1 与 CPU2 执行 VM-entry 操作。按下<F4>键切换到 CPU3 屏幕，接着按下<Enter>键切换到信息列表查看。

在图 5-5 中显示，CPU1 由于执行“lock CPUID”指令产生#UD 异常，`guest1` 进入#UD handler 执行。而 CPU2 执行 MOV-from-CR3 指令由于权限不足产生#GP 异常，导致 VM-exit。VMM 处理时注入#GP 异常，`guest2` 进入#GP handler 执行。

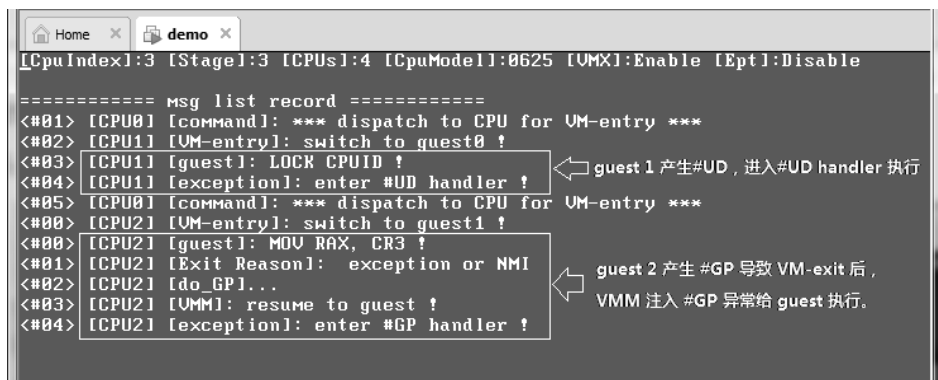
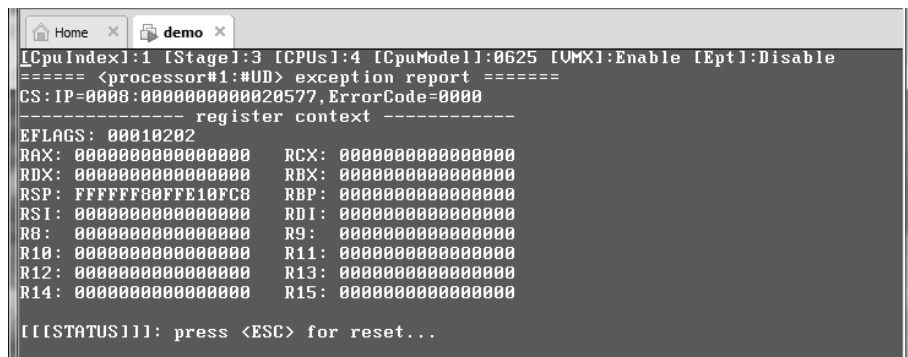


图 5-5

下面，我们按下<F2>键切换到 CPU1 屏幕，查看运行结果。

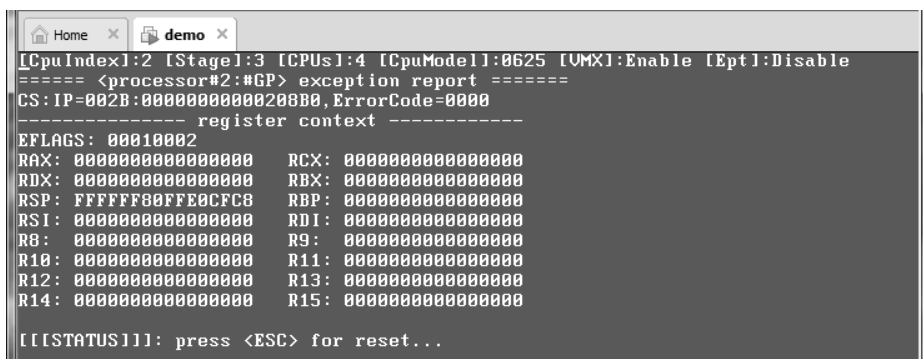
图 5-6 是 CPU1 运行的结果，显示处理器正在执行#UD 异常处理例程，这个例程将打印出发生#UD 时的上下文环境信息。接下来按下<F3>键看看 CPU2 的运行结果。



```
[CpuIndex]:1 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable
===== <processor#1:#UD> exception report =====
CS: IP=0008:000000000020577, ErrorCode=0000
----- register context -----
EFLAGS: 00010202
RAX: 0000000000000000 RCX: 0000000000000000
RDX: 0000000000000000 RBX: 0000000000000000
RSP: FFFFFFFF80FFE10FC8 RBP: 0000000000000000
RSI: 0000000000000000 RDI: 0000000000000000
R8: 0000000000000000 R9: 0000000000000000
R10: 0000000000000000 R11: 0000000000000000
R12: 0000000000000000 R13: 0000000000000000
R14: 0000000000000000 R15: 0000000000000000
[[STATUS]]: press <ESC> for reset...
```

图 5-6

如图 5-7 显示，CPU2 正在运行#GP 异常处理例程，CPU2 打印出了发生#GP 异常的上下文环境信息。



```
[CpuIndex]:2 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable
===== <processor#2:#GP> exception report =====
CS: IP=002B:0000000000208B0, ErrorCode=0000
----- register context -----
EFLAGS: 00010002
RAX: 0000000000000000 RCX: 0000000000000000
RDX: 0000000000000000 RBX: 0000000000000000
RSP: FFFFFFFF80FE0CFC8 RBP: 0000000000000000
RSI: 0000000000000000 RDI: 0000000000000000
R8: 0000000000000000 R9: 0000000000000000
R10: 0000000000000000 R11: 0000000000000000
R12: 0000000000000000 R13: 0000000000000000
R14: 0000000000000000 R15: 0000000000000000
[[STATUS]]: press <ESC> for reset...
```

图 5-7

通过这个简单的例子，我们看到：guest1 执行“lock cupid”指令产生了#UD 异常，处理器将执行#UD 异常处理例程，而不会产生 VM-exit（除非由于 exception bittmap 字段的 bit 6 为 1 而产生 VM-exit）。Guest2 在 3 级权限里执行“mov rax, cr3”指令产生了#GP 异常，由于 exception bitmap 字段的 bit 13 为 1 而产生了 VM-exit，此时并不是由“cr3-store exiting”为 1 而直接产生 VM-exit。

5.7 VM-exit 的处理流程

在 VMX non-root operation 内发生 VM-exit 后，处理器将执行下面几个步骤的工作。

(1) 根据引发 VM-exit 的不同原因，在 VM-exit information 区域内相应的字段里记录 VM-exit 的原因，以及相关信息（例如 exit reason 字段，exit qualification 字段等）。

- (2) 更新 VM-entry control 字段与 VM-entry interruption information 字段值。
- (3) 更新处理器当前的状态信息。
- (4) 将处理器当前的状态信息保存在 guest-state 区域相应的字段里（某些信息需要依据 VM-exit control 字段来决定是否保存）。
- (5) 根据 VM-exit MSR-store 列表，保存相应的 MSR 值（或者没有 MSR 需要保存）。
- (6) 从 host-state 区域相应字段里加载 host 的处理器状态信息（某些信息需要依据 VM-exit control 字段来决定是否加载），附加工作是清除所有地址监控。
- (7) 根据 VM-exit MSR-load 列表，加载相应的 MSR 值（或者没有 MSR 需要加载）。

如同 VM-entry 一样，在 VM-exit 时（*由 non-root 切换到 root 那一刻*）的分支记录处理器不会进行记录，包括 LBR（last-branch record）记录以及 BTS（branch trace store）记录，也不会产生 BTM（branch trace message）消息。关于分支记录详情请参考《x86/x64 体系探索及编程》第 14 章。

同样，在完成 VM-exit 后（*已经切换到 root*），所有分支记录都是可以记录的。

5.8 记录 VM-exit 的相关信息

在 VM-exit 信息区域里包含下面的字段（参见 3.10 节）。

- 基本信息类
 - Exit reason 字段，包含一个 VM-exit 原因码，指示产生 VM-exit 的原因。
 - Exit qualification 字段，记录某些 VM-exit 事件的明细信息。
 - Guest-linear address 字段，记录某些引发 VM-exit 事件中所引用的线性地址值。
 - Guest-physical address 字段，记录由 EPT violation 或者 EPT misconfiguration 产生 VM-exit 时所引用的 GPA（guest-physical address）值。
- 直接向量事件信息类
 - VM-exit interruption information 字段，记录直接引发 VM-exit 向量事件相关的信息，包括中断类型与向量号等。
 - VM-exit interruption error code 字段，记录由于某些硬件异常而引发 VM-exit 时的错误码。
- 间接向量事件信息类
 - IDT-vectoring information 字段，记录间接引发 VM-exit 向量事件的相关信息，包括中断类型与向量号等。
 - IDT-vectoring error code 字段，记录由于某些硬件异常而间接引发 VM-exit 时的错误码。
- 指令信息类
 - VM-exit instruction length 字段，记录由于尝试执行 5.1 节与 5.2 节所列举的指令而产生 VM-exit 的指令长度。

- VM-exit instruction information 字段，记录由于某些指令而产生 VM-exit 的指令信息。
- I/O SMI 信息类
 - I/O RCX 字段，记录执行 I/O 指令前的 RCX 寄存器值。
 - I/O RSI 字段，记录执行 I/O 指令前的 RSI 寄存器值。
 - I/O RDI 字段，记录执行 I/O 指令前的 RDI 寄存器值。
 - I/O RIP 字段，记录执行 I/O 指令前的 RIP 值。
- 指令错误类
 - VM-instruction error 字段，记录在 VMX root operation 内执行 VMX 指令产生了 VMfailValid 类型错误时的错误码（参见 2.6.2 节与 2.6.3 节）。

处理器根据不同的 VM-exit 原因，在上面这 6 大类信息类字段里记录 VM-exit 相关信息，详情请参考 3.10.1 节~3.10.6 节所述。

5.9 更新 VM-entry 区域字段

在记录引发 VM-exit 的相关信息时，处理器也会更新 VM-entry 区域内的某些字段：

- 清 VM-entry interruption information 字段的 bit 31 位 (**valid**) 为 0，强制设置注入事件为无效。
- 如果支持 VM-exit 时保存 IA32_EFER.LMA 值（即 IA32_VMX_MISC[5] 为 1 时），则将 IA32_EFER.LMA 的值写入 VM-entry control 字段的“IA-32e mode guest”位。

注意：如果 VM-exit 是由于 VM-entry 失败而引发的（参见 5.4 节），两个字段不会被更新。

5.10 更新处理器状态信息

处理器的状态（譬如 activity 与 interruptibility 状态等）与环境信息（譬如 CR2，DR7 寄存器等）在运行过程中，随着某些指令的执行或者遇到某些事件而改变。

在发生 VM-exit 时也会更新处理器的状态信息，然后保存在 guest-state 区域相应的字段里。某些处理器的状态信息更新分为“VM-exit 开始前更新”与“VM-exit 完成后更新”两种情形，这取决于某些 VM-exit 事件属于直接退出还是间接退出。

注意：如果 VM-exit 是由于 VM-entry 失败而引发的（参见 5.4 节），处理器的状态不会被更新。例如，“blocking by NMI”阻塞状态是 VM-entry 前的状态。

VM-exit 开始前更新

这类状态信息在 VM-exit 操作开始前已经被更新。应当理解为这些状态信息“在 non-root 内已经被更新”。因此，某些 guest-state 区域的字段保存的是处理器 VM-exit 当

前最新的状态信息。

例如，guest 收到一个 NMI，这个 NMI 并不直接产生 VM-exit。但是，在 NMI 的 delivery 期间间接引发了 VM-exit。那么，处理器在 VM-exit 时就已经存在“blocking by NMI”阻塞状态。

VM-exit 完成后更新

这类状态信息在 VM-exit 操作完成后才被更新。应当理解为“从 non-root 切换到 root 后才被更新，而在 VM-exit 时处理器状态维持不变”。因此，在 VM-exit 时某些 guest-state 区域字段保存的状态信息未得到更新。

例如，guest 收到一个 NMI，这个 NMI 由于 pin-based VM-execution control 字段的“NMI exiting”为 1 而直接产生 VM-exit。那么只有在 VM-exit 完成后（也就是从 non-root 返回 root 后），处理器才存在“blocking by NMI”阻塞状态。

另外，VM-exit 后，处理器总是处于 active 状态，并且不存在“blocking by STI”与“blocking by MOV-SS”阻塞状态（参见 5.14 节），如下面的代码所示。

```
hlt ; 执行 HLT 指令产生 VM-exit
```

执行一条 HLT 指令直接产生 VM-exit。但是，VM-exit 后处理器不会处于 HLT 状态。

```
mov ss, [kernelss] ; 执行 MOV-SS 指令，由于 APIC-access 而产生 VM-exit
```

执行一条 MOV-SS 指令有可能由于 APIC-access 引发 VM-exit，VM-exit 后处理器不会存在“blocking by MOV-SS”阻塞状态。

由某些事件引发 VM-exit 时，处理器状态信息的更新取决于事件是直接引发 VM-exit，还是间接引发 VM-exit。

直接 VM-exit 事件

下面的事件将直接引发 VM-exit（假设没有处理器的 inactive 状态阻塞，参见 4.17.2 节）。

- 收到 INIT，SMI 或者 SIPI。
- 收到 NMI，并且处理器不存在“blocking by NMI”及“blocking by MOV-SS”阻塞状态。
- 发生异常，由于向量号在 exception bitmap 字段对应的位为 1 而直接引发 VM-exit。

间接 VM-exit 事件

在一个向量事件（包括异常、中断、NMI、注入事件及 pending debug exception）的 delivery 期间，由于遇到了另一个异常（即嵌套异常）、triple fault、任务切换、APIC-access、EPT violation 或者 EPT misconfiguration 而间接引发 VM-exit（参见 3.10.3 节与 4.12.2 节）。

这种情况下，显著的特征是：IDT-vectoring information 字段会记录原始的向量事件（即间接引发 VM-exit 的事件，参见 3.10.3 节）。

trap 类型 VM-exit

除了上面的直接与间接 VM-exit 事件外，还与指令引发的 VM-exit 是否属于 trap 类型相关。由 **TPR below threshold**，**EOI 虚拟化**，**APIC-write VM-exit** 或者 “**monitor trap flag**” 引发的 VM-exit 属于 trap 类型。前三者是 local APIC 虚拟化产生的结果，后者是 VMX 支持的调试功能。

由 local APIC 虚拟化产生 trap 类型 VM-exit 也可以通过下面两个途径：

(1) 当 primary processor-based VM-execution control 字段的 “CR8-load exiting” 为 0，并且 “use TPR shadow” 为 1 时，在 64 位模式下执行 MOV to CR8 指令更新 **VPTR** 寄存器。

(2) 当 secondary processor-based VM-execution control 字段的 “virtualize x2APIC mode” 为 1 时，执行 WRMSR 指令，ECX 寄存器提供的 MSR 地址值在 **800H 到 8FFH** 之间，并且 ECX 在 MSR bitmap 中对应的位为 0 时（或者 “use MSR bitmap” 位为 0）。

关于 local APIC 虚拟化产生的 trap 类型 VM-exit，可参见 7.2.12 节所述。

5.10.1 直接 VM-exit 事件下的状态更新

由事件直接引发 VM-exit 时，下面的处理器状态信息不会被更新：

- 由 #DB 异常直接引发 VM-exit 时，DR6，DR7 及 IA32_DEBUGCTL 寄存器不会被更新。
 - 关于 #DB 异常的明细信息会记录在 exit qualification 字段里（参见 3.10.1.5 节）。
- 由 #PF 异常直接引 VM-exit 时，CR2 寄存器不会被更新。
 - 引发 #PF 异常的线性地址会记录在 exit qualification 字段里（参见 3.10.1.6 节）。
- 由 NMI 直接引发 VM-exit 时，处理器 VM-exit 当前不存在 “blocking by NMI” 阻塞状态。只有在 VM-exit 操作完成后（**root 环境**），才存在 “blocking by NMI” 阻塞状态。
 - Guest-state 区域保存的 interruptibility state 字段的 “blocking by NMI” 位为 0 值。
- 由外部中断直接引发 VM-exit 时，取决于 VM-exit control 字段的设置下面的情形。
 - 当 “acknowledge interrupt on exit” 位为 1 时，处理器响应中断控制器，读取中断信息，**清除外部中断的请求位，外部中断不会被 pending（悬挂）**。也就是 VM-exit interruption information 字段将保存外部中断信息（参见 3.7.1 节与 3.10.2.1 节）。
 - 当 “acknowledge interrupt on exit” 位为 0 时，**外部中断请求位保持有效，外部中断被 pending**。也就是 VM-exit interruption information 字段不会记录外部中断信息（参见 3.7.1 节与 3.10.2.1 节）。
- 由于尝试任务切换而引发 VM-exit 时，DR7 寄存器的 L0 到 L3 位不会被清 0。这个任务切换由下面的途径产生：

- 由 CALL, JMP, IRET, INT (包括 INTO, INT3, BOUND, UD2) 指令引起的任务切换。
- 由向量事件的 delivery (包括注入事件与 pending debug exception) 期间引起的任务切换。
- 由于尝试任务切换而直接引发 VM-exit 时, 下面的信息不会被修改:
 - 原任务的 TSS 块内容, 以及新任务的 TSS 块内容。
 - 原 TSS 描述符, 以及新 TSS 描述符。
 - RFLAGS.NT 位, 以及 TR 寄存器。
- LER (last exception record) 记录不会被更新。
- LBR (last branch record) 与 BTS (branch trace store) 记录不会被更新。
- 由 #MC 异常直接引发 VM-exit 时, 不会阻止与机器检查相关的 MSR 更新, 但这些更新限于机器检查自己而不是由 #MC 异常而引起的。
- 当处理器处于 **inactive** 状态时 (即 HLT, shutdown 或者 wait-for-SIPI 状态), 如果一个事件唤醒 inactive 状态直接产生 VM-exit, 处理器只在 VM-exit 完成后 (**non-root 切回 root**) 才被唤醒转入 active 状态 (参见 4.17.3 节)。
 - 在条件满足时, INIT, SIPI, SMI, NMI 及外部中断能唤醒 inactive 状态 (参见 4.17.3 节)。
 - 在条件满足时, 在 4.15.2 节到 4.15.7 节中所描述的 6 种事件能唤醒 inactive 状态。

从另一个角度看, 一个向量事件直接引发 VM-exit, 因此它并没有得到 **delivery**。部分与这个向量事件相关的状态也只有在 VM-exit 完成后才得到更新。

5.10.2 间接 VM-exit 事件下的状态更新

当事件间接引发 VM-exit 时 (参见 3.10.3 节), 下面的处理器状态将被更新:

- 当 #DB 异常间接引发 VM-exit 时, DR6, DR7 以及 IA32_DEBUGCTL 寄存器被更新。即更新 DR6 记录的状态, 而 DR7.GD, IA32_DEBUGCTL.LBR 与 IA32_DEBUGCTL.BTF 位都被清 0。
- 由 #PF 异常间接引发 VM-exit 时, CR2 寄存器被更新为引发异常的线性地址值。
- 由于在 NMI 的 delivery 期间引发 VM-exit 时, 处理器在 VM-exit 开始前更新 interruptibility 状态。也就是 VM-exit 开始前就存在 “blocking by NMI” 阻塞状态。
- 当注入一个 virtual-NMI 事件 (即 pin-based VM-execution control 字段的 “NMI exiting” 与 “virtual-NMIs” 为 1, 并且注入一个 NMI 事件), 在它的 delivery 期间引发 VM-exit 时, 处理器在 VM-exit 开始前更新 interruptibility 状态。也就是 VM-exit 开始前就存在 “blocking by virtual-NMI” 阻塞状态。
- 由外部中断在 delivery 期间引发 VM-exit 时, 处理器已经响应中断控制器, 外部中断不再 pending。
- 当处理器处于 inactive 状态时 (即 HLT, shutdown 或者 wait-for-SIPI 状态), 如果一个事件唤醒 inactive 状态间接引发 VM-exit, 处理器在 VM-exit 前已经转为

active 状态（参见 4.17.3 节）。即在 non-root 里已经转为 active 状态。

- 由事件间接引发 VM-exit，处理器在 VM-exit 开始前已经不会存在“blocking by STI”及“blocking by MOV-SS”阻塞状态。
- 在事件 delivery 期间由于遇到某些错误而间接引发 VM-exit 时，某些状态信息可能会被更新。例如：
 - 段描述符可能会被更新为 accessed（已访问）或者 dirty（已写）。
 - 在向栈中压入返回信息时，可能会由于 RSP 而发生异常（如产生 #PF 异常），但此时可能已经压入了部分数据。
- LER（last exception record）记录的更新与处理器架构实现相关：
 - 某些处理器会得到更新，因为事件已经被 deliver。即使在 delivery 期间遇到了错误而导致 VM-exit 发生，也不影响 LER 记录的更新。
 - 某些处理器不会更新。在这些处理器架构上，LER 记录只有在 delivery 完成后进入事件的 handler 才被更新。delivery 期间发生错误导致 VM-exit 时 LER 记录并没有生成。
- LBR（last branch record）与 BTS（branch trace store）记录只有在 VM-exit 完成后才会产生。因此，在事件的 delivery 期间引发 VM-exit，处理器不会生成 LBR 与 BTS 记录。

从另一个角度看，产生间接 VM-exit 的事件，由于它已经被 deliver（但并没有完成或者成功 deliver），因此，大部分处理器状态信息会在 VM-exit 开始前得到更新。

5.10.3 其他情况下的状态更新

除了上面由事件直接或间接引发 VM-exit 外，还有下面的更新情形：

- 在 pin-based VM-execution control 字段的“NMI exiting”为 0 的前提下，当执行 IRET 指令发生了某些错误（包括异常，任务切换，访问 APIC-access page，EPT violation 或者 EPT misconfiguration）而导致 VM-exit 时，处理器的“blocking by NMI”阻塞状态已经被解除。另外存在下面的附加信息更新：
 - IRET 指令发生异常直接导致 VM-exit 时，在 VM-exit interruption information 字段的 bit 12 会记录这个“blocking by NMI”阻塞状态的解除（参见 3.10.2.1 节）。
 - IRET 指令发生 EPT violation 导致 VM-exit 时，在 exit qualification 字段的 bit 12 会记录这个“blocking by NMI”阻塞状态的解除（参见 3.10.1.14 节）。
 - 当 IDT-vectoring information 字段的 bit 31 为 1 时，VM-exit interruption information 字段的 bit 12 是未定义值（也就是由 IRET 指令产生的异常间接引发 VM-exit）。
- 当 pin-based VM-execution control 字段的“NMI exiting”与“virtual-NMIs”为 1 时（即存在 virtual-NMI 概念），当执行 IRET 指令发生了某些错误（包括异常，任务切换，APIC-access page，EPT violation 或者 EPT misconfiguration）而导致 VM-exit 时，处理器的“blocking by virtual-NMI”阻塞状态已经被解除。另外存在

下面的附加信息更新：

- IRET 指令发生异常直接导致 VM-exit 时，在 VM-exit interruption information 字段的 bit 12 会记录这个“blocking by virtual-NMI”阻塞状态的解除（参见 3.10.2.1 节）。
- IRET 指令发生 EPT violation 导致 VM-exit 时，在 exit qualification 字段的 bit 12 会记录这个“blocking by virtual-NMI”阻塞状态的解除（参见 3.10.1.14 节）。
- 当 IDT-vectoring information 字段的 bit 31 为 1 时，VM-exit interruption information 字段的 bit 12 是未定义值（也就是由 IRET 指令产生的异常间接引发 VM-exit）。
- 与 #MF (x87 FPU 异常) 相关的状态更新。
 - 由 #MF 异常直接引发 VM-exit 时，VM-exit 开始前处理器已经不存在“blocking by STI”与“blocking by MOV-SS”阻塞状态。
 - 由于 #MF 异常而不存在任何阻塞状态时，收到 INIT, SMI, NMI 或者外部中断直接产生 VM-exit 时（因为这些事件优先级高于 #MF 异常），在 VM-exit 开始前处理器已经不存在“blocking by STI”与“blocking by MOV-SS”阻塞状态。
- 由 #MC 引发 VM-exit 时，处理器当前的状态可能不正确，保存在 guest-state 区域的字段也可能不正确。VMM 需要在注入 #MC 异常给 guest 前，根据 IA32_MCG_STATUS 寄存器的 RIPV 与 EIPV 位来进行相应的设置。
- 当执行一条指令可能会由于异常，APIC access，EPT violation 或者 EPT misconfiguration 而导致 VM-exit 发生，或者执行一条指令直接引发 VM-exit 时，有下面的状态更新。
 - VM-exit 之前已经存在 #DB 异常的悬挂时，悬挂的 #DB 异常信息将被记录在 pending debug exception 字段中。
 - VM-exit 之前已经存在“blocking by MOV-SS”或者“blocking by STI”阻塞状态时，interruptibility state 字段保存这些状态信息。

例如下面的代码：

```
... .. ; TF = 1 (启用指令单步调试)
mov ss, ax ; 更新 SS
mov esp, [Stack] ; 由于 EPT violation 而引发 VM-exit, #DB 异常已经被悬挂
```

由 APIC access，EPT violation 或者 EPT misconfiguration 引发的 VM-exit 属于 fault 类型，也就是 guest RIP 字段指向指令本身。这种情况下相当于由指令直接产生了 VM-exit。

- 执行一条指令产生 trap 类型的 VM-exit 时（包括 **TPR below threshold**，**EOI 虚拟化**及 **APIC-write VM-exit**），产生这类 trap VM-exit 也包括使用 MOV to CR8 及 WRMSR 指令，以及由于“**monitor trap flag**”位而引起的 MTF VM-exit。有下面的状态更新。
 - 如果这条指令访问了断点，或者指令单步 #DB 异常 pending 在这条指令后面，在 VM-exit 之前就存在了 #DB 异常的悬挂。Pending debug exception 字段记录 #DB 异常悬挂信息。

- 已经不存在“blocking by MOV-SS”及“blocking by STI”阻塞状态。
- 如果处理器处于 inactive 时，在 VM-exit 操作之前就已经转为 active 状态。

注意：在 VM-exit 后（root 环境里）不会存在某些阻塞状态（包括“blocking by STI”与“blocking by MOV-SS”阻塞状态），并且处理器总是处于 **active** 状态（参见 5.14 节）。

5.11 保存 guest 环境信息

在处理器的状态更新后，guest 的环境信息（处理器当前的状态信息）将保存在 guest-state 区域相应的字段中。

注意：在支持 64 位架构处理器上，由于 natural-width 类型字段属于 64 位宽（参见 3.8 节），在保存这些字段时固定写入 64 位值，而忽略处理器当前的工作模式（不管处理器处于 IA-32e 模式还是 32 位的保护模式）。

5.11.1 保存控制寄存器，debug 寄存器及 MSR

- 当前的 CR0, CR3 及 CR4 寄存器值直接保存在对应的字段里。在 64 位架构处理器上保存 64 位，在 32 位架构处理器上保存 32 位。
- IA32_SYSENTER_CS, IA32_SYSENTER_ESP 及 IA32_SYSENTER_EIP 寄存器的保存如下。
 - IA32_SYSENTER_CS 寄存器只保存 bits 31:0，高 32 位忽略（即使在 64 位架构处理器上）。
 - 在 32 位架构处理器上，只保存 IA32_SYSENTER_ESP 与 IA32_SYSENTER_EIP 寄存器的 bits 31:0，高 32 位忽略。
 - 在 64 位架构处理器上，保存 64 位的 IA32_SYSENTER_ESP 与 IA32_SYSENTER_EIP 寄存器。
- 取决于 VM-exit control 字段相关位的设置，下面的寄存器保存在对应字段里。
 - 当“save debug controls”为 1 时，保存 DR7 与 IA32_DEBUGCTL 寄存器。
 - 当“save IA32_PAT”为 1 时，保存 IA32_PAT 寄存器。
 - 当“save IA32_EFER”为 1 时，保存 IA32_EFER 寄存器。
 当这些位为 0 时，相应的寄存器不会被保存。

5.11.2 保存 RIP 与 RSP

VM-exit 时，RSP 寄存器值直接保存在 RSP 字段里，而 RIP 保存的情况取决于 VM-exit 的类型。

- 由于尝试执行指令（参见 5.1 节及 5.2 节所描述的指令）而直接产生 VM-exit 时（属于 fault 类型），保存的 RIP 值指向这条指令。

- 由于 INIT, SMI 或者 SIPI 引发 VM-exit 时, 保存的 RIP 值是发生这些事件前的 RIP 值。
- 由于 “NMI-window exiting” 或者 “interrupt-window exiting” 而引发 VM-exit 时, 保存的 RIP 值是 VM-exit 之前的值。

```
sti                ; 打开中断窗口
mov eax, 1         ; 存在 “blocking by STI” 阻塞状态
                  ; <--- 在此产生 VM-exit ! (下一条指令执行前)
mov eax, 2         ; RIP 指向这条指令
```

在上面的代码示例中, 执行 STI 指令打开中断窗口后, “mov eax, 1” 指令位置上存在 “blocking by STI” 阻塞状态。此时 RIP 指向下一条指令边界上 (即 “mov eax, 2” 指令), 在 “mov eax, 1” 指令执行完毕后产生 VM-exit。那么保存的 RIP 值将指向 “mov eax, 2” 指令上。

- 由于外部中断, NMI 或者**硬件异常**引发 VM-exit 时, 保存的 RIP 值是**将要**被压入栈中的返回地址值。

如果在 VM-exit 前已经压入了部分数据 (参见 5.10.2 节), 那么保存的 RIP 值可能是已经压入栈中的返回地址值 (或者在尝试任务切换时, 将要保存在旧 TSS 块内的 EIP 值)。

- 由于 INT3 或 INTO 指令产生的**软件异常**引发 VM-exit 时, 保存的 RIP 值指向指令本身。

注意: 由于它们属于 trap 类型的异常, 压入栈中的返回地址指向下一条指令。所以, 这里规定它们产生的 VM-exit 所保存的 RIP 值需要指向指令本身 (**而不是压入栈中的返回地址**)。

- 由于 triple fault 引发 VM-exit 时, 保存的 RIP 值是**#DF 异常将要**压入栈中的返回地址 (由于 triple fault 是在 #DF 异常的 delivery 期间产生的)。

同样, 如果在 VM-exit 前已经压入了部分数据 (参见 5.10.2 节), 那么保存的 RIP 值可能是已经压入栈中的返回地址值 (或者在尝试任务切换时, 将要保存在旧 TSS 块内的 EIP 值)。

- 由于尝试进行任务切换引发 VM-exit 时 (向量号在 IDT 内对应的描述符是 task-gate 描述符), 有下面的情形:
 - 由 CALL, JMP, IRET, INT, INT3, INTO, BOUND 或者 UD2 指令发起时, 保存的 RIP 值指向指令本身。
 - 排除上面所说的指令发起, 在向量事件的 delivery 期间尝试任务切换时, 保存的 RIP 值是将要写入旧 TSS 内的 EIP 值。
- 由 MOV, MOV to CR8 或者 WRMSR 指令引发 trap 类型 VM-exit 时, 保存的 RIP 指向该指令的下一条指令。

5.11.3 保存 RFLAGS

RFLAGS 寄存器的值直接保存在相应的 RFLAGS 字段里，但是 RF 位 (bit 16) 根据不同的 VM-exit 情况，有下面的处理。

- 由于向量事件直接引发 VM-exit 时，保存的 RF 位是将要压入栈中的 RFLAGS.RF 值。

如果在 VM-exit 前已经压入了部分数据 (参见 5.10.2 节)，那么保存的 RF 位可能是已经压入栈中的值 (或者在尝试任务切换时，将要保存在旧 TSS 块内的 EFLAGS.RF 值)。

- 由于 triple fault 引发 VM-exit 时，保存的 RF 位为 1。
- 由于尝试任务切换引发 VM-exit 时，保存的 RF 位是将要写入旧 TSS 内的 RFLAGS.RF 值。
- 由于尝试执行指令而直接引发 VM-exit 时，保存的 RF 位为 0。
- 由于 APIC-access, EPT violation 或者 EPT misconfiguration 而引发 VM-exit 时，保存的 RF 位取决于 VM-exit 是否发生在向量事件的 delivery 期间 (即取决于 IDT-vectoring information 字段的 bit 31 位是否为 1)。
 - 假如没有发生在 delivery 期间，保存的 RF 位为 1。
 - 假如发生在 delivery 期间，保存的 RF 位是将要写入栈中的 RFLAGS.RF 值。
- 对于其他类型的 VM-exit 事件，保存的 RF 位是 VM-exit 之前的值。

5.11.4 保存段寄存器

每个段寄存器有相应的 Selector, Base, Limit 及 Access rights 字段 (参见 3.8.1 节)，段寄存器的 Selector 值直接保存在 Selector 字段里。

而 Base, Limit 及 Access rights 字段保存的情况取决于该段寄存器在 VM-exit 前是否属于 unusable (不可用的)。

(1) 假如段寄存器是 **usable** (可用的)，将直接保存 Base, Limit 及 Access rights 字段 (bits 7:0 及 bits 16:12)。

(2) 假如段寄存器是 **unusable** (不可用的)，保存的 Base, Limit 及 Access rights 字段 (bits 7:0 及 bits 15:12) 是**未定义值**。Access rights 字段的 bit 16 (unusable 位) 为 1。

下面的特殊处理除外：

- CS 寄存器
 - CS.Base 与 CS.Limit 域正常保存在 Base 与 Limit 字段中。
 - CS.L, CS.D 及 CS.G 位保存在 Access rights 字段中，其余位未定义值。
- SS 寄存器
 - SS.DPL 保存在 Access rights 字段中，其余位未定义值。

- 在 64 位架构处理器上，Base 字段的 bits 63:32 位被清为 0 值。
- DS 与 ES 寄存器
 - 在 64 位架构处理器上，Base 字段的 bits 63:32 位被清为 0 值。
- FS 与 GS 寄存器
 - FS.Base 与 GS.Base 域保存在对应的 Base 字段中。
- LDTR 寄存器
 - Base 字段被强置为 canonical 地址形式 (bits 63:47 是 bit 47 位的符号扩展)。

注意：TR 寄存器必须是 usable (可用的)，因此 TR 寄存器总是被保存。而 Access rights 字段的 bits 11:8 及 bits 31:17 总是被清为 0 值。

5.11.5 保存 GDTR 与 IDTR

GDTR 与 IDTR 寄存器被保存在对应的 GDTR 与 IDTR 字段中。

5.11.6 保存 activity 与 interruptibility 状态信息

在发生 VM-exit 时，处理器的 activity 与 interruptibility 状态某些情况下在 VM-exit 开始之前已经被更新，某些情况下在 VM-exit 完成后才被更新（参见 4.16 节，4.17 节及 5.10 节相关内容）。

activity state 与 interruptibility state 字段保存处理器 VM-exit 时的当前状态信息，也就是在 VM-exit 开始前已经被更新的状态信息。

当处理器处于 inactive 状态时，一些**未被阻塞的事件**能将处理器转为 active 状态。当这些事件直接产生 VM-exit 时，只有在 VM-exit 完成后才转为 active 状态（参见 4.17.3 节）。在这种情况下，interruptibility state 字段保存的状态信息仍指示为 **inactive**。

当处理器存在“blocking by STI”或者“blocking by MOV-SS”阻塞状态时，如果在 VM-exit 发生时，处理器当前的这些阻塞状态还没被解除，则保存的 interruptibility state 字段仍指示存在这些阻塞状态。如果发生 **trap** 类型的 VM-exit，在 VM-exit 之前这些阻塞状态已经被解除，则保存的 interruptibility state 字段指示不存在这些阻塞状态。

当 VM-exit 返回到非 SMM 模式时，interruptibility state 字段的 bit 2 被清 0，指示不存在“blocking by SMI”阻塞状态。

5.11.7 保存 pending debug exception 信息

在 VM-exit 当前，如果处理器存在 #DB 异常的悬挂状态，那么 pending debug exception 字段将保存这个悬挂 #DB 异常的信息。否则，pending debug exception 字段被清为 0。

#DB 异常的悬挂，只可能在下面几种类型的 VM-exit 中产生（参见 3.8.7 节及

4.13 节)。

(1) 由 INT, SMI 或者#MC (机器检查) 异常引发的 VM-exit。

(2) 由 trap 类型 VM-exit 引发的 VM-exit。也就是 TPR below threshold, EOI 虚拟化, APIC-write VM-exit 或者由 “monitor trap flag” 引起的 MTF VM-exit。

(3) 在 VM-exit 之前已经存在#DB 异常的悬挂, 但并不是由#DB 异常而引发的 (不是由于 exception bitmap 字段 bit 1 为 1 而直接引发 VM-exit)。

看下面的代码示例:

```
... ... ; TF=1, 开启指令单步
mov ss, ax ; 产生 “blocking by MOV-SS” 阻塞状态
cpuid ; <--- 由 CPUID 指令造成 VM-exit, 在退出 VM 之前已经存在#DB 异常的悬挂状态
```

上面这个示例中 VM-exit 是由于执行 CPUID 指令产生的, 而不是直接由#DB 异常产生的。但是由于启用了指令单步调试, 在 VM-exit 之前已经存在#DB 异常的悬挂。

```
... ... ; TF=1, 开启指令单步
mov ss, ax ; 产生 “blocking by MOV-SS” 阻塞状态
mov rsp, [stack] ; 由于 APIC-access 而引发 VM-exit
```

以上是前面示例的另一个版本, 执行 “mov rsp, [stack]” 指令时, 如果由于 APIC-access, EPT violation 或者 EPT misconfiguration 而引发 VM-exit, 也会造成#DB 异常的悬挂。同样, 因为在 VM-exit 之前已经存在#DB 异常的悬挂 (TF=1), 但这个 VM-exit 并不是由#DB 异常直接引发的。

```
SET_BREAKPOINT 0, BP_READ_WRITE8, rax ; 一个数据断点设置在 rax 地址
mov [rax], rbx ; 由于 TPR below threshold 而产生 VM-exit
```

一条指令如果产生了 trap 类型的 VM-exit (譬如由于 TPR below threshold 而产生 VM-exit, 或者由于 “monitor trap flag” 位为 1 而产生 MTF VM-exit), 同时这条指令由于访问了数据断点或者 I/O 断点, 而导致在 VM-exit 时产生了#DB 异常的悬挂。如上面示例中的 “mov [rax], rbx” 指令。

我们可以思考一下另一种情形。

```
... ... ; TF=1, 启用单步调试
mov ss, ax ; 产生 “blocking by MOV-SS” 阻塞状态
; <--- 在此处由于 INIT, SMI 或者#MC 而产生 VM-exit
mov rsp, [stack] ;
```

在上面的示例中, 启用了指令单步调试, 在 MOV-SS 或者 “mov rsp, [stack]” 指令之后如果收到 INIT, SMI 或者#MC 异常, 由于这些事件的优先级别都比#DB 异常要高, 因而产生 VM-exit, 从而造成#DB 异常的悬挂。

注意: 很难在 guest 里重现这种情形, 毕竟人为地造成在 MOV-SS 指令之后刚好收到 INIT, SMI 或者#MC 异常不是件容易的事。

由上面所述的几种途径产生#DB 异常悬挂时, pending debug exception 字段的保存如下:

- B0 到 B3 (Bits 3:0) 相应的位被置 1, 指示遇到了 DR0 到 DR3 之间的数据断点或 I/O 断点, 同时遇到多个断点则有多个位被置 1。对于单步调试这些位为 0 值。
- “enabled breakpoint” 位 (bit 12) 置 1, 指示遇到数据断点或 I/O 断点。
- BS 位 (bit 14) 置 1, 指示遇到了单步调试。

注意: 当 VM-exit 由于 INIT, SMI, #MC 异常或者 “monitor trap flag” 而产生, 同时 pending debug exception 字段的 BS 位为 1, 并且 IA32_DEBUGCTL.BTF=1 时, 则表明遇到了分支单步调试 #DB 异常 (参见 3.8.7.1 节及 4.13.1 节)。

由上面所述的其他 VM-exit 原因造成 #DB 异常悬挂时, 有下面的设置。

- TF=1, 并且 IA32_DEBUGCTL.BTF=0 时, BS 位为 1。
- TF=0, 或者 IA32_DEBUGCTL.BTF=1 时, BS 位为 0。

VM-entry 后立即 VM-exit 时

由 4.15.2 节~4.15.7 节所描述的事件在 VM-entry 后立即引发 VM-exit 时, pending debug exception 字段在 VM-entry 时加载, 然后在立即引发 VM-exit 后保存回去。因此, pending debug exception 字段的值等于 VM-entry 时加载的值。

5.11.8 保存 VMX-preemption timer 值

当 pin-based VM-execution control 字段 “activate VMX-preemption timer” 位为 1 时, 在 VM-entry 开始时处理器从 guest 的 VMX-preemption timer value 字段加载 32 位的初始计数值并开始递减。

发生 VM-exit 事件后, 当 VM-exit control 字段的 “save VMX-preemption timer value” 位为 1 时, 当前的 VMX-preemption timer 计数值保存在 VMX-preemption timer value 字段中。否则, VMX-preemption timer value 字段值不改变。

VM-exit 由于 VMX-preemption timer 超时而引发, 或者计数值在 VM-entry 期间已经减为 0 值, 此时保存的 VMX-preemption timer value 字段值为 0。

5.11.9 保存 PDPTE

当 secondary processor-based VM-execution control 字段的 “enable EPT” 位为 1, 并且 VM-exit 当前使用 PAE 分页模式 (IA32_EFER.LMA=0, CR0.PG=1, CR4.PAE=1) 时, 处理器内部使用的 4 个 PDPTE 寄存器值将保存在 PDPTE 字段中。

保存的 PDPTE 字段设置如下。

- 由于 PDPTE[11:9] 为忽略位, 因此 PDPTE 字段的 bits 11:9 为未定义值。
- PDPTE 字段的 bit 0 为 0 时 (P=0), 其余的 bits 63:1 忽略 (未定义值)。
- PDPTE 字段的 bit 0 为 1 时 (P=1), bits 63:1 为保存的 PDPTE 表项值。

若“enable EPT”位为 0，或者 VM-exit 时处理器不使用 PAE 分页模式，则 PDPTE 字段不改变。

5.11.10 保存 SMBASE 与 VMCS-link pointer

在 SMM 双重监控处理机制下，当发生 SMM VM-exit 时，处理器内部的 SMBASE 寄存器值保存在 SMBASE 字段中。

VMCS-link pointer 字段的保存取决于 SMM VM-exit 发生在 root 还是 non-root 环境。

- 当从 root 环境中发生时，VMCS-link pointer 字段保存当前的 Current-VMCS pointer 值。而 VM-execution 控制区域中的 Executive-VMCS pointer 字段将保存 VMXON pointer 值。
- 当从 non-root 环境中发生时，Executive-VMCS pointer 字段保存当前的 Current-VMCS pointer 值，而 VMCS-link pointer 字段不改变。

5.12 保存 MSR-store 列表

当 VM-exit MSR-store count 字段的值不为 0 时，处理器将根据 VM-exit MSR-store 列表项中的 MSR index 值 (bits 31:0)，**读取**相应的 MSR 寄存器值写入 VM-exit MSR-store 列表项的 MSR data (bits 127:64) 中。

注意：这是在进行 VM-entry 操作时加载 VM-entry MSR-load 列表的**逆操作**（参见 4.10 节）。在进行 VM-exit 操作时是保存 guest-MSR 列表，而 VM-entry 操作时是加载 guest-MSR 列表。

下面的 C 伪代码描述了 VM-exit MSR-load 列表的保存操作。

```
for (i = MsrCount; i; i--)           // 根据 MSR count 字段值保存 MSR 数量
{
    ecx = MsrIndex;                   // 从 MSR-store 表项的 bits 31:0 提供 MSR 编号
    rdmsr();                           // 执行 RDMSR 指令读取 MSR
    MsrData[31:0] = eax;               // 写入 MSR-store 表项的 bits 95:64
    MsrData[63:32] = edx;              // 写入 MSR-store 表项的 bits 127:96
}
```

根据 MSR index 提供的 MSR 编号来保存相应的 MSR，相当于通过 MSR index 放入 ECX 寄存器然后执行 RDMSR 指令读取 MSR。有多少个 MSR 保存，就需要执行多少次 RDMSR 指令。

VMM 需要设置正确的 VM-exit MSR-store 列表数据，保证在执行 RDMSR 指令时不会出错。在下面的情况下保存 MSR 将会失败。

- VM-exit MSR-store 列表项的保留位 (bits 63:32) 不为 0 值。
- MSR index 提供了**保留**或者**未实现**的 MSR。
- MSR index 值在 00000800H 到 000008FFH 之间，也就是读取 x2APIC 模式下的

local APIC 寄存器。

- 当 VM-exit 完成后处理器在非 SMM 模式下，MSR index 提供了只能在 SMM 模式下读取的 MSR。典型地，**IA32_SMBASE** 寄存器只能在 SMM 模式读取。
- MSR index 提供的 MSR 在读取时发生 #GP 异常，但并不是由权限不足而引起的。
- MSR index 提供的 MSR 由于处理器架构实现的制约，而不能在 MSR-store 列表中保存（正常情况下可以使用 RDMSR 指令读取该 MSR）。在 VMX 当前架构下，有两个 MSR 不支持出现在 MSR-store 列表里：**IA32_BIOS_UPDT_TRIG** 及 **IA32_BIOS_SIGN_ID**（编号分别为 79H 与 8BH）。

当保存 MSR-store 列表发生失败时，将会引发 VMX-abort，最终导致处理器进入 shutdown 状态。

5.13 加载 host 环境

前面 5.8 节~5.12 节的保存工作完毕后，处理器开始从 host-state 区域加载 host 环境信息。处理器切换回 VMX root operation 模式后，当前的状态信息被更新为 host 环境下的状态信息。

5.13.1 加载控制寄存器

Host-state 区域的 CR0, CR3 以及 CR4 字段值分别加载到 CR0, CR3 及 CR4 寄存器。然而，处理器还会对这些寄存器做下面这些强制的设置（另参见 4.4.4.1 节描述）。

(1) CR0 寄存器

- **CR0.ET** 总是设为 1 值（忽略 CR0 字段的 bit 4）。
- **CR0.CD** 与 **CR0.NW** 保持不变（忽略 CR0 字段的 bit 30 与 bit 29）。
- CR0 寄存器的 bits 63:32（在 64 位架构处理器上），bits 28:19, bit 17 及 bits 15:6 清为 0（忽略 CR0 字段对应的位）。
- 其余位从 CR0 字段中加载，但必须符合 VMX 架构对 CR0 寄存器的要求，即满足 Fixed1 与 Fixed0 位的设置（参见 2.5.10 节）。

(2) CR3 寄存器

- 在 64 位架构处理器上，CR3 寄存器的 **bits 63:N** 被清 0（ $N = \text{MAXPHYADDR}$ 值）。

(3) CR4 寄存器

- 当 VM-exit control 字段的“host address-space size”为 1 时，CR4.PAE 置为 1。
- 当 VM-exit control 字段的“host address-space size”为 0 时，CR4.PCIDE 清为 0。
- 其余位从 CR4 字段里加载，但必须符合 VMX 架构对 CR4 寄存器的要求（参见 2.5.10 节），CR4.VMEX 必须为 1。

实际上，除 **CR0.CD**、**CR0.NW** 及 **CR0.ET** 强制设置外（忽略 CR0 字段的 bit 30，

bit 29 及 bit 4 位)，CR0、CR3 及 CR4 字段在 VM-entry 操作时必须通过检查（参见 4.4.4.1 节），CR0 与 CR4 字段必须满足 VMX 架构对 CR0 与 CR4 寄存器的设置要求（即符合 Fixed1 与 Fixed0 位的设置）。例如 **CR0.NE** 必须为 1。

注意：如果 VM-exit 后 host 使用 PAE 分页模式，CR3 的加载也会引发对 PDPTE 寄存器的加载（参见 5.13.6 节）。

5.13.2 加载 DR7 与 MSR

- DR7 寄存器被强制设为 **00000400H** 值（除 bit 10 必须为 1 外，其余位为 0）。
- IA32_DEBUGCTL 寄存器强制清为 00000000_00000000H 值。
- IA32_SYSENTER_CS 寄存器的 bits 31:0 从 IA32_DEBUGCTL 字段中加载，而 bits 63:32 清为 0。
- IA32_SYSENTER_ESP 与 IA32_SYSENTER_EIP 寄存器分别从对应的字段中直接加载。
 - 在 64 位架构处理器上，IA32_SYSENTER_ESP 与 IA32_SYSENTER_EIP 寄存器的值必须是 canonical 地址形式（即 bits 63:48 是 bit 47 的符号扩展位）。
 - 在 32 位架构处理器上，bits 63:32 清为 0。
- 当 VM-exit control 字段的“load IA32_PERF_GLOBAL_CTRL”为 1 时，IA32_PERF_GLOBAL_CTRL 寄存器的值将从 IA32_PERF_GLOBAL_CTRL 字段中加载，保留位不变。
- 当 VM-exit control 字段的“load IA32_PAT”为 1 时，IA32_PAT 寄存器的值将从 IA32_PAT 字段中加载，保留位不变。
- IA32_EFER 寄存器的加载有下面的情形。
 - LMA 与 LME 位被强制设为 VM-exit control 字段“host address-space size”位的值（不管“load IA32_EFER”位是否为 1）。
 - 当 VM-exit control 字段的“load IA32_EFER”为 1 时，IA32_EFER 寄存器的值从 IA32_EFER 字段中加载，保留位不变。

在加载 IA32_EFER 寄存器时，先强制设置 IA32_EFER.LMA 与 IA32_EFER.LME 位，然后再决定是否在 IA32_EFER 字段中加载。但是，**IA32_EFER 字段的值必须满足要求**（参见 4.5.6 节）。

5.13.3 加载 host 段寄存器

在 host-state 区中，对于段寄存器来说，只有 CS、SS、DS、ES、FS 与 GS 寄存器的 selector 字段，以及 FS、GS 与 TR 寄存器的 base 字段（参见 3.9 节）。在 4.4.4.3 节里描述了对这些 selector 字段的检查，4.4.4.4 节描述了对这些 base 字段的检查。

在加载 host 段寄存器时，上述的这些字段值被相应地加载到寄存器的 selector 与 base 域中。对于段寄存器的其余域，处理器会进行强制设置，以保证符合 host 运行环境的要求。

5.13.3.1 加载 selector

- CS.selector 与 TR.selector 分别从对应的 selector 字段中加载。
- SS, DS, ES, FS 及 GS 寄存器的 selector 分别从对应的 selector 字段中加载。
 - 当 selector 被加载为 0 值时, 这些段寄存器属于 **unusable** (不可用的)。

注意: CS 与 TR 寄存器的 selector 字段不允许加载为 0 值, CS 与 TR 寄存器必须是 **usable** (可用的)。VM-exit control 字段的“host address-space size”位决定了是否返回 64 位模式的 host 里, 从而决定了 SS, DS, ES, FS 及 GS 寄存器的使用。

- 当“host address-space size”为 1 时, SS, DS, ES, FS 及 GS 寄存器的 selector 允许为 0 值 (允许为 **unusable**), 使用它们来访问不会出错。
- 当“host address-space size”为 0 时, 有下面的情况。
 - SS.selector **不允许加载**为 0 值 (不能为 **unusable**)。
 - DS, ES, FS 及 GS 寄存器的 selector **允许加载**为 0 值。但是, 在使用它们进行访问时会引发 #GP 异常。

5.13.3.2 加载 base

在 64 位架构处理器上, FS, GS 及 TR 寄存器的 base 字段**必须是 canonical** 地址形式, 不管 FS 与 GS 寄存器是否属于可用的 (也就是它们的 selector 是否为 0)。

注意: FS.base 与 GS.base 分别映射到 **IA32_FS_BASE** 与 **IA32_GS_BASE** 寄存器中。

- FS.base 与 GS.base 分别从对应的 base 字段中加载。
 - 假如 FS.selector 与 GS.selector 被加载为 0 值 (FS 与 GS 寄存器属于 **unusable**), 并且“host address-space size”位为 0, 加载的 base 属于未定义值。
 - 当“host address-space size”位为 1 时, 加载的 base 有效 (不管 FS 与 GS 寄存器是否为 **unusable**)。
- TR.base 从对应的 base 字段中加载。

对于其他段寄存器 (CS, SS, DS 及 ES) 的 base 值, 处理器进行下面的强制设置。

- CS.base 被清为 0。
- 假如 SS, DS 及 ES 寄存器属于 **unusable** (即 selector 被加载为 0 值), base 值未定义。否则, 被清为 0。

5.13.3.3 加载 limit

Host-state 内不存在 limit 字段。因此, 段寄存器内的 limit 域被强制设置为下面的值。

- CS.limit 设为 FFFFFFFFH 值 (4G 段限)。
- 假如 SS, DS, ES, FS 及 GS 寄存器属于 **unusable**, limit 值未定义。否则, 被设为 FFFFFFFFH 值。
- TR.limit 强制设为 **00000067H** 值。这时候 TSS 内不含 I/O permission bitmap (I/O 许可位图)。

5.13.3.4 加载 access rights

host-state 内不存在 access rights 字段。因此，段寄存器内的 access rights 域被设置为下面的值。

(1) CS 寄存器，access rights 被设为 0000A09BH 值或者 0000C09BH 值

- Type 值 (bits 3:0) 为 11。
- S 位 (bit 4) 为 1。
- DPL 值 (bits 6:5) 为 0。
- P 位 (bit 7) 为 1。
- L 位 (bit 13) 等于 VM-exit control 字段 “host address-space size” 位的值。
- D/B 位 (bit 14) 的值设置为：
 - “host address-space size” 位为 1 时，D/B 位为 0。
 - “host address-space size” 位为 0 时，D/B 位为 1。
- G 位 (bit 15) 为 1。

(2) SS 寄存器

- 属于 **unusable** 时，寄存器内的 access rights 设置如下。
 - DPL 值 (bits 6:5) 为 0。
 - D/B 位 (bit 14) 为 1。
 - 其余位 (bit 15, bit 7 及 bits 4:0) 为未定义值。
- 属于 **usable** 时，寄存器内的 access rights 被设置为 **0000C093H**。
 - Type 值 (bits 3:0) 为 3。
 - S 位 (bit 4) 为 1。
 - DPL 值 (bits 6:5) 为 0。
 - P 位 (bit 7) 为 1。
 - D/B 位 (bit 14) 为 1。
 - G 位 (bit 15) 为 1。

(3) ES, DS, FS 及 GS 寄存器

- 属于 **unusable** 时，寄存器内的 access rights 为未定义值。
- 属于 **usable** 时，寄存器内的 access rights 被设置为 **0000C093H**。
 - Type 值 (bits 3:0) 为 3。
 - S 位 (bit 4) 为 1。
 - DPL 值 (bits 6:5) 为 0。
 - P 位 (bit 7) 为 1。
 - D/B 位 (bit 14) 为 1。
 - G 位 (bit 15) 为 1。

(4) TR 寄存器，access rights 被设置为 **0000008BH** 值

- Type 值 (bits 3:0) 为 11。

- S 位 (bit 4) 为 0。
- DPL 值 (bits 6:5) 为 0。
- P 位 (bit 7) 为 1。
- D/B 位 (bit 14) 为 0。
- G 位 (bit 15) 为 0。

注意：host-state 内并不存在 LDTR 寄存器对应的任何字段。LDTR.selector 被清为 0，LDTR 寄存器被设置为 unusable（不可用的），它的 limit、base 及 access rights 为未定义值。

5.13.4 加载 GDTR 与 IDTR

在 host-state 内不存在 GDTR 与 IDTR 寄存器的 limit 字段。

- GDTR.base 与 IDTR.base 分别从对应的 base 字段中加载。
- GDTR.limit 与 IDTR.limit 被设置为 **0000FFFFH** 值（64K 表限）。

5.13.5 加载 RIP, RSP 及 RFLAGS

在 4.4.4.2 节中描述了 VM-entry 时对 host-RIP 字段的检查，但并不检查 host-RSP 字段。另外，在 host-state 区域中并不存在 RFLAGS 字段。

- RFLAGS 字段强制设为 **00000002H** 值（bit 1 固定为 1）。
- RIP 与 RSP 寄存器分别从对应的字段中加载。

注意：由于 host RFLAGS 被设为 02H 值，意味着 IF 位被清为 0。也就是说在 VM-exit 完成后外部中断被屏蔽。直到 VMM 使用 STI 或者 POPF 指令重新置 IF=1，才允许响应外部中断。

参见 3.7.1 节，VM-exit control 字段的“acknowledge interrupt on exit”位决定是否响应中断控制器（8259 或者 local APIC，I/O APIC），从而获取外部中断的 delivery 信息。

另一方面，当“acknowledge interrupt on exit”为 0 时，VMM 也可以通过重新设置 IF=1 夺取外部中断的 delivery 控制权。

5.13.6 加载 PDPTE

在 host-state 区域不存在 PDPTE 字段。当 VM-exit 后 host 使用 PAE 分页模式（IA32_EFER.LMA=0，CR0.PG=1，CR4.PAE=1），属于下面的情况之一时，对 CR3 寄存器的加载也会执行 PDPTE 加载。

- VM-exit 之前，guest 不使用 PAE 分页模式。
- VM-exit 之后，CR3 寄存器的值发生了改变（即 CR3 加载了不同的值）。

在 VM-exit 时，对 host-CR3 寄存器的加载相当于执行一条 MOV to CR3 指令。在加载 PDPTE 前，处理器对 CR3 寄存器所指向的 PDPTE 表项进行检查（参见 4.5.11.1 节）。

假如 CR3 寄存器加载时产生了错误（相当于执行 MOV to CR3 指令产生了 #GP 异常。如由于 PDPTE 表项的保留位不为 0 值），VM-exit 将失败，导致 VMX-abort 发生，处理器进入 shutdown 状态。

5.14 更新 host 处理器状态信息

在 VM-exit 后，下面的处理器状态信息被更新。

- 处理器处于 active 状态。
- 处理器不存在“blocking by STI”及“blocking by MOV-SS”阻塞状态。
- 假如 VM-exit 由 NMI 直接引发（pin-based VM-execution control 字段“NMI exiting”为 1），VM-exit 后处理器的“blocking by NMI”阻塞状态有效（VM-exit 完成后更新，参见 5.10 节）。
- VM-exit 后，处理器不存在 pending debug exception（是指 host 环境不存在）。

5.15 刷新处理器 cache 信息

在 VM-exit 后，取决于 secondary processor-based VM-execution control 字段“enable VPID”位，处理器的 cache 信息会被刷新（另参见 4.8 节）。

- 当“enable VPID”为 1 时，不需要刷新 linear mapping, combined mapping 及 guest-physical mapping 产生的 cache 信息（见 2.6.7 节）。这些 cache 信息在 VM-exit 后保持有效。
- 当“enable VPID”为 0 时，VM-exit 后处理器将进行下面的 cache 刷新处理（参见 6.2 节）：
 - 刷新默认 VPID 值（0000H）下所有 PCID 值对应的 linear mapping 相关 cache 信息。
 - 刷新默认 VPID 值（0000H）下所有 PCID 与 EP4TA 值对应的 combined mapping 相关 cache 信息。

处理器的这个刷新操作，相当于执行了一次 INVVPID 指令，使用 single-context 方式，目标 VPID 值为 0000H（参见 2.6.7.2 节）。

清地址监控

在这一步中，额外的工作将清由 MONITOR/MWAIT 指令设置的地址监控。这个措施可以防止利用地址监控功能检测 non-root 与 root 之间的切换。

5.16 加载 MSR-load 列表

在这一步中，当 VM-exit MSR-load count 字段的值不为 0 时，将从 VM-exit MSR-load 列表中加载 **host-MSR**（参见 4.10 节及 5.12 节描述的 **guest-MSR** 加载及保存操作）。

下面的 C 伪代码描述了 VM-exit MSR-load 列表的加载操作。

```
for (i = MsrCount; i; i--)           // 根据 MSR count 字段值决定加载 MSR 的数量
{
    eax = MsrData[31:0];             // 读取 MSR data 低 32 位
    edx = MsrData[63:32];           // 读取 MSR data 高 32 位
    ecx = MsrIndex;                 // 从 MSR-store 表项的 bits 31:0 提供 MSR 编号
    wrmsr();                         // 执行 WRMSR 指令写 MSR
}
```

这个 VM-exit MSR-load 列表加载的操作相当于执行一系列的 WRMSR 指令。处理器从 VM-exit MSR-load 列表的 MSR data (bits 127:64) 中读取需要加载的 MSR 数据，写入由 MSR index (bits 31:0) 提供的 MSR 编号所对应的 MSR。

VM-exit MSR-load count 字段的值是多少，就代表需要加载多少个 MSR（也就是需要执行多少次 WRMSR 指令）。VMM 需要设置正确的 MSR-load 列表数据，保证在执行 WRMSR 指令时不能出错。

VM-exit MSR-load 列表项数据属于下面的情况之一时，加载 MSR-load 列表将会失败。

- 保留位 (bits 63:32) 不为 0 值。
- MSR index (bits 31:0) 提供了**保留或者未实现的 MSR**。
- MSR index (bits 31:0) 值为 **C0000100H** 或者 **C0000101H**。也就是在 MSR-load 列表中不能加载 **IA32_FS_BASE** 与 **IA32_GS_BASE** 寄存器。
- MSR index (bits 31:0) 值为 **00000800H 到 000008FFH**。也就是在 MSR-load 列表中不能加载 x2APIC 模式使用的 local APIC 寄存器。
- MSR index (bits 31:0) 提供了只能在 SMM 模式下写的 MSR，但 VM-exit 完成后处理器不在 SMM 模式。典型地 **IA32_SMM_MONITOR_CTL** 寄存器只能在 SMM 模式下写。
- MSR index (bits 31:0) 提供的 MSR 在写时发生 #GP 异常（**但不是由权限不足引起的**）。例如，当写 **IA32_EFER** 寄存器时，CR0.PG 为 1，但 MSR data (bits 127:64) 的 LME 位为 0，那么写这个 MSR 时会引起 #GP 异常（由于不能在开启分页下关闭 IA-32e 模式）。
- MSR index (bits 31:0) 提供的某些 MSR 可能会由于处理器架构实现的制约，而在 VM-exit 操作时不能在 VM-exit MSR-load 列表中加载（尽管可以正常使用 WRMSR 指令对该 MSR 进行写）。在 VMX 当前架构下，有两个 MSR 不支持出现在 VM-exit MSR-load 列表中：**IA32_BIOS_UPDT_TRIG** 及 **IA32_BIOS_SIGN_ID**（编号分别为 79H 和 8BH），它们不允许在 VM-exit 时加载。

在加载 VM-exit MSR-load 列表阶段，可能会由于 VM-exit MSR-load 列表时出现

IA32_EFER , IA32_PAT , IA32_PERF_GLOBAL_CTRL , IA32_SYSENTER_CS , IA32_SYSENTER_ESP 及 IA32_SYSENTER_EIP 寄存器, 而导致这些寄存器的重复加载。

参见 5.13.2 节描述这些 MSR 的初始化加载, 另参见 4.10.1 节与 4.10.2 节描述 VM-entry 时这些 MSR 寄存器的重复加载 (在 VM-exit 操作里这些寄存器重复加载情况类似)。

如果在加载 VM-exit MSR-load 列表时出错将会导致 VMX-abort 发生, 从而转入 shutdown 状态。

5.17 VMX-abort

在进行 VM-exit 操作时, 任何错误的发生都会导致 VMX-abort (VMX 中止)。VMX-abort 使得处理器最终转入 **VMX-abort shutdown** 状态。

下面是在 VM-exit 操作时可能发生 VMX-abort 的环节。

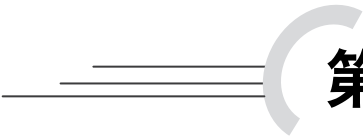
(1) Current-VMCS 内的某些字段或者处理器内部维护的数据 (参见 2.6.5.3 节) 可能已经被破坏而导致 VM-exit 不能成功。这些内部数据用来维护处理器相关的信息, 也可能是用来维护 VMCS 的状态 (参见 3.1 节)。造成这些破坏可能是由于其他逻辑处理器对这个 VMCS 进行 VMCLEAR 操作, VMWRITE 操作等, 或者直接使用 MOV 指令写 VMCS 内存区域。

- 例如, VMM 运行在 IA-32e 模式下, 但 VM-exit control 字段的 “host address-space size” 位被清 0。
- (2) 在保存 VM-exit MSR-store 列表阶段 (参见 5.12 节)。
- (3) 在加载 PDPTE 阶段 (参见 5.13.6 节)。
- (4) 在加载 VM-exit MSR-load 列表阶段 (参见 5.16 节)。
- (5) 在 VM-exit 操作期间发生机器检查。

当 VMX-abort 发生后, 处理器做下面的处理。

- 不修改当前处理器的任何 active VMCS 内的数据 (当前处理器含有多个 active VMCS, 参见 3.1.1 节)。
- 处理器在当前发生 VMX-abort 的 VMCS 中保存非 0 值的 VMX-abort ID (参见 3.2.3 节)。
- 假如处理器处于非 SMX (Safe Mode Extension) 模式将进入 **VMX-abort shutdown** 状态。

进入 VMX-abort shutdown 状态后, 处理器产生特殊的 bus 周期通知芯片组, 只有 RESET 事件才能唤醒 VMX-abort shutdown 状态。外部中断、机器检查事件、INIT、SMI 及 SIPI 都被阻塞 (这一点不同于处理器的 shutdown 状态, 参见 4.17 节)。



第 6 章

内存虚拟化

在虚拟化平台中，每个 VM 有自己独立的（私有的）物理地址空间。思考一下，VM 本身并不知道有其他 VM 的存在，只认为自己独占从 0 到 MAXPHYADDR 这个范围的物理地址空间。这个原理与 OS 下进程独占地址空间一般无异。譬如在 32 位系统下，一个进程认为自己占有独立的 4G 物理地址空间。然而这个地址空间实际上是**虚拟地址空间**，用来隔绝每个进程对物理地址空间的访问。

VMM 负责将 VM 私有的物理地址空间隔离起来，确保 VM 私有的物理地址空间不受其他 VM 及 VMM 本身的影响。VM 私有的物理地址空间被称为 GPA (**Guest-Physical Address**) 空间，而真实平台上的物理地址空间被称为 HPA (**Host-Physical Address**) 空间。

地址空间与地址

对一个 VM 或者 OS 下的每个进程可以访问的**范围**应该描述为“**空间**”。而“**地址**”只是这个空间的一个位置。物理地址空间里可以映射到不同的存储设备，譬如 RAM (DRAM)，ROM，Video buffer，PCI 设备，PCIe 设备等。

在 x86/x64 平台上，最大的物理地址空间是从 0 到 MAXPHYADDR 之间。这个 MAXPHYADDR 值可以使用 CPUID.80000008H:EAX[7:0]查询得到。当前 x64 体系下实现的最大的虚拟地址空间宽度为 48 位，最大的虚拟地址宽度可以从 CPUID.80000008H:EAX[15:8]查询得到。

6.1 EPT（扩展页表）机制

在处理器端，VMX 架构引入 EPT (Extended Page Table, 扩展页表) 机制来实现 VM 物理地址空间的隔离，EPT 机制的实现原理与 x86/x64 体系下的分页机制是一致的。

当 guest 软件发出指令访问内存时, guest 最终生成 GPA (Guest-Physical Address)。EPT 页表结构定义在 host 端, 处理器接收到 guest 传过来的 guest-physical address 后, 通过 EPT 页表结构转换为 HPA (Host-Physical Address) 从而访问平台上的物理地址。

6.1.1 EPT 机制概述

VMM 在设置前应查询处理器是否支持 EPT 机制, 通过检查 secondary processor-based VM-execution control 字段的“enable EPT”位 (bit 1) 是否允许被置为 1 值 (参见 2.5.6.3 节) 来实现。当“enable EPT”允许为 1 时, 表明处理器支持 EPT 机制。否则不支持。

当“enable EPT”位为 1 时, 表明开启 EPT 机制。在 EPT 机制下, 引出下面两个物理地址概念。

- **GPA:** guest 所有访问的物理地址都被视作 GPA。如同 x86/x64 体系下的虚拟地址一般, 需要经过转换才能访问真实的物理地址。
- **HPA:** 这是平台上最终的物理地址。GPA 必须转换为 HPA 后才能访问真实的物理地址。

在 VMM 中并没有这两个概念, 但 VMM 访问的物理地址也可以被视为 HPA。在开启 EPT 机制后 VMM 需要建立 EPT 页表结构, 通过在 EPTP (Extended Page Table Pointer) 字段中提供 EPT 页表结构的指针值, 为每个 VM 准备不同的 EPT 页表结构, 或者在同一个 EPT 页表结构中准备不同的页表项。

当“unrestricted guest”位为 1 时, “enable EPT”位必须为 1 (参见 4.4.1.3 节), 说明 guest 运行在实模式时必须启用 EPT 机制。同时我们也可以推测, 当处理器支持 unrestricted guest 功能时, 也必定支持 EPT 机制。

6.1.1.1 guest 分页机制与 EPT

在实模式下处理器不使用分页机制, guest 访问所使用的 linear address (线性地址) 就是物理地址 (也就是 guest-physical address)。

当 CR0.PG = 1 时 guest 启用分页机制, guest-linear address (guest 线性地址) 通过页表结构转换为物理地址。当“enable EPT”为 1 时, guest 内的**线性地址转换为 guest-physical address**。与此同时, 也产生了两个页表结构的概念。

- **guest paging structure** (guest 页表结构): 这是 guest 内将线性地址转换为 GPA 的页表结构。也就是在 x86/x64 体系分页机制下所使用的页表结构。
- **EPT paging structure** (EPT 页表结构): 负责将 GPA 转换为 HPA 所使用的页表结构。

注意: 当“enable EPT”为 1 时, guest 内所有的“物理地址”都视为“guest-physical address”。例如, 由 CR3 寄存器所指向的 guest paging structure 地址属于 GPA (在

“enable EPT”为 0 时，CR3 的地址是物理地址），并且 guest paging structure 页表项内所引用的地址都属于 GPA。

而 EPT 所指向的 EPT paging structure 地址是 HPA，并且 EPT paging structure 页表项内所引用的地址都属于 HPA。

图 6-1 是在开启 EPT 机制的情况下 guest 的线性地址访问物理地址的转换示意图。guest-linear address 通过 guest paging structure 页表结构转换为 guest-physical address，再经 EPT paging structure 页表结构转换为 host-physical address 后访问属于自己的内存域 (domain)。

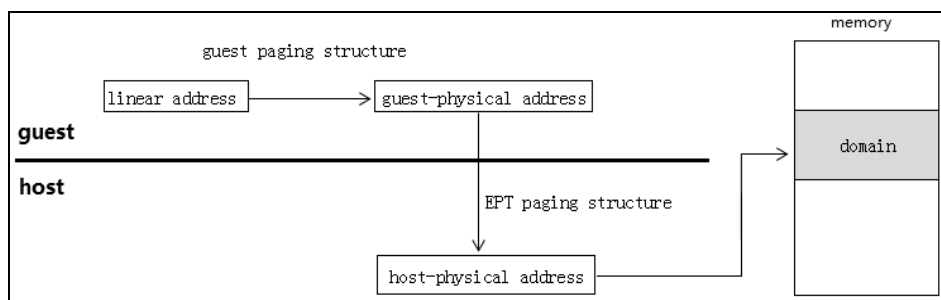


图 6-1

guest 的分页模式

x64 体系上有如下三种分页模式（CR0.PG = 1 时）：

(1) 当 CR4.PAE = 0 时，guest 使用 32 位分页模式。另外，当 CR4.PSE = 1，并且 MAXPHYADDR 值大于等于 40 时，允许在 4M 页面上使用 40 位的物理地址。

(2) 当 IA32_EFER.LMA = 0，并且 CR4.PAE = 1 时，guest 使用 PAE 分页模式。

(3) 当 IA32_EFER.LMA = 1，并且 CR4.PAE = 1 时，guest 使用 IA-32e 分页模式。

guest 的线性地址根据上面的分页模式转换 guest-physical address。当 guest 使用 PAE 分页模式，并且启用了 EPT 机制时，在 VM-entry 时会加载 4 个 PDPTE 字段（参见 4.7.7 节与 4.5.11 节）。

引发 GPA 转换 HPA

下面三个途径会引发 guest-physical address 转换为 host-physical address。

(1) guest 进行内存访问，包括读写访问及执行访问。

(2) guest 使用 PAE 分页模式加载 PDPTE，包括下面的途径：

- 执行 MOV to CR3 指令更新 PDPT 基址（CR3 指向 PDPT）。
- 执行 MOV to CR0 指令修改了 CR0.CD，CR0.NW 或者 CR0.PG 位，从而引起加载 PDPTE（例如，设置 CR0.PG = 1 开启分页机制）。
- 执行 MOV to CR4 指令修改了 CR4.PAE，CR4.PSE，CR4.PGE 或者 CR4.SMEP

位，从而引起加载 PDPTE（例如，设置 CR4.PAE = 1 开启 PAE 分页模式）。

(3) 在 guest-linear address 转换为 guest-physical address 的过程中，处理器访问 guest paging structure 表项内的地址，它们属于 GPA（例如 PDPTE 内的地址值）。

总结来说，GPA 可能是从 **guest-linear address** 转换而来的，或者是直接访问 GPA（即并不是从 guest-linear address 转换而来的）。

guest 分页机制下 GPA 的转换

在分页机制下，完成整个 guest 访问内存操作会引发一系列的 GPA 转换 HPA 过程。

假设 guest 使用 IA-32e 分页模式 (IA32_EFER.LMA = 1, CR4.PAE = 1, CR0.PG = 1)，并且使用 4K 页面。图 6-2 描述了这一系列 GPA 转换过程。

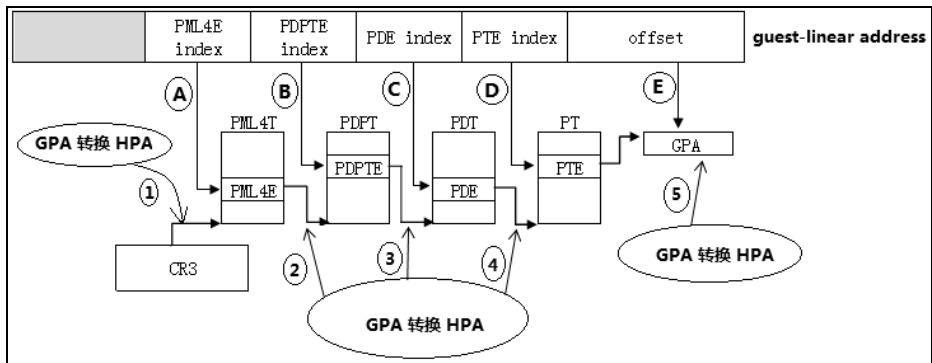


图 6-2

完成这个内存访问操作一共需要进行 5 次 GPA 到 HPA 的转换 ($N = \text{MAXPHYADDR}$)。

(1) CR3 寄存器的 bits $N-1:12$ 提供 PML4T 基址。在定位 PML4T 时需要对 PML4T 基址进行 GPA 转换 (图 6-2 中的第 1 步)。在成功转换 HPA 后得到 PML4T 的物理地址，再由 PML4E index 查找 PML4E (图 6-2 中的 A 点所示)。

(2) PML4E 的 bits $N-1:12$ 提供 PDPT 基址。在定位 PDPT 时需要对 PDPT 基址进行 GPA 转换 (图 6-2 中的第 2 步)。在成功转换 HPA 后得到 PDPT 的物理地址，再由 PDPTE index 查找 PDPTE (图 6-2 中的 B 点所示)。

(3) PDPTE 的 bits $N-1:12$ 提供 PDT 基址。在定位 PDT 时需要对 PDT 基址进行 GPA 转换 (图 6-2 中的第 3 步)。在成功转换 HPA 后得到 PDT 的物理地址，再由 PDE index 查找 PDE (图 6-2 中的 C 点所示)。

(4) PDE 的 bits $N-1:12$ 提供 PT 基址。在定位 PT 时需要对 PT 基址进行 GPA 转换 (图 6-2 中的第 4 步)。在成功转换 HPA 后得到 PT 的物理地址，再由 PTE index 查找 PTE (图 6-2 中的 D 点所示)。

(5) PTE 的 bits $N-1:12$ 提供 4K page frame 基址。这个 page frame 基址加上 guest-

linear address 的 offset 值 (bits 11:0) 得到目标 GPA 值 (图 6-2 中的 E 点所示)。处理器将这个 GPA 转换为 HPA 得到最终的物理地址 (图 6-2 中的第 5 步)，从而完成 guest 内存的访问。

在这一系列的 GPA 转换过程中，任何一个环节都可能会产生 EPT violation 或者 EPT misconfiguration 而导致 VM-exit 发生 (参见 6.1.8 节)。

也可能由于 guest paging structure 而引发 guest 产生 #PF 异常，从而使得 guest 处理 #PF 异常处理例程或者由于 #PF 异常直接或者间接导致 VM-exit。

举一反三，我们可以得到：

- 当 guest 使用 32 位分页模式时，guest 的内存访问操作需要进行 3 次 GPA 转换。即 CR3 寄存器内的 PDT 基址需要进行 GPA 转换，PDE 内的 PT 基址需要进行 GPA 转换，以及合成的 GPA 需要进行转换。
- 当 guest 使用 PAE 分页模式时，guest 的内存访问操作同样需要进行 3 次 GPA 转换。即 PDPTE 寄存器内的 PDT 基址需要进行 GPA 转换，PDE 内的 PT 基址需要进行 GPA 转换，以及合成的 GPA 需要进行转换。

在 PAE 分页模式下，guest 执行 MOV to CR3 指令更新 CR3 寄存器 (也包括更新 CR0 或 CR4 寄存器某些控制位) 引发对 PDPTE 的加载。因此，在加载 PDPTE 表项时也会进行 GPA 的转换。

6.1.2 EPT 页表结构

EPT paging structure (EPT 页表结构) 与 guest paging structure (guest 页表结构) 的实现原理类似。VMX 架构实现最高 4 级 EPT 页表结构，分别为：

- (1) EPT PML4T (**EPT Page Map Level-4 Table**)，它的表项为 EPT PML4E。
- (2) EPT PDPT (**EPT Page Directory Pointer Table**)，它的表项为 EPT PDPTE。
- (3) EPT PDT (**EPT Page Directory Table**)，它的表项为 EPT PDE。
- (4) EPT PT (**EPT Page Table**)，它的表项为 EPT PTE。

软件可以查询 IA32_VMX_EPT_VPID_CAP 寄存器的 bit 6 来确定是否支持 4 级页表结构，为 1 时指示 EPT 支持 4 级页表结构。每个 EPT 页表大小为 4K，每个 EPT 页表项为 64 位宽。

EPT 支持三种页面 (参见 2.5.13 节)：

- 1G 页面，当 IA32_VMX_EPT_VPID_CAP[17] = 1 时处理器支持 1G 页面。PDPTE 的 bit 7 允许置 1 使用 1G 页面。
- 2M 页面，当 IA32_VMX_EPT_VPID_CAP[16] = 1 时处理器支持 2M 页面，PDE 的 bit 7 允许置 1 使用 2M 页面。
- 4K 页面，PDPTE[7] = PDE[7] = 0 时使用 4K 页面，PTE 提供 4K 物理页面地址。

使用 1G 页面时，GPA 转换只需经过两级 EPT 页表的 walk 流程（PML4T 与 PDPT）。使用 2M 页面时，GPA 转换需经过三级 EPT 页表的 walk 流程（PML4T，PDPT 及 PDT）。而使用 4K 页面则需要经过四级 EPT 页表的 walk 流程（如图 6-2 所示）。

6.1.3 guest-physical address

每个 GPA（guest-physical address）值为 64 位宽。但是，当前 VMX 架构实现了最高 48 位有效的 GPA 值（bits 47:0）。然而，在 x86/x64 体系下最高可实现 52 位的物理地址。

更进一步地，实际的 GPA 值宽度与 MAXPHYADDR 值相关。Bits $N-1:0$ （ $N = \text{MAXPHYADDR}$ ）是有效的 GPA 值，而 bits 47: N 会被补为 0 值。在 GPA 转换 HPA 时 bits 63:48 被忽略。

guest-physical address 的产生来自两个方面。

(1) 由 guest-linear address 转换而来。

(2) 不是由 guest-linear address 转换而来（例如，执行 MOV to CR3 指令，或者访问 guest paging structure 页表项）。

由 guest-linear address 转换而来时（在 guest 未分页时 guest-linear address 直接等于 GPA），在正常情况下，32 位分页模式下的线性地址转换为 32 位物理地址，那么 GPA 的高 32 位补为 0 值。在实模式下线性地址为 20 位（即物理地址），那么 GPA 的 bits 63:20 则补为 0 值。

当处理器直接读取 GPA 时，这个 GPA 并不是由 guest-linear address 转换而来的。GPA 的有效宽度必须满足在 MAXPHYADDR 值之内。

- CR3 的 bits 63: N （ $N = \text{MAXPHYADDR}$ ）是保留位。
- EPTP 字段的 bits 63: N （ $N = \text{MAXPHYADDR}$ ）是保留位。
- guest paging structure 里每个表项的 bits 63: N （ $N = \text{MAXPHYADDR}$ ）是保留位，或者被忽略。

这些 GPA 值都被保证不能超越 MAXPHYADDR 之外。思考一下，能支持 48 位以上物理地址的处理器并不多。因此，GPA 的 bits 63:48 被忽略能保证正确性。如果需要处理器支持 48 位以上 GPA，那么可以增加 EPT 页表结构的级数（即 walk 次数）来保证 GPA 的正确转换。

6.1.4 EPTP

EPT 页表结构顶层的 PML4T 物理地址由 EPTP（Extended Page Table Pointer，扩展页表指针）字段提供（参见 3.5.17 节）。PML4T 的物理地址也被称为“EP4TA”（EPTP [$N-1:12$]），它用于标识一个 cache 域。注意，这个物理地址属于 HPA，对齐在 4K 边界上。如图 6-3 所示。

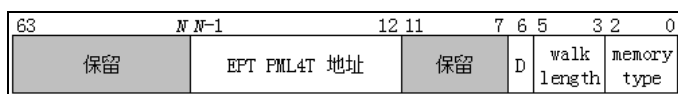


图 6-3

- PML4T 的物理地址 (host-physical address) 值由 EPTP 字段的 bits $N-1:12$ 提供 ($N=MAXPHYADDR$)。例如, 当 $MAXPHYADDR = 36$ 时, EPTP 的 bits 35:12 是 PML4T 地址值, bits 63:36 是保留位。
- bits 2:0 设置 EPT paging structure 所使用的内存类型, 当前 VMX 架构只支持 UC (uncacheable, 值为 0) 与 WB (writeback, 值为 6)。EPTP[2:0]所支持的内存类型可从 IA32_VMX_EPT_VPID_CAP 寄存器的 bit 8 与 bit 14 位查询 (参见 2.5.13 节)。
- bits 5:3 设置 EPT 页表的 walk 长度, 也就是访问 EPT 页表的级数。这个值必须设置为 3 值 (指示需要经过 4 级页表的 walk)。EPT 支持 4 级 EPT 页表可从 IA32_VMX_EPT_VPID_CAP 寄存器 bit 6 查询得到 (参见 2.5.13 节)。
- bit 6 是 EPT 页表项的 dirty 与 accessed 标志开启位。当 IA32_VMX_EPT_VPID_CAP[21] = 1 时, 处理器支持 EPT 的 dirty 与 accessed 标志位 (参见 2.5.13 节)。当 EPTP[6] = 1 时将启用 EPT 的 dirty 以及 accessed 位, EPT 页表项的 bit 8 与 bit 9 被视为 accessed 与 dirty 标志位 (参见 6.1.5 节)。

当“enable EPT”为 1 时, EPTP 值在 VM-entry 时从 EPTP 字段里加载, 记录在处理器内部寄存器中 (推测)。EPTP 字段的设置必须符合设置要求 (参见 4.4.1.3 节)。

注意: EPTP[2:0]所指定的内存类型, 使用在 EPT paging structure (EPT 的页表结构) 数据结构本身中。它属于 VMM 管理的数据区域。它与 EPT paging structure 表项内所指定的内存类型是不同的。

例如, 在 4K 页面下, EPT paging structure 的 PTE[5:3]指定的内存类型使用在 guest-physical address 页面上 (参见 6.1.5 节)。

EP4TA 域

当“enable EPT”为 1 时, 处理器使用 EPT PML4T 地址值 (EPTP 字段的 bits $N-1:12$) 作为标识符, 定义处理器的 cache 域, 用来维护对应的 cache 信息。这个 EPT PML4T 地址被称为“EP4TA”。

注意: 在 Intel 文档中 EP4TA 等于 EPTP 字段的 bits 51:12。但是, 将 bits $N-1:12$ 作为 EP4TA 更为准确合理 (N 值等于 $MAXPHYADDR$)。由于 bits 51: N 属于保留位, 必须为 0 值, 实际上 bits $N-1:12$ 与 bits 51:12 的值是相等的。

6.1.5 4K 页面下的 EPT 页表结构

在 4K 页面下, guest-physical address 被分割为下面的部分。

(1) PML4E index (bits 47:39), 这个 index 值用于在 PML4T 中索引查找 PML4E。

- (2) PDPTE index (bits 38:30), 这个 index 值用于在 PDPT 中索引查找 PDPTE。
- (3) PDE index (bits 29:21), 这个 index 值用于在 PDT 中索引查找 PDE。
- (4) PTE index (bits 20:12), 这个 index 值用于在 PT 中索引查找 PTE。
- (5) Offset (bits 11:0), 这个 offset 值用于在 4K 页面中定位最终的物理地址。

guest-physical address 的 bits 63:48 被忽略 (参见 6.1.3 节)。GPA 转换到 HPA 需要经过 4 级 EPT 页表结构 (walk 次数为 4), 如图 6-4 所示。

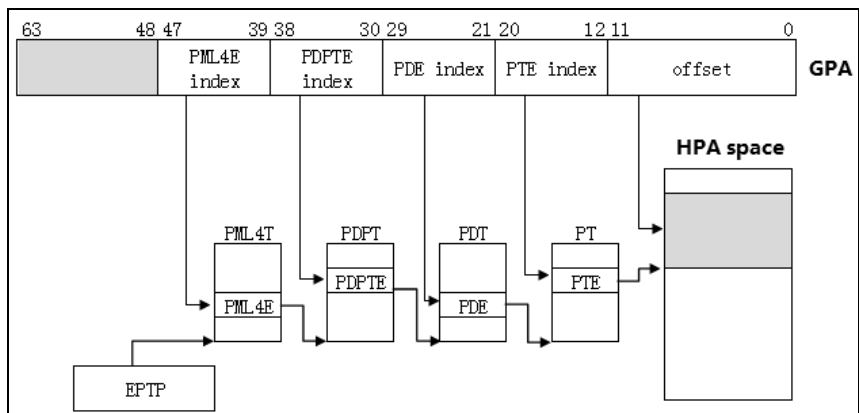


图 6-4

EPTP 字段的 bits $N-1:12$ 提供 PML4T 基址的 bits $N-1:12$ 位, 低 12 位补 0 (对齐在 4K 字节边界)。处理器进行下面的查表转换过程。

(1) 在 $\text{PML4T base} + \text{PML4E index} \times 8$ 中找到 PML4E。PML4E 的 bits $N-1:12$ 提供下一级 PDPT 基址的 bits $N-1:12$ 位, 低 12 位补为 0 值。

(2) 在 $\text{PDPT base} + \text{PDPTE index} \times 8$ 中找到 PDPTE。PDPTE 的 bits $N-1:12$ 提供下一级 PDT 基址的 bits $N-1:12$ 位, 低 12 位补为 0 值。

(3) 在 $\text{PDT base} + \text{PDE index} \times 8$ 中找到 PDE。PDE 的 bits $N-1:12$ 提供下一级 PT 基址的 bits $N-1:12$ 位, 低 12 位补为 0 值。

(4) 在 $\text{PT base} + \text{PTE index} \times 8$ 中找到 PTE。PTE 的 bits $N-1:12$ 提供 4K 页面的 bits $N-1:12$ 位, 低 12 位补为 0 值。

这个 4K 页面基址加上 GPA 的 offset 值 (bits 11:0) 得到最终的 HPA 值, 完成 GPA 的转换。图 6-5 是 4K 页面下的 PML4E, PDPTE, PDE 及 PTE 结构图。

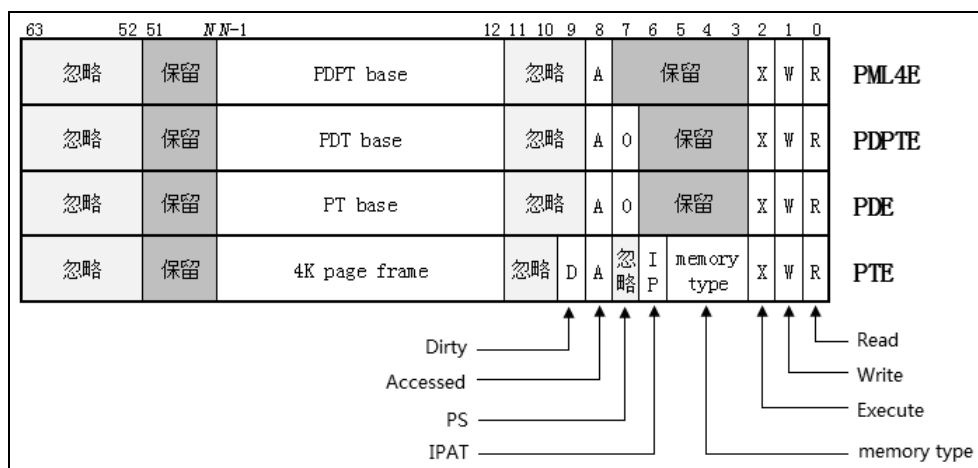


图 6-5

忽略位 (ignored)

下面的位属于忽略位。

- PML4E, PDPTE, PDE 及 PTE 的 Bits 63:52。
- PML4E, PDPTE 与 PDE 的 bits 11:9。
- PTE 的 bits 11:10 与 bit 7。

这些位在 GPA 的转换过程中，处理器忽略不检查。

保留位 (reserved)

下面的位属于保留位。

- PML4E, PDPTE, PDE 及 PTE 的 bits 51: N ($N = \text{MAXPHYADDR}$)。
- PML4E 的 bits 7:3。
- PDPTE 和 PTE 的 bits 6:3。

保留位必须为 0 值。处理器在 GPA 转换过程中，检测到保留位不为 0 值将产生 **EPT misconfiguration** (EPT 配置不当) 从而引发 VM-exit。

由于 x86/x64 体系下最高可实现 52 位的物理地址，而 MAXPHYADDR 值决定了处理器当前支持的物理地址宽度，因此，bits 51: N 为保留位。如果 MAXPHYADDR = 52，那么 bits 51:12 是有效的。

访问权限位 (access rights)

在 EPT 页表结构中，每个 PML4E 管理 512G 的物理空间，每个 PDPTE 管理 1G 的物理空间，每个 PDE 管理 2M 的物理空间，每个 PTE 管理 4K 的物理空间。

每个页表项的 bits 2:0 是访问权限位，指示由该级 EPT 页表项所管理的物理空间具有什么访问权限。包括下面的访问权限。

- **readable** (可读), bit 0 为 1 时页面可读, 否则不可读。
- **writable** (可写), bit 1 为 1 时页面可写, 否则不可写。
- **executable** (可执行), bit 2 为 1 时页面可执行, 否则不可执行。

注意: 当 bits 2:0 为 0 值时, 表示页面是 **not-present** (不存在)。当 access rights 配置不当时, 在 GPA 转换过程中会产生 EPT misconfiguration 而引发 VM-exit (例如, 如果页面是 writable 的, 那么它也必须是 readable, 否则产生 EPT misconfiguration 故障)。

内存类型 (memory type)

bits 5:3 指示 guest-physical address 页面 cache 的内存类型。这个位域只存在于提供 page frame 的表项上 (即 4K 页面下的 PTE, 2M 页面下的 PDE 或者 1G 页面下的 PDPTE)。

这个内存类型值可以为下面的值。

- 0, 指示为 Uncacheable 类型。
- 1, 指示为 WriteCombining 类型。
- 4, 指示为 WriteThrough 类型。
- 5, 指示为 WriteProtected 类型。
- 6, 指示为 WriteBack 类型。

其他的类型值为保留值, 提供保留的类型值将会产生 EPT misconfiguration 而引发 VM-exit。EPT 表项内设置的内存类型使用在 guest-physical address 页面上, 是对 guest-physical address 进行 cache 的类型。

忽略 PAT 位 (ignore PAT)

bit 6 为忽略 PAT 位, 只存在于提供 page frame 的表项上 (即 4K 页面下的 PTE, 2M 页面下的 PDE 或者 1G 页面下的 PDPTE)。当 CR0.CD = 0 时 (cache 有效), 最终的 HPA 页面的内存类型由 **PAT 内存类型** 及 **EPT 内存类型** 联合决定。当 IPAT = 1 时, PAT 内存类型被忽略, 而物理页面的内存类型由 EPT 内存类型决定。

页面尺寸位 (PS, PageSize)

bit 7 指示是否使用 large-page (大页面, 即 1G 或者 2M 页面)。因此, 这个位只存在于 PDPTE 与 PDE 上。在 PML4E 上是保留位, 在 PTE 上是忽略位。

- PDPTE[7] = 1 时, 使用 1G 页面。
- PDE[7] = 1 时, 使用 2M 页面。

在 4K 页面下, PDPTE 与 PDE 的 bit 7 都为 0 值, PTE 的 bit 7 为忽略位。

已访问位 (accessed)

当 EPTP[6] = 1 时, 每个页表项的 bit 8 被视为 “accessed” 标志位。当 EPTP[6] = 0 时, bit 8 为忽略位 (另参考 6.1.4 节)。

当访问到页表项所管理的物理空间时（从另一个角度看，也就是 GPA 转换过程中所 walk 的每一个表项），处理器将 accessed 位置位。accessed 标志位属于“sticky”位，处理器只置位不清位，一旦置位后需要清位，则必须由软件主动进行。

脏位 (dirty)

dirty 位（也可理解为“已写”位）只存在于提供 page frame 的页表项中（即 4K 页面下的 PTE，2M 页面下的 PDE 或者 1G 页面下的 PDPTE）。当 EPTP[6] = 1 时，页表项的 bit 9 被视为 dirty 标志位。当 EPTP[6] = 0 时，bit 9 为忽略位。

当写往 page frame 内的物理地址时，处理器将 dirty 置位。dirty 标志位属于“sticky”位，处理器只置位不清位。一旦置位后需要清位，则必须由软件主动进行。

注意：当 EPTP[6] = 1 时（启用 accessed 与 dirty 标志位），对 guest paging structure 表项的访问被视为“写访问”。

物理基址

在 4K 页面下，每个 EPT 表项的 bits $N-1:12$ ($N = \text{MAXPHYADDR}$) 提供下一级 EPT 页表物理基址，而 PTE 提供 4K 页面物理基址。这些物理地址都属于 HPA (host-physical address)。

- PML4E 由 GPA 的 bits 47:39 来索引查找。从另一个角度看，每个 PML4E 管理 512G 物理地址空间（512G 的 page frame）。GPA 的 bits 47:39 就是这个 512G 页面的 page number。
- PDPTE 由 GPA 的 bits 38:30 来索引查找。从另一个角度看，每个 PDPTE 管理 1G 物理地址空间（1G 的 page frame）。GPA 的 bits 47:30 就是这个 1G 页面的 page number。
- PDE 由 GPA 的 bits 29:21 来索引查找。从另一个角度看，每个 PDE 管理 2M 物理地址空间（2M 的 page frame）。GPA 的 bits 47:21 就是这个 2M 页面的 page number。
- PTE 由 GPA 的 bits 20:12 来索引查找。从另一个角度看，每个 PTE 管理 4K 物理地址空间（4K 的 page frame）。GPA 的 bits 47:12 就是这个 4K 页面的 page number。

6.1.6 2M 页面下的 EPT 页表结构

在 2M 页面下，guest-physical address 被分割为下面几部分。

- PML4E index (bits 47:39)，这个 index 值用于在 PML4T 中索引查找 PML4E。
- PDPTE index (bits 38:30)，这个 index 值用于在 PDPT 中索引查找 PDPTE。
- PDE index (bits 29:21)，这个 index 值用于在 PDT 中索引查找 PDE。
- Offset (bits 20:0)，这个 offset 值用于在 2M 页面内定位最终的物理地址。

guest-physical address 的 bits 63:48 被忽略（参见 6.1.3 节）。GPA 转换到 HPA 需要经过 3 级 EPT 页表结构（walk 次数为 3）。如图 6-6 所示。

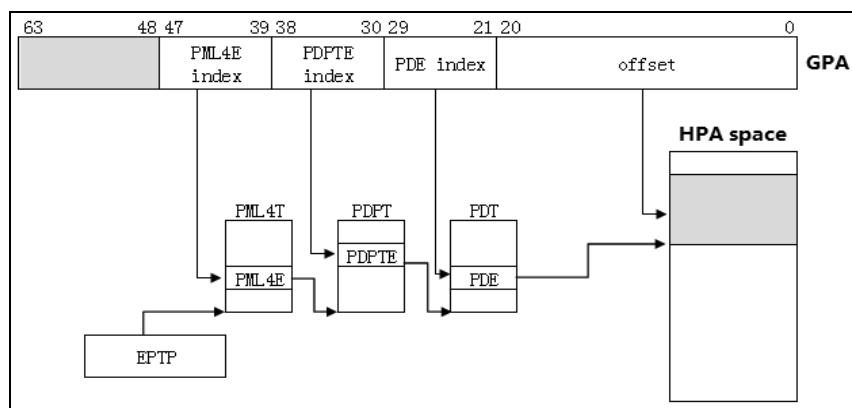


图 6-6

EPTP 字段的 bits $N-1:12$ 提供 PML4T 基址的 bits $N-1:12$ 位，低 12 位补 0（对齐在 4K 字节边界）。处理器进行下面的查表转换过程。

(1) 在 $\text{PML4T base} + \text{PML4E index} \times 8$ 中找到 PML4E。PML4E 的 bits $N-1:12$ 提供下一级 PDPT 基址的 bits $N-1:12$ 位，低 12 位补为 0 值。

(2) 在 $\text{PDPT base} + \text{PDPTE index} \times 8$ 中找到 PDPTE。PDPTE 的 bits $N-1:12$ 提供下一级 PDT 基址的 bits $N-1:12$ 位，低 12 位补为 0 值。

(3) 在 $\text{PDT base} + \text{PDE index} \times 8$ 中找到 PDE。PDE 的 bits $N-1:21$ 提供 2M 页面的 bits $N-1:21$ 位，低 21 位补为 0 值。

最后，由 GPA 的 bits 20:0 作为 offset 值，在 PDE 提供的 2M 页面内定位最终的物理地址，完成 GPA 的转换。

图 6-7 是 2M 页面下 PML4E，PDPTE 及 PDE 的结构图。

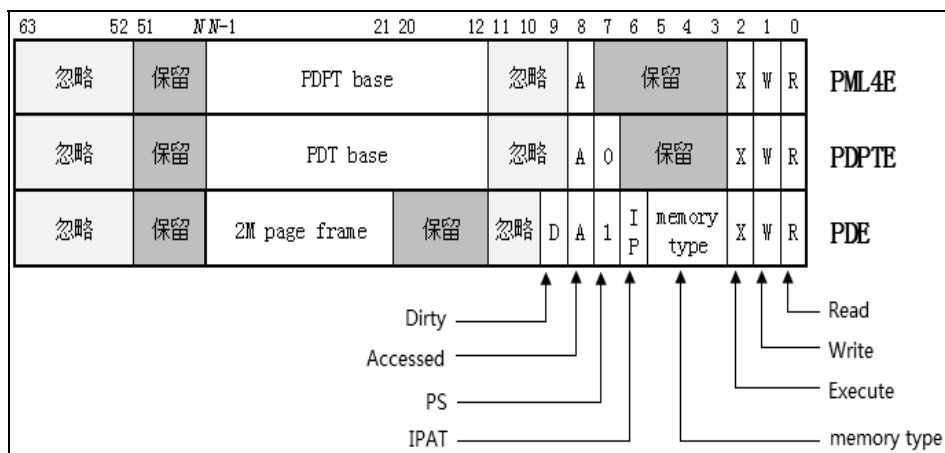


图 6-7

在 2M 页面下，由于 $\text{PDE}[7]$ 为 1，处理器 walk 到 PDE 时结束查表工作，无须继续

walk 下去。此时，PDE 提供最终的 2M page frame 基址。PDE 的结构发生改变。

- Access rights (bits 2:0)，指示 2M 页面的访问权限（需要联合上级页表）。为 0 时指示 2M 页面为 not-present。
- Memory type (bits 5:3)，指示 2M 页面的内存类型（需要联合上级页表及 PAT）。
- IPAT 位 (bit 6)，为 1 时将忽略 PAT 的作用，2M 页面的内存类型由 memory type 提供。
- PS 位 (bit 7)，此时为 1 值。指示使用 2M 页面。
- Accessed 位 (bit 8)，为 1 时指示 2M 页面已经被访问过（当 EPTP[6] = 1 时）。
- Dirty 位 (bit 9)，为 1 时指示 2M 页面已经被写过（当 EPTP[6] = 1 时）。
- Page frame 基址 (bits $N-1:21$)， N 的值等于 MAXPHYADDR。提供 2M 页面的 bits $N-1:21$ 位，低 21 位补为 0 值。
- 忽略位 (bits 63:52 及 bits 11:10)，在 GPA 转换过程中这些位被忽略。
- 保留位 (bits 51: N 及 bits 20:12)， N 的值等于 MAXPHYADDR。保留位必须为 0 值。

6.1.7 1G 页面下的 EPT 页表结构

在 1G 页面下，guest-physical address 被分割为下面几部分。

- PML4E index (bits 47:39)，这个 index 值在 PML4T 中索引查找 PML4E。
- PDPTE index (bits 38:30)，这个 index 值在 PDPT 中索引查找 PDPTE。
- Offset (bits 29:0)，这个 offset 值在 1G 页面内定位最终的物理地址。

guest-physical address 的 bits 63:48 被忽略（参见 6.1.3 节）。GPA 转换到 HPA 需要经过 2 级 EPT 页表结构（walk 次数为 2）。如图 6-8 所示。

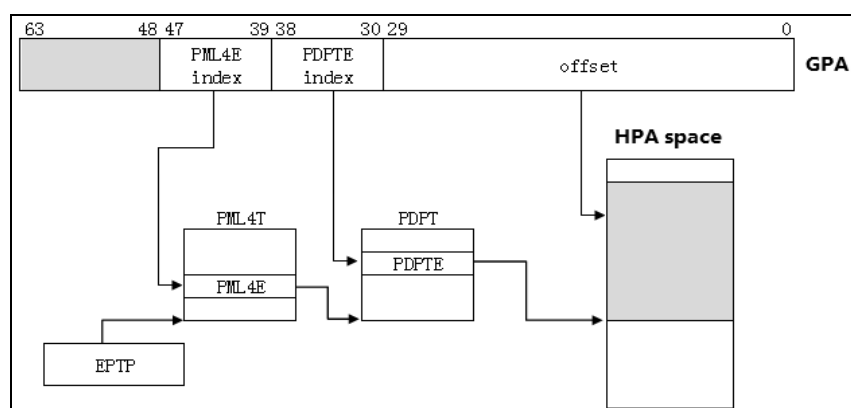


图 6-8

EPTP 字段的 bits $N-1:12$ 提供 PML4T 基址的 bits $N-1:12$ 位，低 12 位补 0（对齐在 4K 字节边界）。处理器进行下面的查表转换过程。

(1) 在 $PML4T\ base + PML4E\ index \times 8$ 中找到 PML4E。PML4E 的 bits $N-1:12$ 提供下一级 PDPT 基址的 bits $N-1:12$ 位，低 12 位补为 0 值。

(2) 在 $PDPT\ base + PDPTE\ index \times 8$ 中找到 PDPTE。PDPTE 的 bits $N-1:30$ 提供最终 1G 页面的 bits $N-1:30$ 位，低 30 位补为 0 值。

最后，由 GPA 的 bits 29:0 作为 offset 值，在 PDPTE 提供的 1G 页面内定位最终的物理地址，完成 GPA 的转换。图 6-9 是 1G 页面下 PML4E 及 PDPTE 的结构图。

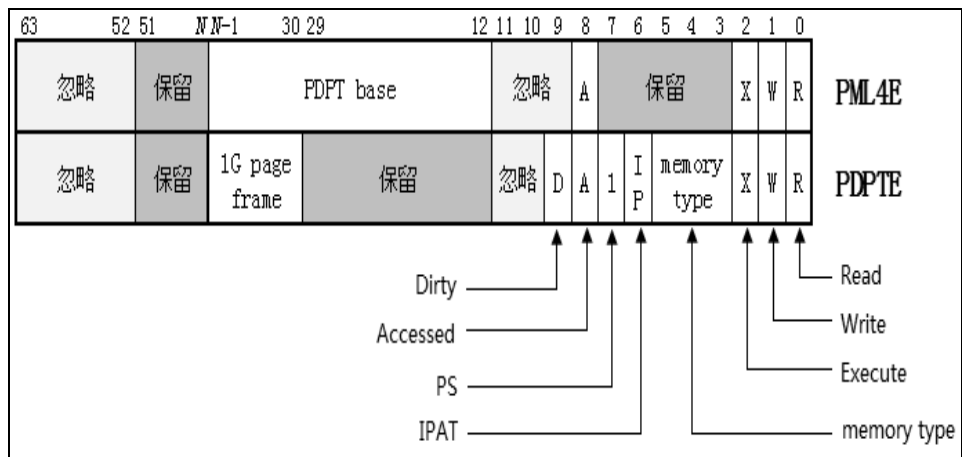


图 6-9

在 1G 页面下，由于 PDPTE[7]为 1，处理器 walk 到 PDPTE 时结束查表工作，无须继续 walk 下去。此时，PDPTE 提供最终的 1G page frame 基址。PDPTE 的结构发生改变。

- access rights (bits 2:0)，指示 1G 页面的访问权限（需要联合上级页表）。为 0 时指示 1G 页面为 not-present。
- memory type (bits 5:3)，指示 1G 页面的内存类型（需要联合上级页表及 PAT）。
- IPAT 位 (bit 6)，为 1 时将忽略 PAT 的作用，1G 页面的内存类型由 memory type 提供。
- PS 位 (bit 7)，此时为 1 值。指示使用 1G 页面。
- accessed 位 (bit 8)，为 1 时指示 1G 页面已经被访问过（当 EPTP[6] = 1 时）。
- dirty 位 (bit 9)，为 1 时指示 1G 页面已经被写过（当 EPTP[6] = 1 时）。
- page frame 基址 (bits $N-1:30$)， N 的值等于 MAXPHYADDR。提供 1G 页面的 bits $N-1:20$ 位，低 30 位补为 0 值。
- 忽略位 (bits 63:52 及 bits 11:10)，在 GPA 转换过程中这些位被忽略。
- 保留位 (bits 51: N 及 bits 29:12)， N 的值等于 MAXPHYADDR。保留位必须为 0 值。

6.1.8 EPT 导致的 VM-exit

在分页机制下，线性地址转换到物理地址的过程中发生错误时，将会引发一个#PF (page fault) 异常。例如，页面属于 not-present，线性地址没有访问权限，或者页表项的保留位不为 0 等。

在 EPT 扩展页表机制下，在 guest 中 GPA (guest-physical address) 转换到 HPA (host-physical address) 的过程中发生错误时，将会引发两类 EPT 故障，被称为“**EPT violation**”及“**EPT misconfiguration**”。作为这两类 EPT 页故障的响应结果，处理器产生 VM-exit。

6.1.8.1 EPT violation

在 guest 访问内存时，由于 guest 对该 guest-physical address (无论是 guest-linear address 转换而来，还是直接访问) 并没有相应的访问权限，从而产生 EPT violation (EPT 违例) 故障。

EPT violation 故障可以发生在下面 GPA 转换 HPA 的环节里。

- 访问的页面属于 not-present。也就是在 walk 过程中，遇到了任何一级 EPT paging structure 表项的 access rights (bits 2:0) 为 0 值 (参见 6.1.5 节)。
- 尝试进行读访问，但目标页面没有可读权限。也就是在 walk 过程中，遇到了任何一级 EPT paging structure 表项的 readable 位 (bit 0) 为 0。
- 尝试进行写访问，但目标页面没有可写权限。也就是在 walk 过程中，遇到了任何一级 EPT paging structure 表项的 writable 位 (bit 1) 为 0。
 - 当 EPTP[6] = 1 时 (启用 EPT paging structure 表项的 accessed 与 dirty 标志位)，处理器对所有 guest paging structure 表项的访问都被视为“写访问”。假如，在对 guest paging structure 表项访问的 GPA 转换过程中，遇到 writable 位为 0 时则产生 EPT violation。
- 尝试执行代码时 (即 fetch 代码)，目标页面没有可执行权限。也就是在 walk 过程中，遇到了任何一级 EPT paging structure 表项的 executable 位 (bit 2) 为 0。

VM-exit 记录信息

由 EPT violation 引发 VM-exit 时，exit qualification 字段记录 EPT violation 的明细信息 (参见 3.10.1.14 节表 3-32)。

从 exit qualification 字段中，我们可以得到下面 GPA 转换的相关信息。

- (1) 访问操作 (access)，bits 2:0 指示发生 EPT violation 时，guest 进行什么访问操作。
- (2) 访问权限 (access rights)，bits 5:3 指示该 guest-physical address 具有什么访问权限。
- (3) EPT violation 发生的位置。

- 当 bit 7 为 1 时，表明 GPA 是来自对 guest-linear address 的转换。此时，bit 8 指示 EPT violation 发生的具体位置：
 - Bit 8 为 0 时，表明 EPT violation 发生在访问 guest paging structure 表项环节上。
 - Bit 8 为 1 时，表明 EPT violation 发生在 GPA 转换为 HPA 阶段。也就是已经通过 guest paging structure 转换而来的 GPA 值。
- 当 bit 7 为 0 时，表明 GPA 并不是由 guest-linear address 转换而来的（例如，执行 MOV to CR3 指令导致 PDPTE 的加载）。

当 exit qualification 字段的 bit 7 为 1 时（存在 guest-linear address），guest-linear address 字段将记录发生 EPT violation 时的 guest-linear address 值。bit 7 为 0 时，guest-linear-address 字段为未定义值（参见 3.10.1.17 节）。guest-physical address 字段记录发生 EPT violation 时的 guest-physical address 值（参见 3.10.1.18 节）。

6.1.8.2 EPT misconfiguration

由于 EPT paging structure 表项的内容配置不当，从而产生 EPT misconfiguration 导致 VM-exit 发生。当 EPT paging structure 表项属于“**present**”（即 bits 2:0 不为 0 值），并且内容配置为下面情况之一时，属于 EPT misconfiguration 故障。

(1) access rights (bits 2:0) 的值为 **010B** (write-only) 或者 **110B** (write/execute)。也就是两种访问权限的设置必须要附加可读权限（即 011B—read/write，或者 111B—read/write/execute）。

(2) 当 IA32_VMX_EPT_VPID_CAP[0] = 0 时，access rights (bits 2:0) 的值为 **100B** (execute-only)。IA32_VMX_EPT_VPID_CAP 寄存器的 bit 0 指示处理器是否支持 EPT 的 execute-only 访问权限（参见 2.5.13 节）。

(3) 保留位不为 0 值，包括下面（参见 6.1.5 节，6.1.6 节及 6.1.7 节）。

- 4K 页面下：
 - PML4E, PDPTE, PDE 及 PTE 的 bits 51:N ($N = \text{MAXPHYADDR}$)。
 - PML4E 的 bits 7:3。
 - PDPTE 及 PTE 的 bits 6:3。
- 2M 页面下：
 - PML4E, PDPTE 及 PDE 的 bits 51:N ($N = \text{MAXPHYADDR}$)。
 - PML4E 的 bits 7:3。
 - PDPTE 的 bits 6:3。
 - PDE 的 bits 20:12。
- 1G 页面下：
 - PML4E 及 PDPTE 的 bits 51:N ($N = \text{MAXPHYADDR}$)。
 - PML4E 的 bits 7:3。
 - PDPTE 的 bits 29:12。

(4) memory type (bits 5:3) 的值属于保留值。EPT 表项里可以使用下面的内存类型值，其余值为保留值。

- 0 (UC, Uncacheable)。
- 1 (WC, WriteCombining)。
- 4 (WT, WriteThrough)。
- 5 (WP, WriteProtected)。
- 6 (WB, WriteBack)。

注意：发生 EPT misconfiguration 故障的前提是 EPT paging structure 表项是“present”的。如果表项属于“not-present”，则发生 EPT violation 故障。

VM-exit 记录信息

对于 EPT misconfiguration 故障，并不会记录明细信息，exit qualification 字段为未定义值。在 guest-physical address 字段中会记录发生 EPT misconfiguration 时的 guest-physical address 地址值（参见 3.10.1.18 节）。

6.1.8.3 EPT 页故障的优先级

与#PF 异常统一接管分页机制下的页故障不同，EPT 页表可能产生两类 EPT 页故障：**EPT violation** 与 **EPT misconfiguration**。因此，一个 EPT paging structure 表项可能既符合 EPT violation，也符合 EPT misconfiguration 的产生条件。

确定 EPT 页故障类型

在 guest-physical address 转换过程中，处理器需要按照一定的规则来确定是产生 EPT violation 还是 EPT misconfiguration 故障，如图 6-10 所示。

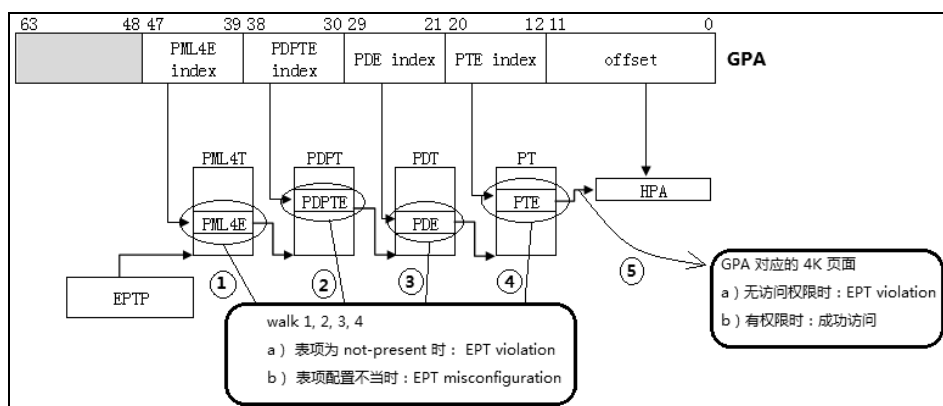


图 6-10

在图 6-10 中，以 EPT paging structure 使用 4K 页面为例，揭示了在 GPA 过程的哪个环节上、如何产生 EPT 页故障。

(1) 在 EPT 页表 walk 的过程中，处理器检查 EPT paging structure 表项的内容。

- 发现表项属于“**not-present**”时（即 bits 2:0 为 0 值），则产生 EPT violation 故障。
- 在属于“**present**”的情况下，发现表项内容设置不当（参见 6.1.8.2 节），则产生 EPT misconfiguration 故障。

(2) 在 4K 页面下，处理器进行了 4 步 walk 流程，相继检查 **PML4E**，**PDPTE**，**PDE** 及 **PTE**。每个表项都按照上面的规则来确定 EPT violation 与 EPT misconfiguration。

(3) 最后，由 PTE 得到最终的 4K 页面物理地址，并且这个 4K 页面的访问权限已经被确定（也就是，**PML4E**[2:0]，**PDPTE**[2:0]，**PDE**[2:0]以及 **PTE**[2:0]进行“**AND**”操作后的结果）。处理器基于这个 4K 页面的访问权限来判断 GPA 是否有权访问（图 6-10 中第 5 步）。

- GPA 无权限访问时，产生 EPT violation 故障。
- GPA 有权限时，成功访问物理地址。

很显然，EPT 表项**不可能**同时产生 EPT violation 与 EPT misconfiguration 故障。处理器首先检查 EPT paging structure 表项是否为“present”来确定是否产生 EPT violation 故障，然后再检查是否产生 EPT misconfiguration 故障。这个步骤相继在各级表项中进行。

当最后一级页表项 PTE 检查通过后，处理器已经确定了最终 page frame 的访问权限。也就是由各级表项的 access rights 进行“AND”操作后得出的值，就是最终 page frame 具有的访问权限。最后，处理器会检查 GPA 对该 page frame 是否具有访问权限。无访问权限时产生 EPT violation 故障。

举一反三，当 EPT paging structure 使用 2M 页面时，处理器按照上面所述的规则相继检查 **PML4E**，**PDPTE** 及 **PDE**，以确定 EPT 页故障的类型（没有 PTE 需要检查）。最后，处理器检查 GPA 对 PDE 得到的 2M 页面是否具有访问权限。

同理，当 EPT paging structure 使用 1G 页面时，处理器按照上述规则相继检查 **PML4E** 及 **PDPTE** 以确定 EPT 页故障的类型（没有 PDE 与 PTE 需要检查）。最后，处理器检查 GPA 对 **PDPTE** 里得到的 1G 页面是否具有访问权限。

EPT 页故障与#PF 异常

当 guest 开启分页机制时，在 guest-linear address 转换为 guest-physical address 的过程中可能会出现 EPT 页故障与#PF 异常。

#PF 异常属于 guest 管辖范围，由 guest 进行处理（由#PF 异常而导致 VM-exit 除外）。而 EPT 页故障则属于 host（VMM）管辖范围，由 host 处理。

在图 6-11 中，以 guest 使用 IA-32e 分页模式的 4K 页面为例，揭示了在 guest-linear address 转换 guest-physical address 的过程中，EPT 页故障与#PF 异常之间的处理次序。

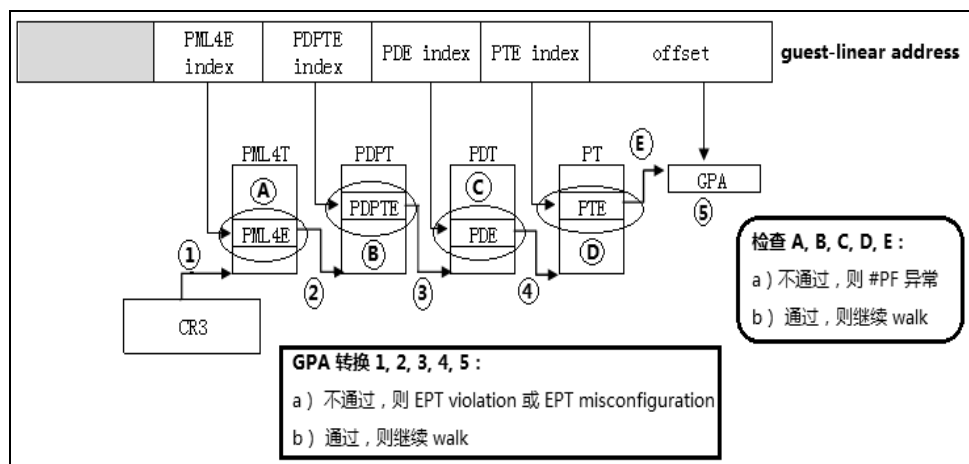


图 6-11

参见 6.1.1.1 节的描述及图 6-2 所示, 以 guest 分页机制使用 4K 页面为例, guest-linear address 的转换需要经过 5 个环节的 GPA 转换 (图 6-11 或图 6-2 中的 1, 2, 3, 4 及 5 点所示), 以及处理器检查 guest paging structure 的 5 个环节 (图 6-11 中的 A, B, C, D 及 E 点)。

(1) 处理器对 CR3 指向的 PML4T 基址进行 GPA 转换 (图 6-11 中第 1 点)。

- 发生错误时, 按照前面图 6-10 所示的规则来确定 EPT 页故障类型 (EPT violation 还是 EPT misconfiguration)。
- 当通过 GPA 转换后, 处理器找到 guest paging structure 的 PML4E。

(2) 处理器对 guest paing structure 的 PML4E 进行检查 (图 6-11 中 A 点所示)。

- 不通过时, 产生 #PF 异常。guest 进行处理或者引发 VM-exit。
- 通过时, 处理器从 PML4E 里得到 PDPT 基址, 继续 walk 下去。

(3) 处理器对得到的 PDPT 基址进行 GPA 转换 (图 6-11 中第 2 点)。

- 发生错误时, 按照前面图 6-10 所示的规则来确定 EPT 页故障类型 (EPT violation 还是 EPT misconfiguration)。
- 当通过 GPA 转换后, 处理器继续 walk 下去, 找到 guest paging structure 的 PDPTE。

(4) 处理器对 guest paging structure 的 PDPTE 进行检查 (图 6-11 中 B 点所示)。

- 不通过时, 产生 #PF 异常。guest 进行处理或者引发 VM-exit。
- 通过时, 处理器从 PDPTE 里得到 PDT 基址, 继续 walk 下去。

(5) 处理器对得到的 PDT 基址进行 GPA 转换 (图 6-11 中第 3 点)。

- 发生错误时, 按照前面图 6-10 所示的规则来确定 EPT 页故障类型 (EPT violation 还是 EPT misconfiguration)。

- 当通过 GPA 转换后，处理器继续 walk 下去，找到 guest paging structure 的 PDE。
- (6) 处理器对 guest paging structure 的 PDE 进行检查（图 6-11 中 C 点所示）。
- 不通过时，产生#PF 异常。guest 进行处理或者引发 VM-exit。
 - 通过时，处理器从 PDE 里得到 PT 基址，继续 walk 下去。
- (7) 处理器对得到的 PT 基址进行 GPA 转换（图 6-11 中第 4 点）。
- 发生错误时，按照前面图 6-10 所示的规则来确定 EPT 页故障类型（EPT violation 还是 EPT misconfiguration）。
 - 当通过 GPA 转换后，处理器继续 walk 下去，找到 guest paging structure 的 PTE。
- (8) 处理器对 guest paging structure 的 PTE 进行检查（图 6-11 中 D 点所示）。
- 不通过时，产生#PF 异常。guest 进行处理或者引发 VM-exit。
 - 通过时，处理器从 PTE 里得到 4K 页面基址，处理器停止 walk。
- (9) 处理器检查 guest-linear address 是否对 4K 页面具有访问权限（图 6-11 中 E 点所示）。
- 有权限时，检查通过。4K 页面基址加上 guest-linear address 的 offset 值（bits 11:0）得到 guest-physical address 值。
 - 无访问权限时，产生#PF 异常。guest 进行处理或引发 VM-exit。
- (10) 处理器对从 guest-linear address 转换得到的 GPA 进行转换（图 6-11 中第 5 点）。
- 发生错误时，按照前面图 6-10 所示的规则来确定 EPT 页故障类型（EPT violation 还是 EPT misconfiguration）。
 - 当通过 GPA 转换后，guest 成功通过 guest-linear address 访问内存。
- 上面是 guest 在使用 4K 页面时，通过线性地址访问内存的 10 个步骤。但是，在 EPT TLB cache 与 TLB cache 联合管理下，可以减少这些步骤来提高性能。

对于图 6-11 中所举的例子，当发生 EPT violation 时，exit qualification 字段的 bit 7 为 1，指示存在 guest-linear address。exit qualification 字段的 bit 8 由下面的情形决定。

- 在第 1，2，3 或者 4 点的 guest paging structure 访问环节上发生 EPT violation 时，exit qualification 字段的 bit 8 为 0。
- 在第 5 点环节上发生 EPT violation 时，exit qualification 字段的 bit 8 为 1。

6.1.8.4 修复 EPT 页故障

在发生 EPT 页故障而导致 VM-exit 时，VMM 需要根据情况来决定是否修复故障。VMM 修复故障后，使用 VMRESUME 指令重新进入 guest 继续执行。

(1) 发生 EPT violation 故障时。

- 如果 EPT paging structure 表项出现“not-present”情况，VMM 可以选择为 guest 分配一块新的物理空间（HPA），将 GPA 进行重新映射。

- 如果 GPA 访问权限不足，VMM 可以选择为 GPA 添加相应的访问权限。例如，guest 需要执行某段区域的代码，但这段区域没有“可执行”的权限。那么，VMM 需要在这段区域的 GPA 所对应的 EPT paging structure 表项里添加可执行权限。

(2) 发生 EPT misconfiguration 故障时。VMM 根据情况来重新设置 EPT paging structure 表项的 access rights (bits 2:0)，memory type (bits 5:3) 值或者清所有保留位。

思考一下：在使用 EPT 机制时，VMM 负责为每个 guest (VM) 分配独立的内存空间 (host-physical address)。那么，应该为每个 guest 分配多大的内存空间呢？显然，这要视虚拟机平台的设计及物理平台有多少内存而定。

当物理平台上的内存已经分配完毕，而 guest 出现“not-present”的 EPT violation 故障时，VMM 不能满足 guest 的分配要求。如何处理内存不足的情况是 guest OS 的责任。

另一方面，VMM 可以为每个 GPA 映射使用全部的访问权限 (read/write/execute)。因为在 guest 运行环境中，哪个页面具有什么样的访问权限，是 guest OS 的职责，属于 guest OS 内存管理范畴。

代码片段 6-1:

```

;-----
; DoEptViolation()
; input:
;     esi - do process code
; output:
;     eax - VMM process code
; 描述:
;     1) 处理由 EPT violation 引发的 VM-exit
;-----
DoEptViolaton:
    push ebp
    push edx
    push ebx
    push ecx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

;;
;; 读取发生 EPT violation 的 guest-physical address 值
;;
    REX.Wrxb
    mov ebx, [ebp + PCB.ExitInfoBuf + EXIT_INFO.GuestPhysicalAddress]

;;
;; 检查页面是否属于 not-present
;; 1) 如果属于 not-present, 则分配物理页面, 进行重新映射
;; 2) 如果无访问权限, 修复映射
;;
    mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]

```

```

    test eax, (EPT_READ | EPT_WRITE | EPT_EXECUTE) << 3      ;
ExitQualification[5:3]
    jz DoEptViolaton.@0

    DEBUG_RECORD        "[DoEptViolation]: fixing access !"

    ;;
    ;; 下面修复由于访问权限引起的 EPT violation 问题
    ;;
#ifdef __X64
    REX.Wrxb
    mov esi, ebx                      ; guest-physical
address
    and eax, 07h                      ; 访问类型
    or eax, FIX_ACCESS                ; 进行 FIX_ACCESS 操作
#else
    xor edi, edi
    mov esi, ebx
    mov ecx, eax
    and ecx, 07h
    or ecx, FIX_ACCESS
#endif
    jmp DoEptViolation.DoMapping

DoEptViolaton.@0:
    ;;
    ;; 从处理器 domain 中分配一个 4K 物理页面
    ;;
    mov esi, 1
    call vm_alloc_pool_physical_page

    REX.Wrxb
    test eax, eax
    jz DoEptViolation.done

DoEptViolaton.remapping:

    DEBUG_RECORD        "[DoEptViolation]: remaping! (eax = HPA, ebx = GPA)"

    ;;
    ;; 下面进行 guest-physical address 映射
    ;; 注意: 这里添加所有访问权限, read/write/execute
    ;; 1) 因为 guest-physical address 可能会进行多种访问, 需要多种权限
    ;;
#ifdef __X64
    ;;
    ;; rsi - guest-physical address
    ;; rdi - host-physical address
    ;; eax - page attribute
    ;;
    REX.Wrxb
    mov esi, ebx                      ; guest-physical address
    REX.Wrxb
    mov edi, eax                      ; host-physical address
    mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]
    or eax, EPT_FIXING | FIX_ACCESS | EPT_READ | EPT_WRITE | EPT_EXECUTE
#else
    ;;

```

```

;; edi:esi - guest-physical address
;; edx:eax - host-physical address
;; ecx      - page attribute
;;
xor edi, edi
xor edx, edx
mov esi, ebx                ; guest-physical address
mov ecx, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]
or ecx, EPT_FIXING | FIX_ACCESS | EPT_READ | EPT_WRITE | EPT_EXECUTE
%endif

DoEptViolation.DoMapping:
;;
;; 执行 guest-physical address 映射工作
;;
call do_guest_physical_address_mapping

;;
;; ### 刷新 cache ###
;;

;;
;; INVEPT 描述符
;;
mov DWORD [ebp + PCB.InvDesc + INV_DESC.Dword1], 0
mov DWORD [ebp + PCB.InvDesc + INV_DESC.Dword2], 0
mov DWORD [ebp + PCB.InvDesc + INV_DESC.Dword3], 0

%ifdef __X64

    GetVmcsField    CONTROL_EPT_POINTER_FULL
    REX.Wrxb
    mov [ebp + PCB.InvDesc + INV_DESC.Eptp], eax
%else
    GetVmcsField    CONTROL_EPT_POINTER_FULL
    mov [ebp + PCB.InvDesc + INV_DESC.Eptp], eax
    GetVmcsField    CONTROL_EPT_POINTER_HIGH
    mov [ebp + PCB.InvDesc + INV_DESC.Eptp + 4], eax
%endif

;;
;; 使用 single-context invalidation 刷新方式
;;
mov eax, SINGLE_CONTEXT_INVALIDATION
invept eax, [ebp + PCB.InvDesc]

mov eax, VMM_PROCESS_RESUME

DoEptViolation.done:
pop ecx
pop ebx
pop edx
pop ebp
ret

```

代码片段 6-1 的 DoEptViolation 处理函数（位于“\lib\VMX\VmxEPT.asm”文件

中) 执行简单的任务。

(1) 从 guest-physical address 字段中读取发生 EPT violation 的 GPA 地址值。

(2) 检查引起 EPT violation 是由于 “not-present” 还是访问权限不足。

(3) 如果是由于访问权限不足, 则修复访问权限。也就是添加相应的访问权限。

(4) 如果是由于 “not-present” 而引起的, 则从 guest 的内存域 (domain) 中分配一块 4K 区域。

(5) 使用 do_guest_physical_address_mapping 函数将这个 GPA 映射到分配的 4K 区域里。

(6) 返回 VMM, 让 VMM 执行 VMRESUME 操作。

注意: do_guest_physical_address_mapping 函数使用 **FIX_ACCESS** 参数来修复 EPT violation 故障 (64 位下是 EAX 寄存器, 32 位下是 ECX 寄存器)。如果由于访问权限不足而引起 EPT violation, 我们在 EPT paging structure 表项中添加了相应的访问权限后, VMM 必须使用 INVEPT 指令来刷新当前 EP4TA 下的 guest-physical mapping 与 combined mapping 相应的 cache 信息 (参见 6.2.6 节)。

在修复 “not-present” 引起的 EPT violation 故障时, 我们重新从处理器的 domain 中分配 4K 物理页面, 并使用所有的访问权限 (read/write/execute) 来进行映射。

代码片段 6-2:

```

;-----
; DoEptMisconfiguration()
; input:
;     none
; output:
;     eax - VMM process code
; 描述:
;     1) 处理由 EPT misconfiguration 引发的 VM-exit
;-----
DoEptMisconfiguration:
    push ebp
    push ecx
    push edx

    ;;
    ;; 发生 EPT misconfiguration 时, 进行修复工作
    ;;

    REX.Wrxb
    mov esi, [ebp + PCB.ExitInfoBuf + EXIT_INFO.GuestPhysicalAddress]

    DEBUG_RECORD        "[DoEptMisconfiguration]: fixing !"

    ;;
    ;; 下面进行修复
    ;;
#endif __X64

```



```

;;
;; rsi - guest-physical address
;; rdi - host-physical address
;; eax - page attribute
;;
REX.Wrb
mov edi, esi
mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]
or eax, FIX_MISCONF
%else
;;
;; edi:esi - guest-physical address
;; edx:eax - host-physical address
;; ecx    - page attribute
;;
xor edi, edi
xor edx, edx
mov eax, esi
mov ecx, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]
or ecx, FIX_MISCONF
%endif
call do_guest_physical_address_mapping

mov eax, VMM_PROCESS_RESUME

DoEptMisconfiguration.done:
pop edx
pop ecx
pop ebp
ret

```

代码片段 6-2 的 DoEptMisconfiguration 处理函数执行更简单的任务。

- (1) 从 guest-physical address 字段中读取发生 EPT misconfiguration 的 GPA 地址值。
- (2) 使用 do_guest_physical_address_mapping 函数执行 GPA 重新映射。
- (3) 返回 VMM，让 VMM 执行 VMRESUME 操作。

注意：这里不需要从 guest domain 里分配内存区域（由于 EPT misconfiguration 不是 “not-present” 引起）。do_guest_physical_address_mapping 函数使用的 page attribute 参数（64 位下是 EAX 寄存器，32 位下是 ECX 寄存器）是 **FIX_MISCONF**，它指示 do_guest_physical_address_mapping 进行 EPT misconfiguration 修复工作。

6.1.9 accessed 与 dirty 标志位

根据前面所述，有两类 accessed 与 dirty 标志位。

- guest paging structure 表项内的 accessed 与 dirty 标志位（也就是 x86/x64 体系的保护模式分页机制下的页表项）。
- EPT paging structure 表项内的 accessed 与 dirty 标志位。

在 EPTP[6] = 1 时（需要处理器支持，参见 2.5.13 节），EPT paging structure 表项启用 accessed 与 dirty 标志位。同时，处理器对所有 guest paging structure 表项的访问都被认

为是“写访问”（参见 6.1.8.1 节）。因此，对 guest paging structure 表项访问时的 GPA 转换需要有写访问的权限（参见 6.1.8.3 节图 6-11 中的第 1, 2, 3 及 4 点）。

如果 guest 使用 PAE 分页模式，执行指令 MOV to CR3 更新 CR3 寄存器时（也包括更新 CR0 或 CR4 寄存器的某些控制位），会引起“CR3 指向的 PDPTE 表项”加载。注意，由这个加载操作所引起的 GPA 转换不需要有写访问权限。

更新 guest paging structure 表项的 accessed 与 dirty 位

处理器在 guest paging structure 页表中进行 walk 时，根据情况会更新 guest paging structure 表项的 accessed 与 dirty 标志位。以 6.1.8.3 节图 6-11 中所描述的 IA-32e 分页模式 4K 页面为例，有下面的更新操作。

(1) 处理器相继分别 walk 到 guest paging structure 的 PML4E, PDPTE, PDE 及 PTE（图 6-11 中的 A, B, C 及 D 点），分别检查这些表项是否为“present”及保留位是否为 0。在任何一级表项的检查不通过时，处理器停止 walk，产生#PF 异常。

(2) 处理器检查合并后的 page frame（4K 页面）访问权限（即 PML4E, PDPTE, PDE 及 PTE 访问权限进行“AND”操作，对于 NX 位是“OR”操作）。Guest-linear address 对这个 page frame 没有访问权限时，产生#PF 异常（图 6-11 中的 E 点）。

(3) 当 guest-linear address 对这个 page frame 的访问权限检查通过后，处理器将置 PML4E, PDPTE, PDE 及 PTE 的“accessed”标志位（之前设置位时）。

(4) 当 guest-linear address 对这个 page frame 进行写访问时，处理器将置 PTE 的“dirty”标志位（之前设置位时）。

举一反三，当 guest paging structure 使用 2M 页面时，处理器相继分别 walk 到 PML4E, PDPTE 及 PDE。对这些表项进行“present”与保留位的检查，并对 2M page frame 进行访问权限检查。通过后置 PML4E, PDPTE 及 PDE 的 accessed 标志位（之前设置位时）。guest 进行写访问则置 PDE 的 dirty 标志位（之前设置位时）。

当 guest paging structure 使用 1G 页面时，处理器相继分别 walk 到 PML4E 及 PDPTE。对这些表项进行“present”与保留位的检查，并对 1G page frame 进行访问权限检查。通过后置 PML4E 及 PDPTE 的 accessed 标志位（之前设置位时）。guest 进行写访问则置 PDPTE 的 dirty 标志位（之前设置位时）。

更新 EPT paging structure 表项的 accessed 与 dirty 位

处理器在 EPT paging structure 页表中进行 walk 时，根据情况会更新 EPT paging structure 表项的 accessed 与 dirty 标志位。以 6.1.8.3 节图 6-10 中所描述的 EPT paging structure 使用 4K 页面为例，有下面的更新操作。

(1) 处理器相继分别 walk 到 EPT paging structure 的 PML4E, PDPTE, PDE 及 PTE（图 6-10 中的第 1, 2, 3 及 4 点），在任何一级表项中发生 EPT 页故障（EPT violation 或者 EPT misconfiguration，参见 6.1.8 节）时停止 walk。

(2) 处理器检查合并后的 page frame (4K 页面) 访问权限 (即 PML4E, PDPTE, PDE 及 PTE 访问权限进行“AND”操作)。guest-physical address 对这个 page frame 没有访问权限时, 将产生 EPT violation 故障 (图 6-10 中的第 5 点)。

(3) 当 guest-physical address 对这个 page frame 的访问权限通过后, 处理器分别置 PML4E, PDPTE, PDE 及 PTE 的“accessed”标志位 (之前设置位时)。

(4) 当 guest-physical address 对这个 page frame 进行写访问时, 处理器置 PTE 的“dirty”标志位 (之前设置位时)。

举一反三, 当 EPT paging structure 使用 2M 页面时, 处理器相继分别 walk 到 PML4E, PDPTE 及 PDE, 检查这些表项是否发生 EPT 页故障, 并对 page frame 进行访问权限检查。检查通过后, 处理器置 PML4E, PDPTE 及 PDE 的 accessed 标志位 (之前设置位时), 如果属于写访问, 则置 PDE 的 dirty 标志位 (之前设置位时)。

当 EPT paging structure 使用 1G 页面时, 处理器相继分别 walk 到 PML4E 及 PDPTE, 检查这些表项是否发生 EPT 页故障, 并对 page frame 的访问权限进行检查。检查通过后, 处理器置 PML4E 及 PDPTE 的 accessed 标志位 (之前设置位时), 如果属于写访问, 则置 PDPTE 的 dirty 标志位 (之前设置位时)。

6.1.10 EPT 内存类型

在 VMX 架构中, 可以为两类区域指定 cache 的内存类型:

(1) VMM 所管理的区域。例如 VMCS 区域及 EPT paging structure 数据结构属于 VMM 所管理的区域。

(2) guest 使用的区域。例如 guest-physical address 页面。

这两类区域设置 cache 内存类型的方法不同。对于 VMM 所管理的区域 (VMCS 与 EPT paging structure), 当前 VMX 架构下只支持两种内存类型: UC (Uncacheable) 及 WB (WriteBack)。分别由 IA32_VMX_BASIC[53:50]以及 IA32_VMX_EPT_VPID_CAP 寄存器的 bit 8 与 bit 14 查询得到 (参见 2.5.4 节及 2.5.13 节)。

对于 guest-physical address 页面类型, 可支持的内存类型为: UC, WC (WriteCombining), WT (WriteThrough), WP (WriteProtected) 及 WB。

EPT paging structure 内存类型

EPTP 字段的 bits 2:0 设置 EPT paging structure 的内存类型 (参见 6.1.4 节), 但也受到 CR0.CD 控制位的影响。

- 当 CR0.CD = 0 时, 由 EPTP[2:0]设置的内存类型就是 EPT paging structure 的内存类型。
- 当 CR0.CD = 1 时, EPT paging structure 的内存类型只能为 UC。

MTRR 寄存器不能影响 EPT paging structure 的内存类型的设置。

guest-physical address 页面内存类型

guest-physical address 页面的有效内存类型由 EPT 内存类型及 PAT 内存类型决定，也受到 CR0.CD 控制位的影响。

- **PAT 内存类型**，由 guest paging structure 表项内的 PAT，PCD 及 PWT 位决定。
 - 当 CR0.PG = 0 时，PAT 内存类型固定为 WB。
 - 当 CR0.PG = 1 时，PAT 内存类型依据 PAT，PCD 及 PWT 位组成的值在 IA32_PAT 寄存器中索引选择（详见《x86/x64 体系探索及编程》第 335 页 11.7 节描述）。
- **EPT 内存类型**，由 EPT paging structure 内提供 page frame 基址的表项 bits 5:3 提供。

当 CR0.CD = 1 时，guest-physical address 页面的有效内存类型只能是 UC。

当 CR0.CD = 0 时，EPT paging structure 内提供 page frame 基址的表项 IPAT 位 (bit 6) 决定是否忽略 PAT 内存类型（参见 6.1.5 节）。

- 当 IPAT = 1 时，忽略 PAT 内存类型。那么 guest-physical address 页面有效内存类型就是 EPT 内存类型。
- 当 IPAT = 0 时，guest-physical address 页面有效内存类型是 PAT 内存类型与 EPT 内存类型的组合（如表 6-1 所示）。

表 6-1

EPT 内存类型	PAT 内存类型	GPA 页面有效内存类型
UC	UC	UC
	UC-	UC
	WC	WC
	WT	UC
	WB	UC
	WP	UC
WC	UC	UC
	UC-	WC
	WC	WC
	WT	UC
	WB	WC
	WP	UC
WT	UC	UC
	UC-	UC
	WC	WC
	WT	WT
	WB	WT

续表

WT	WP	WP
WP	UC	UC
	UC-	WC
	WC	WC
	WT	WT
	WB	WP
	WP	WP
WB	UC	UC
	UC-	UC
WB	WC	WC
	WT	WT
	WB	WB
	WP	WP

MTRR 寄存器不影响 guest-physical address 页面的有效内存类型。

6.1.11 EPTP switching

VMX 架构提供了可以在 VMX non-root operation 模式内调用 VM function 的机制，使 guest 软件执行 VMFUNC 指令而不会产生 VM-exit。

开启 VM functions 机制

VMM 设置 secondary processor-based VM-execution control 字段的“enable VM functions”位为 1 时，表明允许在 VMX non-root operation 模式内执行 VMFUNC 指令，否则将产生#UD 异常。

此时，由 VM-functions control 字段的控制位来开启或关闭相应的 VM 功能，每一个控制位对应一个 VM 功能。例如，VM-functions control 字段的 bit 0 为 1 时，表明允许调用 0 号 VM 功能。当 bit 0 为 0 时，调用 0 号 VM 功能则产生 VM-exit。

EPTP-switching 功能

当前 VMX 架构只实现了 0 号 VM 功能“**EPTP switching**”（EPTP 切换），这个功能支持在 VMX non-root operation 模式内完成 EPTP 的切换。

使用前，VMM 设置 VM-funcitons control 字段的 bit 0 为 1，开启“EPTP switching”功能。guest 类似下面的代码示例，执行 VMFUNC 指令调用“EPTP switching”功能。

```
mov eax, 0          ; 0 号 VM 功能
mov ecx, 0          ; EPTP-list index
vmfunc
```

EAX 寄存器提供 VM 功能号 0，表明调用“EPTP switching”功能。EAX 寄存器的

值记为 N ，执行 VMFUNC 指令由于这个 N 值可能产生下面的情况。

- 如果 $N > 63$ ，则产生 #UD 异常。
- 如果 VM-functions control 字段的 bit N 为 0，则产生 VM-exit。

当产生 VM-exit 时，exit reason 字段记录的值为 59，指示“VM-exit 由 VMFUNC 指令引发”，VM-exit instruction length 字段记录 VMFUNC 指令的长度。

在 EPTP switching 功能中，ECX 寄存器提供的是 EPTP-list 表项的索引值（关于 EPTP-list 结构参见 3.5.21 节），如果 ECX 寄存器的值大于 511，则产生 VM-exit。

EPTP switching 功能允许在 VMX non-root operation 模式内切换 EPTP 值，从而可以使用不同 EPT 页表结构进行 guest-physical address 的转换。在执行 VMFUNC 指令前，VMM 必须提前准备好 EPTP-list 列表。EPTP-list 列表的 64 位物理地址提供在 EPTP-list address 字段中（参见 3.5.21 节）。

EPTP switching 功能执行的操作如下代码所示。

```
do_eptp_switching(ECX)
{
    if (ECX > 511)
    {
        do_vmexit_processing(EXIT_NUMBER_VMFUNC); // 处理由 VMFUNC 引发的 VM-exit
    }

    NewEptp = EptpList[ECX]; // 根据 ECX 读取 EPTP-list 表项

    if (check_valid_for_eptp(NewEptp) == FALSE)
    {
        /*
         * EPTP 值无效，产生 VM-exit
         */
        do_vmexit_processing(EXIT_NUMBER_VMFUNC); // 处理由 VMFUNC 引发的 VM-exit
    }

    /*
     * 切换 EPTP 值：NewEptp 值写入 VMCS 的 EPTP 字段
     */
    EPTP = NewEptp;
}
```

首先检查 ECX 寄存器值，如果大于 511，则产生 VM-exit（ECX 寄存器的值在 0 到 511 之间），然后根据 ECX 寄存器的值从 EPTP-list 中读取相应的表项，这个表项为 8 字节的 EPTP 值。

检查新的 EPTP 值是否有效（参见 4.4.1.3 节），无效时将产生 VM-exit。检查通过后，新的 EPTP 值将写入在 VMCS 区域内的 EPTP 字段（参见 3.5.17 节）。

VMFUNC 指令执行完毕后，处理器接着执行下一条指令。此时，EPTP 已经被切换，处理器将使用新的 EPT 页表结构转换 guest-physical address。

然而在这一时刻，原有层级 guest paging structure 的“顶层指针”不变。也就是说，原有的 guest paging structure 保持不变，guest-linear address 仍旧通过原来的 guest paging

structure 进行转换。**顶层指针**取决于 guest 的分页模式而存放在不同的位置：

(1) guest 使用 32-bit 或者 IA-32e 分页模式时，CR3 寄存器的 guest physical address 指向 guest paging structure 的顶层页表结构，分别为 PDT 或 PML4T 结构。

(2) guest 使用 PAE 分页模式时，处理器内部的 PDPTE 寄存器存放 4 个 PDPTE 表项，分别指向各自对应 PDT 结构。4 个 PDPTE 寄存器已经被成功加载，它们的值不变。

因此，VMM 需要特别注意的是：在下一条指令的 fetch 过程中，可能会由于使用新的 EPT 页表结构转换 guest-physical address，而产生 **EPT violation** 或者 **EPT misconfiguration** 故障，从而导致指令 fetch 失败。发生这种情况时，VMM 需要做出相应的处理。

注意：执行 EPT switching 操作本身并不会产生 EPT violation 或者 EPT misconfiguration 故障。只有在 EPT switching 操作完成后才可能由于 guest 访问内存而产生 EPT violation 或者 EPT misconfiguration 故障。

Cache 维护

完成 EPT switching 操作后，**EP4TA** 值 (EPT[N-1:12]) 已经变更，但原 VPID 及 PCID 保持不变。处理器对所有内存访问使用新的 **EP4TA** 值建立 guest-physical mapping cache 信息，并结合原 VPID 与 PCID 值建立 combined mapping cache 信息 (cache 相关描述参见 6.2 节)。

6.1.12 实现 EPT 机制

在本书例子中，VMM 在初始化 VMCS 前应设置 VMB (VMCS manage block) 结构中的 GuestFlag 标志位值。

```
mov eax, GUEST_FLAG_PG | GUEST_FLAG_PE | GUEST_FLAG_EPT
mov [rbp + PCB.GuestA + VMB.GuestFlags], eax
```

上面是配置 GuestFlag 的一个示例，VMM 打算在 guestA 上使用分页保护模式，并启用 EPT 机制。initialize_vmcs_buffer 函数将根据 GuestFlag 值做相应的设置。

代码片段 6-3:

```
;;
;; 如果 "Enable EPT" = 1, 必须设置 EPT
;;
mov BYTE [ebp + PCB.EptEnableFlag], 0
test DWORD [ExecutionControlBufBase + EXECUTION_CONTROL.ProcessorControl2],
ENABLE_EPT
jz init_vm_execution_control_fields.@2

mov BYTE [ebp + PCB.EptEnableFlag], 1 ; 更新 EptEnableFlag 值

REX.Wrb
mov esi, [edx + VMB.Ep4taPhysicalBase]

#ifdef __X64
```

```

    mov edi, [edx + VMB.Ep4taPhysicalBase + 4]
%endif
;;
;; 初始化 EPTP 字段
;;
call init_eptp_field

```

上面的代码来自 `init_vm_execution_control_fields` 函数，当检查到 `secondary processor-based VM-execution control` 字段的“enable EPT”位为 1 时，将调用 `init_eptp_field` 函数来设置 EPTP 字段值。

代码片段 6-4:

```

;-----
; init_eptp_field()
; input:
;   x86: edi:esi - EP4TA (64 位值)
;   x64: rsi - EP4TA
; output:
;   none
; 描述:
;   1) 设置 Extended-page-table pointer (EPTP) 域
;-----
init_eptp_field:
    push ebp

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

#define ExecutionControlBufBase          (ebp + PCB.ExecutionControlBuf)

;;
;; 设置 EPTP 说明:
;; [2:0]    - EPT memory type; 支持 UC 和 WB 两类
;; [5:3]    - 3 (指示 EPT 的 page walk length 为 4 级)
;; [6]      - 1 (由 IA32_VMX_EPT_VPID_CAP[21]决定)
;; [11:7]   - 保留位, 为 0
;; [N-1:12] - EPT pointer 值
;; [63:N]   - 保留位, 为 0
;;
    REX.Wrb
    and esi, ~0FFFh                ; 保证 4K 边界

;;
;; 保证 EPT pointer 在 MAXPHYADDR 值内
;;
#ifdef __X64
    REX.Wrb
    and esi, [ebp + PCB.MaxPhyAddrSelectMask]
#else
    and edi, [ebp + PCB.MaxPhyAddrSelectMask + 4]
#endif

;;
;; 读 IA32_VMX_EPT_VPID_CAP[21] 位, 决定是否支持 dirty 标志
;;

```



```

xor eax, eax
test DWORD [ebp + PCB.EptVpidCap], (1 << 21)
setnz al
shl eax, 6                                ; EPTP[6]

;;
;; 设置 EPT memory type
;;
or eax, [ebp + PCB.EptMemoryType]        ; EPTP[2:0]

;;
;; 设置 page walk length = 3
;;
or eax, 18h                              ; EPTP[5:3] = 3

;;
;; 合成最终 EPT pointer 值
;;
REX.Wrxb
or esi, eax

;;
;; 更新 execution control buffer 的 Ept pointer
;;
REX.Wrxb
mov [ExecutionControlBufBase + EXECUTION_CONTROL.EptPointer], esi

#ifdef __X64
mov [ExecutionControlBufBase + EXECUTION_CONTROL.EptPointer + 4], edi
#endif

#ifdef ExecutionControlBufBase
pop ebp
ret

```

当支持 EPT paging structure 的 accessed 与 dirty 标志位时（从 IA32_VMX_EPT_VPID_CAP[21]检查），设置 EPTP[6]为 1。EPTP 的 walk length 值设置为 3（实际 length 为 3+1），EPTP[N-1:12]由参数 edi:esi（x86 下的 64 位）或者 rsi（x64）提供。

最终生成的 EPTP 字段值写入 execution control buffer 的 EptPointer 中。

guest-physical address 映射函数

具体负责 guest-physical address 的映射由 do_guest_physical_address_mapping 函数实现，分为 32 位与 64 位两个版本。

- do_guest_physical_address_mapping64：64 位版本实现在\lib\VMX\VmxFpage64.asm 文件中。
- do_guest_physical_address_mapping32：32 位版本实现在\lib\VMX\VmxFpage32.asm 文件中。

do_guest_physical_address_mapping 是个符号，根据__X64 符号的定义分别链接到 64

或 32 位版本中，在 inc\LibDef.inc 文件中定义。目的是在 32 位代码和 64 位代码中使用统一的符号。

do_guest_physical_address_mapping_n 函数执行多个页面的映射，同样被定义为符号，分别链接到 do_guest_physical_address_mapping32_n 或 do_guest_physical_address_mapping64_n 函数中。

代码片段 6-5:

```

#ifdef __STAGE1
...
#elifdef __X64
;;
;; 定义在 X64 下
;;
...

#define do_guest_physical_address_mapping do_guest_physical_address_mapping64
#define do_guest_physical_address_mapping_n do_guest_physical_address_mapping64_n
...

#else
;;
;; 定义在 X86 下
;;
...

#define do_guest_physical_address_mapping do_guest_physical_address_mapping32
#define do_guest_physical_address_mapping_n do_guest_physical_address_mapping32_n
...

#endif

```

以 64 位版本为例，下面看 do_guest_physical_address_mapping64 函数的实现（32 位版本原理一样）。这个函数原型声明看起来如下面 C 代码所示。

```

STATUS_CODE
do_guest_physical_address_mapping(
    UINT64 GuestPhysicalAddress,
    UINT64 HostPhysicalAddress,
    UINT PageAttribute
);

```

输入的 GuestPhysicalAddress 与 HostPhysicalAddress 参数是 64 位，PageAttribute 提供目标页面的属性及映射工作的指示代码。

```

mov rsi, 200000h
mov rdi, 08801000h
mov eax, EPT_READ | EPT_WRITE | EPT_EXECUTE
call do_guest_physical_address_mapping

```

上面是 do_guest_physical_address_mapping 函数的调用示例，将 GPA 200000H 映射到

HPA 08801000H, guest-physical address 的访问权限是 read/write/execute。

代码片段 6-6:

```

;-----
; do_guest_physical_address_mapping64()
; input:
;     rsi - guest physical address
;     rdi - physical address
;     eax - page attribute
; output:
;     0 - successful, otherwise - error code
; 描述:
;     1) 如果进行映射工作, 则映射 guest-physical address 到 physical address
;     2) 如果进行修复工作, 则修复 EPT violation 及 EPT misconfiguration
;
; page attribute 说明:
;     eax 传递过来的 attribute 由下面的标志位组成:
;     [0]      - Read
;     [1]      - Write
;     [2]      - Execute
;     [23:3]   - 忽略
;     [24]     - GET_PTE
;     [25]     - GET_PAGE_FRAME
;     [26]     - FIX_ACCESS, 置位时, 进行 access right 修复工作
;     [27]     - FIX_MISCONF, 置位时, 进行 misconfiguration 修复工作
;     [28]     - EPT_FIXING, 置位时, 需要进行修复工作, 而不是映射工作
;               - EPT_FIXING 清位时, 进行映射工作
;     [29]     - EPT_FORCE, 置位时, 强制进行映射
;     [31:30]  - 忽略
;-----
do_guest_physical_address_mapping64:
    push rbp
    push rdx
    push rbx
    push rcx
    push r10
    push r11

    ;;
    ;; EPT 映射说明:
    ;; 1) 所有映射均使用 4K-page 进行
    ;; 2) 在 PML4T(PXT), PDPT(PPT) 及 PDT 表上, 都具有 Read/Write/Execute 访问权限
    ;; 3) 在最后一级 PT 表上, 访问权限是输入的 ecx 参数(page attribute)
    ;; 4) 所有页表都需要使用 get_ept_page 动态分配 (从 kernel pool 内分配)
    ;;
    ;;
    ;; page attribute 使用说明:
    ;; 1) FIX_MISCONF=1 时, 表明修复 EPT misconfiguration 错误
    ;; 2) FIX_ACCESS=1 时, 表明修复 EPT violation 错误
    ;; 3) GET_PTE=1 时, 表明需要返回 PTE 值
    ;; 4) GET_PAGE_FRAME=1 时, 表明需要返回 page frame
    ;; 5) EPT_FORCE=1 时, 进行强制映射
    ;;

    mov r10, rsi                ; r10 = GPA
    mov r11, rdi                ; r11 = HPA

```

```

mov ebx, eax                ; ebx = page attribute
mov ecx, (32 - 4)           ; ecx = EPT 表项 index 左移位数 (shld)

;;
;; 读取当前 VMB 的 EP4TA 值
;;
mov rbp, [gs: PCB.CurrentVmbPointer]
mov rbp, [rbp + VMB.Ep4taBase] ; rbp = EPT PML4T 虚拟基址

do_guest_physical_address_mapping64.walk:
;;
;; EPT paging structure walk 处理
;;
shld rax, r10, cl
and eax, 0FF8h

;;
;; 读取 EPT 表项
;;
add rbp, rax                ; rbp 指向 EPT 表项
mov rsi, [rbp]              ; rsi = EPT 表项值

;;
;; 检查 EPT 表项是否为 not present, 包括:
;; 1) access right 不为 0
;; 2) EPT_VALID_FLAG
;;
test esi, 7                 ; access right = 0 ?
jz do_guest_physical_address_mapping64.NotPrsent
test esi, EPT_VALID_FLAG    ; 有效标志位 = 0 ?
jz do_guest_physical_address_mapping64.NotPrsent

;;
;; 当 EPT 表项为 Present 时
;;
test ebx, FIX_MISCONF
jz do_guest_physical_address_mapping64.CheckFix

;;
;; 修复 EPT misconfiguration 故障
;;
mov rax, ~EPT_LEVEL_MASK
and rax, rsi                ; 清掉错误 level 类型
mov esi, ecx
call get_ept_entry_level_attribute
or rsi, rax                 ; 设置 level 属性
call do_ept_entry_misconf_fixing64
cmp eax, MAPPING_SUCCESS
jne do_guest_physical_address_mapping64.Done
mov [rbp], rsi              ; 写回表项值

do_guest_physical_address_mapping64.CheckFix:
test ebx, FIX_ACCESS
jz do_guest_physical_address_mapping64.CheckGetPageFrame

;;
;; 修复 EPT violation 故障
;;

```

```

        mov edi, ebx                                ; rsi = 表项, ebx = page
attribute
        call do_ept_entry_violation_fixing64
        cmp eax, MAPPING_SUCCESS
        jne do_guest_physical_address_mapping64.Done
        mov [rbp], rsi                                ; 写回表项值

do_guest_physical_address_mapping64.CheckGetPageFrame:
    ;;
    ;; 读取表项内容
    ;;
    and rsi, ~0FFFh                                ; 清 bits 11:0
    and rsi, [gs: PCB.MaxPhyAddrSelectMask]        ; 取地址值

    ;;
    ;; 检查是否属于 PTE
    ;;
    cmp ecx, (32 - 4 + 9 + 9 + 9)
    jne do_guest_physical_address_mapping64.Next

    ;;
    ;; 如果需要返回 PTE 表项内容
    ;;
    test ebx, GET_PTE
    mov rax, [rbp]
    jnz do_guest_physical_address_mapping64.Done

    ;;
    ;; 如果需要返回 page frame 值
    ;;
    test ebx, GET_PAGE_FRAME
    mov rax, rsi
    jnz do_guest_physical_address_mapping64.Done

    ;;
    ;; 如果属于强制映射
    ;;
    test ebx, EPT_FORCE
    jnz do_guest_physical_address_mapping64.BuildPte

    mov eax, MAPPING_USED
    jmp do_guest_physical_address_mapping64.Done

do_guest_physical_address_mapping64.Next:
    ;;
    ;; 继续向下 walk
    ;;
    call get_ept_pt_virtual_address
    mov rbp, rax                                ; rbp = EPT 页表基址
    jmp do_guest_physical_address_mapping64.Nextwalk

do_guest_physical_address_mapping64.NotPrsent:
    ;;
    ;; 当 EPT 表项为 not present 时
    ;; 1) 检查 FIX_MISCONF 标志位, 如果尝试修复 EPT misconfiguration, 错误返回
    ;; 2) 如果尝试读取 page frame 值, 错误返回
    ;;

```

```

    test ebx, (FIX_MISCONF | GET_PAGE_FRAME)
    mov  eax, MAPPING_UNSUCCESS
    jnz  do_guest_physical_address_mapping64.Done

do_guest_physical_address_mapping64.BuildPte:
    ;;
    ;; 生成 PTE 表项值
    ;;
    mov  edx, ebx
    and  edx, 07                                ; 提供的 page frame 访问权限
    or   edx, EPT_VALID_FLAG                    ; 有效标志位
    or   rdx, r11                              ; 提供的 HPA

    ;;
    ;; 检查是否属于 PTE 层级
    ;; 1) 是: 写入生成的 PTE 值
    ;; 2) 否: 分配 EPT 页面
    ;;
    cmp  ecx, (32 - 4 + 9 + 9 + 9)
    je   do_guest_physical_address_mapping64.WriteEptEntry

    ;;
    ;; 下面分配 EPT 页面, 作为下一级页表
    ;;
    call get_ept_page                          ; rdx:rax = pa:va
    or   rdx, EPT_VALID_FLAG | EPT_READ | EPT_WRITE | EPT_EXECUTE

do_guest_physical_address_mapping64.WriteEptEntry:
    ;;
    ;; 生成表项值, 写入页表
    ;;
    mov  esi, ecx
    call get_ept_entry_level_attribute         ; 得到 EPT 表项层级属性
    or   rdx, rsi

    ;;
    ;; 写入 EPT 表项内容
    ;;
    mov  [rbp], rdx
    mov  rbp, rax                             ; rbp = EPT 表基址

do_guest_physical_address_mapping64.Nextwalk:
    ;;
    ;; 执行继续 walk 流程
    ;;
    cmp  ecx, (32 - 4 + 9 + 9 + 9)
    lea  rcx, [rcx + 9]                        ; 下一级页表的 shld count
    jne  do_guest_physical_address_mapping64.walk

    mov  eax, MAPPING_SUCCESS

do_guest_physical_address_mapping64.Done:
    pop  r11
    pop  r10
    pop  rcx
    pop  rbx
    pop  rdx
    pop  rbp
    ret

```

在设计中，每个 VM 使用不同的 EPTP 值，也就是每个 VM 有自己独立的 EPT paging structure。因此，每个 VM 所对应的 EP4TA 值在调用 initialize_vmcs_buffer 函数初始化 VMCS 区域时使用 get_vmcs_access_pointer 函数在 kernel pool 中分配。

EPT paging structure 并没有固定的区域，do_guest_physical_address_mapping64 函数在映射过程中，根据情况，PDPT，PDT 及 PT 都使用 get_ept_page 函数动态地在 kernel pool 中分配，如图 6-12 所示。

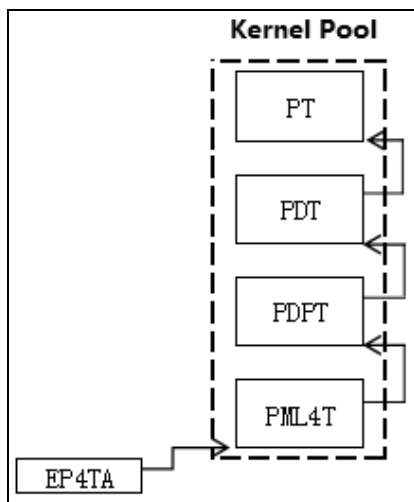


图 6-12

do_guest_physical_address_mapping64 函数大致的工作流程如下所示。

- (1) 根据 GPA 值，在 EPT 页表中读取 EPT 表项（最初是 PML4T）。
- (2) 检查 EPT 表项是否为 present。
 - 属于 present 时，来到第 (3) 步。
 - 属于 not-present 时，来到第 (6) 步。
- (3) 检查 PageAttribute 参数，是否需要修复 EPT misconfiguration。
 - 是，则调用 do_ept_entry_misconf_fixing64 函数尝试修复后，来到第 (4) 步。
 - 否，来到第 (4) 步。
- (4) 检查 PageAttribute 参数，是否需要修复 EPT violation。
 - 是，则调用 do_ept_entry_violation_fixing64 函数尝试修复后，来到第 (5) 步。
 - 否，来到第 (5) 步。
- (5) 检查 PageAttribute 参数，是否需要返回 page frame 地址值。
 - 是，则判断是否属于 page frame 表项，是则返回 page frame 值。否则继续下一步。
 - 否，则来到第 (9) 步。

(6) 在 not-present 的情况下, 检查 PageAttribute 参数, 是否修复 EPT misconfiguration。

- 是, 直接出错返回 (not-present 下不能修复 EPT misconfiguration)。
- 否, 继续第 (7) 步。

(7) 检查 PageAttribute 参数, 是否需要返回 page frame 地址值。

- 是, 直接出错返回 (not-present 下不能返回 page frame 地址值)。
- 否, 继续第 (8) 步。

(8) 使用 get_ept_page 函数分配一个 4K 页面。生成 EPT 表项内容, 然后写入 EPT 表项中。

(9) 调整 EPT 表项 index 值, 检查是否到达 PTE。

- 是, 则成功返回。
- 否, 则继续向下一级 EPT 页表 walk, 重复第 (1) 步。

do_guest_physical_address_mapping64 函数中使用的 shld count 值被使用于 SHLD 指令。

```
shld rax, r10, cl ; cl = shld count
```

如上面的指令所示, r10 存放 guest-physical address 值, 而 cl 是 shld count 值。初始的 shld count 值为 32-4 (即 28)。那么, 执行 SHLD 指令后 rax 寄存器的值是 $(\text{GPA} \gg 38) * 8$, 也就是 PML4E 的 offset 值。每次 walk 时 shld count 增加 9, 结束 walk 的 shld count 值为 $(32 - 4 + 9 + 9 + 9)$ 。

返回 GPA 对应的 host 虚拟地址

当 guest OS 启用分页机制时, guest-linear address 经过 guest paging structure 转换为 GPA。Host/VMM 出于内存管理目的, 可能需要访问 guest paging structure (另参考 6.3.3 节)。

现在, 我们需要实现 get_system_va_of_guest_pa 函数来完成 GPA 转换为 host-linear address, 这个函数看起来像下面的 C 代码:

```
PVOID
get_system_va_of_guest_pa(
    UINT64 GuestPhysicalAddress
);
```

向 get_system_va_of_guest_pa 函数提供 GPA 地址值, 返回 host-linear address。它实现在 \lib\VMX\VmxLib.asm 文件中。

代码片段 6-7:

```
;-----
; get_system_va_of_guest_pa()
; input:
;     esi - guest-physical address
; output:
;     eax - virtual address
; 描述:
;     1) 返回当前 guest 的 guest-physical address 对应的 system 的虚拟地址
```



```

;      2) 失败时返回 0 值
;-----
get_system_va_of_guest_pa:
    push ebx
    push ecx
    push edx

    xor ebx, ebx

    ;;
    ;; 读取 guest-physical address 的 VM domain 地址
    ;;
%ifdef __X64
    mov eax, GET_PAGE_FRAME
%else
    xor edi, edi
    mov ecx, GET_PAGE_FRAME
%endif
    call do_guest_physical_address_mapping
    cmp eax, MAPPING_UNSUCCESS
    je get_system_va_of_guest_pa.done

    ;;
    ;; 得到 system 地址
    ;;
%ifdef __X64
    or eax, SYSTEM_DATA_SPACE_BASE32
    mov ebx, 0FFFFFF800h
    REX.Wrxb
    shl ebx, 32
    REX.Wrxb
    or ebx, eax
%else
    mov ebx, SYSTEM_DATA_SPACE_BASE
    add ebx, eax
%endif

get_system_va_of_guest_pa.done:
    REX.Wrxb
    mov eax, ebx
    pop edx
    pop ecx
    pop ebx
    ret

```

其核心实现是通过 `do_guest_physical_address_mapping` 函数来返回 GPA 对应的 HPA，使用 `GET_PAGE_FRAME` 参数。然后加上 `SYSTEM_DATA_SPACE_BASE` 值。这个值在 x86 下是 `80000000H`，在 x64 下是 `FFFFFF800_80000000H`。

返回 guest-linear address 对应的 host 虚拟地址

很多时候 VMM 也需要根据 guest 的虚拟地址来读取数据分析。例如，当 guest 产生 VM-exit 后，VMM 可以根据 guest-RIP 值来读取分析 guest 当前执行的指令。VMM 有时也可能需要向 guest-RSP 写入某些数据（往 guest 当前栈中压入数据）。

现在，我们实现 `get_system_va_of_guest_va` 函数，将 guest-linear address 转化为 host-linear address。它的原型类似下面这样。

```
PVOID
get_system_va_of_guest_va(
    PVOID    GuestLinearAddress
);
```

它实现在 lib\Vm\VmLib.asm 文件中，如下面的代码所示。

代码片段 6-8:

```
;-----
; get_system_va_of_guest_va()
; input:
;     esi - guest-linear address
; output:
;     eax - host linear address
; 描述:
;     1) 得到 guest 线性地址对应的 host 线性地址
;-----
get_system_va_of_guest_va:
    ;;
    ;; 读取 guest-linear address 对应的 guest-physical address
    ;;
    call get_guest_pa_of_guest_va
    cmp eax, -1
    jne get_system_va_of_guest_va.@1
    mov eax, PCB.LastStatusCode
    mov DWORD [gs: eax], STATUS_GUEST_PAGING_ERROR
    mov eax, 0
    ret

get_system_va_of_guest_va.@1:
    ;;
    ;; 读取 guest-physical address 对应的 host linear address
    ;;
    REX.Wrb
    mov esi, eax
    call get_system_va_of_guest_pa
    ret
```

这个函数的实现是：首先将 guest-linear address 转化为 guest-physical address，然后调用前面所说的 get_system_va_of_guest_pa 函数返回 host 虚拟地址值。

然而，当 guest 运行在实模式时，由于关闭了分页机制，guest-linear address 就等于 guest-physical address。在这种情形下，我们不能使用 get_system_va_of_guest_va 函数来返回 host 虚拟地址。

因此，我们实现了一个 get_system_va_of_guest_os 函数来完成这个任务。实际上，它封装了 get_system_va_of_guest_va 函数与 get_system_va_of_guest_pa 函数，如下面的代码所示。

代码片段 6-9:

```
;-----
; get_system_va_of_guest_os()
; input:
;     esi - guest-physical address 或 guest-linear address
; output:
;     eax - host virtual address
```

```

; 描述:
; 1) 假如 guest OS 启用 paging, 则返回 guest-linear address 对应的 host virtual
address
; 2) 假如 guest OS 关闭 paging, 则返回 guest-physical address 对应的 host virtual
address
;-----
get_system_va_of_guest_os:
    push ebx

    REX.Wrxb
    mov ebx, esi

    ;;
    ;; 检查 guest OS 是否开启 paging
    ;; 1) 是, 调用 get_system_va_of_guest_va
    ;; 2) 否, 调用 get_system_va_of_guest_pa
    ;;
    GetVmcsField    GUEST_CR0
    test eax, CR0_PG
    mov esi, get_system_va_of_guest_pa
    mov eax, get_system_va_of_guest_va
    cmovz eax, esi
    REX.Wrxb
    mov esi, ebx
    call eax
    pop ebx
    ret

```

get_system_va_of_guest_os 函数读取 guest 的 CR0 寄存器, 判断 guest 是否开启分页机制, 然后进行相应的函数调用。假如 guest 开启了分页机制, 则调用 get_system_of_guest_va 函数, 否则调用 get_system_of_guest_pa 函数。

6.2 Cache 管理

由于 VMX 架构下支持对 guest-physical address 的转换, 在原有处理器架构基础上, 需要增加基于 EPT 扩展页表机制的 cache 管理。

地址转换的 cache 信息

对地址的转换（如线性地址转换物理地址），处理器可以缓存两类 cache 信息。

- **TLB cache**, 或称为“**translation cache**”。这类 cache 信息缓存线性地址到物理地址的转换结果。包括了线性 page number 对应的物理 page frame 地址值及 page frame 的访问权限与内存类型。例如在 4K 页面下, 有下面的 TLB cache 信息:
 - page number（线性地址 bits 47:12）对应的 4K page frame。
 - 合成后的 page frame 访问权限与内存类型。
- **paging-structure cache**, 这类 cache 信息缓存 paging structure 页表项内容。例如在 4K 页面下, 有下面的 paging-structure cache 信息:
 - PML4E number（线性地址的 bits 47:39）对应的 PML4E。
 - PDPTE number（线性地址的 bits 47:30）对应的 PDPTE。

➤ PDE number（线性地址的 bits 47:21）对应的 PDE。

在 EPT 机制下，guest 使用线性地址访问内存时，线性地址经过两种地址映射机制转换为平台上的物理地址：基于 guest paging structure 的 guest-linear address 映射，以及基于 EPT paging structure 的 guest-physical address 映射。

处理器能缓存由这两种地址映射机制产生的前面所述两类 cache 信息。即 **TLB cache** 与 **paging-structure cache** 信息。

6.2.1 linear mapping（线性映射）

线性映射是指线性地址通过 CR3 引伸出来层级的 paging-structure 映射到物理地址空间上。通俗地讲，就是线性地址转换为物理地址。

由线性映射产生下面两类的 cache 信息（关闭 EPT 机制）：

(1) **linear TLB cache**，包括了线性地址中 page number 对应的物理 page frame 及 page frame 的访问权限与内存类型。

- 在 32 位分页模式下，page frame 信息是：
 - 使用 4K 页面时，page number（线性地址的 bits 31:12）对应的 4K page frame。
 - 使用 4M 页面时，page number（线性地址的 bits 31:22）对应的 4M page frame。
- 在 PAE 分页模式下，page frame 信息是：
 - 使用 4K 页面时，page number（线性地址的 bits 31:12）对应的 4K page frame。
 - 使用 2M 页面时，page number（线性地址的 bits 31:21）对应的 2M page frame。
- 在 IA-32e 分页模式下（long-mode 分页模式），page frame 信息是：
 - 使用 4K 页面时，page number（线性地址的 bits 47:12）对应的 4K page frame。
 - 使用 2M 页面时，page number（线性地址的 bits 47:21）对应的 2M page frame。
 - 使用 1G 页面时，page number（线性地址的 bits 47:30）对应的 1G page frame。

(2) **linear paging-structure cache**，缓存处理器在 paging structure 中进行 walk 时所经过的页表项，但不缓存提供 page frame 的表项（即 4K 页面时的 PTE，2M 与 4M 页面时的 PDE 及 1G 页面时的 PDPTE）。

- 在 32 位分页模式下，cache 下面的信息：
 - 使用 4K 页面时，PDE number（线性地址的 bits 31:22）对应的 PDE。
 - 使用 4M 页面时，没有任何 paging structure 表项信息被 cache。
- 在 PAE 分页模式下，cache 下面的信息（注意，由于 PDPTE 被加载到 PDPTE 寄存器里。因此，没有 PDPTE 需要被 cache）。
 - 使用 4K 页面时，PDE number（线性地址的 bits 31:21）对应的 PDE。
 - 使用 2M 页面时，没有任何 paging structure 表项信息被 cache。
- 在 IA-32e 分页模式下（long-mode 分页模式），cache 下面的信息：
 - 使用 4K 页面时，PML4E number（线性地址的 bits 47:39）对应的 PML4E，

PDPTE number (线性地址的 bits 47:30) 对应的 PDPTE, 以及 PDE number (线性地址的 bits 47:21) 对应的 PDE。

- 使用 2M 页面时, PML4E number (线性地址的 bits 47:39) 对应的 PML4E, 以及 PDPTE number (线性地址的 bits 47:30) 对应的 PDPTE。
- 使用 1G 页面时, PML4E number (线性地址的 bits 47:39) 对应的 PML4E。

当开启 EPT 机制时 (secondary processor-based VM-execution 字段的 “enable EPT” 为 1), 处理器不会缓存由线性映射产生的 cache 信息 (缓存的是由 **combined mapping** 产生的 cache 信息)。也就是当 “enable EPT” 为 0 时 (或者在 **VMX root operation** 模式里) 才会产生线性映射 cache 信息。

6.2.2 guest-physical mapping (guest 物理映射)

guest 物理映射是指 guest-physical address 通过 EPTP 引伸出来层级的 EPT paging structure 映射到物理地址空间上 (host-physical address), 也就是 GPA 转换为 HPA。

guest-physical mapping 与线性地址的转换无关, 它产生下面两类 cache 信息。

(1) **EPT TLB cache**, 包括从 guest-physical address 转换得到的物理 page frame, 以及 page frame 的访问权限与内存类型信息。

- EPT 使用 4K 页面时, 缓存 page number (GPA 的 bits 47:12) 对应的 4K page frame。
- EPT 使用 2M 页面时, 缓存 page number (GPA 的 bits 47:21) 对应的 2M page frame。
- EPT 使用 1G 页面时, 缓存 page number (GPA 的 bits 47:30) 对应的 1G page frame。

(2) **EPT paging-structure cache**, 缓存处理器在 EPT paging structure 里进行 walk 时所经过的页表项, 但不缓存提供 page frame 的表项 (即 4K 页面时的 PTE、2M 与 4M 页面时的 PDE, 以及 1G 页面时的 PDPTE)。

- EPT 使用 4K 页面时, 缓存下面的 EPT 表项信息。
 - PML4E number (GPA 的 bits 47:39) 对应的 PML4E。
 - PDPTE number (GPA 的 bits 47:30) 对应的 PDPTE。
 - PDE number (GPA 的 bits 47:21) 对应的 PDE。
- EPT 使用 2M 页面时, 缓存下面的 EPT 表项信息。
 - PML4E number (GPA 的 bits 47:39) 对应的 PML4E。
 - PDPTE number (GPA 的 bits 47:30) 对应的 PDPTE。
- EPT 使用 1G 页面时, 缓存下面的 EPT 表项信息。
 - PML4E number (GPA 的 bits 47:39) 对应的 PML4E。

只有开启 EPT 机制后 (secondary processor-based VM-execution 字段的 “enable EPT” 为 1), 处理器才缓存上面的 cache 信息。当 “enable EPT” 为 0 时 (或者在 **VMX root operation** 模式下) 不会产生 guest-physical mapping 的 cache 信息。

6.2.3 combined mapping (合并映射)

合并映射 cache 信息是合并了 **linear mapping** 与 **guest-physical mapping** 这两种映射 cache 的结果。其中 combined TLB 直接缓存由 guest-linear address 到 host-physical address 的转换结果。同样有下面两类 cache 信息（在启用 EPT 机制后）。

(1) **combined TLB cache**，包括了从 guest-linear address 转换得到的最终物理 page frame（即 HPA），以及 page frame 访问权限与内存类型。

- 在 guest 32 位分页模式下，最终的物理 page frame（HPA）信息是：
 - 使用 4K 页面时，page number（线性地址的 bits 31:12）对应的 4K 物理 page frame。
 - 使用 4M 页面时，page number（线性地址的 bits 31:22）对应的 4M 物理 page frame。
- 在 guest PAE 分页模式下，最终的物理 page frame（HPA）信息是：
 - 使用 4K 页面时，page number（线性地址的 bits 31:12）对应的 4K 物理 page frame。
 - 使用 2M 页面时，page number（线性地址的 bits 31:21）对应的 2M 物理 page frame。
- 在 guest IA-32e 分页模式下，最终的物理 page frame（HPA）信息是：
 - 使用 4K 页面时，page number（线性地址的 bits 47:12）对应的 4K 物理 page frame。
 - 使用 2M 页面时，page number（线性地址的 bits 47:21）对应的 2M 物理 page frame。
 - 使用 1G 页面时，page number（线性地址的 bits 47:30）对应的 1G 物理 page frame。

(2) **combined paging-structure cache**，这部分 cache 信息与 linear paging-structure cache 信息是一致的（参见前面 6.2.1 节）。缓存处理器在 guest-paging structure 中进行 walk 时所经过的页表项，但不缓存提供 page frame 的表项（即 4K 页面时的 PTE、2M 与 4M 页面时的 PDE，以及 1G 页面时的 PDPTE）。

- 在 guest 32 位分页模式下，cache 下面的信息：
 - 使用 4K 页面时，PDE number（线性地址的 bits 31:22）对应的 PDE。
 - 使用 4M 页面时，没有任何 paging structure 表项信息被 cache。
- 在 guest PAE 分页模式下，cache 下面的信息（注意，由于 PDPTE 被加载到 PDPTE 寄存器中，因此，没有 PDPTE 需要被 cache）。
 - 使用 4K 页面时，PDE number（线性地址的 bits 31:21）对应的 PDE。
 - 使用 2M 页面时，没有任何 paging structure 表项信息被 cache。
- 在 guest IA-32e 分页模式下（long-mode 分页模式），cache 下面的信息：
 - 使用 4K 页面时，PML4E number（线性地址的 bits 47:39）对应的 PML4E，

PDPTE number (线性地址的 bits 47:30) 对应的 PDPTE, 以及 PDE number (线性地址的 bits 47:21) 对应的 PDE。

- 使用 2M 页面时, PML4E number (线性地址的 bits 47:39) 对应的 PML4E, 以及 PDPTE number (线性地址的 bits 47:30) 对应的 PDPTE。
- 使用 1G 页面时, PML4E number (线性地址的 bits 47:39) 对应的 PML4E。

只有开启 EPT 机制后 (secondary processor-based VM-execution 字段的 “enable EPT” 为 1), 处理器才缓存 combined mapping 的 cache 信息。当 “enable EPT” 为 0 时 (或者在 **VMX root operation** 模式下) 不会产生 guest-physical mapping 的 cache 信息。

由于在 EPT 机制下处理器不会缓存关于 linear mapping 的 cache 信息, 因此, combined mapping 的 cache 信息在某种程度上用来作为与 linear mapping 相关的 cache 信息。

6.2.4 cache 域

处理器在建立 cache 信息时, 也必须维护 cache 的归属信息, 也就是 cache 所属的域。另一方面, 在刷新 (invalidate) cache 操作时, 也需要指明刷新哪份 cache 信息 (哪个域的 cache)。

处理器引入 VMX 架构后, 在原基础上添加了新的 cache 归属标识。当前处理器支持三个归属标识, 分别为 PCID, VPID 及 EP4TA。它们用在不同的场合。

PCID (process context identifier, 进程上下文 ID)

顾名思义, PCID 用来为每个进程提供一个 ID 值。引入初衷是当 OS 进行进程切换时可能也需要进行 CR3 切换 (每个进程具有独立的 paging-structure 与线性地址空间)。

在更新 CR3 寄存器时, PCID 值提供在 CR3 寄存器的 bits 11:0 (12 位的 PCID 值) 中。因此, 软件可以为每份 paging structure 定义一个 ID 标识 (线性地址空间), 处理器通过这个 PCID 值来维护线性地址通过对应的 paging structure 转换而产生的 cache 信息。

在使用 PCID 功能前, 软件需要通过 CPUID 指令来检查处理器是否支持。

- 当 CPUID.01H:ECX[17] = 1 时, 软件可以置 CR4.PCIDE = 1 开启 PCID 功能。否则对 CR4.PCIDE 置位将产生 #GP 异常。
- PCID 只能在 IA-32e 模式下使用 (IA32_EFER.LMA = 1), 在其他模式下置 CR4.PCIDE = 1 则产生 #GP 异常。

在开启 PCID 功能后, 软件可以为每份 paging structure 提供一个非 0 的 PCID 值。当 CR4.PCIDE = 0 或者在其他模式下时, 默认的 PCID 值为 000H。在这种情况下, 处理器只维护 PCID 值为 000H 的那份 cache 信息。关于 PCID 更详细的信息请参考《x86/x64 体系探索及编程》11.5.1.3 节。

注意: 处理器用 PCID 来维护与线性地址转换相关的 cache 信息 (即 linear mapping 及 combined mapping, 参见前面 6.2.1 节与 6.2.3 节)。

EP4TA (EPT PML4T address, 扩展页表的 PML4T 地址)

在启用 EPT 机制后, guest-physical address 需要通过 EPT paging structure 转换到 host-physical address。层级 EPT paging structure 的地址则需要在 EPTP 字段 **bits N-1:12** 中提供 (参见 6.1.4 节)。

EPT paging structure 中顶层的页表 PML4T 地址被称为“**EP4TA**” (EPT PML4T address, 扩展页表的 PML4T 地址)。在 EPT 机制中, 一个 EP4TA 值也被作为 ID 标识符, 用来标识由 EPT paging structure 所映射的 **guest-physical address space**。

注意: 处理器用 EP4TA 来维护与 guest-physical address 转换相关的 cache 信息, 包括 **guest-physical mapping** 与 **combined mapping** 所产生的 cache 信息 (参见前面 6.2.2 节与 6.2.3 节)。

如果在每次进行 VM-entry 时提供不同的 EP4TA 值 (即不同的 EPTP 字段值), 处理器将维护不同 EP4TA 值所对应的 cache 信息。

取决于 secondary processor-based VM-execution control 字段“enable VPID”位的值, 处理器在每次 VM-entry 与 VM-exit 时决定是否刷新所有 EP4TA 值对应的 combined mapping 产生的 cache 信息, 但不刷新 guest-physical mapping 产生的 cache 信息。

VPID (virtual-processor identifier, 虚拟处理器 ID)

VPID 用来为每个虚拟处理器提供一个 ID 值。在第 4 章开头我们已经了解过“**虚拟处理器**”的概念, 每次 VM-entry 与 VM-exit 都相当于进行一次虚拟处理器的切换。

一个 VM (虚拟机) 就代表着一个虚拟处理器。在每次 VM 切换时 (在一个逻辑处理器下可能管理多个 VM 实例), VPID 用来标识该 VM (一个 VPID 对应一个 VM)。从另一个角度看, **VPID 用来标识 VM 在处理器中的 cache 域**。

当 secondary processor-based VM-execution control 字段的“enable VPID”为 1 时, VMM 需要在 virtual-processor identifier 字段提供一个 16 位的非 0 值作为 VPID (参见 3.5.18 节与 4.4.1.3 节)。VMM 应该为每个 VM 准备**独立的 VPID 值**, 用来**隔离每个 VM 的 cache 域**。也就是每个 VM 应该有自己的 VPID 值, 用来标识自己的 cache 空间。

有下面的情形之一, 处理器使用**默认的 VPID 值 (0000H)** :

- 当“enable VPID”为 0 时 (VMX non-root operation 模式)。
- 处理器处于 VM root operation 模式 (VMM 或 host)。
- 处理器运行在非 VMX 模式 (没执行 VMXON 指令, 或执行了 VMXOFF 指令)。
- 处理器切换到 SMM 模式 (在 VMX 中使用默认的 SMM 处理机制)。

每个 VM 使用默认的 VPID 值时, 处理器在 VM-entry 与 VM-exit 时维护当前 VPID 为 0000H 的那份 cache 信息。也就是 VM 与 VMM 的都在同一个 VPID cache 域下。

处理器用 VPID 来维护与**线性地址转换**相关的 cache 信息, 包括下面的情形。

- 当 VM 没有 EPT 机制时 (或者在 host 里), 处理器在 VPID 域中结合 PCID 为

linear mapping 建立相应的 cache 信息（参见 6.2.1 节）。

- 当 VM 启用 EPT 机制时，处理器在 VPID 与 EP4TA 联合的域中结合 PCID 为 combined mapping 建立相应的 cache 信息（参见 6.2.3 节）。

注意：当“enable VPID”为 0 时，处理器在 VM-entry 与 VM-exit 时会进行下面的刷新处理。

- 刷新 **VPID 值为 0000H 时所有 PCID** 对应的由 linear mapping 与 combined mapping 产生的 cache 信息。
- 刷新 **VPID 值为 0000H 时所有 EP4TA** 对应的由 combined mapping 产生的 cache 信息。

由于 VPID 用来维护与 VM 线性地址转换相关的 cache 信息。因此，处理器刷新 VPID 值管理下所有 PCID 与 EP4TA 值所对应的 cache 信息。当“enable VPID”为 1 时，没有任何 cache 需要被刷新。

可以推测，在每次 VM-entry 时为同一个 VM 提供不同的 VPID 值，处理器将维护这个 VM 多份 VPID 对应的 cache 信息。另一方面，当为不同的 VM 提供同一个 VPID 值时，这几个 VM 的 cache 都在这一个 VPID 下维护。

PCID, EP4TA 及 VPID 的关联关系

假设一个逻辑处理器中管理三个 VM 实例，分别为 VM 1，VM 2 及 VM 3。在每个虚拟机对应的 VMCS 中，secondary processor-based VM-execution control 字段的“enable VPID”位都为 1 值。VMM 为这三个虚拟机分别设置的 VPID 值为：0001H，0002H 及 0003H，如图 6-13 所示。

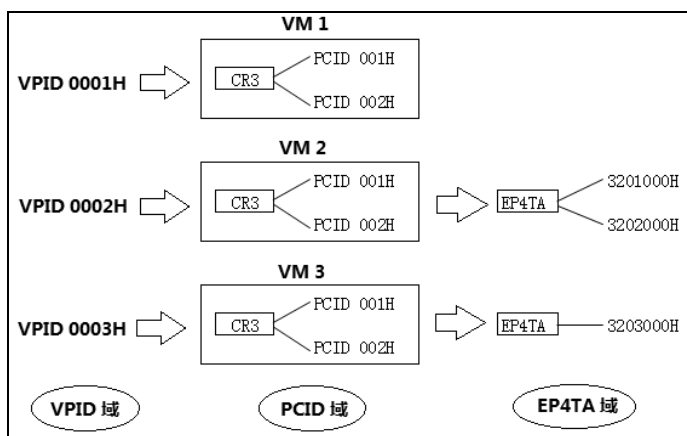


图 6-13

每个 VM 中的 guest OS 在切换 CR3 寄存器时，都使用了两个 PCID 值，分别为 001H 与 002H。VM 1 并没有开启 EPT 转换机制。VM 2 使用了两个 EP4TA 值，分别为 3201000H 与 3202000H。VM 3 使用一个 EP4TA 值，为 3203000H。

那么，处理器维护的 cache 信息如下：

(1) VM1，维护由 linear mapping 产生的 cache 信息。

- linear mapping 在下面域：
 - VPID 为 0001H 值。
 - PCID 为 001H 与 002H 值。

(2) VM2，维护由 guest-physical mapping 与 combined mapping 产生的 cache 信息。

- guest-physical mapping 在下面域：
 - EP4TA 为 3201000H 与 3202000H 值。
- combined mapping 在下面域：
 - VPID 为 0002H 值。
 - PCID 为 001H 与 002H 值。
 - EP4TA 为 3201000H 与 3202000H 值。

(3) VM3，维护由 guest-physical mapping 与 combined mapping 产生的 cache 信息。

- guest-physical mapping 在下面域：
 - EP4TA 为 3203000H 值。
- combined mapping 在下面域：
 - VPID 为 0003H 值。
 - PCID 为 001H 与 002H 值。
 - EP4TA 为 3203000H 值。

软件可使用 INVLPG, INVPCID, INVVPID 及 INVEPT 指令对相应的 cache 进行刷新（参见 6.2.6 节）。例如，执行 INVVPID 指令，提供的目标 VPID 为 0002H 值并且使用 single-context 刷新方式。那么，处理器将刷新 VPID 为 0002H 下所有 PCID 与 EP4TA（也就是 PCID 为 001H 与 002H，EP4TA 为 3201000H 与 3202000H 值）对应的 combined mapping cache 信息，以及 VPID 为 0002H 下所有 PCID（也就是 001H 与 002H）对应的 linear mapping cache 信息。

注意：尽管 VM2 不产生 linear mapping 的 cache 信息，但是 INVVPID 指令也会刷新 linear mapping 的 cache 信息。

对于 cache 域的设置，VM 与 VMM 的职能如下：

- guest 更新 CR3 寄存器时，guest OS 可以在 CR3 寄存器中为 paging-structure 所管理的线性地址空间设置 PCID 值。
- EP4TA 与 VPID 值的设置属于 VMM 的管理职责，在 VM-entry 时由 VMM 在 VMCS 区域相应的字段里提供。
- Host OS（或者 VMM）在更新 CR3 寄存器时，可以为 host 环境中的 paging-structure 所管理的线性地址空间设置 PCID 值。

guest OS 不能对 EP4TA 与 VPID 值进行设置，只能根据需要设置 PCID 值。

VMM 在设置 VPID 与 EP4TA 值时，应注意：

- 不要为不同的 VM 使用相同的 VPID 值。当使用 INVVPID 指令对目标 VPID 域的 cache 进行刷新时，结果会让不同 VM 的 cache 都得到刷新。
- 不要为不同的 VM 使用相同的 EP4TA 值。这样做会污染这些 VM 的 guest-physical address 空间，造成 VM 之间 GPA 转换的互相干扰。

如上面所述，在同一个逻辑处理器下可能需要维护多份 cache 信息（多个 VM 实例）。注意，在支持 Hyper-Threading 的处理器上，core 内的两个逻辑处理器（SMT）可能会共用 cache。如果一个逻辑处理器更新了 cache 信息，可能会影响另一个逻辑处理器的 cache 信息。

图 6-13 中，VM 1 没启用 EPT 转换机制，处理器为 VM 1 建立由 linear mapping 产生的 cache 信息。为 VM 2 与 VM 3 建立由 guest-physical mapping 与 combined mapping 产生的 cache 信息。

思考一下，处理器维护的每份 cache 具有附加的 tag 属性，用来标记 cache 域，如图 6-14 所示。

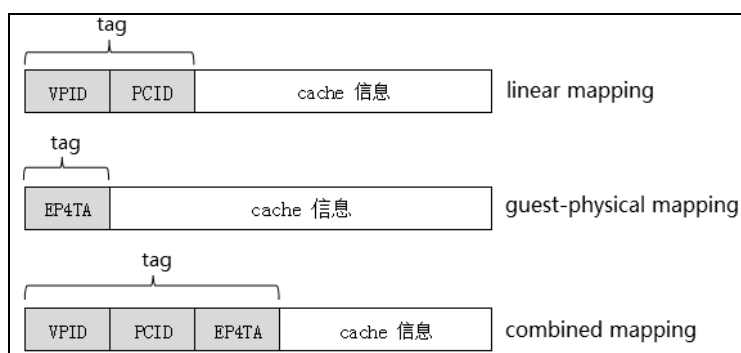


图 6-14

如图 6-14 所示，不同的映射途径产生的 cache 信息具有的 tag 属性也不同，也就是具有不同的域。

- linear mapping 产生的 cache 具有 VPID 与 PCID 域。
- guest-physical mapping 产生的 cache 只有 EP4TA 域。
- combined mapping 产生的 cache 具有 VPID，PCID 及 EP4TA 域。

处理器在建立这些相应的 cache 信息时标记上这些 tag 属性，在刷新 cache 时根据这些 tag 来找到对应的 cache 进行刷新。

在首次发起 VM-entry 前，在 host 环境中处理器使用默认 VPID 值（0000H），当前的 PCID 值提供在 CR3 寄存器中（CR4.PCIDE=1 时，否则当前 PCID 值为 000H）。注意，host 环境中并不存在 EP4TA 值，处理器为 host 环境只建立 linear mapping 相关的 cache 信息。

思考一下，当发生 VM-exit 后处理器切换回到 host 环境，“enable VPID”为 1 时，处理器保持所有 linear mapping, guest-physical mapping 及 combined mapping 的 cache 信息有效。

切换回到 host 环境后，处理器当前 linear mapping 的 cache 域可能为：

- 取决于在 VM-exit 时对 CR4.PCIDE 位的加载设置（参见 5.13.1 节），当前 PCID 值为 CR3 寄存器的 bits 11:0 或者 000H（默认 PCID 值）。
- 当前 VPID 值为 0000H（默认 VPID 值）。

cache 域刷新

VMX 指令集里提供了两条基于 VPID 与 EP4TA 域的 cache 刷新指令：

- INVVPID 指令，刷新 VPID 域下与线性地址映射相关的 cache 信息。包括：
 - 由 linear mapping 产生的 cache 信息。
 - 由 combined mapping 产生的 cache 信息。
- INVEPT 指令，刷新 EP4TA 域下与 GPA 映射相关的 cache 信息。包括：
 - 由 guest-physical mapping 产生的 cache 信息。
 - 由 combined mapping 产生的 cache 信息。

在 Intel64 体系上，也提供了 INVPCID 指令来刷新 PCID 域下的 linear mapping 及 combined mapping 产生的 cache 信息。

6.2.5 cache 建立

当一个物理页面被首次成功访问时，处理器将为这个地址转换建立对应的 cache 信息。取决于处理器当前是否使用 EPT 机制，建立不同的 cache 信息。

注意：当物理页面访问失败时，没有任何 cache 信息需要被建立。这个失败包括：

- 由于线性地址的转换产生 #PF 异常（页面属于 not-present，或者无访问权限等）。
- 由于 guest-physical address 的转换产生 EPT violation 故障（参见 6.1.8.1 节）。
- 由于 guest-physical address 的转换产生 EPT misconfiguration 故障（参见 6.1.8.2 节）。

处理器成功访问页面，并且对应的 cache 尚未被建立时（页面被首次访问），处理器才可能建立相应的 cache 信息。

非 EPT 机制下的 cache 建立

当 secondary processor-based VM-execution control 字段的“enable EPT”位为 0 时，VM 不使用 EPT 机制映射地址。或者处理器处于 VMX root operation 模式也不使用 EPT 机制映射地址。因此，没有任何 guest-physical mapping 及 combined mapping 相关的 cache 信息需要被建立。

当 CR0.PG = 0 时，没有线性地址需要转换。因此，没有任何 linear mapping 相关的 cache 信息需要被建立。

当 CR0.PG=1 时，线性地址由 CR3 寄存器引伸出来的 paging-structure 进行转换，处理器在当前 VPID 与 PCID 域中为 **linear mapping** 建立相应的 cache 信息。

- 当前 VPID 值提供在 VMCS 的 VPID 字段里。当 secondary processor-based VM-execution control 字段“enable VPID”位为 0，或者在 VMX root operation 模式下时，当前 VPID 的值为 0000H。
- 当前 PCID 值提供在 CR3 寄存器的 bits 11:0 中。当 CR4.PCIDE = 0 时，当前 PCID 值为 000H。

EPT 机制下的 cache 建立

当 secondary processor-based VM-execution control 字段的“enable EPT”位为 1 时，VM 使用 EPT 机制映射地址。没有任何 linear mapping 相关的 cache 信息需要被建立。

guest-physical address 由 EP4TA 引伸出来的 EPT paging structure 进行转换，处理器在当前的 EP4TA 域建立 **guest-physical mapping** 相关的 cache 信息。

- 当前 EP4TA 提供在 EPTP 字段的 bits $N-1:12$ 中 ($N = \text{MAXPHYADDR}$)。

当 CR0.PG = 0 时，没有 guest-linear address 需要转换。因此，没有任何 combined mapping 相关的 cache 信息需要被建立。

当 CR0.PG = 1 时，guest-linear address 由 CR3 寄存器引伸出来的 guest-paging structure 进行转换，处理器在当前 VPID 与 EP4TA 域，结合当前的 PCID 建立 **combined mapping** 相关的 cache 信息（如图 6-14 所示）。

- 当前 VPID 值提供在 VMCS 的 VPID 字段中。当“enable VPID”为 0 时（或者在 VMX root operation 模式），当前 VPID 的值为 0000H。
- 当前 EP4TA 值提供在 EPTP 字段的 bits $N-1:12$ ($N = \text{MAXPHYADDR}$)。
- CR4.PCIDE = 1 时，当前 PCID 值提供在 CR3 寄存器的 bits 11:0 中。CR4.PCIDE = 0 时，当前 PCID 值为 000H。

global TLB cache 建立

global page（全局页面）被定义在由 CR3 寄存器引伸出来的 paging structure 内提供 page frame 的表项里。因此，它是与**线性地址转换**相关的。在下面的表项中定义 global page。

- 使用 4K 页面时，定义在 PTE 的 G 位 (bit 8) 中。
- 使用 2M 或者 4M 页面时，定义在 PDE 的 G 位 (bit 8) 中。
- 使用 1G 页面时，定义在 PDPTE 的 G 位 (bit 8) 中。

当页面是 global page 时，处理器建立下面的 cache 信息。

- 当“enable EPT”为 0 时，在当前 VPID 域下为 **linear mapping** 建立 global TLB cache 信息 (PCID 为任何值)。
- 当“enable EPT”为 1 时，在当前 VPID 与 EP4TA 的联合域下为 **combined**

mapping 建立 global TLB cache 信息（PCID 为任何值）。

在 EPT paging structure 中并不支持 global 标志位。因此，不存在与 guest-physical address 转换相关的 global TLB cache。

6.2.6 cache 刷新

由于 VMX 架构的引入，在 Intel 处理器上提供了 4 条对 TLB 与 paging-structure cache 进行刷新的指令，它们是：INVLPG，INVPCID，INVVPID 及 INVEPT 指令。INVLPG 与 INVPCID 是 Intel64 体系原有的指令，而 INVVPID 与 INVEPT 是 VMX 架构新增的指令。

注意：当处理器支持 Hyper-Threading 技术时，表示在处理器的一个 core 内拥有两个执行单元（SMT，同步多线程），也就是 core 内拥有两个逻辑处理器。由于 core 内的两个逻辑处理器共享 cache，因此一个逻辑处理器进行 cache 刷新时，就会影响到另一个逻辑处理器的 cache 信息。

本文中所提到的处理器指的是“逻辑处理器”，为了简便，大多数时候省去了“逻辑”二字（另见第 4 章开头所述），务必注意。

刷新的映射途径

关于这些指令对何种地址映射途径所产生的 cache 进行刷新，有下面的情形。

- INVLPG，INVPCID 及 INVVPID 指令刷新与**线性地址转换**相关的 cache 信息。它们刷新以下映射途径的 cache 信息：
 - 由 linear mapping 产生的 cache 信息。
 - 由 combined mapping 产生的 cache 信息。
- INVEPT 指令刷新在 EPT 机制下与 **guest-physical address 转换**相关的 cache 信息。它刷新以下映射途径的 cache 信息：
 - 由 **guest-physical mapping** 产生的 cache 信息。
 - 由 **combined mapping** 产生的 cache 信息。

注意：执行指令主动刷新 cache（或者由于页故障而引起 cache 刷新）时。当刷新 linear mapping 时并不会因为“enable EPT”为 1 而受到影响。当刷新 guest-physical mapping 或者 combined mapping 时也并不会因为“enable EPT”为 0 而受到影响。

思考一下，VM 可能在前一次 VM-entry 时启用了 EPT 机制，从而已经建立了 guest-physical mapping 与 combined mapping 相关的 cache 信息。在后续的 VM-entry 中虽然关闭了 EPT 机制（或者其他 VM 并没有使用 EPT 机制），但是处理器依然留存着已经被建立的 guest-physical mapping 与 combined mapping 相关的 cache 信息。

刷新的 cache 域

这些指令对刷新的 cache 域有下面的不同（参见图 6-14 所示的 cache 域）。

- INVLPG 指令
 - 刷新当前 VPID 与 PCID 下的 linear mapping 相关的 cache 信息。
 - 刷新当前 VPID 与 PCID 下，所有 EP4TA 对应的 combined mapping 相关的 cache 信息。
- INVPCID 指令
 - 刷新当前 VPID 下，目标 PCID（提供在指令操作数中）中的 linear mapping 相关的 cache 信息。
 - 刷新当前 VPID 下，目标 PCID（提供在指令操作数中）中里所有 EP4TA 对应的 combined mapping 相关的 cache 信息。
- INVVPID 指令
 - 刷新目标 VPID 下（提供在指令操作数中），所有 PCID 值的 linear mapping 相关的 cache 信息。
 - 刷新目标 VPID 下（提供在指令操作数中），所有 PCID 与 EP4TA 对应的 combined mapping 相关的 cache 信息。
- INVEPT 指令
 - 刷新目标 EP4TA 下（提供在指令操作数中） guest-physical mapping 相关的 cache 信息。
 - 刷新目标 EP4TA 下（提供在指令操作数中），所有 VPID 与 PCID 域对应的 combined mapping 相关的 cache 信息。

Mov-to-CR 引起的 cache 刷新

执行写控制寄存器 CR0，CR3 或者 CR4 将引起下面的 cache 刷新。

- Mov to CR0
 - 当 CR0.PG 从 1 清为 0 时，刷新当前 VPID 下，所有 PCID 对应的 linear mapping 相关的 cache（包括 global page）。刷新当前 VPID 下，所有 PCID 与 EP4TA 对应的 combined mapping 相关的 cache（包括 global page）。
 - 改变 CR0.NW 或 CR0.CD 时，也进行上面的刷新动作。
- Mov to CR3
 - CR4.PCIDE = 0 时，刷新当前 VPID 下，默认 PCID（000H）对应的 linear mapping 相关的 cache（不包括 global page）。刷新当前 VPID 下，默认 PCID（000H）下所有 EP4TA 对应的 combined mapping 相关的 cache（不包括 global page）。
 - CR4.PCIDE = 1，并且 CR3[63]为 0 时，刷新当前 VPID 与 PCID（提供在 CR3[11:0]中）的 linear mapping 相关的 cache（不包括 global page）。刷新当前 VPID 与 PCID（提供在 CR3[11:0]中）下，所有 EP4TA 对应的 combined mapping 相关的 cache（不包括 global page）。
- Mov to CR4
 - 当 CR4.PCIDE 从 1 清为 0，或者改变 CR4.PGE、CR4.PSE 位时，刷新当前

VPID 下，所有 PCID 对应的 linear mapping 相关 cache（包括 global page）。刷新当前 VPID 下，所有 PCID 与 EP4TA 对应的 combined mapping 相关 cache（包括 global page）。

- 当 CR4.SMEP 从 0 置为 1，或者改变 CR4.PAE 时，刷新当前 VPID 与 PCID 下对应的 linear mapping 相关的 cache（包括 global page）。刷新当前 VPID 与 PCID 下，所有 EP4TA 对应的 combined mapping 相关的 cache（包括 global page）。

由页故障引起的 cache 刷新

在线性地址（包括 guest-linear address）经过 CR3 寄存器引伸出来的 paging structure 进行转换时，如果发生 #PF 异常（即**线性地址无访问权限**），处理器将刷新由这个线性地址转换而建立的 TLB 与 paging-structure cache 信息。

- 刷新当前 VPID 与 PCID 域下的 **linear mapping** 相关的 cache 信息。
- 刷新当前 VPID 与 PCID 域下，所有 EP4TA 对应的 **combined mapping** 相关的 cache 信息。

如果访问的页面**属于 not-present**，处理器不可能建立该线性地址转换而来的 cache 信息（参见 6.2.5 节）。但是，如果属于线性地址无访问权限而引起 #PF 异常，该线性地址可能已经由于首次成功访问而建立了对应的 cache 信息。在后续的再次访问中可能并没有足够的权限而产生 #PF 异常。

若 guest-physical address 通过 EPT paging structure 进行转换时发生 EPT violation 故障，处理器将刷新由这个 guest-physical address 转换而建立的 TLB 与 paging-structure cache 信息。

- 刷新当前 EP4TA 域下的 **guest-physical mapping** 相关 cache 信息。
- 刷新当前 EP4TA 域下，当前的 VPID 与 PCID 对应的 **combined mapping** 相关的 cache 信息。

同样，一个 EPT misconfiguration 故障不会引起刷新 cache 的动作（一个 EPT 表项能引起 EPT misconfiguration 故障是不可能建立相应的 cache 信息）。

VM-entry 与 VM-exit 对 cache 的刷新

参见 4.8 节与 5.15 节所述，在 VM-entry 与 VM-exit 时若 secondary processor-based VM-execution control 字段的“enable VPID”为 0，处理器将对 linear mapping 与 combined mapping 相关的 cache 信息进行刷新。

在 VM-entry 与 VM-exit 时，没有任何 guest-physical mapping 相关的 cache 信息需要被刷新。

另外，执行 VMXON 与 VMXOFF 指令时，没有任何 linear mapping, guest-physical mapping 及 combined mapping 相关的 cache 信息需要被刷新。

6.2.6.1 INVLPG 指令刷新 cache

INVLPG 指令提供一个内存操作数（线性地址），处理器根据这个线性地址的转换关系找到对应的 TLB cache 信息进行刷新。INVLPG 指令也刷新当前 PCID 下所有的 paging-structure cache 信息。INVLPG 指令也刷新所有 global TLB cache 信息（忽略 PCID）。

下面是 INVLPG 指令使用的示例。

```
invlpg [rax] ; 刷新线性地址(rax)对应的 linear mapping 与 combined mapping
```

INVLPG 指令能刷新当前 VPID 与 PCID 域下的 linear mapping 相关的 cache 信息，也对当前 VPID 与 PCID 域下，所有 EP4TA 对应的 combined mapping 相关的 cache 信息进行刷新。

INVLPG 指令不能刷新 guest-physical mapping 所产生的 cache 信息。

6.2.6.2 INVPCID 指令刷新 cache

INVPCID 指令根据提供的目标 PCID 域来刷新 linear mapping 与 combined mapping 相关的 cache 信息。INVPCID 指令提供两个操作数：operand1 提供刷新的类型值，operand2 提供 INVPCID 描述符。

下面是 INVPCID 指令使用的示例。

```
invpcid rax, [rsi] ; 根据[rsi]提供的 INVPCID 描述符来进行相应的刷新动作 (rax)
```

INVPCID 指令能刷新当前 VPID 下，目标 PCID 域的 linear mapping 相关的 cache 信息。也对当前 VPID 下，目标 PCID 域所有 EP4TA 对应的 combined mapping 相关的 cache 信息进行刷新。

INVPCID 指令不能刷新 guest-physical mapping 所产生的 cache 信息。

INVPCID 支持的刷新类型如下。

(1) **individual-address invalidation**（单个地址无效）：当类型值为 0 时，根据 INVPCID 描述符提供的目标 PCID 值 (bits 11:0)，刷新目标线性地址 (bits 127:64) 所对应的 cache 信息。但不刷新 global page 相关的 cache 信息。

(2) **single-context invalidation**（单个环境无效）：当类型值为 1 时，刷新 INVPCID 描述符提供的目标 PCID 值 (bits 11:0) 下所有的 cache 信息。但不刷新 global page 相关的 cache 信息。

(3) **all-context invalidation, including global translations**（所有环境无效，包括全局转换）：当类型值为 2 时，刷新所有 PCID 值（包括 000H）下的 cache 信息，包括 global page 相关的 cache 信息。

(4) **all-context invalidation**（所有环境无效）：当类型值为 3 时，刷新所有 PCID 值（包括 000H）下的 cache 信息。但不刷新 global page 相关的 cache 信息。

INVPCID 描述符的结构如图 6-15 所示。

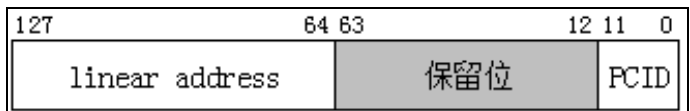


图 6-15

INVPCID 描述符的 bits 11:0 提供目标 PCID 值，bits 127:64 提供目标线性地址值。这个线性地址仅在刷新类型值为 0 时（individual-address invalidation）使用。128 位的 INVPCID 描述符保存在内存中，描述符的地址值提供给 INVPCID 指令作为 operand2。

6.2.6.3 INVVPID 指令刷新 cache

INVVPID 指令根据提供的目标 VPID 域来刷新 linear mapping 与 combined mapping 相关的 cache 信息。INVVPID 指令提供两个操作数：operand1 提供刷新的类型值，operand2 提供 INVVPID 描述符。

下面是 INVVPID 指令的使用示例。

```
invvpid rax, [rsi] ; 根据[rsi]提供的 INVVPID 描述符来进行相应的刷新动作 (rax)
```

INVVPID 指令能刷新目标 VPID 下，所有 PCID 域的 linear mapping 相关的 cache 信息。也对目标 VPID 下，所有 PCID 与 EP4TA 对应的 combined mapping 相关的 cache 信息进行刷新。

INVVPID 支持的刷新类型如下。

(1) **individual-address invalidation**（单个地址无效）：当类型值为 0 时，根据 INVVPID 描述符提供的目标 VPID 值（bits 15:0），刷新目标线性地址（bits 127:64）所对应的 cache 信息。包括刷新 global page 相关的 cache 信息。

(2) **single-context invalidation**（单个环境无效）：当类型值为 1 时，刷新 INVVPID 描述符提供的目标 VPID 值（bits 15:0）下所有的 cache 信息。包括刷新 global page 相关的 cache 信息。

(3) **all-context invalidation**（所有环境无效）：当类型值为 2 时，刷新所有 VPID 值（除了 0000H 外）下的 cache 信息，包括刷新 global page 相关的 cache 信息。

(4) **single-context invalidation, retaining global translations**（单个环境无效，保留全局转换）：当类型值为 3 时，刷新 INVVPID 描述符提供的目标 VPID 值（bits 15:0）下所有的 cache 信息。但不刷新 global page 相关的 cache 信息。

INVVPID 描述符的结构如图 6-16 所示。

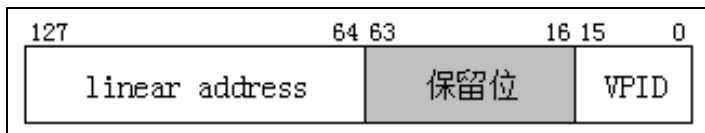


图 6-16

INVVPID 描述符的 bits 15:0 提供目标 VPID 值, bits 127:64 提供目标线性地址值。这个线性地址仅在刷新类型值为 0 时 (individual-address invalidation) 使用。128 位的 INVVPID 描述符保存在内存中, 描述符的地址值提供给 INVVPID 指令作为 operand2。

6.2.6.4 INVEPT 指令刷新 cache

INVEPT 指令根据提供的目标 EP4TA 值 (EPTP[N-1:12]) 刷新 guest-physical mapping 与 combined mapping 相关的 cache 信息。INVEPT 指令提供两个操作数: operand1 提供刷新的类型值, operand2 提供 INVEPT 描述符。

下面是 INVEPT 指令的使用示例。

```
invept rax, [rsi] ; 根据[rsi]提供的 INVEPT 描述符来进行相应的刷新动作 (rax)
```

INVEPT 指令能刷新目标 EP4TA 下所有的 guest-physical mapping 相关的 cache 信息。也对目标 EP4TA 下, 所有 VPID 与 PCID 对应的 combined mapping 相关的 cache 信息进行刷新。

INVEPT 指令不能刷新 linear mapping 所产生的 cache 信息, 它支持如下刷新类型。

(1) **Single-context invalidation** (单个环境无效): 当类型值为 1 时, 刷新 INVEPT 描述符 EPTP (bits 63:0) 所提供的 EP4TA (EPTP[N-1:12]) 下所有的 cache 信息。

(2) **All-context invalidation** (所有环境无效): 当类型值为 2 时, 刷新所有 EP4TA 的 cache 信息。

INVEPT 描述符结构如图 6-17 所示。

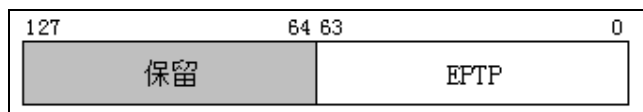


图 6-17

INVEPT 描述符的 bits 63:0 提供 EPTP (EPT pointer) 值, 目标的 EP4TA 值为 EPTP[N-1:12] ($N = \text{MAXPHYADDR}$), bits 127:64 为保留位。

6.2.6.5 INVVPID 指令使用指南

假如 VM 没有开启 EPT 机制, 使用 INVVPID 指令可以完成如下模拟任务。

(1) 模拟 guest 执行 INVLPG 指令。

- INVVPID 描述符的 VPID 为 guest 当前使用的 VPID 值。
- INVVPID 描述符中的线性地址为 INVLPG 指令的操作数。
- 执行 INVVPID 指令时, 使用刷新类型为 **Individual-address invalidation** (值为 0, 单个地址无效)。

(2) 模拟 guest 执行 MOV to CR3 指令, 它不刷新 global page。

- INVVPID 描述符的 VPID 为 guest 当前使用的 VPID 值。

- 执行 INVVPID 指令时，使用刷新类型为 **single-context invalidation, retaining global translations**（值为 3，单个环境无效，保留全局转换）。

(3) 模拟 guest 执行 MOV to CR0 或者 MOV to CR4 指令，包括刷新 global page。

- INVVPID 描述符的 VPID 为 guest 当前使用的 VPID 值。
- 执行 INVVPID 指令时，使用刷新类型为 **single-context invalidation**（值为 1，单个环境无效）。

一般情况下，在 5.3 节中描述了当 secondary processor-based VM-execution control 字段的“virtualize APIC accesses”为 1 时，若干个 guest 线性访问（或者 guest-physical 访问）APIC-access page 页面会产生 APIC-access VM-exit 的情形。

当“virtualize APIC accesses”为 0 时，guest 线性访问（或者 guest-physical 访问）APIC-access page 不会产生 APIC-access VM-exit。此时，guest 能成功访问 APIC-access page 内的任何值（由 APIC-access address 字段提供物理地址值，参见 3.5.9 节）。处理器会维护由线性地址（或 guest-physical address）转换到物理地址产生的 cache 信息。

在上述访问 APIC-access page 页面的 cache 信息已经被建立后，此时 VMM 重新将“virtualize APIC accesses”置为 1 时，guest 线性访问（或 guest-physical 访问）APIC-access page 内的值可能不会产生 APIC-access VM-exit。因此，VMM 需要使用 INVVPID 指令刷新 cache。

Intel 推荐在下面的情形里，VMM 需要使用 INVVPID 指令刷新 cache。

- “virtualize APIC accesses”位由 0 设为 1 时。
- APIC-access page 页面地址改变时（改变 APIC-access address 字段的值）。
- paging structure 改变时（CR3 寄存器更新，或者线性地址转换到 APIC-access page 的 paging structure 表项更改）。

VMM 执行 INVVPID 指令时，INVVPID 描述符的 VPID 值是 guest 所对应的 VPID 值，并且使用 single-context invalidation（值为 1）刷新类型。当 guest（VM）没有使用 VPID 及 EPT 机制时，不需要进行 cache 刷新。

Intel 也推荐在执行 VMXON 指令后（开启 VMX 模式后）及执行 VMXOFF 指令前（关闭 VMX 模式前），VMM 需要紧接着执行 INVVPID 指令并且使用 all-context invalidation（值为 2）刷新类型进行 cache 刷新。

6.2.6.6 INVEPT 指令使用指南

VM 开启 EPT 机制后，当 VMM 修改 EPT paging structure 表项的以下内容时，需要执行 INVEPT 指令刷新 cache。

(1) 将任何一级表项 bits 2:0 中的任何一位从 1 清为 0。也就是：取消某一访问权限或者将表项置为 not-present（bits 2:0 的值改为 0）。

(2) 更改任何一级表项内的物理地址值。

(3) 启用 accessed 标志位时，清任何一级表项的 accessed 标志位 (bit 8)。

(4) 更改提供 page frame 的表项 (即 PTE, 2M 页面下的 PDE 或者 1G 页面下的 PDPTE)，包括下面的内容之一：

- 清 PDPTE 或者 PDE 的 bit 7 (取消 large-page)。
- 更改 bits 5:3 (内存类型) 或者 bit 6 (Ignore PAT 位)，也就是更改 page frame 的内存类型。
- 启用 dirty 标志位时，清 bit 9 (dirty 位)。

执行 INVEPT 指令时，INVEPT 描述符的 EPTP 值为更改的 EPT paging structure 所属的 EPTP 值。使用的刷新类型是 single-context invalidation (值为 1)。

当发生 VM-exit 后，VMM 修改 EPTP 的 bit 6 (accessed 与 dirty 开启控制位) 从 0 到 1 重新进行 VM-entry 时，VMM 需要使用 single-context invalidation (值为 1) 刷新类型执行 INVEPT 指令刷新 cache。

当将 EPT paging structure 任何一级表项 bits 2:0 中的任何一位从 0 置为 1 时 (但不是从 not-present 修改为 present)，VMM 也需要用 single-context invalidation (值为 1) 刷新类型执行 INVEPT 指令刷新 cache。

注意：在 guest-physical address 转换过程中引起 EPT misconfiguration 故障，或者由于 not-present 而引起 EPT violation 故障时，处理器不会为这个 guest-physical address 转换建立任何 cache。因此，VMM 修复 EPT misconfiguration 故障，或者由 not-present 引起的 EPT violation 故障后重新 VM-entry 时，无须使用 INVEPT 刷新 cache。

与 6.2.6.5 节中关于访问 APIC-access page 产生 APIC-access VM-exit 的描述一致，当“virtualize APIC accesses”为 0 时，由于 guest 线性访问 (或 guest-physical 访问) APIC-access page 已经建立相关的 cache 信息，此时，在 VMM 将“virtualize APIC accesses”从 0 置为 1 时，guest 访问 APIC-access page 可能并不会产生 APIC-access VM-exit。

Intel 推荐在下面的情形下，需要刷新 cache。

(1) “virtualize APIC accesses”位从 0 置为 1 时。

(2) APIC-access page 页面地址改变时 (改变 APIC-access address 字段的值)。

VMM 在 VM-entry 前，执行 INVEPT 指令使用 single-context invalidation (1 值) 刷新类型进行 cache 刷新。

VMM 也可以在执行 VMXON 指令后 (开启 VMX 模式后) 及执行 VMXOFF 指令前 (关闭 VMX 模式前)，紧接着执行 INVEPT 指令并且使用 all-context invalidation (2 值) 刷新类型进行 cache 刷新。

6.3 内存虚拟化管理

一个虚拟化平台中存在多个 VM（虚拟机）实例，每个虚拟机就像一个独立的物理平台，有自己的物理资源（包括内存、设备等）。guest OS 运行在虚拟机环境里，与普通 OS 一样负责管理虚拟机里的资源。更重要的是，guest OS 并不知道自己运行在虚拟机环境中。

6.3.1 分配物理内存

VMM 需要为每个 VM 分配独立的物理内存，然而物理平台上的内存是有限的，怎样合理有效地分配和管理每个虚拟机的物理内存是 VMM 设计的难点之一。

在本书的例子中，每个逻辑处理器支持 4 个虚拟机。每个虚拟机对应一个 VMB（VMCS manage block）结构，分别为 GuestA，GuestB，GuestC 及 GuestD。这样可以支持一个逻辑处理器同时运行和管理 4 个虚拟机。实际上本书所有的例子并没有同时开启 4 个虚拟机，但要做到这一点并不难。

图 6-18 是物理平台内存的分配策略图。本书的例子中，物理平台上的内存区域分为两大部分：host/VMM 所使用的区域，以及 Guest/DMA 所使用的区域，各个 VM 从这个区域里动态分配。

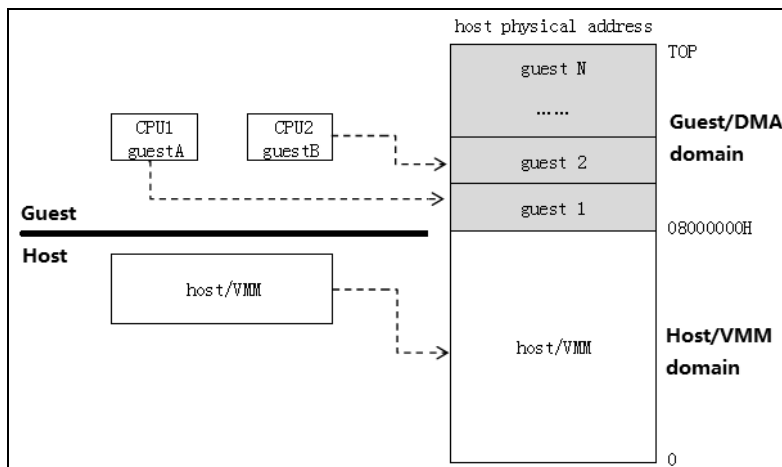


图 6-18

Guest/DMA 域

简单起见，本书例子中以 256MB 系统内存作为参考。每个 guest 域大小为 8M，guest/DMA 域从物理地址 08000000H（128M）开始到最高物理内存。

VMM 按照 VM 的初始化顺序来分配内存。假如 VMM 先对 CPU2 的 guest B 进行初始化，再对 CPU1 的 guest A 进行初始化。那么 CPU2 的 guest B 域物理地址为 08000000H，CPU1 的 guest A 域物理地址为 08800000H。

```
PVOID
vm_alloc_domain (
    VOID
);
```

vm_alloc_domain 函数声明看起来像上面这样，它用来在 domain pool 中分配 guest domain。domain pool 的物理地址基址是 08000000H，记录在 SDA.DomainPhysicalBase 中。虚拟地址基址在 x86 下是 88000000H，在 x64 下是 FFFFFFF800_88000000H，记录在 SDA.DomainBase 中（参见 1.3 节）。

代码片段 6-10:

```
-----
; vm_alloc_domain()
; input;
;     none
; output:
;     eax - 虚拟地址
;     edx - 物理地址
; 描述:
;     1) 在 domain pool 中分配 VM domain 区域
;-----
vm_alloc_domain:
    push ebp
    push ebx
    push ecx

#ifdef __X64
    LoadFsBaseToRbp
#else
    mov ebp, [fs: SDA.Base]
#endif

    mov ebx, DOMAIN_SIZE
    mov edx, ebx
    REX.Wrb
    xadd [ebp + SDA.DomainBase], ebx
    REX.Wrb
    xadd [ebp + SDA.DomainPhysicalBase], edx

    ;;
    ;; 在 x64 下使用 2M 页面，在 x86 下使用 4K 页面
    ;;
#ifdef __X64
    REX.Wrb
    mov esi, ebx
    REX.Wrb
    mov edi, edx
    REX.wrxB
    mov eax, PAGE_2M | PAGE_WRITE | PAGE_P
    REX.wrxB
    mov ecx, (DOMAIN_SIZE / 200000h)
    call do_virtual_address_mapping_n
#else
    REX.Wrb
    mov esi, ebx
    REX.Wrb
    mov edi, edx
```

```

    REX.wrxB
    mov eax, PAGE_WRITE | PAGE_P
    REX.wrxB
    mov ecx, (DOMAIN_SIZE + 0FFFh) / 1000h
    call do_virtual_address_mapping_n
%endif

    REX.wrxB
    mov eax, ebx
    pop ecx
    pop ebx
    pop ebp
    ret

```

DOMAIN_SIZE 值定义为 800000H，从 domain pool 分配后使用 do_virtual_address_mapping_n 函数将 guest domain 映射到系统地址空间里。在 64 位环境下使用 2M 页面，在 32 位环境下使用 4K 页面。

vm_alloc_domain 函数在执行 initialize_vmcs_buffer 函数初始化 VMCS 阶段时调用，分配后的 guest domain 值保存在 VMB.DomainBase（虚拟地址）及 VMB.DomainPhysicalBase（物理地址）中。

6.3.2 实模式 guest OS 内存处理

为了支持 guest OS 运行在实模式环境下，VMX 架构引进了 unrestricted guest 功能（参见 3.5.2.2 节）。secondary processor-based VM-execution control 字段的“unrestricted guest”位为 1 时，允许 guest OS 运行在实模式或者非分页保护模式。

(1) 当 VMCS 的 guest-state 被配置为实模式或者非分页保护模式时，guest OS 所使用的线性地址就是物理地址（EPT 机制下的 guest-physical address）。VM 必须开启 EPT 机制来管理 GPA 转换为 HPA。

(2) 当 guest OS 运行在分页保护模式或者 IA-32e 模式时，VM 可以不必开启 EPT 机制。

当 guest OS 运行在实模式时，guest OS 可能会关闭 A20 地址线来模拟 8086 处理器的 20 位地址。由于在 VMX 模式下不允许关闭 A20 地址线，因此 VMM 必须监控 guest OS 对 A20 gate 的访问。

```

    mov ax, 0FFF0h
    mov es, ax
    mov ax, [es: 0F000h]           ; 尝试访问内存 FFF00h + F000h = 10EF00h

```

当实模式的 guest OS 尝试进行关闭 A20 地址线时，guest OS 的这个动作**被 VMM 接管后忽略**（通过在 I/O bitmap 里禁止访问 92h 端口），实际上 A20 地址线并没有真正被关闭。

在上面的代码中，guest OS 执行“mov ax, [es: 0F000h]”指令尝试访问 **10EF00h** 地址，guest OS 认为 A20 地址线已经关闭，这个地址会回卷到 0EF00h 地址（A20 关闭后的结果）。但是，由于 VMM 接管后 A20 地址线并没有真正关闭，guest OS 会得到

10EF00h 地址中的值。

鉴于这种情况，VMM 必须限制 guest OS 只能访问 20 位地址内的区域。那么，VMM 有如下两种方案实现这个限制。

(1) VMM 只映射 GPA 地址从 0 到 FFFFFh (1M) 的空间。当 guest OS 的 GPA 尝试访问 10EF00h 地址时产生 EPT violation 故障，VMM 经分析发现属于这种情况后，返回 GPA 等于 0EF00h 时的值给 guest OS。

(2) 最简单及有效的处理是：将 GPA 为 100000h 到 10FFFFh 的空间与 00000h 到 0FFFFh 空间映射相同的 HPA 地址。造成 100000h 到 10FFFFh 之间的地址直接回卷到 00000h 到 0FFFFh (如图 6-19 所示)。

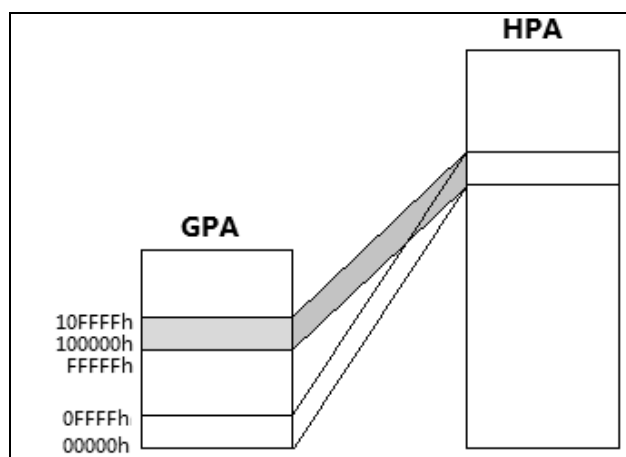


图 6-19

很显然，第 2 种是最佳的解决方案。因为在实模式 16 位代码下，最高的地址值就是 FFFF:FFFFh (等于 10FFEFh 值)。将超过 1M 的 16K 区域与**最低的 16K 区域**映射相同的 HPA 后，VMM 无须插手，GPA 超过 1M 的地址自动转换为 1M 以下对应的 HPA 值。

6.3.3 guest 内存虚拟化

我们知道，通常 OS 都会切换到保护模式或者 IA-32e (long-mode) 模式执行，并且开启分页机制。

- 当启用 EPT 机制后，guest 内的线性地址经过两次地址转换：guest-linear address 转换到 guest-physical address，以及 guest-physical address 转换到 host-physical address。在这种情况下，VMM 可以通过 EPT 机制的 GPA 映射 HPA 来实现 guest 内存的虚拟化。guest OS 可以自由地实施它的内存管理策略，而 VMM 无须插手。
- 当关闭 EPT 机制时，guest 内线性地址转换后的物理地址就是物理平台上的物理地址。VMM 需要非常细致，非常小心地处理每个 guest 之间的内存隔离问题。避免

VM 之间及 VM 与 VMM 之间的相互干扰。

在下面的章节中将讨论 VM 在关闭 EPT 机制后 guest 内存的虚拟化。

6.3.3.1 guest 虚拟地址转换

当 VM 没有启用 EPT 机制时，guest 与 VMM 共用物理地址空间。guest 的线性地址经过转换后也直接映射到平台的物理地址空间。

例如，host/VMM 将一个线性地址 C0400000H 转换到物理地址 400000H，而 guest OS 通过自己的 paging-structure 将另一个线性地址 80200000H 转换到物理地址 400000H。那么，VM 会污染 VMM 的线性地址空间和物理地址空间，造成 VM 与 VMM 互相干扰。因此，VMM 必须避免这样的情况发生。

我们沿用 guest domain 的概念，VMM 为每个 guest 分配独立的物理内存区域。VMM 必须监控 guest OS 对 CR3 寄存器的切换，为 guest 准备一份独立的 paging structure 数据结构，并且在这个独立的 paging structure 中重建 guest OS 的每一个线性地址映射（如图 6-20 所示）。

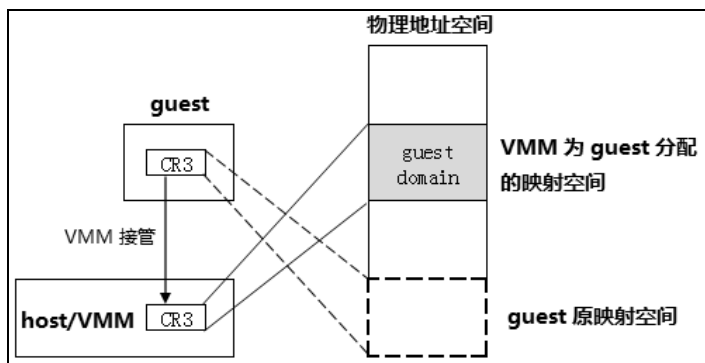


图 6-20

- VMM 接管 guest OS 对 CR3 的更新。将 primary processor-based VM-execution control 字段的“CR3-load exiting”位置为 1，并且将 CR3-target count 设为 0 值，或者在 CR3-target value 中设置为给 guest 而准备的 CR3 值（参见 3.5.8 节）。
- VMM 接管 guest OS 对 CR3 的读取。将 primary processor-based VM-execution control 字段的“CR3-store exiting”位置为 1。

当 guest OS 切换 CR3 寄存器时，由于“CR3-load exiting”而产生 VM-exit。VMM 保存 guest 将要写入的 CR3 寄存器值，并将另一个 CR3 值写入 guest-CR3 字段，然后 VMRESUME 重启 VM 执行。guest OS 并不知道这个 CR3 寄存器值已经被替代。

当 guest OS 读取 CR3 寄存器时，由于“CR3-store exiting”而产生 VM-exit。VMM 返回 guest OS 原来想要切换的 CR3 寄存器值。guest OS 得到了它想要的值，并没有发现什么问题。

重建 guest OS 映射

VMM 接管 guest OS 的 CR3 更新后, 通过 guest 的 CR3 寄存器找到 guest 的所有 paging structure 数据。VMM 可以维护 guest OS 的一份线性映射列表 (也就是 guest OS 进行了哪些线性地址映射), VMM 使用新的 CR3 值替换 guestCR3 的原值, 并且这个替换后的 CR3 所指向的 paging structure 中重建 guest OS 所有的线性地址映射。

注意: VMM 应该为每个 guest 准备独立的 paging structure。如果多个 guest 共用一份 paging structure, 或者 guest 与 host/VMM 共用一份 paging structure, 会同时互相污染线性地址空间与物理地址空间 (这个情况与 OS 的进程切换有相似之处)。

例如, guest OS 将线性地址从 C0000000H 到 C07FFFFFFH 映射到物理地址 200000H 到 9FFFFFFH。那么, VMM 可以在为 guest 准备的 paging structure 中将线性地址从 C0000000H 到 C07FFFFFFH 映射到物理地址从 **200000H+domain 基址**到 **9FFFFFFH+domain 基址**。

分析 guest 虚拟地址映射

VMM 利用读取的 guest CR3 值分析 guest 的虚拟地址映射情况。注意: VMM 需要将 guest 的 domain 区域映射到 host 的系统地址空间中, 使得 VMM 可以读取 guest 的 domain 区域。假设, guest 的 domain 物理基址为 08000000H。guest OS 在进行虚拟地址映射时从 domain 区动态分配物理页面 (即从 08000000H 开始, 08001000H...)。

假设 guest OS 使用 IA-32e 映射模式。guest 的 CR3 值为 200000H (即 guest-paging structure 基址)。其中, guest 虚拟地址 20000H 映射到 guest 物理地址 20000H。

```

[Home] [demo]
[CpuIndex]:1 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Enable
[Host]: launch VM ...
[Host]: Guest OS running ...
[Host]: press <F2> key switch to GUEST screen !
=== dump guest-linear address (000000000020000) ===
[PML4E] : 000000000020003
--> [PML4E] : 0010000003228807
--> [PDPTE] : 0020000003229807 ← 202000H ( GPA ) 的 EPT 表项
--> [PDE] : 0030000003236807
--> [PTE] : 0040000008004837
[PDPTE] : 0000000000205003
--> [PML4E] : 0010000003228807
--> [PDPTE] : 0020000003229807 ← 205000H ( GPA ) 的 EPT 表项
--> [PDE] : 0030000003236807
--> [PTE] : 0040000008007837
[PDE] : 0000000000208003
--> [PML4E] : 0010000003228807
--> [PDPTE] : 0020000003229807 ← 208000H ( GPA ) 的 EPT 表项
--> [PDE] : 0030000003236807
--> [PTE] : 004000000800A837
[PTE] : 000000000020003
--> [PML4E] : 0010000003228807
--> [PDPTE] : 0020000003229807 ← 20000H ( GPA ) 的 EPT 表项
--> [PDE] : 003000000322A807
--> [PTE] : 0040000008001837
  
```

图 6-21

图 6-21 显示的结果是打印 guest 虚拟地址 20000H 转换为 host-physical address 的转换关系, 包括了 guest paging structure 与 EPT paging structure 的表项值。

(1) guest paging structure

- PML4E 提供 PDPT 的物理基址为 202000H（属于 guest-physical address）。
- PDPTE 提供 PDT 的物理基址为 205000H（属于 guest-physical address）。
- PDE 提供 PT 的物理基址为 208000H（属于 guest-physical address）。
- PTE 提供 4K 页面的基址为 20000H（属于 guest-physical address）。

(2) EPT paging structure

- 将 GPA 202000H 转换为 HPA。
 - PML4E 提供 PDPT 的物理基址为 3228000H（属于 host-physical address）。
 - PDPTE 提供 PDT 的物理基址为 3229000H（属于 host-physical address）。
 - PDE 提供 PT 的物理基址为 3236000H（属于 host-physical address）。
 - PTE 提供 4K 页面基址为 08004000H（在 VM domain 区域内）。
- 将 GPA 205000H 转换为 HPA。
 - PML4E 提供 PDPT 的物理基址为 3228000H（属于 host-physical address）。
 - PDPTE 提供 PDT 的物理基址为 3229000H（属于 host-physical address）。
 - PDE 提供 PT 的物理基址为 3236000H（属于 host-physical address）。
 - PTE 提供 4K 页面基址为 08007000H（在 VM domain 区域内）。
- 将 GPA 208000H 转换为 HPA。
 - PML4E 提供 PDPT 的物理基址为 3228000H（属于 host-physical address）。
 - PDPTE 提供 PDT 的物理基址为 3229000H（属于 host-physical address）。
 - PDE 提供 PT 的物理基址为 3236000H（属于 host-physical address）。
 - PTE 提供 4K 页面基址为 0800A000H（在 VM domain 区域内）。
- 将 GPA 20000H 转换为 HPA。
 - PML4E 提供 PDPT 的物理基址为 3228000H（属于 host-physical address）。
 - PDPTE 提供 PDT 的物理基址为 3229000H（属于 host-physical address）。
 - PDE 提供 PT 的物理基址为 322A000H（属于 host-physical address）。
 - PTE 提供 4K 页面基址为 08001000H（在 VM domain 区域内）。

guest OS 的虚拟地址 20000H 转换为 GPA 20000H，而 GPA 20000H 最终转换为 8001000H 物理地址。处理器每经过一个 guest paging structure 表项时，同时也会进行 GPA 到 HPA 的转换（参见 6.1.8.3 节与图 6-11 所示）。

当 guest OS 不使用 EPT 机制时，guest OS 的虚拟地址经过 guest paging structure 转换为最终的物理地址。如前面所述，VMM 必须要接管 guest OS 对 CR3 的更新操作，然后通过 CR3 引伸出来的 paging-structure 进行分析，在 VM 独立的 paging structure 中重建新的虚拟地址映射。

6.3.3.2 guest OS 的 cache 管理

guest OS 会执行一系列的内存管理及 cache 管理操作。在强力监管的策略下，VMM 可能需要接管 guest 的下面这些操作。

- VMM 接管 guest 对 IA32_PAT 与 MTRR 系列寄存器的访问。通过 MSR read bitmap 与 MSR write bitmap 字段的设置达到监控的目的（参见 3.5.15 节）。
- VMM 接管 guest 对 CR0.CD 与 CR0.NW 位的访问。通过设置 CR0 的 guest/host mask 与 shadow 字段达到监控的目的（参见 3.5.7 节）。
- VMM 接管 guest 执行 INVLPG 指令。通过设置 primary processor-based VM-execution control 字段的“INVLPG exiting”位。另外，guest 尝试执行 INVD 指令则无条件产生 VM-exit。

guest OS 通过 MTRR 设置内存类型时，VMM 根据设置的目标内存区域调整到 guest 所属的 domain 区域内达到虚拟化目的。对于 IA32_PAT 的设置，VMM 通过设置 VM-entry control 字段的“load IA32_PAT”位以及 VM-exit control 字段的“save IA32_PAT”与“load IA32_PAT”位，保持 guest 与 host/VMM 的 IA32_PAT 寄存器独立。在这种情况下，VMM 直接允许 guest OS 设置 IA32_PAT。

guest OS 修改 CR0.CD 与 CR0.NW 位，以及执行 INVD 指令是 VMM 虚拟化 cache 的难点之一。CR0.CD 与 CR0.NW 位的修改实时地反映在 guest 与 host/VMM 环境里，因为在进行 VM-entry 及 VM-exit 操作时，CR0.CD 与 CR0.NW 位将保持不变（参见 4.7.1 与 5.13.1 节）。也就是说：Guest OS 对 CR0.CD 与 CR0.NW 的修改，在 host/VMM 环境里也保持有效。同样，host/VMM 对 CR0.CD 与 CR0.NW 的修改，在 guest 环境里也保持有效。

尽管如此，VMM 必须允许 guest OS 的下面动作：

- 允许 guest OS 对 CR0.CD 与 CR0.NW 位的修改。
- 允许 guest OS 执行 INVD 指令刷新 cache。

可以认为：VMM 只有允许 guest OS 的这些操作，才能保证 guest 环境的正确运行。VMM 根据 guest OS 的这些动作来调整 host/VMM 自身的内存管理策略。

VMM 通过 CR0 寄存器对应的 guest/host mask 与 read shadow 字段监控 guest OS 对 CR0.CD 与 CR0.NW 位的访问（参见 3.5.7 节）。

VMM 可以设置 CR0 guest/host mask 字段的 bit 30 与 bit 29 为 1 值（对应 CR0.CD 与 CR0.NW），CR0 read shadow 字段的 bit 30 与 bit 29 为 0 值。当 guest OS 尝试将 CR0.CD 或者 CR0.NW 置 1 时产生 VM-exit，从而被 VMM 捕捉。

VMM 对 CR0.CD 与 CR0.NW 位的处理，如以下 C 伪码所示：

```
DoCR0_CD_NW(int value)
{
    Cr0CdNwMask = value & (CR0_CD | CR0_NW);           // 取要写入的 CR0.CD 与 CR0.NW 值
    Cr0ReadShadow = GetVmcsField(CR0_READ_SHADOW);      // 读取 CR0 read shadow 值

    if (Cr0CdNwMask == CR0_NW)
    {
        /*
         * 当 CR0.CD = 0, CR0.NW = 1 时，产生 #GP(0) 异常！
         * VMM 注入 #GP 异常事件给 guest OS 执行！
         */
    }
}
```

```

        inject_event(GP_EVENT, 0);                // error code = 0
    }
    else
    {
        /*
         * 更新当前的 CR0 值，并重新调整 CR0 read shadow 字段
         */
        do_MovCr0((cr0 & ~(CR0_CD | CR0_NW)) | Cr0CdNwMask);

        Cr0ReadShadow &= ~(CR0_CD | CR0_NW);      // 清原 CR0.CD, CR0.NW
        Cr0ReadShadow |= Cr0CdNwMask;             // 设置新 CR0.CD, CR0.NW

        SetVmcsField(CR0_READ_SHADOW, Cr0ReadShadow);
    }
    ... ..
}

```

VMM 读取 guest OS 要写入 CR0 寄存器的值，检查 CR0.CD 与 CR0.NW 位的值是否符合要求。当 CR0.CD = 0，并且 CR0.NW = 1 时，需要注入 **#GP 异常** 让 guest OS 处理。

符合要求的前提下，VMM 更新当前的 CR0 寄存器值：

- $CR0 \&= \sim(CR0_CD | CR0_NW)$ ，清当前的 CR0.CD 与 CR0.NW 位。
- $CR0 |= Cr0CdNwMask$ ，设置要写入的 CR0.CD 与 CR0.NW 值。

同时，为了继续保持对 CR0.CD 与 CR0.NW 位的监控，VMM 必须更新 CR0 read shadow 字段值：

- $CrReadShadow \&= \sim(CR0_CD | CR0_NW)$ ，清 read shadow 原 CR0.CD 与 CR0.NW 位。
- $CrReadShadow |= Cr0CdNwMask$ ，设置 read shadow 新的 CR0.CD 与 CR0.NW 位。

这样，当 guest OS 尝试再次更新 CR0.CD 或 CR0.NW 值时，VMM 再一次捕捉 guest OS 这个动作。VMM 需要评估设置 CR0.CD 或 CR0.NW 后对 host/VMM 及其他 guest 的影响。在保证 guest OS 运行正确的前提下，尽量减少对 host/VMM 的干扰或者做一些补偿措施。

另外，guest OS 尝试执行 INVD 指令时产生 VM-exit。VMM 应该完成执行 INVD 指令来刷新 cache，可是，这个动作也将影响到 host/VMM。更重要的是它会影响同一个逻辑处理器下管理的其他 VM。但鉴于不刷新 cache 会令 guest OS 的行为不正确，因此，由 VMM 完成刷新 cache 工作似乎是必需的。

在捕捉 guest OS 执行的 INVLPG 指令后，VMM 可以使用 INVVPID 指令来模拟执行 INVLPG 指令，完成 cache 的刷新。

INVVPID 描述符使用下面的参数：

- 线性地址，从 exit qualification 字段里读取 INVLPG 指令所需要刷新的线性地址。
- VPID，从 virtual-processor identifier 字段里读取 VPID 值。

VMM 使用 Individual-address invalidation (0 值) 刷新类型执行 INVVPID 指令。

6.4 例子 6-1

示例 6-1: 使用实模式的 guest, 并启用 EPT 机制

本节将构建一个完整的实验例子。对这个例子不感兴趣的读者, 可以跳过这一节直接阅读第 7 章。这个例子实现下面的功能。

- (1) 一个逻辑处理器里开启两个 VM, 并且模拟软件切换在两个 VM 之间进行切换。
- (2) 每个 guest OS 模拟从实模式 boot 启动开始, 到进入 OS kernel 阶段 (32 位或 64 位模式)。
- (3) 每个 VM 都使用独立的 EPT paging structure 对 guest 进行内存虚拟化管理。

在首次发起 VM-entry 时, 我们的 guest 进入实模式, 然后在 guest 的运行过程中切换到保护模式或者 IA-32e 模式 (取决于是否定义了符号 GUEST_X64)。与例子相关的代码在 chap06\ex6-1\目录下。

编译代码

进入 chap06\ex6-1 目录下, 可以执行下面几种形式的编译命令。

- **build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64 -DGUEST_X64**, 这个命令将代码编译为 64 位, 并且 guest 也使用 64 位环境。
- **build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64**, 这个命令将代码编译为 64 位, 但 guest 使用 32 位环境。
- **build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE**, 这个命令将代码编译为 32 位, 当然 guest 也只能使用 32 位环境。

为了观察执行流程, 所以需要开启调试记录功能, 并且需要开启 guest 支持 (GUEST_ENABLE)。

```

e:\x86-II\chap06\ex6-1>build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64 -DGUEST_X64
..\lib\Vm\VmInit.asm:1178: warning: dword data exceeds bounds
<All rights reserved! DengZhi, Bug: mik@mouseos.com>
entry #0:      ..\..\common\boot ---> c.img:      success
entry #1:      ..\..\common\setup ---> c.img:      success
entry #2:      ..\..\common\long ---> c.img:      success
entry #3:      ..\..\lib\Guest\GuestBoot ---> c.img:   success
entry #4:      ..\..\lib\Guest\GuestKernel ---> c.img:  success
entry #5:      ..\..\common\boot ---> demo.img:    success
entry #6:      ..\..\common\setup ---> demo.img:    success
entry #7:      ..\..\common\long ---> demo.img:    success
entry #8:      ..\..\lib\Guest\GuestBoot ---> demo.img:  success
entry #9:      ..\..\lib\Guest\GuestKernel ---> demo.img: success
<Write Error>: \.\g:, 系统找不到指定的文件。
  
```

图 6-22

lib\Vm\VmInit.asm 文件内有一处警告信息, 是由 “and eax, ~(CR0_PG | CR0_PE)” 这条指令引起的。nasm 认为将这个源操作数取反后将扩展为超过 32 位的数, 这里可以忽

略这个警告。这里并没有插入 U 盘，因此显示找不到\\.\g:文件。

这里新增了一个 GUEST_ENABLE 符号定义，目的是加入对 GuestBoot 与 GuestKernel 模块的编译，并写入映像文件（c.img 与 demo.img）。另外，定义 GUEST_X64 符号的目的是将 guest 编译并生成 64 位环境。

6.4.1 GuestBoot 模块

我们的 guest OS 的运行分两个阶段：boot 阶段及 kernel 阶段。分别对应 GuestBoot 模块与 GuestKernel 模块，它们实现在\lib\guest 目录下。

这个例子中，我们将模拟 guest OS 从磁盘加载到内存开始，也就是模拟 GuestBoot 模块被 INT 19h 加载到 7C00H 位置上。例子中，将 guest RIP 字段的值设置为 7C00H + 4（由于 GuestBoot 模块被编译后首 DWORD 存放着模块的长度，所以需要加上 4）。

代码片段 6-11:

```
;;
;; 注意:
;; 1) 现在处理器处于 real mode 下
;; 2) 模拟 GuestBoot 已经被加载到 7C00h 位置上, GUEST_BOOT_ENTRY 定义为 7C00h
;;

[SECTION .text]
org GUEST_BOOT_ENTRY
dd GUEST_BOOT_LENGTH                ;; GuestBoot 模块长度

bits 16

GuestBoot.Start:
cli
NMI_DISABLE                        ; 关闭 NMI
FAST_A20_ENABLE                    ; 开启 A20 位

; set BOOT_SEG environment
mov ax, cs
mov ds, ax
mov ss, ax
mov es, ax
mov sp, GUEST_BOOT_ENTRY          ; 设 stack 底为
GUEST_BOOT_ENTRY

; *****
; * 下面切换到保护模式 *
; *****

lgdt [Guest.GdtPointer]           ; 加载 GDT
lidt [Guest.IdtPointer]           ; 加载 IDT

;;
;; 设置 TSS
;;
mov WORD [tss_desc], 67h
```



```

mov WORD [tss_desc + 2], Guest.Tss
mov BYTE [tss_desc + 5], 89h

mov eax, cr0
bts eax, 0 ; CR0.PE = 1
mov cr0, eax

jmp GuestKernelCs32 : GuestBoot.Entry32

;;
;; 以下是 32 位 protected 模式代码
;;

bits 32

GuestBoot.Entry32:
mov ax, GuestKernelSs32 ; 设置 data segment
mov ds, ax
mov es, ax
mov ss, ax

;;
;; 加载 TSS
;;
mov ax, GuestKernelTss
ltr ax

;;
;; 下面转入 GuestKernel 模块, 入口在 GUEST_KERNEL_ENTRY + 4
;;
jmp GUEST_KERNEL_ENTRY + 4

[SECTION .data]
;;
;; Guest 模块的 GDT
;;
Guest.Gdt:
null_desc dq 0 ; NULL descriptor
kernel_code64_desc dq 0x0020980000000000 ; DPL=0, L=1
kernel_data64_desc dq 0x0000920000000000 ; DPL=0
user_code32_desc dq 0x00cff8000000ffff ; non-conforming, DPL=3, P=1
user_data32_desc dq 0x00cff2000000ffff ; DPL=3, P=1, writeable, expand-up
user_code64_desc dq 0x0020f80000000000 ; DPL = 3
user_data64_desc dq 0x0000f20000000000 ; DPL = 3
kernel_code32_desc dq 0x00cf9a000000ffff ; non-conforming, DPL=0, P=1
kernel_data32_desc dq 0x00cf92000000ffff ; DPL=0, P=1, writeable, expand-up
tss_desc dq 0 ; TSS
Guest.Gdt.End:

;;
;; Guest 模块的 IDT
;;
Guest.Idt:

```

```

        times 256      dq 0                ; 保留 256 个 vector
Guest.Idt.End:

;;
;; Guest 模块的 TSS
;;
Guest.Tss:
        dd 0
        dd 7FFFh                ; esp0
        dd GuestKernelss32      ; ss0
        dq 0                    ; ss1/esp1
        dq 0                    ; ss2/esp2
        dq 0                    ; reserved
        dq 0FFFF8000FFF008F0h   ; IST1
        times 17 dd 0
        dw 0
        dw 0                    ; I/O permission bitmap offset = 0
Guest.Tss.End:

;;
;; Guest 模块的 Gdt pointer
;;
Guest.GdtPointer:
gdt_limit      dw      (Guest.Gdt.End - Guest.Gdt) - 1
gdt_base       dd      Guest.Gdt

;;
;; Guest 模块的 Idt pointer
;;
Guest.IdtPointer:
idt_limit      dw      (Guest.Idt.End - Guest.Idt) - 1
idt_base       dd      Guest.Idt

GUEST_BOOT_LENGTH      EQU      $ - GUEST_BOOT_ENTRY

```

这个 GuestBoot 模块非常简单，只是象征性地实现了实模式模块。GuestBoot 模块最后切换到未分页的保护模式，设置了保护模式的基本环境。

GuestBoot 模块在系统 boot 阶段被加载到 GUEST_BOOT_SEGMENT 区域（值为 10000H，定义在 inc\support.inc 文件中），入口地址是 GUEST_BOOT_ENTRY（值为 7C00H，定义在 lib\guest\guest.inc 文件中）。

加载 GuestBoot 模块

在初始化 guestA 阶段，为了模拟 GuestBoot 被加载到 7C00H，VMM 对 GuestBoot 的加载处理如下面的代码所示。

代码片段 6-12:

```

;;
;; step 1: 将 GuestBoot 模块安装到 domain
;;

```

```

mov ecx, [GUEST_BOOT_SEGMENT]
add ecx, 0C00h + 0FFFh
shr ecx, 12
mov esi, ecx
call vm_alloc_pool_physical_page      ;; 分配 n 个 4K domain 页面给 GuestBoot 模块
mov rdx, rax

;;
;; 将 GuestBoot 模块入口映射到 VM domain
;;
mov esi, GUEST_BOOT_ENTRY            ;; rsi - guest physical address
mov rdi, rdx                         ;; rdi - host physical address
mov eax, EPT_READ | EPT_WRITE | EPT_EXECUTE ;; eax - page attribute
                                           ;; ecx - count of page
call do_guest_physical_address_mapping_n ;; 将 GuestBoot 模块入口映射到
domain

;;
;; 将 GuestBoot 模块复制到 VM domain
;;
mov esi, GUEST_BOOT_SEGMENT          ;; GuestBoot
mov rdi, SYSTEM_DATA_SPACE_BASE
or rdi, rdx                          ;; VM domain
add rdi, 0C00h
mov r8d, [GUEST_BOOT_SEGMENT]
call memcpy

```

这段代码摘自 `init_guest_a` 函数（位于 `chap06\ex6-1\ex.asm` 文件内）。它的处理如下。

- (1) 为 GuestBoot 模块在 guest 自己的 VM domain 里分配相应的区域（参见 6.3.1 节）。
- (2) 使用 `do_guest_physical_address_mapping_n` 函数将 7C00H（GUEST_BOOT_ENTRY）映射到为 GuestBoot 模块分配的区域。这个 7C00H 属于 guest-physical address。
- (3) 从 GUEST_BOOT_SEGMENT 里将 GuestBoot 模块复制到为 GuestBoot 模块分配的区域。

在映射 7C00H 地址时，使用了所有权限（read/write/execute）。在复制 GuestBoot 模块时，目标地址是 host 的虚拟地址，值等于 SYSTEM_DATA_SPACE 加上 GuestBoot 模块的 domain 地址。

假设为 GuestBoot 模块分配的 domain 地址值为 08000000H。在 x64 下，GuestBoot 模块 domain 在系统中的虚拟地址就是 `FFFFF800_80000000 + 08000000H = FFFFF800_88000000H`。依此类推，如果为 guest 分配的一块 domain 地址为 08005000H，则对应的 host 虚拟地址为 `FFFFF800_88005000H`。

VMM 就是通过 guest domain 对应的 host 虚拟地址来访问 guest 的内容（参见 6.3.3.1 节中对 guest paging structure 的访问）。

6.4.2 GuestKernel 模块

例子中，GuestKernel 模块在系统 boot 阶段被加载到 GUEST_KERNEL_SEGMENT

区域（值为 11000H，定义在 inc\support.inc 文件中）。

代码片段 6-13:

```
[SECTION .text]
org GUEST_KERNEL_ENTRY
dd GUEST_KERNEL_LENGTH

;;
;; 当前 guest 已经位于未分页的 protected 模式下
;;
bits 32

GuestKernel.Start:

mov esi, Guest.StartMsg
call PutStr

mov eax, cr4
or eax, CR4_PAE
mov cr4, eax                ; 开启 CR4.PAE

;;
;; 根据 GUEST_X64 符号来决定 guest 进入 IA-32e 还是 protected 模式
;;

#ifdef GUEST_X64
;;
;; 初始化 long mode 模式页表
;;
call init_longmode_page

mov eax, GUEST_PML4T_BASE
mov cr3, eax

;;
;; 开启 long mode
;;
mov ecx, IA32_EFER
rdmsr
or eax, EFER_LME
wrmsr

;;
;; 开启分页
;;
mov eax, cr0
or eax, CR0_PG
mov cr0, eax

jmp GuestKernelCs64 : GuestKernel.@0

GuestKernel.@0:

bits 64

mov ax, GuestKernelSs64
mov ds, ax
mov es, ax
```

```

mov ss, ax
mov rsp, 0FFFF8000FFF00FF0h

mov rax, (0FFFF800080000000h + GuestKernel.Next)
jmp rax

%else

;;-----
;; 32 位保护模式
;;-----

call init_pae_page

mov eax, Guest.Pdpt
mov cr3, eax

;;
;; 开启分页
;;
mov eax, cr0
or eax, CR0_PG
mov cr0, eax

mov esp, 0FFF00FF0h
mov eax, (80000000h + GuestKernel.Next)
jmp eax

%endif

;;#####
;;    guest kernel
;;#####

GuestKernel.Next:
;;
;; 输出信息
;;
mov esi, Guest.DoneMsg
call PutStr
mov esi, Guest.RunMsg
call PutStr
mov esi, Guest.TscMsg
call PutStr
rdtsc
mov esi, eax
mov edi, edx
call PrintQword

;;
;; guest_ex.asm 是 guest 的示例文件
;;

```

```
%include "guest_ex.asm"

jmp $

%include "..\..\lib\Guest\GuestLib.asm"
%include "..\..\lib\Guest\GuestCrt.asm"

;#####
;;
;;
;;      Guest 的数据区域
;;
;;
;#####

[SECTION .data]

Guest.PoolPhysicalBase      DD      GUEST_POOL_PHYSICAL_BASE
;;
;; 处理器状态标志位
;;
Guest.ProcessorFlag         DD      0

Guest.VideoBufferPtr        DQ      0B8000h

%ifndef GUEST_X64
;;
;; 下面是 PAE paging 下的 PDPT 表
;; 1) 每个 PDPTE 为 8 字节
;;
ALIGNB 32
Guest.Pdpt                  DQ      GUEST_PDT0_BASE | P
                             DQ      GUEST_PDT1_BASE | P
                             DQ      GUEST_PDT2_BASE | P
                             DQ      GUEST_PDT3_BASE | P

%endif

;;
;; 信息
;;
Guest.StartMsg               db      '[OS]: start ...', 10, 0
Guest.DoneMsg                 db      '[OS]: initialize done ...', 10, 0
Guest.RunMsg                  db      '[OS]: running ...', 10, 0
Guest.TscMsg                  db      '[OS]: TSC = ', 0

GUEST_KERNEL_LENGTH         EQU     $ - GUEST_KERNEL_ENTRY
```

出于实验目的，GuestKernel 模块也非常简单，主要的工作是开启分页机制。当定义了 GUEST_X64 符号时调用 `init_longmode_page` 函数初始化页表结构，并进入 IA-32e 模

式 (long-mode)，否则调用 `init_pae_page` 函数初始化 PAE 页表结构，并进入 32 位保护模式。

这个模块包含的 `guest_ex.asm` 文件是每个 `guest` 端的示例文件，用于不同的例子示范，和 `ex.asm` 文件的作用相同。

Guest 的分页机制

当 `host/VMM` 处于 IA-32e 模式时，取决于 `GUEST_X64` 符号的定义，`guest OS` 允许使用 64 位或者 32 位环境。以 64 位环境为例，下面是 IA-32e 分页模式的初始化。

代码片段 6-14:

```

;-----
; init_longmode_page()
; input:
;     none
; output:
;     none
; 描述:
;     1) 在未分页下调用
;-----
init_longmode_page:
    push ebx
    push edx
    push ecx
    push ebp

    ;;
    ;; ----- 虚拟地址 -----          ----- 物理地址 -----
    ;; 1) FFFF8000_C0200000h - FFFF8000_C05FFFFFh ==> 00200000h - 005FFFFFh (2M 页面)
    ;; 2) FFFF8000_80020000h - FFFF8000_8002FFFFh ==> 00020000h - 0002FFFFh (4K 页面)
    ;; 3) 00020000h - 0002FFFFh ==> 00020000h - 0002FFFFh (4K 页面)
    ;; 4) FFFF8000_FFF00000h - FFFF8000_FFF0FFFFh ==> AllocPhysicalPage() (4K 页面)
    ;; 5) 000B8000h - 000B8FFFh ==> 000B8000h - 000B8FFFh (4K 页面)
    ;; 6) 00007000h - 00008FFFh ==> 00007000h - 00008FFFh (4K 页面)
    ;; 7) FFFF8000_81000000h - FFFF8000_8100FFFFh ==> 01000000h - 0100FFFFh (4K 页面)
    ;;

    ;;
    ;; #### step 1: 设置 PML4E ####
    ;;

    ;;
    ;; 设置 FFFF8000_xxxxxxxx 的 PML4E
    ;;
    mov esi, 1
    call AllocPhysicalPage
    mov ebx, eax                ;; ebx = PML4T[100h]
    or eax, RW | P
    mov [GUEST_PML4T_BASE + 100h * 8], eax                ;; PML4T[100h]
    mov DWORD [GUEST_PML4T_BASE + 100h * 8 + 4], 0

    ;;
    ;; 设置 00000000_xxxxxxxx 的 PML4E
    ;;

```

```

mov esi, 1
call AllocPhysicalPage
mov edx, eax                ;; edx = PML4T[0]
or eax, RW | US | P
mov [GUEST_PML4T_BASE + 0 * 8], eax                ;; PML4T[0]
mov DWORD [GUEST_PML4T_BASE + 0 * 8 + 4], 0

;;
;; ##### step 2: 设置 PDPTE #####
;;

;;
;; 设置 FFFF8000_Cxxxxxxx 的 PDPTE
;; 设置 FFFF8000_8xxxxxxx 的 PDPTE
;; 设置 FFFF8000_Fxxxxxxx 的 PDPTE
;;
mov esi, 1
call AllocPhysicalPage
mov ebp, eax                ;; ebp
or eax, RW | P
mov [ebx + 2 * 8], eax
mov DWORD [ebx + 2 * 8 + 4], 0

mov esi, 1
call AllocPhysicalPage
mov ecx, eax
or eax, RW | P
mov [ebx + 3 * 8], eax
mov DWORD [ebx + 3 * 8 + 4], 0

;;
;; 设置 00000000_0xxxxxxx 的 PDPTE
;;
mov esi, 1
call AllocPhysicalPage
mov ebx, eax                ;; ebx
or eax, RW | US | P
mov [edx + 0 * 8], eax
mov DWORD [edx + 0 * 8 + 4], 0

;;
;; ##### step 3: 设置 PDE #####
;;

;;
;; 设置 FFFF8000_C02xxxxx 的 PDE
;;
mov DWORD [ecx + 1 * 8], 200000h | PS | RW | P
mov DWORD [ecx + 1 * 8 + 4], 0
mov DWORD [ecx + 2 * 8], 400000h | PS | RW | P
mov DWORD [ecx + 2 * 8 + 4], 0

;;
;; 设置 FFFF8000_FFFxxxxx 的 PDE
;;
mov esi, 1
call AllocPhysicalPage

```



```

mov edi, eax
or eax, RW | P
mov [ecx + 1FFh * 8], eax
mov DWORD [ecx + 1FFh * 8 + 4], 0
mov ecx, edi

;;
;; 设置 FFFF8000_8002xxxx 的 PDE
;;
mov esi, 1
call AllocPhysicalPage
mov edx, eax                ;; edx
or eax, RW | P
mov [ebp + 0 * 8], eax
mov DWORD [ebp + 0 * 8 + 4], 0

;;
;; 设置 FFFF8000_810xxxxx 的 PDE
;;
mov esi, 1
call AllocPhysicalPage
or eax, RW | P
mov [ebp + 8 * 8], eax
mov DWORD [ebp + 8 * 8 + 4], 0

;;
;; 设置 FFFF8000_81000000h 映射的 PTE
;;
and eax, ~0FFFh
mov DWORD [eax + 0 * 8], 01000000h | RW | P
mov DWORD [eax + 0 * 8 + 4], 0

;;
;; 设置 00000000_00020xxx 的 PDE
;;
mov esi, 1
call AllocPhysicalPage
mov ebp, eax
or eax, RW | US | P
mov [ebx + 0 * 8], eax
mov DWORD [ebx + 0 * 8 + 4], 0

;;
;; #### step 4: 设置 PTE ####
;;

;;
;; 设置 FFFF8000_FFF00000 的 PTE
;;
mov esi, 1
call AllocPhysicalPage
or eax, RW | P
mov [ecx + 100h * 8], eax
mov DWORD [ecx + 100h * 8 + 4], 0

;;
;; 设置 FFFF8000_80020000h, 00020000h 的 PTE
;;
mov ecx, 20h

```

```

    mov esi, 20000h | RW | P
init_longmode_page.loop:
    mov [ebp + ecx * 8], esi
    mov DWORD [ebp + ecx * 8 + 4], 0
    or DWORD [ebp + ecx * 8], PAGE_USER ; 20000h 具有 USER 权限
    mov [edx + ecx * 8], esi
    mov DWORD [edx + ecx * 8 + 4], 0
    add esi, 1000h
    inc ecx
    cmp ecx, 2Fh
    jbe init_longmode_page.loop

;;
;; 设置 0B8000h 的 PTE
;;
mov DWORD [ebp + 0B8h * 8], 0B8000h | RW | US | P
mov DWORD [ebp + 0B8h * 8 + 4], 0

;;
;; 设置 7000h - 8FFFh 的 PTE
;;
mov DWORD [ebp + 7 * 8], 7000h | RW | US | P
mov DWORD [ebp + 7 * 8 + 4], 0
mov DWORD [ebp + 8 * 8], 8000h | RW | US | P
mov DWORD [ebp + 8 * 8 + 4], 0

pop ebp
pop ecx
pop edx
pop ebx
ret

```

作为示例，这个 `init_longmode_page` 函数只是使用“硬编码”方式简单地逐个设置 paging structure 表项值（实现在 `lib\guest\GuestLib.asm` 文件里）。`GUEST_PML4T_BASE` 的值为 **200000H**，它将被加载到 CR3 寄存器中。

下面以线性地址 `FFFF8000_C0200000H` 与 `20000H` 为例，它们的转换关系如图 6-23 所示。

```

=== dump guest-linear address (FFFF8000C0200000) ===
[PML4E] : 0000000000201003
[PDPTE] : 0000000000204003
[PDE]   : 0000000000200083
=== dump guest-linear address (0000000000200000) ===
[PML4E] : 0000000000202003
[PDPTE] : 0000000000205003
[PDE]   : 0000000000208003
[PTE]   : 000000000020003

```

图 6-23

如图 6-23 中所示，线性地址 `FFFF8000_C0200000H` 使用 2M 页面映射到 GPA `200000H` 上，线性地址 `20000H` 使用 4K 页面映射到 GPA `20000H` 上。

在 EPT 机制下，guest paging structure 表项的值都是 GPA 地址值，它们使用 guest OS 内的 `AllocPhysicalPage` 函数来分配。

图 6-24 所显示的结果是对 GPA 20000H 进行深度的 dump 行为。每个 GPA 都必须转换为 HPA 后才能访问内存，GPA 20000H 最终被转换为物理地址 08001000H。每个 EPT paging structure 表项内的地址值为 HPA。其中 PML4E，PDPTE 及 PDE 内的物理地址从 Kernel Pool 里分配（从 32x_xxxx 开始）。PTE 内的物理地址从 VM domain 内分配（从 0800_xxxx 开始）。

```

[Home] [demo]
[CpuIndex]:1 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Enable
[Host]: launch VM ...
[Host]: Guest OS running ...
[Host]: press <F2> key switch to GUEST screen !
=== dump guest-linear address (00000000020000) ===
[PML4E] : 00000000020003
----> [PML4E] : 0010000003228807
----> [PDPTE] : 0020000003229807 ← 202000H (GPA) 的 EPT 表项
----> [PDE] : 0030000003236807
----> [PTE] : 0040000008080837
[PDPTE] : 000000000205003
----> [PML4E] : 0010000003228807
----> [PDPTE] : 0020000003229807 ← 205000H (GPA) 的 EPT 表项
----> [PDE] : 0030000003236807
----> [PTE] : 0040000008080837
[PDE] : 000000000208003
----> [PML4E] : 0010000003228807
----> [PDPTE] : 0020000003229807 ← 208000H (GPA) 的 EPT 表项
----> [PDE] : 0030000003236807
----> [PTE] : 0040000008080837
[PTE] : 00000000020003
----> [PML4E] : 0010000003228807
----> [PDPTE] : 0020000003229807 ← 20000H (GPA) 的 EPT 表项
----> [PDE] : 0030000003229807
----> [PTE] : 0040000008081837

```

图 6-24

加载 GuestKernel 模块

在 `init_guest_a` 和 `init_guest_b` 函数里，分别对 `GuestBoot` 与 `GuestKernel` 模块进行加载，模拟当 guest OS 被加载到系统空间时的情形（6.4.1 节已经介绍了 `GuestBoot` 模块的加载）。

代码片段 6-15:

```

;;
;; step 2: 将 GuestKernel 模块安装到 domain 中
;;
mov ecx, [GUEST_KERNEL_SEGMENT]
add ecx, 0FFFh
shr ecx, 12
mov esi, ecx
call vm_alloc_pool_physical_page      ;; 分配 domain 内存用来容纳 GuestKernel 模块
mov rdx, rax

;;
;; 将 GuestKernel 模块入口映射到 VM domain
;;
mov esi, GUEST_KERNEL_ENTRY
mov rdi, rdx
mov eax, EPT_READ | EPT_WRITE | EPT_EXECUTE
call do_guest_physical_address_mapping_n      ;; 将 GuestKernel 模块入口映射到
domain 内存
;;

```

```
;; 将 GuestKernel 模块复制到 VM domain
;;
mov esi, GUEST_KERNEL_SEGMENT          ;; GuestKernel
mov rdi, SYSTEM_DATA_SPACE_BASE        ;; VM domain
or rdi, rdx
mov r8d, [GUEST_KERNEL_SEGMENT]
call memcpy
```

这段代码摘自 `init_guest_a` 函数（位于 `chap06\ex6-1\ex.asm` 文件内）。它的处理如下。

- ① 在 VM domain 区域内分配物理页面来供 GuestKernel 模块使用。
- ② 将 GuestKernel 模块的入口（GPA 20000H）映射到已分配的 domain 物理页面。
- ③ 从 GUEST_KERNEL_SEGMENT（11000H）中复制到已分配的 domain 物理页面上。

在复制 GuestKernel 模块时，同样使用 host 虚拟地址值 **FFFFFF800_80000000H** 加上已分配的物理页面基址（参见 6.4.1 节的 GuestBoot 模块加载）。

6.4.3 VSB 结构

每个 VM 有独立的 VSB（VM Store Block，虚拟机存储块）区域，每个 VSB 区域大小为 8K。由 VMB 内的 `VsbBase` 指针指向它。

VSB 结构定义在 `\inc\vmcs.inc` 文件中，如下。

代码片段 6-16:

```
struc VSB
    .Base                RESQ    1
    .PhysicalBase        RESQ    1

    ;;
    ;; VM video buffer 管理记录
    ;;
    .VmVideoBufferHead    RESQ    1
    .VmVideoBufferPtr     RESQ    1
    .VmVideoBufferLastChar RESD    1
    .VmVideoBufferSize    RESD    1

    ;;
    ;; VM keyboard buffer 管理记录
    ;;
    ALIGNB 8
    .VmKeyBufferHead      RESQ    1
    .VmKeyBufferPtr       RESQ    1
    .VmKeyBufferSize      RESD    1

    ;;
    ;; guest 的 context 信息
    ;;
    .Context:
    .Rax                  RESQ    1
    .Rcx                  RESQ    1
```

```

.Rdx                RESQ        1
.Rbx                RESQ        1
.Rsp                RESQ        1
.Rbp                RESQ        1
.Rsi                RESQ        1
.Rdi                RESQ        1
.R8                 RESQ        1
.R9                 RESQ        1
.R10                RESQ        1
.R11                RESQ        1
.R12                RESQ        1
.R13                RESQ        1
.R14                RESQ        1
.R15                RESQ        1
.Rip                RESQ        1
.RFlags             RESQ        1

;;
;; FPU 单元 context
;;
ALIGNB 16
.FpuStateImage:
.FpuEnvironmentImage: RESB      28
.FpuStackImage:       RESB      80

;;
;; XMM image 区域, 用于 FXSAVE/FXRSTOR 指令 (512 字节)
;;
ALIGNB 16
.XMMStateImage:       RESB      512

;;
;; VM keyboard buffer 区域, 共 256 个字节, 保存按键扫描码
;;
.VmKeyBuffer          RESB      256
.Reserve              RESB      4096 - $

;;
;; VM video buffer 区域, 存放处理器的屏幕信息
;; 4K, 存放约 25 × 80 × 2 信息
;;
.VmVideoBuffer        RESB      (4096 * 2 - $)

VM_STORAGE_BLOCK_SIZE EQU      $
VSB_SIZE              EQU      $
endstruc

```

VSB 区域用来存储每个 guest 的 context 信息, 并且作为 VM 的 video buffer 及键盘 buffer。

初始化 VSB 区域

在调用 `initialize_vmcs_buffer` 函数初始化 VMCS 阶段, 使用 `init_vm_storage_block` 函数来初始化 VSB 区域。

代码片段 6-17:

```

;-----
; init_vm_storage_block()
; input:
;     esi - VMB pointer
; output:
;     none
; 描述:
;     1) 初始化 VM 私有存储区域
;-----
init_vm_storage_block:
    push ebx
    push edx

    REX.Wrb
    mov ebx, esi

    ;;
    ;; 分配 VSB (VM storage block) 区域
    ;;
    mov esi, ((VSB_SIZE + 0FFFh) / 1000h)
    call alloc_kernel_pool_n
    REX.Wrb
    mov [ebx + VMB.VsbBase], eax                ; edx:eax = PA:VA
    REX.Wrb
    mov [ebx + VMB.VsbPhysicalBase], edx

    ;;
    ;; 初始化 VSB 管理记录
    ;;
    REX.Wrb
    mov [eax + VSB.Base], eax
    REX.Wrb
    mov [eax + VSB.PhysicalBase], edx

    ;;
    ;; 初始化 VM video buffer 管理记录
    ;;
    REX.Wrb
    lea esi, [eax + VSB.VmVideoBuffer]
    REX.Wrb
    mov [eax + VSB.VmVideoBufferHead], esi
    REX.Wrb
    mov [eax + VSB.VmVideoBufferPtr], esi

    ;;
    ;; 初始化 VM keyboard buffer 管理记录
    ;;
    REX.Wrb
    lea esi, [eax + VSB.VmKeyBuffer]
    REX.Wrb
    mov [eax + VSB.VmKeyBufferHead], esi
    REX.Wrb
    mov [eax + VSB.VmKeyBufferPtr], esi
    mov DWORD [eax + VSB.VmKeyBufferSize], 256

    ;;
    ;; 更新处理器状态
    ;;

```

```

mov eax, PCB.ProcessorStatus
or DWORD [gs: eax], CPU_STATUS_GUEST

pop edx
pop ebx
ret

```

init_vm_store_block 函数完成的工作如下。

(1) 使用 alloc_kernel_pool_n 函数在 kernel pool 内分配若干页面作为 VSB 区域，将地址值保存在 VMB.VsbBase 值中。

(2) 初始化 VM video buffer 与 keyboard buffer 管理记录。

(3) 加上处理器状态 CPU_STATUS_GUEST，表明 VM 拥有 VSB 区域。

保存与恢复 guest context 信息

VSB 区域存在 context buffer，可以保存处理器当前的通用寄存器组，x87 单元与 MMX 寄存器组，以及 XMM/YMM 寄存器组。每个 VM 拥有私有 context buffer。

每当发生 VM-exit 时，VMM 使用 store_guest_context 函数在 VSB 区域中保存 guest 环境信息。在下次 VM-entry 操作前，VMM 调用 restore_guest_context 函数将保存在 VSB 区域内的数据恢复为 guest 当前的环境信息。

guest 屏幕

当 host 与 guest 端的代码需要打印输出信息时使用私有的 video buffer，当切换屏幕时将 host 或 guest 私有的 video buffer 刷新到物理屏幕。Host 端使用 LSB (Local Storage Block, 本地存储块) 区域内的 video buffer，而 guest 端则使用 VSB (VM Storage Block, 虚拟机存储块) 区域内的 video buffer。

代码片段 6-18:

```

;;
;; step 3: 将 B8000h 映射到 VM video buffer
;;
mov esi, 0B8000h
mov rdi, [gs: PCB.CurrentVmbPointer]
mov rdi, [rdi + VMB.VsbPhysicalBase]
add rdi, VSB.VmVideoBuffer
mov eax, EPT_READ | EPT_WRITE | EPT_EXECUTE
call do_guest_physical_address_mapping

```

在 init_guest_a 与 init_guest_b 函数初始化阶段，将 guest 的物理地址 B8000H 映射到 VSB 区域内的 video buffer。这样，当 guest 打印时直接写入 VM 私有的 video buffer。

6.4.4 VMM 初始化 guest

例子中的主体文件是 ex.asm，位于 chap06\ex6-1 目录下。ex.asm 文件内的 init_guest_a 函数负责对 guest A 进行初始化，init_guest_b 函数负责对 guest B 进行初始化。TargetCpuVmentry 函数负责从 ready 队列中取出 guest，切换到 VM 执行。

初始化 VM 的 VMCS 区域

init_guest_a 与 init_guest_b 执行完全相同的工作，下面是 init_guest_a 函数关于 VMCS 初始化工作的部分代码。

代码片段 6-19:

```
mov R5, [gs: PCB.Base]

;;
;; guest 启用 "unrestricted guest" 以及 "enable EPT"
;;
mov eax, GUEST_FLAG_UNRESTRICTED | GUEST_FLAG_EPT
mov [R5 + PCB.GuestA + VMB.GuestFlags], eax

;;
;; 设置 guest RIP 以及 host-RIP
;;
mov DWORD [R5 + PCB.GuestA + VMB.GuestEntry], GUEST_BOOT_ENTRY + 4
mov DWORD [R5 + PCB.GuestA + VMB.HostEntry], VmmEntry

;;
;; 初始化 VMCS buffer
;;
mov R6, [R5 + PCB.VmcsA]
call initialize_vmcs_buffer

;;
;; 执行 VMCLEAR 操作
;;
vmclear [R5 + PCB.GuestA]
jc TargetCpuVmentry.Failure
jz TargetCpuVmentry.Failure

;;
;; 加载 VMCS pointer
;;
vmptlrd [R5 + PCB.GuestA]
jc TargetCpuVmentry.Failure
jz TargetCpuVmentry.Failure

;;
;; 更新当前 VMB 指针
;;
mov R0, [R5 + PCB.VmcsA]
mov [R5 + PCB.CurrentVmbPointer], R0

call setup_vmcs_region
```

为了支持 guest OS 使用实模式，将 guest A 与 guest B 对应的 VMB.GuestFlags 值设置为 GUEST_FLAG_UNRESTRICTED | GUEST_FLAG_EPT。也在 VM 对应的 VMCS 设置中都启用了“**unrestricted guest**”功能，并且开启 EPT 机制。

Guest-RIP 的值设置为 GUEST_BOOT_ENTRY + 4（也就是 **7C04H** 值），Host-RIP 值设置为 VmmEntry 函数入口。VMM 接着使用 initialize_vmcs_buffer 函数初始化 VMCS 区域的 buffer，执行完 VMPTRLD 指令加载 current-VMCS 指针，使用 setup_vmcs_region

函数将 VMCS buffer 数据刷新到当前 VMCS 中。

维护当前的 guest

每当切换 guest 时，VMM 需要维护当前 guest。在本书代码中，每个 VMB（VM Manage Block，VM 管理块）结构负责维护 VM 所对应的数据。一个 VMB 结构对应着一个 VM（也就是 guest）。示例中所有的操作都基于当前的 VMB 结构。

PCB 内的 CurrentVmbPointer 值指向当前 VMB 区域。因此，CurrentVmbPointer 值用来维护当前 guest。

代码片段 6-20：

```
;;
;; 加载 VMCS pointer
;;
vmptlrd [R5 + PCB.GuestA]
jc TargetCpuVmentry.Failure
jz TargetCpuVmentry.Failure

;;
;; 更新当前 VMB 指针
;;
mov R0, [R5 + PCB.VmcsA]
mov [R5 + PCB.CurrentVmbPointer], R0
```

在切换 guest 时，VMM 执行 VMPTRLRD 指令加载当前 VMCS。同时，VMM 需要在 CurrentVmbPointer 值中保存当前的 VMB 地址。

guest 状态

每个逻辑处理器支持 4 个 guest，分别为 guest A，guest B，guest C 以及 guest D。每个 guest 的状态可以为：

- GUEST_INITIALIZED，已初始化状态。
- GUEST_READY，就绪状态。
- GUEST_RUNNING，正在运行状态。
- GUEST_SUSPENDED，暂停状态。

这些状态值记录在每个 guest 所属的 VMB 区域的 GuestStatus 值中，当 VMM 完成 guest 的初始化后，将 guest 的状态置为 ready 状态。

为了支持 guest 的切换，每个逻辑处理器有两个队列：ready 队列与 running 队列。在 VMM 完成对 guest 的初始化后，将 guest 插入 ready 队列。在 VMM 切换到 guest 运行前，将 guest 插入到 running 队列。

guest 的 ready 队列

由于每个逻辑处理器只支持 4 个 guest，这里可以使用一个 32 位的 GuestReadyQueue 值来构建 ready 队列，GuestReadyIndex 值指向 ready 队列下一个将要插入的位置，如图 6-25 所示。

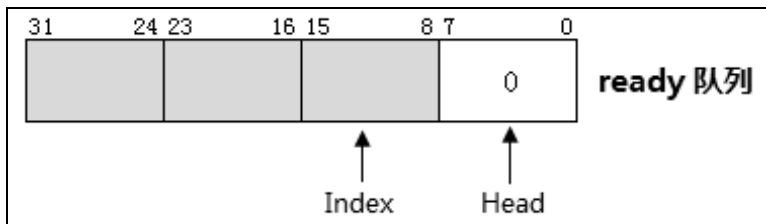


图 6-25

ready 队列中的每个字节对应一个 guest 槽，能同时容纳 4 个 guest。每个 guest 以 8 位的编号来表示（从 0 到 3）。0 代表 guest A，3 代表 guest D。当 ready 队列的 4 个槽已经装满时，ready 队列为满状态，不能再进行新的 guest 入队列操作。

在入 ready 队列时，每次在 GuestReadyIndex 值指向位置写入 guest 编号值。因此，GuestReadyIndex 的初始值为 0。每次出 ready 队列时，固定从 head 处取 guest 编号（也就是 byte 0 处取）。

代码片段 6-21:

```

;-----
; in_ready_queue()
; input:
;     esi - guest index
; output:
;     none
; 描述:
;     1) 在 guest ready 队列中插入 guest 编号 (0, 1, 2, 3)
;-----
in_ready_queue:
    push ebp
    push ecx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    ;;
    ;; 往 GuestReadyQueue[index] 里插入 guest 编号
    ;;
    mov eax, esi
    cmp DWORD [ebp + PCB.GuestReadyStatus], GUEST_QUEUE_FULL
    je in_ready_queue.done
    mov ecx, [ebp + PCB.GuestReadyIndex]
    mov [ebp + PCB.GuestReadyQueue + ecx], al
    INCV ecx
    cmp ecx, 3
    jbe in_ready_queue.@1
    mov DWORD [ebp + PCB.GuestReadyStatus], GUEST_QUEUE_FULL
    jmp in_ready_queue.done
in_ready_queue.@1:
    mov DWORD [ebp + PCB.GuestReadyStatus], GUEST_QUEUE_NORMAL
    mov [ebp + PCB.GuestReadyIndex], ecx
in_ready_queue.done:

```

```

    pop ecx
    pop ebp
    ret

```

in_ready_queue 函数实现入 ready 队列操作（在 lib\Vm\VmLib.asm 文件里），输入的参数为将要插入 ready 队列的 guest 编号。

当检测到 ready 队列满时，这个函数直接返回。否则向 GuestReadyIndex 指向的字节中写入 guest 编号值，然后，增加 GuestReadyIndex 值。如果 GuestReadyIndex 为 3，将 ready 队列置为 FULL 状态，否则为 NORMAL 状态。

代码片段 6-22:

```

;-----
; out_ready_queue()
; input:
;     none
; output:
;     eax - guest index
; 描述:
;     1) 在 guest ready 队列中取出 guest 编号 (0, 1, 2, 3)
;-----
out_ready_queue:
    push ebp
    push ecx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    mov eax, -1
    cmp DWORD [ebp + PCB.GuestReadyStatus], GUEST_QUEUE_EMPTY
    je out_ready_queue.done
    movzx eax, BYTE [ebp + PCB.GuestReadyQueue]
    mov ecx, [ebp + PCB.GuestReadyIndex]
    DECv ecx
    jl out_ready_queue.@1
    shr DWORD [ebp + PCB.GuestReadyQueue], 8
    mov [ebp + PCB.GuestReadyIndex], ecx
    jmp out_ready_queue.done
out_ready_queue.@1:
    mov DWORD [ebp + PCB.GuestReadyStatus], GUEST_QUEUE_EMPTY
out_ready_queue.done:
    pop ecx
    pop ebp
    ret

```

out_ready_queue 函数实现出 ready 队列操作。返回的值为从 ready 队列头中取出来的 guest 编号值。

当 ready 队列为空时，返回-1 值。否则，从 ready 队列的 byte 0 处读取 guest 编号值，然后，将 GuestReadyIndex 值减 1。当减为-1 值时，表示 ready 队列为 EMPTY 状态。否则将 ready 队列整体向前移一格（GuestReadyQueue 向右移 8 位）。

guest 的 running 队列

与 ready 队列一样，使用 32 位的 GuestRunningQueue 值来构建 running 队列。GuestRunningIndex 值指向 running 队列下一个将要插入的位置。

running 队列的结构与 ready 队列完全一样。同理，使用 in_running_queue 函数来插入 running 队列，使用 out_running_queue 函数来取出将要运行的 guest 编号。

与 running 队列唯一的区别是：由于在一个逻辑处理器中当前只有一个 guest 在运行，因此，在 running 队列中永远只有一个 guest 在排队。

6.4.5 使用 VMX-preemption timer

为了支持在 guest A 与 guest B 之间来回地切换，例子中使用了 VMX-preemption timer 功能，它允许 VMM 指定每个 guest 运行的时间。

启用 VMX-preemption timer

在执行 initialize_vmcs_buffer 函数前，VMB.VmxTimerValue 的值为 0 时则关闭 VMX-preemption timer 功能。在后续设置中可以使用下面的代码开启 VMX-preemption timer 功能。

代码片段 6-23:

```
;;
;; 启用 VMX-preemption timer, guestA 运行时间为 50μs
;;
SET_PINBASED_CTLS      ACTIVATE_VMX_PREEMPTION_TIMER
SET_VM_EXIT_CTLS       SAVE_VMX_PREEMPTION_TIMER_VALUE
mov esi, 50
call set_vmx_preemption_timer_value
```

上面的代码摘自 init_guest_a 与 init_guest_b 函数。通过使用 SET_PINBASED_CTLS 宏来置 pin-based VM-execution control 字段的“activate VMX-preemption timer”位，并且使用 SET_VM_EXIT_CTLS 宏来置 VM-exit control 字段的“save VMX-preemption timer value”位。在每次 VM-exit 时自动保存当前 VMX-preemption timer 的计数值。

接着使用 set_vmx_preemption_timer_value 函数为 guest A 与 guest B 指定 50μs 的运行时间。这是一个很短的时间值，为了观察在 guest A 与 guest B 之间频繁地进行切换。

代码片段 6-24:

```
;-----
; set_vmx_preemption_timer_value()
; input:
;     esi - us
; output:
;     none
; 描述:
;     1) 设置 VMX-preemption timer 初始计数值
;-----
set_vmx_preemption_timer_value:
```

```

        push ebp
        push edx
        push ecx

#ifdef __X64
        LoadGsBaseToRbp
#else
        mov ebp, [gs: PCB.Base]
#endif
        mov ecx, esi
        GetVmcsField    CONTROL_PINBASED
        test eax, ACTIVATE_VMX_PREEMPTION_TIMER
        jz set_vmx_preemption_timer_value.done

        ;;
        ;; 生成 vmx preemption timer value
        ;;
        mov eax, [ebp + PCB.ProcessorFrequency]
        mul ecx                      ; VmxTimerValue = (ProcessorFrequency * μs)
        mov ecx, [ebp + PCB.VmxMisc]
        and ecx, 1Fh                 ; VMX-preemption timer 计数频率
        shr eax, cl                  ; VmxTimerValue = (ProcessorFrequency *
μs) >> 频率

        ;;
        ;; 写入 VMX-preemption timer value 字段
        ;;
        SetVmcsField    GUEST_VMX_PREEMPTION_TIMER_VALUE, eax

set_vmx_preemption_timer_value.done:
        pop ecx
        pop edx
        pop ebp
        ret

```

set_vmx_preemption_timer 函数提供的输入参数以 μs 为单位（实现在 `\lib\Vm\VmLib.asm` 文件里），输入的 μs 数值乘上处理器的频率得到目标 TSC 值，需要根据 VMX-preemption timer 的计数频率来调整目标 TSC 值（目标 TSC 值向右位移计数频率），最终得到 VMX-preemption timer 的初始计数值。

假设处理器的频率为 $2400/\mu\text{s}$ (2.4GHz)，VMX-preemption timer 的计数频率为 32 (IA32_VMX_MISC[4:0]的值为 5)。输入的参数为 $50\mu\text{s}$ 时，目标的 TSC 值 = $2400 \times 50 = 120000$ 。最终的 VMX-preemption timer 初始计数值为 $120000 \gg 5 = 3750$ 。

使用 SetVmcsField 宏将这个初始计数值写入 guest-state 区域的 VMX-preemption timer value 字段中。

注册一个 VMX-preemption timer 处理例程

在 `lib\Vm\VmVMM.asm` 文件中有一个默认的 VMX-preemption timer 处理例程 DoVmPreemptionTimer 函数，这个默认例程只是让 VMM 打印 VMCS 信息。例子中，我们额外注册一个 VMX-preemption timer 例程去完成 guest 之间的切换工作。

代码片段 6-25:

```
;;
;; 注册另一个 VMX preemption timer 处理例程
;;
mov esi, Ex.DoVmxPreemptionTimer
mov [DoVmExitRoutineTable + EXIT_NUMBER_VMX_PREEMPTION_TIMER * 4], esi
```

在 chap06\ex6-1\ex.asm 文件中，TargetCpuVmentry 函数为 VMM 注册 ex 模块的 Ex.DoVmxPreemptionTimer 函数作为例子的 VMX-preemption timer 处理例程。

VMX-preemption timer 超时处理

每当 VM 的 VMX-preemption timer 计数值减为 0 值时将产生 VM-exit，VMM 将调用前面注册的那个 Ex.DoVmxPreemptionTimer 函数来处理 VM-exit。

代码片段 6-26:

```
;------
; Ex 模块的 VMX-preemption timer 处理例程
;------
Ex.DoVmxPreemptionTimer:
    push R5
    push R1
    push R3

    DEBUG_RECORD    "[Ex.DoPreemptionTimer]: switch guest !"

    ;;
    ;; 移除当前运行的 guest, 放入 ready 队列
    ;;
    call out_running_queue
    mov R3, [gs: PCB.Vmcsa + R0 * 8]
    mov esi, eax
    call in_ready_queue
    mov DWORD [R3 + VMB.GuestStatus], GUEST_SUSPENDED

    ;;
    ;; 从 ready 队列中取出需要运行的 guest, 放入 running 队列
    ;;
    call out_ready_queue
    mov R3, [gs: PCB.Vmcsa + R0 * 8]
    mov ecx, eax

    mov esi, [Ex.SwitchMsg + R0 * 4]
    call puts

    ;;
    ;; 运行 guest
    ;;
    vmptrld [R3 + VMB.PhysicalBase]
    jc Ex.DoPreemptionTimer.failure
    jz Ex.DoPreemptionTimer.failure

    mov [gs: PCB.CurrentVmbPointer], R3

    ;;
    ;; guest 运行时间为 50μs
```

```

;;
mov esi, 50
call set_vmx_preemption_timer_value

cmp DWORD [R3 + VMB.GuestStatus], GUEST_READY
je Ex.DoPreemptionTimer.ready
cmp DWORD [R3 + VMB.GuestStatus], GUEST_SUSPENDED
jne Ex.DoPreemptionTimer.done

;;
;; guest 目前处于 suspended 状态
;;
mov DWORD [R3 + VMB.GuestStatus], GUEST_RUNNING
mov esi, ecx
call in_running_queue

mov eax, VMM_PROCESS_RESUME
jmp Ex.DoPreemptionTimer.done

Ex.DoPreemptionTimer.ready:
;;
;; guest 目前处于 ready 状态
;;
mov DWORD [R3 + VMB.GuestStatus], GUEST_RUNNING
mov esi, ecx
call in_running_queue

mov eax, VMM_PROCESS_LAUNCH
jmp Ex.DoPreemptionTimer.done

Ex.DoPreemptionTimer.failure:
call dump_vmcs

Ex.DoPreemptionTimer.done:
pop R3
pop R1
pop R5
ret

```

当由于 VMX-preemption timer 超时而产生 VM-exit 后，说明 guest 的运行时间片已经用完。Ex.DoVmxPreemptionTimer 函数所做的处理有下面 8 个步骤：

- (1) 使用 out_running_queue 函数将当前运行的 guest 移出 running 队列。
- (2) 使用 in_ready_queue 函数将从 running 队列移出的 guest 插入 ready 队列，并且将这个 guest 状态置为 GUEST_SUSPENDED 暂停执行状态。
- (3) 使用 out_ready_queue 函数将下一个等待运行的 guest 从 ready 队列中取出。
- (4) 使用 VMPTRLD 指令加载从 ready 队列取出的 guest 所对应的 VMCS，作为 current-VMCS。并且更新 PCB.CurrentVmbPointer 值，VMM 记录当前的 VMB 指针。
- (5) 重新为等待运行 guest 指定 50μs 的时间片。
- (6) 检查等待运行 guest 的状态。

- 如果是 GUEST_READY 状态, 说明这个 guest 是首次进入 VM。则通知 VMM 使用 VMLAUNCH 指令进入 VM (返回 VMM_PROCESS_LAUNCH 值给 VmxVMM 函数)。
- 如果是 GUEST_SUSPENDED 状态, 说明这个 guest 产生了 VM-exit 等待再次进入 VM。则通知 VMM 使用 VMRESUME 指令进入 VM (返回 VMM_PROCESS_RESUME 值给 VmxVMM 函数)。

(7) 上面的操作完成后, 使用 in_running_queue 函数将等待运行的 guest 插入 running 队列, 并且将 guest 状态置为 GUEST_RUNNING 状态。

(8) 最后返回 VmxVMM 函数, 让 VmxVMM 完成收尾工作。

通过使用 VMX-preemption timer 功能, 在同一个逻辑处理器下, 可以在 guest A 与 guest B 之间进行不断的切换 (例子中只开启了两个 guest)。

6.4.6 host 处理流程

为了便于测试与观察, 本书例子期望读者的物理处理器中拥有至少 4 个逻辑处理器 (例如, 双核 4 线程处理器)。其中, 使用 CPU3 观察输出的调试记录 (debug record), CPU0 作为命令控制台, CPU1 与 CPU2 作为 guest 运行的处理器。

在一个只有两个逻辑处理器的平台上 (例如, 不支持超线程的双核处理器), 例子中使用 CPU0 作为 guest 运行处理器, 而使用 CPU1 观察输出的调试记录。

代码片段 6-27:

```
;;
;; 检查 CPU 个数
;;
cmp DWORD [fs: SDA.ProcessorCount], 2
je Ex.Next

;;
;; 调度 CPU3 执行 dump_debug_record() 函数
;;
mov esi, 3
mov edi, dump_debug_record
call dispatch_to_processor

;;
;; 等待用户选择命令
;;
call do_command

Ex.Next:
;;
;; 注册另一个 external interrupt 处理例程
;;
mov esi, Ex.DoExternalInterrupt
mov [DoVmExitRoutineTable + EXIT_NUMBER_EXTERNAL_INTERRUPT * 4], esi
```



```

;;
;; 设置 host 对 external interrupt 的控制权
;;
mov DWORD [Ex.ExtIntHold], 1

;;
;; 调度到 CPU1 执行 dump_debug_record()
;;
mov esi, 1
mov edi, dump_debug_record
call dispatch_to_processor

;;
;; 发起 VM-entry
;;
call TargetCpuVmentry

```

ex.asm 文件是例子的主体代码，当检测到只有两个处理器时：

(1) 由于 CPU0 被作为 guest 运行处理器，每个 VM 设置了外部中断发生时将由于“external interrupt exiting”而产生 VM-exit。这里，VMM 注册 Ex.DoExternalInterrupt 函数作为另一个外部中断处理例程。

(2) 设置 host 对外部中断控制权的标志位，允许 host 接管外部中断请求。

(3) 调度 CPU1 执行 dump_debug_record 函数来打印调试记录。

(4) 让 CPU0 发起 VM-entry 操作。

当有 4 个以上的处理器时，host 调度 CPU3 来执行 dump_debug_record 函数，然后让 CPU0 执行 do_command 函数作为命令选择器。

代码片段 6-28：

```

;;
;; 检查 host 是否需要对外部中断进行控制
;;
cmp DWORD [Ex.ExtIntHold], 1
jne init_guest_a.@1
;;
;; 关闭 "acknowledge interrupt on exit" 位
;;
CLEAR_VM_EXIT_CTLTS      ACKNOWLEDGE_INTERRUPT_ON_EXIT

init_guest_a.@1:

```

上面的代码是在 init_guest_a 与 init_guest_b 函数中，当 Ex.ExtIntHold 值为 1 时，需要 host 对外部中断的接管。

通过 CLEAR_VM_EXIT_CTLTS 宏来关闭 VM-exit control 字段的“acknowledge interrupt on exit”位，当由于外部中断而产生 VM-exit 时，处理器不会响应中断控制器（参见 3.5.1 节与 3.7.1 节），从而在 host 打开中断窗口时，处理器才响应中断控制器，取得外部中断 delivery 信息。

代码片段 6-29:

Ex.DoExternalInterrupt:

```
;;
;; 打开中断 window, 由 host 执行中断处理
;;
sti
mov eax, VMM_PROCESS_RESUME
ret
```

Ex.DoExternalInterrupt 函数非常简单, 只是使用 STI 指令打开中断窗口。在 STI 指令后面存在一个“blocking by STI”阻塞状态, 在执行完下一条指令后(在 RET 指令执行之前), host 将响应由用户的按键而产生的外部中断, 转而调入 keyboard_8259_handler 函数。

如果这里不执行 STI 指令, 则 host 无法接管外部中断, 从而不能响应用户按键。最后返回 VmxVMM 函数让 VMM 重新进入 VM 执行。

图 6-26 显示了只有两个逻辑处理器时的运行结果。CPU0 进入 VM 执行 guest 代码, 用户按<F2>键后可切换到 CPU1 观察, 当前 CPU1 执行打印调试记录。

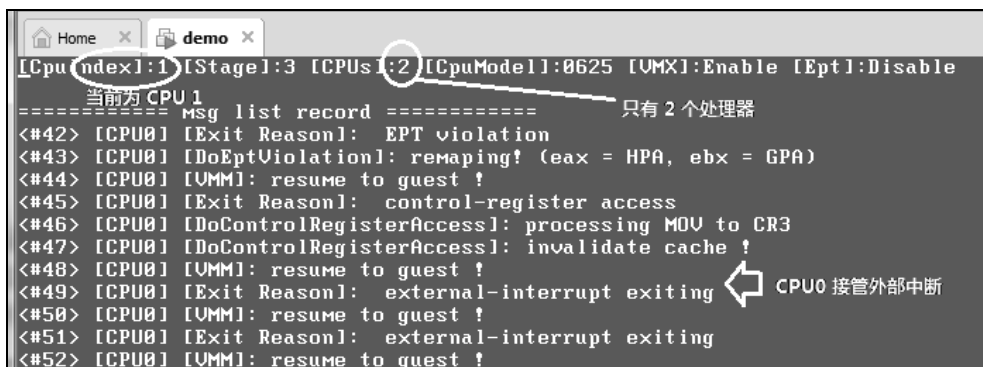


图 6-26

在第 49 条调试记录中显示, 由于按键而产生了 external interrupt exiting 事件, 然后 CPU0 的 host 代码接管了外部中断的执行切换到 CPU1。

当拥有 4 个以上的逻辑处理器时, CPU0 执行 do_command 函数, 用户通过按键选择相应的指令。当按下数字<1>键时, 调度 CPU1 执行 TargetCpuVmentry 函数, 按下数字<2>键时调度 CPU2 执行 TargetCpuVmentry 函数。

代码片段 6-30:

TargetCpuVmentry:

```
mov R5, [gs: PCB.Base]

;;
;; 初始化 GUEST A 与 B
;;
call init_guest_a
call init_guest_b
```

```

;;
;; 从 ready 队列取出
;;
call out_ready_queue
mov ecx, eax
mov R3, [R5 + PCB.VmcsA + R0 * 8]

;;
;; 加载 VMCS pointer
;;
vmptlrd [R3 + VMB.PhysicalBase]
jc TargetCpuVmentry.Failure
jz TargetCpuVmentry.Failure

;;
;; 更新当前 VMB 指针
;;
mov [R5 + PCB.CurrentVmbPointer], R3
call update_system_status

mov edi, ecx

;;
;; 配置 VMM MSR-load
;;
mov ecx, IA32_TIME_STAMP_COUNTER
rdtsc
mov ecx, edi
mov esi, IA32_TIME_STAMP_COUNTER
call append_vmexit_msr_load_entry

;;
;; 注册另一个 VMX preemption timer 处理例程
;;
mov esi, Ex.DoVmXPreemptionTimer
mov [DoVmExitRoutineTable + EXIT_NUMBER_VMX_PREEMPTION_TIMER * 4], esi

;;
;; 打印信息
;;
mov esi, Ex.Msg1
call puts
mov esi, Ex.Msg2
call puts
mov esi, Ex.Msg3
call puts
mov esi, [R5 + PCB.ProcessorIndex]
inc esi
call print_dword_decimal
mov esi, Ex.Msg4
call puts

;;
;; 插入 running 队列
;;
mov esi, ecx
call in_running_queue
mov DWORD [R3 + VMB.GuestStatus], GUEST_RUNNING

```

```
;;
;; 进入 guest 环境
;;
call reset_guest_context
vmlaunch
```

TargetCpuVmentry.Failure:

```
call dump_vmcs
call wait_esc_for_reset
ret
```

TargetCpuVmentry 函数依次调用 init_guest_a 与 init_guest_b 对 guest A 与 guest B 对应的 VMCS 进行初始化设置。Current-VMCS 与 CurrentVmbPointer 更新为 guest A，然后对 guest A 发起 VM-entry 操作，执行 guest A 代码。

6.4.7 运行结果

在笔者使用的 VMware Workstation 9.0.2 版本中仍然不支持 VMX-preemption timer 功能。因此，要得到 guest 之间的切换效果，需要使用 U 盘在真实的机子上运行。

运行例子代码后，在 CPU0 的命令选择器上按数字<1>键，调度 CPU1 执行 TargetCpuVmentry 函数。然后按下<F2>键切换到 CPU1 屏幕。图 6-27 是 CPU1 在 host 端的屏幕输出信息。

```
[CpuIndex]:1 [Stage]:3 [CPUs]:4 [CpuModel]:8625 [VMX]:E
[Host]: launch VM ...
[Host]: Guest OS running ...
[Host]: press <F2> key switch to GUEST screen !
[Host]: switch to guestB
[Host]: switch to guestA
[Host]: switch to guestB
[Host]: switch to guestA ← geustA 与 guestB 之间来回切换
[Host]: switch to guestB
[Host]: switch to guestA
[Host]: switch to guestB
[Host]: switch to guestA
```

图 6-27

图 6-27 显示，在当前 CPU1 下，VMM 发起 VM-entry 操作后 CPU1 首次执行的是 guest A，接下来切换到 guest B，然后再切换回 guest A，从而不断地在 guest A 与 guest B 之间进行切换（本例中只使用了 guest A 与 guest B）。可以进一步扩展，当使用了 4 个 guest 时，便会在 guest A, B, C 及 D 之间切换。

图 6-27 是 host 端的输出信息。那么，此时按<F2>键将可以切换到 guest 端的屏幕，查看 guest 的输出信息，如图 6-28 所示。

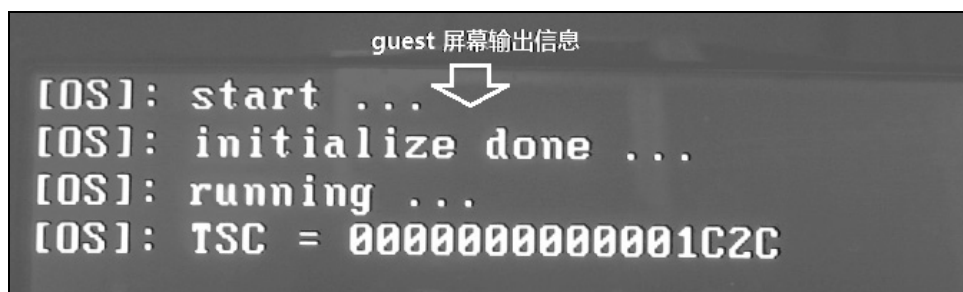


图 6-28

参见 6.4.2 节中的 GuestKernel 模块代码。guest 调用 PutStr 函数来打印信息，这些信息被保存在 VSB 区域内的 video buffer。当按下<F2>键切换到 guest 屏幕时，显示出 guest 打印的信息。

可以再次按下<F2>键切换回 host 屏幕显示，如图 6-27 所示。注意，在 guest 屏幕显示中并没有系统状态信息，因为 guest 并没有输出系统状态信息。

处理 I/O 指令

在 6.4.1 节介绍的 GuestBoot 模块的代码中，有下面的代码：

```
GuestBoot.Start:
    cli
    NMI_DISABLE                ; 关闭 NMI
    FAST_A20_ENABLE            ; 开启 A20 位
```

guest 执行 NMI_DISABLE 与 FAST_A20_ENABLE 宏关闭 NMI 及开启 A20 地址线，这是 VMM 需要监控的行为。

代码片段 6-31：

```
;;
;; 屏蔽 guest 对 NMI_EN_PORT (70h) 与 SYSTEM_CONTROL_PORTA (92h) 端口的访问
;;
mov R6, [R5 + PCB.VmcsA]
mov edi, NMI_EN_PORT
call set_vmcs_iomap_bit

mov R6, [R5 + PCB.VmcsA]
mov edi, SYSTEM_CONTROL_PORTA
call set_vmcs_iomap_bit
```

在 init_guest_a 与 init_guest_b 函数初始化 VMCS 时，在 I/O bitmap 中设置了对端口 70H 与 92H 访问的监控。当 guest 尝试访问这些端口时将产生 VM-exit。

按下<F4>键切换到 CPU3，按<PgDn>键翻到第 2 条记录。在图 6-29 的调试记录里显示 guest 在 7C05H 的地址上执行了 I/O 指令。这条指令产生了 VM-exit 而被 VMM 拦截了下来。

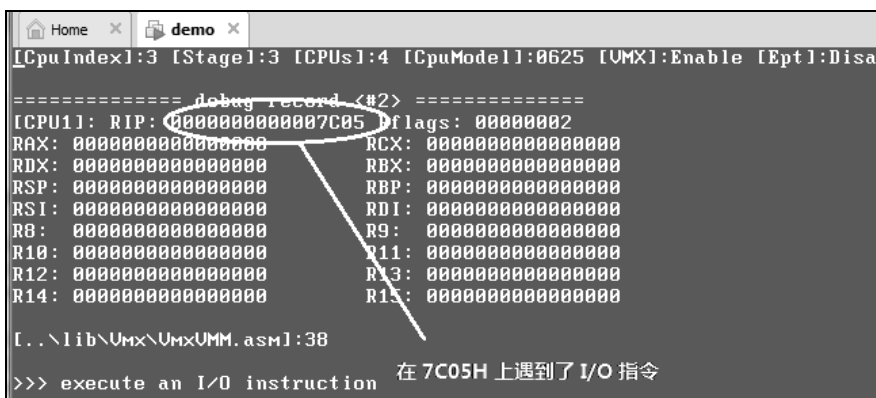


图 6-29

通过翻查 debug record 信息，实际上 guest 执行的 I/O 指令一共被 VMM 拦截了 4 次，分别为：

- (1) 7C05H 位置。
- (2) 7C09H 位置。
- (3) 7C0BH 位置。
- (4) 7C0FH 位置。

VMM 简单地将这些对端口 70H 与 92H 的访问忽略了，如图 6-30 所示。

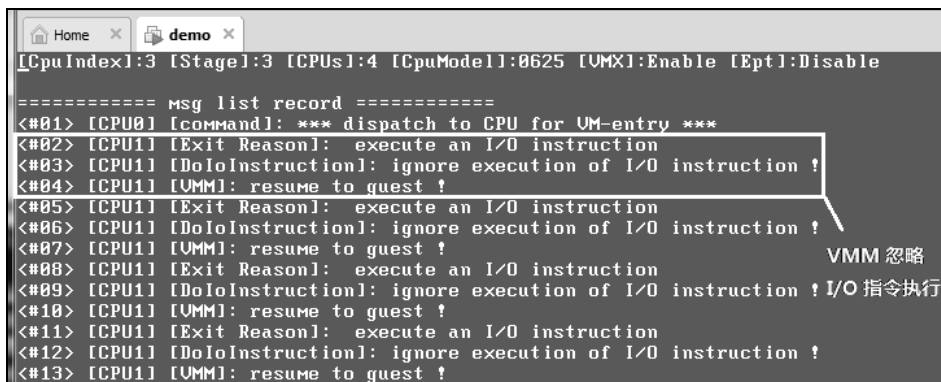


图 6-30

在 msg-list 记录中显示，VMM 对这 4 次拦截的 I/O 指令忽略，然后重新 resume 到 guest 继续执行。

处理描述符表寄存器的访问

在 6.4.1 节介绍的 GuestBoot 模块的代码中，guest 使用 LGDT 与 LIDT 指令加载了 GDTR 与 IDTR 描述符表寄存器。

```

lgdt [Guest.GdtPointer]      ; 加载 GDT
lidt [Guest.IdtPointer]      ; 加载 IDT

```

作为示例，尽管这里 VMM 拦截了 guest 对 GDTR 与 IDTR 寄存器的访问，但是，VMM 并没有什么实际的工作。实际上，VMM 应该允许 guest OS 对 GDTR 与 IDTR 寄存器的访问。

因此在 DoGDTR_IDTR 函数中，使用 CLEAR_SECONDARY_PROCBASED_CTLs 宏来清掉“descriptor-table exiting”位，允许 guest OS 访问描述符表寄存器。

代码片段 6-32:

```
;;
;; 清"descriptor-table exiting"位
;;
CLEAR_SECONDARY_PROCBASED_CTLs  DESCRIPTOR_TABLE_EXITING
```

接着，VMM 重新进入 VM，让 guest 继续执行 LGDT 与 LIDT 指令。

处理访问 CR0 与 CR4 寄存器

在 GuestBoot 模块的最后执行了 MOV-to-CR0 指令开启未分页的保护模式，在 GuestKernel 模块开头执行了 MOV-to-CR4 指令来开启 CR4.PAE 位。在初始化 VMCS 区域时，对 CR0 与 CR4 的监控设置如下面的代码所示。

代码片段 6-33:

```
;;
;; CR0 guest/host mask 设置，根据提供的 guest flag 来进行设置
;; 1) CR0.NE 属于 host 权限
;; 2) 当 GUEST_FLAG_PE = 1, CR0.PE 属于 host 权限，否则为 guest 权限
;; 3) 当 GUEST_FLAG_PG = 1, CR0.PG 属于 host 权限，否则为 guest 权限
;; 5) CR0.CD 属于 host 权限
;; 6) CR0.NW 属于 host 权限
;;
;; CR0 read shadow 设置:
;; 1) CR0.PE 等于 CR0 guest/host mask 的 CR0.PE
;; 2) CR0.PG 等于 CR0 guest/host mask 的 CR0.PG
;; 3) CR0.NE = 1
;; 4) CR0.CD = 0
;; 5) CR0.NW = 0
;;
mov eax, [esi + VMB.GuestFlags]
and eax, (GUEST_FLAG_PG | GUEST_FLAG_PE)
or eax, CR0_NE | CR0_CD | CR0_NW
mov edx, eax
and edx, ~(CR0_CD | CR0_NW)
mov DWORD [ExecutionControlBufBase + EXECUTION_CONTROL.Cr0GuestHostMask], eax
mov DWORD [ExecutionControlBufBase + EXECUTION_CONTROL.Cr0GuestHostMask + 4], 0FFFFFFFh
mov DWORD [ExecutionControlBufBase + EXECUTION_CONTROL.Cr0ReadShadow], edx
mov DWORD [ExecutionControlBufBase + EXECUTION_CONTROL.Cr0ReadShadow + 4], 0

;;
;; CR4 guest/host mask 与 read shadow 设置
;; 1) CR4.VMXE 及 CR4.VME 属于 host 权限
;; 2) 当 GUEST_FLAG_PG = 1 时，CR4.PAE 属于 host 权限，否则属于 guest 权限
;;
mov eax, 00002021h
mov edi, 00002020h
```

```

test DWORD [esi + VMB.GuestFlags], GUEST_FLAG_PG
jnz init_vm_execution_control_fields.Cr4

mov eax, 00002001h                ;; guest/host mask[PAE] = 0
mov edi, 00002000h                ;; read shadow[PAE] = 0

init_vm_execution_control_fields.Cr4:
mov DWORD [ExecutionControlBufBase + EXECUTION_CONTROL.Cr4GuestHostMask], eax
mov DWORD [ExecutionControlBufBase + EXECUTION_CONTROL.Cr4GuestHostMask + 4],
0FFFFFFFFh
mov DWORD [ExecutionControlBufBase + EXECUTION_CONTROL.Cr4ReadShadow], edi
mov DWORD [ExecutionControlBufBase + EXECUTION_CONTROL.Cr4ReadShadow + 4], 0

```

这段代码摘自 `init_vm_execution_control_fields` 函数（实现在 `lib\Vm\VmInit.asm` 文件）。当 guest 运行在实模式时，允许 guest OS 对 CR0.PE 与 CR0.PG 进行置位（guest 有权设置）。也允许 guest OS 对 CR4.PAE 进行置位（guest 有权限设置该位）。

因此，guest OS 在开启保护模式及 PAE 分页模式时，VMM 无须接管，直接让 guest 通行。

处理 EPT violation

在 `GuestKernel` 模块中，首要的任务是开启分页机制。当定义了 `GUEST_X64` 符号时，调用 `init_longmode_page` 函数建立 IA-32e 分页模式页表结构。否则，调用 `init_pae_page` 函数建立 PAE 分页模式页表结构。

这里以 64 位的 host 和 64 位的 guest 为例。在调用 `init_longmode_page` 函数时会引发一连串的林 EPT violation 故障而导致 VM-exit。

在图 6-31 所示的 msg-list 记录里，从第 16 条记录开始连续产生多个 EPT violation 故障。在 `VmxVMM` 函数中调用 `DoEptViolation` 函数处理 EPT violation 故障。

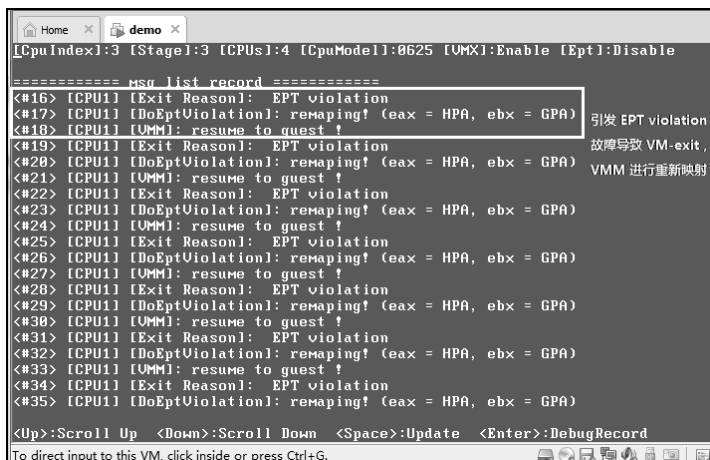


图 6-31

代码片段 6-34:

```

;-----
; DoEptViolation()

```



```

; input:
;     esi - do process code
; output:
;     eax - VMM process code
; 描述:
;     1) 处理由 EPT violation 引发的 VM-exit
;-----
DoEptViolaton:
    push ebp
    push edx
    push ebx
    push ecx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    ;;
    ;; 读取发生 EPT violation 的 guest-physical address 值
    ;;
    REX.Wrb
    mov ebx, [ebp + PCB.ExitInfoBuf + EXIT_INFO.GuestPhysicalAddress]

    ;;
    ;; 检查页面是否属于 not-present
    ;; 1) 如果属于 not-present, 则分配物理页面, 进行重新映射
    ;; 2) 如果属于无访问权限, 则修复映射
    ;;
    mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]
    test eax, (EPT_READ | EPT_WRITE | EPT_EXECUTE) << 3
ExitQualification[5:3]
    jz DoEptViolaton.@0

    DEBUG_RECORD        "[DoEptViolation]: fixing access !"

    ;;
    ;; 下面修复由于访问权限引起的 EPT violation 问题
    ;;
#ifdef __X64
    REX.Wrb
    mov esi, ebx
    and eax, 07h
    or eax, FIX_ACCESS
    ; guest-physical address
    ; 访问类型
    ; 进行 FIX_ACCESS 操作
#else
    xor edi, edi
    mov esi, ebx
    mov ecx, eax
    and ecx, 07h
    or ecx, FIX_ACCESS
#endif
    jmp DoEptViolation.DoMapping

DoEptViolaton.@0:
    ;;
    ;; 从处理器 domain 里分配一个 4K 物理页面
    ;;

```

```

mov esi, 1
call vm_alloc_pool_physical_page

REX.Wrxb
test eax, eax
jz DoEptViolation.done

DoEptViolation.remaping:

    DEBUG_RECORD        "[DoEptViolation]: remaping! (eax = HPA, ebx = GPA)"

    ;;
    ;; 下面进行 guest-physical address 映射
    ;; 注意: 这里添加所有访问权限, read/write/execute
    ;; 1) 因为 guest-physical address 访问, 可能会进行多种访问, 需要多种权限
    ;;
#ifdef __X64
    ;;
    ;; rsi - guest-physical address
    ;; rdi - host-physical address
    ;; eax - page attribute
    ;;
    REX.Wrxb
    mov esi, ebx                ; guest-physical address
    REX.Wrxb
    mov edi, eax                ; host-physical address
    mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]
    or eax, EPT_FIXING | FIX_ACCESS | EPT_READ | EPT_WRITE | EPT_EXECUTE
#else
    ;;
    ;; edi:esi - guest-physical address
    ;; edx:eax - host-physical address
    ;; ecx     - page attribute
    ;;
    xor edi, edi
    xor edx, edx
    mov esi, ebx                ; guest-physical address
    mov ecx, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]
    or ecx, EPT_FIXING | FIX_ACCESS | EPT_READ | EPT_WRITE | EPT_EXECUTE
#endif

DoEptViolation.DoMapping:
    ;;
    ;; 执行 guest-physical address 映射工作
    ;;
    call do_guest_physical_address_mapping

    ;;
    ;; ### 刷新 cache ###
    ;;

    ;;
    ;; INVEPT 描述符
    ;;
    mov DWORD [ebp + PCB.InvDesc + INV_DESC.Dword1], 0
    mov DWORD [ebp + PCB.InvDesc + INV_DESC.Dword2], 0
    mov DWORD [ebp + PCB.InvDesc + INV_DESC.Dword3], 0

```

```

#ifdef __X64

    GetVmcsField    CONTROL_EPT_POINTER_FULL
    REX.Wrb
    mov [ebp + PCB.InvDesc + INV_DESC.Eptp], eax
#else
    GetVmcsField    CONTROL_EPT_POINTER_FULL
    mov [ebp + PCB.InvDesc + INV_DESC.Eptp], eax
    GetVmcsField    CONTROL_EPT_POINTER_HIGH
    mov [ebp + PCB.InvDesc + INV_DESC.Eptp + 4], eax
#endif

;;
;; 使用 single-context invalidation 刷新方式
;;
mov eax, SINGLE_CONTEXT_INVALIDATION
invept eax, [ebp + PCB.InvDesc]

mov eax, VMM_PROCESS_RESUME

DoEptViolation.done:
pop ecx
pop ebx
pop edx
pop ebp
ret

```

如果是由 EPT paging structure 表项出现 not-present 而引起 EPT violation 故障，则调用 `vm_alloc_pool_physical_page` 函数从 VM domain 中分配一个 4K 页面进行重新映射。

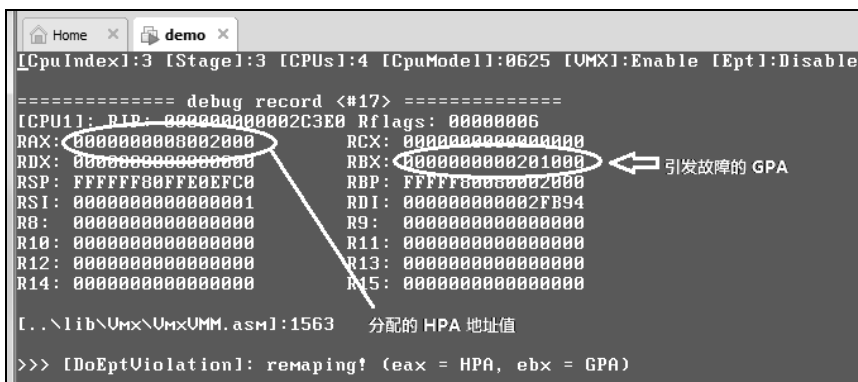


图 6-32

如图 6-32 中的 debug record 所示，引发 EPT violation 故障的地址值是 201000H。这个地址是在 `init_longmode_page` 函数内调用 `AllocPhysicalPage` 函数分配页面而来的。在对这个页面进行清 0 而引发 EPT violation 故障。

所有的 EPT violation 故障都在 `init_longmode_page` 函数内引发，总共产生了 10 次 EPT violation 故障。也就是 VMM 进行了 10 次重新映射，如表 6-2 所示。

表 6-2

序 号	引发故障的 GPA	重新映射的 HPA
1	201000H	8002000H
2	200800H	8003000H
3	202000H	8004000H
4	203000H	8005000H
5	204000H	8006000H
6	205000H	8007000H
7	206000H	8008000H
8	207000H	8009000H
9	208000H	800A000H
10	209000H	800B000H

每当 VMM 修复一次 EPT violation 故障后回到 guest 继续执行，guest 并不知道自己发生了故障。实际上，当 guest 发生这一连串的 EPT violation 故障时，已经由于 guest 时间片用完（50μs）发生过 guest 切换。

guest 切换

处理器首次运行的是 guest A。在 guest A 的运行过程中，第一次的 guest 切换发生在 2009DH 位置，如图 6-33 所示。

```
[CpuIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:8625 [VM]
===== debug record <#31> =====
[CPU1]: RIP: 00000000002009D Rflags: 00000016
RAX: 0000000000000000 RCX: 0000000000000000
RDX: 0000000000202000 RBX: 0000000000204000
RSP: 0000000000007BDC RBP: 0000000000203000
RSI: 0000000000001000 RDI: 0000000000204000
RB: 0000000000000000 R9: 0000000000000000
R10: 0000000000000000 R11: 0000000000000000
R12: 0000000000000000 R13: 0000000000000000
R14: 0000000000000000 R15: 0000000000000000

[.\lib\Umx\UmxVMM.asm]:39
>>> VMX-preemption timer expired
```

由于 VMX-preemption timer 超时
而引发 VM-exit

图 6-33

在第 31 条调试记录中显示，由于 VMX-preemption timer 超时而引发 VM-exit。VmxVMM 函数调用 Ex.DoVmxPreemptionTimer 函数来处理这个 VM-exit 事件（参见 6.4.5 节）。

Ex.DoVmxPreemptionTimer 函数首次切换时，检查到 guest B 属于 ready 状态，则执行 VMLAUNCH 指令发起 VM-entry 执行 guest B 代码。在 guest B 运行过程中由于 VMX-preemption timer 超时而导致 VM-exit 时，Ex.DoVmxPreemptionTimer 函数检查到 guest A

属于 suspended 状态，则执行 VMRESUME 指令发起 VM-entry 切换到 guest A 执行。

如图 6-34 所示，在第 34 条调试记录中显示，此时已经从 guest A 切换到 guest B。同样，guest B 也从 7C04H 位置开始执行，在 7C05H 上由于执行了 I/O 指令而被 VMM 拦截下来（参见前面的图 6-29）。

```
[CpuIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VM]

===== debug_record <#34> =====
[CPU1]: RIP: 0000000000007C05 Rflags: 00000002
RAX: 0000000000000000 RCX: 0000000000000000
RDX: 0000000000000000 RBX: 0000000000000000
RSP: 0000000000000000 RBP: 0000000000000000
RSI: 0000000000000000 RDI: 0000000000000000
R8: 0000000000000000 R9: 0000000000000000
R10: 0000000000000000 R11: 0000000000000000
R12: 0000000000000000 R13: 0000000000000000
R14: 0000000000000000 R15: 0000000000000000

[..\lib\Umx\UmxVMM.asm]:39
>>> execute an I/O instruction

guest B 从 7C04H 开始执行，同样在 7C05H 位
置上执行了 I/O 指令被 VMM 拦截！
```

图 6-34

表 6-3 列出了 guest 切换时地址。注意，最后 guest A 与 guest B 都执行 JMP 指令进入死循环。因此，只要 VMX-preemption timer 超时，就不断地进行 guest 切换。

表 6-3

序 号	guest	被中断地址	描 述
1	A	2009DH	从 guest A 切换到 guest B
2	B	200B0H	从 guest B 切换到 guest A
3	A	201E7H	从 guest A 切换到 guest B
4	B	200B0H	从 guest B 切换到 guest A
5	A	2008DH	从 guest A 切换到 guest B
6	B	2008DH	从 guest B 切换到 guest A
...	...	...	...

guest 进入 HLT 状态

作为另一种更好的选择，我们可以让 guest A 与 guest B 在空闲的时候，执行 HLT 指令进入 HLT 状态。

代码片段 6-35:

```
;;
;; 进入 HLT 状态
;;
hlt
jmp $ - 1
```

上面的代码来自 `guest_ex.asm` 模块，作为例子 6-1 的最终结果，让 `guest` 进入 HLT 状态。

代码片段 6-36:

```
DoHLT:
    DEBUG_RECORD    "[DoHLT]: enter HLT state"

    ;;
    ;; 将 guest 设置为 HLT 状态
    ;;
    SetVmcsField    GUEST_ACTIVITY_STATE, GUEST_STATE_HLT

    mov eax, VMM_PROCESS_RESUME
    ret
```

当 VMM 拦截到 HLT 指令后，将 `guest-state` 区域的 `activity state` 字段设为 1 值（HLT 状态），重新进入后令 VM 进入 HLT 状态。当然，更简单的方法是直接让 `guest` 执行 HLT 指令，VMM 并不拦截 HLT 指令。

但是，恢复 `guest` 执行让 VM 进入 HLT 状态，并不能提高虚拟化平台的利用率，这将使处理器一直处于等待状态（等待事件唤醒 HLT 状态）。

另一种选择是：当 VMM 拦截 `guest` 执行 HLT 指令后，并不恢复 `guest` 的执行，VMM 去处理另一些待处理的任务，或者执行另一个 `guest`（前提是这个 `guest` 处于正常状态）。

第 7 章

中断虚拟化

VMM 可以监控和接管 VM 内发生的异常（包括硬件异常与软件异常）、外部中断、SMI 及 NMI（参见 5.3 节）。VMM 通过下面的设置进行拦截。

- 异常，通过将异常向量号在 exception bitmap 字段相应位置 1。
- 外部中断，通过将 pin-based VM-execution control 字段 “external-interrupt exiting” 位置 1。
- SMI，通过 “SMM 双重监控处理机制” 切入到 STM（SMM-transfer monitor）端进行处理。
- NMI，通过将 pin-based VM-execution control 字段 “NMI exiting” 位置 1。

VMM 拦截这些事件后，可以直接反射给 guest 执行，或者根据情况做相应地虚拟化处理后反射虚拟化结果给 guest。另外，当在 VM 内发生 INIT、SMI 及 SIPI 事件而不被阻塞时，将直接产生 VM-exit（参见 5.3 节和 4.17.2 节）。

7.1 异常处理

当 guest 在运行过程中可能会遇到某些错误引发异常，而由于该异常向量号在 exception bitmap 字段对应的位为 1 而导致 VM-exit。

如果这个异常不直接导致 VM-exit，也可能会有这个异常（或其他向量事件）的 delivery 期间遇到了另一个错误（包括**异常**、**triple fault**、**任务切换**、**EPT violation**、**EPT misconfiguration** 或者访问 **APIC-access page**）而间接导致 VM-exit。

尽管 VMM 接管了这个异常（向量事件）的控制权，并且可以做某些处理，但是如果这个异常的发生是由于 guest 自身原因而导致，那么 guest OS 总是期望能得到对这个异常的最终处理。此时，VMM 不应该忽略掉这个异常，而总是需要使用事件注入方式将异常交回 guest 进行处理，这样能保证 guest OS 正确的行为。

因此，VMM 对异常如何处理，取决于这个异常是否由于 guest 自身原因而产生。

- 当异常是由于 guest 自身条件而引发时，VMM 需要反射这个异常给 guest 处理，VMM 可以不做任何处理。
- 当异常是由于 host/VMM 出于某些目的设置了某些条件或者 VMM 的错误设置而引发时，VMM 根据自身设置的条件做相应处理，或者修正自身的错误设置后反射异常给 guest 处理，然后恢复 guest 的运行。

例如，当 guest 产生 #PF 异常而导致 VM-exit 时，如果 #PF 异常是由于 guest OS 并没有映射线性地址而产生，那么 VMM 不能插手处理，需要直接反射给 guest 处理。但是，如果这个线性地址是由于 VMM 的有意安排而产生 #PF 异常，那么 VMM 根据这个情况做一些处理后无须反射给 guest（参考 7.1.2.2 节的例子 7-1）。

又如，在异常 delivery 期间由于 EPT misconfiguration 而导致 VM-exit 时，VMM 应该修复这个故障，然后注入这个原始异常事件（并不是直接引发 VM-exit 的异常）让 guest 恢复继续执行（即这个异常间接导致 VM-exit）。

然而有一个特殊的例子，在 VMX non-root operation 模式里尝试进行任务切换将直接产生 VM-exit。当 guest 运行在 legacy 模式时，并不是 guest 的错误，也不是 VMM 本身的错误。因此，VMM 必须要帮助 guest 来完成这个任务切换操作（参考 7.1.3 节例子 7-2）。

7.1.1 反射异常给 guest

由于异常而引发 VM-exit 时，VM-exit interruption information 字段将记录这个异常的 delivery 信息（参见 3.10.2.1 节），并且有错误码时，在 VM-exit interruption error code 字段里记录异常的错误码。

一般情况下，VMM 可以直接将 VM-exit interruption information 字段的值复制给 VM-entry interruption information 字段，将 VM-exit interruption error code 字段的值复制给 VM-entry interruption error code 字段（存在错误码时）。

1. 处理 NMI unblocking 位

VM-entry interruption information 字段的 bits 30:12 属于保留位，在注入事件时必须清为 0 值。然而，VM-exit interruption information 字段的 bit 12 为“NMI unblocking”位。当这个异常是在执行 IRET 指令时引发并导致 VM-exit，VM-exit interruption information 字段的 bit 12 被置位。此时，VMM 需要将复制到 VM-entry interruption information 字段的 bit 12 清 0，否则将会产生 VM-entry 失败。

因此，每次反射异常给 guest 时，VMM 需要检查 VM-exit interruption information 字段的 bit 12，做出相应的处理。

2. 反射 #DF 异常

当 IDT-vectoring information 字段的 bit 31 (valid 位) 为 1 时，表明这个字段所记录的向量事件并不直接引发 VM-exit，而是由于在异常 delivery 期间遇到某些错误而导致 VM-exit（参见 3.10.3 节）。那么，IDT-vectoring information 字段记录原始向量事件的

delivery 信息，IDT-vectoring error code 字段记录原始异常的错误码。

在如表 7-1 所示的情况下 VMM 需要反射一个#DF（double fault）异常给 guest。

表 7-1

类 型	IDT-vectoring information	VM-exit interruption information	反射给 guest
第 1 类	#DE (0)、 #TS (10)、 #NP (11)、 #SS (12)、 #GP (13)	#DE (0)	#DF (8)
		#TS (10)	
		#NP (11)	
		#SS (12)	
		#GP (13)	
第 2 类	#PF (14)	#PF (14)	
		#DE (0)	
		#TS (10)	
		#NP (11)	
		#SS (12)	
		#GP (13)	

当 guest 在执行过程中引发了表 7-1 中 IDT-vectoring information 字段所记录的异常，或者注入了表中 IDT-vectoring information 字段所记录的硬件异常（中断类型为硬件异常），但这个异常并不直接导致 VM-exit，而在 delivery 期间遇到了表 7-1 中 VM-exit interruption information 字段所记录的嵌套异常而导致 VM-exit（另外参见 4.12.2 节）。

也就是下面两类情况，#DF 异常最终会被产生。

第 1 类情况：

- 遇到的异常（或者注入的硬件异常）是#DE（向量号 0）、#TS（向量号 10）、#NP（向量号 11）、#SS（向量号 12）或者#GP（向量号 13）之一。
- 异常 delivery 期间遇到了#DE（向量号 0）、#TS（向量号 10）、#NP（向量号 11）、#SS（向量号 12）或者#GP（向量号 13）之一。

第 2 类情况：

- 遇到的异常（或者注入的硬件异常）是#PF（向量号 14）。
- 异常 delivery 期间遇到了#PF（向量号 14）、#DE（向量号 0）、#TS（向量号 10）、#NP（向量号 11）、#SS（向量号 12）或者#GP（向量号 13）之一。

属于上面两类情况时，在正常情况下这两个异常会转化为一个#DF 异常（double fault，双重故障）。但由于嵌套异常 delivery 期间引发的异常直接引发了 VM-exit，为了模拟 guest 产生的#DF 异常，VMM 需要注入一个硬件异常#DF 给 guest，而不是嵌套异常。

举例来说，guest 在执行中产生了#SS 异常，但这个#SS 异常并不引发 VM-exit，继而在这个#SS 异常的 delivery 期间遇到了一个#GP 异常而导致 VM-exit。此时 VMM 需要反

射一个#DF 异常给 guest 处理，而不是#GP 异常。

VMM 注入#DF 异常时，VM-entry interruption information 字段设置为：

- Bits 7:0 (向量号) 为 8，指示#DF 异常。
- Bits 10:8 (中断类型) 为 3，指示属于硬件异常。
- Bit 11 (error code) 为 1，指示需要提供错误码。
- Bit 31 (valid) 为 1，指示这个字段有效。
- Bits 32:12 (保留位) 清为 0。

VM-entry interruption error code 字段值设置为 0，指示错误码为 0 值。

另一情况是：guest 遇到异常但并不导致 VM-exit，继而在这个异常 delivery 期间引发了另一个嵌套异常，这个嵌套异常也不导致 VM-exit。从而这两个异常被转化为#DF 异常，由于 exception bitmap 字段的 bit 8 为 1 而导致 VM-exit。

此时，VM-exit interruption information 字段记录的值为 **80000B08H**，指示由#DF 异常导致 VM-exit，并且存在 error code。VM-exit interruption error code 字段的值为 0，指示异常错误码为 0。在这种情况下，VMM 只需要使用前面所说的“使用直接复制方法”注入硬件异常#DF 让 guest 处理。

3. 处理 triple fault

下面的三种情况会产生 triple fault（异常属于前面所说的第 1 类或者第 2 类情况）：

(1) 当 guest 遇到异常但并不导致 VM-exit，继而在这个异常 delivery 期间引发了另一个异常，这个异常也不导致 VM-exit，从而这两个异常被转化为#DF 异常。#DF 异常也不导致 VM-exit，继而在#DF 异常的 delivery 期间再次引发了一个异常，最终转化为 triple fault 而导致 VM-exit。

(2) VMM 注入一个硬件异常，在这个注入异常的 delivery 期间引发了一个异常，这个异常并不导致 VM-exit，从而转化为#DF 异常。#DF 异常也不导致 VM-exit，继而在#DF 异常的 delivery 期间再次引发了一个异常，最终转化为 triple fault 而导致 VM-exit。

(3) VMM 注入一个硬件异常#DF，在这个注入的#DF 异常 delivery 期间引发了一个异常，最终转化为 triple fault 而导致 VM-exit。

由 triple fault 直接引发 VM-exit 时，VMM 不应该注入任何事件给 guest 执行。VMM 可以选择终止 guest 的运行，或者将 VM 置为 shutdown 状态。当 VM 处于 shutdown 状态时，NMI、SMI 及 INIT 事件能唤醒 shutdown 状态转为 active 状态（参见 4.17.3 节）。

4. 直接反射异常

当异常由 guest 自身条件所引发时，在下面的情况下 VMM 可以直接反射异常给 guest 处理。

(1) 由异常直接导致 VM-exit 时，也就是 IDT-vectoring information 字段 bit 31 为 0。

(2) 不属于表 7-1 中的**第 1 类**和**第 2 类**情况时（不产生#DF 异常），由于向量事件 delivery 期间遇到一个异常而导致 VM-exit。

例如，guest 运行过程中引发了#GP 异常，#GP 异常不导致 VM-exit。但在#GP 异常 delivery 期间遇到了#PF 异常而导致 VM-exit。这种情况下**并不会产生#DF 异常**，VMM 需要直接将#PF 异常反射给 guest 处理。

VMM 可以直接将 VM-exit interruption information 字段的值赋值给 VM-entry interruption information 字段，将 VM-exit interruption error code 字段的值直接赋给 VM-entry interruption error code 字段（存在错误码时），通过事件注入反射给 guest 处理。

7.1.2 恢复 guest 异常

当 guest 引发异常的条件是 host/VMM 所设置时，VMM 取消引发 guest 异常的条件后使用 VMRESUME 指令恢复 guest 的运行。

7.1.2.1 直接恢复

当异常直接引发 VM-exit，也就是 IDT-vectoring information 字段 bit 31 为 0（不是间接引发 VM-exit）时。尽管 VM-exit interruption information 字段记录了引发 VM-exit 的异常。但是，由于异常是 host 的原因引发，并不是 guest 造成的，因此 **VMM 无须反射异常给 guest 处理**（忽略这个异常），可以直接执行 VMRESUME 指令恢复 guest 执行。

处理 NMI unblocking 位

当 VM-exit interruption information 字段的“NMI unblocking”位为 1 时，表明由于尝试执行 IRET 指令而解除了“blocking by NMI”阻塞状态，但是 **IRET 指令并没有成功执行**。

在这种情况下，VMM 在执行 VMRESUME 指令前，需要将 **interruptibility state 字段 bit 3 置 1**，用来反馈 guest 在执行 IRET 指令前存在“blocking by NMI”阻塞状态。恢复 guest 执行后，guest 完成 IRET 指令的执行将自动解除“blocking by NMI”阻塞状态。

当 pin-based VM-execution control 字段的“virtual NMIs”为 1（“NMI exiting”也为 1），VM-exit interruption information 字段的“NMI unblocking”为 1 时，表明尝试执行 IRET 指令而解除了“blocking by virtual-NMI”阻塞状态。

同样，VMM 需要将 **interruptibility state 字段 bit 3 置 1**，反馈 guest 在执行 IRET 指令前存在“blocking by virtual-NMI”阻塞状态，然后执行 VMRESUME 指令恢复 guest 执行。

注意：当 VM-exit interruption information 字段记录的向量号为 8（#DF 异常）时，VM-exit interruption information 字段的 bit 12 是**未定义**（参见 3.10.2.1 节），VMM 不需要对 interruptibility state 字段进行额外的设置。

7.1.2.2 例子 7-1

示例 7-1：实现对 SYSENTER 指令的监控

现在，我们用一个例子来说明 VMM 直接恢复 guest 执行而忽略产生的异常事件，这个例子实现了对 guest 用户层代码执行 SYSENTER 指令的监控。本例子的代码在 chap07\ex7-1 目录下。

1. 监控 SYSENTER 指令的原理及实现

用户层代码通过 SYSENTER 指令来快速切入 kernel 的服务例程，目标代码入口的线性地址提供在 IA32_SYSENTER_EIP 寄存器里。VMM 通过设置 IA32_SYSENTER_EIP 寄存器在 MSR write bitmap 区域里对应的位置 1，来监控 guest 更新 IA32_SYSENTER_EIP 寄存器。

当 guest 尝试使用 WRMSR 指令向 IA32_SYSENTER_EIP 寄存器写入系统服务例程入口地址时产生 VM-exit。VMM 将拦截 guest 的写入动作，使用一个预先定义的**签名值**来代替这个系统服务例程入口地址值写入 IA32_SYSENTER_EIP 寄存器。

当 guest 执行 SYSENTER 指令时，由于目标地址无效（写入的签名值）而产生 #PF 异常，VMM 拦截了 #PF 异常后（由于 #PF 异常而导致 VM-exit），VMM 检查引发 #PF 异常的源地址值是否为这个签名值，如果属于则找回 guest 原系统服务例程的入口地址，直接恢复 guest 的执行。

代码片段 7-1：

```
;;
;; 定义一个 SYSENTER_EIP hook 签名
;;
%define SYSENTER_HOOK_SIGN          'HOOK'
```

在 ex.asm 文件的开头，我们定义一个 SYSENTER_HOOK_SIGN 常量作为 hook IA32_SYSENTER_EIP 寄存器的签名值。

代码片段 7-2：

```
;;
;; 设置拦截对 IA32_SYSENTER_EIP 的写操作
;;
mov esi, IA32_SYSENTER_EIP
call set_msr_write_bitmap
```

在 init_guest_a 与 init_guest_b 函数里，调用 set_msr_write_bitmap 函数将 IA32_SYSENTER_EIP 寄存器在 MSR write bitmap 对应位置位。

代码片段 7-3：

```
;;
;; 注册另一个 WRMSR 指令处理例程
;;
mov esi, Ex.DOWRMSR
mov [DoVmExitRoutineTable + EXIT_NUMBER_WRMSR * 4], esi

;;
```

```
;; 注册另一个 VMM 的 page fault 处理例程
;;
mov esi, Ex.DoPageFault
mov [DoExceptionTable + 14 * 4], esi
```

接下来，在 TargetCpuVmentry 函数里分别注册 WRMSR 与 #PF 的处理例程：Ex.DoWRMSR 与 Ex.PageFault 函数。当由于 WRMSR 指令与 #PF 异常而产生 VM-exit 时，VMM 将调用它们来进行相关的处理。

2. 处理 WRMSR 指令产生的 VM-exit

guest 尝试写 IA32_SYSENTER_EIP 寄存器时将产生 VM-exit，VMM 将替换 guest 想要写入的值。

代码片段 7-4:

```
;------
; Ex 模块的 WRMSR 处理例程
;------
Ex.DOWRMSR:
    push R5
    push R3
    push R1
    push R2
    mov R5, [gs: PCB.Base]

    ;;
    ;; 当前 VM store block
    ;;
    mov R3, [R5 + PCB.CurrentVmbPointer]
    mov R3, [R3 + VMB.VsbBase]

    ;;
    ;; 检查是否属于 IA32_SYSENTER_EIP
    ;;
    mov eax, [R3 + VSB.Rcx]
    cmp eax, IA32_SYSENTER_EIP
    jne Ex.DOWRMSR.@1

    ;;
    ;; 设置 IA32_SYSENTER_EIP 值为 SYSENTER_HOOK_SIGN, 用于验证
    ;;
    SetVmcsField    GUEST_IA32_SYSENTER_EIP, SYSENTER_HOOK_SIGN

    ;;
    ;; 保存原 IA32_SYSENTER_EIP
    ;;
    mov eax, [R3 + VSB.Rax]
    mov edx, [R3 + VSB.Rdx]
    mov [Ex.SysenterEip], eax
    mov [Ex.SysenterEip + 4], edx

    DEBUG_RECORD    "[Ex.DOWRMSR]: set SYSENTER_HOOK_SIGN !"

    jmp Ex.DOWRMSR.@2

Ex.DOWRMSR.@1:
    ;;
```

```

;; 配置 VM-entry/VM-exit MSR-load/MSR-store
;;
mov esi, eax
mov eax, [R3 + VSB.Rax]
mov edx, [R3 + VSB.Rdx]
call append_vmentry_msr_load_entry

```

Ex.DoWRMSR.@2:

```

;;
;; 更新 guest-RIP
;;
call update_guest_rip

mov eax, VMM_PROCESS_RESUME
pop R2
pop R1
pop R5
pop R3
ret

```

Ex.DoWRMSR 函数从当前的 VSB (VM store block) 块里读取 guest 的 RCX 寄存器值, 然后判断是否为 IA32_SYSENTER_EIP 寄存器。

(1) 是, 则使用 SetVmcsField 宏设置 guest 的 IA32_SYSENTER_EIP 字段值为预定义的签名值。并且读取 guest 的 RAX 与 RDX 寄存器值, 保存 guest 想要写入的值。

(2) 否, 则读取 guest 的 RAX 与 RDX 寄存器值, 使用 append_vmentry_msr_load_entry 函数在 MSR-load 列表里添加这个 MSR 的表项, 在 VM-entry 时加载 guest 的 MSR 内容。

最后, 调用 update_guest_rip 函数跳过 guest 的 WRMSR 指令, 恢复 guest 的下一条指令执行。注意, VMM 必须要保存 guest 写入的原值, 以便后续处理及恢复。lib\Vm\Vm\Msr.asm 文件里也实现了一个 AppendMsrVte 函数用来保存写入 MSR 的原值。

3. 处理#PF 异常

guest 在尝试执行 SYSENTER 指令时产生#PF 异常从而导致 VM-exit。但是, 这个#PF 异常并不是由 guest 本身原因而导致的。VMM 的处理如代码片段 7-5 所示。

代码片段 7-5:

```

;-----
; Ex 模块的 page fault 处理例程
;-----
Ex.DoPageFault:
    push R5
    push R2
    mov R5, [gs: PCB.Base]

    ;;
    ;; 读取引发 #PF 的线性地址
    ;;
    mov R0, [R5 + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]

    ;;
    ;; 检查线性地址是否由 VMM 设置的地址值
    ;;
    cmp R0, SYSENTER_HOOK_SIGN

```

```

mov eax, VMM_PROCESS_DUMP_VMCS
jne Ex.DoPageFault.Reflect

;;
;; 读取原 IA32_SYSENTER_EIP 值
;;
mov R0, [Ex.SysenterEip]

DEBUG_RECORD "[Ex.DoPageFault]: call SysRoutine !"

;;
;; 更新 guest-RIP
;;
SetVmcsField GUEST_RIP, R0

jmp Ex.DoPageFault.Done

Ex.DoPageFault.Reflect:
;;
;; 反射 #PF 异常给 guest
;;
SetVmcsField VMENTRY_INTERRUPTION_INFORMATION, INJECT_EXCEPTION_PF
mov eax, [R5 + PCB.ExitInfoBuf + EXIT_INFO.InterruptonErrorCode]
SetVmcsField VMENTRY_EXCEPTION_ERROR_CODE, eax

Ex.DoPageFault.Done:
mov eax, VMM_PROCESS_RESUME
pop R2
pop R5
ret

```

Ex.DoPageFault 函数读取引发#PF 异常的线性地址值，判断是否为我们所设置的签名值。

(1) 是，则读取 guest 想写入 IA32_SYSENTER_EIP 寄存器的原值。使用 SetVmcsField 将这个值写入 guest RIP 字段，从而直接跳转到目标的服务例程里执行。

(2) 否，则需要注入一个#PF 异常给 guest 处理。

注意：当 Ex.DoPageFault 函数检查到不是 VMM 所设置的条件而引起#PF 异常时，VMM 应该反射#PF 异常交回 guest 处理。这里直接设置 VM-entry interruption information 字段，复制错误码到 VM-entry interruption error code 字段里。

4. guest 代码

下面是 guest 的部分代码，实现在 chap07\ex7-1\guest_ex.asm 文件里。

代码片段 7-6:

```

;;
;; 设置 SYSENTER 使用环境
;;
xor edx, edx
xor eax, eax
mov ax, cs
mov ecx, IA32_SYSENTER_CS
wrmsr

```

```

%if __BITS__ == 64
    mov rax, rsp
    shld rdx, rax, 32
%else
    mov eax, esp
    xor edx, edx
%endif
    mov ecx, IA32_SYSENTER_ESP
    wrmsr

%if __BITS__ == 64
    mov rax, 0FFFF800080000000h + SysRoutine
    shld rdx, rax, 32
%else
    mov eax, 80000000h + SysRoutine
    xor edx, edx
%endif
    mov ecx, IA32_SYSENTER_EIP
    wrmsr

    ;;
    ;; 下面进入 user 权限
    ;;

%if __BITS__ == 64
    push GuestUserSs64 | 3
    push 7FF0h
    push GuestUserCs64 | 3
    push GuestEx.UserEntry
    retf64
%else
    push GuestUserSs32 | 3
    push 7FF0h
    push GuestUserCs32 | 3
    push GuestEx.UserEntry
    retf
%endif

;;#####
;; 下面是 guest 的 User 层代码
;;#####

GuestEx.UserEntry:
%if __BITS__ == 64
    mov ax, GuestUserSs64
%else
    mov ax, GuestUserSs32
%endif
    mov ds, ax
    mov es, ax

    ;;
    ;; 调用系统服务例程
    ;;
    call FastSysCallEntry

```



```

mov esi, GuestEx.Msg2
call PutStr

jmp $

```

这段 guest 代码将配置 SYSENTER/SYSEXIT 的使用环境，IA32_SYSENTER_EIP 寄存器设置的目标地址为 SysRoutine 例程。

接着，guest 切换到 user 层代码。由于 SYSEXIT 指令只能返回到 3 级权限里，因此，SYSENTER 指令被期望在 3 级权限里执行。

在 user 层代码里，使用 FastSysCallEntry 函数来调用系统服务例程，然后打印出一条信息。

代码片段 7-7:

```

FastSysCallEntry:
%if __BITS__ == 64
    mov rcx, rsp
    mov rdx, [rsp]
%else
    mov ecx, esp
    mov edx, [esp]
%endif
    sysenter
    ret

```

FastSysCallEntry 只是一个中转 stub 函数，它执行 SYSENTER 指令切入到 SysRoutine 例程里。

代码片段 7-8:

```

;-----
; SYSENTER 指令的服务例程
;-----
SysRoutine:
    mov esi, GuestEx.Msg1
    call PutStr
    REX.Wrxb
    sysexit

```

作为示范例子，这个 SysRoutine 服务例程只是打印一条信息，不做其他任务。

5. 编译及运行

我们可以使用下面的命令编译及生成映像文件：

- build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64 -DGUEST_X64
- build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64
- build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE

在 VMware 里加载 demo.img 映像文件运行后，在 CPU0 的命令选择器里按下数字 <1> 键让 CPU1 进入 VM 执行，再按**两次**<F2> 键切换到 CPU1 的 guest 屏幕后，如图 7-1 所示。

```

[OS]: start ...
[OS]: initialize done ...
[OS]: running ...
[OS]: TSC = 0000000000018230

now, enter sysenter service routine... ← 调用 SysRoutine 例程
system service done ...

```

图 7-1

我们看到图 7-1 的下面打印了两条信息，一条信息是在 SysRoutine 服务例程里打印的，另一条信息是从 SysRoutine 例程返回后打印的。guest 在此期间并不知道发生了什么，它只知道运行是正常的。但我们知道，在 guest 的这个系统服务例程调用过程中已经被 VMM 监管和处理过。

下面，我们再来看一下调试记录信息。按下<F4>键切换到 CPU3，再按<Enter>键切换到 msg-list 列表，按多次向下箭头键翻到第 60 条记录上。如图 7-2 所示。

```

[CpuIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable

===== msg list record =====
<#58> [CPU1] [DoControlRegisterAccess]: invalidate cache !
<#59> [CPU1] [VMX]: resume to guest !
<#60> [CPU1] [Exit Reason]: VM-exit due to WRMSR ← 处理 WRMSR 引发的 VM-exit
<#61> [CPU1] [Ex.DoWRMSR]: set SYSENTER_HOOK_SIGN !
<#62> [CPU1] [VMX]: resume to guest !
<#63> [CPU1] [Exit Reason]: exception or NMI ← 处理 #PF 异常引发的 VM-exit
<#64> [CPU1] [Ex.DoPageFault]: call SysRoutine !
<#65> [CPU1] [VMX]: resume to guest !
<#66> [CPU1] [Exit Reason]: exception or NMI
<#67> [CPU1] [do_NMI]: call NMI handler !
<#68> [CPU1] [VMX]: resume to guest !
<#69> [CPU1] [Exit Reason]: exception or NMI
<#70> [CPU1] [do_NMI]: call NMI handler !
<#71> [CPU1] [VMX]: resume to guest !

```

图 7-2

第 60 条记录显示由于 WRMSR 指令发生了 VM-exit，第 61 条记录显示在 Ex.DoWRMSR 函数里进行了一些处理：设置了 SYSENTER_HOOK_SIGN 值。第 63 条记录显示由于异常而引发 VM-exit，第 64 条记录显示在 Ex.DoPageFault 函数里恢复 guest 调用 SysRoutine 服务例程。

敬请留意：在只有 2 个逻辑处理器的情况下，CPU0 进入 VM 执行，CPU1 显示调试记录，需要按<F2>键切换到 CPU1 来观察 msg-list 列表。

通过这个例子，说明了当异常（例子中的#PF 异常）是由于 VMM 所设定条件而引起时，VMM 不应该让 guest 来处理这个异常。VMM 忽略这个异常直接恢复 guest 运行（当然这里做了一些额外处理）。

7.1.2.3 恢复原始向量事件

在 guest 运行过程中产生一个异常，但这个异常不直接导致 VM-exit。而在这个异常的 delivery 期间遇到了下面的错误而导致 VM-exit（或者 VMM 注入一个向量事件，在向

量事件 delivery 期间遇到错误)：

- 遇到另一个嵌套的异常。
- 尝试执行任务切换。
- 遇到 EPT violation 故障。
- 遇到 EPT misconfiguration 故障。
- 尝试访问 APIC-access page 页面内容。

此时，IDT-vectoring information 字段的 bit 31 为 1，并且记录 guest 的这个原始向量事件的 delivery 情况。但是，这些错误是由于 host/VMM 的原因而产生的（但不包括任务切换，这里假设异常 delivery 期间遇到的另一个异常也是由于 host/VMM 原因而产生的）。

此时，VMM 修复这些错误后需要注入 **guest 原始异常/向量事件**（这个异常是由于 guest 本身产生）。VMM 应该将 IDT-vectoring information 字段的值复制到 VM-entry interruption information 字段里。

IDT-vectoring information 字段 bit 11 为 1 时，VMM 也需要将 IDT-vectoring error code 字段值复制到 VM-entry interruption error code 字段里。然后执行 VMRESUME 指令恢复 guest 原始异常的处理。

当原始向量事件的类型为**软件中断**、**特权级软件中断**或者**软件异常**时（属于注入事件或者 guest 执行了软件中断/软件异常指令），VMM 也需要将 VM-exit instruction length 字段的值复制到 VM-entry instruction length 字段里。

1. 处理 NMI unblocking

IDT-vectoring information 字段的 bit 12 为未定义（参见 3.10.3.1 节），有可能为 1 值。VM-entry interruption information 字段的 bits 30:12 必须为 0 值。因此，在复制后需要清 VM-entry interruption information 字段 bit 12。

假如，在异常 delivery 期间由于 EPT violation 故障而导致 VM-exit，exit qualification 字段记录 EPT violation 的明细信息，但由于间接引发 VM-exit，此时它的 bit 12 是未定义位（参见 3.10.1.14 节）。VMM 此时应忽略 exit qualification 字段的 bit 12。

2. 处理 NMI 阻塞状态

当注入一个 virtual-NMI 事件时（pin-based VM-execution control 字段的“NMI exiting”与“virtual-NMIs”位都为 1 值），在注入的 virtual-NMI 事件 delivery 期间遇到错误而导致 VM-exit，那么在 VM-exit 开始前就存在“blocking by virtual-NMI”阻塞状态（参见 5.10.2 节）。

此时，interruptibility state 字段记录的 bit 3 为 1 值，指示存在 NMI 阻塞状态。VMM 在重新恢复原始异常让 guest 处理时（virtual-NMI 事件），需要清 interruptibility state 字段的 bit 3，否则将产生 VM-entry 失败（参见 4.5.8 节）。

7.1.3 处理任务切换

在 VMX non-root operation 模式内，尝试 TSS 机制的任务切换会产生 VM-exit。包括下面的途径：

- guest 使用 TSS selector 或者 Task-gate selector 作为 far pointer 来执行 CALL、JMP 指令。
- guest 执行 IRET 指令时，由于 EFLAGS.NT = 1 而发起任务切换。
- 在一个向量事件（中断或异常）delivery 期间尝试任务切换，即向量号在 guest-IDT 对应的描述符是 task-gate 描述符。

由于 VMX 不支持 guest 进行任务切换（利用 TSS 机制），因此在 guest 由于尝试进行任务切换而产生 VM-exit 时，VMM 需要模拟 guest 的任务切换来代替处理器的操作（guest 运行在 legacy 模式下）。

VMM 整个模拟任务切换的过程大致可以分为三个阶段：

- (1) 处理器检查是否满足任务切换条件阶段。
- (2) VMM 模拟处理器任务切换阶段。
- (3) VMM 根据情况进行 guest 相应恢复执行阶段。

其中，在第 2 阶段的任务切换处理非常烦琐。而在第 3 阶段里，VMM 需要根据情况反射一个异常给 guest 处理，或者跳转去执行 guest 的新任务。第 1 阶段的工作由处理器自动完成，主要是检查是否允许进行任务切换。

7.1.3.1 检查任务切换条件

进行 TSS 任务切换是一个非常复杂的过程，在 long-mode (IA-32e) 模式下处理器不支持 TSS 的任务切换机制。在由于任务切换而导致 VM-exit 之前，处理器进行一系列的检查，确认满足任务切换的条件。如果检查不通过则引发异常，如果检查通过则产生 VM-exit。

1. 使用 TSS selector 进行任务切换

CALL 或 JMP 指令使用 TSS selector 作为 far pointer 进行任务切换时，处理器进行下面的检查。

- (1) 检查 selector 是否为 NULL selector。属于 NULL selector 时产生 #GP 异常。
- (2) 检查 selector 是否超过描述符表的 limit。超过 limit 则产生 #GP 异常。
- (3) 读取描述符进行类型检查：
 - 属于 TSS 描述符，在 IA-32e 模式下则产生 #GP 异常。
 - 属于其他不支持的类型（非代码段描述符、TSS 描述符以及 Task-gate 描述符），则产生 #GP 异常。

- (4) 检查访问权限。假如 TSS 描述符的 DPL 小于 CPL 或者 RPL，则产生#GP 异常。
- (5) 检查 TSS selector 的 TI 位 (bit 2)。TI = 1 时产生#GP 异常。
- (6) 检查 TSS 描述符是否为 busy。属于 busy 则产生#GP 异常。
- (7) 检查 TSS 描述符是否为 present。属于 not-present 则产生#NP 异常。

2. 使用 Task-gate 进行任务切换

CALL 或 JMP 指令使用 Task-gate selector 作为 far pointer 尝试任务切换。中断、异常或注入事件 delivery 期间引用 IDT 的 task-gate 描述符时，处理器进行下面的检查。

- (1) CALL 或 JMP 指令检查 Task-gate selector。
 - 检查 Task-gate selector 是否为 NULL selector。属于 NULL selector 时产生#GP 异常。
 - 检查 Task-gate selector 是否超过描述符表的 limit。超过 limit 时产生#GP 异常。
- (2) 中断、异常或者注入事件 deliver 时检查向量号。
 - 检查向量号是否超过 IDT limit。超过 IDT limit 时产生#GP 异常。
- (3) 根据 Task-gate selector 读取 Task-gate 描述符检查。
 - 在 IA-32e 模式下产生#GP 异常。
- (4) 检查访问权限，产生或者注入的 NMI、外部中断、硬件异常不需要检查权限。
 - 产生或者注入软件中断或软件异常 (INT、INT3 及 INTO)：Task-gate 描述符的 DPL 小于 CPL 时，产生#GP 异常。
 - 由 CALL 或 JMP 指令发起：Task-gate 描述符的 DPL 小于 CPL 或者 RPL 时，产生#GP 异常。
- (5) 检查 Task-gate 描述符是否为 present。属于 not-present 则产生#NP 异常。
- (6) 从 Task-gate 描述符里读取 TSS selector，检查 TSS selector。
 - TSS selector 的 TI 位 (bit 2) 为 1 时，产生#GP 异常。
 - TSS selector 超过 GDT limit 时，产生#GP 异常。
- (7) 根据 TSS selector 读取 TSS 描述符检查。
 - TSS 描述符属于 busy 时，产生#GP 异常。
 - TSS 描述符属于 not-present 时，产生#NP 异常。

3. 使用 IRET 进行任务切换

当执行 IRET 指令，EFLAGS.NT = 1 时，处理器进行下面的检查。

- (1) 在 IA-32e 模式下，则产生#GP 异常。
- (2) 从当前 TSS 块内的 Task-link 读取 TSS selector 检查。

- TSS selector 的 TI 位 (bit 2) 为 1 时, 产生#GP 异常。
- TSS selector 超过 GDT limit 时, 产生#GP 异常。

(3) 根据 TSS selector 读取 TSS 描述符检查。

- 如果不是 TSS 描述符, 产生#TS 异常。
- TSS 描述符不属于 busy 时, 产生#TS 异常。

(4) TSS 描述符属于 not-present 时, 产生#NP 异常。

在对上面三个途径下的任务切换进行相应的检查后, 处理器确认**允许执行任务切换的条件满足**。但是如前面所述, VMX non-root operation 下不允许进行任务切换。因此, 接着会产生 VM-exit。

7.1.3.2 VMM 处理任务切换

guest 在尝试任务切换时, 经过前面的满足条件检查后产生 VM-exit。VMM 拦截了 guest 任务切换动作, 但是 VMM 必须支持 guest OS 进行任务切换。因此, 在这个阶段里 VMM 的任务是帮助 guest 完成任务切换操作。

VMM 不能通过事件注入方式反射事件给 guest 处理, VMM 应该模拟处理器的任务切换动作。在\lib\Vm\VmVMM.asm 文件里实现了 DoTaskSwitch 函数处理 guest 的任务切换。

代码片段 7-9:

```

;-----
; DoTaskSwitch()
; input:
;     none
; output:
;     none
; 描述:
;     1) 处理由 task switch 引发的 VM-exit
;-----
DoTaskSwitch:
    push ebp
    push ecx
    push edx
    push ebx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    DEBUG_RECORD    "[DoTaskSwitch]: the VMM to complete the task switching"

    ;;
    ;; 收集任务切换 VM-exit 的相关信息
    ;;
    call GetTaskSwitchInfo

    ;;

```

```

;; ### VMM 需要模拟处理器的任务切换动作 ###
;; 注意:
;; 1) 不能使用事件注入重启任务切换!
;; 2) 这里的“当前”指“旧任务”
;;

mov ecx, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.Source]    ;; 读发起源

;;
;; step 1: 处理当前的 TSS 描述符
;; a) JMP, IRET 指令发起: 则清 busy 位。
;; b) CALL, 中断或异常发起: 则 busy 位保持不变 (原 busy 为 1)
;;
DoTaskSwitch.Step1:
    cmp ecx, TASK_SWITCH_JMP
    je DoTaskSwitch.Step1.ClearBusy
    cmp ecx, TASK_SWITCH_IRET
    jne DoTaskSwitch.Step2

DoTaskSwitch.Step1.ClearBusy:
    ;;
    ;; 清当前 TSS 描述符 busy 位
    ;;
    REX.Wrxb
    mov ebx, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.CurrentTssDesc]
    btr DWORD [ebx + 4], 9

    ;;
    ;; step 2: 在当前 TSS 里保存 context 信息
    ;;
DoTaskSwitch.Step2:
    REX.Wrxb
    mov ebx, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.CurrentTss] ;; 当前 TSS 块
    REX.Wrxb
    mov edx, [ebp + PCB.CurrentVmbPointer]
    REX.Wrxb
    mov edx, [edx + VMB.VsbBase]                                ;; 当前 VM store block

    ;;
    ;; 将 VSB 保存的 guest context 复制到当前 TSS 块
    ;;
    mov eax, [edx + VSB.Rax]
    mov [ebx + TSS32.Eax], eax                                ;; 保存 eax
    mov eax, [edx + VSB.Rcx]
    mov [ebx + TSS32.Ecx], eax                                ;; 保存 ecx
    mov eax, [edx + VSB.Rdx]
    mov [ebx + TSS32.Edx], eax                                ;; 保存 edx
    mov eax, [edx + VSB.Rbx]
    mov [ebx + TSS32.Ebx], eax                                ;; 保存 ebx
    mov eax, [edx + VSB.Rsp]
    mov [ebx + TSS32.Esp], eax                                ;; 保存 esp
    mov eax, [edx + VSB.Rbp]
    mov [ebx + TSS32.Ebp], eax                                ;; 保存 ebp
    mov eax, [edx + VSB.Rsi]
    mov [ebx + TSS32.Esi], eax                                ;; 保存 esi
    mov eax, [edx + VSB.Rdi]
    mov [ebx + TSS32.Edi], eax                                ;; 保存 edi
    mov eax, [edx + VSB.Rflags]
    mov [ebx + TSS32.Eflags], eax                            ;; 保存 eflags

```

```

;;
;; 注意: 保存 EIP 时需要加上指令长度
;;
mov eax, [edx + VSB.Rip]
add eax, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.InstructionLength]
mov [ebx + TSS32.Eip], eax                ;; 保存 eip

;;
;; 读取 guest selector 与 CR3 保存在当前 TSS 里
;;
GetVmcsField    GUEST_CS_SELECTOR
mov [ebx + TSS32.Cs], ax                ;; 保存 cs selector
GetVmcsField    GUEST_ES_SELECTOR
mov [ebx + TSS32.Es], ax                ;; 保存 es selector
GetVmcsField    GUEST_DS_SELECTOR
mov [ebx + TSS32.Ds], ax                ;; 保存 ds selector
GetVmcsField    GUEST_SS_SELECTOR
mov [ebx + TSS32.Ss], ax                ;; 保存 ss selector
GetVmcsField    GUEST_FS_SELECTOR
mov [ebx + TSS32.Fs], ax                ;; 保存 fs selector
GetVmcsField    GUEST_GS_SELECTOR
mov [ebx + TSS32.Gs], ax                ;; 保存 gs selector
GetVmcsField    GUEST_LDTR_SELECTOR
mov [ebx + TSS32.LdtrSelector], ax      ;; 保存 ldt selector
GetVmcsField    GUEST_CR3
mov [ebx + TSS32.Cr3], eax              ;; 保存 cr3

;;
;; step 3: 处理当前 TSS 内的 eflags.NT 标志位
;; a) IRET 指令发起: 则清 TSS 内 eflags.NT 位
;; b) CALL, JMP, 中断或异常发起: TSS 内 eflags.NT 位保持不变
;;
DoTaskSwitch.Step3:
    cmp ecx, TASK_SWITCH_IRET
    jne DoTaskSwitch.Step4
    ;;
    ;; 清当前 TSS 内的 eflags.NT 位
    ;;
    btr DWORD [ebx + TSS32.EFlags], 14

;;
;; step 4: 处理目标 TSS 的 eflags.NT 位
;; a) CALL, 中断或异常发起: 置 TSS 内的 eflags.NT 位
;; b) IRET, JMP 发起: 保持 TSS 内的 eflags.NT 位不变
;;
DoTaskSwitch.Step4:
    REX.Wrxb
    mov ebx, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.NewTaskTss] ;; 目标 TSS 块

    cmp ecx, TASK_SWITCH_CALL
    je DoTaskSwitch.Step4.SetNT
    cmp ecx, TASK_SWITCH_GATE
    jne DoTaskSwitch.Step5

DoTaskSwitch.Step4.SetNT:
    ;;
    ;; 置目标 TSS 的 eflags.NT 位

```



```

;;
bts DWORD [ebx + TSS32.EFlags], 14

;;
;; step 5: 处理目标 TSS 描述符
;; a) CALL, JMP, 中断或异常发起: 置 busy 位
;; b) IRET 发起: busy 位保持不变
;;
DoTaskSwitch.Step5:
    cmp ecx, TASK_SWITCH_IRET
    je DoTaskSwitch.Step6
    ;;
    ;; 置目标 TSS 描述符 busy 位
    ;;
    REX.Wrxb
    mov ebx, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.NewTaskTssDesc]
    bts DWORD [ebx + 4], 9

    ;;
    ;; step 6: 加载目标 TR 寄存器
    ;;
DoTaskSwitch.Step6:
    REX.Wrxb
    mov edx, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.NewTaskTssDesc]    ; 目标
TSS 描述符
    mov eax, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.NewTrSelector]    ; 目标
TSS selector

    SetVmcsField    GUEST_TR_SELECTOR, eax
    movzx eax, WORD [edx]
    SetVmcsField    GUEST_TR_LIMIT, eax    ;; 读取 limit
    movzx eax, WORD [edx + 5]              ;; 设置 TR.limit
    and eax, 0F0FFh                        ;; 读取 access rights
    SetVmcsField    GUEST_TR_ACCESS_RIGHTS, eax    ;; 设置 TR access rights
    mov eax, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.NewTaskTssBase]
    SetVmcsField    GUEST_TR_BASE, eax    ;; 设置 TR base

    ;;
    ;; step 7: 加载目标任务 context
    ;;
DoTaskSwitch.Step7:
    REX.Wrxb
    mov ebx, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.NewTaskTss] ;; 目标 TSS 块
    REX.Wrxb
    mov edx, [ebp + PCB.CurrentVmbPointer]
    REX.Wrxb
    mov edx, [edx + VMB.VsbBase]    ;; 当前 VM store block

    ;;
    ;; 将目标 TSS 内的值复制到当前 VSB 内
    ;;
    mov eax, [ebx + TSS32.Eax]
    mov [edx + VSB.Rax], eax    ;; 加载 eax
    mov eax, [ebx + TSS32.Ecx]
    mov [edx + VSB.Rcx], eax    ;; 加载 ecx
    mov eax, [ebx + TSS32.Edx]
    mov [edx + VSB.Rdx], eax    ;; 加载 edx
    mov eax, [ebx + TSS32.Ebx]
    mov [edx + VSB.Rbx], eax    ;; 加载 ebx

```

```

mov eax, [ebx + TSS32.Ebp]
mov [edx + VSB.Rbp], eax          ;; 加载 ebp
mov eax, [ebx + TSS32.Esi]
mov [edx + VSB.Rsi], eax          ;; 加载 esi
mov eax, [ebx + TSS32.Edi]
mov [edx + VSB.Rdi], eax          ;; 加载 edi

;;
;; 设置 guest ESP, EIP, EFLAGS, CR3
;;
mov eax, [ebx + TSS32.Esp]
SetVmcsField GUEST_RSP, eax        ;; 加载 esp
mov eax, [ebx + TSS32.Cr3]
SetVmcsField GUEST_CR3, eax        ;; 加载 cr3
mov eax, [ebx + TSS32.Eip]
SetVmcsField GUEST_RIP, eax        ;; 加载 eip
mov eax, [ebx + TSS32.Eflags]
SetVmcsField GUEST_RFLAGS, eax     ;; 加载 eflags

;;
;; 加载 SS
;;
mov esi, [ebx + TSS32.Ss]
call load_guest_ss_register
cmp eax, TASK_SWITCH_LOAD_STATE_SUCCESS
jne DoTaskSwitch.Done

;;
;; 加载 CS
;;
mov esi, [ebx + TSS32.Cs]
call load_guest_cs_register
cmp eax, TASK_SWITCH_LOAD_STATE_SUCCESS
jne DoTaskSwitch.Done

;;
;; 加载 ES
;;
mov esi, [ebx + TSS32.Es]
call load_guest_es_register
cmp eax, TASK_SWITCH_LOAD_STATE_SUCCESS
jne DoTaskSwitch.Done

;;
;; 加载 DS
;;
mov esi, [ebx + TSS32.Ds]
call load_guest_ds_register
cmp eax, TASK_SWITCH_LOAD_STATE_SUCCESS
jne DoTaskSwitch.Done

;;
;; 加载 FS
;;
mov esi, [ebx + TSS32.Fs]
call load_guest_fs_register
cmp eax, TASK_SWITCH_LOAD_STATE_SUCCESS
jne DoTaskSwitch.Done

;;

```

```

;; 加载 GS
;;
mov esi, [ebx + TSS32.Gs]
call load_guest_gs_register
cmp eax, TASK_SWITCH_LOAD_STATE_SUCCESS
jne DoTaskSwitch.Done

;;
;; 加载 LDTR
;;
mov esi, [ebx + TSS32.LdtrSelector]
call load_guest_ldtr_register
cmp eax, TASK_SWITCH_LOAD_STATE_SUCCESS
jne DoTaskSwitch.Done

;;
;; step 8: 在目标 TSS 内保存当前 TR selector
;;
DoTaskSwitch.Step8:
mov eax, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.CurrentTrSelector]
mov [ebx + TSS32.TaskLink], ax          ;; 保存 task link

;;
;; step 9: 设置 CR0.TS 位
;;
DoTaskSwitch.Step9:
GetVmcsField    GUEST_CR0
or eax, CR0_TS
SetVmcsField    GUEST_CR0, eax

DoTaskSwitch.Done:
mov eax, VMM_PROCESS_RESUME
pop ebx
pop edx
pop ecx
pop ebp
ret

```

DoTaskSwitch 函数的工作是模拟处理器的任务切换操作，处理器执行任务切换的动作大致有下面的 9 个步骤。下面所指的“当前”任务为**旧任务**，“目标”任务为需要切换的**新任务**。发起源的中断或异常包括 NMI、pending debug exception 与注入事件。

(1) 处理当前的 TSS 描述符：

- 由 JMP 和 IRET 指令发起时，清 busy 位。
- 由 CALL、中断或异常发起时，保持 busy 位不变（busy 位为 1）。

(2) 在当前 TSS 里保存 guest 的 context 信息。

(3) 处理当前 TSS 内 EFLAGS 寄存器的 NT 标志位：

- 由 IRET 指令发起时，清当前 TSS 内的 eflags.NT 位。
- 由 CALL、JMP、中断或异常发起时，保持 eflags.NT 位不变。

(4) 处理目标 TSS 内 EFLAGS 寄存器的 NT 标志位：

- 由 CALL、中断或异常发起时，置目标 TSS 内的 eflags.NT 位。

- 由 IRET、JMP 指令发起时，保持目标 TSS 内的 eflags.NT 位不变。

(5) 处理目标 TSS 描述符：

- 由 CALL、JMP、中断或异常发起时，置目标 TSS 描述符的 busy 位。
- 由 IRET 指令发起时，目标 TSS 描述符的 busy 位保持不变。

(6) 根据 TSS selector（或者 Task-gate 描述符内的 selector）加载目标 TR 寄存器。

(7) 从目标 TSS 内加载新任务的 context 信息。

(8) 在目标 TSS 内保存当前的 TR selector 作为 task-link。

(9) 置 CR0 寄存器的 TS 位（bit 3）。

在这 9 个步骤完成后，处理器转入执行新任务。

1. 收集任务切换信息

VMM 能模拟处理器的任务切换操作，前提是必须有能力访问 guest 的数据。譬如，访问 guest 的 GDT 等。在 6.3.1 节里介绍了 VMM 为每个 VM 分配一个 domain 区域。那么，VMM 通过访问属于 guest 的 VM domain 区域来达到访问 guest 的所有数据。

在 lib\Vmx\VmxExit.asm 模块里实现了在 guest 产生 VM-exit 后的收集信息例程。其中 GetTaskSwitchInfo 函数实现收集任务切换时的信息。

代码片段 7-10：

```

;-----
; GetTaskSwitchInfo()
; input:
;     none
; output:
;     none
; 描述:
;     1) 收集由 exception 或者 NMI 引发的 vector 信息
;-----
GetTaskSwitchInfo:
    push ebp
    push ebx
    push edx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    REX.Wrbx
    lea ebx, [ebp + PCB.GuestExitInfo]

    ;;
    ;; 读取 VM-exit 信息
    ;;
    mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]
    movzx esi, ax
    mov [ebx + TASK_SWITCH_INFO.NewTrSelector], esi        ; 记录目标 TSS
selector

```

```

shr eax, 30
and eax, 3
mov [ebx + TASK_SWITCH_INFO.Source], eax          ; 任务切换源
GetVmcsField    GUEST_TR_SELECTOR
mov [ebx + TASK_SWITCH_INFO.CurrentTrSelector], eax      ; 记录当前 TSS selector

;;
;; 读取指令长度
;;
mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.InstructionLength]
mov [ebx + TASK_SWITCH_INFO.InstructionLength], eax

;;
;; 读取 GDT/IDT base
;;
GetVmcsField    GUEST_GDTR_BASE
REX.Wrxb
mov esi, eax
call get_system_va_of_guest_va
REX.Wrxb
mov [ebx + TASK_SWITCH_INFO.GuestGdtBase], eax
REX.Wrxb
mov edx, eax          ; edx = GuestGdtBase
GetVmcsField    GUEST_IDTR_BASE
REX.Wrxb
mov esi, eax
call get_system_va_of_guest_va
REX.Wrxb
mov [ebx + TASK_SWITCH_INFO.GuestIdtBase], eax

;;
;; 读取 GDT/IDT limit
;;
GetVmcsField    GUEST_GDTR_LIMIT
mov [ebx + TASK_SWITCH_INFO.GuestGdtLimit], eax
GetVmcsField    GUEST_IDTR_LIMIT
mov [ebx + TASK_SWITCH_INFO.GuestIdtLimit], eax

;;
;; 读取 current/new-task TSS 描述符地址
;;
mov eax, [ebx + TASK_SWITCH_INFO.CurrentTrSelector]
REX.Wrxb
lea eax, [edx + eax]          ; Gdt.Base + selector
REX.Wrxb
mov [ebx + TASK_SWITCH_INFO.CurrentTssDesc], eax      ; 记录当前 TSS 描述符地址
mov eax, [ebx + TASK_SWITCH_INFO.NewTrSelector]      ; 目标 TSS selector
REX.Wrxb
lea eax, [edx + eax]          ; 目标 TSS 描述符地址 = Gdt.Base + selector
REX.Wrxb
mov [ebx + TASK_SWITCH_INFO.NewTaskTssDesc], eax

;;
;; 读取 current TSS 地址
;;
GetVmcsField    GUEST_TR_BASE
REX.Wrxb
mov esi, eax
call get_system_va_of_guest_va

```

```

    REX.Wrxb
    mov [ebx + TASK_SWITCH_INFO.CurrentTss], eax           ; 当前 TSS 地址

    ;;
    ;; 读取 new-task TSS 地址
    ;; *** 注意, 不需要读取 64 位 TSS 地址。在 longmode 下不支持任务切换! ***
    ;;
    REX.Wrxb
    mov eax, [ebx + TASK_SWITCH_INFO.NewTaskTssDesc]
    mov esi, [eax]                                         ; 描述符 low 32
    mov edi, [eax + 4]                                     ; 描述符 high 32
    shr esi, 16
    and esi, 0FFFFh                                       ; TSS 地址 bits 15:0
    mov eax, edi
    and eax, 0FF000000h                                    ; TSS 地址 bits 31:24
    shl edi, (23 - 7)
    and edi, 00FF0000h                                    ; TSS 地址 bits 23:16
    or edi, eax
    or esi, edi                                           ; TSS 地址 bits 31:0
    mov [ebx + TASK_SWITCH_INFO.NewTaskTssBase], esi     ; TR.base 的 GPA
    call get_system_va_of_guest_va
    REX.Wrxb
    mov [ebx + TASK_SWITCH_INFO.NewTaskTss], eax         ; 目标 TSS 地址

    pop edx
    pop ebx
    pop ebp
    ret

```

GetTaskSwitchInfo 函数收集任务切换而导致 VM-exit 的相关信息, 以供 VMM 作为处理的依据。收集信息包括下面几点。

- 当前 TSS selector 与目标 TSS selector。
- 任务切换的发起源。
- 发起任务切换指令的长度。包括由注入软件中断、特权级软件中断或者软件异常时发起的指令长度 (参见 4.4.3.3 节)。
- guest 的 GDT 与 IDT 基址。注意: 这些基址是 **guest 虚拟地址**对应的 **host 虚拟地址**。
- guest 的 GDT 与 IDT 的 limit 值。
- guest 的当前 TSS 描述符地址。同样, 这个地址是 host 虚拟地址。
- guest 的目标 TSS 描述符地址 (新任务 TSS 描述符)。同样, 这个地址是 host 虚拟地址。
- guest 的当前 TSS 段地址。同样, 这个地址是 host 虚拟地址。
- guest 的目标 TSS 段地址。同样, 这个地址是 host 虚拟地址。
- guest 的目标 TSS 段的 guest-linear address。

这些信息收集到处理器 PCB 块内的 TASK_SWITCH_INFO 结构区域里, 这个结构定义在 inc\VmxEit.inc 文件里。

DoTaskSwitch 例程正是通过这些数据进行任务切换处理。它通过收集的 TSS 描述符和 TSS 段地址来访问 guest 的 TSS 描述符和 TSS 段, 进行相应的设置和保存或加载

context 信息。

GetTaskSwitchInfo 例程使用 `get_system_va_of_guest_va` 函数将 guest-linear address 转换为 host 的线性地址值。这个 `get_system_va_of_guest_va` 函数的定义如下所示。

```
PVOID
get_system_va_of_guest_va(
    PVOID    GuestLinearAddress
);
```

输入的参数是 guest-linear address 值，而返回的是 host-linear address 值。内部调用 `get_guest_pa_of_guest_va` 函数将 guest-linear address 转换为 guest-physical address，再调用 `get_system_va_of_guest_pa` 函数将 guest-physical address 转换为最终的 host-linear address 值。这些函数实现在 `lib\Vm\VmLib.asm` 文件里。

2. 保存当前 context 信息

在代码片段 7-9 的第 2 步骤里，DoTaskSwitch 例程在当前 TSS 里保存当前（旧任务）的 context 信息。原理是：由于在 VM-exit 时，guest 的 context 信息保存在 VM 对应的 VSB（VM store block）区域内。因此将 VSB 区域的 context 数据复制到 guest 的当前 TSS 段里。

注意：在保存 EIP 值时，**必须将 EIP 值加上指令长度**，令 EIP 值指向引起任务切换操作指令的下一条指令。如代码片段 7-11 所示。

代码片段 7-11:

```
;;
;; 注意：保存 EIP 时需要加上指令长度
;;
mov eax, [edx + VSB.Rip]
add eax, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.InstructionLength]
mov [ebx + TSS32.Eip], eax                ;; 保存 eip
```

当保存 CS、ES、DS、SS、FS、GS 以及 LDTR 寄存器 selector 值时，需要使用 `GetVmcsField` 宏从当前 VMCS 区域里读取，然后写入当前 TSS 段里。

3. 加载目标任务 context 信息

在代码片段 7-9 的第 7 步骤里，DoTaskSwitch 例程从目标 TSS 段内加载新任务的 context 信息。原理是：由于在恢复 guest 运行时，VMM 从 VM 对应的 VSB 区域内恢复 guest 的 context 信息。因此，需要从目标 TSS 段内将 context 信息复制到 VSB 区域内。

注意：在加载 ESP、EIP、EFLAGS 以及 CR3 寄存器时，需要使用 `SetVmcsField` 宏写入当前 VMCS 区域内。如代码片段 7-12 所示。

代码片段 7-12:

```
;;
;; 设置 guest ESP, EIP, EFLAGS, CR3
;;
mov eax, [ebx + TSS32.Esp]
SetVmcsField    GUEST_RSP, eax                ;; 加载 esp
mov eax, [ebx + TSS32.Cr3]
```

```

SetVmcsField    GUEST_CR3, eax          ;; 加载 cr3
mov eax, [ebx + TSS32.Eip]
SetVmcsField    GUEST_RIP, eax          ;; 加载 eip
mov eax, [ebx + TSS32.Eflags]
SetVmcsField    GUEST_RFLAGS, eax       ;; 加载 eflags

;;
;; 加载 SS
;;
mov esi, [ebx + TSS32.Ss]
call load_guest_ss_register
cmp eax, TASK_SWITCH_LOAD_STATE_SUCCESS
jne DoTaskSwitch.Done

```

接着分别使用了 `load_guest_ss_register`、`load_guest_cs_register`、`load_guest_es_register`、`load_guest_ds_register`、`load_guest_fs_register`、`load_guest_gs_register` 以及 `load_guest_ldtr_register` 函数来加载 SS、CS、ES、DS、FS、GS 以及 LDTR 寄存器。这些函数实现在 `lib\Vm\VmLib.asm` 模块里。

注意：在代码片段 7-9 第 7 步的加载操作里有可能会产生错误而引发 #TS 异常，我们将在后面的第 7.1.3.3 节里进行阐述。

`DoTaskSwitch` 例程的 9 个步骤处理完毕后，代表着任务切换的第二阶段完成。

7.1.3.3 恢复 guest 运行

在第三阶段里，VMM 将根据第二阶段的处理结果来决定是让 guest 转入新任务运行，还是反射异常让 guest 处理。在《Intel 手册》的描述里，第二阶段的任何一个步骤都可能产生错误，第二阶段是一个相当复杂的阶段。作为演示，`DoTaskSwitch` 例程并没有完全考虑到所有可能发生的情况。

下面以 `DoTaskSwitch` 例程代码片段的第 7 步中的加载 SS 寄存器为例子，说明第 7 步可能发生的错误情况。

代码片段 7-13:

```

;-----
; load_guest_ss_register()
; input:
;     esi - selector
; output:
;     eax - error code
; 描述:
;     1) 从 guest GDT 里加载 SS 寄存器
;     2) 在 task switch 例程内部使用!
;-----
load_guest_ss_register:
    push ebp
    push ebx
    push edx
    push ecx

#ifdef __x64
    LoadGsBaseToRbp
#else

```



```

    mov ebp, [gs: PCB.Base]
%endif

    mov ecx, esi
    REX.Wrxb
    mov ebx, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.NewTaskTss]
    REX.Wrxb
    mov edx, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.GuestGdtBase]

    ;;
    ;; 进行检查, 包括:
    ;; 1) SS selector 是否为 NULL selector
    ;; 2) SS selector 是否超出 limit
    ;; 3) SS.RPL == SS.DPL == CS.RPL, CS.DPL 视情况而定
    ;; 4) SS 段是否为可写数据段
    ;;
    test cx, 0FFF8h
    jz load_guest_ss_register.Error

    ;;
    ;; (selector & 0xFFF8 + 7) > limit ?
    ;;
    and esi, 0FFF8h
    add esi, 7
    cmp esi, [ebp + PCB.GuestExitInfo + TASK_SWITCH_INFO.GuestGdtLimit]
    ja load_guest_ss_register.Error

    ;;
    ;; 检查 SS.RPL == CS.RPL
    ;;
    movzx eax, WORD [ebx + TSS32.Cs]
    xor eax, ecx
    test eax, 03h
    jnz load_guest_ss_register.Error

    ;;
    ;; 检查 SS.RPL == SS.DPL
    ;;
    mov esi, ecx
    shl esi, 13
    movzx eax, WORD [ebx + TSS32.Ss]
    and eax, 0FFF8h
    mov eax, [edx + eax + 4]
    xor esi, eax
    test esi, 6000h
    jnz load_guest_ss_register.Error

    ;;
    ;; 检查 SS.DPL == CS.DPL
    ;;
    movzx esi, WORD [ebx + TSS32.Cs]
    and esi, 0FFF8h
    mov esi, [edx + esi + 4]
    xor esi, eax
    test esi, 6000h
    jz load_guest_ss_register.@1

    ;;
    ;; SS.DPL <> CS.DPL 时, 检查是否为 conforming 段
    ;;

```

```

xor esi, eax
test esi, (1 << 10)
jz load_guest_ss_register.Error

;;
;; 检查 CS.DPL <= SS.RPL
;;
shr esi, 13
and esi, 03h                ;; CS.DPL
mov edi, ecx
and edi, 03                ;; SS.RPL
cmp esi, edi
ja load_guest_ss_register.Error    ;; CS.DPL > SS.RPL 时出错

load_guest_ss_register.@1:
;;
;; 检查 SS 描述符: P = S = W = 1
;;
test eax, 9200h
jz load_guest_ss_register.Error

;;
;; ### 下面加载 selector, limit, base, access rights ###
;;

;;
;; 设置 selector
;;
SetVmcsField    GUEST_SS_SELECTOR, ecx

;;
;; 找到 SS segment 描述符
;;
mov esi, ecx
and esi, 0FFF8h
REX.Wrxb
add edx, esi

;;
;; 设置 limit
;;
movzx eax, WORD [edx]                ; limit bits 15:0
mov esi, [edx + 4]
and esi, 0F0000h                    ; limit bits 19:16
or eax, esi
;;
;; G = 1 时: limit32 = limit20 * 1000h + 0FFFh
;;
test DWORD [edx + 4], (1 << 23)
jz load_guest_ss_register.@2
shl eax, 12
or eax, 0FFFh
load_guest_ss_register.@2:
SetVmcsField    GUEST_SS_LIMIT, eax

;;
;; 设置 base
;;

```

```

        mov esi, [edx]                ; 描述符 low 32
        mov edi, [edx + 4]            ; 描述符 high 32
        shr esi, 16
        and esi, 0FFFFh              ; base bits 15:0
        mov eax, edi
        and eax, 0FF000000h          ; base bits 31:24
        shl edi, (23 - 7)
        and edi, 00FF0000h          ; base bits 23:16
        or eax, esi
        or eax, edi                  ; base bits 31:0
        SetVmcsField    GUEST_SS_BASE, eax

        ;;
        ;; 设置 access rights
        ;;
        movzx eax, WORD [edx + 5]
        and eax, 0F0FFh
        SetVmcsField    GUEST_SS_ACCESS_RIGHTS, eax

        mov eax, TASK_SWITCH_LOAD_STATE_SUCCESS
        jmp load_guest_ss_register.Done

load_guest_ss_register.Error:
        ;;
        ;; 发生错误时, 注入 #TS 异常
        ;;
        SetVmcsField    VMENTRY_EXCEPTION_ERROR_CODE, ecx
        SetVmcsField    VMENTRY_INTERRUPTINFORMATION, INJECT_EXCEPTION_TS

        mov eax, TASK_SWITCH_LOAD_STATE_ERROR

load_guest_ss_register.Done:
        pop ecx
        pop edx
        pop ebx
        pop ebp
        ret

```

load_guest_ss_register 函数模拟处理器加载 SS 段寄存器的过程。在成功加载之前进行下面的检查：

- (1) 检查 SS selector 是否为 NULL selector，如果是则产生 #TS 异常。
- (2) 检查 SS selector 是否超出描述符表的 limit，如果是则产生 #TS 异常。
- (3) 检查权限（参见 4.5.4.2 节）。必须要 **SS.RPL == SS.DPL == CS.RPL**，而 CS.DPL 则取决于代码段是否为 conforming 类型而决定。如果检查不通过则产生 #TS 异常。
 - 属于 conforming 段时，需要 **CS.DPL <= SS.DPL**。
 - 属于 non-conforming 段时，需要 **CS.DPL == SS.DPL**。
- (4) 检查 SS 段是否为可写数据段。必须要 SS 描述符的 **P = S = W = 1**，否则产生 #TS 异常。

当检查到产生 #TS 异常条件时，load_guest_ss_register 函数将注入 #TS 异常事件反射给 guest 处理。在使用 load_guest_cs_register 函数加载 CS 寄存器时，使用相同的权限检查。

在对 CS 描述符进行检查时，使用下面的代码。

代码片段 7-14:

```
;;
;; 检查 CS 描述符: P = S = C/D = 1
;;
test esi, 9800h
jz load_guest_cs_register.Error
```

CS 描述符的 P、S 以及 C/D 位必须为 1，表明可执行的代码段。我们看到在 context 加载阶段加载任何一个寄存器值出错时都会产生 #TS 异常。

因此，VMM 在第三阶段里的处理如下。

- 在 DoTaskSwitch 例程处理 guest 数据阶段发生错误，VMM 不能直接恢复 guest 的运行，应该将错误反射给 guest 处理。例如下面的错误：
 - 由于目标 TSS 段的地址不可用而产生 #PF 异常。
 - 由于在加载目标 TSS 内的 guest context 时产生 #TS 异常。
- DoTaskSwitch 例程处理过程没有发生错误，完成后 VMM 让 guest 转入执行新任务。

7.1.3.4 例子 7-2

示例 7-2: 实现 guest 的任务切换

有了前面的知识，现在我们完成这个例子，实现在 legacy 模式下对 guest 任务切换的支持。这是保证 guest 环境正确运行的必要条件。

1. guest 代码

在 guest 端里，我们构造了一个任务切换的环境，如下面的代码所示。

代码片段 7-15:

```
%ifndef GUEST_X64
;;
;; 新任务所使用的 TSS selector
;;
NewTssSelector      EQU    GuestReservedSel0
TargetTaskSelector  EQU    GuestReservedSel0

;;
;; 构造一个 TSS 描述符
;;
sgdt [GuestEx.GdtPointer]
mov ebx, [GuestEx.GdtBase]
mov DWORD [ebx + NewTssSelector + 4], GuestEx.Tss
mov DWORD [ebx + NewTssSelector + 2], GuestEx.Tss
mov WORD [ebx + NewTssSelector], 67h
mov WORD [ebx + NewTssSelector + 5], 89h

;;
;; 构造一个 task-gate 描述符
;;
sidt [GuestEx.IdtPointer]
mov ebx, [GuestEx.IdtBase]
```

```

mov WORD [ebx + 60h * 8 + 2], NewTssSelector
mov BYTE [ebx + 60h * 8 + 5], 85h

;;
;; 设置新任务的 TSS 块内容
;;
mov ebx, GuestEx.Tss
mov DWORD [ebx + TSS32.Esp], 7F00h
mov WORD [ebx + TSS32.Ss], GuestKernelSs32
mov eax, cr3
mov [ebx + TSS32.Cr3], eax
mov DWORD [ebx + TSS32.Eip], GuestEx.NewTask
mov DWORD [ebx + TSS32.EFlags], FLAGS_IF | 02h
mov WORD [ebx + TSS32.Cs], GuestKernelCs32
mov WORD [ebx + TSS32.Ds], GuestKernelSs32

;;
;; ### 下面进行任务切换操作。###
;; 注意：不要使用 INT 指令！
;; 因为，在例子 ex7-4 里拦截了执行 INT 指令，并没有对中断的任务切换进行处理！！
;; 所以，这里使用 CALL 指令进行任务切换！！
;;
call TargetTaskSelector : 0

;;
;; 打印切换回来的信息
;;
mov esi, GuestEx.Msg2
call PutStr

jmp $

;;
;; ##### 目标任务 #####
;;
GuestEx.NewTask:
    mov esi, GuestEx.Msg1
    call PutStr
    cts
    iret

%endif

;;
;; GDT 表指针
;;
GuestEx.GdtPointer:
    GuestEx.GdtLimit    dw    0
    GuestEx.GdtBase     dd    0

GuestEx.IdtPointer:
    GuestEx.IdtLimit    dw    0
    GuestEx.IdtBase     dd    0

```

```

;;
;; TSS 区域
;;
ALIGNB 4
GuestEx.Tss:    times 104      db      0

GuestEx.Msg1    db      'now, switch to new task', 10, 0
GuestEx.Msg2    db      'now, switch back old task', 10, 0

```

Guest_ex.asm 文件实现在 chap07\ex7-2 目录下，它构造了一个新的 TSS 描述符、一个 task-gate 描述符，以及 TSS 段区域。在这个 TSS 段里只设置了必要的环境信息。

例子中，guest 使用 **CALL** 指令进行任务切换，目标任务是 GuestEx.NewTask 例程，它只是简单地输出一条信息“now, switch to new task”。注意，不要使用 INT 指令进行中断任务切换，由于在 7.3.1.5 节的例子 7-4 里拦截了 INT 指令，并且没有进行相应的处理，导致这里不能使用 INT 指令进行任务切换。

2. 编译及运行

由于在 IA-32e 模式下不支持任务切换，因此这个例子需要将 guest 编译为 legacy 模式下运行。可以使用下面的命令进行编译：

(1) build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64

(2) build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE

第 1 种方式将 host 编译为 64 位，将 guest 编译为 32 位。第 2 种方式将 host 编译为 32 位，将 guest 编译为 32 位。

在 VMware 加载 demo.img 映像文件运行后，按数字<1>键让 CPU1 发起 VM-entry 操作。按两次<F2>键切换到 CPU1 的 guest 屏幕，如图 7-3 所示。



图 7-3

图 7-3 的信息显示，guest 在切入到新任务后，在新任务里输出了一条信息“now, switch to new task”。然后从新任务切换回旧任务时，输出了一条信息“now, switch back old task”。guest 一共进行了两次任务切换，分别由 CALL 指令和 IRET 指令发起。

接下来我们看看 msg-list 信息，按<F4>键切换到 CPU3 屏幕，按<enter>键切换到 msg-list 列表，按向下键翻行，如图 7-4 所示。

```

Home x demo x
[CpuIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:8625 [VMX]:Enable [Ept]:Disable

===== msg list record =====
<#50> [CPU1] [VMM]: resume to guest !
<#51> [CPU1] [Exit Reason]: access to GDTR or IDTR
<#52> [CPU1] [VMM]: resume to guest !
<#53> [CPU1] [Exit Reason]: access to GDTR or IDTR
<#54> [CPU1] [VMM]: resume to guest !
<#55> [CPU1] [Exit Reason]: control-register access
<#56> [CPU1] [VMM]: resume to guest !
<#57> [CPU1] [Exit Reason]: occur task switch
<#58> [CPU1] [DoTaskSwitch]: the VMM to complete the task switching
<#59> [CPU1] [VMM]: resume to guest !
<#60> [CPU1] [Exit Reason]: occur task switch
<#61> [CPU1] [DoTaskSwitch]: the VMM to complete the task switching
<#62> [CPU1] [VMM]: resume to guest !
  
```

↑
VMM 完成了两次任务切换操作，一次由 CALL 指令发起，另一个由 IRET 指令发起！

图 7-4

图 7-4 的 msg-list 列表信息中显示共进行了两次任务切换：第 57 条和第 60 条显示由任务切换而产生 VM-exit，第 58 条和第 61 条显示 VMM 调用 DoTaskSwitch 例程帮助 guest 完成任务切换。

注意：此时 host 是运行在 IA-32e 模式，而 guest 运行在 legacy 保护模式。

7.2 Local APIC 虚拟化

每个逻辑处理器都有属于自己的 local APIC 组件。当 guest 读取 local APIC 寄存器时，将返回物理的 local APIC 寄存器值。同样，当 guest 写 local APIC 寄存器时，也将写入物理的 local APIC 寄存器。

例如，guest 可以通过写 local APIC 的 ICR 寄存器向其他处理器发送 IPI，也可以提升 local APIC 的 TPR 值来屏蔽外部中断。guest 的这些行为将更改物理资源，严重地干扰 host 环境。

为了保护物理资源，VMM 必须限制 guest 对物理 local APIC 的访问。VMM 可以选择使用下面的方案来限制访问 local APIC。

- 通过 EPT 映射机制或者 MSR-bitmap 限制访问。
- 通过 virtual-APIC 机制进行虚拟化处理 local APIC 访问。

virtual-APIC 机制是 VMX 为 local APIC 而引进的虚拟化内存机制。guest 代码对 local APIC 进行访问实际上访问的是 virtual-APIC page 区域。

7.2.1 监控 guest 访问 local APIC

VMM 通过 EPT 映射机制或者 MSR-bitmap 来限制 guest 访问 local APIC，其原理是监控 guest 访问 local APIC 的行为并做出相应处理。

1. 监控基于内存映射的 local APIC 访问

当 local APIC 使用 xAPIC 模式时，local APIC 寄存器通过 memory-mapped（内存映射）方式映射在物理空间的 4K 页面上，我们可以称这个 4K 页面为 APIC-page。APIC-page 物理基址在 IA32_APIC_BASE 寄存器的 bits $N-1:12$ 里提供（ $N = \text{MAXPHYADDR}$ ）。

在这种情况下，VMM 可以通过 EPT 映射机制来监控 guest 访问 local APIC。

2. 监控基于 MSR 的 local APIC 访问

当 local APIC 使用 x2APIC 模式时（xAPIC 的扩展模式），local APIC 寄存器映射到 MSR 空间上，MSR 地址范围为 **800H~8FFH**。guest 软件使用 RDMSR 指令读取 local APIC 寄存器，使用 WRMSR 指令写入 local APIC 寄存器。

在这种情况下，VMM 可以通过设置 MSR-bitmap 来监控 guest 访问 local APIC。

7.2.1.1 例子 7-3

示例 7-3：使用 EPT 机制实现 local APIC 虚拟化

上文提及，当 local APIC 使用 xAPIC 模式时，我们可以通过 EPT 映射机制来实现监控 guest 访问 local APIC。在这一节里，我们将使用 EPT 映射机制来实现 local APIC 虚拟化。

1. 监控 guest 访问 IA32_APIC_BASE 寄存器

VMM 必须监控 guest 对 IA32_APIC_BASE 寄存器的访问（包括读与写访问），只有这样才能够监控 guest 访问 local APIC 寄存器，如代码片段 7-16 所示。

代码片段 7-16：

```
;;
;; 设置拦截对 IA32_APIC_BASE 的读操作
;;
mov esi, IA32_APIC_BASE
call set_msr_read_bitmap

;;
;; 设置拦截对 IA32_APIC_BASE 的写操作
;;
mov esi, IA32_APIC_BASE
call set_msr_write_bitmap
```

在 chap07\ex7-3\ex.asm 模块的 init_guest_a 与 init_guest_b 函数里，分别设置了对 IA32_APIC_BASE 的读/写进行拦截。set_msr_read_bitmap 函数设置 MSR 的读访问限制，set_msr_write_bitmap 函数设置 MSR 的写访问限制。

2. 处理写 IA32_APIC_BASE 时产生的 VM-exit

当 guest 执行 WRMSR 指令尝试写 IA32_APIC_BASE 寄存器时产生 VM-exit。例如，下面的 chap07\ex7-3\guest_ex.asm 模块代码片段 7-17 所示。

代码片段 7-17:

```
;;
;; 设置 local APIC base 值为 01000000h
;;
mov ecx, IA32_APIC_BASE
mov eax, 01000000h | APIC_BASE_BSP | APIC_BASE_ENABLE
xor edx, edx
wrmsr
```

guest 企图设置 APIC-page 的基址为 01000000H。在产生 VM-exit 后, VMM 调用 DoWRMSR 函数处理由于 WRMSR 指令而产生的 VM-exit。

代码片段 7-18:

```
;;
;; 读取 MSR index
;;
mov ecx, [ebx + VSB.Rcx]
cmp ecx, IA32_APIC_BASE
jne DOWRMSR.@1

;;
;; 处理写 IA32_APIC_BASE 寄存器
;;
call DOWriteMsrForApicBase
```

DOWRMSR.@1:

DoWRMSR 函数实现在 lib\Vmx\VmxVMM.asm 文件里。它读取 VM-exit 时的 ECX 寄存器值, 属于 IA32_APIC_BASE 寄存器时, 调用 DoWriteMsrForApicBase 来处理 guest 对 IA32_APIC_BASE 寄存器的写操作。

代码片段 7-19:

```
;-----
; DOWriteMsrForApicBase()
; input:
;     none
; output:
;     none
; 描述:
;     1) 处理 guest 访问 IA32_APIC_BASE 寄存器
;-----
DOWriteMsrForApicBase:
    push ebp
    push ebx
    push edx
    push ecx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif
    REX.Wrb
    mov ebx, [ebp + PCB.CurrentVmbPointer]
    REX.Wrb
    mov ebx, [ebx + VMB.VsbBase]
```

```

;;
;; 读取 guest 写入的 MSR 值
;;
mov eax, [ebx + VSB.Rax]
mov edx, [ebx + VSB.Rdx]

DEBUG_RECORD "[DowriteMsrForApicBase]: write to IA32_APIC_BASE"

;;
;; ### 检查写入值是否合法 ###
;; 1) 保留位 (bits 7:0, bit 9, bits 63:N) 需为 0
;; 2) 检查 bit 11 与 bit 10 的设置
;;    a) 当 bit 11 = 1, bit 10 = 0 时, 设置 bit 11 = 1, bit 10 = 1 开启 x2APIC
模式
;;    b) 当 bit 11 = 0, bit 10 = 1 时, 无效
;;    c) 当 bit 11 = 0, bit 10 = 0 时, 关闭 local APIC
异常
;;    d) 当 bit 11 = 1, bit 10 = 1 时, 设置 bit 11 = 1, bit 10 = 0 时, 产生 #GP
;;

;;
;; 检查保留位, 不为 0 时注入 #GP 异常
;;
test eax, 2Fh
jnz DowriteMsrForApicBase.Error
mov esi, [ebp + PCB.MaxPhyAddrSelectMask + 4]
not esi
test edx, esi
jnz DowriteMsrForApicBase.Error

;;
;; 检查 xAPIC enable (bit 11) 与 x2APIC enable (bit 10)
;;
test eax, APIC_BASE_X2APIC
jz DowriteMsrForApicBase.Check.@1

;;
;; 当 bit 10 = 1 时, 检查 CPUID.01H:ECX[21].x2APIC 位
;; 1) 为 0 时表明不支持 x2APIC 模式, 注入 #GP(0) 异常
;;
test DWORD [ebp + PCB.CpuidLeaf01EcX], (1 << 21)
jz DowriteMsrForApicBase.Error

;;
;; 当 bit 10 = 1 时, bit 11 = 0, 无效设置则注入 #GP(0) 异常
;;
test eax, APIC_BASE_ENABLE
jz DowriteMsrForApicBase.Error

DowriteMsrForApicBase.x2APIC:
;;
;; 现在 bit 10 = 1, bit 11 = 1
;; 1) 使用 x2APIC 模式的虚拟化设置
;;
mov esi, IA32_APIC_BASE
call AppendMsrVte
;; 保存 guest 写入原值

```

```

;;
;; 检查 secondary processor-based VM-execution control 字段 “virtualize x2APIC
mode” 位
;; 1) 为 1 时, 使用 VMX 原生的 x2APIC 虚拟化, 直接返回
;; 2) 为 0 时, 监控 800H - 8FFH MSR 的读写
;;
GetVmcsField    CONTROL_PROCBASED_SECONDARY
test eax, VIRTUALIZE_X2APIC_MODE
jnz DOWriteMsrForApicBase.Done

;;
;; 现在监控 x2APIC MSR 的读写, 范围从 800H 到 8FFH
;;
call set_msr_read_bitmap_for_x2apic
call set_msr_write_bitmap_for_x2apic
jmp DOWriteMsrForApicBase.Done

DOWriteMsrForApicBase.Check.@1:
;;
;; bit 10 = 0, bit 11 = 0, 关闭 local APIC, 不进行虚拟化处理
;; 1) 写入 IA32_APIC_BASE 寄存器
;; 2) 恢复映射
;;
test eax, APIC_BASE_ENABLE
jnz DOWriteMsrForApicBase.Check.@2

;;
;; guest 尝试关闭 local APIC
;; 1) 恢复 guest 对 IA32_APIC_BASE 寄存器的写入
;; 2) 恢复 EPT 映射
;;
mov esi, IA32_APIC_BASE
mov eax, [ebx + VSB.Rax]
mov edx, [ebx + VSB.Rdx]
call append_vmentry_msr_load_entry

#ifdef __X64
    REX.Wrb
    mov esi, [ebx + VSB.Rax]
    mov edi, 0FEE00000h
    mov eax, EPT_WRITE | EPT_READ
    call do_guest_physical_address_mapping
#else
    mov esi, [ebx + VSB.Rax]
    mov edi, [ebx + VSB.Rdx]
    mov eax, 0FEE00000h
    mov edx, 0
    mov ecx, EPT_WRITE | EPT_READ
    call do_guest_physical_address_mapping
#endif

    jmp DOWriteMsrForApicBase.Done

DOWriteMsrForApicBase.Check.@2:
;;
;; 读取原 guest 设置的 IA32_APIC_BASE 值
;; 1) 假如返回 0 值, 则表明 guest 第 1 次写 IA32_APIC_BASE
;;
mov esi, IA32_APIC_BASE
call GetMsrVte

```

```

test eax, eax
jz DoWriteMsrForApicBase.xAPIC

;;
;; 如果原值 bit 11 = 1, bit 10 = 1, 当设置 bit 11 = 1, bit 10 = 0 时, 将产生 #GP 异常
;;
test DWORD [eax + MSR_VTE.Value], APIC_BASE_X2APIC
jnz DoWriteMsrForApicBase.Error

DoWriteMsrForApicBase.xAPIC:
;;
;; ### 下面虚拟化 local APIC 的 xAPIC 模式 ###
;;
mov esi, IA32_APIC_BASE
mov eax, [ebx + VSB.Rax]
mov edx, [ebx + VSB.Rdx]
call AppendMsrVte ; 保存 guest 写入值

REX.Wrb
mov edx, eax

;;
;; 1) 检查是否开启了 "virtualize APIC access"
;; a) 是, 则设置 APIC-access page 页面
;; b) 否, 则提供 GPA 例程处理 local APIC 访问
;; 2) 检查是否开启了 "enable EPT"
;; a) 是, 则映射 IA32_APIC_BASE[N-1:12], 将 APIC-access page 设置为该 HPA 值
;; b) 否, 则直接将 IA32_APIC_BASE[N-1:12] 设为 APIC-access page
;;

GetVmcsField CONTROL_PROCBASED_SECONDARY

test eax, VIRTUALIZE_APIC_ACCESS
jz DoWriteMsrForApicBase.SetForEptViolation
test eax, ENABLE_EPT
jz DoWriteMsrForApicBase.EptDisable

;;
;; 执行 EPT 映射到 0FEE00000H
;;

#ifdef __X64
    REX.Wrb
    mov esi, [edx + MSR_VTE.Value]
    mov edi, 0FEE00000h
    mov eax, EPT_READ | EPT_WRITE
    call do_guest_physical_address_mapping
#else
    mov esi, [edx + MSR_VTE.Value]
    mov edi, [edx + MSR_VTE.Value + 4]
    mov eax, 0FEE00000h
    mov edx, 0
    mov ecx, EPT_READ | EPT_WRITE
    call do_guest_physical_address_mapping
#endif

mov eax, 0FEE00000h
mov edx, 0
jmp DoWriteMsrForApicBase.SetApicAccessPage

```

```

DOWriteMsrForApicBase.EptDisable:
    REX.Wrb
    mov eax, [edx + MSR_VTE.Value]
    mov edx, [edx + MSR_VTE.Value + 4]
    REX.Wrb
    and eax, ~0FFFh

DOWriteMsrForApicBase.SetApicAccessPage:
    SetVmcsField    CONTROL_APIC_ACCESS_ADDRESS_FULL, eax
%ifndef __X64
    SetVmcsField    CONTROL_APIC_ACCESS_ADDRESS_HIGH, edx
%endif

    call update_guest_rip
    jmp DOWriteMsrForApicBase.Done

DOWriteMsrForApicBase.SetForEptViolation:
    ;;
    ;; 处理 guest 写入 IA32_APIC_BASE 寄存器的值:
    ;; 1) 将 IA32_APIC_BASE[N-1:12] 映射到 host 的 IA32_APIC_BASE 值, 但是为 not-
present
    ;; 2) GPA 不进行任何映射
    ;;

    ;;
    ;; 为 GPA 提供处理例程
    ;;
    REX.Wrb
    mov esi, [edx + MSR_VTE.Value]
    REX.Wrb
    and esi, ~0FFFh
    mov edi, EptHandlerForGuestApicPage
    call AppendGpaHte

    call update_guest_rip
    jmp DOWriteMsrForApicBase.Done

DOWriteMsrForApicBase.Error:
    ;;
    ;; 反射 #GP(0) 给 guest 处理
    ;;
    SetVmcsField    VMENTRY_INTERRUPT_INFORMATION, INJECT_EXCEPTION_GP
    SetVmcsField    VMENTRY_EXCEPTION_ERROR_CODE, 0

DOWriteMsrForApicBase.Done:
    pop ecx
    pop edx
    pop ebx
    pop ebp
    ret

```

DoWriteMsrForApicBase 函数实现在 lib\Vm\Vm\Msr.asm 文件里, 它根据 guest 设置 local APIC 的模式 (xAPIC 还是 x2APIC), 以及 secondary processor-based VM-execution control 字段的设置进行虚拟化配置。

- 当 local APIC 为 xAPIC 模式时：
 - 如果“virtualize APIC accesses”为 1，则使用 VMX 提供的 local APIC 虚拟化功能。
 - 如果“virtualize APIC accesses”为 0，则利用 EPT violation 故障来达到 local APIC 虚拟化。
- 当 local APIC 为 x2APIC 模式时：
 - 如果“virtualize x2APIC mode”为 1，则使用 VMX 提供的 x2APIC 模式虚拟化功能。
 - 如果“virtualize x2APIC mode”为 0，则利用 MSR bitmap 来达到 local APIC 虚拟化。

DoWriteMsrForApicBase 例程通过检查 guest 写入 IA32_APIC_BASE 寄存器的值确定 guest 的 local APIC 处于何种工作模式。在检查写入值不合法时将注入 #GP 异常给 guest 处理，例如保留位（bits 63:N、bit 9 以及 bits 7:0）必须为 0。

当利用 EPT violation 来虚拟化 local APIC 时，下面是简单的处理流程。

(1) 读取 EAX 与 EDX 寄存器值。两个寄存器组合的 64 位值是 guest 尝试写入 IA32_APIC_BASE 寄存器的值。

(2) 保存 guest 写入 IA32_APIC_BASE 寄存器的值。这个值在 guest 读取 IA32_APIC_BASE 寄存器时需要反馈给 guest。

(3) 为 guest 的写入值提供一个 EPT violation 处理例程。

3. 保存 guest 的写入值

在前面介绍的 DoWriteMsrForApicBase 函数里，使用 AppendMsrVte 函数将 guest 的写入值保存在了 guest 对应的 MSR VTE (Value Table Entry) 区域里。这个 MSR VTE 区域在 init_vmcs_buffer 函数初始化时从 kernel pool 里分配。

在 inc\vmcs.inc 文件里定义了一个 MSR_VTE 结构，这个结构用来定义 MSR VTE 区域里的每个表项。

```
PMSR_VTE
AppendMsrVte(
    UINT    MsrIndex,
    DWORD   MsrLow,
    DWORD   Msrhigh
);
```

AppendMsrVte 函数的定义看起来像上面的 C 代码，实现在 lib\Vm\VmMsr.asm 文件里。

4. 提供 EPT violation 处理例程

由于 VMM 拦截了 guest 写 IA32_APIC_BASE 寄存器，但并没有为 guest 写入的 guest-physical address 地址值进行 EPT 映射。因此，当 guest 尝试访问 local APIC 寄存器

时，会产生 EPT violation 故障。

那么，在 DoWriteMsrForApicBase 函数里使用 AppendGpaHte 函数提供了由这个 guest-physical address 产生 EPT violation 而导致 VM-exit 的处理例程。

```
PGPA_HTE
AppendGpaHte(
    PVOID GuestPhysicalAddress,
    PVOID Handler
);
```

AppendGpaHte 函数的定义看起来像上面的 C 代码，实现在 lib\Vmx\VmxPage.asm 文件里。在 GPA HTE (Handler Table Entry) 区域建立 GPA 与 EPT violation 处理例程的对应关系。这个 GPA HTE 区域在 init_vmcs_buffer 函数初始化时在 kernel pool 里分配。GPA_HTE 结构定义在 inc\vmcs.inc 文件里。

5. 处理读 IA32_APIC_BASE 产生的 VM-exit

当 guest 尝试读取 IA32_APIC_BASE 寄存器的值时，VMM 拦截了 RDMSR 指令而产生 VM-exit。DoRDMSR 例程检查到 ECX 的值是 IA32_APIC_BASE 寄存器时，调用 DoReadMsrForApicBase 例程来处理。

代码片段 7-20:

```
-----
; DoReadMsrForApicBase()
; input:
;     none
; output:
;     none
; 描述:
;     1) 处理 guest 读 IA32_APIC_BASE 寄存器
;-----
DoReadMsrForApicBase:
    push ebp
    push ebx
    push edx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    REX.Wrb
    mov ebx, [ebp + PCB.CurrentVmbPointer]
    REX.Wrb
    mov ebx, [ebx + VMB.VsbBase]

    mov esi, IA32_APIC_BASE
    call GetMsrVte
    REX.Wrb
    test eax, eax
    jz DoReadMsrForApicBase.Done

    mov edx, [eax + MSR_VTE.Value + 4]
    mov eax, [eax + MSR_VTE.Value]
    mov [ebx + VSB.Rax], eax
```

```

mov [ebx + VSB.Rdx], edx

DEBUG_RECORD    "[DowriteMsrForApicBase]: read from IA32_APIC_BASE"

call update_guest_rip
DoReadMsrForApicBase.Done:
pop edx
pop ebx
pop ebp
ret

```

DoReadMsrForApicBase 函数实现在 lib\Vm\Vm\Msr.asm 文件里，它的主要工作如下：

- (1) 调用 GetMsrVte 函数从 MSR VTE 区域里找到对应的 VTE 表项。
- (2) 从 VTE 表项里读取 guest 尝试写入 IA32_APIC_BASE 寄存器的原值，写入 guest 对应的 VSB 区域里。

GetMsrVte 函数实现在 lib\Vm\Vm\Msr.asm 文件里，它的定义看起来像下面的 C 代码。

```

PMSR_VTE
GetMsrVte(
    UINT    MsrIndex
);

```

当没有 MSR 对应的 MSR VTE 表项时，返回 0 值。从 VTE 表项读取 MSR 值复制到 VSB (VM store block) 区域后，在执行 VMRESUME 指令前，从 VSB 区域里恢复 guest 的 context 信息。

6. 处理 guest 访问 local APIC

guest 尝试读或者写 APIC-page 内的值时，由于 APIC-page 的基址 (IA32_APIC_BASE 提供) 并没有 EPT 映射，从而导致由于 EPT violation 故障而产生 VM-exit。

代码片段 7-21:

```

;;
;; 读取发生 EPT violation 的 guest-physical address 值
;;
REX.Wrb
mov ebx, [ebp + PCB.ExitInfoBuf + EXIT_INFO.GuestPhysicalAddress]

;;
;; 检查 GPA 是否需要额外处理
;;
REX.Wrb
mov esi, ebx
REX.Wrb
and esi, ~0FFFh
call GetGpaHte
REX.Wrb
test eax, eax
jz DoEptViolaton.next
REX.Wrb
mov eax, [eax + GPA_HTE.Handler]
call eax

```



```

;;
;; 处理完后，是否需要修复 EPT violation 故障
;; a) 需要则执行下面的修复工作
;; b) 否则直接返回
;;
cmp eax, EPT_VIOLATION_FIXING
jne DoEptViolation.resume

```

DoEptViolation.next:

上面的代码片段来自 DoEptViolation 例程，它读取引发 EPT violation 故障的 GPA 地址值，然后检查这个 GPA 是否需要额外处理。使用 GetGpaHte 函数来查找 GPA 地址值对应的 HTE (Handler Table Entry) 表项，如果找到则调用 GPA 对应的处理例程来处理。处理完后检查返回值来确定是否需要修复 EPT violation 故障。

代码片段 7-22:

```

;-----
; EptHandlerForGuestApicPage()
; input:
;     none
; output:
;     eax - 处理码
; 描述:
;     1) 处理由于 guest APIC-page 而引起的 EPT violation
;-----
EptHandlerForGuestApicPage:
    push ebp
    push ebx
    push ecx
    push edx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    REX.Wrb
    mov ebx, [ebp + PCB.CurrentVmbPointer]

    ;;
    ;; EPT violation 明细信息
    ;;
    mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.ExitQualification]

    ;;
    ;; guest 访问 APIC-page 的偏移量
    ;;
    REX.Wrb
    mov edx, [ebp + PCB.ExitInfoBuf + EXIT_INFO.GuestPhysicalAddress]
    and edx, 0FFFh

    ;;
    ;; 检查 guest 访问类型
    ;;

```

```

test eax, EPT_READ
jnz EptHandlerForGuestApicPage.Read
test eax, EPT_EXECUTE
jz EptHandlerForGuestApicPage.Write

;;
;; 处理 guest 尝试执行 APIC-page 页面, 注入一个 #PF(0x11) 异常
;;
SetVmcsField    VMENTRY_INTERRUPTION_INFORMATION, INJECT_EXCEPTION_PF
SetVmcsField    VMENTRY_EXCEPTION_ERROR_CODE, 0011h
REX.Wrxb
mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.GuestLinearAddress]
REX.Wrxb
mov cr2, eax
jmp EptHandlerForGuestApicPage.Done

EptHandlerForGuestApicPage.Write:
;;
;; 读取源操作数值
;;
GetVmcsField    GUEST_RIP
REX.Wrxb
mov esi, eax
call get_system_va_of_guest_va
REX.Wrxb
mov esi, eax
mov al, [esi]                ; opcode
cmp al, 89h
je EptHandlerForGuestApicPage.Write.Opcod89
cmp al, 0C7h
je EptHandlerForGuestApicPage.Write.OpcodC7

;;
;; ### 注意, 作为示例, 这里不处理其他指令情况, 包括:
;; 1) 使用其他 opcode 的指令
;; 2) 含有 REX prefix (4xH) 的指令
;;
jmp EptHandlerForGuestApicPage.Done

EptHandlerForGuestApicPage.Write.OpcodC7:
;;
;; 分析 ModRM 字节
;;
mov al, [esi + 1]
mov cl, al
and ecx, 7
cmp cl, 4
sete cl                ; 如果 ModRM.r/m = 4, 则 cl = 1, 否则 cl = 0
shr al, 6
jz EptHandlerForGuestApicPage.Write.@2 ; ModRM.Mod = 0, 则 ecx += 0
cmp al, 1                ; ModRM.Mod = 1, 则 ecx += 2
je EptHandlerForGuestApicPage.Write.OpcodC7.@1
add ecx, 2

EptHandlerForGuestApicPage.Write.OpcodC7.@1:
add ecx, 2                ; ModRM.Mod = 2, 则 ecx += 4
                        ; ModRM.Mod = 3, 属于错误 encode

EptHandlerForGuestApicPage.Write.@2:
;;
;; 读取写入立即数
;;

```

```

    mov eax, [esi + ecx + 2]

    jmp EptHandlerForGuestApicPage.Write.Next

EptHandlerForGuestApicPage.Write.Opcode89:
    ;;
    ;; 读取源操作数
    ;;
    mov esi, [esi + 1]
    shr esi, 3
    and esi, 7
    call get_guest_register_value

EptHandlerForGuestApicPage.Write.Next:
    ;;
    ;; virtual APIC-page 页面
    ;;
    REX.Wrb
    mov esi, [ebx + VMB.VirtualApicAddress]

    ;;
    ;; APIC-page 可写的 offset 值, 写其他区域忽略
    ;; 1) 80h:      TPR
    ;; 2) 80h:      EOI
    ;; 3) D0h:      LDR
    ;; 4) E0h:      DFR
    ;; 5) F0h:      SVR
    ;; 6) 2F0h - 370h: LVT
    ;; 7) 380h:      TIMER-ICR
    ;; 8) 3E0h:      TIMER-DCR
    ;;
    cmp edx, 80h
    jne EptHandlerForGuestApicPage.Write.@1

    DEBUG_RECORD    "[EptHandlerForGuestApicPage]: wirte to APIC-page"

    ;;
    ;; 写入 TPR
    ;;
    mov [esi + 80h], eax
    jmp EptHandlerForGuestApicPage.Done

EptHandlerForGuestApicPage.Write.@1:

    jmp EptHandlerForGuestApicPage.Done

EptHandlerForGuestApicPage.Read:
    ;;
    ;; 分析指令
    ;;
    GetVmcsField    GUEST_RIP
    REX.Wrb
    mov esi, eax
    call get_system_va_of_guest_va
    REX.Wrb
    mov esi, eax
    mov al, [esi]                                ; opcode
    cmp al, 8Bh

```

```

je EptHandlerForGuestApicPage.Read.Opcode8B

;;
;; ### 注意，作为示例，这里不处理其他指令情况，包括：
;; 1) 使用其他 opcode 的指令
;; 2) 含有 REX prefix (4xH) 的指令
;;
jmp EptHandlerForGuestApicPage.Done

EptHandlerForGuestApicPage.Read.Opcode8B:
;;
;; 读取目标操作数 ID
;;
mov esi, [esi + 1]
shr esi, 3
and esi, 7

;;
;; APIC-page 内下面的 offset 为可读区域
;; 1) 20h:      APIC ID
;; 2) 30h:      VER
;; 3) 80h:      TPR
;; 4) 90h:      APR
;; 5) A0h:      PPR
;; 6) B0h:      EOI
;; 7) C0h:      RRD
;; 8) D0h:      LDR
;; 9) E0h:      DFR
;; 10) F0h:      SVR
;; 11) 100h - 170h:  ISR
;; 12) 180h - 1F0h:  TMR
;; 13) 200h - 270h:  IRR
;; 14) 280h:      ESR
;; 15) 2F0h - 370h:  LVT
;; 16) 380h:      TIMER-ICR
;; 17) 3E0h:      TIMER-DCR
;;

cmp edx, 80h
jne EptHandlerForGuestApicPage.Read.@1

DEBUG_RECORD    "[EptHandlerForGuestApicPage]: read from APIC-page"

;;
;; 写入目标寄存器
;;

REX.Wrxb
mov eax, [ebx + VMB.VirtualApicAddress]
mov edi, [eax + 80h]
call set_guest_register_value
jmp EptHandlerForGuestApicPage.Done

EptHandlerForGuestApicPage.Read.@1:

EptHandlerForGuestApicPage.Done:
call update_guest_rip
mov eax, EPT_VIOLATION_NO_FIXING
pop edx
pop ecx

```

```
pop ebx
pop ebp
ret
```

EptHandlerForGuestApicPage 例程用来模拟 guest 访问 local APIC，这是一个相当烦琐的操作。这个函数实现在 lib\Vm\VmxApic.asm 文件里。作为示例，这个函数只实现了基本的框架功能，**并没有处理所有可能发生的情况。**

这个例程需要处理 guest 的两大类操作：读和写 APIC-page 区域。VMCS 区域的 Exit Qualification 字段记录 EPT violation 的明细信息，其中 bits 2:0 指示访问的类型（参见 3.10.1.14 节）。

- bit 0 为 1 时，表示进行了读访问。
- bit 1 为 1 时，表示进行了写访问。
- bit 2 为 1 时，表示进行了执行访问。

EptHandlerForGuestApicPage 例程可以根据 Exit Qualification 字段的 bits 2:0 来判断 guest 当前对 APIC-page 进行了什么访问，从而做出相应的处理。

当属于读 APIC-page 页面时，这个例程进行下面的处理：

(1) 分析 guest 访问指令。典型地，一个读访问使用 MOV 指令来完成，这个例程只处理了 MOV 指令（opcode 码为 8BH）。另外，local APIC 不能支持超过 32 位的访问宽度，并且某些处理器也不支持低于 32 位的访问宽度（譬如 16 位）。这里也没有处理 guest 使用 64 位的访问宽度的情况（即指令存在 REX prefix）。

(2) 读取目标操作数 ID 值。使用 MOV 指令读 APIC-page 时，目标操作数必定是寄存器。通过分析指令编码的 ModRM 字段来获取目标操作数的 ID 值。

(3) 分析读 APIC-page 页面的 offset 值是否合法。APIC-page 的某些区域是保留的，访问这些保留区域是未定义的。因此，EptHandlerForGuestApicPage 例程需要检查访问的 offset 是否合法。作为示例，这里只处理了访问 TPR 寄存器的情况（offset 为 80H）。

(4) 从 virtual-APIC page 页面的 80H（TPR）读取相应的值写入目标寄存器里。

这个 virtual-APIC page 是 APIC-page 的虚拟页面，每个 guest 都有自己独立的虚拟的 APIC-page 页面，用来存放 local APIC 的虚拟数据。当写 local APIC 时往这个虚拟的页面写入数据，当读 local APIC 时从这个虚拟的页面读取数据。

当属于写 APIC-page 页面时，这个例程进行下面的处理：

(1) 分析 guest 指令。写 local APIC 操作的指令相对更丰富些。典型地，可以有下面的指令形式：

```
mov [ebx + TPR], 50h
mov eax, 50h
mov [ebx + TPR], eax
```

这两条写 local APIC 的 MOV 指令，它们的 opcode 码是不同的。作为示例，在这个例程只处理了 opcode 码为 89H 以及 C7H 时的指令情况。

- 指令 opcode 码为 89H 时，源操作数是寄存器。从 ModRM 字节的 bits 5:3 读取寄存器 ID 值，然后使用 `get_guest_register_value` 函数读取寄存器值。
- 指令 opcode 码为 C7H 时，源操作数是 32 位立即数，则分析 ModRM 字节是否存在 SIB 以及 displacement 值，然后从指令 encode 中读取 32 位的立即数值。

(2) 分析写入 APIC-page 的 offset 值，检查是否属于 local APIC 可写的寄存器（只读寄存器以及保留区域都不可写）。作为示例，这里只处理了 TPR 寄存器（80H）。

当 guest 尝试执行 APIC-page 页面时，这个例程反射一个 #PF 异常给 guest 处理，错误码为 0011H，指示页面不可执行。将 CR2 寄存器设置为 guest-linear address 值。注意：这个处理实际上与 guest 相关，要取决于 guest 如何映射 APIC-page 区域。

一个实用的 `EptHandlerForGuestApicPage` 例程需要完善很多细节，各种可能遇到的情况都要考虑进去，需要进行烦琐的处理工作才能完全模拟 guest 访问 local APIC。

7. guest 端代码

在 `chap07\ex7-3` 目录下的 `guest_ex.asm` 文件是这个例子的 guest 主体代码，如代码片段 7-23 所示。

代码片段 7-23:

```
;;
;; 设置 local APIC base 值为 01000000h
;;
mov ecx, IA32_APIC_BASE
mov eax, 01000000h | APIC_BASE_BSP | APIC_BASE_ENABLE
xor edx, edx
wrmsr

mov esi, GuestEx.Msg0
call PutStr
mov ecx, IA32_APIC_BASE
rdmsr
mov esi, eax
and esi, ~0FFFh
mov edi, edx
call PrintQword
call PrintLn

mov R3, GUEST_APIC_BASE

;;
;; TPR = 50h
;;
mov eax, 50h
mov [R3 + LAPIC_TPR], eax
mov esi, GuestEx.Msg1
call PutStr
mov esi, [R3 + LAPIC_TPR]
call PrintValue
call PrintLn

;;
;; TPR = 60h
```

```

;;
mov DWORD [R3 + LAPIC_TPR], 60h
mov esi, GuestEx.Msg1
call PutStr
mov esi, [R3 + LAPIC_TPR]
call PrintValue

jmp $

GuestEx.Msg0 db 'APIC base: ', 0
GuestEx.Msg1 db 'TPR: ', 0

```

这个 guest 代码的工作是：

- (1) 尝试将 IA32_APIC_BASE 寄存器的 bits $N-1:12$ 设置为 01000000H（即新的 APIC-page 基址）。
- (2) 从 IA32_APIC_BASE 寄存器读取 APIC-page 基址，并打印出来。
- (3) 尝试向 local APIC 的 TPR 寄存器（80H）写入值 50H。注意，这里 MOV 指令使用寄存器作为源操作数（即 opcode 码为 89H）。
- (4) 读取 TPR 寄存器（80H），并打印出来。
- (5) 尝试再次向 TPR 寄存器（80H）写入值 60H。注意，这里 MOV 指令使用立即数作为源操作数（即 opcode 码为 C7H）。
- (6) 再次读取 TPR 寄存器（80H），并打印出来。

8. 编译及运行

这个例子，我们可以使用下面的命令进行编译：

- (1) build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64 -DGUEST_X64
- (2) build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64
- (3) build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE

第 1 个命令将 host 与 guest 都编译为 64 位。第 2 个命令将 host 编译为 64 位，guest 编译为 32 位。第 3 个命令将 host 与 guest 都编译为 32 位。

下面以 host 与 guest 都是 64 位环境为例，观察例子的运行结果。

在 VMware 里加载 demo.img 映像文件运行，在 CPU0 的命令选择器上按数字<1>键，让 CPU1 进入 guest 运行。按下两次<F2>键切换到 CPU1 的 guest 屏幕，如图 7-5 所示。



图 7-5

guest 打印出来的 local APIC 基址为 01000000H，这是 guest 尝试向 IA32_APIC_BASE 寄存器写入的 local APIC 基址值。它由前面介绍的 DoReadMsrForApicBase 例程进行处理。接下来分别打印 guest 两次尝试写入的 TPR 寄存器值 50H 与 60H。

我们来看看记录的调试信息，按下<F4>键切换到 CPU3 屏幕，按下<enter>键切换到 msg-list 信息窗口，如图 7-6 所示。

```

===== msg list record =====
<#52> [CPU1] [VMM]: resume to guest !
<#53> [CPU1] [Exit Reason]: VM-exit due to WRMSR
<#54> [CPU1] [DoWriteMsrForApicBase]: write to IA32_APIC_BASE
<#55> [CPU1] [VMM]: resume to guest !
<#56> [CPU1] [Exit Reason]: VM-exit due to RDMSR
<#57> [CPU1] [DoWriteMsrForApicBase]: read from IA32_APIC_BASE
<#58> [CPU1] [VMM]: resume to guest !
<#59> [CPU1] [Exit Reason]: EPT violation
<#60> [CPU1] [EptHandlerForGuestApicPage]: write to APIC-page
<#61> [CPU1] [VMM]: resume to guest !
<#62> [CPU1] [Exit Reason]: EPT violation
<#63> [CPU1] [EptHandlerForGuestApicPage]: read from APIC-page
<#64> [CPU1] [VMM]: resume to guest !
<#65> [CPU1] [Exit Reason]: EPT violation
<#66> [CPU1] [EptHandlerForGuestApicPage]: write to APIC-page
<#67> [CPU1] [VMM]: resume to guest !
<#68> [CPU1] [Exit Reason]: EPT violation
<#69> [CPU1] [EptHandlerForGuestApicPage]: read from APIC-page
<#70> [CPU1] [VMM]: resume to guest !
<#71> [CPU1] [Exit Reason]: exception or NMI
  
```

图 7-6

第 53 条到第 57 条记录显示 VMM 处理了 guest 写和读 IA32_APIC_BASE 寄存器。第 59 条到第 69 条记录显示发生 EPT violation 而导致 VM-exit，VMM 调用了 EptHandlerForGuestApicPage 来处理 EPT violation 故障，用来模拟 guest 写入及读取 local APIC 寄存器。

当 local APIC 使用 x2APIC 模式时，VMM 监控 guest 访问 local APIC 寄存器变得非常简单可靠。VMM 只需要在 MSR bitmap 里限制 guest 访问 local APIC 寄存器对应的 MSR 即可。

7.2.2 local APIC 虚拟化机制

前面的 7.2.1 节所介绍的 local APIC 虚拟化原理是使用 EPT 映射机制或者 MSR-bitmap 机制来监控 guest 访问 local APIC 页面，并模拟写入和读取 local APIC 寄存器。

VMX 引进了原生的 local APIC 虚拟化机制，由 VMCS 的 secondary processor-based VM-execution control 字段提供下面的功能。

- **Virtualize APIC accesses**（虚拟化 APIC 访问）。当“virtualize APIC accesses”位（bit 0）为 1 时，将启用 APIC-access page 页面。处理器虚拟化 guest 线性访问 APIC-access page 页面的行为，取决于“APIC-register virtualization”位的设置，产生两种结果：

- 产生 APIC access VM-exit, 或者 APIC write VM-exit (参见 7.2.8 节)。
- 成功访问 virtual-APIC page 页面的数据, 或者虚拟化某项操作 (譬如 EOI 虚拟化)。
- **Virtualize x2APIC mode** (虚拟化 x2APIC 模式)。当 “virtualize x2APIC mode” 位 (bit 4) 为 1 时, 处理器虚拟化通过 MSR 来访问 800H~8FFH local APIC 寄存器的行为, 取决于 “APIC-register virtualization” 位的设置, 产生两种结果:
 - 成功访问 virtual-APIC page 页面的数据, 或者虚拟化某项操作 (譬如 EOI 虚拟化)。
 - 取决于 local APIC 是否为 x2APIC 模式, 产生 #GP 异常或者访问到 MSR 数据。
- **Use TPR shadow** (启用 TPR 影子)。当 primary processor-based VM-execution control 字段的 “Use TPR shadow” 位为 1 时, 启用 virtual-APIC page 页面。在 **virtual-APIC page** 页面的 **80H** 位置上存在一份 TPR shadow 数据, 即 **VTPR** (virtual TPR, 虚拟 TPR 寄存器)。当 guest 访问 APIC-access page 页面内 80H 位置时, 将访问到这份 VPTR 数据。
- **APIC-register virtualization** (APIC 寄存器虚拟化)。当 “APIC-register virtualization” 位 (bit 8) 为 1 时, 将启用 **virtual-APIC page** 页面内可访问的 local APIC 虚拟寄存器 (譬如 VISR、VEOI 等, 参见 7.2.5 节表 7-2)。
- **Virtual-interrupt delivery** (虚拟中断的提交)。当 “virtual-interrupt delivery” 位 (bit 9) 为 1 时, 修改 virtual local APIC 的某些状态将引起 virtual-interrupt 的悬挂时, 对 pending virtual-interrupt 进行评估, 通过仲裁后允许 virtual-interrupt 通过 guest-IDT 提交处理。
- **Posted-interrupt processing** (通告的中断处理)。当 pin-based VM-execution control 字段的 “process posted interrupts” 位为 1 时, 处理器接收到 “通知” 的外部中断时不会产生 VM-exit, 并且将 posted-interrupt descriptor (通告中断描述符) 内的 posted-interrupt request (通告的中断请求) 复制到 virtual-APIC page 内 VIRR 寄存器的虚拟中断请求列表里, 形成虚拟中断的请求。处理器进行虚拟中断的评估和 delivery。

local APIC 存在两种访问模式: **xAPIC** 和 **x2APIC** 模式。**xAPIC** 模式基于 memory-mapped (内存映射) 方式访问 local APIC 寄存器 (使用 MOV 指令), 而 **x2APIC** 基于 MSR 访问 local APIC 寄存器 (使用 RDMSR 与 WRMSR 指令)。

VMX 提供了这两种途径访问的虚拟化功能, 分别在 “virtualize APIC accesses” 位与 “virtualize x2APIC mode” 位里设置。但是, 不能同时使用这两种途径的虚拟化 (参见 4.4.1.3 节)。

Local APIC 虚拟化设置主要基于 secondary processor-based VM-execution control 字段, 而 primary processor-based VM-execution control 字段则提供了对 “use TPR shadow” 位的设置。

7.2.3 APIC-access page

当“virtualize APIC accesses”位为 1 时，虚拟化基于**内存映射**（memory mapped）的方式访问 local APIC 寄存器。引入了一个被称为“**APIC-access page**”（APIC 访问页面）的 4K 页面来对应**物理的 local APIC 页面**。APIC-access page 的 64 位**物理地址**由 VMM 提供在 APIC-access address 字段里（参见 3.5.9 节）。

由于 EPT 映射机制的引入，guest 访问 APIC-access page 页面可能出现三种途径：**linear access**（线性访问）、**guest-physical access**（客户端物理访问）以及 **physical access**（物理访问）。

1. linear access（线性访问）

线性访问是指下面两种途径：

(1) 当“enable EPT”为 0 时，guest 尝试使用**线性地址**访问，这个线性地址经过 paging-structure 转换后的**物理地址**落在 APIC-access page 页面内。

(2) 当“enable EPT”为 1 时，guest-linear address 经过 guest paging structure 以及 EPT paging structure 转换的最终 host-physical address 落在 APIC-access page 页面内。

这两种情况下，对 APIC-access page 页面的访问发起源都是线性地址。也就是：**访问的最终物理地址是由线性地址转换而来**。

2. guest-physical access（客户端物理访问）

guest-physical 访问是在启用 EPT 机制的情况下，guest 发起访问 APIC-access page 页面的地址是 guest-physical address。包括下面的几种情况。

(1) 处理器在转换 guest-linear address 时（参见 6.1.8.3 节及图 6-11），由于 guest paging structure 表项里的 **GPA 转换后的 HPA** 地址值落在了 APIC-access page 页面内，也就是处理器在各级 guest paging structure 进行 **walk 流程**时遇到 GPA 访问了 APIC-access page 页面。

(2) 在 guest 使用 PAE 分页模式的情况下，由于执行了 MOV-to-CR3 指令而引起了加载 PDPTEs 表项，CR3 寄存器内的 GPA 转换后的 HPA 地址值落在了 APIC-access page 页面内。

(3) 当更新 guest paging structure 表项内的 dirty 或者 accessed 标志位时（这时使用 GPA 地址访问），表项地址转换后的 HPA 落在了 APIC-access page 页面内。

在这种情况下，发起 guest-physical 访问 APIC-access page 页面不需要经过 guest-linear address。也就是：**guest-physical address 并不是由 guest-linear address 转换而来**。

3. Physical access（物理访问）

物理访问是以物理地址作为发起源来访问 APIC-access page 页面，包括 VMCS 区域内引用的物理区域（譬如 I/O bitmap，MSR bitmap 等）。

取决于 secondary processor-based VM-execution control 字段“enable EPT”位的设置，下面的情况属于物理访问。

- 当“enable EPT”位为 0 时（关闭 EPT 机制）：
 - 在处理器转换线性地址时（进行 walk 流程），由于 paging structure 表项里的物理地址落在了 APIC-access page 页面内。
 - 在 guest 使用 PAE 分页模式的情况下，由于执行了 MOV-to-CR3 指令而引起了加载 PDPTEs 表项，CR3 寄存器内的物理地址落在了 APIC-access page 页面内。
 - 在更新 paging structure 表项的 dirty 或者 accessed 标志位时，表项地址落在了 APIC-access page 页面内。
- 当“enable EPT”位为 1 时（启用 EPT 机制）：
 - 在处理器转换 guest-physical address 时，由于 EPT paging structure 表项里的物理地址落在了 APIC-access page 页面内。
 - 在更新 EPT paging structure 表项的 dirty 或者 accessed 标志位时（启用了 dirty 标志位），表项的物理地址落在了 APIC-access page 页面内。
- 访问的 VMCS 区域落在了 APIC-access page 页面内。从另一个角度看，也就是 VMCS 区域的物理地址为 APIC-access page 地址。
- 访问 VMCS 区域内所引用的数据区域的物理地址落在了 APIC-access page 页面内。包括 I/O bitmap、MSR bitmap 以及 virtual-APIC page 区域。
- 在发生 SMM 模式的切换时（进入或者退出 SMM）访问了 APIC-access page 页面发生。包括下面的情况（注意：实际上这不可能发生在 **VMX non-root operation 模式内**。）：
 - 在默认 SMM 处理下，从 SMI 的 delivery 到执行 RSM 指令期间访问的 SMRAM 落在 APIC-access page 页面内。
 - 在 SMM 双重监控处理下，从产生 SMM VM-exit 到 VM-entry that return from SMM 期间访问了 APIC-access page 页面（譬如 MSEG 区域）。

在物理访问里，发起访问 APIC-access page 页面的物理地址，**不是由线性地址或者 guest-physical address 转换而来**。取决于“use TPR shadow”、“APIC-register virtualization”以及“virtual-interrupt delivery”位的设置，guest 访问 APIC-access page 页面可以产生 VM-exit，成功访问 virtual-APIC page 内的虚拟寄存器或者执行某些虚拟化操作。

7.2.3.1 APIC-access page 的设置

引入 APIC-access page，目的是监控 guest 访问 local APIC（参见 7.2.1 节描述），但这个监控由处理器自动实施（并非通过 EPT violation 或者 MSR bitmap 实现）。

显然，我们需要将 APIC-access page 页面的物理地址设置为 APIC-page 的物理地址。当 guest 尝试线性访问 local APIC 时，会落入 APIC-access page 页面内。

- 当“enable EPT”为 0 时（关闭 EPT 机制），VMM 将 guest 写入 IA32_APIC_BASE

寄存器的 local APIC 基址写到 APIC-access address 字段里。

- 当“enable EPT”为 1 时（启用 EPT 机制），VMM 需要将 guest 写入 IA32_APIC_BASE 寄存器的 local APIC 基址（GPA）转换后的 host-physical address 写到 APIC-access address 字段里。

因此，取决于 secondary processor-based VM-execution control 字段“virtualize APIC accesses”位的设置，对监控 guest 尝试使用 WRMSR 指令写 IA32_APIC_BASE 寄存器有不同的处理。如下面 C 伪码所示。

```
void DoWriteMsrForApicBase()
{
    ...

    if (ProcessorBasedExecutionControl2 & VIRTUALIZE_APIC_ACCESSSES)
    {
        //
        // 如果启用了“virtualize APIC accesses”功能
        // 则设置 APIC access address 字段
        //
        ... ..
    }
    else
    {
        //
        // 否则，利用 EPT violation 来完成虚拟化 local APIC
        //
        ... ..
    }

    ... ..
}
```

当 VM 启用了“virtualize APIC accesses”功能时，需要设置相应的 APIC access address 字段值。否则，利用 EPT violation 来达到虚拟化 local APIC 的目的（参见 7.2.1.1 节的例子 7-3）。

代码片段 7-24:

```
GetVmcsField    CONTROL_PROCBASED_SECONDARY

test eax, VIRTUALIZE_APIC_ACCESS
jz DoWriteMsrForApicBase.SetForEptViolation
test eax, ENABLE_EPT
jz DoWriteMsrForApicBase.EptDisable

;;
;; 执行 EPT 映射到 0FEE00000h
;;
#ifdef __x64
    REX.Wrb
    mov esi, [edx + MSR_VTE.Value]
    mov edi, 0FEE00000h
    mov eax, EPT_READ | EPT_WRITE
    call do_guest_physical_address_mapping
#else
    mov esi, [edx + MSR_VTE.Value]
```

```

        mov edi, [edx + MSR_VTE.Value + 4]
        mov eax, 0FEE00000H
        mov edx, 0
        mov ecx, EPT_READ | EPT_WRITE
        call do_guest_physical_address_mapping
%endif

        mov eax, 0FEE00000h
        mov edx, 0
        jmp DOWriteMsrForApicBase.SetApicAccessPage

DOWriteMsrForApicBase.EptDisable:
        REX.Wrxb
        mov eax, [edx + MSR_VTE.Value]
        mov edx, [edx + MSR_VTE.Value + 4]
        REX.Wrxb
        and eax, ~0FFFh

DOWriteMsrForApicBase.SetApicAccessPage:
        SetVmcsField    CONTROL_APIC_ACCESS_ADDRESS_FULL, eax
%ifndef __X64
        SetVmcsField    CONTROL_APIC_ACCESS_ADDRESS_HIGH, edx
%endif

        call update_guest_rip
        jmp DOWriteMsrForApicBase.Done

```

在上面的 DoWriteMsrForApicBase 例程片段里，当检查到 VMCS 里设置了“virtualize APIC accesses”位之后，则对 APIC-access page 页面进行设置。

在 lib\Vm\VmInit.asm 文件的 int_vm_execution_control_fields 函数初始化期间，默认的 APIC-access page 页面物理地址设置为 0FEE00000H。这里保持了一致，将 guest 尝试写入 IA32_APIC_BASE 寄存器的基址映射到 0FEE00000H 地址上。

1. 设置 guest 默认 IA32_APIC_BASE

由于 host 可能会更改 IA32_APIC_BASE 寄存器的值，为了保持 guest 环境在#REST 后的状态，需要在 init_guest_a 和 init_guest_b 函数里，将 guest 默认的 IA32_APIC_BASE 值设置为 0FEE00900H。

代码片段 7-25:

```

;;
;; 配置 guest 的初始 IA32_APIC_BASE 寄存器值
;;
mov esi, IA32_APIC_BASE
mov eax, 0FEE00000h | APIC_BASE_BSP | APIC_BASE_ENABLE
xor edx, edx
call append_vmentry_msr_load_entry

```

通过在 VM-entry MSR-load 列表里增加 IA32_APIC_BASE 寄存器数据，来设置 guest 的 MSR 初始化值。

2. 恢复 host 的 IA32_APIC_BASE

当 guest 修改 IA32_APIC_BASE 寄存器值时，为了保持 host 环境正确，需要在 VM-

exit 时恢复 host 原来的 IA32_APIC_BASE 寄存器值。

代码片段 7-26:

```
;;
;; 保存当前的 IA32_APIC_BASE 值供 VM-exit 时加载
;;
mov ecx, IA32_APIC_BASE
rdmsr
mov esi, IA32_APIC_BASE
call append_vmexit_msr_load_entry
```

通过在 VM-exit MSR-load 列表里增加 IA32_APIC_BASE 寄存器数据来恢复 host 原值。然而，由于 VMM 监控了 guest 尝试更改 IA32_APIC_BASE 寄存器的操作。实际上，IA32_APIC_BASE 寄存器并没有被修改过。在这种情况下，恢复 host 的 IA32_APIC_BASE 寄存器操作不是必需的！

7.2.4 虚拟化 x2APIC MSR 组

在 x2APIC 模式里，local APIC 寄存器被映射到 MSR 地址空间从 800H 到 8FFH 的区域。软件通过 RDMSR 与 WRMSR 指令读/写 MSR 寄存器来访问 local APIC 寄存器。

VMM 可以通过下面两种方式来虚拟化 x2APIC 模式的 local APIC：

(1) 当“virtualize x2APIC mode”位为 0 时，通过设置 **MSR bitmap** 来监控 guest 访问 local APIC，从而达到虚拟化 local APIC。

(2) 当“virtualize x2APIC mode”位为 1 时，使用 VMX 提供的原生 (native) 虚拟化 x2APIC 模式 local APIC。处理器将虚拟化 guest 基于 **MSR** 访问 local APIC 寄存器。

当虚拟化 x2APIC 模式 local APIC 时，**不存在 APIC-access page 页面！** APIC-access page 页仅存在于使用 VMX 原生的 (native) 基于**内存映射**访问 local APIC 的虚拟化中。

注意：“virtualize APIC-accesses”与“virtualize x2APIC mode”位不能同时设为 1 值。也就是 local APIC 不能同时使用 xAPIC 与 x2APIC 两种模式。

1. 监控 guest 访问 local APIC MSR

在前面的例子 7-3 里，我们学习了如何利用 EPT violation 故障来监控 guest 访问 local APIC 页面。当“virtualize x2APIC mode”位为 0 时，我们也利用 MSR bitmap 来监控 guest 访问 local APIC MSR。

代码片段 7-27:

```
;;
;; 检查 secondary processor-based VM-execution control 字段“virtualize x2APIC
mode”位
;; 1) 为 1 时，使用 VMX 原生的 x2APIC 虚拟化，直接返回
;; 2) 为 0 时，监控 800H - 8FFH MSR 的读/写
;;
GetVmcsField CONTROL_PROCBASED_SECONDARY
test eax, VIRTUALIZE_X2APIC_MODE
```

```

jnz DoWriteMsrForApicBase.Done

;;
;; 现在监控 x2APIC MSR 的读/写, 范围从 800H 到 8FFH
;;
call set_msr_read_bitmap_for_x2apic
call set_msr_write_bitmap_for_x2apic
jmp DoWriteMsrForApicBase.Done

```

上面的代码片段出现在前面介绍过的 DoWriteMsrForApicBase 例程里。它检查“virtualize x2APIC mode”位是否为 1，为 0 时调用 set_msr_read_bitmap_for_x2apic 与 set_msr_write_bitmap_for_x2apic 函数设置 local APIC MSR 对应的读/写 MSR bitmap（参见 3.5.15 节）。

2. 处理访问 local APIC MSR

当 guest 使用 RDMSR 与 WRMSR 指令访问 local APIC MSR 时，由于我们已经在 MSR bitmap 里设置了拦截功能而产生 VM-exit。VMM 应该根据 local APIC 的虚拟化操作流程来处理这个 VM-exit（参见下面的 7.2.7 与 7.2.8 节）。

这里没有实现例子完成处理访问 local APIC MSR，这个工作留待读者自行完成。

7.2.5 virtual-APIC page

当“use TPR shadow”为 1 时，引入了一个被称为“**virtual-APIC page**”的影子页面（shadow page）。这个影子页面与物理 APIC-page 一样是 4K 大小，是 local APIC 的虚拟页面。

- 当“APIC-register virtualization”为 1 时，virtual-APIC page 页面内存在每个 local APIC 寄存器对应的一份**影子数据**（即虚拟 local APIC 寄存器）。
- 当“APIC-register virtualization”为 0 时，还细分有下面的情况：
 - “virtual-interrupt delivery”为 0 时，只有偏移量为 **80H** 的 **VTPR** 寄存器（virtual-TPR）是有效的。其他位置的数据属于无效（不可访问）。
 - “virtual-interrupt delivery”为 1 时，除了偏移量为 **80H** 的 **VTPR** 寄存器外，还有偏移量为 **B0H** 的 **VEOI** 寄存器（virtual-EOI），以及偏移量为 **300H** 的 **VICR** 寄存器（virtual-ICR）的低 32 位可以进行写访问，但不能进行读访问！其他位置的数据属于无效（不可访问）。

virtual-APIC page 页面的 64 位物理地址需要在 virtual-APIC address 字段里提供。这个 4K 的页面属于 VMCS 区域所引用的数据结构区域（例如 I/O bitmap 与 MSR bitmap），VMM 在初始化 VMCS 区域时需要为 VM 分配这样的一个 4K 页面。

注意：无论是虚拟化 xAPIC 模式还是 x2APIC 模式，都会存在 virtual-APIC page 页面。

1. 虚拟 local APIC 寄存器

virtual-APIC page 页面内存在物理 local APIC 对应的一份虚拟寄存器。表 7-2 列出了

可访问的虚拟 local APIC 寄存器。

表 7-2

offset	虚拟寄存器	读写属性
020H - 023H	local APIC ID	读/写
030H - 033H	local APIC version	只读
080H - 083H	VTPR	读/写
0B0H - 0B3H	VEOI	
0D0H - 0D3H	VLDR	
0E0H - 0E3H	VDFR	
0F0H - 0F3H	VSVR	
100H - 103H	VISR	
110H - 113H		
120H - 123H		
130H - 133H		
140H - 143H		
150H - 153H		
160H - 163H		
170H - 173H		
180H - 183H	VTMR	只读
190H - 193H		
1A0H - 1A3H		
1B0H - 1B3H		
1C0H - 1C3H		
1D0H - 1D3H		
1E0H - 1E3H		
1F0H - 1F3H		
200H - 203H	VIRR	
210H - 213H		
220H - 223H		
230H - 233H		
240H - 243H		
250H - 253H		
260H - 263H		
270H - 273H		
280H - 283H	VESR	读/写
300H - 303H	VICR	

续表

offset	虚拟寄存器	读写属性
310H - 313H	VICR	读/写
320H - 323H	LVT	
330H - 333H		
340H - 343H		
350H - 353H		
360H - 363H		
370H - 373H		
380H - 383H	APIC timer initial count	
3E0H - 3E3H	APIC timer divide configuration	

如前面所述，只有在“APIC-register virtualization”为 1 时，表中所有的虚拟寄存器是可访问的。“APIC-register virtualization”与“virtual-interrupt delivery”都为 0 时，只有 VTPR 是可访问的。“APIC-register virtualization”为 0 且“virtual-interrupt delivery”为 1 时，只有 VTPR（读/写）、VEOI（只写）以及 VICR（偏移量 300H，只写）是可访问的。

部分 local APIC 对应的虚拟寄存器是不可访问的，譬如偏移量为 A0H 的 VPPR（virtual processor-priority register，虚拟处理器优先级别寄存器）。尝试访问则产生 VM-exit。

2. APIC-access page 与 virtual-APIC page 的关系

APIC-access page 与 virtual-APIC page 之间的关系类似于页映射中的虚拟地址与物理地址。当 guest 尝试线性访问 APIC-access page 页面时（不包括 guest-physical 访问与物理访问），实际上访问了 virtual-APIC page 页面的数据。如图 7-7 所示。

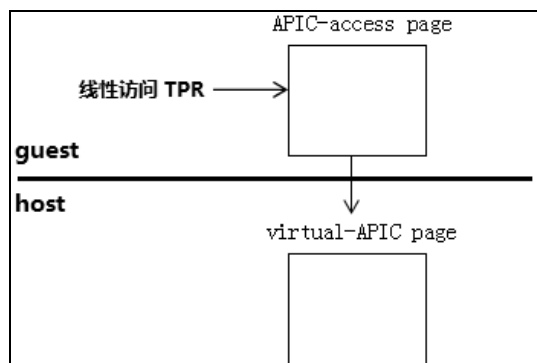


图 7-7

当启用了“virtualize APIC-accesses”和“use TPR shadow”后，如图 7-7 所示 guest 尝试线性访问 TPR 寄存器，处理器返回的是 virtual-APIC page 页面内的 VTPR 寄存

器值。

当 guest 尝试以 **guest-physical** 方式访问 APIC-access page 页面时，不能得到 virtual-APIC page 页面内的虚拟寄存器数据，而是产生 VM-exit。以物理方式访问时可能会访问到，也可能产生 VM-exit。

3. local APIC MSR 与 virtual-APIC page 的关系

当使用 x2APIC 模式虚拟化机制时，guest 访问 local APIC MSR 同样访问到 virtual-APIC page 页面内的虚拟寄存器。如同图 7-7 所示，所不同的是通过 MSR 来访问 local APIC 物理页面。

7.2.6 APIC-access VM-exit

当“virtualize APIC accesses”为 1 时，guest 尝试进行下面的访问将产生 VM-exit，这个 VM-exit 可称为“**APIC-access VM-exit**”。

(1) 尝试线性访问（包括读和写）APIC-access page 页面内**不支持的 offset 位置**将产生 APIC-access VM-exit。

- 当“use TPR shadow”为 0 时，所有的 offset 位置都不支持。
- 当“use TPR shadow”为 1，且“APIC-register virtualization”与“virtual-interrupt delivery”都为 0 时，仅有 offset 为 **80H** 位置（TPR）被支持。
- 当“use TPR shadow”为 1，且“APIC-register virtualization”为 0，“virtual-interrupt delivery”为 1 时，支持下面的访问：
 - offset 为 **80H** 位置（TPR）支持读/写。
 - offset 为 **B0H** 位置（EOI）与 **300H** 位置（ICR 低 32 位）支持写访问。
- 当“use TPR shadow”为 1，且“APIC-register virtualization”为 1 时，表 7-2 中所列的所有 offset 位置可访问。

(2) 尝试 guest-physical 访问（包括读和写）APIC-access page 页面内**任何的 offset 位置**将产生 APIC-access VM-exit。

《Intel 手册》明确表明：guest 尝试 physical access（物理访问）APIC-access page 页面**可能会也可能不会产生** VM-exit。当不产生 APIC-access VM-exit 时，可能会访问到 virtual-APIC page 页面内的数据。

7.2.6.1 APIC-access VM-exit 优先级别

当 guest 尝试访问 APIC-access page 页面时，可能会存在多个引发 VM-exit 或其他事件的条件。例如下面的代码。

```
mov eax, [ebx + LAPIC_ID] ; 读取 APICID 值
```

在上面的代码里，guest 尝试通过线性地址读取 local APIC 的 ID 寄存器值。这条指令可能会产生多个异常。例如：它可能由于地址的 offset 值超过了 DS 的 limit 而产生 #GP

异常，也可能由于线性地址而产生#PF 异常，也可能由于 guest-physical address 而产生 EPT violation 或者 EPT misconfiguration 导致 VM-exit，也可能由于访问 APIC-access page 而产生 APIC-access VM-exit。

APIC-access VM-exit 与其他 VM-exit 及事件之间的优先级别如下（关于优先级别的描述另参见第 5.6 节、4.15 节以及 6.1.8.3 节）。

- APIC-access VM-exit 的优先级别**低于**#PF 异常、EPT violation 或者 EPT misconfiguration 故障。也就是说，当发生#PF 异常、EPT violation 或者 EPT misconfiguration 时不会产生 APIC-access VM-exit。
- 假如伴随着其他异常或事件（例如#GP，#AC 或者#NP 异常），APIC-access VM-exit 的优先级别等同于#PF 异常、EPT violation 或者 EPT misconfiguration 故障。
- 假如产生 APIC-access VM-exit，则**直到处理器完成更新** paging structure（包括 guest paging structure 与 EPT paging structure）表项内的 dirty 或者 accessed 标志位后才响应 APIC-access VM-exit。

假如，在一条含有 REP 前缀的循环串指令里（譬如 MOVS、INS/OUTS 等），在前面的 N 次循环里正常执行并没有产生 APIC-access VM-exit 或者异常，但第 $N+1$ 次可能会由于 ESI 或 EDI 指针的递增而产生 APIC-access VM-exit，则处理器记录当前的 ECX、ESI 或 EDI 寄存器值。

7.2.7 虚拟化读取 APIC-access page

当“virtualize APIC accesses”为 1 时，启用 APIC-access page 页面。APIC-access page 页面物理地址提供在 APIC-access address 字段里。

guest 尝试线性读取 APIC-access page 页面内的值（参见 7.2.3 节）可能会产生下面的两种结果之一：

(1) 产生 APIC-access VM-exit。

(2) 返回 virtual-APIC page 页面（参见 7.2.5 节）内相应偏移量的虚拟 local APIC 寄存器值。

1. 产生 APIC-access VM-exit

属于下面情况之一时，访问 APIC-access page 页面内任何 offset 值都将产生 APIC-access VM-exit：

- 当“use TPR shadow”位为 0 时（表示不存在 virtual-APIC page 页面）。
- 尝试执行 APIC-access page 页面（即尝试执行访问）。
- 访问的数据 size 超过 32 位。例如，尝试读取 64 位数据。
- 访问 APIC-access page 页面的动作发生在虚拟化写 APIC-access page 页面操作的其中一个流程。例如，在虚拟化 TPR 操作时（即写 APIC-access page 页面 80H 位置），最后的步骤是进行 pending virtual-interrupt 的 delivery 评估。当评估通过后

虚拟中断将被 deliver 执行，而在 delivery 期间访问了 APIC-access page 页面。

- 尝试对 APIC-access page 页面进行跨 local APIC 寄存器边界访问（访问不是完整地包含在 local APIC 寄存器内），也就是访问地址的 **bits 3:2 必须为 0 值**。

以访问 TPR 寄存器（偏移量为 **80H**）为例，有下面的情况：

- 访问 WORD（16 位）数据时，使用偏移量为 83H 则属于跨寄存器边界。而偏移量为 80H 到 82H 之间不属于跨寄存器边界。
- 访问 DWORD（32 位）数据时，使用偏移量为 81H 到 83H 之间都属于跨寄存器边界。只有偏移量为 80H 才不属于跨寄存器边界。

注意：64 位或者 256 位的 local APIC 寄存器（譬如 ICR、IRR）在 APIC-page 内被拆分为多个 32 位的寄存器，这些寄存器都是对齐在 16 位字节边界上。

2. 返回 virtual-APIC page 页面数据

当不是由于上述的原因产生 APIC-access VM-exit 时，取决于 secondary processor-based VM-execution control 字段的“APIC-register virtualization”位的设置，guest 尝试线性读取 APIC-access page 页面，将返回 virtual-APIC page 页面的虚拟寄存器值（参见 7.2.5 节）。

- 当“APIC-register virtualization”位为 0 时，有下面的情况：
 - 线性读取偏移量为 **80H**（TPR）的位置时，从 virtual-APIC page 页面返回 VTPR 值。
 - 线性读取其他的偏移量时，则产生 APIC-access VM-exit。
- 当“APIC-register virtualization”位为 1 时，以下面的偏移量进行读访问，则返回 virtual-APIC page 页面对应偏移量位置的数据（虚拟 local APIC 寄存器，参见 7.2.5 节的表 7-2）。
 - 020H – 023H: local APIC ID。
 - 030H – 033H: local APIC version。
 - 080H – 083H: TPR。
 - 0B0H – 0B3H: EOI。
 - 0D0H – 0D3H: LDR。
 - 0E0H – 0E3H: DFR。
 - 0F0H – 0F3H: SVR。
 - 100H – 103H、110H – 113H、120H – 123H、130H – 133H、140H – 143H、150H – 153H、160H – 163H、170H – 173H: ISR。
 - 180H – 183H、190H – 193H、1A0H – 1A3H、1B0H – 1B3H、1C0H – 1C3H、1D0H – 1D3H、1E0H – 1E3H、1F0H – 1F3H: TMR。
 - 200H – 203H、210H – 213H、220H – 223H、230H – 233H、240H – 243H、250H – 253H、260H – 263H、270H – 273H: IRR。
 - 280H – 283H: ESR。
 - 300H – 303H、310H – 313H: ICR。

- 320H – 323H、330H – 333H、340H – 343H、350H – 353H、360H – 363H、370H – 373H: LVT。
- 380H – 383H: APIC-timer initial count。
- 3E0H – 3E3H: APIC-timer divide configuration。

读取除上述之外的其他偏移量，则产生 APIC-access VM-exit。

注意：guest 尝试以 **guest-physical 读取** APIC-access page 页面则直接产生 APIC-access VM-exit。

7.2.8 虚拟化写入 APIC-access page

当 guest 尝试线性写入 APIC-access page 页面时，产生下面的结果之一。

(1) 产生 APIC-access VM-exit。

(2) 值写入到 virtual-APIC page 页面相应偏移量的位置上，并且执行称为“**APIC-write emulation**”的后续处理。最终结果可能为：

- 产生 APIC-write VM-exit。
- 进行 TPR 虚拟化、EOI 虚拟化，或者 Self-IPI 虚拟化操作。

如上所述，在写入 virtual-APIC page 页面后会进行相应的**后续处理**，而读访问 APIC-access page 则直接返回 virtual-APIC page 页面内的数据。

1. 产生 APIC-access VM-exit

属于下面情况之一时，尝试写 APIC-access page 页面内任何 offset 值都产生 APIC-access VM-exit。

- 当“use TPR shadow”位为 0 时（表示不存在 virtual-APIC page 页面）。
- 访问的数据 size 超过 32 位。例如，尝试写入 64 位数据。
- 访问 APIC-access page 页面的动作发生在虚拟化写 APIC-access page 页面操作的其中一个流程。例如，在虚拟化 TPR 操作时（即写 APIC-access page 页面 80H 位置），最后的步骤是进行 virtual-interrupt 的评估及 delivery 操作（参见 7.2.13 节）。当评估通过后虚拟中断将被 deliver 执行，而在这个在 delivery 期间访问了 APIC-access page 页面。
- 尝试对 APIC-access page 页面进行跨 local APIC 寄存器边界访问（访问不是完整地包含在 local APIC 寄存器内）。也就是访问地址的 **bits 3:2 必须为 0 值**。

以写入 TPR 寄存器（偏移量为 **80H**）为例，有下面的情况：

- 写入 WORD (16 位) 数据时，使用偏移量为 83H 则属于跨寄存器边界，而偏移量为 80H 到 82H 之间不属于跨寄存器边界。
- 写入 DWORD (32 位) 数据时，使用偏移量为 81H 到 83H 之间都属于跨寄存器边界，只有偏移量为 80H 才不属于跨寄存器边界。

注意：64 位或者 256 位的 local APIC 寄存器（譬如 ICR，IRR）在 APIC-page 内被拆分为多个 32 位的寄存器，这些寄存器都是对齐在 16 位字节边界上。

2. 写入 virtual-APIC page 页面

当不是由于上述原因产生 APIC-access VM-exit 时，取决于“APIC-register virtualization”与“virtual-interrupt delivery”位的设置，guest 尝试线性写入 APIC-access page 页面时，值将写入 virtual-APIC page 页面相应偏移量的位置上（**虚拟 local APIC 寄存器**），并执行 APIC-write emulation 操作。

- 当“APIC-register virtualization”与“virtual-interrupt delivery”位都为 0 时，有下面的情况：
 - 线性写 APIC-access page 页面偏移量为 **80H**（TPR）的位置时，值写入 VTPR。
 - 线性写其他偏移量的位置时，将产生 APIC-access VM-exit。
- 当“APIC-register virtualization”位为 0，且“virtual-interrupt delivery”位为 1 时，有下面的情况：
 - 线性写 APIC-access page 页面偏移量为 **80H**（TPR）、**B0H**（EOI）及 **300H**（ICR 低 32 位）的位置时，值分别写入对应的 VTPR、VEOI 及 VICR（检查通过后写低 32 位）。
 - 线性写其他偏移量的位置时，将产生 APIC-access VM-exit。
- 当“APIC-register virtualization”位为 1 时，guest 允许线性写入 APIC-access page 页面偏移量位置如下所示。也就是**可写的** local APIC 寄存器（参见 7.2.5 节的表 7-2）：
 - 020H – 023H: local APIC ID。
 - 080H – 083H: TPR。
 - 0B0H – 0B3H: EOI。
 - 0D0H – 0D3H: LDR。
 - 0E0H – 0E3H: DFR。
 - 0F0H – 0F3H: SVR。
 - 280H – 283H: ESR。
 - 300H – 303H, 310H – 313H: ICR。
 - 320H – 323H、330H – 333H、340H – 343H、350H – 353H、360H – 363H、370H – 373H: LVT。
 - 380H – 383H: APIC-timer initial count。
 - 3E0H – 3E3H: APIC-timer divide configuration。

guest 尝试线性写除上述以外的偏移量位置将产生 APIC-access VM-exit。允许被写入的 local APIC 必须是可写的。因此 local APIC version、ISR、TMR 以及 IRR 不允许写入。

注意：guest 尝试 guest-physical 写 APIC-access page 页面则直接产生 APIC-access VM-exit。

当值成功写入 virtual-APIC page 页面对应的虚拟寄存器后，处理器进行后续的 APIC-write emulation 处理。

3. 执行 APIC-write emulation

执行 APIC-write emulation 的最终结果是：产生 **APIC-write VM-exit**，或者进行 local APIC 的虚拟化操作。

在“APIC-register virtualization”位为 1 的前提下，一个值写入 virtual-APIC page 相应偏移量的**虚拟 local APIC 寄存器**后，执行下面的 APIC-write emulation 操作。

(1) 当“virtual-interrupt delivery”为 0 时，有下面的情况。

- 线性写 APIC-access page 页面偏移量为 **80H** (TPR) 的位置时，值写入 VTPR，并且处理器清 VPTR 的 bits 31:8，接着执行 TPR 虚拟化操作。
- 线性写 APIC-access page 页面其他偏移量的位置时，值写入 local APIC 虚拟寄存器后产生 APIC-write VM-exit。

(2) 当“virtual-interrupt delivery”为 1 时，有下面的情况。

- 线性写 APIC-access page 页面偏移量为 **80H** (TPR) 的位置时，值写入 VTPR，并且处理器清 VPTR 的 bits 31:8，接着执行 **TPR 虚拟化**操作。
- 线性写 APIC-access page 页面偏移量为 **B0H** (EOI) 的位置时，处理器清 VEOI，接着执行 **EOI 虚拟化**操作。
- 线性写 APIC-access page 页面偏移量为 **300H** (ICR 低 32 位) 的位置时，处理器检查下面写入的值是否满足 Self-IPI 虚拟化：
 - 保留位 (bits 31:20, bits 17:16 及 bit 13) 必须为 0。
 - Destination shorthand (bits 19:18) 必须为 **01B**，也就是目标为 **Self**。
 - Delivery status 位 (bit 12) 必须为 0。
 - Trigger mode 位 (bit 15) 必须为 0。
 - Delivery mode (bits 10:8) 必须为 **000B**，也就是使用 **Fixed** 交付模式。
 - Vector 的高 4 位 (bits 7:4) **不能为 0000B**，也就是 Vector 从 **10H 到 FFH**。

当写入值满足上面所有条件则执行 **Self-IPI 虚拟化**操作。否则产生 APIC-write VM-exit。

- 线性写 APIC-access page 页面偏移量为 **310H** (ICR 高 32 位) 的位置时，处理器清 64 位 VICR 的 bits 55:32 (保留 bits 63:56)，不会发生虚拟化操作以及 APIC-write VM-exit。

(3) 线性写 APIC-access page 页面，除了上面所述的情况外将产生 APIC-write VM-exit (包括写其他偏移量的位置，即表 7-2 列出的**可写 local APIC 寄存器**)。

如上所述，APIC-write emulation 其中的一个结果是 APIC-write VM-exit。APIC-write VM-exit 属于 trap 类型，处理器在 VM-exit 时保存的 guest RIP 将指向下一条指令。

4. APIC-write emulation 优先级别

APIC-write emulation 具有很高的优先级别。高于 SMI、INIT 以及其他低优先级别的事件（关于某些事件的优先级别可参见 4.15 节）。

可是，由于 APIC-write emulation 属于 **trap 类型** 事件。因此，在 APIC-write emulation 发生之前，guest 尝试线性写 APIC-access page 页面可能会产生**异常**或者 **VM-exit**。

- 发生异常时，APIC-write emulation 操作发生在异常 delivery 后，异常 handler 执行之前。
- 发生 VM-exit 时，将不会引发 APIC-write emulation 操作。

APIC-write emulation 不能被 **eflags.IF** 标志位、“blocking by MOV-SS”及“blocking by STI”状态阻塞。

7.2.9 虚拟化基于 MSR 读 local APIC

当“virtualize x2APIC mode”位为 1 时，使用基于 MSR 访问 local APIC 寄存器的虚拟化机制。guest 尝试使用 RDMSR 指令读取 local APIC 寄存器，ECX 寄存器提供的 MSR 地址从 **800H** 到 **8FFH** 之间。假设执行 RDMSR 指令不产生异常或 VM-exit（由于 MSR bitmap 而产生 VM-exit），取决于“APIC-register virtualization”位与 ECX 寄存器的值，处理器进行下面的处理。

(1) 当“APIC-register virtualization”位为 0 时，有下面的情况：

- ECX = **808H** 时 (IA32_X2APIC_TPR)，则从 virtual-APIC page 页面的偏移量 **80H** 位置返回 64 位值（包括 VPTR 与它之上的 32 位）到 EDX:EAX。注意：**即使 local APIC 处于非 x2APIC 模式也不受影响。**
- ECX 为 800H 到 8FFH 之间的其他值时，执行 RDMSR 指令被作为正常情况对待。
 - 假如 local APIC 处于 x2APIC 模式，并且 ECX 指示可读的 local APIC 寄存器，则返回物理的 local APIC 寄存器值到 EDX:EAX。
 - 假如 local APIC 处于非 x2APIC 模式（xAPIC 模式或关闭 local APIC），或者 ECX 指示为不可读的 local APIC 寄存器，则产生 #GP 异常。

(2) 当“APIC-register virtualization”位为 1 时，ECX 提供的 MSR 地址在 800H 到 8FFH 之间，处理器从 virtual-APIC page 页面的 **(ECX & 0FFH) << 4** 位置开始读取 64 位值到 EDX:EAX。例如 ECX = 808H 时则从 80H 位置返回 64 位值。**即使 local APIC 处于非 x2APIC 模式也不受影响。**

《Intel 手册》并没有明确指出：当 ECX 提供的 MSR 地址值并没有实现对应的 local APIC 寄存器时会产生 #GP 异常（例如，当 ECX = 800H 时并没有对应的 local APIC 寄存

器)。因此,以 $ECX = 800H$ 为例,我们可以认为处理器将从 virtual-APIC page 页面的 0 位置返回 64 位值。

7.2.10 虚拟化基于 MSR 写 local APIC

当“virtualize x2APIC mode”位为 1 时, guest 尝试使用 WRMSR 指令写 local APIC 寄存器, ECX 寄存器提供的 MSR 地址在 $800H$ 到 $8FFH$ 之间。假设执行 WRMSR 指令不产生异常或 VM-exit (由于 MSR bitmap 而产生 VM-exit), 取决于“virtual-interrupt delivery”位与 ECX 寄存器的值, 处理器进行下面的虚拟化写 local APIC 处理。

(1) $ECX = 808H$ 时 (IA32_X2APIC_TPR), 处理器进行以下步骤的处理:

- 检查 EAX 与 EDX 寄存器的值, EDX 与 $EAX[31:8]$ 必须为 0, 否则产生 #GP 异常。
- $EDX:EAX$ 的 64 位值写入 virtual-APIC page 页面内偏移量为 $80H$ 的位置 (VTPR)。
- 接着执行 TPR 虚拟化操作。

(2) $ECX = 80BH$ (IA32_X2APIC_EOI), 并且“virtual-interrupt delivery”为 1 时, 处理器进行以下步骤的处理:

- 检查 EAX 与 EDX 寄存器的值, $EDX:EAX$ 必须为 0, 否则产生 #GP 异常。
- $EDX:EAX$ 的 64 位值写入 virtual-APIC page 页面内偏移量为 $B0H$ 的位置 (VEOI)。
- 接着执行 EOI 虚拟化操作。

(3) $ECX = 83FH$ (IA32_X2APIC_SELF_IPI), 并且“virtual-interrupt delivery”为 1 时, 处理器进行下面的处理:

- 检查 EAX 与 EDX 寄存器的值, EDX 与 $EAX[31:8]$ 必须为 0, 否则产生 #GP 异常。
- $EDX:EAX$ 的 64 位值写入 virtual-APIC page 页面内偏移量为 $3F0H$ 的位置 (Self-IPI)。
- 检查 EAX 寄存器的 bits 7:4, 不能为 0 值 (即 vector 为 $10H$ 到 FFH 之间)。为 0 时, 将产生 APIC-write VM-exit。
- 不为 0 时, 接着执行 Self-IPI 虚拟化操作。

(4) ECX 为 $800H$ 到 $8FFH$ 之间的其他值, 或者 $ECX = 80BH$ 或 $83FH$ 并且“virtual-interrupt delivery”为 0 时, 执行 WRMSR 指令被作为正常情况对待:

- 假如 local APIC 处于 x2APIC 模式, 并且 ECX 指示为可写的寄存器, $EDX:EAX$ 的 64 位值写入物理的 local APIC 寄存器。没有后续的虚拟化操作!
- 假如 local APIC 处于非 x2APIC 模式, 或者 ECX 指示为不可写的寄存器, 则产生 #GP 异常。

综上所述, 在基于 MSR 的写 local APIC 寄存器虚拟化里, 不存在“APIC-write emulation”概念。但是, 虚拟化写 IA32_X2APIC_TPR、IA32_X2APIC_EOI 或者

IA32_X2APIC_SELF_IPI 时，实际上相当于进行了 APIC-write emulation 处理。

另一方面，尝试虚拟化写 IA32_X2APIC_SELF_IPI 寄存器时（ECX = 83FH），处理器检查到提供的 vector 不符合要求时会产生 APIC-write VM-exit。

7.2.11 虚拟化基于 CR8 访问 TPR

在 64 位模式下（CR8 寄存器仅能使用于 64 位模式下），处理器支持通过 CR8 寄存器作为接口访问 TPR 寄存器。

VMM 可以使用下面两种方式虚拟化 CR8 访问 TPR：

（1）通过设置 primary processor-based VM-execution control 字段的“CR8-load exiting”以及“CR8-store exiting”位来监控 guest 访问 CR8 寄存器。

（2）使用原生的（native）CR8 虚拟化机制。

VMM 设置“CR8-load exiting”以及“CR8-store exiting”位为 1 时，guest 尝试访问 CR8 寄存器产生 VM-exit 后，VMM 进行相应的处理达到虚拟化目的。

在执行 MOV-CR8 指令不产生异常或 VM-exit 时（即“CR8-load exiting”与“CR8-store exiting”位为 0），当“use TPR shadow”位为 1 时，guest 访问 CR8 寄存器则访问到 virtual-APIC page 页面内偏移量为 80H 的位置（即 VTPR）。

- 执行 MOV-from-CR8 指令：将 virtual-APIC page 页面内 VTPR 的 bits 7:4 值返回到目标寄存器的 bits 3:0，目标寄存器的 bits 63:4 清为 0 值。
- 执行 MOV-to-CR8 指令：将源寄存器 bits 3:0 值写入 virtual-APIC page 页面内 VTPR 的 bits 7:4，VTPR 的 bits 3:0 与 31:8 清为 0 值。接着执行 TPR 虚拟化操作。

7.2.12 local APIC 虚拟化操作

APIC-write emulation 的另一个结果是执行 local APIC 状态的虚拟化操作，包括下面几个虚拟化操作。

- TPR 虚拟化
 - 在虚拟化 xAPIC 模式时，由 guest 线性写 APIC-access page 页面偏移量 80H（TPR）产生的结果。
 - 在虚拟化 x2APIC 模式时，由 guest 执行 WRMSR 指令写 IA32_X2APIC_TPR 寄存器（ECX = 808H）产生的结果。
 - 在 64 位模式下，guest 执行 MOV-to-CR8 指令产生的结果。
- PPR 虚拟化：由 TPR 虚拟化、EOI 虚拟化或者 VM-entry 而引发。
- EOI 虚拟化
 - 在虚拟化 xAPIC 模式时，由 guest 线性写 APIC-access page 页面偏移量 B0H（EOI）产生的结果。

- 在虚拟化 x2APIC 模式时，由 guest 执行 WRMSR 指令写 IA32_X2APIC_EOI 寄存器 (ECX = 80BH) 产生的结果。
- Self-IPI 虚拟化
 - 在虚拟化 xAPIC 模式时，由 guest 线性写 APIC-access page 页面偏移量 300H (ICR 低 32 位) 产生的结果。
 - 在虚拟化 x2APIC 模式时，由 guest 执行 WRMSR 指令写 IA32_X2APIC_SELF_IPI 寄存器 (ECX = 83FH) 产生的结果。

在 TPR 虚拟化、EOI 虚拟化以及 Self-IPI 虚拟化操作最后阶段里，会进行 **virtual-interrupt** 的评估及 delivery 操作（参见 7.2.13 节）。当仲裁通过后，虚拟中断通过 guest-IDT 进行 delivery 处理。

7.2.12.1 TPR 虚拟化

处理器在完成写入 virtual-APIC page 页面偏移量 80H 的 VTPR 后（前面 7.2.12 节 TPR 虚拟化所述的三种途径写 VTPR），执行的最后工作是进行 TPR (task priority register) 虚拟化处理。

```
do_tpr_virtualization()
{
    if ("virtual-interrupt delivery" == 0)
    {
        if (VTPR[7:4] < TPR_threshold[3:0])
        {
            /* 由于低于 TPR threshold 值而产生 VM-exit */

            do_vmexit_processing(EXIT_NUMBER_TPR_BELOW_THRESHOLD);
        }
    }
    else
    {
        do_ppr_virtualization();          /* 执行 PPR 虚拟化 */
        do_evaluate_virtual_interrupt();  /* 评估虚拟中断的 delivery */
    }
}
```

如上面伪代码所示，处理器执行下面的 TPR 虚拟化处理：

- 当“virtual-interrupt delivery”为 1 时。
 - 引发下一步 PPR 虚拟化的执行（参见 7.2.12.2 节）。
 - 接着进行 virtual-interrupt 的评估及 delivery 操作（参见 7.2.13 节）。
- 当“virtual-interrupt delivery”为 0 时。
 - 处理器检查到写入 VTPR 的 bits 7:4 值低于 TPR threshold 字段值 bits 3:0 时，将产生 VM-exit（参见 3.5.11 与 5.3 节）。

由 TPR 低于 threshold 而产生的“TPR below threshold VM-exit”属于 **trap** 类型的 VM-exit 事件（参见 5.10 节相关描述）。

7.2.12.2 PPR 虚拟化

PPR (processor priority register) 是只读寄存器, 不能通过写 VPPR 发起 PPR 虚拟化操作。如前面第 7.2.12 节所述, 能引发 PPR 虚拟化的是 **TPR 虚拟化**、**EOI 虚拟化** 及 **VM-entry** 这三种途径。

PPR 虚拟化执行更新 VPPR 动作。VPPR 的更新取决于 guest interrupt status 字段中的 **SVI** (Servicing virtual interrupt) 状态值 (参见 3.8.11 节) 和 **VTPR** 值, 取两者的最大值。

```
do_ppr_virtualization()
{
    /*
     * 更新 VPPR 值
     */
    if (VTPR[7:4] >= SVI[7:4])
    {
        VPPR = VTPR & 0xFF;
    }
    else
    {
        VPPR = SVI & 0xF0;
    }

    VPPR[31:8] = 0;          /* 清 VPPR[31:8] */
}
```

如上面伪码所示, 处理器执行下面的 PPR 虚拟化处理:

- 更新 VTPR 值。
 - 当 VTPR[7:4] 大于或等于 SVI[7:4] 时, 使用 VTPR 来更新 VPPR。
 - 当 VTPR[7:4] 低于 SVI[7:4] 时, 使用 SVI 来更新 VPPR。
- 清 VPPR 的 bits 31:8 值。

注意: 在 virtual-interrupt 的 delivery 操作里, 处理器修改 SVI、VISR 以及 RVI 值不会引发 PPR 虚拟化 (尽管处理器也修改 VPPR 值, 但不属于 PPR 虚拟化操作)。

7.2.12.3 EOI 虚拟化

在一个中断例程返回前应该向**中断控制器** (例如 8259、local APIC) 发送 EOI 命令, EOI 命令引发中断控制器进行一些**复位**或者**更新状态**工作 (譬如清中断 In-service 位), 以便为**下一个**中断例程服务。

EOI 命令的发送被假设在中断例程内执行, EOI 虚拟化需要在 virtual-interrupt (虚拟中断) 例程内进行, 因此, 引发 EOI 虚拟化的前提是 “virtual-interrupt delivery” 位为 1。

处理器在完成写入 virtual-APIC page 页面偏移量 **B0H** 的 VEOI 后 (前面第 7.2.12 节所述的两种途径), 执行 EOI 虚拟化操作。

```
do_eoi_virtualization()
{
    vector = SVI;          /* 当前正在运行的中断例程向量号 */
    VISR[vector] = 0;      /* 清 VISR 对应位 */

    /*
```

```

    * 更新 SVI 值
    */
    if (VISR != 0)
    {
        SVI = get_highest_index_of_bitset(VISR); /* 得到 VISR 中为 1 位的最高 index
值 */
    }
    else
    {
        SVI = 0; /* 不存在中断例程的嵌套，则 SVI 为 0 */
    }

    do_ppr_virtualization(); /* 执行 PPR 虚拟化 */

    if (eoi_exit_bitmap[vector] == 1)
    {
        /*
        * 由于 EOI-exit bitmap 而产生 VM-exit
        */
        do_vmexit_processing(EXIT_NUMBER_EOI);
    }
    else
    {
        /*
        * 执行 pending virtual-interrupt delivery 评估
        */
        do_evaluate_virtual_interrupt();
    }
}

```

如上面伪码所示，处理器执行下面的 EOI 虚拟化处理：

- 清 VISR 的 In-service 位。当虚拟中断例程发送 EOI 命令时，处理器将清向量号（由 SVI 指示）在 VISR 对应的位。表示正在执行的虚拟中断例程已经结束，可以执行另一个虚拟中断例程。
- 更新 SVI 值。
 - 当 VISR 的值为 0 时，表示目前没有虚拟中断例程的嵌套（即低优先级的中断例程被高优先级中断例程所中断），当前的 In-service 列表为空，SVI 被设为 0 值。
 - 当 VISR 的值不为 0 时，处理器从 VISR 中取出**优先级别最高的**虚拟中断例程向量号，将这个向量号赋予 SVI。指示下一个正在运行的**虚拟中断例程**。
- 接着进行 PPR 虚拟化处理（参见 7.2.12.2 节）。
- 检查 EOI-exit bitmap 字段（参见 3.5.12 节），决定是否产生 VM-exit。
 - 当发送 EOI 命令的虚拟中断向量号在 EOI-exit bitmap 字段对应位为 1 时，产生 VM-exit。
 - 否则进行 virtual-interrupt 的评估及 delivery 操作。

如上所述，EOI 虚拟化使用 SVI（servicing virtual-interrupt）状态值来进行虚拟中断例程的收尾工作，并且更新 SVI 值，以便于处理器服务下一个虚拟中断例程。

由 EOI 虚拟化产生的 VM-exit 属于 trap 类型（另见 5.10 节描述）。

7.2.12.4 Self-IPI 虚拟化

引发 Self-IPI 虚拟化操作是由于 guest 线性写 APIC-access page 页面偏移量 **300H** 位置，或者执行 WRMSR 指令写 IA32_X2APIC_SELF_IPI 寄存器 (**ECX = 83FH**)。同样，能引发 Self-IPI 虚拟化的前提是“virtual-interrupt delivery”位为 1。

当处理器完成写入 virtual-APIC page 页面的 **300H** 或者 **3F0H** (IA32_X2APIC_SELF_IPI) 位置后，执行 Self-IPI 虚拟化处理。

```
do_self_ipi_virtualization()
{
    vector = VICR[7:0];                /* 从 VICR 或 IA32_X2APIC_SELF_IPI 中取得中断
向量号 */
    VIRR[vector] = 1;                  /* VIRR 对应位置位，表示有虚拟中断请求 */

    RVI = get_highest_index_of_bitset(VIRR); /* 从 VIRR 里取出优先级最高的虚拟中断
请求向量号 */

    do_evaluate_virtual_interrupt();      /* 进行虚拟中断评估 */
}
```

如上面伪码所示，处理器执行下面的 Self-IPI 虚拟化处理：

- 从 VICR 的 bits 7:0 里取出虚拟中断向量号。注意，当写 IA32_X2APIC_SELF_IPI 寄存器时，则从 virtual-APIC page 页面的 **3F0H** 位置的 bits 7:0 取中断向量号。
- 置中断向量号在 VIRR（虚拟中断请求寄存器）的对应位，指示有**虚拟中断请求**出现。
- 从 VIRR 列表中取出优先级别最高的虚拟中断请求向量号赋予 **RVI**（Requesting virtual interrupt，参见 3.8.11 节）。指示下一个将要 **delivery** 的虚拟中断例程。
- 执行 virtual-interrupt 的评估及 delivery 操作。

在 Self-IPI 虚拟化处理里，依赖 **RVI**（参见 3.8.11 节）状态值而发出虚拟中断请求，并且更新 RVI 值。

注意：在这个操作里并不会产生 VM-exit 事件。

7.2.13 虚拟中断的评估与 delivery

能引发虚拟中断的评估及 delivery 操作的前提是“virtual-interrupt delivery”位为 1，并且“**external-interrupt exiting**”位也必须为 1（参见 4.4.1.1 节）。注意，当处理器接收到 external interrupt（外部中断）时会产生 VM-exit。

因此，一个外部中断不能引起虚拟中断的评估与 **delivery**。外部中断是指：

- 由 local APIC 的 LVT 寄存器产生的本地中断（例如由 **LVT timer**、**LVT perfmon** 产生的中断）。
- local APIC 接收到的外部中断请求。包括 8259 中断控制器发出的中断请求，I/O APIC 中断控制器发出的中断消息，以及 **IPI**（处理器间的中断消息）。

因此，虚拟中断的评估与 delivery 只能由下面几个途径引发（另参见 7.2.12 节）：

(1) **TPR 虚拟化、EOI 虚拟化、Self-IPI 虚拟化。**

(2) 在 **VM-entry** 操作时引发。

(3) 在利用 “**Posted-interrupt processing**” 机制处理外部中断时引发（参见 7.2.14 节）。

除了上述的途径外，没有其他的途径可以引发 virtual-interrupt 的评估及 delivery 操作，即使改变了 RVI 或 VPPR 的值。

1. 注入虚拟中断

VMM 可以通过 VM-entry interruption information 字段来注入一个**外部中断**或**软件中断**给 guest 处理（参见 4.12 节）。另一方面，VMM 可以通过 guest-interrupt status 字段的 RVI（参见 3.8.11 节）来注入一个**虚拟中断**给 guest 处理。

```
SetVmcsField    GUEST_INTERRUPT_STATUS, 60h    ; 注入虚拟中断 60h
```

上面的代码示例中，VMM 设置 guest-interrupt status 字段的值为 60H，从而注入一个虚拟中断 60H 给 guest 处理。在完成 VM-entry 操作后（包括注入事件 delivery），处理器接着执行 **PPR 虚拟化**，然后执行 **virtual-interrupt 的评估与 delivery** 操作（参见 4.9 节）。

在 guest 端里，guest 只能通过写 VICR[31:0]或者 IA32_X2APIC_SELF_IPI 寄存器执行 Self-IPI 虚拟化操作来主动产生虚拟中断。

2. 发送通知 IPI

在 host 端里，VMM 除了可以**注入虚拟中断**外，也可以利用 “posted-interrupt processing” 机制来实现虚拟中断的 delivery。VMM 需要通过 posted-interrupt notification vector 字段来发送一个**通知 IPI 中断**给 guest，guest 收到通知 IPI 后接着处理 VMM 设置的 **posted-interrupt**（通告中断）。

7.2.13.1 虚拟中断的评估

处理器在执行虚拟中断（virtual-interrupt）delivery 之前，需要评估**是否允许组织生成虚拟中断的 pending 信息**。一旦生成了虚拟中断的 pending 信息，处理器将 deliver 这个虚拟中断执行。

```
do_evaluate_virtual_interrupt()
{
    if ("interrupt-window exiting" == 0)
    {
        /*
         * 评估 virtual-interrupt 是否允许组织
         */
        if (RVI[7:4] > VPPR[7:4])
        {
            /*
             * 通过评估，组织一个 pending virtual-interrupt
            */
        }
    }
}
```

```

        */
        do_recognize_virtual_interrupt();

        /*
        * 检查 virtual-interrupt 是否允许 deliver
        */
        if (eflags.IF == 1 && "blocking by STI" == 0
            && "blocking by MOV-SS" == 0)
        {
            do_deliver_virtual_interrupt(); /* 执行 deliver 虚拟中断 */
        }
    }
}

```

如上面的伪码所示，当 primary processor-based VM-execution control 字段的 “interrupt-window exiting” 位为 0 时，才允许组织虚拟中断的 pending 信息。当 “interrupt-window exiting” 为 1 时，在打开中断窗口后会直接引发 VM-exit（参见 3.5.2.1 及 4.15.7 节）。

处理器接着评估虚拟中断优先级别。当 **RVI[7:4]** 大于 **VPPR[7:4]** 时，处理器组织一个虚拟中断 pending 在下一条指令边界上（当在 VM-entry 时则在 guest 第 1 条指令边界上）。处理器接着检查虚拟中断是否被阻塞，当不被阻塞时通过 guest-IDT 进行 deliver 执行。

1. 虚拟中断的阻塞

虚拟中断（virtual-interrupt）具有与外部中断（external-interrupt）一样的**阻塞属性**。虚拟中断是否允许 deliver 取决于处理器当前的 **interruptibility** 以及 **activity** 状态（参见 4.16 与 4.17 节描述）。

- **interruptibility（可中断性）**：当 $IF = 1$ ，并且不存在 “blocking by STI” 以及 “blocking by MOV-SS” 阻塞状态时，虚拟中断才允许 deliver。
- **activity（活动性）**：当处理器不在 shutdown 以及 wait-for-SIPI 状态时，虚拟中断才允许 deliver。

因此，虚拟中断在处理器的 HLT 状态下不会被阻塞。虚拟中断的 deliver 可以唤醒处理器的 HLT 状态进入 active 状态。

2. 虚拟中断的优先级

虚拟中断的优先级别与 “interrupt-window exiting” 一致（参见 4.15 与 4.15.7 节）。可是，在 “interrupt-window exiting” 为 1 时，虚拟中断不会被组织 pending。

在前面 4.15 节的描述里，虚拟中断的优先级别**低于** “NMI-window exiting”、“NMI exiting”、NMI 以及其他更高优先级的事件。虚拟中断的优先级等同于 “interrupt-window exiting”，**高于** “external-interrupt exiting” 和 external-interrupt（外部中断请求）。

注意：虚拟中断与外部中断是不同的，虚拟中断只能通过 TPR 虚拟化、EOI 虚拟化、self-IPI 虚拟化、VM-entry 操作以及 posted-interrupt processing 机制产生，而外部中断通过中断控制器发送请求或者接收到 IPI 消息产生。

但是，在 posted-interrupt（通告中断）的处理过程中，虚拟中断的评估及 delivery 操作是在处理器检查“external-interrupt exiting”之后发生，也就是在 guest 收到 notification vector（通知向量）后。

7.2.13.2 虚拟中断的 delivery

虚拟中断通过 guest-IDT 进行 delivery 时，处理器将更新 virtual-APIC 的状态，包括 VIRR、VISR、RVI 及 SVI 状态。如下面的代码所示。

```
do_deliver_virtual_interrupt()
{
    /*
     * 更新 virtual-APIC 状态
     */
    vector = RVI;                /* 将要 deliver 的中断请求 */
    VISR[vector] = 1;           /* 置 VISR 的 In-service 位 */
    SVI = vector;               /* 指示正在 In-service 的中断例程 */
    VPPR = vector & 0xF0;       /* 更新 VPPR 为当前 In-service 的中断例程的优先级 */
    VIRR[vector] = 0;           /* 清 VIRR 的中断请求位 */

    /*
     * 更新 RVI 状态值，指向下一个将要 deliver 的虚拟中断
     */
    if (VIRR != 0)
    {
        RVI = get_highest_index_of_bitset(VIRR);
    }
    else
    {
        RVI = 0;
    }

    do_delivery_of_guest_idt(vector); /* 通过 guest-IDT 进行 delivery */
}
```

处理器执行下面的虚拟中断 delivery 处理。

- 更新 virtual-APIC 状态。
 - 设置 VISR 的 In-service 位：读取 RVI 值，指示将要 deliver 的虚拟中断向量号。根据这个向量号在 VISR 对应位置位，指示将要在 In-service（进入服务）的虚拟中断。注意：在 Self-IPI 虚拟化里更新 RVI 值，在 EOI 虚拟化里更新 SVI 值。
 - 更新 SVI 值：SVI 被赋予将要 deliver 的虚拟中断向量号，指示正在运行的虚拟中断。
 - 更新 VPPR 状态：根据 deliver 的虚拟中断向量号更新 VPPR 值（vector & F0H）。指示当前处理器的中断优先级别。
 - 更新 VIRR 状态：处理器从 VIRR 的**虚拟中断请求列表**中取出中断请求后需要清对应的请求位，表明已经响应虚拟中断。
 - 更新 RVI 值：RVI 指向下一条将要 deliver 的虚拟中断。假如 VIRR 的请求列表还存在另一个虚拟中断，则需要从 VIRR 取出**优先级别最高的**虚拟中断请求赋予 RVI。假如 VIRR 请求列表为空，则 RVI 为 0 值。

- 处理器根据虚拟中断向量号通过 guest-IDT 进行 deliver 执行。

当然，在虚拟中断的 delivery 期间可能会发生异常而产生 VM-exit，VMM 需要根据情况进行相应的处理（参见 7.1 节）。

7.2.14 posted-interrupt 处理

posted-interrupt processing（通告中断处理）是一项很实用的技术，在较早前的处理器并不支持。VMM 通过检查 Pin-based VM-execution control 字段的“process posted interrupts”位（bit 7）是否允许设为 1 值来检查处理器是否支持（参见 2.5.6.1 节）。

posted-interrupt（直译为“通告中断”）是指 VMM 张贴出来给 guest 进行处理的**虚拟中断**。guest 收到一个**通知**（向量号为 notification vector 的**外部中断**）后，将张贴的**通告中断请求列表**复制到 virtual-APIC page 页面的 VIRR 里，从而**更新虚拟中断请求列表**，然后进行虚拟中断的评估与 delivery。

1. 启用 posted-interrupt 处理机制

posted-interrupt processing 机制用来处理 VMM 预先为 guest 设置的 **virtual-interrupt**（虚拟中断）。VMM 将“process posted interrupts”位置 1 开启 posted-interrupt processing 机制，同时也需要满足下面的设置（参见 4.4.1.1 节）。

- pin-based VM-execution control 字段的“external-interrupt exiting”位必须为 1。
- VM-exit control 字段的“acknowledge interrupt on exit”位必须为 1。

由此可见，当 local APIC 收到外部中断请求时，要么产生 VM-exit，要么就执行虚拟中断的评估与 delivery 操作。“acknowledge interrupt on exit”位为 1，表明处理器收到外部中断请求后，将响应中断控制器（local APIC）从而取得中断向量信息。

处理器响应 local APIC 得到的中断向量被称为“**physical vector**”（物理向量号），处理器将使用这个 physical vector 与“**notification vector**”（通知向量）进行对比。

2. posted-interrupt 的通知向量号

VMM 需要在 posted-interrupt notification vector 字段里提供一个**通知向量号**（参见 3.5.13 节），那么，local APIC 接收到的这个外部中断可以被称为“**通知中断**”，其作用是通知 guest 进行 posted-interrupt 的处理。

在 guest 里，当 local APIC 收到外部中断请求时，处理器检查这个外部中断请求的向量号与 notification vector（通知向量号）是否相等（即检查是否为通知中断）。相等则将 posted-interrupt 描述符内的 PIR 请求列表复制到 VIRR，形成新的虚拟中断请求列表，然后执行 virtual-interrupt 的评估与 delivery 操作。如图 7-8 所示。

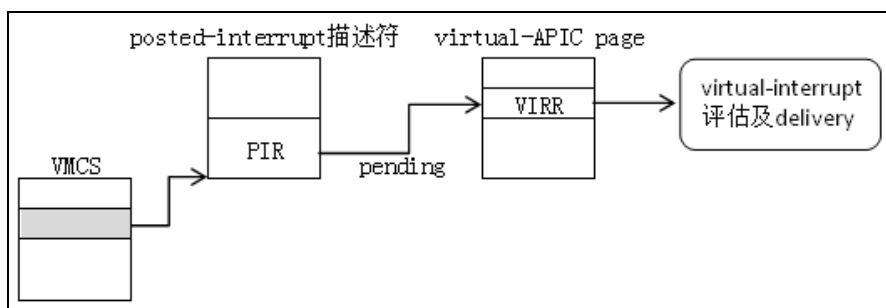


图 7-8

当外部中断向量号不等于 notification vector 时，则产生 VM-exit。由于“acknowledge interrupt on exit”位为 1，在 VM-exit 时处理器从 local APIC 里获取外部中断的信息，保存在 VM-exit interruption information 字段里（参见 3.7.1 节）。

3. posted-interrupt 描述符

VMM 需要在 posted-interrupt descriptor address 字段里提供一个 64 位物理地址（必须是 64 字节边界对齐，参见 4.4.1.1 节），指向一个被称为“**posted-interrupt descriptor**”（通告中断描述符）的数据结构（参见 3.5.14 节）。posted-interrupt 描述符结构为 512 位宽，如表 7-3 所示。

表 7-3

位 域	名 字	描 述
255:0	posted-interrupt requests	PIR (posted-interrupt request)，某位为 1 时表明存在相应的 posted-interrupt 请求
256	outstanding notification	ON (待处理通知位)，为 1 时表明有一个或多个 posted-interrupt 请求需要处理
511:257	software available	AVL (软件可用位)，此部分保留给软件使用，处理器不会修改此域

描述符的 bits 255:0 是 PIR (posted-interrupt request) 域，每一位对应一个中断向量号，（譬如 bit 60 对应中断向量号 60）。某位为 1 时表明出现了该位对应的中断请求（中断向量号对应为该位）。bit 256 是 ON (outstanding notification) 位，为 1 时表明 PIR 中有一个或多个 posted-interrupt 需要处理。处理器不使用及修改 bits 511:257 位域部分，保留给软件使用。

注意：posted-interrupt 描述符与 VMCS 使用的其他数据结构不同（譬如 exception bitmap、I/O bitmap、MSR bitmap 等）。当 VMCS 所对应的 VM 正在运行时（进入 VMX non-root operation 模式），VMM 不能修改这些数据结构。

而 posted-interrupt 描述符允许在 VM 运行时被 VMM 修改（修改其他逻辑处理器的 posted-interrupt 描述符），但必须使用 locked 的“**read-modify-write**”操作。如下面的代码所示：

```
lock or DWORD [rbx + VMB.PostedInterruptDesc + (61h / 8)], (1 << (61h & 7))
```

上面的代码中，VMM 将 61h 号中断向量在 posted-interrupt 描述符 **PIR** 对应的位置 1，使用了加了 **lock** 前缀的 OR 指令完成。这个操作允许在目标 VMCS 对应的 guest 正在运行时设置。

4. posted-interrupt 处理流程

在启用 posted-interrupt processing 机制后，处理器对外部中断的处理将有很大的不同。

- 在一般情况下，当“external-interrupt exiting”为 1，处理器接收到外部中断则产生 VM-exit。“external-interrupt exiting”为 0，则通过 guest-IDT 进行 deliver 执行。
- 在 posted-interrupt processing 机制下，“external-interrupt exiting”与“acknowledge interrupt on exit”位必须为 1。当处理器接收到的外部中断向量号是**通知向量号**（notification vector）时不会产生 VM-exit，否则将产生 VM-exit。

当接收到的外部中断向量号等于通知向量号时不会产生 VM-exit，而是进行 PIR（posted-interrupt request）的 pending 处理（如图 7-8 所示）。

处理器执行的 posted-interrupt 处理流程如下面的伪码所示。

```
do_posted_interrupt_processing()
{
    /*
     * 检查外部中断的向量号是否等于“通知向量号”
     */
    if (vector != notification_vector)
    {
        /*
         * 不相等时，由于“external-interrupt exiting”产生 VM-exit
         */
        do_vmexit_processing(EXIT_NUMBER_EXTERNAL_INTERRUPT);
    }
    else
    {
        /*
         * 相等时，进行 pending posted-interrupt 的评估与 delivery
         */
        PostedInterruptDesc.On = 0; /* 清 posted-interrupt 描述符的 ON 位 */
        send_eoi_to_local_apic(vector); /* 根据 vector 向 local APIC 发送 EOI 命令 */
        PIR = PostedInterruptDesc.Pir; /* 读取 posted-interrupt 描述符的 PIR */
        VIRR |= PIR; /* VIRR “OR” PIR */
        PostedInterruptDesc.Pir = 0; /* 清空 posted-interrupt 描述符的 PIR */

        /*
         * 从 PIR 列表中取出最高优先级别的 posted-interrupt 请求
         */
        pi_vector = get_highest_index_of_bitset(PIR);

        /*
         * 更新 RVI 值，取决于 PIR 请求列表是否为空
         */
        if (PIR != 0)
        {
            /*
             * 从原 RVI 与 PIR 列表的最高优先级别中断请求两者间比较，取最高值作为新的 RVI 值
            */
        }
    }
}
```

```

        */
        RVI = pi_vector > RVI ? pi_vector : RVI;
    }
    else
    {
        /* 当 PIR 列表为空时, RVI 值维持不变 */
    }

    /*
     * 执行虚拟中断的评估与 delivery 操作
     */
    do_evaluate_virtual_interrupt();
}
}

```

当 local APIC 接收或产生外部中断时, 处理器执行下面的 posted-interrupt 处理流程:

(1) 处理器响应 local APIC 后, 读取外部中断向量号 (即前面所说的 physical vector)。

(2) 检查外部中断的向量号是否等于 notification vector (通知向量号), **不等于则产生 VM-exit**, 相等则继续下面的处理。在这个步骤里检查这个外部中断是否为通知中断。

(3) 处理器清 posted-interrupt 描述符内的 ON 位 (outstanding notification, bit 256), 这个动作类似于执行一条 locked 的 AND 指令 (AND 指令清位操作)。清 ON 位的作用是锁上 posted-interrupt 描述符不允许修改。

(4) 处理器**自动**向 local APIC 发送一条 EOI 命令 (清 EOI 寄存器), 这个操作将**驱散通知中断**。因此, 通知中断不会被 deliver 执行。

(5) 处理器从 posted-interrupt 描述符里读取 PIR (bits 255:0), 将 PIR 执行“**逻辑或**”在 VIRR 上 (也就是将 PIR 的中断请求列表 pending 在 VIRR 里, 形成新的虚拟中断请求), 并且将 PIR 列表清空。在这一步操作里, 没有其他的访问者 (譬如处理器、芯片组等) 可以访问 PIR 位域。

(6) 处理器更新 RVI 值。从 PIR 请求列表里取出优先级别最高的中断请求, 与原 RVI 值进行比较。取**两者最高的值作为新的 RVI 值**。显然, 当 PIR 中断请求列表为空时, RVI 值维护不变。

(7) 处理器最后执行虚拟中断的评估与 delivery 操作 (参见 7.2.13 节)。虚拟中断是否能被组织 pending 取决于评估的结果。

在上面的 1 到 7 步骤里不允许被其他事件中断。在第 7 步里, 虚拟中断一旦被组织就会立即执行虚拟中断的 delivery (参见 7.2.13.2)。如前面 7.2.13.1 节所述, 未被屏蔽的虚拟中断 delivery 可以唤醒处理器的 HLT 状态, 但在 shutdown 或 wait-for-SIPI 状态下被阻塞。

5. posted-interrupt 的使用

guest 收到的通知中断可以是来自中断控制器 (譬如 8259、I/OAPIC、local APIC), 也可以是处理器发出的 IPI。posted-interrupt processing 机制的典型用法是: 在多处理器平

台里，一个逻辑处理器正处于 VMX non-root operation 模式里运行（guest），而另一个逻辑处理器以 IPI 的形式发送一个通知中断给这个 guest。如图 7-9 所示。

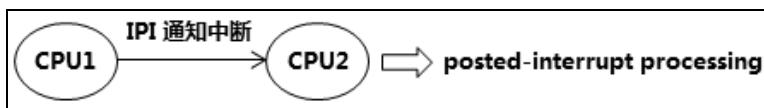


图 7-9

VMM 分配一个具有**高优先级别**的向量号作为 notification vector（通知向量）保存在 posted-interrupt notification vector 字段里，以便在**虚拟中断的评估环节**里可以获得评估通过（参见 7.2.13.1 节）。

VMM 也要为目标 guest 分配虚拟中断向量号，通过 guest-IDT 进行 deliver 执行。VMM 使用通知向量号发送 IPI 给目标处理器（正在运行 guest），目标处理器接收到 IPI 后进行 posted-interrupt 处理。

另外，VMM 可以在 posted-interrupt 描述符内的 PIR 里张贴多个虚拟中断请求，那么 guest 根据仲裁情况可以执行多个虚拟中断处理。

7.3 中断处理

在 VMX 架构下，VMM 能监控 guest 的**外部中断**（external-interrupt）、**SMI** 以及 **NMI**。VMM 也能监控 guest 引发的**硬件异常**以及**软件异常**（#BP 与 #OF，由 INT3 与 INTO 指令引发）。然而 VMX 却没有提供对 **INT** 指令（**软件中断**）的监控能力，因此 VMM 拦截 guest 执行 INT 指令需要通过其他途径。

7.3.1 拦截 INT 指令

当 guest 执行一条 INT 指令时，VMX 不能通过指令条件设置来监控 INT 指令的执行。由于 INT 指令需要访问 IDT（interrupt descriptor table），那么 VMM 监控 INT 指令可行的方法是：**监控 guest 访问 IDT**。

思考一下，应该怎样来监控 guest 访问 IDT？注意，**下面的方法是不可行的**。

- 替换 IDTR.base 值。这个方法企图通过替换 IDTR 寄存器的 base 值来监控 guest 访问 IDT，这是行不通的。主要原因有下面两个：
 - 无法处理 guest 产生的异常，IDTR.base 值必须是一个可用的线性地址值。VMM 可能会企图将 IDTR.base 替换为一个不存在的地址值，或者不映射这个 IDTR.base 对应的 guest-physical address。可是，当 guest 发生异常时，IDTR.base 的地址必须有效，处理器才能 deliver 这个异常进行处理。从这个角度看 VMM 也只能监控一次 INT 指令的执行。
 - 可以被 guest OS 识别出来，并且在设置 IDT 时会产生一定的混乱。当 guest OS

直接通过线性地址读写 IDT 描述符时，如果 VMM 利用 EPT violation 来捕捉 guest OS 的这个访问，必须将 IDT 内容保持同步更新，否则会被 guest 检测出来，但这样做也就失去了监控 IDT 的意义。

- 替换 IDT 中断描述符内的目标地址值。这个做法同样会被 guest 检测出来，并且 guest 更新描述符时还需要同步更新 IDT 描述符内容。

因此，我们看到在访问 IDT 描述符上下工夫是行不通的，唯一可行的方法是**替换 IDT 的 limit 值**。例如，我们将 IDT 设置为只能容纳 32 个中断描述是一个不错的选择。

```
do_int_process(UCHAR vector)
{
    ... ..

    if (EFER.LMA == 1)
    {
        if ((vector * 16 + 15) > IDTR.limit)
        {
            /*
             * 产生 #GP 异常
             */
            do_GP(ERROR_CODE(vector, IDT, EXT));
        }
        ... ..
    }
    else
    {
        if ((vector * 8 + 7) > IDTR.limit)
        {
            /*
             * 产生 #GP 异常
             */
            do_GP(ERROR_CODE(vector, IDT, EXT));
        }
        ... ..
    }
    ... ..
}
```

在上面的 do_int_process 伪码里显示，处理器在执行 INT 指令时首先会检查使用 vector 读取描述符时是否超出 IDTR.limit 值。

- 在 IA-32e 模式下检查是否 $\text{vector} \times 16 + 15 > \text{IDTR.limit}$ 。
- 在 legacy 模式下检查是否 $\text{vector} \times 8 + 7 > \text{IDTR.limit}$ 。

当超出 IDT 的 limit 时，产生 #GP 异常。VMM 利用这一点将 IDTR.limit 设置一个较小的值，从而迫使 guest 在执行 INT 指令时产生 #GP 异常导致 VM-exit，VMM 则进行相应的处理。

7.3.1.1 处理 IDTR.limit

我们知道，在 x86/x64 体系上 0 到 31 号向量是 Intel 架构保留的，主要作为异常向量号。软件不应该将 0 到 31 作为中断向量号，并且在 local APIC 上使用 0 到 15 作为中断

向量号将产生错误。

因此，我们可以将 IDT 设置为只容纳 32 个描述符。

- 在 IA-32e 模式下 (long-mode)，IDTR.limit 的值设为 **1FFh** ($31 \times 16 + 15$)。
- 在 legacy 模式下，IDTR.limit 的值设为 **FFh** ($31 \times 8 + 7$)。

当 guest 执行 INT 指令使用的向量号超过 31 时，将产生 #GP 异常。VMM 拦截 guest 的 #GP 异常后进行分析，如果属于软件中断、外部中断以及特权级软件中断，则由 VMM 完成中断 delivery 操作。

设置 IDTR.limit 值

在 guest 使用 LIDT 指令加载 IDTR 寄存器时，VMM 设置 secondary processor-based VM-execution control 字段的“descriptor-table exiting”位为 1，拦截 LIDT 指令的执行。

代码片段 7-28:

```
DoGDTR_IDTR.Lidt:
    DEBUG_RECORD    "[DoGDTR_IDTR]: load IDTR"

    ;;
    ;; 处理 LIDT 指令: 写入 GIMB.IdtPointer 以及加载 guest IDTR
    ;;
    REX.Wrxb
    lea ecx, [ecx + VMB.GuestImb + GIMB.IdtPointer]
    movzx eax, WORD [ebx]
    mov [ecx + GIMB.IdtLimit], ax                ; 保存 guest 原 IDTR.limit

    ;;
    ;; 在 IA-32e 模式下是 1FFh, 否则为 0FFh
    ;;
    GetVmcsField    GUEST_IA32_EFER_FULL
    mov esi, (31 * 8 + 7)
    test eax, EFER_LMA
    mov eax, (31 * 16 + 15)
    cmovz eax, esi

    ;;
    ;; 设置 guest IDTR.limit
    ;;
    SetVmcsField    GUEST_IDTR_LIMIT, eax        ; 加载 guest IDTR.limit
    mov WORD [ecx + GIMB.HookIdtLimit], ax      ; 保存 VMM 设置的
IDTR.limit
    mov eax, [ebx + 2]
    and eax, 00FFFFFFh
    cmp DWORD [edx + INSTRUCTION_INFO.OperandSize], INSTRUCTION_OPS_WORD
    cmovne eax, [ebx + 2]
    cmp DWORD [edx + INSTRUCTION_INFO.OperandSize], INSTRUCTION_OPS_QWORD
    REX.Wrxb
    cmov eax, [ebx + 2]
    REX.Wrxb
    mov [ecx + GIMB.IdtBase], eax                ; 保存 guest 原 IDTR.base
    SetVmcsField    GUEST_IDTR_BASE, eax        ; 加载 guest IDTR.base
```

上面的代码片段来自 \lib\Vm\VmxBMM.asm 文件内的 DoGDTR_IDTR 例程。VMM 先将 guest 尝试写入 IDTR 寄存器的 limit 值保存在 GIMB (Guest IDT Manage Block) 结

构内的 IdtLimit 里。

当 guest 处于 IA-32e 模式时, 设置的 limit 值为 1FFh, 否则为 0FFh 值。这个值将写入 IDTR 寄存器的 limit 字段里, 并且保存在 GIMB.HookIdtLimit 里。

```

    if (OperandSize == 32)
    {
        IDTR.base = *((int *)(IdtPointer + 2));           // 读取 32 位值
    }
    else if (OperandSize == 64)
    {
        IDTR.base = *((int64 *)(IdtPointer + 2));        // 读取 64 位值
    }
    else
    {
        IDTR.base = *((int *)(IdtPointer + 2)) & 0x00FFFFFF; // 读取 24 位值
    }

```

接下来根据指令操作数 size 来加载 IDTR.base 值, 在 64 位模式下加载 64 位的 base 值, 在非 64 位模式下加载 32 位 (操作数为 32 位) 或者 24 位 (操作数为 16 位)。

guest 尝试写入的 IDTR.base 值也保存在 GIMB.IdtBase 值里。GIMB 结构定义在 \inc\vmcs.inc 文件里。DoGDTR_IDTR 例程开头使用 GetDescTableRegisterInfo 函数来收集相关信息, 这个函数实现在 \lib\Vm\VmxEit.asm 文件里。

代码片段 7-29:

```

DoGDTR_IDTR.SgdtSidt:
    ;;
    ;; 处理 SGDT 与 SIDT 指令: 保存 GDT/IDT pointer
    ;;
    mov ax, [esi]
    mov [ebx], ax
    mov eax, [esi + 2]
    mov [ebx + 2], eax

    ;;
    ;; 检查 operand size, 如果是 64 位则写入 10 bytes
    ;;
    cmp DWORD [edx + INSTRUCTION_INFO.OperandSize], INSTRUCTION_OPS_QWORD
    jne DoGDTR_IDTR.Done
    mov eax, [esi + 6]
    mov [ebx + 6], eax
    jmp DoGDTR_IDTR.Done

```

上面的代码同样来自 \lib\Vm\VmVMM.asm 文件内的 DoGDTR_IDTR 例程, 用来处理 guest 尝试执行 SIDT 指令。它将 GIMB.IdtBase 和 GIMB.IdtLimit 值分别写入 guest 的目标内存地址里。

另外, VMM 也需要拦截 guest 修改 IA32_EFER 寄存器。当 guest 开启 IA-32e 模式时, 需要将 IDTR.limit 值相应地调整为 1FFh 值 (假设当前 IDTR.limit 值设为 0FFh)。这样才能有效地拦截软件中断的执行。实现这个功能的代码在 \lib\Vm\VmMsr.asm 文件内的 DoWriteMsrEfer 函数里。

7.3.1.2 处理#GP 异常

当 guest 尝试执行下面的软件中断指令时，以 60h 中断向量为例。

```
int 60h ; 中断在 delivery 期间，由于向量号超出 IDTR.limit 而产生#GP 异常
```

处理器在进行中断 delivery 期间，由于我们设置了 IDTR.limit 值为 0FFh，向量号 60h 超出了 IDT 的 limit 范围而产生#GP 异常。同时，VMM 设置 exception bitmap 的 bit 13 为 1 而导致 VM-exit。

那么，软件中断在 delivery 期间产生了#GP 异常而导致 VM-exit，属于间接向量事件（参见 3.10.3 节）。处理器在 VM-exit 时将记录：

- IDT-vectoring information 字段的值为 **80000460H**。指示软件中断，向量号为 60h。
- VM-exit interruption information 字段的值为 **8000B0DH**。指示硬件异常#GP，并且存在错误码。
- VM-exit interruption error code 字段的值为 **00000303H**。指示向量号为 60h，IDT 与 EXT 位为 1。

VMM 可以根据这些信息来判断#GP 发生在什么环节，VMM 的处理逻辑如下面的伪码所示。

```
if (InterruptType == SOFTWARE_INTERRUPT ||
    InterruptType == EXTERNAL_INTERRUPT ||
    InterruptType == PRIVILEGE_INTERRUPT)
{
    do_int_process(vector);          // 执行中断 delivery 操作
}
else if (InterruptType == HADRWARE_EXCEPTION)
{
    if (vector == DE_VECTOR || vector == TS_VECTOR || vector == NP_VECTOR
        || vector == SS_VECTOR || vector == GP_VECTOR || vector == PF_VECTOR)
    {
        do_reflect_exception(#DF);    // 反射 #DF 给 guest 处理
    }
    else if (vector == DF_VECTOR)
    {
        do_enter_shutdown();          // 处理器进入 shutdown 状态
    }
    else
    {
        do_reflect_exception(#GP);    // 反射 #GP 给 guest 处理
    }
}
else
{
    do_reflect_exception(#GP);        // 反射 #GP 给 guest 处理
}
```

根据原始向量事件（IDT-vectoring information）的中断类型进行相应的处理：

（1）属于软件中断、外部中断或者特权级软件中断时，由 VMM 完成中断的 delivery 工作。

（2）属于硬件中断时，有下面的处理：

- 属于#DE、#TS、#NP、#SS、#GP 或者#PF 异常时，VMM 需要反射一个#DF (double fault) 异常给 guest 处理（参见 7.1.1 节）。
- 属于#DF 异常时，VMM 需要让虚拟处理器进入 shutdown 状态。
- 属于其他异常时（实际上并不存在），反射一个#GP 异常给 guest 处理。

(3) 属于其他中断类型时，直接反射#GP 给 guest 处理。

在\lib\Vm\VmVMM.asm 文件里的 do_GP 例程实现这个处理，如代码片段 7-30 所示。

代码片段 7-30:

```

;-----
; do_GP()
; input:
;     none
; output:
;     eax - VMM 处理码
; 描述:
;     1) 处理由 #GP 引发的 VM-exit
;-----
do_GP:
    push ebp
    push ebx
    push edx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    DEBUG_RECORD    "[do_GP]..."

    REX.Wrbx
    mov ebx, [ebp + PCB.CurrentVmbPointer]

    call get_interrupt_info                ; 收集中断相关信息

    ;;
    ;; 属于 software interrupt, external-interrupt 以及 privileged interrupt 时，执行
中断处理
    ;;
    movzx eax, BYTE [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.InterruptType]
    cmp al, INTERRUPT_TYPE_SOFTWARE
    je do_GP.DoInterrupt
    cmp al, INTERRUPT_TYPE_EXTERNAL
    je do_GP.DoInterrupt
    cmp al, INTERRUPT_TYPE_PRIVILEGE
    je do_GP.DoInterrupt

    ;;
    ;; 反射处理:
    ;; 1) 当 IDT-vectoring information 记录异常为#DE,#TS,#NP,#SS,#PF 或者#GP 时，需要反射
#DF 异常
    ;; 2) 当 IDT-vectoring information 记录异常为#DF 异常时，需要处理 triple fault
    ;;

```

```

    cmp eax, INTERRUPT_TYPE_HARD_EXCEPTION
    jne do_GP.ReflectGp
    mov al, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Vector]
    cmp al, DF_VECTOR
    je do_GP.TripleFault
    cmp al, DE_VECTOR
    je do_GP.ReflectDf
    cmp al, TS_VECTOR
    je do_GP.ReflectDf
    cmp al, NP_VECTOR
    je do_GP.ReflectDf
    cmp al, SS_VECTOR
    je do_GP.ReflectDf
    cmp al, PF_VECTOR
    je do_GP.ReflectDf
    cmp al, GP_VECTOR
    jne do_GP.ReflectGp

do_GP.ReflectDf:
    mov eax, 0
    mov ecx, INJECT_EXCEPTION_DF
    jmp do_GP.ReflectException

do_GP.ReflectGp:
    mov eax, [ebp + PCB.ExitInfoBuf + EXIT_INFO.InterruptErrorCode]
    mov ecx, INJECT_EXCEPTION_GP
    jmp do_GP.ReflectException

do_GP.TripleFault:
    ;;
    ;; 处理 triple fault: 进入 shutdown 状态
    ;;
    SetVmcsField    GUEST_ACTIVITY_STATE, GUEST_STATE_SHUTDOWN
    jmp do_GP.Done

    ;;
    ;; ##### 下面 VMM 进行中断的 delivery 处理 #####
    ;;

do_GP.DoInterrupt:
    DEBUG_RECORD    "process INT instruction"

    movzx esi, BYTE [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Vector]
    call do_int_process
    jmp do_GP.Done

do_GP.ReflectException:
    SetVmcsField    VMENTRY_EXCEPTION_ERROR_CODE, eax
    SetVmcsField    VMENTRY_INTERRUPTION_INFORMATION, ecx

do_GP.Done:
    mov eax, VMM_PROCESS_RESUME
    pop edx
    pop ebx
    pop ebp
    ret

```

get_interrupt_info 函数负责收集相关的中断 delivery 信息。函数的实现在 \lib\Vm\VmExit.asm 文件里, INTERRUPT_INFO 结构定义在 \inc\VmExit.inc 文件里。

注意：当 guest 执行的 INT 指令使用**向量号低于 32** 时，VMM 并不能由于 IDTR.limit 而拦截到。如下面的代码所示。

```
int 3          ; INT 指令向量号为 3 (指令码 CD 03)
int 1Fh        ; INT 指令向量号为 31
```

然而，上面的 INT 指令使用示例不属于 guest 正常使用范围。因此，对于 OS 以正常使用 INT 指令 VMM 可以监控到，我们这个目的已达到，这已经足够了！

另一方面，产生硬件与软件异常的指令（也就是 BOUND、UD2、INT3 与 INTO 指令），它们的向量号不会超过 32。因此，它们可以正常使用而不受影响。

当然，guest 异常处理是不受影响的，因为异常向量号都低于 32。

7.3.1.3 处理中断 delivery

VMM 拦截 INT 指令后，需要帮助处理器完成中断例程的 delivery 操作确保中断例程的执行。处理器执行中断的 delivery 操作是非常复杂的。

VMM 要处理 4 个模式下的中断 delivery：

- 实模式下。作为示例，我们这里忽略实模式下的中断 delivery 处理。
- virtual-8086 模式下。作为示例，我们这里忽略 virtual-8086 下的中断 delivery 处理。
- 保护模式下，我们需要处理。
- IA-32e 模式（即 long-mode）下，我们需要处理。

VMM 还要细分目标代码是在**权限内**（包括 conforming 段），还是**权限外**的情况。作为示例，这里也忽略了对 task-gate 的处理。

下面是 do_int_process 例程实现中断 delivery 的 C 伪代码。

```
do_int_process(UCHAR Vector)
{
    if (IA32_EFER.LMA == 0)
    {
        /* step 1: 检查 vector 是否超出 IDTR.limit */
        if ((Vector * 8 + 7) > IDTR.limit)
        {
            do_GP(ERROR_CODE(Vector, IDT, EXT));
        }

        /* step 2: 读取 IDT 描述符 */
        IdtDesc = read_idt_desc(vector);

        /* step 3: 检查 IDT 描述符是否属于 gate */
        if (IdtDesc.Type != TASK_GATE && IdtDesc.Type != INTERRUPT_GATE &&
            IdtDesc.Type != TRAP_GATE)
        {
            do_GP(ERROR_CODE(Vector, IDT, EXT));
        }

        /* step 4: 检查权限 */
        if (InterruptType == SOFTWARE_INTERRUPT)
        {

```

```

        if (CPL > IdtDesc.Dpl)
        {
            do_GP(ERROR_CODE(Vector, IDT, 0));
        }
    }

    /* step 5: 检查 IDT-gate 是否为 present */
    if (IdtDesc.P == 0)
    {
        do_NP(ERROR_CODE(Vector, IDT, EXT));
    }

    /* step 6: 检查 gate 类型 */
    if (IdtDesc.Type == TASK_GATE)
    {
        do_task_switch();
        return;
    }
}
else
{
    /*
     * ##### IA-32e 模式下的中断处理 #####
     */

    /* step 1: 检查 vector 是否超出 IDTR.limit */
    if ((Vector * 16 + 15) > IDTR.limit)
    {
        do_GP(ERROR_CODE(Vector, IDT, EXT));
    }

    /* step 2: 读 IDT 描述符 */
    IdtDesc = read_idt_desc64(Vector);

    /* step 3: 检查 IDT 描述符是否属于 interrupt-gate, trap-gate */
    if (IdtDesc.Type != INTERRUPT_GATE && IdtDesc.Type != TRAP_GATE)
    {
        do_GP(ERROR_CODE(Vector, IDT, EXT));
    }

    /* step 4: 检查权限 */
    if (InterruptType == SOFTWARE_INTERRUPT)
    {
        if (CPL > IdtDesc.Dpl)
        {
            do_GP(ERROR_CODE(Vector, IDT, 0));
        }
    }

    /* step 5: 检查 IDT-gate 是否为 present */
    if (IdtDesc.P == 0)
    {
        do_NP(ERROR_CODE(Vector, IDT, EXT));
    }
}

/*
 * ##### 处理 interrupt-gate 与 trap-gate 的情况 #####
 */

```

```

/* step 7: 检查 code-segment selector 是否为 NULL */
if (TargetCs == NULL_SELECTOR)
{
    do_GP(ERROR_CODE(0, 0, EXT));
}

/* step 8: 检查 code-segment selector 是否超出 GDTR.limit */
/* #### 作为示例, 这里忽略 LDT #### */
if (((TargetCs & 0xFFF8) + 7) > GDTR.limit)
{
    do_GP(ERROR_CODE(TargetCs & 0xFFF8, 0, EXT));
}

/* step 9: 读取 code-segment 描述符 */
TargetCsDesc = read_gdt_desc(TargetCs);

/* step 10: 检查是否属于 code-segment */
if (TargetCsDesc.CD == 0)
{
    do_GP(ERROR_CODE(TargetCs & 0xFFF8, 0, EXT));
}

/* step 11: 检查权限 */
if (CPL < TargetCsDesc.Dpl)
{
    do_GP(ERROR_CODE(TargetCs & 0xFFF8, 0, EXT));
}

/* step 12: 检查 code-segment 是否为 present */
if (TargetCsDesc.P == 0)
{
    do_NP(ERROR_CODE(TargetCs & 0xFFF8, 0, EXT));
}

/* step 13: 根据权限进行相应处理 */
if (TargetCsDesc.Dpl < Cpl)
{
    /*
     * CPL > DPL 时, 进行权限外的中断处理
     */
    if (IA32_EFER.LMA == 1)
    {
        do_interrupt_for_inter_privilege_longmode(TargetCsDesc.Dpl);
    }
    else
    {
        do_interrupt_for_inter_privilege(TargetCsDesc.Dpl);
    }
}
else
{
    /*
     * CPL == DPL 时, 进行权限内的中断处理
     */
    if (IA32_EFER.LMA == 1)
    {
        do_interrupt_for_intra_privilege_longmode();
    }
}

```

```

        else
        {
            do_interrupt_for_intra_privilege();
        }
    }
}

```

do_int_process 例程实现在\lib\Vmx\VmxException.asm 文件里，它的主要工作如下。

(1) 检查在 vector 读取 IDT 描述符时是否超出 IDTR.limit 值，当超出时产生#GP 异常。

- IA-32e 模式下：检查 $\text{vector} \times 16 + 15 > \text{IDTR.limit}$ 。
- Legacy 模式下：检查 $\text{vector} \times 8 + 7 > \text{IDTR.limit}$ 。

(2) 根据 vector 读取 IDT 描述符。

- IA-32e 模式下：从 $\text{IDTR.base} + \text{vector} \times 16$ 处读取 16 个字节的 IDT 描述符。
- Legacy 模式下：从 $\text{IDTR.base} + \text{vector} \times 8$ 处读取 8 个字节的 IDT 描述符。

(3) 检查 IDT 描述符是否为 gate，不符合要求时产生#GP 异常。

- IA-32e 模式下：只支持 interrupt-gate 与 trap-gate 描述符。
- Legacy 模式下：支持 task-gate、interrupt-gate 以及 trap-gate 描述符。

(4) 当属于软件中断时，检查权限： $\text{CPL} \leq \text{IdtDesc.Dpl}$ ，不通过时产生#GP 异常。

当属于外部中断或者特权级软件中断时，不需要检查权限。

(5) 检查 IDT 描述符是否为 present，为 not-present 时产生#NP 异常。

(6) 根据 gate 类型进行相应处理。

- IA-32e 模式下，进行下一步的 interrupt-gate 与 trap-gate 处理。
- Legacy 模式下，属于 task-gate 时进行任务切换，否则进行下一步。

(7) 检查目标 CS selector 是否为 NULL selector，为 NULL selector 时产生#GP 异常。

(8) 检查目标 CS selector 是否超出 GDTR.limit： $(\text{TargetCs} \& \text{FFF8h}) + 7 > \text{GDTR.limit}$ (这里忽略了 LDT)。超出时产生#GP 异常。

(9) 从 $\text{GDTR.base} + \text{TargetCs} \& \text{FFF8h}$ 处读取 8 个字节的 code-segment 描述符到 TargetCsDesc 里。

(10) 检查 TargetCsDesc 的 C/D 位是否为 code-segment。不属于 code-segment 时产生#GP 异常。

(11) 检查权限：需要 $\text{TargetCsDesc.Dpl} \leq \text{CPL}$ ，否则产生#GP 异常。

(12) 检查 TargetCsDesc 是否为 present，为 not-present 时产生#NP 异常。

(13) 根据 TargetCsDesc.Dpl 进行相应的处理。

- IA-32e 模式下：
 - 权限内（同级权限）调用 do_interrupt_for_intra_privilege_longmode 例程。

- 权限外（切换到高权限）调用 `do_interrupt_for_inter_privilege_longmode` 例程。
- Legacy 模式下：
 - 权限内（同级权限）调用 `do_interrupt_for_intra_privilege` 例程。
 - 权限外（切换到高权限）调用 `do_interrupt_for_inter_privilege` 例程。

在 `do_int_process` 例程里检查产生的异常，将采用事件注入方式反射相应的异常给回 guest 处理，VMM 不做任何处理。

注意：由 VMM 反射 #GP 回给 guest 处理不会打乱 `do_GP` 例程的处理。因为，事件注入不会因为 exception bitmap 而产生 VM-exit。

下面的 `do_int_process` 例程按照前面描述的 C 伪码来实现。

代码片段 7-31:

```

;-----
; do_int_process()
; input:
;     esi - vector
; output:
;     none
; 描述:
;     1) 处理中断 delivery 操作
;-----
do_int_process:
    push ebp
    push ebx
    push edx
    push ecx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    DEBUG_RECORD    "[do_int_process]..."

    REX.Wrb
    mov ebx, [ebp + PCB.CurrentVmbPointer]

    mov ecx, esi                ; ecx = vector

    ;;
    ;; 检查是否处于 IA-32e 模式
    ;; 1) 是，处理 IA-32e 模式下的 INT 指令
    ;; 2) 否，处理 protected 模式下的 INT 指令
    ;;
    test DWORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.GuestStatus],
    GUEST_STATUS_LONGMODE
    jnz do_int_process.Longmode

do_int_process.Protected:
    DEBUG_RECORD    "[do_int_process.Protected]..."

    ;;
    ;; ##### 处理 protected 模式下的 INT 指令执行 #####

```

```

;;
shl ecx, 3

;;
;; step 1: 检查 vector 是否超出 IDT limit
;; 1) (vector * 8 + 7) > limit ?
;;
mov edx, ecx
lea esi, [edx + 7]
cmp si, [ebx + VMB.GuestImb + GIMB.IdtLimit]
jbe do_int_process.Protected.ReadDesc

do_int_process.Gp_vector_11B:
;;
;; error code = vector | IDT | EXT
;;
mov eax, ecx
or eax, 3
mov ecx, INJECT_EXCEPTION_GP
jmp do_int_process.ReflectException

do_int_process.Gp_vector_10B:
;;
;; error code = vector | IDT | 0
;;
mov eax, ecx
or eax, 2
mov ecx, INJECT_EXCEPTION_GP
jmp do_int_process.ReflectException

do_int_process.Gp_CsSelector_01B:
do_int_process.Gp_vector_01B:
;;
;; error code = vector | 0 | EXT
;;
mov eax, ecx
or eax, 1
mov ecx, INJECT_EXCEPTION_GP
jmp do_int_process.ReflectException

do_int_process.Gp_01B:
mov eax, 1
mov ecx, INJECT_EXCEPTION_GP
jmp do_int_process.ReflectException

do_int_process.Np_vector_11B:
;;
;; error code = vector | 1 | EXT
;;
mov eax, ecx
or eax, 3
mov ecx, INJECT_EXCEPTION_NP
jmp do_int_process.ReflectException

do_int_process.Np_CsSelector_01B:
;;
;; error code = selector | 0 | EXT
;;
mov eax, ecx
or eax, 1

```

```

        mov ecx, INJECT_EXCEPTION_NP
        jmp do_int_process.ReflectException

do_int_process.Protected.ReadDesc:
    ;;
    ;; step 2: 读 IDT 描述符
    ;;
    REX.Wrxb
    add edx, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtBase]
    mov esi, [edx]
    mov edi, [edx + 4]
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc], esi
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc + 4], edi

do_int_process.Protected.CheckType:
    ;;
    ;; step 3: 检查 IDT 描述符是否属于 gate
    ;;
    shr edi, 8
    and edi, 0Fh
    cmp edi, 0101B                ; task-gate
    je do_int_process.Protected.CheckPrivilege
    cmp edi, 1110B                ; interrupt-gate
    je do_int_process.Protected.CheckPrivilege
    cmp edi, 1111B                ; trap-gate
    jne do_int_process.Gp_vector_11B

do_int_process.Protected.CheckPrivilege:
    ;;
    ;; step 4: 当属于 software-interrupt 时, 检查权限: CPL <= IDT-gate.DPL
    ;;
    movzx eax, BYTE [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.InterruptType]
    cmp eax, INTERRUPT_TYPE_SOFTWARE
    jne do_int_process.Protected.CheckPresent

    movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Cp1]
    mov esi, [edx + 4]
    shr esi, 13
    and esi, 3
    cmp esi, eax
    jb do_int_process.Gp_vector_10B

do_int_process.Protected.CheckPresent:
    ;;
    ;; step 5: 检查 gate 是否为 present
    ;;
    test DWORD [edx + 4], (1 << 15)
    jz do_int_process.Np_vector_11B

do_int_process.Protected.GateType:
    ;;
    ;; step 6: 检查 gate 类型
    ;;
    test DWORD [edx + 4], (1 << 9)
    jnz do_int_process.InterruptTrap

    ;;
    ;; ### 保留处理 task-gate ###

```

```

;;
jmp do_int_process.Done

do_int_process.InterruptTrap:
;;
;; step 7: 检查 code-segment selector 是否为 NULL
;;
movzx ecx, WORD [edx + 2]
mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCs], cx
and ecx, 0FFF8h
jz do_int_process.Gp_01B

;;
;; step 8: 检查 code-segment selector 是否超出 GDT limit
;;
mov esi, ecx
add esi, 7
cmp si, [ebx + VMB.GuestGmb + GGMB.GdtLimit]
ja do_int_process.Gp_CsSelector_01B

;;
;; step 9: 读取 code-segment 描述符
;;
REX.Wrxb
mov edx, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.GdtBase]
mov esi, [edx + ecx]
mov edi, [edx + ecx + 4]
mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc], esi
mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc + 4], edi

;;
;; step 10: 检查描述符 C/D 位, 是否为 code-segment
;;
test edi, (1 << 11)
jz do_int_process.Gp_CsSelector_01B

;;
;; step 11: 检查权限: DPL <= CPL
;;
mov esi, edi
shr esi, 13
and esi, 3
cmp si, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Cpl]
ja do_int_process.Gp_CsSelector_01B

;;
;; step 12: 检查 code-segment 描述符是否为 present
;;
test edi, (1 << 15)
jz do_int_process.Np_CsSelector_01B

do_int_process.InterruptTrap.Next:
;;
;; step 13: 根据权限进行相应处理
;;
;; 注意: ### 作为例子, 保留实现对 conforming 类型段的处理 ###
;;      ### 作为例子, 保留实现对 virtual-8086 模式下的中断处理 ###

```

```

;;
mov eax, do_interrupt_for_inter_privilege
mov edi, do_interrupt_for_intra_privilege
cmp si, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Cpl]
cmovz eax, edi
mov ecx, do_interrupt_for_inter_privilege_longmode
mov edi, do_interrupt_for_intra_privilege_longmode
cmovz ecx, edi
test DWORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.GuestStatus],
GUEST_STATUS_LONGMODE
cmovnz eax, ecx
call eax
jmp do_int_process.Done

do_int_process.Longmode:
;;
;; 处理 longmode 模式下的 INT 指令执行
;;
DEBUG_RECORD "[do_int_process.Longmode]..."

;;
;; step 1: 检查 vector 是否超出 IDT.limit
;;
shl ecx, 4
lea esi, [ecx + 15]
cmp si, [ebx + VMB.GuestImb + GIMB.IdtLimit]
ja do_int_process.Gp_vector_11B

;;
;; step 2: 读 IDT 描述符
;;
REX.Wrxb
mov edx, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtBase]
REX.Wrxb
add edx, ecx
REX.Wrxb
mov esi, [edx]
REX.Wrxb
mov edi, [edx + 8]
REX.Wrxb
mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc], esi
REX.Wrxb
mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc + 8], edi

;;
;; step 3: 检查 IDT 描述符是否属于 interrupt-gate, trap-gate
;;
mov edi, [edx + 4]
shr edi, 8
and edi, 0Fh
cmp edi, 1110B ; interrupt-gate
je do_int_process.Longmode.CheckPrivilege
cmp edi, 1111B ; trap-gate
jne do_int_process.Gp_vector_11B

do_int_process.Longmode.CheckPrivilege:
;;
;; step 4: 当属于 software-interrupt 时, 检查权限: CPL <= IDT-gate.DPL
;;

```

```

movzx eax, BYTE [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.InterruptType]
cmp eax, INTERRUPT_TYPE_SOFTWARE
jne do_int_process.Longmode.CheckPresent

mov eax, [edx + 4]
shr eax, 13
and eax, 3
cmp al, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Cp1]
jnb do_int_process.Gp_vector_10B

do_int_process.Longmode.CheckPresent:
;;
;; step 5: 检查 IDT-gate 是否为 present
;;
test DWORD [edx + 4], (1 << 15)
jnz do_int_process.InterruptTrap

;;
;; #NP (error code = vector | IDT | EXT)
;;
movzx eax, BYTE [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Vector]
shl eax, 3
or eax, 3
mov ecx, INJECT_EXCEPTION_NP

do_int_process.ReflectException:
;;
;; 注入异常
;;
SetVmcsField VMENTRY_EXCEPTION_ERROR_CODE, eax
SetVmcsField VMENTRY_INTERRUPT_INFORMATION, ecx
mov eax, DO_INTERRUPT_ERROR

do_int_process.Done:
pop ecx
pop edx
pop ebx
pop ebp
ret

```

do_int_process 例程只是处理了中断 delivery 的一部分工作，剩下的主要工作由下面 4 个例程去完成：

(1) do_interrupt_for_inter_privilege 函数执行 legacy 模式下切入高权限的中断 delivery。

(2) do_interrupt_for_intra_privilege 函数执行 legacy 模式下同级权限的中断 delivery。

(3) do_interrupt_for_inter_privilege_longmode 函数执行 IA-32e 模式下切入高权限的中断 delivery。

(4) do_interrupt_for_intra_privilege_longmode 函数执行 IA-32e 模式下同级权限的中断 delivery。

这 4 个例程实现在 \lib\Vm\VmException.asm 文件里。

7.3.1.4 完成中断的 delivery 操作

do_int_process 例程进行了中断 delivery 的前半部分工作，这里我们以 IA-32e 模式下为例，介绍剩余的中断 delivery 工作。

1. 权限外的 delivery 处理

do_interrupt_for_inter_privilege_longmode 例程完成 IA-32e 模式下切入高权限代码的 delivery 工作。它的工作流程如下面的 C 伪码所示。

```
do_interrupt_for_inter_privilege_longmode(UINT Dpl)
{
    /* step 1: 根据 IST 值计算 stack pointer 位置 */
    StackOffset = IdtDesc.Ist ? IdtDesc.Ist * 8 + 28 : Dpl * 8 + 4;

    /* step 2: 检查 stack pointer 位置是否超出 TSS limit */
    if ((StackOffset + 7) > TR.limit)
    {
        do_TS(ERROR_CODE(TssSelector, 0, EXT));
    }

    /* step 3: 读取 stack pointer 值 */
    TargetRsp = read_stack_pointer_from_TSS(StackOffset); // 读取 8 个字节的 RSP
    TargetSs = Dpl;                                     // SS = NULL selector

    /* step 4: 检查 RSP 是否为 canonical 地址形式 */
    if (!IS_CANONICAL(TargetRsp))
    {
        do_SS(ERROR_CODE(0, 0, EXT));
    }

    /* step 5: RSP 向下调整到 16 字节边界对齐 */
    TargetRsp &= 0xFFFFFFFFFFF0;
    Rsp = get_system_va_of_guest_os(TargetRsp)           // TargetRsp 取得 system 地址

    /* step 6: 读取目标 RIP, 并检查 RIP 是否为 canonical 地址形式 */
    TargetRip = get_target_rip(IdtDesc);                 // 从 IDT 描述符里读取 RIP 值
    if (!IS_CANONICAL(TargetRip))
    {
        do_GP(ERROR_CODE(0, 0, EXT));
    }

    /* step 7: 加载 RSP 与 SS */
    GUEST_RSP = TargetRsp - 40;                          // 调整目标 RSP
    GUEST_SS = LoadSsSegment(TargetSs);                 // 加载目标 SS

    /* step 8: 压入返回信息 */
    *(Rsp - 1) = OldSs;
    *(Rsp - 2) = OldRsp;
    *(Rsp - 3) = OldFlags;
    *(Rsp - 4) = OldCs;
    *(Rsp - 5) = OldRip;

    /* step 9: 加载 CS:RIP */
    GUEST_RIP = TargetRip;
    GUEST_CS = LoadCsSegment(TargetCs);                 // 加载目标 CS
}
```

```

/* step 10: 更新 rflags 寄存器 */
TmpFlags = OldFlags;
TmpFlags.TF = 0;
TmpFlags.VM = 0;
TmpFlags.RF = 0;
TmpFlags.NT = 0;

if (IdtDesc.Type == INTERRUPT_GATE)
{
    TmpFlags.IF = 0;
}

GUEST_RFLAGS = TmpFlags;
}

```

do_interrupt_for_inter_privilege_longmode 例程主要进行下面的工作。

(1) 根据 IDT 描述符的 IST 值，计算出目标权限级别的 stack pointer 在 TSS 的位置。

- IST = 0 时，StackOffset = $DPL \times 8 + 4$ 。
- IST > 0 时，StackOffset = $DPL \times 8 + 28$ 。

(2) 检查 stack pointer 位置是否超出 TSS 的 limit: $(StackOffset + 7) > TR.limit$ 时产生 #TS 异常。

(3) 从 $TR.base + StackOffset$ 位置上读取 8 个字节的 RSP 值到 TargetRsp。

(4) 检查 TargetRsp 是否为 canonical 地址形式，属于 non-canonical 时产生 #SS 异常。

(5) TargetRsp 值向下调整到 16 字节边界对齐。

(6) 从 IdtDesc 里读取 8 个字节的 offset 值到 TargetRip，检查 TargetRip 是否为 canonical 地址形式，属于 non-canonical 时产生 #GP 异常。

(7) 加载 SS:RSP。guest 的 RSP 值需要减去 40（这是压入 5 个值后的 RSP 值），SS 被加载为 NULL selector（值等于 TargetCsDesc.Dpl）。

(8) 在 TargetRsp 指向的目标栈中（由 TargetRsp 对应的 Rsp）相继写入 OldSs、OldRsp、OldFlags、OldCs 及 OldRip 值。

(9) 加载 CS:RIP 值，分别写入 guest-CS 的 selector、access rights、limit 及 base 字段。Guest-RIP 字段加载为 TargetRip 值。

(10) 更新 RFLAGS 寄存器。TF、VM、RF 以及 NT 位都被清 0。

- 中断描述符属于 interrupt-gate 时，IF = 0。
- 中断描述符属于 trap-gate 时，IF 位保持不变。

下面是完整的 do_interrupt_for_inter_privilege_longmode 例程实现代码。

代码片段 7-32:

```

;-----
; do_interrupt_for_inter_privilege_longmode()
; input:
;     esi - privilege level

```



```

; output:
;     eax - status code
; 描述:
;     1) 处理 longmode 模式下的特权级内的中断
;-----
do_interrupt_for_inter_privilege_longmode:
    push ebp
    push ebx
    push edx
    push ecx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    DEBUG_RECORD    "[do_interrupt_for_inter_privilege_longmode]..."

    REX.Wrxb
    mov ebx, [ebp + PCB.CurrentVmbPointer]
    mov ecx, esi

    ;;
    ;; step 1: 根据 IST 值计算 stack pointer 偏移量
    ;;
    mov edx, esi
    shl edx, 3
    add edx, 4
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc + 4]
    and eax, 7
    lea esi, [eax * 8 + 28]
    cmovnz edx, esi

    ;;
    ;; step 2: 检查 stack pointer 是否超出 TSS limit
    ;;
    mov esi, edx
    add esi, 7
    cmp esi, [ebx + VMB.GuestTmb + GTMB.TssLimit]
    ja do_interrupt_for_inter_privilege_longmode.Ts_TsSelector_01B

    ;;
    ;; step 3: 读取 stack pointer
    ;;
    REX.Wrxb
    add edx, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TssBase]
    REX.Wrxb
    mov esi, [edx]                ;; new RSP
    mov edi, [edx + 4]
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetSs], cx    ;; 保存 SS
                                ;; SS selector 被加载为 NULL

    ;;
    ;; step 4: 检查 RSP 是否为 canonical 地址形式, 否则产生 #SS(EXT)
    ;;
    shrd eax, edi, 16
    sar eax, 16
    cmp eax, edi
    jne do_interrupt_for_inter_privilege_longmode.Ss_01B

```

```

;;
;; step 5: RSP 向下调整到 16 字节边界对齐
;;

    REX.Wrxb
    and esi, ~0Fh                ;; new RSP & FFFF_FFFF_FFFF_FFF0h
    REX.Wrxb
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRsp], esi
    call get_system_va_of_guest_os
    REX.Wrxb
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Rsp], eax        ;; 保存 RSP

;;
;; step 6: 检查 RIP 是否为 canonical 地址形式, 否则产生 #GP(EXT)
;;
    movzx esi, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc]
    mov edi, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc + 4]
    and edi, 0FFFF0000h
    or esi, edi
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc + 8]
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRip], esi
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRip + 4], eax
    shr esi, 16
    sar esi, 16
    cmp esi, eax
    jne do_interrupt_for_inter_privilege_longmode.Gp_01B

;;
;; step 7: 加载 RSP, SS = NULL-selector
;;
    REX.Wrxb
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRsp]
    REX.Wrxb
    sub eax, (5 * 8)
    SetVmcsField    GUEST_RSP, eax
    movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetSs]
    SetVmcsField    GUEST_SS_SELECTOR, eax
    shl eax, 5                ; SS.DPL
    or eax, 93h              ; P = S = W = A = 1
    SetVmcsField    GUEST_SS_ACCESS_RIGHTS, eax
    SetVmcsField    GUEST_SS_LIMIT, 0
    SetVmcsField    GUEST_SS_BASE, 0

;;
;; step 8: 压入返回信息到 stack 中
;;
    REX.Wrxb
    mov edx, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Rsp]
    movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldSs]
    REX.Wrxb
    mov [edx - 8], eax
    REX.Wrxb
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldRsp]
    REX.Wrxb
    mov [edx - 16], eax
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldFlags]
    REX.Wrxb
    mov [edx - 24], eax
    movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldCs]

```

```

    REX.Wrxb
    mov [edx - 32], eax
    REX.Wrxb
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldRip]
    REX.Wrxb
    mov [edx - 40], eax

    ;;
    ;; step 9: 加载 CS:RIP
    ;;
    movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCs]
    SetVmcsField    GUEST_CS_SELECTOR, eax
    movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc + 5]
    and eax, 0F0FFh
    SetVmcsField    GUEST_CS_ACCESS_RIGHTS, eax
    DoVmWrite       GUEST_CS_LIMIT, [ebp + PCB.GuestExitInfo +
INTERRUPT_INFO.TargetCsLimit]
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc + 2]
    and eax, 00FFFFFFh
    mov esi, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc + 4]
    and esi, 0FF000000h
    or eax, esi
    SetVmcsField    GUEST_CS_BASE, eax
    REX.Wrxb
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRip]
    SetVmcsField    GUEST_RIP, eax

    ;;
    ;; step 10: 更新 rflags
    ;;
    mov esi, ~(FLAGS_TF | FLAGS_VM | FLAGS_RF | FLAGS_NT)
    mov edi, ~(FLAGS_TF | FLAGS_VM | FLAGS_RF | FLAGS_NT | FLAGS_IF)
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldFlags]
    test DWORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc], (1 << 8)
    cmovz esi, edi
    and eax, esi
    SetVmcsField    GUEST_RFLAGS, eax
    mov eax, DO_INTERRUPT_SUCCESS
    jmp do_interrupt_for_inter_privilege_longmode.Done

do_interrupt_for_inter_privilege_longmode.Ts_TsSelector_01B:
    movzx eax, WORD [ebx + VMB.GuestTmb + GTMB.TssSelector]
    and eax, 0FFF8h
    or eax, 1
    mov ecx, INJECT_EXCEPTION_TS
    jmp do_interrupt_for_inter_privilege_longmode.ReflectException

do_interrupt_for_inter_privilege_longmode.Gp_01B:
    mov eax, 01
    mov ecx, INJECT_EXCEPTION_GP
    jmp do_interrupt_for_inter_privilege_longmode.ReflectException

do_interrupt_for_inter_privilege_longmode.Ss_01B:
    mov eax, 1
    mov ecx, INJECT_EXCEPTION_SS

```

```
do_interrupt_for_inter_privilege_longmode.ReflectException:
;;
;; 注入异常
;;
SetVmcsField    VMENTRY_EXCEPTION_ERROR_CODE, eax
SetVmcsField    VMENTRY_INTERRUPT_INFORMATION, ecx
mov eax, DO_INTERRUPT_ERROR

do_interrupt_for_inter_privilege_longmode.Done:
pop ecx
pop edx
pop ebx
pop ebp
ret
```

IA-32e 模式的切入高权限下, guest 完整的 INT 指令执行路线是: INT 60h → do_GP() → do_int_process() → do_interrupt_for_inter_privilege_longmode()。

2. 权限内的 delivery 处理

do_interrupt_for_intra_privilege_longmode 例程负责完成 IA-32e 模式下同级权限的中断 delivery 工作, 它的处理流程如下面的 C 伪码所示。

```
do_interrupt_for_intra_privilege_longmode()
{
    /*
     * step 1: 检查 IST, 如果 IST > 0 则使用 IST
     */
    if (IdtDesc.Ist != 0)
    {
        if ((IdtDesc.Ist * 8 + 28 + 7) > Tsslimit)
        {
            do_TS(ERROR_CODE(TssSelector, 0, EXT));
        }

        TargetRsp = read_stack_pointer_from_TSS(IdtDesc.Ist * 8 + 28);
    }
    else
    {
        TargetRsp = OldRsp;
    }

    /* step 2: 检查 TargetRsp 是否为 canonical 地址 */
    if (!IS_CANONICAL(TargetRsp))
    {
        do_SS(ERROR_CODE(0, 0, EXT));
    }

    /* step 3: 读取 RIP, 并检查 RIP 是否为 canonical 地址 */
    TargetRip = read_target_rip(IdtDesc);
    if (!IS_CANONICAL(TargetRip))
    {
        do_GP(ERROR_CODE(0, 0, EXT));
    }

    /* step 4: 压入返回信息 */
    *(TargetRsp - 1) = OldSs;
    *(TargetRsp - 2) = OldRsp;
}
```

```

*(TargetRsp - 3) = OldFlags;
*(TargetRsp - 4) = OldCs;
*(TargetRsp - 5) = OldRip;

/* step 5: 加载新的 RSP */
GUEST_RSP = TargetRsp - 40;

/* step 6: 加载 CS:RIP */
GUEST_RIP = TargetRip;
GUEST_CS = LoadCsSegment(TargetCs);           // 加载目标 CS

/* step 7: 更新 rflags */
TmpFlags = OldFlags;
TmpFlags.TF = 0;
TmpFlags.VM = 0;
TmpFlags.RF = 0;
TmpFlags.NT = 0;

if (IdtDesc.Type == INTERRUPT_GATE)
{
    TmpFlags.IF = 0;
}

GUEST_RFLAGS = TmpFlags;
}

```

do_interrupt_for_intra_privilege_longmode 例程主要进行下面的工作。

- (1) 根据中断描述符的 IST 值决定目标 RSP 值。
 - IST = 0 时，原 RSP 就是目标 RSP (TargetRsp)。
 - IST > 0 时，TargetRsp 从 TR.base + IST × 8 + 28 位置上读取。
- (2) 检查 TargetRsp 是否为 canonical 地址。不属于时产生 #SS 异常。
- (3) 读取 TargetRip，并检查是否为 canonical 地址。不属于时产生 #GP 异常。
- (4) 在 TargetRsp 里压入返回信息。
- (5) 为 guest 加载新的 RSP 值，等于 TargetRsp - 40。
- (6) 加载 guest 的 CS:RIP 为 TargetCs 与 TargetRip 值。
- (7) 更新 RFLAGS 寄存器。TF、VM、RF 以及 NT 位都被清 0。
 - 中断描述符属于 interrupt-gate 时，IF = 0。
 - 中断描述符属于 trap-gate 时，IF 位保持不变。

do_interrupt_for_intra_privilege_longmode 例程的处理流程相对简单得多，完整的实现代码如下所示。

代码片段 7-33:

```

;-----
; do_interrupt_for_intra_privilege_longmode()
; input:
;     none

```

```

; output:
;     eax - status code
; 描述:
;     1) 处理 longmode 模式下的特权级外的中断
;-----
do_interrupt_for_intra_privilege_longmode:
    push ebp
    push ebx
    push edx
    push ecx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    DEBUG_RECORD    "[do_interrupt_for_intra_privilege_longmode]..."

    REX.Wrb
    mov ebx, [ebp + PCB.CurrentVmbPointer]

    ;;
    ;; 当前 RSP
    ;;
    REX.Wrb
    mov esi, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldRsp]
    REX.Wrb
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRsp], esi

    ;;
    ;; step 1: 检查 IST
    ;;
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc + 4]
    and eax, 7
    jz do_interrupt_for_intra_privilege_longmode.CheckRsp
    lea eax, [eax * 8 + 28]
    lea esi, [eax + 7]
    cmp esi, [ebx + VMB.GuestTmb + GTMB.TssLimit]
    ja do_interrupt_for_intra_privilege_longmode.Ts_TsSelector_01B

    ;;
    ;; 读取 IST pointer
    ;;
    REX.Wrb
    add eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TssBase]
    REX.Wrb
    mov esi, [eax]
    REX.Wrb
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRsp], esi

do_interrupt_for_intra_privilege_longmode.CheckRsp:
    ;;
    ;; step 2: 检查 RSP 是否为 canonical 地址形式
    ;;
    REX.Wrb
    mov eax, esi
    REX.Wrb
    shl eax, 16

```

```

    REX.Wrxb
    sar eax, 16
    REX.Wrxb
    cmp eax, esi
    jne do_interrupt_for_intra_privilege_longmode.Ss_01B

    REX.Wrxb
    and esi, ~0Fh                                ;; new RSP &
FFFF_FFFF_FFFF_FFF0h
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRsp], esi
    call get_system_va_of_guest_os
    REX.Wrxb
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Rsp], eax        ;; 保存 RSP

    ;;
    ;; step 3: 读取 RIP, 并检查 RIP 是否为 canonical 地址
    ;;
    movzx esi, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc] ;
offset[15:0]
    mov edi, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc + 4]
    and edi, 0FFFF0000h                                ; offset[31:16]
    or esi, edi
    mov edi, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc + 8]
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRip], esi    ; 保存目标 RIP
    mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRip + 4], edi
    shl edi, 16
    sar edi, 16
    cmp edi, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRip + 4]
    jne do_interrupt_for_intra_privilege_longmode.Gp_01B

    ;;
    ;; step 4: 压入返回信息
    ;;
    REX.Wrxb
    mov edx, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.Rsp]
    movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldSs]
    REX.Wrxb
    mov [edx - 8], eax
    REX.Wrxb
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldRsp]
    REX.Wrxb
    mov [edx - 16], eax
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldFlags]
    REX.Wrxb
    mov [edx - 24], eax
    movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldCs]
    REX.Wrxb
    mov [edx - 32], eax
    REX.Wrxb
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldRip]
    REX.Wrxb
    mov [edx - 40], eax

    ;;
    ;; step 5: 加载新的 RSP 值
    ;;
    REX.Wrxb
    mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRsp]
    REX.Wrxb
    sub eax, 40

```

```

SetVmcsField    GUEST_RSP, eax

;;
;; step 6: 加载 CS:RIP
;;
movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCs]
SetVmcsField    GUEST_CS_SELECTOR, eax
movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc + 5]
and eax, 0F0FFh
SetVmcsField    GUEST_CS_ACCESS_RIGHTS, eax
mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc + 2]
and eax, 0FFFFFFh
mov esi, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc + 4]
and esi, 0FF00000h
or eax, esi
SetVmcsField    GUEST_CS_BASE, eax
movzx eax, WORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc]    ;
limit[15:0]
mov esi, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc + 4]
and esi, 0F0000h
or eax, esi    ; limit[19:0]
test DWORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsDesc + 4], (1 <<
23)
jz do_interrupt_for_intra_privilege_longmode.SetCsLimit
shl eax, 12
add eax, 0FFFh
do_interrupt_for_intra_privilege_longmode.SetCsLimit:
mov [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetCsLimit], eax
SetVmcsField    GUEST_CS_LIMIT, eax
REX.Wrb
mov eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.TargetRip]
SetVmcsField    GUEST_RIP, eax

;;
;; step 7: 更新 rflags
;;
mov eax, ~(FLAGS_TF | FLAGS_NT | FLAGS_VM | FLAGS_RF)
mov esi, ~(FLAGS_TF | FLAGS_NT | FLAGS_VM | FLAGS_RF | FLAGS_IF)
test DWORD [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.IdtDesc + 4], (1 << 8)
cmovz eax, esi
and eax, [ebp + PCB.GuestExitInfo + INTERRUPT_INFO.OldFlags]
SetVmcsField    GUEST_RFLAGS, eax
mov eax, DO_INTERRUPT_SUCCESS
jmp do_interrupt_for_intra_privilege_longmode.Done

do_interrupt_for_intra_privilege_longmode.Gp_01B:
mov eax, 01
mov ecx, INJECT_EXCEPTION_GP
jmp do_interrupt_for_intra_privilege_longmode.ReflectException

do_interrupt_for_intra_privilege_longmode.Ts_TsSelector_01B:
movzx eax, WORD [ebx + VMB.GuestTmb + GTMB.TssSelector]
or eax, 01
mov ecx, INJECT_EXCEPTION_TS
jmp do_interrupt_for_intra_privilege_longmode.ReflectException

do_interrupt_for_intra_privilege_longmode.Ss_01B:
mov eax, 01

```



```

        mov ecx, INJECT_EXCEPTION_SS

do_interrupt_for_intra_privilege_longmode.ReflectException:
    ;;
    ;; 注入异常
    ;;
    SetVmcsField    VMENTRY_EXCEPTION_ERROR_CODE, eax
    SetVmcsField    VMENTRY_INTERRUPT_INFORMATION, ecx
    mov eax, DO_INTERRUPT_ERROR

do_interrupt_for_intra_privilege_longmode.Done:
    pop ecx
    pop edx
    pop ebx
    pop ebp
    ret

```

IA-32e 模式的同级权限下，guest 完整的 INT 指令执行路线是：INT 60h → do_GP() → do_int_process() → do_interrupt_for_intra_privilege_longmode()。

7.3.1.5 例子 7-4

示例 7-4：拦截 INT 指令，并帮助 guest 完成中断 delivery

经过前面 7.3.1.1 节到 7.3.1.4 节的篇幅介绍，现在我们来完成例子 7-4，实现 VMM 拦截 INT 指令，并且帮助 guest 完成中断的 delivery。我们拦截 INT 指令不是为了实现模拟处理器的 delivery 工作，而是为了可以监控 guest 执行 INT 指令，从而让 VMM 可以做一些额外的工作。但最终 VMM 的方式来模拟处理器的 delivery 操作是必需的。

例子 7-4 以 guest 执行 INT 60h 指令为例，实现在 0 级和 3 级权限进行中断调用。

1. guest 端代码

guest_ex.asm 模块要做的工作是：添加一个 60h 号的中断例程，并且在 guest-IDT 里设置中断描述符。

代码片段 7-34：

```

    ;;
    ;; 设置 60h 中断 handler
    ;;
    sidt [Guest.IdtPointer]
    mov ebx, [Guest.IdtPointer + 2]

%if __BITS__ == 64
    add ebx, (60h * 16)
    mov rsi, (0FFFF800080000000h + GuestEx.Int60Handler)
    mov [rbx + 4], rsi
    mov [rbx], si
    mov WORD [rbx + 2], GuestKernelCs64
    mov WORD [rbx + 4], 0EE01h ; DPL = 3, IST = 1
    mov DWORD [rbx + 12], 0
%else
    add ebx, (60h * 8)
    mov esi, (80000000h + GuestEx.Int60Handler)
    mov [ebx + 4], esi
    mov [ebx], si

```

```

        mov WORD [ebx + 2], GuestKernelCs32
        mov WORD [ebx + 4], 0EE00h                ; DPL = 3
%endif
        call PrintLn

        int 60h
        mov esi, GuestEx.Msg1
        call PutStr

        ;;
        ;; 进入 user 权限
        ;;
%if __BITS__ == 64
        push GuestUserSs64 | 3
        push 2FFF0h
        push GuestUserCs64 | 3
        push GuestEx.UserEntry
        retf64
%else
        push GuestUserSs32 | 3
        push 2FFF0h
        push GuestUserCs32 | 3
        push GuestEx.UserEntry
        retf
%endif

GuestEx.UserEntry:
        mov ax, GuestUserSs32
        mov ds, ax
        mov es, ax
        mov esi, GuestEx.Msg2
        call PutStr
        int 60h
        mov esi, GuestEx.Msg1
        call PutStr
        jmp $

;;
;; guest 的 60h 中断例程
;;
GuestEx.Int60Handler:
        mov esi, GuestEx.Msg0
        call PutStr
%if __BITS__ == 64
        iretq
%else
        iret
%endif

GuestEx.Msg0    db    'guest Int 60h !', 10, 0
GuestEx.Msg1    db    'interrupt end!', 10, 0
GuestEx.Msg2    db    'enter User !', 10, 0

```

guest_ex.asm 模块执行下面的工作。

(1) 在 IDT 里设置向量 60h 的中断描述符。

(2) 在 0 级权限里执行 INT 60h 指令进行中断调用。

(3) 切换到 3 级权限。

(4) 在 3 级权限里执行 INT 60h 指令。

作为示例，60h 中断服务例程只是简单地打印出一条信息 “guest Int 60h !”。注意，在 IA-32e 模式下，为了方便，我们的中断描述符使用了 IST (Interrupt Stack Table) 指针，设置的 IST 值为 1。并且，描述符的 DPL 值设为 3，这样可以在 3 级权限下调用中断。

2. 编译及运行

我们可以使用下面的三个命令进行编译及生成目标映像文件：

- build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64 -DGUEST_X64
- build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64
- build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE

第 1 条命令将 host 与 guest 编译为 64 位。第 2 条命令将 host 编译为 64 位，guest 编译为 32 位。第 3 条命令将 host 与 guest 都编译为 32 位。同时，必须要开启 GUEST_ENABLE 符号。

我们在 VMware 里运行后，在 CPU0 的命令选择器上按**数字<1>**键，向 CPU1 发起 VM-entry 操作切入到 guest 运行。然后按<F2>键切换到 CPU1 屏幕，再按一次<F2>键切换到 CPU1 的 guest 屏幕。

如图 7-10 所示的 guest 端打印了两次 “guest Int 60h” 信息，一次在 0 级权限里调用中断 60h，另一次是在 3 级权限里调用中断 60h。



```

[OS]: start ...
[OS]: initialize done ...
[OS]: running ...
[OS]: TSC = 000000000004CB4D

guest Int 60h ! ← 在 0 级权限里执行 INT 60h
interrupt end!
enter User !
guest Int 60h ! ← 在 3 级权限里执行 INT 60h
interrupt end!
  
```

图 7-10

现在，我们按下<F4>键切换到 CPU3，再按下<Enter>键切换到 msg-list 列表显示信息屏幕。按<向下箭头>键翻到我们所需要的记录，如图 7-11 所示。

```

[CPUIndex]:3 [Stage]:3 [CPUs]:4 [CpuModel]:0625 [VMX]:Enable [Ept]:Disable

===== msg list record =====
<#62> [CPU1] [Exit Reason]: exception or NMI
<#63> [CPU1] [do_GP]...
<#64> [CPU1] process INT instruction
<#65> [CPU1] [do_int_process]...
<#66> [CPU1] [do_int_process.Longmode]...
<#67> [CPU1] [do_interrupt_for_intra_privilege_longmode]...
<#68> [CPU1] [VMX]: resume to guest !
<#69> [CPU1] [Exit Reason]: EPT violation
<#70> [CPU1] [DoEptViolation]: remapping! (eax = HPA, ebx = GPA)
<#71> [CPU1] [VMX]: resume to guest !
<#72> [CPU1] [Exit Reason]: exception or NMI
<#73> [CPU1] [do_GP]...
<#74> [CPU1] process INT instruction
<#75> [CPU1] [do_int_process]...
<#76> [CPU1] [do_int_process.Longmode]...
<#77> [CPU1] [do_interrupt_for_inter_privilege_longmode]...
<#78> [CPU1] [VMX]: resume to guest !
<#79> [CPU1] [Exit Reason]: exception or NMI
<#80> [CPU1] [do_NMI]: call NMI handler !
<#81> [CPU1] [VMX]: resume to guest !

```

图 7-11

图 7-11 中所显示的是 guest 在 IA-32e 模式下，记录的 VMM 进行中断处理流程的信息。VMM 进行了两次中断的 delivery 处理。第 1 次最终调用 do_interrupt_for_intra_privilege_longmode 例程来处理，第 2 次最终调用 do_interrupt_for_inter_privilege_longmode 例程来处理。它们的调用路线是：

- (1) guest 执行 INT 60 指令。
- (2) 产生#GP 异常。
- (3) VMM 拦截#GP 异常，进入 do_GP 例程进行处理。
- (4) VMM 检查到原始事件属于软件中断，则调用 do_int_process 例程进入中断处理。
- (5) VMM 检查到 guest 处于 IA-32e 模式，则进入 do_int_process.Longmode 进行处理。
- (6) VMM 根据执行 INT 60h 指令时的权限及目标代码权限进行处理：
 - 属于同级调用，则进入 do_interrupt_for_intra_privilege_longmode 例程处理。
 - 属于高权限调用，则进入 do_interrupt_for_inter_privilege_longmode 例程处理。

VMM 可以在进入 do_int_process 例程进行中断的 delivery 操作前，进行一些相应的动作，从而达到监控 guest 执行 INT 指令的某种目的，这是我们所期望的。

经过 VMM 的处理，**guest** 并不知道其中发生了什么事情，**guest** 得到了它所想要的结果（也就是中断服务例程得到正常的执行）。

注意：在两次中断调用之间发生了一次 EPT violation 故障，那是因为 User 代码使用的 RSP 值为 2FFF0h，这个值并没有被 EPT 映射，从而 VMM 额外做了一次 EPT violation 故障修复工作。

就本例来说，VMM 监控 INT 指令能不能被 guest 检测出来？答案是可能会，也可能不会！一个非简聪明的 **guest** 端，软件可能会根据检测“为什么 INT 指令需要这么长时

间完成”而推测可能处于 guest 中。但是，VMM 也可以拦截 guest 读取 TSC 值来隐瞒真实的指令执行时间，而令 guest 无法推测出来。

7.3.2 处理 NMI

处理器可以从下面的途径接收到 NMI 请求：

- (1) 来自芯片组的 NMI。
- (2) 来自 I/O APIC 以 NMI delivery 模式发送的中断消息。
- (3) 来自处理器 local APIC 以 NMI delivery 模式发送的 IPI 消息。

芯片组上的 NMI 由 local APIC 的 LINT1 接口传递给 local APIC，来自 I/O APIC 与 local APIC 的中断消息直接由 local APIC 接收。NMI 不需要经过 local APIC 的仲裁直接提交到处理器执行，因此，不能通过 local APIC 的 TPR 以及 eflags.IF 标志位进行屏蔽。

但是，NMI 能被相关的 interruptibility state 以及 activity state 阻塞。详见 4.16 节与 4.17 节所述。

7.3.2.1 拦截 NMI

VMM 可以通过设置 pin-based VM-execution control 字段的“NMI exiting”位为 1，来拦截 guest 的 NMI。在产生 VM-exit 后回到 host 环境里，存在“blocking by NMI”阻塞状态（参见 5.10 节）。处理器期待执行一个 IRET 指令来解除阻塞状态（参见 4.16.2 节），或者通过 VMRESUME 时加载 interruptibility state 字段来解除。

本书的所有例子中，按功能键<F1>、<F2>、<F3>、<F4>等，通过向处理器以 NMI 发送 IPI 消息的方式实现。因此，当按下<F4>键时，由 CPU0 产生键盘中断，然后调用 force_dispatch_to_processor 例程来发送 NMI 给目标的处理器。

force_dispatch_to_processor 的 C 语言原型看起来如下所示：

```
VOID
force_dispatch_to_processor(
    UINT    ProcessorIndex,
    PVOID    RoutineEntry
);
```

force_dispatch_to_processor 例程实现在 lib\smp.asm 文件里，由于 NMI 不受 eflags.IF 标志位影响，因此，不管 ProcessorIndex 指示的目标处理器是否关闭中断允许，都可以执行我们派送的任务（RoutineEntry）。

VMM 在接管 NMI 后，有两种处理方式：

- (1) 在 host 端直接执行 INT 02h 指令调用 NMI 处理例程。
- (2) VMM 将 NMI 反射回 guest 处理。

它们的区别是：一种在 host 端处理 NMI，另一种在 guest 端处理 NMI。在实际应用

中应该需要反射回 **guest** 处理！鉴于本书例子的特殊性，选择使用第 1 种方式处理（在 host 端处理）。

代码片段 7-35:

```

;-----
; do_NMI()
; input:
;     none
; output:
;     eax - VMM 处理码
; 描述:
;     1) 处理由 NMI 引发的 VM-exit
;-----
do_NMI:
    push ebp
#ifdef __X64
    LoadFsBaseToRbp
#else
    mov ebp, [fs: SDA.Base]
#endif

    DEBUG_RECORD    "[do_NMI]: call NMI handler !"

    ;;
    ;; 下面采用主动调用 NMI handler 方式在 VMM 内完成 NMI
    ;;
    int NMI_VECTOR

%if 0
    ;;
    ;; 下面注入 NMI 让 guest 完成
    ;; 1) 将这种方式更改为 VMM 完成
    ;;
    DEBUG_RECORD    "[do_NMI]: inject a NMI event !"

    ;;
    ;; 注入 NMI 事件
    ;;
    SetVmcsField    VMENTRY_INTERRUPTION_INFORMATION, INJECT_NMI
    SetVmcsField    VMENTRY_INSTRUCTION_LENGTH, 0
%endif

    mov eax, VMM_PROCESS_RESUME
    pop ebp
    ret

```

本书例子中，将 `switch_to_processor` 例程作为 `RoutineEntry` 传递给 `force_dispatch_to_processor` 来完成处理器的切换。由于 `switch_to_processor` 例程必须在 host 端处理，不能受目标处理器运行的 guest 环境影响（目标 guest 不支持），所以，在收到 NMI 而产生 VM-exit 时，VMM 选择在 host 端直接执行 `INT 02h` 指令完成 `switch_to_processor` 例程的工作。

如图 7-12 所示，我们按了两次<F2>键切换到 CPU1 的 guest 屏幕后，产生了两次 VM-exit，然后在 VMM 里调用了两次 NMI handler 完成处理器的切换工作。

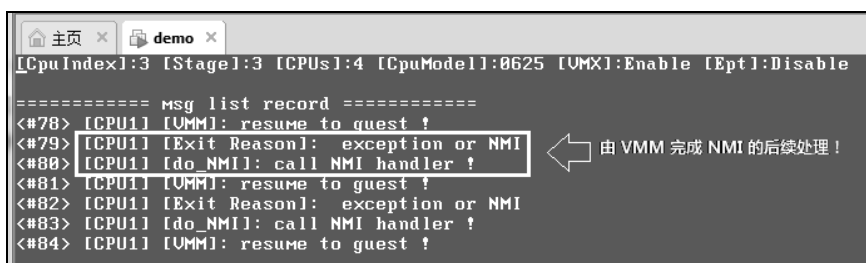


图 7-12

7.3.2.2 虚拟 NMI

虚拟 NMI 必须建立在“NMI exiting”位为 1 的基础上，这一点与虚拟中断（virtual-interrupt）原理相似，虚拟中断需要建立在“external-interrupt exiting”位为 1 的基础上。

虚拟 NMI 产生的前提是“virtual-NMIs”位为 1，可以通过事件注入方式进行 deliver。而虚拟中断的 delivery 前提是设置 secondary processor-based VM-execution control 字段的“virtual-interrupt delivery”位为 1（参见 7.2.13 节所述）。

7.3.3 处理外部中断

对于外部中断，处理器可以接收从中断控制器（8259、IO APIC 或者 local APIC）来的中断请求，也可以接收来自处理器间以 Fixed delivery 模式发送的 IPI 消息。

7.3.3.1 拦截外部中断

VMM 设置 pin-based VM-execution control 字段的“external-interrupt exiting”位为 1，当处理器收到**未被屏蔽及阻塞**的中断请求时，将直接产生 VM-exit。由外部中断产生的 VM-exit **不能被 eflags.IF 标志位屏蔽**，但可以被 local APIC 的 TPR 屏蔽。

在拦截外部中断产生 VM-exit 后，取决于 VM-exit control 字段“acknowledge interrupt on exit”位的设置，VMM 有不同的应对处理。

- “acknowledge interrupt on exit”为 1 时，在 VM-exit 时处理器响应中断控制器，从中断控制器里取出外部中断的信息，记录在 VM-exit interruption information 字段里（参见 3.10.2.1 节）。
- “acknowledge interrupt on exit”为 0 时，外部中断在中断控制器 **IRR**（interrupt request register）里的**请求位保持有效**，中断被悬挂着，VM-exit interruption information 字段属于无效。VMM 执行 STI 指令后重新打开中断时，处理器才响应中断控制器取得中断向量号。

当 VMM 想夺取对外部中断 delivery 的控制权时，需要设置“acknowledge interrupt on exit”为 0，发生 VM-exit 后，VMM 在重新打开中断许可时（执行 STI 或 POPF 指令），处理器响应中断控制器并通过 **host-IDT** 进行外部中断的 delivery 操作。

VMM 需要反射外部中断给 guest 处理时，则需要设置“acknowledge interrupt on

exit”为 1。产生 VM-exit 后，VMM 将 VM-exit interruption information 字段的值复制到 VM-entry interruption information 字段后，执行 VMRESUME 指令回到 guest 运行，外部中断通过 **guest-IDT** 进行 delivery 操作。

7.3.3.2 转发外部中断

在一个有多个逻辑处理器的平台里，8259 中断控制器的 INTR 输出端接在 CPU0 的 LINT0 接口里，其余逻辑处理器的 LINT0 空闲着。但 I/O APIC 的中断消息可以发送到某一个处理器，或者某一组处理器。

下面我们以 8259 中断控制器发送中断请求给 CPU0 为例，讲解 VMM 转发外部中断的处理过程。假设处理器拥有 4 个逻辑处理器，当用户按下键盘后 8259 的 IRQ1 产生中断请求发给 CPU0，VMM 需要判断转发给哪个 CPU 执行。如图 7-13 所示。

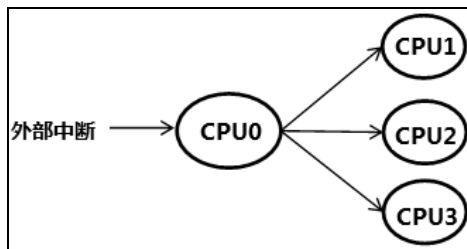


图 7-13

如果当前是在 CPU1 的 guest 运行中用户按下了键盘，那么 VMM 应该将外部中断转发给 CPU1 处理。显然，只有在 **guest 运行中产生了中断 VMM 才需要转发**。当 CPU1 并没有 guest 正在运行，那么就不需要将外部中断转发给 CPU1。

另一个情况是，当 CPU0 的 guest 运行中产生了中断，CPU0 的 VMM 就直接拦截了。此时不需要转发给其他的 CPU 处理。

```

keyboard_handler()
{
    ... ..

    if (processor = get_processor_for_interrupt_dispatch())
    {
        /*
         * 有处理器需要转发中断，则转发给目标处理器
         */
        send_ipi_to_processor(processor, vector);    // 向目标处理器发送 IPI
    }

    ... ..
}
  
```

如上面的伪代码所示，在 CPU0 的中断例程里需要检查是否有处理器需要转发中断。如果有则取出中断向量号，以 Fixed delivery 模式发送 IPI 到目标处理器。

目标处理器收到 IPI 后，由于“external-interrupt exiting”为 1 而产生 VM-exit。因此，目标处理器的 VMM 拦截了外部中断，然后反射给 guest 处理。

当然，如果目标处理器 VMCS 的“external-interrupt exiting”为 0，则直接由 guest 接收到 CPU0 发过来的中断，从而直接在 guest-IDT 里 deliver 执行。

如果使用 I/O APIC 作为中断控制器，VMM 则可以设置物理 I/O APIC 的发送目标，从而达到转发中断目的。取决于 VMM 的设计模型，可以让 VMM 与 guest 强制使用相同的某类中断控制器（8259 或者 I/O APIC），或者 VMM 与 guest 使用不同类型的中断控制器。

注意：中断转发需要满足下面三个条件。

- (1) 目标处理器处于焦点状态，这个条件由 SDA.InFocus 来控制。
- (2) 目标处理器的 guest 正在运行状态，这个状态由 PCB.ProcessorStatus 记录。
- (3) 目标处理器的 guest 拥有焦点状态，这个状态由 PCB.ProcessorStatus 记录。

下面是来自 lib\pic8259A.asm 文件的 keyboard_8259_handler 例程部分代码，它执行上述的条件检查并发送 IPI 消息。

代码片段 7-36:

```
;;
;; 检查目标处理器是否拥有焦点
;;
cmp edx, [ebx + EXTINT_RTE.ProcessorIndex]
jne keyboard_8259_handler.SendExtIntIpi.Next

mov esi, edx
call get_processor_pcb
REX.Wrb
test eax, eax
jz keyboard_8259_handler.SendExtIntIpi.Next

;;
;; 检查目标处理器是否处于 guest，并且拥有焦点
;;
mov esi, [eax + PCB.ProcessorStatus]
xor esi, CPU_STATUS_GUEST | CPU_STATUS_GUEST_FOCUS
test esi, CPU_STATUS_GUEST | CPU_STATUS_GUEST_FOCUS
jnz keyboard_8259_handler.SendExtIntIpi.Next

;;
;; 发送外部中断到目标处理器
;;
mov esi, [eax + PCB.ApicId]
movzx edi, BYTE [ebx + EXTINT_RTE.Vector]          ; 8259 的 IRQ0 vector
incv edi                                           ; 得到键盘中断 IRQ1 vector
or edi, FIXED_DELIVERY | PHYSICAL
SEND_IPI_TO_PROCESSOR esi, edi
```

CPU_STATUS_GUEST 位为 1 时，表明 guest 正在运行中。CPU_STATUS_GUEST_FOCUS 为 1 时，表明 guest 拥有焦点。通过后 keyboard_8259_handler 例程使用 SEND_IPI_TO_PROCESSOR 宏发送中断消息给目标处理器。

7.3.3.3 监控 guest 设置 8259

为了能在 VMware 里运行，本书的例子以 **8259 作为中断控制器** 为例。在 guest OS 的运行过程中，少不了会进行中断控制器的初始化工作。那么，VMM 需要监控 guest 对中断控制器的设置。

以 8259 为例，VMM 至少需要拦截 guest 访问 20h、21h、A0h 以及 A1h 这 4 个 I/O 端口，它们代表着 8259 的 master 片与 slave 片。不能让 guest 的设置影响到物理平台上的 8259 中断控制器。

处理 8259 初始化流程

在设计完善的虚拟化平台里，VMM 需要非常清楚 8259 的工作原理，一些细节也不能有误。但作为示例，**本书例子只是进行了一些简单的处理**，敬请读者留意。

按照 8259 的初始化工作原理，它的初始化流程大致为下面的伪码所示。

```
init_8259()
{
    ... ..

    if (write_ICW1(0x11))
    {
        /*
         * 向 20h 端口写入 11h 值，被认为是写入 ICW1 字
         * 假如写入了 ICW1 字，则认定处于 8259 的初始化流程里
         */

        write_ICW2(vector);           // 后续写入 ICW2
        write_ICW3(...);             // 写入 ICW3
        write_ICW4(...);             // 写入 ICW4
    }

    ... ..
}
```

当向 20h 端口写入的值为 11h 时，则认为写入 ICW1 字，那么 guest 正处于 8259 的初始化当中。8259 期望后续向 21h 端口分别写入 ICW2、ICW3 及 ICW4 字（这几个初始化字使用同一个端口）。

VMM 必须根据这个原理来监控 guest 初始化 8259。下面的伪代码是当 guest 尝试执行 I/O 指令被拦截后，VMM 做出的粗略处理。

```
do_guest_io_process()
{
    ... ..

    if (IoOperation == IO_FLAGS_IN)
    {
        /*
         * 处理 IN/INS 指令
         */
        ... ..
    }
    else
    {
```

```

/*
 * 处理 OUT/OUTS 指令
 */
if (IoFlags == IO_FLAGS_STRING)
{
    /* 处理串指令 */
    ... ..
}
else
{
    AppendIoVte(IoPort, value);           // 保存 guest 写入值

    if (IoPort == 0x20)
    {
        if (value == 0x11)
        {
            /*
             * guest 处于 8259 初始化流程里
             */
            IoOperationFlags |= IOP_FLAGS_8259_MASTER_INIT;
        }
        else if (value == 0x20)
        {
            /*
             * guest 发送 EOI 到 8259
             */
            send_eoi_to_lapic();           // 发送 EOI 到 local APIC
        }
    }
    else if (IoPort == 0x21)
    {
        if (IoOperationFlags & IOP_FLAGS_8259_MASTER_INIT)
        {
            /*
             * guest 写入 ICW2 字
             */
            AppendExtIntRte(value);        // 将 value 写入 RTE 表项
            IoOperationFlags &= ~IOP_FLAGS_8259_MASTER_INIT;
        }
    }
}
... ..
}

```

VMM 需要区分 guest 操作属于读 I/O 端口 (IN/INS)，还是写 I/O 端口 (OUT/OUTS)。在写端口处理里：

- (1) 检查写入端口是否为 20h。端口为 20h 时，检查写入值是否为 11h。
 - 是，则标记 guest 处于 8259 初始化流程 (IOP_FLAGS_8259_MASTER_INIT)。
 - 否，则检查写入值是否为 20h (EOI 命令)，写入值为 20h 则由 VMM 向 local APIC 发送 EOI 命令，驱散外部中断的 In-service 位。
- (2) 写入端口不是 20h，而是 21h 时，则进行下面的处理。
 - 如果检查到存在 IOP_FLAGS_8259_MASTER_INIT 标记，则表明 guest 正在写

ICW2 字。VMM 调用 AppendExtIntRte 例程添加外部中断的 route 表项，用于转发外部中断。然后清 8259 初始化标志位。

- 否则忽略。

下面是 do_guest_io_process 例程的代码，实现在 lib\Vmx\VmxIo.asm 文件里。

代码片段 7-37:

```

;-----
; do_guest_io_process()
; input:
;     none
; output:
;     eax - status code
; 描述:
;     1) 进行 guest I/O 指令的相应处理
;-----
do_guest_io_process:
    push ebp
    push ebx
    push ecx
    push edx

#ifdef __X64
    LoadGsBaseToRbp
#else
    mov ebp, [gs: PCB.Base]
#endif

    REX.Wrb
    mov ebx, [ebp + PCB.CurrentVmbPointer]

    cmp DWORD [ebp + PCB.LastStatusCode], STATUS_GUEST_PAGING_ERROR
    je do_guest_io_process.Pf

    mov edx, [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.IoFlags]
    test edx, IO_FLAGS_IN
    jnz do_guest_io_process.In

    ;;
    ;; 处理 OUT/OUTS
    ;;
do_guest_io_process.Out:
    DEBUG_RECORD    "processing OUT instruciton ..."

    ;;
    ;; #### 作为示例，保留实现串指令的处理 ####
    ;;
    test edx, IO_FLAGS_STRING
    jnz do_guest_io_process.Done

    ;;
    ;; 将 guest 尝试写 I/O 寄存器的值保存在 IO-VTE 里
    ;;
    mov ecx, [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.IoPort]
    mov esi, ecx
    mov edi, [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.Value]
    call AppendIovte

```

```

;;
;; 检查 guest 是否处于 8259 初始化工作流程!
;; 1) 检查是否写 MASTER_ICW1_PORT 端口
;; a) 是, 则检查下一个是否为 MASTER_ICW2_PORT 端口
;; b) 否, 则忽略
;;

;;
;; 检查是否为 20h 端口
;;
cmp ecx, 20h
jne do_guest_io_process.Out.@1

movzx eax, BYTE [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.Value]
;;
;; 检查写入值:
;; 1) bit 4 = 1 时: 写入 ICW 字
;; 2) bit 5 = 1 时, 写入 EOI 字
;;
test eax, (1 << 4)
jnz do_guest_io_process.Out.20h.ICW1
test eax, (1 << 5)
jz do_guest_io_process.Done

;;
;; 属于 EOI 命令, 则由 VMM 向 local APIC 写入 EOI
;;
LAPIC_EOI_COMMAND
jmp do_guest_io_process.Done

do_guest_io_process.Out.20h.ICW1:
;;
;; 设置 8259 MASTER 初始化标志位
;;
or DWORD [ebx + VMB.IoOperationFlags], IOP_FLAGS_8259_MASTER_INIT
jmp do_guest_io_process.Done

do_guest_io_process.Out.@1:
;;
;; 检查接下来是否写 MASTER_ICW2_PORT 端口
;;
cmp ecx, MASTER_ICW2_PORT
jne do_guest_io_process.Done
test DWORD [ebx + VMB.IoOperationFlags], IOP_FLAGS_8259_MASTER_INIT
jz do_guest_io_process.Done

;;
;; 清 8259 MASTER 初始化标志位
;;
and DWORD [ebx + VMB.IoOperationFlags], ~IOP_FLAGS_8259_MASTER_INIT

;;
;; 将 vector 添加到 ExtIntRte 表项里
;;
mov esi, [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.Value]
call AppendExtIntRte
jmp do_guest_io_process.Done

```

```

do_guest_io_process.In:
    DEBUG_RECORD    "processing IN instruction ..."

    ;;
    ;; 处理 IN/INS
    ;;
    mov esi, [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.IoPort]
    call GetIoVte
    REX.Wrxb
    test eax, eax
    jz do_guest_io_process.Done

    mov ecx, [eax + IO_VTE.Value]                ; I/O port 原值
    REX.Wrxb
    mov ebx, [ebx + VMB.VsbBase]                ; VSB 区域

    ;;
    ;; 检查是否属于串指令
    ;;
    test edx, IO_FLAGS_STRING
    jnz do_guest_io_process.In.String

    ;;
    ;; 处理 IN al/ax/eax, IoPort
    ;;
    mov esi, [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.OperandSize]
    cmp esi, IO_OPS_BYTE
    je do_guest_io_process.In.Byte
    cmp esi, IO_OPS_WORD
    je do_guest_io_process.In.Word

    ;;
    ;; 写入 dword 值
    ;;
    REX.Wrxb
    mov [ebx + VSB.Rax], ecx
    jmp do_guest_io_process.Done

do_guest_io_process.In.Byte:
    ;;
    ;; 写入 byte 值
    ;;
    mov [ebx + VSB.Rax], cl
    jmp do_guest_io_process.Done

do_guest_io_process.In.Word:
    ;;
    ;; 写入 word 值
    ;;
    mov [ebx + VSB.Rax], cx
    jmp do_guest_io_process.Done

do_guest_io_process.In.String:
    ;;
    ;; ##### 作为示例，保留实现对串指令的处理 #####
    ;;
    jmp do_guest_io_process.Done

```

```

%if 0
    ;;
    ;; 处理 INS 指令
    ;;
    REX.Wrxb
    mov edi, [[ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.LinearAddress]

    test DWORD [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.Flags], IO_FLAGS_REP
    jz do_guest_io_process.In.String.@1
    mov ecx, [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.Count]

do_guest_io_process.In.String.@1:

%endif

do_guest_io_process.Pf:
    mov ecx, INJECT_EXCEPTION_PF
    mov eax, 2
    test DWORD [ebp + PCB.GuestExitInfo + IO_INSTRUCTION_INFO.IoFlags],
IO_FLAGS_IN
    jz do_guest_io_process.ReflectException
    mov eax, 0

do_guest_io_process.ReflectException:
    SetVmcsField    VMENTRY_EXCEPTION_ERROR_CODE, eax
    SetVmcsField    VMENTRY_INTERRUPT_INFORMATION, ecx

do_guest_io_process.Done:
    pop edx
    pop ecx
    pop ebx
    pop ebp
    ret

```

这个 `do_guest_io_process` 例程实现得比较粗糙，有许多细节未处理。这里作为示例，展现了 VMM 如何处理 guest 初始化 8259A 工作。

有一点比较重要的是：当 guest 发送 EOI 命令给 8259A 完成中断例程时，需要由 VMM 主动向 local APIC 发送一个 EOI 命令。这是因为：CPU0 以 IPI 的形式向目标处理器转发外部中断。这种情况下，必须向 local APIC 发送 EOI 命令（而不是 8259A），向 8259A 发送 EOI 属于 CPU0 的职责。

7.3.3.4 例子 7-5

示例 7-5：实现外部中断转发，处理 CPU1 guest 的键盘中断

在例子 7-4 的代码基础上，我们扩展一下实现例子 7-5。这个例子需要实现外部中断的转发，用来处理来自 CPU1 guest 的键盘中断。

1. guest 端代码

在 `guest_ex.asm` 模块里，我们首先对 8259A 进行初始化设置，然后建立 8259A 的 IRQ1 中断例程。

代码片段 7-38:

```

;;
;; 初始化 8259
;;
cli
call init_pic8259
call disable_8259
    call enable_8259_keyboard
sti

;;
;; 设置 IRQ1 中断 handler
;;
sidt [Guest.IdtPointer]
mov ebx, [Guest.IdtPointer + 2]

%if __BITS__ == 64
    add ebx, (GUEST_IRQ1_VECTOR * 16)
    mov rsi, (0FFFF800080000000h + GuestKeyboardHandler)
    mov [rbx + 4], rsi
    mov [rbx], si
    mov WORD [rbx + 2], GuestKernelCs64
    mov WORD [rbx + 4], 08E01h                ; DPL = 0, IST = 1
    mov DWORD [rbx + 12], 0
%else
    add ebx, (GUEST_IRQ1_VECTOR * 8)
    mov esi, (80000000h + GuestKeyboardHandler)
    mov [ebx + 4], esi
    mov [ebx], si
    mov WORD [ebx + 2], GuestKernelCs32
    mov WORD [ebx + 4], 08E00h                ; DPL = 0
%endif

    mov esi, GuestEx.Msg0
    call PutStr

GuestEx.Loop:
    call waitKey
    movzx esi, BYTE [KeyMap + R0]
    call PutChar
    jmp GuestEx.Loop

    jmp $

;;
;; 进入 user 权限
;;
%if __BITS__ == 64
    push GuestUserSs64 | 3
    push 2FFF0h
    push GuestUserCs64 | 3
    push GuestEx.UserEntry
    retf64
%else
    push GuestUserSs32 | 3
    push 2FFF0h
    push GuestUserCs32 | 3

```



```

        push GuestEx.UserEntry
        retf
%endif

GuestEx.UserEntry:
    mov ax, GuestUsersSs32
    mov ds, ax
    mov es, ax
    jmp $

;-----
; GuestKeyboardHandler:
; 描述:
;     使用于 8259 IRQ1 handler
;-----
GuestKeyboardHandler:
    push R3
    push R6
    push R7
    push R0

    in al, I8408_DATA_PORT                ; 读键盘扫描码
    test al, al
    js GuestKeyboardHandler.done          ; 为 break code

    mov ebx, KeyBufferPtr
    mov esi, [ebx]
    inc esi

    ;;
    ;; 检查是否超过缓冲区长度
    ;;
    mov edi, KeyBuffer
    cmp esi, KeyBuffer + 255
    cmovae esi, edi
    mov [esi], al                          ; 写入扫描码
    mov [ebx], esi                        ; 更新缓冲区指针

GuestKeyboardHandler.done:
mov al, 00100000B                        ; OCW2 select, EOI
    out MASTER_OCW2_PORT, al
    pop R0
    pop R7
    pop R6
    pop R3
%if __BITS__ == 64
    iretq
%else
    iret
%endif

GuestEx.Msg0    db    'wait any keys: ', 0

```

guest 的工作主要是：

- (1) 初始化 8259A 后，屏蔽其他 IRQ，但打开 IRQ1。
- (2) 设置 IRQ1 对应的 IDT 描述符，并且提供一个 IRQ1 中断对应的服务例程。
- (3) 进入循环，不断调用 WaitKey 函数等待用户按键。WaitKey 直到发生按键后返回键盘扫描码，guest 根据扫描码打印出字符。

GuestKeyboardHandler 中断例程只是简单地读取扫描码，保存在键盘 buffer 里。WaitKey 函数将会在键盘 buffer 里读取扫描码返回。WaitKey 实现在 lib\Guest\GuestCrt.asm 文件里。

2. 监控访问 8259A 端口

ex.asm 模块在调用 init_guest_a 与 init_guest_b 初始化 VMCS 时，调用 set_io_bitmap_for_8259 来设置对 8259A 端口 20h、21h、A0h 以及 A1h 的访问监控。

3. 编译及运行

可以使用下面的三个命令进行编译及生成目标映像文件：

- build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64 -DGUEST_X64
- build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE -D__X64
- build -DDEBUG_RECORD_ENABLE -DGUEST_ENABLE

在 VMware 上运行 demo.img 映像后，在 CPU0 屏幕的命令选择器里按下数字<1>键，让 CPU1 进入 guest 运行。

接着按下两次<F2>键，切换到 CPU1 的 guest 屏幕，如图 7-14 所示。



图 7-14

这是 guest 进入循环后，调用 WaitKey 函数的结果。接下来，我们尝试输入一条信息“hello, mik”，看看结果如何。

注意：当在屏幕上按键输入信息，**这个信息并没有立即回显在屏幕！**那是因为：guest 的输出信息保存在 guest 的 VSB（VM store block）区域内的 **video buffer** 里。

我们需要将 VSB 区域的 video buffer 内容刷新到屏幕上，才能看到所输入的信息。

现在，按下两次<F2>键再次切换到 CPU1 的 guest 屏幕（效果等于刷新 video buffer 到物理 video）！

如图 7-15 所示，现在屏幕上已经有了刚才输入的信息。说明键盘中断已转发给 CPU1 的 guest 处理！



图 7-15

接下来，我们看看这个中断处理流程到底发生了什么。按下<F4>键切换到 CPU3 处理器，然后再按下<Enter>键切换到 msg-list 列表记录。不断按向下键翻页，找出处理外部中断所产生的第 1 记录信息。

如图 7-16 所示，我们分析一下图中所框的信息。

(1) 第 101 条记录，显示 CPU0 发送一条 IPI 给目标处理器。说明 CPU0 已经转发了外部中断给 CPU1。

(2) 第 102 条记录，显示 CPU1 由于外部中断而产生 VM-exit。

(3) 第 103 条记录，显示由 DoExternalInterrupt 例程通过注入事件反射外部中断给 guest 处理。

(4) 第 105 条记录，显示由于异常而产生 VM-exit。这是由中断向量号超出 guest-IDT 的 limit 而产生的#GP 异常（参见 7.3.1.5 节的例子 7-4）。

(5) 接下来从第 106 到 110 条记录，就是处理 guest 中断的 delivery 操作（参见 7.3.1.5 节的例子 7-4），在这里它处理外部中断的 delivery 操作。

用户每次按键都会经过上面的几个步骤进行处理，这个过程耗时很大。当然，如果省去例子 7-4 的监控 INT 指令机制，那么不会有什么太大的损耗。

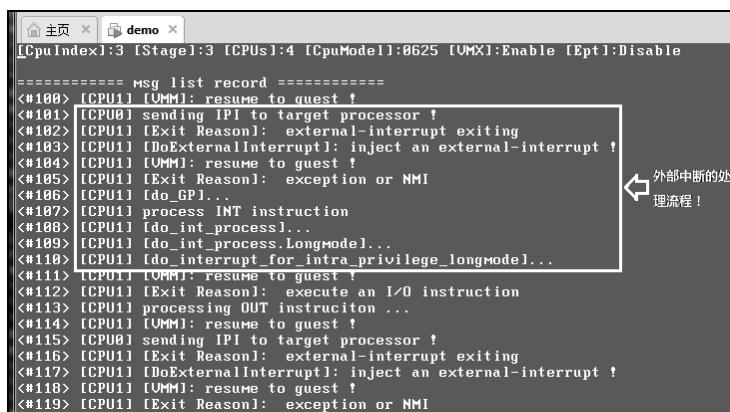


图 7-16

注意：VMM 必须要发送 EOI 给 local APIC，处理器才可能响应下一次的中断消息。所以，每次经过前面所述的处理中断步骤后，就遇到 guest 执行 OUT 指令发送 EOI 指令。VMM 拦截后，由 VMM 来写 EOI 命令。